

Byeharu

Project History — Development Log

Source	docs/DEV_LOG.md
Repository commit	23ac23302910330d09bdde949197a40cbb246025
Source size	12,957 lines / 1,005,824 bytes
Entries	241 dated entries (newest first)
Generated	2026-07-23
Renderer	ReportLab Platypus (Malgun Gothic / Consolas / Segoe UI Symbol embedded)

This document is a complete, unabridged rendering of the development log in its natural file order (newest entry first, oldest entry last).

Contents

2026-07-23 — Movement-truth audit, dead-wrapper retirement, rollback fail-closed, elite stat wiring (0272 merged, deploy deferred), PROJECTMAP function-ownership authority	9
2026-07-23 — World Editor roadmap CLOSURE: chain 0263→0271 deployed (prod head 0271) + client V5 shipped	11
2026-07-21 — Combat-content program (E0–E4) activation-prep authored (NOT run)	12
2026-07-21 — OPERATIONAL RECORD: World Editor V1.5 audit-read backend DEPLOYED & PRODUCTION-PROVEN (migration 0256, prod head 0256)	12
2026-07-20 — MILESTONE: WORLD EDITOR FOUR-DOMAIN AUTHORING V1 — COMPLETE (full owner-gated publish surface deployed, migrations 0244→0255, prod head 0255; reversible zone lifecycle proven live in production)	13
2026-07-20 — SESSION WRAP: World Editor V1A (merged) + V1B-0 Owner Security Spine (merged, mig 0243 — DEPLOYED LIVE) + V1B-1 Location Drafts (merged); V1B-2 validation in progress; V1C coordinate-authority inventory open	14
2026-07-18 — COMBAT SLICE 0: the module-attribute foundation (mig 0229, DARK module_range_attributes_enabled)	15
2026-07-15 — SESSION WRAP: the FLEET RESHAPE + audit-fix batch is COMPLETE (all merged, all DARK) — pending state = owner-gated	15
2026-07-15 — CRON-GUARD: per-row exception isolation for the two hottest legacy crons (mig 0206, pure hardening, NO flag)	16
2026-07-15 — COMMAND-BUFFS: the FINALE of the fleet reshape — fleet-wide command buffs (mig 0205, DARK behind command_buffs_enabled)	17
2026-07-14 — ROOMS-8: configurable ship rooms — 8 slots, a large room catalog, per-slot config (mig 0203, DARK behind captain_assignment_enabled)	18
2026-07-14 — FLEET-CONTROL: the fleet control-model (mig 0204, DARK behind fleet_control_enabled)	19
2026-07-14 — REPAIR-ECON: paid hull-repair economy (mig 0201, DARK behind repair_economy_enabled)	20
2026-07-14 — MOD2-2: the Mk-II module tier (mig 0202, DARK behind the EXISTING module gates)	21
2026-07-13 — NO-HOME: launch-from-dock + dock-at-return (mig 0199, DARK behind launch_from_dock_enabled)	21
2026-07-13 — NANGUARD: fix the inherited no-op NaN guards at all three knob sites (mig 0198, inert)	22
2026-07-13 — SHIELD-2: out-of-combat regen + commission copy + the UI meter pair (mig 0197, inert — P13 complete dark)	23
2026-07-13 — DECKS-3: station affinity bonuses — the adapter's knob-gated matched-station multiplier (mig 0196, inert)	24
2026-07-13 — SHIPYARD-2: hull-build DELIVERY (mig 0194, dark) — all pre-flip seams closed	25
2026-07-13 — SHIELD-1: the shield enters the live combat engine (mig 0195, inert while all pools are 0/NULL)	27
2026-07-13 — SOUL-1: ship traits go functional — commission roll hook + adapter trait fold (mig 0193, dark)	28
2026-07-13 — TEAMMOVE-1: a docked team moves onward as one (mig 0190, LIVE gate)	29
2026-07-13 — SHIELD-0: the shield schema foundation (mig 0191, deploy-inert, data-gated)	29
2026-07-13 — SHIPYARD-1: the hull-build ORDER command (mig 0188, dark)	30
2026-07-12 — SOUL-0: the per-ship traits foundation (mig 0186, doubly dark)	32
2026-07-12 — SHIPYARD-0: the ship-production foundation (mig 0185, dark)	33
2026-07-12 — MOD2-1: the shield line + mining rig module seeds (mig 0183, dark)	34
2026-07-12 — EV-1: the world-events producer — worldstate_tick narrates threshold events (dark)	35
2026-07-12 — ACT-HAUL: the delivery-contracts activation script	37
2026-07-12 — C2-2: captain level → stats adapter fold (dark)	38
2026-07-12 — HAUL-2: the contract accept/deliver RPCs (dark)	39
2026-07-12 — COORD-GUARD (queue #14.5): the A0-fix + the coordinate-travel flip script	40
2026-07-12 — CAPXP-0/1 (queue #13): captain-XP foundation (dark)	41
2026-07-12 — HAUL-0/1 (queue #12): delivery-contracts foundation (dark)	43
2026-07-12 — RANK-SEASON + ACT-RANKING (queue #9): the ranking activation script	44
2026-07-12 — ZONES2-1+2 (queue #7+#8): "Ember Reach" content expansion — seeded HIDDEN + the reveal script	45
2026-07-12 — SALVAGE-0/1 (queue #6): the combat-loot → port-economy feed (dark)	46
2026-07-12 — ACT-TRADE (queue #5): the trade-market activation script + the proof chain finally green	47
2026-07-12 — ECON-SEED-1: differentiated three-port economy seed (migration 0173, DARK)	48

2026-07-12 — ACTIVATION PREP: exploration + mining flip scripts + the 0172 writer reconcile (Phases 11/12; no flip)	49
2026-07-12 — DEPLOYS: prod migration head 0169; team-command roadmap merged (main @ 9a292ed)	50
2026-07-12 — TEAM-COMMAND Slice D complete (D0–D4): team combat, dark (migrations 0166–0169)	50
2026-07-12 — UI RENEWAL R0–R4: Mission Control (PRs #86, #87–#91; frontend-only)	51
2026-07-12 — TEAM-COMMAND Slices C0 + C1: captains into teams, dark (migration 0165)	51
2026-07-11 — TEAM-COMMAND Slice B complete: send/stop BY TEAM, dark (migrations 0161–0164 + UI + proof)	51
2026-07-09 — A0 FOUNDATION FIXUP + TEAM-COMMAND Slice A: team model (migrations 0159, 0160)	52
2026-07-09 — PR #75 merged: recall/return-home removed + stop-zoom fix; session handoff (#76)	52
2026-07-08 — PORT-CENTRIC ACTIVATED on prod + station storage (migrations 0157/0158)	52
2026-07-07 — STOP = HOLD, Slice D2: client copy/comments now match the hold semantics	52
2026-07-06 — STOP = HOLD, Slice C: verifier proves the legacy hold round-trip	53
2026-07-06 — STOP = HOLD, Slice B: RESUME from the held point (migration 0156)	53
2026-07-06 — STOP = HOLD-IN-OPEN-SPACE (legacy), Slice A: stop settlement (migration 0155)	54
2026-07-06 — GOAL WRAP-UP: stop fix + UI rebuild + map declutter — final verification & the ONE apply-time checklist	55
2026-07-06 — UI REBUILD (2b): Map interior — detail panel humanized, overlays organized, selector dedup	56
2026-07-06 — UI REBUILD (2b): Command interior — home base in the shared design language	57
2026-07-06 — UI REBUILD (2b): Port interior — one docked-services surface, DockServicesPanel folded	57
2026-07-06 — UI REBUILD (2b): Ship interior — the MainShipPreview + MainShipPanel MERGE	58
2026-07-06 — UI REBUILD (2b): the persistent four-destination nav shell (structure + navigation)	58
2026-07-06 — MAP DECLUTTER: waypoint relocation (migration 0154, data-only)	59
2026-07-06 — STOP/MOVE FIX, slice 2: the send→stop→send→stop verifier	60
2026-07-06 — STOP/MOVE FIX, slice 1: consumed idempotency keys cleared on success (client-only)	61
2026-07-06 — MAINSHIP LEGACY SPATIAL-STATE FIX, slice 3: the end-to-end round-trip verifier	61
2026-07-06 — MAINSHIP LEGACY SPATIAL-STATE FIX, slice 2: legacy arrival docks the ship (migration 0153)	62
2026-07-06 — MAINSHIP LEGACY SPATIAL-STATE FIX, slice 1: departure/halt pair-writes (migration 0152)	63
2026-07-06 — VISUAL FOLLOW-ON item 4 (final sweep): palette-literal inventory; last live straggler converted	64
2026-07-06 — VISUAL FOLLOW-ON item 3: CombatReportPage restyled onto the shared design system	64
2026-07-06 — VISUAL FOLLOW-ON item 2: legacy /map list view retired (fully superseded by /galaxy)	65
2026-07-06 — VISUAL FOLLOW-ON item 1: AuthPage restyled onto the shared design system	65
2026-07-05 — UX CLEANUP PASS COMPLETE — final consolidation: full-suite check + PR-ready handoff (no product changes)	65
2026-07-05 — UX CLEANUP (item 5, slice D) — market surface on the design system (COMPLETES item 5's screen coverage; presentational-only)	66
2026-07-05 — UX CLEANUP (item 5, slice C) — dock/port surfaces on the design system (presentational-only)	67
2026-07-05 — UX CLEANUP (item 5, slice B) — Galaxy Map on the design system + map-look elevation (presentational-only)	67
2026-07-05 — UX CLEANUP (item 5, slice A) — the ONE shared design system + Command Center conversion (presentational-only)	68
2026-07-05 — UX CLEANUP — reconcile the stale verify:m2 world-count pin (test-only; suite green again, 13/13)	69
2026-07-05 — UX CLEANUP (item 6, part B) — on-demand LEGACY main-ship arrival settlement (migration 0151: helper extraction + RPC + unified client due-trigger)	69
2026-07-05 — UX CLEANUP (item 6, part A) — on-demand OSN arrival settlement (migration 0150 + client due-trigger)	70
2026-07-05 — UX CLEANUP (item 3) — consistent in-transit "stop" for LEGACY main-ship moves (migration 0149 + shared stop UI reuse)	71
2026-07-05 — UX CLEANUP (item 2) — honest docking UX at non-dock waypoints (frontend-only; server eligibility untouched)	72
2026-07-05 — UX CLEANUP (item 4) — one-word location names (forward-only data migration 0148 + every caller updated same step)	73
2026-07-05 — UX CLEANUP (item 1) — retire the legacy Train Ships / Training Queue / map ExpeditionCommand UI (frontend-only; backend RPCs intentionally untouched)	74
2026-07-05 — RANKING-P17 POST-AUDIT FIX (item 5) — schedule the ranking-accrual cron; world-tick already scheduled, no redundant cron (migration 0147)	75

2026-07-05 — CAPTAIN-P16 POST-AUDIT UI (panel 4 of 4) — the dark Captain Progression (recruit) screen, EXTENDING src/features/captains/ (frontend-only; no server change)	76
2026-07-05 — CAPTAIN-P15 POST-AUDIT UI (panel 3 of 4) — the dark src/features/captains/ assign/unassign screen (frontend-only; no server change)	77
2026-07-05 — LOCATION-INVEST-P18 POST-AUDIT UI (panel 2 of 4) — the dark src/features/investment/ Port Investment screen (frontend-only; no server change)	78
2026-07-05 — RANKING-P17 POST-AUDIT UI (panel 1 of 4) — the dark src/features/ranking/ leaderboard screen (frontend-only; no server change)	79
2026-07-05 — EXPLORATION-P11 POST-AUDIT FIX — convert the racing duplicate-scan insert into a clean already_discovered (migration 0146)	79
2026-07-05 — RANKING-P17 POST-AUDIT FIX — the deferred commit-safety PROOF scripts/ranking-p17-commit-safe-accrual-proof.sh (no migration/flag change)	80
2026-07-05 — RANKING-P17 POST-AUDIT FIX, SLICE B — ranking_accrue_standings made COMMIT-SAFE by folding through the ranking_counted_grants ledger (migration 0145)	81
2026-07-05 — RANKING-P17 POST-AUDIT FIX, SLICE A — the commit-safe consumption-ledger SCHEMA ranking_counted_grants (migration 0144; schema only, no writer)	82
2026-07-05 — MINING-P12 POST-AUDIT RECONCILIATION — the 0143 double-extract race is NOT reachable today; honest reframing + deferred concurrency proof (no code/migration change)	83
2026-07-05 — MINING-P12 POST-AUDIT FIX — add a per-(player, field) advisory lock to mining_extract as defense-in-depth for the double-extract race (migration 0143)	84
2026-07-05 — PHASE20-POLISH CLEANUP — independent re-audit of the closed polish milestone; ALL THREE audits CLEAN, ZERO remediation (docs-only close-out)	84
2026-07-04 — PHASE20-POLISH SLICE 7 (FINAL) — the dark-posture verifier verify-phase20-polish.mjs + verify:phase20-polish; Phase 20 CLOSED	86
2026-07-04 — PHASE20-POLISH SLICE 6 — icon resolver (src/features/assets/) + severity icons on the World Events panel, fail-closed	87
2026-07-04 — PHASE20-POLISH SLICE 5 — the World Events display feature (src/features/events/) wired into the galaxy map, fail-closed	87
2026-07-04 — PHASE20-POLISH SLICE 4 — the UI asset-key vocabulary ui_asset_catalog (static reference table + seed) + its flag-gated read surface get_ui_asset_catalog(...) (0142)	88
2026-07-04 — PHASE20-POLISH SLICE 3 — the World Events flag-gated READ surface get_world_events(...) (fail-closed, authenticated-only) (0141)	89
2026-07-04 — PHASE20-POLISH SLICE 2 — the World Events sole-writer functions world_events_publish / world_events_set_active (service-role, idempotent) + the dedup_key idempotency column (0140)	90
2026-07-04 — PHASE20-POLISH SLICE 1 — the Phase-20 dark master flag phase20_polish_enabled + the World Events schema world_events (0139)	91
2026-07-04 — WORLD-BALANCE-P19 CLEANUP — restore the 0092 docked-resolve dedup that 0136 regressed; re-route all three Trade Market RPCs through mainship_resolve_docked_location (0138)	92
2026-07-04 — WORLD-BALANCE-P19 SLICE 4 (FINAL) — the dark-posture verifier verify-world-balance.mjs + verify:world-balance; Phase 19 CLOSED	92
2026-07-04 — WORLD-BALANCE-P19 SLICE 3 (FINAL) — RESOURCE-FIELD DEPLETION (dark): World-State mining_field_state reserve, composed into mining_extract, regenerated by the tick (0137)	93
2026-07-04 — WORLD-BALANCE-P19 SLICE 2 — PRICE DRIFT (dark): World-State price multiplier folded into location_state, composed into all three Trade Market prices (0136)	95
2026-07-04 — WORLD-BALANCE-P19 SLICE 1 — PIRATE PRESSURE (dark): wire the defeat_pressure seam in worldstate_tick() (0135)	96
2026-07-04 — LOCATION-INVEST-P18 SLICE 3 (FINAL) — the dark-posture verifier verify-location-investment.mjs + verify:location-investment; Phase 18 CLOSED	97
2026-07-04 — LOCATION-INVEST-P18 SLICE 2 — the dark PUBLIC read surface (development vs seasonal score) + the ONE season-window helper	98
2026-07-04 — LOCATION-INVEST-P18 SLICE 1 — the sole-writer invest command (invest_in_location → location_investment_invest) + min-amount tunable	99
2026-07-04 — LOCATION-INVEST-P18 SLICE 0 — dark flag + the location_investments root table (foundations only)	100
2026-07-04 — RANKING-P17 CLEANUP — correct two stale figures in the verify-harness.mjs header (docs-in-code only)	101
2026-07-04 — RANKING-P17 SLICE 5 — the dark-posture verifier scripts/verify-ranking.mjs + verify:ranking (FINAL Phase-17 slice)	101
2026-07-04 — RANKING-P17 SLICE 4 — the dark read surface get_ranking_seasons + get_ranking_leaderboard (migration 0131)	102

2026-07-04 — RANKING-P17 SLICE 3 — the core standings-scoring accrual ranking_accrue_standings (the SOLE writer of ranking_standings, reading reward_grants DOWNWARD; migration 0130)	103
2026-07-04 — RANKING-P17 SLICE 2 — the season-management writer ranking_season_open (the SOLE writer of ranking_seasons; migration 0129)	104
2026-07-04 — RANKING-P17 SLICE 1 — the standings (leaderboard) schema ranking_standings (migration 0128)	105
2026-07-04 — RANKING-P17 SLICE 0 — dark flag ranking_enabled + the Ranking-owned root table ranking_seasons (migration 0127)	106
2026-07-04 — CAPTAIN cleanup audit — SYSTEM_BOUNDARIES \$1-matrix doc-sync (rows captain_types / captain_recipe_ingredients / ship_captain_assignments)	107
2026-07-04 — CAPTAIN-P16 SLICE 4 — the dark-posture verifier scripts/verify-captain-progression.mjs (the verify-captain.mjs analogue) + the self-approved "no new read RPC" decision	107
2026-07-04 — CAPTAIN-P16 SLICE 3 — the dark recruit command: captain_recruit_receipts + private production_recruit_captain + two-layer recruit_captain (the 0109 craft_module analogue, point-for-point)	108
2026-07-04 — CAPTAIN-P16 SLICE 2 — the recruitment recipe catalog captain_recipe_ingredients + seeds (the 0107 module_recipe_ingredients analogue)	109
2026-07-04 — CAPTAIN-P16 SLICE 1 — the dark capability flag captain_progression_enabled = 'false' (flag-only, the 0107/0117 seed idiom) + the self-approved Phase-16 design decision	110
2026-07-04 — CAPTAIN-P15 SLICE I — the DARK-posture verifier scripts/verify-captain.mjs (the verify-fitting.mjs analogue; the sole remaining Phase-15 deliverable)	111
2026-07-04 — CAPTAIN-P15 SLICE G — the dark read surface: get_my_captain_instances() + get_my_ship_captains(ship) (the 0110/0116 analogues)	112
2026-07-04 — CAPTAIN-P15 SLICE F — assigned captains feed calculate_expedition_stats (the 0115 analogue; third feed block, headcount-capped)	112
2026-07-04 — CAPTAIN-P15 SLICE E — the settled-SAFE rule for captain assignment + the mainship_space_assert_settled_safe shared-leaf extraction (fitting re-created behavior-identically)	113
2026-07-04 — CAPTAIN-P15 SLICE D — the dark assign/unassign command + captain_assignment_receipts (settled-SAFE rule deliberately deferred, the 0113→0114 split)	114
2026-07-04 — CAPTAIN-P15 SLICE C — ship_captain_assignments schema + the ONE assignment writer (inert AND dark)	115
2026-07-04 — CAPTAIN-P15 SLICE B — captain_instances schema + the single Captain mint writer (inert AND dark)	116
2026-07-04 — CAPTAIN-P15 SLICE A — dark flag captain_assignment_enabled + the captain_types catalog (foundations only)	116
2026-07-04 — CLEANUP SLICE 4 (final, scripts-only) — auto-cleanup part 4: the trade proof scripts' five duplicated blocks extracted into the sourced scripts/lib/trade-proof-lib.sh	117
2026-07-04 — CLEANUP SLICE 3 (frontend) — auto-cleanup part 3: the four-way duplicated guard body extracted into runGuardedCommand...	
2026-07-04 — CLEANUP SLICE 2 (docs-only) — auto-cleanup part 2: the fleets commission writes recorded as the sanctioned Main-Ship shim (with retirement condition)	118
2026-07-04 — CLEANUP SLICE 1 (docs-only) — module-fitting-milestone auto-cleanup part 1: market_offers law-doc sync	119
2026-07-04 — FITTING-P14 SLICE G (final) — verify:fitting dark-posture script. Phase 14 Module fitting — dark implementation complete (slices A–G)	119
2026-07-04 — FITTING-P14 SLICE F — dark frontend: the fitting section EXTENDS src/features/modules/ (fit/unfit controls inside ModulesPanel; renders nothing while either flag is dark). Frontend only — no migration	121
2026-07-04 — FITTING-P14 SLICE E — the dark read surface 0116 (get_my_ship_fittings()). Server side of Phase 14 complete, fully dark	122
2026-07-04 — FITTING-P14 SLICE D — stats integration 0115 (fitted modules feed calculate_expedition_stats under the module_slots hard cap; re-create of the 0044 adapter)	122
2026-07-04 — FITTING-P14 SLICE C2 — settled-SAFE ship-state rule 0114 (corrects the 0113 'home' literal; ship_not_home → ship_not_settled)	
2026-07-04 — FITTING-P14 SLICE C — the dark two-layer fit/unfit command 0113 (module_fitting_receipts + fit_module_to_ship/unfit_module_from_ship → ONE private fitting_execute_command)	124
2026-07-04 — FITTING-P14 SLICE B — ship_module_fittings + the single Fitting writer 0112 (fitting_apply; FIT and UNFIT through THE ONE writer)	126
2026-07-04 — FITTING-P14 SLICE A — locked design decisions + dark flag/stats-catalog migration 0111 (module_fitting_enabled + module_types.slot_cost/stats_json)	127
2026-07-04 — MODULES-P13 SLICE F (final) — verify:modules dark-posture script. Phase 13 Module crafting — dark implementation complete (slices A–F)	127
2026-07-04 — MODULES-P13 SLICE E — dark frontend src/features/modules/ (catalog + craft + instances panel; renders nothing while the server says dark). Frontend only — no migration	129

2026-07-04 — MODULES-P13 SLICE D — the dark read surface 0110 (get_my_module_instances()). Server side of Phase 13 complete, fully dark	130
2026-07-04 — MODULES-P13 SLICE C — the dark craft command 0109 (module_craft_receipts + craft_module → private production_craft_module)	130
2026-07-04 — MODULES-P13 SLICE B — module_instances schema + the single Modules mint writer 0108 (modules_mint_instance; idempotent by mint_key)	132
2026-07-04 — MODULES-P13 SLICE A — locked design decisions + dark flag/catalog migration 0107 (module_types + module_recipe_ingredients)	132
2026-07-04 — MINING-CLEANUP SLICE 2 (final) — MarketPanel migrated onto useActivityPanelGuards. Guard-hook extraction complete — no local copies remain	133
2026-07-04 — MINING-CLEANUP SLICE 1 — panel guard scaffold extracted to src/lib/useActivityPanelGuards.ts; Exploration + Mining migrated	134
2026-07-04 — MINING-P12 SLICE G (final) — verify: mining dark-posture script. Phase 12 Mining — dark implementation complete (slices A–G)	135
2026-07-04 — MINING-P12 SLICE F — dark frontend src/features/mining/ + shared src/lib/rewardBundle.ts/osnState.ts extraction (exploration repointed same-step)	136
2026-07-04 — MINING-P12 SLICE E — the dark read surface 0106 (get_my_mining_extractions()). Server side of Phase 12 complete, fully dark ¹³⁶	136
2026-07-04 — MINING-P12 SLICE D — the securing processor 0105 (process_mining_securing + pg_cron; deposits via reward_grant('mining', ...))	137
2026-07-04 — MINING-P12 SLICE C — the dark extraction command 0104 (command_mining_extract → private mining_extract)	138
2026-07-04 — MINING-P12 SLICE B — mining schema migration 0103 (mining_fields + mining_extractions, dark, no writer exists)	139
2026-07-04 — MINING-P12 SLICE A — design decisions (self-approved) + dark config/flag migration 0102	139
2026-07-04 — EXPLORATION CLEANUP step 4 (final) — delete dead EXPLORATION_DISABLED_REASON export; closes recon finding #3. Exploration cleanup complete (findings 1–3 all fixed)	140
2026-07-04 — EXPLORATION CLEANUP step 3 — extract shared verify harness (scripts/lib/verify-harness.mjs); closes recon finding #2	140
2026-07-04 — EXPLORATION CLEANUP step 2 — ARCHITECTURE.md doc-sync (docs-only; closes recon finding #1)	141
2026-07-04 — EXPLORATION-P11 SLICE H (final): verify: exploration dark-posture script + wiring. Phase 11 Exploration: dark implementation complete (slices A–H)	141
2026-07-04 — EXPLORATION-P11 SLICE G: dark frontend surface src/features/exploration/ (scan control + discoveries panel; renders nothing while the server says dark). Frontend only — no migration	143
2026-07-04 — EXPLORATION-P11 SLICE F: dark read surface get_my_exploration_discoveries() (reveal-after-discovery) + ACTIVITIES.md as-built reconciliation	144
2026-07-04 — EXPLORATION-P11 SLICE E: dark securing/deposit path — process_exploration_securing cron → reward_grant('exploration', ...) on safe settlement. DESIGN RE-DECISION: corrects the Slice-C carrier law	144
2026-07-04 — EXPLORATION-P11 SLICE D: dark command_exploration_scan — OSN-proximity scan → pending discovery bundle (server-rejected while exploration_enabled=false; nothing deposits yet)	145
2026-07-04 — EXPLORATION-P11 SLICE C: hidden exploration_sites + per-player exploration_discoveries (tables + seed + RLS only; no RPC, no client path, fully dark)	147
2026-07-04 — EXPLORATION-P11 SLICE B: reward-item catalog entries + exploration_enabled=false dark flag (foundations only; nothing client-reachable, no behavior change)	147
2026-07-04 — EXPLORATION-P11 SLICE A: activity-agnostic deposit-on-arrival carrier (reward_source_type; refactor only, combat behavior unchanged, everything dark)	148
2026-07-03 — Trade-economy cleanup audit: ROADMAP Phase-10 doc-sync (docs-only; one stale figure fixed)	149
2026-07-03 — TRADE-ECONOMY-BOOTSTRAP proof wired into existing trade-v1-proof.yml (disposable-only; no new workflow)	150
2026-07-03 — TRADE-ECONOMY-BOOTSTRAP proof: disposable, self-rolling-back seed + relief exercise (no CI yet)	150
2026-07-03 — TRADE-MARKET-1 no-softlock floor: relief claim RPC market_claim_relief (DARK; server-rejected)	151
2026-07-03 — TRADE-MARKET-1 no-softlock floor: relief ledger + tunables + dark flag (schema slice; NO RPC)	151
2026-07-03 — TRADE-MARKET-1 seed capital: starting_credits tunable + single shared wallet_ensure (DARK)	152
2026-07-03 — Docs-only roadmap reconciliation: Phase-10 label + live migration head (no code/flag change)	153
2026-07-03 — Trading V1 cleanup: CI proof workflow trade-v1-proof.yml (disposable DB only; no production)	153
2026-07-03 — Trading V1 cleanup: extract shared docked-location helper (migration 0092; behavior-identical)	154

2026-07-03 — Trading V1 cleanup pass: SYSTEM_BOUNDARIES doc-sync (docs-only; no behavior/flag change)	154
2026-07-03 — TRADE-UI-1 landed DARK + PR-ready (ship-switcher + buy/sell + \$2.5 sole-ship shim retirement)	155
2026-07-03 — Repo/docs sync + PORT-ENTRY player UI landing recorded (no new build)	155
2026-07-02 — Trading V1 design record — FIXED product direction (volume-only per-ship cargo + multi-ship foundation) + TRADE-FLEET-0A read-only audit (DESIGN RECORD ONLY; nothing built)	156
2026-06-30 — OSN-COORD-ENABLE (dark) → PORT-ENTRY-1 first-ship commission/normalize → production verifier (head 0070 → 0072) .	157
2026-06-29 — OSN enabled → Phase 9 docked-port surface → coordinate-gate hardening → Phase 10 Trading design (head 0068 → 0070)	157
2026-06-27 — PORT-LAUNCH: public port launch (foundation → reveal → independent verification)	158
2026-06-26 — Session wrap-up + FORWARD PLAN (notes/design only; nothing started)	159
2026-06-26 — OSN-HUB-1A FORMALLY CLOSED — dark canonical location-target navigation, deployed + verified (flag OFF)	160
2026-06-23 — ANCHOR-2 P0-A census closed + PORT-CENTRIC direction (durable handoff; design/ops only)	161
2026-06-23 — MSP-0: Main Ship Progression ↔ Movement integration contract (design only)	161
2026-06-23 — OSN-ANCHOR-1B: empty canonical-anchor schema (space_anchors) — DEPLOYED & CLOSED (flag OFF)	162
2026-06-23 — OSN-ANCHOR-1A: production catalog-parity verification — CLOSED	162
2026-06-22 — OSN-ANCHOR-1A: truthful-origin guard (dark) — DEPLOYED & CLOSED (flag OFF)	163
2026-06-22 — OSN-3 S6C: flag-dark empty-space coordinate command surface — CLOSED (flag OFF)	163
2026-06-22 — OSN-3 S6B: fixed-space frontend coordinate foundation — CLOSED (flag OFF, read-only)	163
2026-06-21 — OSN-3 S6A: public coordinate-command boundary (flag-dark) — CLOSED (flag OFF)	164
2026-06-21 — OSN-3 S5: coordinate-complete trusted destruction primitive — CLOSED (flag OFF)	165
2026-06-21 — OSN-3 S4: coordinate-arrival processor — CLOSED (flag OFF)	165
2026-06-21 — Legacy main-ship send: controlled production activation (config-only, reversible)	166
2026-06-21 — OSN-3 S3: first internal coordinate-movement writer — CLOSED (flag OFF)	167
2026-06-21 — OSN-3 S2: internal transition boundary + validation core — CLOSED (flag OFF)	168
2026-06-21 — OSN-3 S1: coordinate-domain schema + invariants + read-model — CLOSED (flag OFF)	168
2026-06-21 — OSN-1 / OSN-2a / OSN-2b (Open-Space Navigation, read side) — CLOSED	169
2026-06-19 — Design correction: HIGH-STAKES ships (destructible) + emergency restart (docs)	170
2026-06-18 — Design correction: deprecate support capacity / support craft (UI + docs)	170
2026-06-18 — Phase 10B: read-only main-ship expedition preview (implemented; pending verify)	171
2026-06-18 — Follow-up: M4.5 browser test self-cleaning + orphan cleanup (test hygiene)	171
2026-06-18 — Phase 9C: Expedition UI Reframe (BUILD + VERIFY + BROWSER GREEN)	171
2026-06-18 — Phase 9B: Map-based Expedition Send (BUILD + VERIFY + BROWSER GREEN)	172
2026-06-18 — Phase 9A: Read-only Visual Galaxy Map (BUILD + VERIFY GREEN)	173
2026-06-18 — Prevention Phase C: self-cleaning verify runs (DEPLOYED + VERIFIED)	173
2026-06-18 — Prevention Phase B: safe retention cleanup (DEPLOYED + VERIFIED)	174
2026-06-18 — Prevention Phase A: combat logging controls + DB visibility (DEPLOYED + VERIFIED)	174
2026-06-18 — Phase 8: calculate_expedition_stats() (DEPLOYED + VERIFIED)	175
2026-06-18 — Phase 7: Main Ship Instance (DEPLOYED + VERIFIED)	176
2026-06-18 — Phase 6: Support Craft Reframe (DEPLOYED + VERIFIED)	176
2026-06-18 — Phase 5: Multi-Item Pirate Loot (DEPLOYED + VERIFIED)	177
2026-06-17 — Phase 4: Pending Loot Bundle (DEPLOYED + VERIFIED)	178
2026-06-17 — Phase 3: generic inventory foundation (DEPLOYED + VERIFIED)	178
2026-06-17 — Phase 2: Expedition Activity Architecture (design doc only)	179
2026-06-17 — Phase 1: roadmap / architecture reconciliation (docs only)	179
2026-06-17 — M4.5 CLOSED (browser acceptance passed)	179
2026-06-17 — M4.5 Core UX + production queue law fix (CLOSED — see entry above)	179

2026-06-17 — M7 Ship Training (implemented; pending deploy/verify + click-through)	180
2026-06-17 — M5 balance correction: pressure decay toward baseline (follow-up #3, Option A)	180
2026-06-17 — M6 CLOSED (frontend depth / player clarity)	181
2026-06-17 — M6 Frontend Depth (implementation record — CLOSED above)	181
2026-06-17 — M5 CLOSED (deployed + verified green in CI)	182
2026-06-17 — M5 Living World (built; pending deploy + verify)	182
2026-06-17 — M4 cleanup (loose ends; verified 40/40)	183
2026-06-17 — M4 CLOSE (combined final pass; all verified)	183
2026-06-17 — M4 final checklist audit (all pass)	183
2026-06-17 — M4 combat clarity pass (verified 28/28)	184
2026-06-17 — M4 combat overhaul: pacing + per-unit HP (verified 27/27)	184
2026-06-16 — M4 fixes from browser feedback (verified 26/26)	185
2026-06-16 — M4 frontend (active combat UI, display-only)	185
2026-06-16 — M4 backend: server-authoritative pirate combat (verified)	186
2026-06-16 — M3 COMPLETE	186
2026-06-16 — M3 frontend (Command Center)	186
2026-06-16 — M3 backend built, deployed, and verified live	187
2026-06-16 — M2 verified live against real Supabase	187
2026-06-16 — System boundaries approved; M2 (read-only world map)	188
2026-06-16 — Foundation architecture & milestone plan (no code)	188
2026-06-16 — Rename to Byebaru	189
2026-06-16 — Milestone 1: Scaffold + auth	189

Byeharu — Dev Log

Running record of **requests**, **work done**, **bugs**, and **fixes**. Newest entries at the top. Dates are absolute (YYYY-MM-DD).

2026-07-23 — Movement-truth audit, dead-wrapper retirement, rollback fail-closed, elite stat wiring (0272 merged, deploy deferred), PROJECTMAP function-ownership authority

DEPLOY STATE — read this first. main carries migration 0272. **Production is still 0271.** The Deploy Supabase migrations run for the 0272 merge (29979341800) is waiting **at the production environment approval gate** — the owner deliberately deferred the approval. **Nothing in 0272 is live.** Every statement below about elite behaviour describes merged code, not production.

The unified-movement production smoke has NOT been performed. Movement verification remains **classification B — evidence incomplete**. docs/MOVEMENT_SMOKE_PACKET.md is a prepared, unexecuted packet requiring an owner-named expendable fleet and an explicit production-write authorisation.

Merged to `main` **this session** (in merge order):

PR	SHA	What
#282	68ea475	World Editor roadmap closure doc (docs/WORLD_EDITOR_ROADMAP_CLOSURE.md)
#283	86c2c73	World Editor function-level ownership section in docs/SYSTEM_BOUNDARIES.md + the first ## 2026-07-23 DEV_LOG closure entry (below)
#286	b9e2560	Movement cutover documentation truth + new docs/MOVEMENT_ROLLBACK_DEFECT.md
#288	a086800	Legacy movement rollback made fail closed (activation path proven byte-identical)
#285	2279b45	Retired two dead client wrappers <code>sendShipGroup</code> / <code>moveShipGroup</code>
#284	b11b3bd	Elite stat wiring — migration 0272 (DARK; merged, not deployed)
#287	ce26486	World Editor UX comfort pass (client-only) — current main head

Closed without merging (disposition comments posted; branches preserved):

- **#162** — edited an **already-applied** migration (0205). A migration that has already run is never re-run, so the change was a guaranteed production **no-op**. Must be re-authored as a forward migration if the intent is still wanted.
- **#245** — the **×17 coordinate normalize**. Rejected direction (see the coordinate program). Slot 0253 is **intentionally reserved and absent from main** — do not fill it casually.
- **#221** — Zone Templates plan. Predates the entire V1→V5 World Editor program; should be rewritten against the current architecture rather than merged.

Still open: #163 (PROJECTMAP tool branch, feat-project-map — intentionally not on main).

1. Movement verification — classification B, and the true head correction

An audit of the unified-movement cutover established what is and is not proven.

Proven (code + live PostgREST probes):

- **Exactly ONE movement authority.** Every legacy per-ship mover is physically absent from production — probed by distinguishing PGRST02 (function does not exist) from 42501 (exists but revoked).
- **The canonical mover is** `command_ship_group_go`, **TRUE HEAD** 20260618000233_...:589 — **not 0207** or **0208**, which are superseded bodies. (docs/MOVEMENT_UNIFICATION_CHARTER.md \$true-heads table.)
- **One fleet ⇒ one movement by construction** — 0233:768-786 keys the fleet on `group_id` + `main_ship_id IS NULL`, takes `FOR UPDATE`, and returns `fleet_ambiguous` rather than picking one when more than one live unified fleet matches.
- **Replay is a redirect, not a duplicate.**
- `fleet_control_enabled = false` is **VALID alongside unified movement**. 0204:24-25 records that the flag gates **only** (a) the command-ship-required movement check and (b) the 8-ship assign cap — neither of which the unified mover reads. A false `fleet_control_enabled` is therefore not evidence that movement is broken.
- `tsc -b clean`; **1160** frontend specs pass; `scripts/fleetgo-proof.sh selftest` green.

NOT proven — the whole of the runtime: `fleets`, `fleet_movements`, `main_ship_instances`, `ship_groups` and `location_presence` are all RLS-scoped and return **zero rows to an anonymous reader**, so no live movement was observed. The real-Postgres `osn3-fleetgo-realchain-proof` **did not run** (no Docker / Supabase CLI / `psql` on this machine). Hence **classification B — evidence incomplete**.

Cutover provenance. The four movement flags were written by an **act script**, not a migration: `scripts/activate-unified-movement.sql:242-256` (the four `set_game_config` writes at `:246-249`), commit `56a84c3`. Grepping `supabase/migrations/` for the flag change finds **nothing** — a trap that has already misled one reader.

2. The rollback defect (#286 documented it, #288 fixed it)

`0232` dropped all **20** legacy movement functions; `0231:1559-1562` dropped `main_ship_instances.spatial_state / space_x / space_y`, and `0231:1552-1556` removed `'stationary'` from the status CHECK constraint.

`command_main_ship_stop_transit` **survives by design** (`0232:67` defers the stop-trio; `0232:354-355` self-asserts it must still exist) — but its `0155` body still **reads** `spatial_state` (`0155:102-103`) and **writes** `space_x / space_y` (`0155:152-154`). Consequence: re-lighting `mainship_send_enabled` would convert a clean `feature_disabled` reject into a runtime `column does not exist` error. The "flip the flags back" rollback in the older handoff was therefore **not a rollback at all**.

#288's fix: make it fail closed. The four inverse `set_game_config` writes were deleted from the rollback block; what remains is a commented block that **raises on its first statement** if anyone runs it. The **activation path is unchanged** — proven byte-identical (comment-stripped executable SQL identical to `main`).

3. Pirate interception is INTENDED, not a defect

Recorded explicitly because it was briefly mis-reported as a bug this session.

`20260618000236_pirate_intercept_reliable_ambush.sql` deliberately set `pirate_intercept_base_risk = 1.0 (:51)`, `min_risk = 0.98 (:52)`, `max_risk = 1.0 (:53)` and `exposure_floor = 1.0 (:54)`, per an explicit owner directive recorded in the migration header (`:15` — "owner expects RELIABLE combat on entry, not a rare roll"). Any leg touching an **active** danger zone is intercepted with probability $\in [0.98, 1.0]$ **regardless of fleet strength, by design**; the ~2% escape is retained deliberately ("there is ALWAYS a risk"). Live-verified: `pirate_intercept_enabled = true`, `spatial_combat_enabled = true`.

Do not "repair" this. A future reader who finds strong fleets still being intercepted is looking at the intended behaviour, not a balance bug.

4. Dead client wrappers retired (#285)

`sendShipGroup` and `moveShipGroup` had **zero callers**, and their target RPCs (`send_ship_group_expedition`, `move_ship_group_to_location`) were **dropped from production by 0232** — so they could only ever have returned `unavailable`. Deleted rather than left as decoration.

5. Elite stat wiring — migration 0272 (#284) — merged, deployment deferred

Design. The elite roll happens **once, at encounter materialization**, inside `resolve_location_encounter` (the sole `create` or `replace` function in the file, `0272:90`). The elite subset is emitted as its **own** `units[]` **entry** carrying `base_difficulty × encounter_elite_difficulty_multiplier` (`0272:208`, default 2). Because the tick derives every enemy stat of a resolved wave from that one `base_difficulty` field, the existing spawn arm materialises both entries through its **identical existing insert** — so `process_combat_ticks` is **NOT re-created** and the damage resolver never learns what "elite" means.

Blast radius. No new flag (elite rides the existing `encounter_resolver_enabled` quad-gate). The sole `game_config` write is an additive insert ... on conflict (key) do nothing of the numeric multiplier (`0272:75-82`). **Zero DROP / ALTER / DELETE / TRUNCATE statements in the file.**

CI proof markers, all green: `ELITE_PASS_SOURCE`, `ELITE_PASS_LEGACY_PARITY`, `ELITE_PASS_DETERMINISM`, `ELITE_PASS_SPLIT_PLAN`, `ELITE_PASS_FLAGOFF_SYNTHETIC`, `ELITE_PASS_SPAWN_STATS`, `ELITE_PASS_WEAPONS_DAMAGE`.

Two honest v1 tradeoffs, recorded so nobody is surprised at go-live:

- Elite is a COUPLED buff.** `base_difficulty` scales hp **and** attack **and** range **and** speed together (`0260:658-665`). Decoupling (hp-only or attack-only elites) would require re-creating `process_combat_ticks` — deliberately not done.
- Rewards do NOT scale with elites.** The resolved reward is derived from the reward profile / `reward_tier` / danger (`0261:818`), **not** from `units[]`. An elite wave is therefore harder for the same loot until a reward-adaptor slice lands.

What it unblocks. The `ELITE-READINESS` FAIL refusal in `scripts/activate-encounter-resolver.sql` (duplicated in `scripts/activate-combat-content-all.sql`) is repointed by this PR to an informational `ACTE3_PASS_ELITE_WIRED` notice, with a new `ELITE-WIRING` FAIL raise if the deployed resolver lacks the `':enc:elite:'` salt (i.e. if someone tries to activate before `0272` is deployed). **Honest note: that refusal was not blocking anything today** — production has **0** members with `elite_chance > 0`.

6. Encounter content audit (read live from production)

Two bindings exist; both are inactive.

- Binding** `2f7bcf88` → Reaver → profile `canary_encounter` (difficulty 1, cap 1, **cooldown 30 s**) → template `canary_fleet` → archetype `canary_pirate` (`base_difficulty = 1`, `elite_chance = 0`) → reward `canary_reward` (metal-only, base 7). **Complete and activation-ready** — selected as the canary chain.

- **Binding** 2d491cde → Snare → profile `pirate_basic`. Structurally complete but `cooldown_seconds = 0` — no spawn throttle at all. **Rejected as the first canary.**

It is NOT a clean slate. `encounter_runtime_state` shows the canary chain **already ran**: `last_spawn_at = 2026-07-22T06:03:27Z`, `active_count = 2` — roughly four minutes before `encounter_resolver_enabled` was set `false` at 06:07:02Z. Note `active_count` **only ever increments** and is **not** the live-encounter authority (the cap authority derives from `combat_encounters`).

E0–E2 authoring flags are LIVE in production: `enemy_content_registry_enabled`, `encounter_authoring_enabled` and `encounter_binding_authoring_enabled` are all `true`. Only `encounter_resolver_enabled` is `false`. (A forthcoming `docs/ENCOUNTER_CANARY_PACKET.md` carries the canary decision packet.)

7. PROJECTMAP — a real ownership defect found and fixed

The scanner now parses **function-level** ownership as a concern **separate from table ownership**. All **19** World Editor RPCs resolve to a `World Editor` system, while `locations` / `sectors` / `zones` remain owned by **Map** — ownership of a function transfers no ownership of a table.

Graph deltas: systems **29 → 30**, nodes **741 → 742**, edges **2159 → 2198**, timeline **236 → 240** entries, now reaching 2026-07-23. Drift guard clean, `--allow-shrink` never used, `graph.json` byte-reproducible.

The defect. `src/tree.js` and `src/layout.js` inferred function ownership **differently** — four bidirectional call-vote passes versus one, plus different handling of unowned-table votes. **49 of 274 functions silently disagreed** between the `조직도` Tree and the 3D map, and **no test could detect it**, because each file was internally consistent. Fixed by extracting **ONE authority**, `src/systems.js`, consumed by both; divergence is now **0**. This is the no-spaghetti law in miniature: two readers of the same concept is one reader too many.

Also removed the distance fog (`src/main.js:89`, `FogExp2(0x07090f, 0.0016)`), which faded the graph into the background at exactly the zoom level where you most want to see the whole thing.

Scanner hardening was load-bearing, not cosmetic. Against the merged `SYSTEM_BOUNDARIES.md`, the OLD parser ingests **90 pairs / 30 owners** versus the new **71 / 29** — **19 function names would have been swallowed as phantom tables**.

8. What remains (nothing here is done)

1. **Approve the production gate for 0272** — owner-only. Until then prod stays at 0271 and elite is not live anywhere.
2. **Execute the unified-movement smoke** (`docs/MOVEMENT_SMOKE_PACKET.md`) with an owner-named **expendable** fleet, to move movement from classification **B** to **A**.
3. **Encounter canary** — activate the `canary_encounter` chain (binding `2f7bcf88`), not `pirate_basic`.
4. **Reward-adaptor slice** if elite waves should pay more than non-elite waves.

2026-07-23 — World Editor roadmap CLOSURE: chain 0263→0271 deployed (prod head 0271) + client V5 shipped

Migration chain deployed (0263→0271). Production migration head is 0271. Recorded from the **deploy workflow's Supabase CLI log** — this is not a direct production database read; nothing in this entry was read off prod SQL from this machine.

Migra tion	File	What it did
0263	20260618000263_worldeditor_v1c_pr_d_read_cutover.sql	<code>get_world_map()</code> READ-cutover onto <code>space_anchors</code> (authority only — byte-identical rendered world, no coordinate moved)
0264	20260618000264_worldeditor_v1c_anchor_write_authority.sql	<code>space_anchors</code> becomes the SOLE location-coordinate authority for write too (<code>location_create</code> / <code>location_update</code>); the 0263 fail-safe fallback tightened to fail-closed
0265	20260618000265_canonical_coord_validation_authority.sql	<code>canonical_coord_violation</code> — the ONE canonical ±10000 point-coordinate validation authority ; the six point-coordinate-writing RPCs redefined to call it instead of six duplicated inline checks
0266	20260618000266_worldeditor_publish_zone_update.sql	<code>zone_update</code> — the zone EDIT command, the last missing edge of the publish domain matrix
0267	20260618000267_worldeditor_unified_revert.sql	<code>world_editor_revert</code> — ONE unified revert across all four domains, replaying an audit before snapshot through the domain's own <code>*_update</code> command (never a direct table write)
0268	20260618000268_worldeditor_zone_set_active.sql	<code>zone_set_active</code> — zone RE-ACTIVATE, closing lifecycle parity across all four domains
0269	20260618000269_worldeditor_entity_catalog.sql	<code>world_editor_entity_catalog</code> — the owner-only lifecycle catalog (active and inactive entities; every other world read is active-only)
0270	20260618000270_worldeditor_entity_detail.sql	<code>world_editor_entity_detail</code> — owner-only inactive-entity detail read supplying the optimistic-concurrency expected fields reactivation demands

Migration	File	What it did
0271	20260618000271_worldeditor_catalog_markerstyle.sql	catalog marker-style enrichment — the four location styling fields (<code>location_type</code> / <code>activity_type</code> / <code>reward_tier</code> / <code>base_difficulty</code>), so the editor map can be sourced from the catalog alone

Client V5 (no migration). Entity search + camera jump · coordinate jump · a global lifecycle filter (active / inactive) · a pending-drafts indicator · an unsaved-draft navigation guard · inactive-entity selection and reactivation.

Lifecycle-parity note. All four domains now have create · update · lifecycle flip. Location reactivation runs through `location_update` (status) — **there is no** `location_set_active`, by design.

Verification posture — read this before quoting the closure.

- Production closure verification was a **READ-ONLY smoke**: the owner-gated editor loaded, the entity catalog listed, search and the lifecycle filters returned, and the History panel read back. **No writes were performed against production** in that smoke.
- **Mutation paths (create / update / set_active / unpublish / revert) are covered by the CI disposable apply-proofs, NOT by a live production mutation smoke.** The disposable proofs apply the full migration chain to a throwaway Postgres and run each migration's own self-asserts. That is a different and weaker claim than "mutated live production and observed the result" — do not merge the two.
- **Live game_config flag values were NOT verified.** They were not readable from this machine; no claim is made about any flag's current production value.

Ownership. `docs/SYSTEM_BOUNDARIES.md` §6 now records the World Editor as a **function-owning, table-owning-nothing** system. sectors / zones / locations remain **Map**-owned; the World Editor reaches them only through owner-gated SECURITY DEFINER RPCs, which transfers no table ownership.

2026-07-21 — Combat-content program (E0–E4) activation-prep authored (NOT run)

Authored the go-live tooling for the E0–E4 combat-content program (PRs #257–#261, migrations 0257–0260, all DARK/seeded-false): four dependency-guarded per-flag `scripts/activate-*.sql` (+ matching `.sh` runners) and a combined `scripts/activate-combat-content-all.sql`, plus the program summary. Scripts are **AUTHORED-NOT-RUN** — the owner flips them at go-live in order E0→E1→E2→E3. Full architecture, merge/deploy/flag-flip order, the E3 byte-identity guarantee, and deferred items: see [docs/COMBAT_CONTENT_PROGRAM.md](#).

2026-07-21 — OPERATIONAL RECORD: World Editor V1.5 audit-read backend DEPLOYED & PRODUCTION-PROVEN (migration 0256, prod head 0256)

Deployed migration: `20260618000256_worldeditor_audit_read.sql` (owner-only audit reader `public.world_editor_audit_list(jsonb)`). **PRs:** docs closure #252 (merge 335d948) + backend #253 (merge 5e234e1), both admin-merged 2026-07-21. **Deployment run:** 29792617330 @ 5e234e1 → success (job 88517404749, 26s). **Production migration head:** 0256 (recorded once; 0253 remains reserved/absent).

Function security & privilege posture (verified on target): SECURITY DEFINER; `search_path=''`; volatility STABLE; owner role postgres; in-body `is_owner()` authorization. EXECUTE: PUBLIC none, anon none, authenticated only. `world_editor_audit` remains RLS-enabled with **0 policies** and **no client SELECT** (anon/authenticated) — the reader is the sole read path; no table grant, no unrelated grant/policy change.

Bounded read-only production proof — PASS: anonymous → not_authenticated; authenticated non-owner → not_authorized; owner small-limit → bounded page; exact request-ID lookup on the two existing canary audit rows; empty filter → clean empty page; keyset cursor continuation (page 1 → page 2) with no repeated item. Sanitization (key-level): **0** snapshots contain `reward_bundle_json`, `created_by`, or raw actor — `actor_is_owner` + `redactions[]` returned instead (`created_by` appears only as a redaction label, never a field); create record before = null; unpublish record sanitized before/after (active→inactive); `boundary_wkt` present as the intended owner-visible geometry. **Read-only invariance:** `world_editor_audit` row count 2 → 2 (the reader wrote nothing).

Before/after audit row count: 2 → 2. **Not observable in current production dataset (covered by disposable proof):** a second full pagination page beyond 2 rows; edit-command (non-zone) snapshots — the disposable proof covers both.

Verdict: WORLD EDITOR V1.5 AUDIT-READ BACKEND: PRODUCTION-PROVEN.

Residual risks: no index on `world_editor_audit(created_at, command_type)` — fine at single-owner tens-to-hundreds-of-rows scale, revisit if the ledger grows; the reader returns `boundary_wkt/coords` (owner-visible, needed for the upcoming history→map focus). Next: the read-only frontend History UI on a separate branch (no migration).

2026-07-20 — MILESTONE: WORLD EDITOR FOUR-DOMAIN AUTHORIZING V1 — COMPLETE (full owner-gated publish surface deployed, migrations 0244→0255, prod head 0255; reversible zone lifecycle proven live in production)

What this entry is. The close-out of the World Editor publishing program: every publish domain is now an owner-gated, server-authoritative write RPC through the 0243 spine, deployed live, and the zone lifecycle (create → live → unpublish → inert) was proven end-to-end against production under a controlled canary. This completes **World Editor four-domain authoring V1**.

Coordinate direction — DECIDED: keep physical gameplay coordinates. After a 6-agent C0 audit, the physical x17 DB normalize (migration 0253, PR #245, k=17) was **rejected** as the coordinate direction and is held as historical research only — a physical rescale would change gameplay, invert intended near/far content tiering, detach frozen danger-zone polygons, and need a coordinated migration of active movement / parked fleets / anchors / ranges / radii / speeds / combat / verifiers. Migration slot 0253 remains intentionally reserved/absent from the deployed chain. The approved direction is the **C1 display-adapter contract** below.

Point-domain publish RPCs (migrations 0244–0252, deployed). The exploration / mining / location domains each got owner-gated create + edit (+ set-active) commands, all following the one contract (authn → `is_owner()` authz → `request_id` idempotency → server re-validation → optimistic concurrency for updates → atomic apply + one `world_editor_audit` row with before/after snapshots → typed `{ok,request_id,result|error}` envelope; EXECUTE to authenticated only, table client-write revoked, deploy-time self-assert): 0244 `exploration_site_create`, 0246 `mining_field_create`, 0247 `exploration_site_update`, 0248 `mining_field_update`, 0249 `location_update`, 0250 `exploration/mining set_active` (the canonical **soft unpublish/restore** via an `is_active` flag — no hard delete), 0252 `location_create`.

PR #249 — C1 canonical coordinate + display-adapter contract (merged, client-only, no migration). Stored gameplay coordinates never change; the editor's view of the world is controlled by typed display adapters + the **camera only**. `worldEditorCoordinates.ts` re-exports the ONE `worldToViewBox` projection (no second transform authority); `worldEditorFocus.ts` frames one domain at a time via the shared `galaxyCamera` fit; the legacy `/dev/zones ZoneEditor` + its bespoke `makeFit` were retired; the mining/exploration overlap radii are threaded from `game_config` into the pure validators (750 demoted to a labeled fallback). Structural guard tests (`tests/worldEditorC1Guards.spec.ts`) prove stored coords are unwritten, there is no second projection, and `ZoneEditor` is gone.

PR #248 + #250 — zone_create (migration 0254, deployed) and its ACL fix. `zone_create(p_request_id, p_payload)` materializes a zone draft's seed geometry (circle `ST_Buffer` / polygon closed-ring `ST_MakePolygon`) into ONE `public.danger_zones` row (source='drawn', status='active') with server-side PostGIS as the geometry authority (`ST_IsValid` + `ST_Area` → typed `invalid_geometry`; bad attach → `invalid_attach`). **Prod grant-drift caught by 0254's own self-assert:** production `danger_zones` carried a Supabase project-default GRANT ALL (anon/authenticated INSERT/UPDATE/DELETE) that no migration had revoked — the fresh-chain CI apply-proof never reproduced the default, so the assert had passed vacuously in CI and the first deploy fail-closed rolled back (prod untouched). **Fix = PR #250:** 0254 now REVOKES client write on `danger_zones` before asserting it (mirroring the sibling publish slices), then deployed clean (prod head 0254). Lesson recorded: publish migrations must **establish** their table-write lockdown, not assume it.

PR #251 — zone_unpublish (migration 0255, deployed; prod head → 0255). The zone UNPUBLISH twin of `zone_create`: flips ONE `danger_zones` row from status='active' to 'inactive' (the CHECK already models 'inactive' — no schema change, no hard delete). All three player-active reads (`danger_zones_select_when_lit` RLS, `get_danger_zones()`, and the interception leaf `pirate_intercept_leg_zone_hits()`) gate on status='active', so the flip removes the zone from the player map + flag-gated read + pirate interception at once, while the row (geometry / name / attach / created_by) is preserved for a future republish. Mirrors the 0250 `set_active` contract, plus two zone guards: a **protected-zone** rule (only source='drawn' editor zones are unpublishable — the seeded 'circle' zones Blackden/Reaver/Snare can never be targeted → not_unpublishable/protected_zone) and `already_inactive`. Client: `commandContract` gained the `zone_unpublish` member + `not_unpublishable` code; `ZoneInspectorActions.tsx` issues the real owner-gated command from the live-zone inspector (no client-side success simulation). Adversarial security review = SECURE (no CRITICAL/HIGH/MED); disposable proof + build + 900/900 frontend specs green.

Controlled production live proof — PASS (four-domain V1 complete). Under a controlled canary (CANARY-0255-LIVEPROOF, drawn circle centre (-900,-900) r25, provably isolated: nearest content 1100+ world units, zero unfinished movement legs), the full lifecycle ran against production: owner **create** → the zone was active, in the player read, and interception-relevant → owner **unpublish** → gone from the player read AND interception, row retained inert — all inside a **529 ms** active window. Then, with

the canary inactive: create + unpublish **replays** were idempotent (no second row/audit); **non-owner** → `not_authorized` and **anonymous** → `not_authenticated` for both calls (authorization precedes replay disclosure — no owner-result leak); the three seeded zones rejected unpublish as `protected_zone` and stayed active. Final prod state: 4 zones (3 seeded active + 1 canary inactive), exactly 1 `zone_create` + 1 `zone_unpublish` audit row (zero from the rejected attempts), `pirate_zone_create/delete` still client-locked, `danger_zones` client-write revoked, RLS + SELECT-only policy intact, flags unchanged. The inert canary row is retained as the audit artifact (no SQL cleanup, no hard delete).

World Editor four-domain V1 — verified capabilities: unified owner editor (`/dev/world`, flag-gated, `is_owner()`-authoritative); Location + Mining + Exploration create / edit / unpublish; Zone geometry authoring + create + unpublish; owner authorization, server validation, idempotency, optimistic concurrency, auditing (before/after snapshots), player-read isolation, and a **reversible production lifecycle proof**.

2026-07-20 — SESSION WRAP: World Editor V1A (merged) + V1B-0 Owner Security Spine (merged, mig 0243 — DEPLOYED LIVE) + V1B-1 Location Drafts (merged); V1B-2 validation in progress; V1C coordinate-authority inventory open

What this entry is. A consolidated close-out for the overnight 2026-07-19→20 World Editor program (design doc: `docs/ZONE_TEMPLATES_ARCH.md` SWE), plus the combat-prerequisite track that closed first the same session. Three slices merged to `main`, one of them (the owner security spine) additionally **deployed live**; one slice is in progress; one is inventory-only (no code yet).

Combat-prerequisite track closed first (migrations 0239–0242, deployed). Before the World Editor work started: `pirate_zone_create/pirate_zone_delete` EXECUTE-grant lockdown (#223, mig 0239); `combat_units` uniqueness re-scoped to aggregate player rows, fixing a multi-pirate stall (#224, mig 0240); `combat_ticks` result-check widened to admit `next_wave_incoming` (#226, mig 0241); sticky spatial encounter mode so a fight no longer de-spatializes mid-combat (#227, mig 0242).

PR #228 — World Editor Foundation V1 (merged, `main`). The ONE owner-only World Editor shell on the REAL game map, built strictly READ-ONLY: `worldEditorTypes.ts` (typed read/resolve/inspect contract only — create/edit/publish/enable/disable/archive are declared for explicit-disable UI, never adapter methods), `worldEditorGeometry.ts` (the one map-representation resolver over the shared `openSpaceTransform`), `worldEditorAdapters.ts` (4 typed read-only layer adapters — `LocationLayerAdapter` / `MiningLayerAdapter` / `ExplorationLayerAdapter` / `ZoneLayerAdapter` — each grounded in its real read source), `worldEditorData.ts` (read-only snapshot over `get_world_map` / `get_active_mining_fields` / `exploration_sites` SELECT / `get_danger_zones`, fail-closed), `worldEditorRegistry.ts` + `WorldEditor.tsx` (layer registry/toggles, shared `galaxyCamera` map, typed inspectors). Reuses the shared map primitives (`openSpaceTransform`/`galaxyCamera`/marker styling) and the existing `dev_zone_editor_enabled` / `hidden-/dev/world` / `RequireAuth` gate — no new flag, no migration, no write RPC, no mutation.

PR #229 — V1B-0 Owner Security Spine (merged commit `ec4e71e`; migration 0243 20260618000243_world_editor_owner_security_spine.sql — DEPLOYED LIVE). The reusable, server-authoritative owner boundary every future World Editor command reuses: `app_owners` (deny-all allow-list, RLS on, no client grant), `is_owner()` (SECURITY DEFINER STABLE, keyed off `auth.uid()`, pinned `search_path=''` as defense-in-depth hardening — commit `15d0928`), `world_editor_audit` (`request_id` UNIQUE idempotency + audit ledger), `world_editor_ping(text, jsonb)` (the guarded **no-op** command endpoint template — `authn` → `authz` → idempotent → audit → typed envelope, ZERO world mutation). Frontend `commandClient.ts` ships typed-client-only, not wired to any write control. Docs: `docs/WORLD_EDITOR_V1B0_OWNERSPINE.md`. Independently verified **no hole** by the lead plus an adversarial Opus review across 8 vectors before merge. **Deploy verified live on prod:** migration head = 0243, owner seeded into `app_owners` (`is_owner()` = true for the owner), the 0239 pirate-zone lockdown still intact, combat unchanged (`spatial_combat_enabled=true`, `combat_tick_logging=false`).

PR #230 — V1B-1 Location Drafts & Preview (merged commit `1a0d4d3`). Client-side location draft model, wired into `WorldEditor.tsx`: `locationDraftTypes.ts` / `locationDraftModel.ts` (pure, FNV-1a source fingerprint) / `useLocationDrafts.ts` (reducer + `localStorage`, rehydrate re-validation) / `LocationDraftPanel.tsx` / `DraftPreview.tsx`. Dark behind `dev_zone_editor_enabled`; **zero** live locations write, no migration, no publish. Guard tests (`tests/locationDraftGuards.spec.ts`, `tests/locationDraftModel.spec.ts`) prove there is no mutation surface.

V1B-2 Location Validation — IN PROGRESS. Client-side typed validation ruleset over the drafts from V1B-1, backed by a single enum source-of-truth; still no publish path.

V1C Canonical `space_anchors` coordinate authority — inventory complete, no code yet. Key finding: two incompatible coordinate scales coexist — the map-seed frame (sectors/zones/locations, roughly ±300 after the 0227 ×3 world-rebalance, **no bounds**) versus the OSN frame (±10000, used by `space_anchors` / mining / exploration / fleets). A staged normalize-then-cutover migration is in design (approach A: uniform-scalar normalize `locations` into the ±10000 frame, then

make `space_anchors` the single coordinate authority — the same "no third authority" law already stated in `ZONE_TEMPLATES_ARCH.md` §WE.4/§WE.14). This is **world-affecting** and deferred behind a separate deploy approval — not part of this session's merges.

Open follow-ups

- V1B-2 (validation ruleset) — continue in progress.
- V1C — turn the inventory into a concrete staged migration plan for owner review; nothing merged yet.
- Every future World Editor write command must route through the 0243 owner spine (`is_owner()` + `world_editor_audit` + idempotent `request_id`) — no parallel authorization path.

2026-07-18 — COMBAT SLICE 0: the module-attribute foundation (mig 0229, DARK `module_range_attributes_enabled`)

Request. Owner intent: "each module (mining laser, missile launcher) has its own range, ammo speed, power — shown on map as a radius; the range concept is used in combat AND mining." Lay the catalog foundation both future spatial combat (S3, parallel branch) and mining will read, and migrate mining's flat radius onto the fitted mining module — additive, dark, byte-parity except one marked hunk.

Work done (branch `slice-combat-s0-module-attrs`, worktree `byeharu-combat-s0`):

- `module_types` += 6 nullable-by-default catalog columns: `range` (world units), `projectile_speed` (units/sec; NULL = hitscan), `power` (weapon damage OR mining extraction rate), `ammo_type` (FK `item_types(item_id)`; NULL = energy/unlimited), `ammo_per_shot` (int, default 0), `cooldown_seconds` (numeric, default 0) — plus 5 nonneg CHECKs. NULL is the honest "no spatial/combat reach" answer.
- Backfill: `autocannon_battery` (attack 10) → range 150 / `projectile_speed` 300 / power 10 / cooldown 2s; `autocannon_battery_mk2` (attack 18) → range 180 / `projectile_speed` 320 / power 18 / cooldown 2.5s (fires slightly slower — the 0202 "bigger guns draw more heat" narrative extended to rate of fire); `mining_rig_extension` (the ONE existing mining module, 0183 — no new "Mining Laser" needed) → range 120 / power 8 (mirrors its own existing `stats_json.mining=8`, not a second invented number) / hitscan (`projectile_speed` NULL). Every other module type (engine/cargo/sensor/defense) stays explicit NULL — no spatial reach. Ranges picked in the low-hundreds band to read sanely once the parallel world-rebalance (mig 0227, 3x map distances) lands — this slice reads none of that geometry, just future-proofs the scale.
- New read-leaf `ship_weapon_modules(ship)` — owner-scoped (authenticated-only; empty for a ship you don't own), composes the EXISTING fitting join (`ship_module_fittings` → `module_instances` → `module_types`, the exact join `calculate_expedition_stats` already performs) filtered to `range IS NOT NULL`. This is the ONE shape S3 (spatial combat) and the client map-radius layer will both reuse.
- `mining_extract` re-created BYTE-PARITY from the 0104 head with exactly ONE marked hunk: while `module_range_attributes_enabled` is dark, the radius is the exact original flat `coalesce(cfg_num('mining_extract_radius'), 750)` one-liner; lit, it becomes `max(module_types.range)` over the ship's OWN fitted mining modules (identified by `slot_type='mining'` OR a mining `stats_json` key — never a weapon's range, even though both now carry `range`), falling back to the SAME flat default when no mining module is fitted (mining is never hard-blocked for lacking one).
- Flag `module_range_attributes_enabled` seeded `false`. Self-asserted at deploy time: column shape, FK, CHECKs, exact backfill per module type, flag dark, leaf ACL (authenticated-only, never anon) + shape, and `mining_extract` prosrc byte-parity outside the one marked hunk (comment-stripped probes, the 0221 lesson) with unchanged ACL on both `mining_extract/command_mining_extract`.
- `docs/SYSTEM_BOUNDARIES.md` synced: `module_types` row, the Mining row's radius-source clause + new Fitting/Modules read edge, and the Fitting row's inbound-edge note for the new leaf.

Verify: `bash scripts/team-command-proof.sh selftest` and `bash scripts/fleetgo-proof.sh selftest` both pass (DB-free harness checks, unaffected by this slice). No TS touched — `tsc -b` not required this slice (no client change; a client `moduleTypes` field is left for a later slice to add on demand).

Independent of S1 (parallel per-ship-damage combat slice, different files) — not yet applied to any deployed environment; `main` still ends at migration 0222 plus whichever of 0223+ lands first.

2026-07-15 — SESSION WRAP: the FLEET RESHAPE + audit-fix batch is COMPLETE (all merged, all DARK) — pending state = owner-gated

What this entry is. A consolidated close-out for the fleet-control-model reshape and the 7-agent-audit fixes. Every slice below is **merged to main** and **deploy-inert** (each new behavior sits behind a `game_config` flag/knob seeded `false/0`, so the LIVE game

is unchanged until the owner flips). This wrap also records the lighter client slices that shipped without their own top-level entry, and the exact owner-gated state so a future session can resume cold.

The reshape — the owner's new control model ("everything moves as a fleet; a lone ship goes nowhere; a fleet needs a command ship; each ship carries a command buff that only fires as command ship and applies fleet-wide; 8 configurable rooms + 8 captains per ship"):

- **FLEET-RENAME** (#153, client) — "team" → **fleet**, and the "main ship" jargon the owner hated ("WTF is move main ship?") replaced with the ship's own name throughout. `data-testids` and internal identifiers (`ship_groups/group_id`) kept STABLE — pure display rename, no schema churn.
- **ROOMS-8** (#154, mig 0203, DARK `captain_assignment_enabled`) — 8 configurable room slots per ship over a large room catalog; captain slots 6→8. `_(own entry below.)_`
- **FLEET-CONTROL** (#155, mig 0204, DARK `fleet_control_enabled`) — command-ship control model; a fleet is inert without one; 8-ship cap. `_(own entry below.)_`
- **RETURN-PORT** (#156, client, gated `launch_from_dock_enabled`) — the owner's "I want to move freely, not return to whatever port I came from" fix: a return-port `<select>` on fleet hunts (reusing `sendableDestinations` via new `returnPortOptions`); the MOVE path already docks-at-destination. Replaces the hard-coded return-to-origin.
- **COMMAND-BUFFS** (#157, mig 0205, DARK `command_buffs_enabled`) — the finale: per-tier buff catalog, random-at-commission roll, fleet-wide through the active command ship. `_(own entry below.)_`

The audit fixes:

- **POLISH** (#158, client) — killed the "At home port"/"home" label leak (→ "Idle"); de-duped the map double-drawing a docked fleet (`deriveTeamRepresentedShipIds` excludes fleet-badge ships from the chevron layer); Mk-II module glyphs + "Mk-II" labels.
- **ACT-SCRIPTS** (#159) — the owner-run flip scripts the audit found MISSING (`activate-{shield,soul,salvage,decks3,shipyard}. {sql,sh}` + new docs/ACTIVATION_GUIDE.md). The headline: **activate-shipyard opens the build loop** (`shipyard_enabled` + `blueprint_fragment_drop_rate` 0→0.15) — the audit's #1 gap, the faucet that walled ships off from being buildable at all. All NEW files; no shared-doc collisions.
- **CRON-GUARD** (#160, mig 0206) — per-row exception isolation for the two hottest legacy crons; pure hardening, no flag. `_(own entry directly below.)_`

Migration chain now at 0206. New dark flags this batch: `fleet_control_enabled`, `command_buffs_enabled` (both false). ROOMS-8 rides the existing `captain_assignment_enabled`.

PENDING — ALL owner-gated (nothing here is mine to do; recorded so it isn't lost):

1. Two decisions.

`<font name="Sym">` `</font>` `_Command-ship buff STACKING._` Current fold SUMS every command ship's buff, so a player who designates all 8 ships in a fleet as command ships stacks 8 buffs. Owner leaned "backups" (one active buff; extras are insurance). If backups-only: a **1-line limit 1** in the adapter's command-buff fold. Left as-is (sum) pending the owner's word.

`<font name="Sym">` `</font>` `_ACT-flip [D] numbers._` The `activate-*` scripts carry default values (shields 100/130/85 + 0.02/0.10; soul backfill=yes, catalog count=8; salvage 0174 prices; decks3 station-affinity +0.15; shipyard faucet 0.15). Owner says "defaults" or adjusts before the flip.

2. **Deploy migs 0203/0204/0205/0206** to prod (owner runs the approval one-liner — classifier-gated away from me).

3. **Flip order** (per docs/ACTIVATION_GUIDE.md): captains → **decks3; mining + combat → shipyard** (= the build-a-ship loop); **fleet_control → command_buffs**; shields/salvage pair with hunting.

Bugs / fixes (this batch, already resolved in-flight). MOD2-2 zero-arg RAISE "50%" → % (swept all 41 sites); FLEET-CONTROL deploy-window regression (`fetchMyShipGroupMap` widened its live query with `is_command_ship` → would blank all fleets pre-migration; fixed by decoupling the read with a null-safe fold); CRON-GUARD CI broken-pipe harness flake (proof logic passed; re-run merged clean).

2026-07-15 — CRON-GUARD: per-row exception isolation for the two hottest legacy crons (mig 0206, pure hardening, NO flag)

Request (audit). The 7-agent audit flagged a latent global-wedge landmine: the two hottest legacy crons — `process_fleet_movements` (30s) and `process_combat_ticks` (3s) — process all their rows in ONE transaction with NO per-row exception isolation. A single failing row aborts the WHOLE cron run and re-raises every tick FOREVER, for EVERY player. Reachable today: a movement whose target location has an allowed-but-undispatched `activity_type` (e.g. `mine_resource` — in the 0002 CHECK domain, but `activity_start` 0018 raises 'unknown activity') wedges every player's arrivals every 30s; any composed writer raise inside the tick body wedges every player's combat every 3s.

Fix — the house pattern, already proven. `process_build_queue` (mig 0194, the EV-1 0182 / SHIPYARD-2 precedent) already wraps each order's completion in its OWN begin/exception subtransaction. This slice applies that same per-row guard to the

two unguarded crons: each per-row body (per-movement in `process_fleet_movements`, per-encounter in `process_combat_ticks`) now runs in its own subtransaction — on a row raise it logs a `WARNING` (row id + `sqlerrm`), rolls that row back to its pre-iteration state, and `CONTINUES` to the next row; `query_canceled` is RE-RAISED (never swallow a statement-timeout cancel — the 0194/0182 posture). `v_count` sits inside the guard, so a failed row is `UNCOUNTED`.

Parity discipline (ABSOLUTE — LIVE hot crons). Each function was re-created from its grep-verified `TRUE` head (`process_fleet_movements` ← 0151, `process_combat_ticks` ← 0195 SHIELD-1) with `ONE` marked `CRON-GUARD` (0206) hunk — the begin/exception wrapper. Extract-and-diff verified: after removing the added wrapper lines the body is `BYTE-IDENTICAL` to its head (empty diff). Every accumulated hunk survives — combat keeps the `SHIELD-1` regen/absorb/pool/gated-leaf + hp sync, the hull-only integrity + both defeat predicates, the `D1` legacy-key idiom, the whole reward pipeline. On the `SUCCESS` path a subtransaction around a non-erroring body is transparent → behavior byte-identical to today (the existing `TEAMSETTLE` / `COMBATPARITY` / `TEAMHUNT` settle/tick proofs run their exacts against these re-created bodies and stay green). The change manifests **ONLY** on the error path, where skip+log+continue is strictly safer than aborting the whole cron for everyone.

No flag — pure hardening, deploy-safe (byte-identical on the normal path; strictly-safer on the error path). No gate, no data flip, no schedule change. Server-only — zero `src/` changes.

Trade-off (documented; matches 0194 exactly). A persistently-failing row is left in its pre-iteration state and retries every tick — it can spin forever — but it **NO LONGER WEDGES OTHERS**. Advancing a poison row to a terminal state to stop the retry (and the audit's `G3` legacy arrival status re-check) are **SEPARATE** behavior changes — deliberately out of scope, named follow-ups.

Proof. `scripts/team-command-proof.{sql,sh}` extended with `TEAMCMD_PASS_CRONGUARD` (the 30th marker): on a `FRESH` fixture user per arm, a `POISONED` row (a `mine_resource`-repointed movement whose settle raises; a `ship_hp-0` unit forcing the tick's `ceil(hp/ship_hp)` to divide by zero) does **NOT** wedge the run — a `SECOND HEALTHY` row in the **SAME** run still processes, the poison is logged+skipped (left in place, `UNCOUNTED`, never silently completed), and un-poisoning self-heals it next run (the `SHIPYARD-2 DELIVERY_GUARD` shape). Selftest green + 3 mutation guttings (the two anti-wedge asserts + the not-silently-completed assert) fail-then-restore. The migration's own self-asserts pin the guard shape, the loop-body wrap (token order), byte-parity of both bodies, every surviving hunk, determinism, and the closed client `ACL`.

Open follow-ups

- (follow-up, separate behavior change) `G3`: the legacy arrival status re-check.
- (follow-up, separate behavior change) advance a persistently-failing cron row to a terminal state to stop the infinite per-tick retry (a poison row currently spins forever, but no longer wedges others).

2026-07-15 — COMMAND-BUFFS: the FINALE of the fleet reshape — fleet-wide command buffs (mig 0205, DARK behind `command_buffs_enabled`)

Request (owner). "Each tier will have ~10 buffs, assigned **RANDOMLY** when bought or manufactured. More buffs added later — keep in mind for spaghetti." + "Command ship will provide those buffs ... a buff slot, activated only when the ship is set on command ship." + (on the fleet-wide themed bonus) "That is the spirit." Each ship rolls a tiered command buff at build; that buff applies **FLEET-WIDE**, but only when the ship is set as its fleet's **COMMAND SHIP**.

No spaghetti — this maps onto SHIP-SOUL almost exactly (reuse, never a new mechanism).

1. **Catalog** — `command_buff_types` (the `ship_trait_types` 0186 mold: `buff_id` collate-"C" / tier / name / description / stats_json in the shared 0180/0198 adapter vocabulary), organized by ship **TIER** via the new additive `main_ship_hull_types.tier` column (starter_frigate=T0; bulk_hauler/strike_corvette=T1 — the 0185 split). ~10 buffs seeded per tier (themed: gunnery→fleet attack, engineering→fleet speed, logistics→fleet cargo, ...; modest **FLEET-WIDE** magnitudes). **EXTENSIBLE**: additive on-conflict-do-nothing seeds — a later buff never re-derives a rolled ship (the **SOUL** catalog-growth handling).
2. **Roll** — the ship **BUFF SLOT** `main_ship_instances.command_buff_id` (nullable FK), rolled **DETERMINISTICALLY** (`hashtextextended('<ship_id>:cmdbuff',0)` → the tier pool's collate-"C" order — the 0186 pure-hash law, no RNG) by an **AFTER-INSERT** trigger (`trg_command_buff_roll` — the **ROOMS-8** 0203 seed-trigger seam, covers every commission path **WITHOUT** re-creating a commission function) + a monotonic backfill, both through the **ONE** writer `command_buff_roll_for_ship`. **IMMUTABLE** once set (NULL-guarded write). **ALWAYS-ON** additive data (NOT flag-gated) — inert until the fold.

The adapter fold (the ONE re-create — parity critical). `calculate_expedition_stats` `TRUE` head grep-verified **0198** (`NANGUARD`; `FLEET-CONTROL` 0204 and `ROOMS-8` 0203 did **NOT** touch it). Re-created from that head **VERBATIM** with `ONE` marked **COMMAND-BUFFS** hunk: when `command_buffs_enabled` **AND** the ship is in a fleet (`group_id` not null), fold the fleet's **ACTIVE** command ship(s)' `command_buff_id` → `command_buff_types.stats_json` **FLEET-WIDE** into **THIS** ship's totals (the shared additive 8-key fold; multiple command ships sum — backups; **NO** tradeoff **CASE** / **NO** scaling — a command buff is a pure fleet bonus). **DOUBLE-GATED**: flag false → the loop is skipped entirely (dark = byte-identical); flag true + ungrouped / no command ship → empty loop (byte-identical) — the **DECKS-3/level** double inertness. **Extract-and-diff CONFIRMED**: the re-create is the

0198 body byte-identical except the marked hunk (3 declares + the fold loop + + v_cmdbuff_speed_bonus in the ONE speed multiplier) — **every accumulated 0115 modules / 0122 captains / 0170 hull / 0180 level / 0193 traits / 0196 affinity / 0198 nanguard hunk survived byte-for-byte** (re-pinned by the \$9 accumulated-hunk law). DEPENDENCY (documented): the fold needs fleet_control_enabled too (is_command_ship is only meaningfully set through FLEET-CONTROL); the fold itself gates on command_buffs_enabled alone.

Client. ShipDossier gains a runtime-flag-gated **Command buff** line (name + effect + "applies to the whole fleet when this ship is the command ship") — commandBuff.ts (pure view, reuses the SOUL-2 traitEffects formatter) + commandBuffApi.ts (gate-first reads: config → catalog + the ship's command_buff_id) + the dossier section. DARK today (command_buffs_enabled false) → hidden, byte-identical.

Proof / activation / docs. team-command-proof gains TEAMCMD_PASS_CMDBUFF (29th marker): DARK parity (a designated command ship's buff inert, adapter byte-identical); LIT in-txn (flip command_buffs_enabled + fleet_control_enabled) — a fleet with a command ship whose buff = +X → every member's totals gain X exactly (independent catalog derivation), a non-command-ship's buff dormant, two command ships sum both buffs, a zero-command fleet folds nothing, an ungrouped ship folds nothing; the commission trigger's roll = the inline hash re-derivation. .sh selftest pins + the 3 gutting targets (buff-fold exactness / dark parity / no-command-no-buff). ACT-COMMAND-BUFFS (scripts/activate-command-buffs.{sql,sh}, command_buffs_enabled→true, catalog-freeze + buff-slot-coverage + FLEET-CONTROL-dependency preconditions). Docs: FULL_CAPACITY_PLAN \$FLEET buff slice, SYSTEM_BOUNDARIES (command_buff_id writer = the roll trigger; the command_buff_types catalog; the adapter's new read). LAWS held: parity ABSOLUTE (extract-and-diff, marked hunk only, dark byte-identical), no spaghetti (SOUL catalog+roll + ROOMS-8 trigger + shared adapter fold idiom reused), 0041 determinism, deploy-inert dark. Left UNCOMMITTED.

2026-07-14 — ROOMS-8: configurable ship rooms — 8 slots, a large room catalog, per-slot config (mig 0203, DARK behind captain_assignment_enabled)

Request (owner). "Each ship will have 8 captain slots, and the captains will be inserted accordingly." + "create many rooms as possible" (Command Deck, Gunnery, Engine Room, Outpost, Medbay, Sensors, Armory, Cargo Hold, Bridge, Sickbay ...) + "we can also change room by modifying the ships ... able to choose." Reshape ships so each has 8 CONFIGURABLE room-slots drawn from a large extensible catalog, captains staff the rooms. EXTEND the 0189 decks system — do NOT fork a parallel one.

The load-bearing constraint (kept). DECKS-3's adapter calculate_expedition_stats (head 0196) folds a captain's affinity via left join ship_stations st on st.station_id = a.station. ROOMS-8 keeps ship_captain_assignments.station = a ship_stations.station_id (now a "room type"), so that read-shape is BYTE-UNTOUCHED and the adapter is **NOT re-created** (confirmed absent from the diff — a parallel COMMAND-BUFFS slice owns the next adapter re-create; two in flight = the 0143/0146 collision).

Design (built exactly — mig 0203).

- Captain slots 6→8** — the 0171 pinned-bump idiom verbatim: every hull's base_captain_slots→8 + the instance backfill, <-guarded (monotonic/idempotent). Player-visible-cosmetic on deploy (the 0171 accepted posture — no seat can be occupied while dark). IRREVERSIBILITY LAW: roll back flags, never slots.
- A large room catalog** — ship_stations IS the room catalog; ADD eight rooms (command_deck, armory, cargo_hold, workshop, comms, outpost, sickbay, observatory; sorts 7..14, affinities in the 0117 vocabulary or NULL) via on-conflict-do-nothing. The frozen six (0189/0196) keep their ids/sorts/affinities — the DECKS-3 mapping is verbatim. Extensible: more rooms = one additive seed.
- 8 configurable room-slots per ship** — NEW ship_room_slots (main_ship_id, slot_index 1..8, room_type_id → ship_stations; **distinct rooms per ship** unique, so a slot's room is a unique placement key exactly like the 0189 station axis). Defaults = the 8 lowest-sort rooms, seeded by an AFTER-INSERT trigger on main_ship_instances (covers EVERY commission path without re-creating one)


```
<font name="Sym"> </font> a monotonic backfill for existing ships. Sole writer ship_room_configure (client wrapper
      configure_ship_room); read get_my_ship_room_slots; both dark-gated on the SAME captain flag (no new flag).
```
- Slot-scoped captain assignment** — captain_assign_apply re-created from its TRUE head (0189, its only re-create) with the marked -- ROOMS-8 (0203) hunks ONLY: the explicit-room existence check + the auto-assign walk now scope to THIS ship's configured slots (was the whole catalog). Signature unchanged → create-or-replace; the station insert, headcount cap, and every 0189 reject/order are byte-identical (extract-and-diff). A captain can only staff a room the ship has FITTED (unknown_station for any non-fitted catalog room).
- Client (ShipDossier).** The Captains section becomes the ROOM-SLOT board: 8 rows, each a room picker (room-pick-<i> → configure_ship_room via the CaptainsPanel runGuardedCommand idiom, disabled while a captain staffs it) + the staffing captain (dossier-captain-<id> + CaptainXpBar) or "Empty room". New pure helpers roomSlotBoard/roomPickerOptions (specs tests/roomSlots.spec.ts). Server-lit gated → renders NOTHING on prod today (deploy-inert). deckBoard kept (still tested).

Gating. Everything rides the existing captain gate `captain_assignment_enabled` (DARK, unflipped). The slot bump + catalog seeds + slot rows are additive/monotonic data no live path reads while dark; the config/read RPCs reject `captain_assignment_disabled` before any read (anti-probe, no oracle).

Proof. `scripts/decks-proof.{sql,sh}` extended (DECKS is its own standalone pair): 8-slot default seed, slot-scoped named/auto/non-fitted assign, room config happy + room_duplicate/unknown_room/ invalid_slot/room_occupied rejects, cap-first authority at the 9th captain, the **DECKS-3 affinity fold still fires through the preserved read-shape** (`ROOMS8_PASS_AFFINITY` — knob raised in-txn → combat_power rises), deterministic monotonic station + slot backfills. Selftest green + 3 mutation guttings fail-then-restore. The 6→8 raise's blast radius (team-command-proof, shipyard-proof, activate-captains.{sql,sh}) reconciled 6→8 the same way 0171 did 2→6.

Verify. `tsc -b + vite build` green; `roomSlots+deckStations+ship/team` pure battery green; eslint clean on touched files; the adapter confirmed NOT in the diff. Left UNCOMMITTED (the human Supabase apply gate).

Known fail-safe follow-up (NOT fixed here — separate small client slice). `CaptainsPanel`'s assign station-picker still lists FREE rooms from the WHOLE `ship_stations` catalog (14) rather than the ship's 8 FITTED slots. It is fail-safe: its AUTO default resolves the lowest-sort fitted free room server-side, and an explicit pick of a non-fitted room gets the server's honest `unknown_station` reject. Scoping the picker to fitted-only rooms (it would need to read `get_my_ship_room_slots`) is deferred — the panel is dark, and ROOMS-8's client scope was the ShipDossier room board.

2026-07-14 — FLEET-CONTROL: the fleet control-model (mig 0204, DARK behind `fleet_control_enabled`)

Request (owner directive — the core of the fleet control-model reshape). After FLEET-RENAME (team→fleet copy) merged, add the MECHANICS, DARK behind a new flag `fleet_control_enabled` (seeded false) so the LIVE fleet-command game is byte-unchanged until the owner flips it: (1) caps — max 3 fleets (already), up to 8 ships each, min 1; (2) a fleet needs ≥1 COMMAND SHIP to be active; (3) everything moves as a FLEET — a lone ship can't move.

Design (built exactly, all gated on the flag).

- **Command-ship designation.** Additive `main_ship_instances.is_command_ship` boolean not null default false
`<font name="Sym"> </font>` a partial index `idx_msi_command_ship (group_id)` where `is_command_ship`. Meaningful only relative to the ship's fleet (`group_id`); a fleet may have MULTIPLE command ships (extras are backups). New RPC `set_fleet_command_ship(p_main_ship_id, p_is_command)` — owner-scoped (the 0081 resolver), the SOLE writer. DECISION (documented in the migration): the designation is ALWAYS settable (NOT flag-gated — additive data, inert until the flag gates movement); setting to true requires the ship be in a fleet (`ship_not_in_fleet`), clearing is always allowed. **The command role is PER-FLEET and cleared on move** (review): when `assign_ship_to_group` changes a ship's `group_id` (moved to a different fleet, or unassigned) it resets `is_command_ship` to false — always-on, inert while dark — so command must be explicitly re-designated in each fleet (never a silently-inherited role).
- **A fleet with zero command ships is INACTIVE.** The three LIVE group movement RPCs (`send_ship_group_expedition` 0187 head, `move_ship_group_to_location` 0190 head, `send_ship_group_hunt` **0199** head — the NO-HOME 3-arg widening, NOT 0168) are re-created from their TRUE heads with ONE marked flag-gated hunk each: `if v_fleet_control then <reject fleet_inactive_no_command if no command ship> end`. Placed after the empty-group check, BEFORE the destination/lock/readiness reads (reject-before-read on the caller's own fleet). DARK = byte-identical (extract-and-diff confirmed: the only non-hunk delta is the added `v_fleet_control` declare + an unused `v_members` counter in assign).
- **The 8-ship-per-fleet cap.** `assign_ship_to_group` (0161 head) re-created with a marked flag-gated hunk: `lit → assigning to a fleet already holding 8 OTHER members rejects fleet_full`; `dark → no cap` (current behavior). Unassign and re-assign of the same ship are never capped.
- **Client (all runtime-flag-gated via `strictConfigFlag('fleet_control_enabled')` / `fetchFleetControlEnabled`, dark = today byte-identical).** `teamApi.setFleetCommandShip` wrapper + `fetchMyShipGroupMap` widened with `is_command_ship`. Pure mirrors in `teamRoster.ts` (`fleetCommandState`, `fleetCapacityState`, `canToggleCommandShip`, `FLEET_MAX_SHIPS=8`) + `tests/fleetControl.spec.ts`. `TeamRosterPanel`: the "Set as command ship" / "Stand down" control + a "Command ship" badge, the fleet Active/inactive indicator ("Fleet inactive — set a command ship"), and the 8-cap surfaced in the add-ship picker. Deploy-window safety (review HIGH): `fetchMyShipGroupMap`'s base membership select keeps ONLY pre-existing columns and reads `is_command_ship` in a SEPARATE query folded by the pure `buildShipGroupMap` — a missing column on a pre-0204 DB (the client auto-deploys ahead of the approval-gated migration) degrades to "no command-ship data", NEVER to blanked live rosters; pinned by a `tests/fleetControl.spec.ts` null-command-read spec. `TeamMapSend`: disables Send/Move/Hunt + a hint when the fleet is inactive. `MainShipCommand`: hides the per-ship Move affordance when lit and shows guidance ("Add this ship to a fleet to move it" / "Move ... with its fleet from the Fleets screen"); wired via new shell fields `fleetControlEnabled` + `mainShipInFleet`. `teamReasonMessage` maps the three new rejects.
- **Proof.** `team-command-proof.{sql,sh}` gained `TEAMCMD_PASS_FLEETCTRL` (the 27th marker; NOHOME is now the 28th tail): DARK arm (a zero-command fleet SENDS; a 9th assign SUCCEEDS — no cap) then in-txn LIT arm (zero command →

`fleet_inactive_no_command` on send/move/hunt; designate → active + `is_command_ship` persists; 8th assign OK, 9th `fleet_full`, held at 8; ungrouped → `ship_not_in_fleet`; stand-down re-inactivates). The flag flips in-txn only + the committed-false honesty loop covers it. Selftest green; 3 mutation guttings fail-then-restore verified.

- **Activation.** `scripts/activate-fleet-control.{sql,sh}` — FLAG-ONLY flip with preconditions (0204 recorded, the surface prosrc-pinned, `team_command` committed true) + a smoke FYI counting the fleets that go inactive at flip time. NO client PR needed (runtime-flag-gated).

Anti-spaghetti / parity. Reused `ship_groups` + the group RPCs; `is_command_ship` is ONE additive column; NO adapter (`calculate_expedition_stats` untouched), NO decks/ShipDossier touched. Parity discipline held: the 3 live group RPCs re-created from true heads, marked flag-gated hunks, DARK byte-identical. Migration 0204 (0203 taken by ROOMS-8, already merged). tsc + vite + eslint green; team pure battery green.

2026-07-14 — REPAIR-ECON: paid hull-repair economy (mig 0201, DARK behind `repair_economy_enabled`)

Request (FULL_CAPACITY_PLAN §C P9 / gap G8). Ships come home DAMAGED (not deleted); repair is the recovery loop. Today `repair_main_ship` (0052 head, re-created verbatim-else-branch by 0199) is the FREE, INSTANT safelock — it only recovers a destroyed ship (restores full hp + re-homes/docks), the temporary no-progress-loss guarantee. G8: the ECONOMY is what's missing, not that destroyed-recovery should cost. Build the paid repair as a real economy, DARK behind a new flag, WITHOUT breaking the safelock.

Design (built exactly).

- **A NEW additive RPC, not a re-create of the safelock.** `repair_ship_hull_at_port(p_main_ship_id, p_repair_hp, p_request_id)` mends a DAMAGED-but-alive ship's HULL for credits at a port. The live `repair_main_ship / dev_set_main_ship_destroyed` are UNTOUCHED (not in the diff) — zero parity risk.
- **THE SEAM (the G8 mandate).** DESTROYED ships (`status='destroyed'`) are REJECTED by the paid path (`ship_destroyed`) and keep the FREE, ungated, instant `repair_main_ship` safelock. The two recovery paths never overlap: destroyed → free safelock (unchanged); damaged-alive → new paid repair.
- **Reject-before-read, all-or-nothing, idempotent** (the 0174 salvage mold): `not_authenticated` → `repair_economy_disabled` (gate FIRST) → `invalid_amount` (integer hp — `null/≤0/fractional/>1e6`) → `ship_not_found` → `ship_destroyed` → `not_docked` → `idempotent_replay` → `nothing_to_repair` → `repair_misconfigured` → `insufficient_credits` → ok. Fan-out downward only: reused `mainship_resolve_owned_ship` (ownership) / `mainship_space_lock_context` (per-ship lock) / `mainship_resolve_docked_location` (the settled-safe dock rule) → `cfg_num` knob → `wallet_debit` (0093, false-if-poor) → its own hp heal → its own `repair_receipts` (new table, the salvage_receipts shape: unique (main_ship_id, request_id), owner-read via the ship). The request is CLAMPED to the actual missing hull (over-request tops up to max_hp, never over-charges).
- **Cost model [D owner-tunable].** `total = hp_restored × repair_credits_per_hp`, seeded conservatively **0.5**: a full 500-hp Frigate rebuild = 250 cr (one ship's price); a ~120-hp combat dent = 60 cr (a Snare-run's salvage, the 0174 30..80 band). A retune is a one-row `set_game_config` write, no deploy.
- **v1 scope (deliberate — the honest-minimal slice).** HULL only (shield SELF-REGENS, 0197 — paying to top a self-refilling bar is a non-feature); CREDITS only; INSTANT at-port. FULL_CAPACITY_PLAN RR-1's richer model (`repair_parts` materials + M4.5-queue duration) and MAINSHIP_TRANSITION §13's open cost/duration questions are the documented FOLLOW-UP behind the SAME flag ([D] `repair_parts_per_hp / repair_seconds_per_hp`) — a re-create of THIS new RPC, never the live safelock. (ROADMAP's Repair & Recovery section points to §13's cost/duration as OPEN design; the directive's steer was v1 = simplest honest, so instant + credits-only, divergence documented.)

Client (dark, the SalvageMarketPanel mold). `RepairPanel` on the Port main rail: strict `strictConfigFlag` fold of `repair_economy_enabled` read FIRST from public `game_config` (renders NOTHING while dark → production byte-unchanged), sticky-lit, an owner-read of the chosen ship's hull (Meter bar), a whole-hp stepper (default = full mend) + `Full` button, server-priced cost, ONE Repair (server- receipted, advise-on-shortfall). A DESTROYED ship shows the free-recovery note here, never a paid button. Pure mirror `repairEconomy.ts` (reuses `strictConfigFlag` + salvage's `foldStartingCredits / salvageStickyLit / salvageWalletDisplay`), `repairApi.ts`, `repairReasonMessage.ts`.

Proof. Standalone `scripts/repair-econ-proof.{sql,sh}` + `.github/workflows/repair-econ-proof.yml` (NOT team-command-proof — a parallel MOD2-2 slice owns that block), reusing `scripts/lib/trade-proof-lib.sh`. REPAIR_PASS_* markers: dark gate (no oracle) · 0.5 knob seed · full mend (120 hp → clamp → restore 120, debit 60 exact, hull→max, 1 receipt) · partial mend (40 of 100 → 20 cr) · replay idempotency (no double debit/heal/receipt) · guards (`invalid_amount` 0/-3/2.5, cross-player `ship_not_found`, full-hull `nothing_to_repair`, in-transit `not_docked`, broke `insufficient_credits` — all zero-write) · THE SAFELock SEAM (destroyed rejected by the paid path + recovered FREE by `repair_main_ship` even with the economy flag LIT). Selftest green; 3 mutation guttings fail-then-restore.

ACT-REPAIR. `scripts/activate-repair-econ.{sql,sh}` (the activate idiom): one timed UTC BEGIN..COMMIT flipping `repair_economy_enabled`; preconditions pin the paid-RPC gate/seam bodies AND that `repair_main_ship` stays UNGATED + destroyed-only (the free path is unaffected by the flip); a zero-write gate-open smoke under a fake JWT. Awaits the owner's flip.

Verify. `repair-econ-proof.sh selftest + activate-repair-econ.sh selftest` green; 3 guttings caught; `tsc -b + vite build` green (RepairPanel mounts on the Port screen); full pure battery (637 specs, `galaxy.spec.ts` excluded) green incl. 21 new repair specs; eslint clean on touched files. **Migration 0201; DARK; deploy-inert; UNCOMMITTED.**

2026-07-14 — MOD2-2: the Mk-II module tier (mig 0202, DARK behind the EXISTING module gates)

Request. P7 MODULES-2, next slice: the Mk-II module tier, dark behind the existing module gates (NO new flag). MOD2-1 (mig 0183) shipped the base defense/mining line; MOD2-2 is the higher tier the P7 line specs — `autocannon_battery_mk2` (attack 18) "same for shield", `slot_cost 2` so 3-slot ships face a real tradeoff. Mirror MOD2-1 exactly: pure catalog content, zero engine edits.

Built (mig 0202, uncommitted — the exact 0183 shape).

- **Two Mk-II module_types rows.** `autocannon_battery_mk2` (slot_type weapon, slot_cost 2, {"attack": 18}) and `shield_lattice_mk2` (slot_type defense, slot_cost 2, {"defense": 20}). The tier is a flat +8 on the base stat across both lines (autocannon 10→18, shield 12→20) — one rule, [D] owner-tunable. `slot_cost 2` is the capacity-tradeoff law made real: a 3-slot frigate carries at most one Mk-II + one base module. Both REUSE shipped archetypes (no new slot_type) — the weapon arm pays the adapter's existing attention/speed tradeoff ($\times \text{slot_cost}$), the defense arm the else-0 posture (0183's shield).
- **Six recipe rows (live drops only, F4).** Both Mk-II share the progression pair `blueprint_fragment 2 + artifact_core 1`; the third is the line's base component (`weapon_parts 6` for the autocannon, `repair_parts 6` for the shield — mirroring the base tier's split). Grounding self-asserted: `blueprint_fragment/weapon_parts/repair_parts` vs the deployed `pirate_loot_for_wave` prosrc (head 0185); `artifact_core` vs the Singularity Scar mining-field bundle (0103, the crystal/ore idiom — it is no combat drop). **HONEST FAUCET GATE:** `blueprint_fragment`'s combat faucet (`blueprint_fragment_drop_rate`) is committed 0, and its only non-combat source is the SINGLE one-shot exploration site (0098, qty 1 — it is NOT a mining-field drop; 0103 seeds `artifact_core` but no `blueprint_fragment`). So the shipped-config lifetime ceiling for `blueprint_fragment` is 1, but each Mk-II needs qty 2 → **both Mk-II modules are UNCRAFTABLE under every shipped config until the combat faucet is lit** (`blueprint_fragment_drop_rate > 0`) — a SEPARATE activation this slice does not ride. This is a deliberate deep-gate that self-corrects at the flip, the EXACT dependency SHIPYARD-0's T1 ships already carry (`bulk_hauler / strike_corvette` each need `blueprint_fragment 2` behind the same closed faucet, 0185).
- **Self-asserts (0183 verbatim shape).** exact seeded shapes (2 modules + 6 recipe rows, no strays); each fits the smallest hull (`slot_cost 2 ≤ min base_module_slots`, today `bulk_hauler`'s 2); every ingredient item-typed with a live drop source; the stats-key prosrc pins (`attack/defense` are adapter-read, `combat_power/survival` outputs present) PLUS a new pin that the weapon slot_type tradeoff ($\text{attention } 2 / \text{speed } 0.03 \text{ per slot_cost}$) is a live adapter rule — so a future adapter can't silently make the Mk-II a free upgrade; craftable shape; **no new flag** (the gates keep their committed value — the seed is deploy-inert additive catalog content behind the SAME `module_crafting_enabled/module_fitting_enabled` gates as MOD2-1).

Deploy-inert. Landing 0202 changes NO player-visible behavior until (and only until) the existing module gates are lit — the same captains/module activation path MOD2-1 rides. No new mechanic, no flag.

Proof — the sibling TEAMCMD_PASS_MOD22 block (extends `team-command-proof.{sql,sh}`; marker count 26→27). Two fresh fixture users, one Mk-II each (a slot-2 + slot-2 pair overflows the 3-slot starter frigate): the shield fit lands **survival +20 exactly** on the else-0 arm (nothing else but 2 slots), and the autocannon fit lands **combat_power +18** plus the **first end-to-end pin of the weapon tradeoff arm — pirate_attention +4** ($\text{weapon } 2 \times \text{slot_cost } 2$) and **speed $\times 0.94$ exactly** (the slot-2 penalty, guarded against a 0-penalty false-green), nothing else but 2 slots. Both: exact-price craft spends to 0 with the `insufficient_items` boundary + minus-key isolation. Also bumped the module-catalog verify contracts (`verify-modules/verify-fitting`) 6→8 types / 18→24 recipe rows.

Verify. Proof selftest green with the new MOD22 block; 3 mutation guttings (`survival +20 → +19`, `combat_power +18 → +17`, `speed penalty 0.06 → 0.0`) each fail-then-restore; verify scripts syntax-clean with the bumped contracts. Server-only slice — zero `src/` changes. Left UNCOMMITTED.

2026-07-13 — NO-HOME: launch-from-dock + dock-at-return (mig 0199, DARK behind `launch_from_dock_enabled`)

Request (owner's absolute law). There is NO home base; ports are the only base; a ship acts from WHEREVER it is docked. Today SEND (send_main_ship_expedition, 0169) and HUNT (send_ship_group_hunt, 0168) require ship status='home' and launch the outbound fleet from the legacy invisible base at (0,0); a docked ship (status='stationary' + spatial_state='at_location') can MOVE (move_main_ship_to_location, 0156) but cannot SEND/HUNT — the bug. Fix it, DARK behind a new flag so prod is untouched until the owner flips it.

Design (locked with the owner, built exactly).

- **Additive dual-state launch, not removal.** Send/hunt ACCEPT a docked ship as a legal launch state IN ADDITION to home — mirroring the settled-safe rule the game already uses for explore/mine/fit/ captain (spatial_state in ('home', 'at_location'), 0100/0105/0114/0121). A docked launch departs FROM its docked port (the present-fleet origin, the 0156 template), NOT the (0,0) base.
- **Player-chosen return port.** Send/hunt gain a trailing p_return_location_id uuid DEFAULT NULL. The port is recorded on the fleet (additive nullable fleets.return_location_id). A send-TO-a-port docks at the destination (the return param is recorded-but-inert; movement_settle_arrival already docks a dockable arrival). A HUNT (non-dockable combat site) docks the team at the recorded return port after combat: the reconciler process_mainship_expeditions DOCKS the returning ship there (the canonical pair via the 0153 helper mainship_mark_docked_at_location + a leaf that re-presents the fleet at the port) instead of re-homing.
- **Flag-gated.** New flag launch_from_dock_enabled seeded false. DARK → send/hunt/reconciler/ repair behave EXACTLY as today (home-only, base origin, re-home) — every else-branch is the grep-verified TRUE head body verbatim. LIT → the dual-state + docked launch + dock-at-return.

Work done.

- **Migration 0199** — flag seed (dark) + fleets.return_location_id; re-creates (parity discipline, if v_launch_from_dock then <NOHOME> else <head verbatim> end): send_main_ship_expedition (DROP+CREATE +param), send_ship_group_hunt (DROP+CREATE +param), process_mainship_expeditions (CREATE OR REPLACE + the nohome_dock_returning_ship leaf), repair_main_ship (CREATE OR REPLACE — LIT revives docked, not home; recovery still always works). send_ship_group_expedition (0187) is UNTOUCHED — it calls the single send with 2 positional args, which resolve to the widened function via the DEFAULT. Self-asserts pin the flag dark, the column, and the DARK heads verbatim.
- **ACT-NOHOME** — scripts/activate-nohome.{sql,sh} (the activate-team-command idiom): flips the flag true with preconditions + smoke, one timed BEGIN..COMMIT, commented rollback. Selftest green.
- **Proof** — scripts/team-command-proof.{sql,sh} gains TEAMCMD_PASS_NOHOME (the 26th marker): DARK arm = a docked ship's send RAISES home-only (byte-identical witness); LIT arm in-txn = a docked ship re-departs its OWN present fleet from the port + docks at the destination, and a docked team hunt launches from the port with the reconciler docking the returning member at the recorded return port — never home. The flag is added to the committed-false honesty loop + the gate lists.
- **Client (DARK byte-identical — every gate flag defaults false):** fetchLaunchFromDockEnabled (catalog.ts, REUSING strictConfigFlag); teamMapSendAction gains launchFromDock (lit → docked_unready becomes send); TeamMapSend + TeamRosterPanel read the flag and treat a docked-together team as hunt-ready + pass the return port; teamApi.sendShipGroupHunt gains an optional returnLocationId; mainshipStatusLabel maps stationary → "Docked – ready to launch". Single-ship docked launch already works via move_main_ship_to_location (canMoveHere), so MainShipCommand needed no gate change.

Verification. tsc -b + vite build green; eslint clean on touched files; the pure battery (team + teammap + resolver + status-label specs, 171+12 cases) green, incl. new NO-HOME classifier cases; ACT-NOHOME + ACT-TEAMCMD + team-command-proof .sh selftests green. **DB-dependent proof (selftest apply + mutation guttings) NOT run — docker/supabase-local unavailable on this machine (the standing Supabase apply gate); the SQL + proof are written to be applied+run by the human behind that gate.**

2026-07-13 — NANGUARD: fix the inherited no-op NaN guards at all three knob sites (mig 0198, inert)

Request. The NANGUARD follow-up (FULL_CAPACITY_PLAN queue row 20, found by the DECKS-3 hostile review): the house case when v_raw <> v_raw then 0 else v_raw end NaN-detect idiom is a NO-OP in PostgreSQL — PG deviates from IEEE-754 and makes 'NaN'::float8 = 'NaN'::float8 TRUE, so the x <> x arm is UNREACHABLE and a knob mis-set to the JSON string "NaN" passes straight through greatest() (NaN sorts above every numeric in PG) to poison / abort the guarded math. A real latent bug, inert only because the knobs are seeded '0' today.

Work done

- **True heads (grep-verified across ALL migrations):** calculate_expedition_stats ← 0196 (creates 0044→0115→0122→0170→0180→0193→0196, nothing later) — TWO guards live here: the 0180 LEVEL knob (captain_level_bonus_per_level) and the 0196 AFFINITY knob (station_affinity_bonus). process_mainship_expeditions ←

0197 (creates 0050→0169→0197) — ONE guard: the 0197 IDLE-REGEN knob (`shield_regen_idle_pct`). **Checked and rejected a 4th site:** `process_combat_ticks` (0195) reads `shield_regen_combat_pct` as a bare `coalesce(cfg_num(...),0)` — NO `x <> x` guard at all, so there is nothing to fix there. **THREE guard sites, not four.**

- **The fix (mig 0198, uncommitted):** re-created both affected functions from their TRUE heads VERBATIM except the ONE marked `-- NANGUARD` (0198) hunk per guard — the operator `<>` becomes the working `= 'NaN':double precision` (the 0182 worldstate-knob precedent, 0182:204-207) and the adjacent honesty-note comment that called the arm a no-op is corrected. Extract-and-diff clean: the ONLY code delta is the guard operator at each of the three sites; every accumulated hunk (0115/0122/0170/0180/0193/0196 in the adapter; the 0169 CTEs + 0197 regen hunk in the reconciler) is byte-identical.
- **Behaviorally inert at the seeds.** All three knobs are '0' today, and 0 is not NaN under either idiom — so the fix changes NOTHING at the committed seeds. Every existing proof stayed green unchanged. It is a correctness/robustness fix that bites ONLY a post-flip bad knob write.
- **Proof.** Added `TEAMCMD_PASS_NANGUARD` (25th marker) to `team-command-proof.{sql,sh}`: it POISONS a knob with the jsonb "NaN" string in-txn and proves the fixed guard floors it to 0 — the affinity witness (a real gunnery-matched captain, non-vacuous) leaves the adapter byte-identical to the knob-0 baseline with a finite `combat_power` (never NaN); the idle-regen witness leaves the reconciler a clean no-op (shield stays 3, no `ceil(NaN)::integer` abort). Both knobs restored to '0' in-txn. This block would have FAILED before 0198 (NaN propagates/aborts) — the witness the inert envelope can't show. Self-asserts in 0198 pin the working token present + the dead `<>` token gone at all three sites, the accumulated head hunks surviving, and the knobs still committed '0'.

Verification. `team-command-proof.sh selftest` green (25 markers); 3 mutation guttings of the new NaN-floor pins fail-then-restore. Server-only — zero `src/` changes (tsc/vite N/A). Left uncommitted.

2026-07-13 — SHIELD-2: out-of-combat regen + commission copy + the UI meter pair (mig 0197, inert — P13 complete dark)

Request. The SHIELD charter, final slice: the idle regen home, the deferred commission base_shield copy, and the shield/hull meter pair — everything inert while pools are 0/0 and `shield_regen_idle_pct` is '0'. Only the human ACT-SHIELD flip remains for P13.

Work done

- **True heads (grep-verified on main today):** `process_mainship_expeditions` ← 0169:446 (created 0050:234; ONLY re-create 0169 — SHIPYARD-2/SOUL-1/SHIELD-1 confirmed untouched); `ensure_main_ship_for_player` ← 0193:175 (0043 → 0077 → 0078 → 0193); `port_entry_commission_build` head is now **0194 on main** (SHIPYARD-2, PR #138, merged) — re-created from that merged 0194 body + the shield hunk (see collision notes).
- **mig 20260618000197_shield2_regen.sql** (0196 landed as DECKS-3, merged; 0197 is the next free slot on rebased main; recorded in the header):

Idle regen — the reconciler re-created (0169 head verbatim, extract-and-diff verified: only the marked hunk + one blank line differ) with ONE guarded set-based statement AFTER the two D3 CTEs: `shield := least(max_shield, shield + ceil(max_shield × knob))` where `shield < max_shield` (the 0191 partial index) and `status <> 'destroyed'` and NOT EXISTS an **active/retreating encounter membership** — the charter §1.3.2 double-writer EXCLUSION predicate: in a live encounter the 3s tick is the sole shield writer, outside it the reconciler is (a disjoint-writers partition, never a second lock). **DOUBLE-GUARDED zero-knob:** `least(max, shield+0)` = shield, but a same-value UPDATE still fires row writes — so if `v_idle > 0` then skips the statement ENTIRELY (zero writes = byte-inert cron, stated in the hunk). Knob read ONCE per invocation (declare initializer), carrying the C2-2 NaN-guard SHAPE for parity — **a documented no-op in PG** (NaN = NaN is true, so `x<>x` never fires; a 'NaN' knob is NOT caught — NANGUARD follow-up), floored at 0 (a negative can never drain). Return envelope unchanged (`v_count + v_team` — regen rows uncounted).

Commission copy — both creators' enumerated inserts gain `shield, max_shield := h.base_shield, h.base_shield`: **BORN FULL** (the `h.base_hp`, `h.base_hp` posture mirrored; [D] a fresh hull leaves the yard charged — ACT-SHIELD's backfill uses the same `shield=max` shape). 0/0 while every hull is 0 (migration-asserted) = byte-identical to the defaults it replaces. `ensure_main_ship_for_player` gets the copy too — its hull lookup was already in scope (two added column refs, the cheapest honest form) and the two creators must not fork the day `base_shield` rises.

Self-asserts: idle knob '0' + every hull/instance 0 (deploy-inert); regen hunk tokens + the guard-appears-before-statement order pin + exactly-one knob read + both D3 CTEs/race-guards byte-intact + envelope token; exactly 1 reconciler cron job; build re-signing (old (`uuid`) gone, starter default, Sparrow + `h.name` + FMRN + the SOUL-1 hook exactly once + fleets insert unmoved + the copy INSIDE the insert) with both callers still single-arg; ensure's copy + create-branch hook + commission lock; no `random()`; ACLs server-only; a live knob-0 reconciler smoke pass moves nothing.

- **UI — the meter pair (data-gated, NO flag — the 0191 posture):** ONE pure view-model (`src/features/ship/meterPair.ts` — shield reading null unless `max_shield > 0`, both pcts clamped 0–100, non-finite fails closed, the sr-only pair label) + ONE shared component (`src/features/ship/MeterPairBars.tsx` — shield ABOVE hull, the classic pair; hull markup = the pre-SHIELD-2 `ShipStatusCard` block verbatim). `ShipStatusCard` renders the pair in place of its hull bar (byte-identical DOM while shieldless); `ShipDossier` gains the WHOLE pair only when `max_shield > 0` (it had no bar before — zero new DOM on prod where every ship is 0/0); sr-only "Shield x/y · Hull x/y" rides the shield-shown branch only. `shield`, `max_shield` added to the enumerated owner-ship selects (`mainshipApi SHIP_COLS + MainShipRow`, the `useGalaxyMapData MainShipLite` read) — additive columns. `TeamRosterPanel/TeamMapSend` member rows carry NO hp today (`TeamRosterPanel:379` records it) → no shield readout there (skipped, documented — the pair arrives when member rows learn hp). Specs: `tests/shipMeterPair.spec.ts` (zero-max hidden / partial / full / clamping both ways / fail-closed non-finite / sr label).
- **Proof — TEAMCMD_PASS_SHIELD2** (24th marker; DECKS-3 landed as the 23rd, SHIELD-2 appended as the 24th — the established both-blocks-kept idiom): committed-'0' entry pin (so `TEAMSETTLE`'s five reconciler passes above ARE the knob-0 byte-parity witness — stated, not re-run); knob-0 ZERO-WRITES via a 1-hour-old `updated_at` sentinel on a 40/3 fixture (the statement never FIRES); lit arm in-txn via real `set_game_config` — knob 0.03 pins CEIL (3 → 5, floor would land 4), 0.25 climbs 5 → 15 exactly, least caps 35 → 40, a FULL pool with a re-pinned sentinel is never rewritten; the exclusion pins on REAL fixtures (a live 1-ship team-hunt encounter — instance shield untouched, non-vacuous vs the regen fixture — then the one-step-wipe defeat and a re-armed DESTROYED hull stays at 3); the commission copy births 25/25 through BOTH real creators under a sanctioned `base_shield = 25` hull surgery, restored. `.sh`: idle knob moved never-touch → raised-and-restored-in-txn (its consumer arrived — the SHIELD-1 combat-knob precedent); the hull-table negative grep TIGHTENED not dropped (insert/delete/copy always fail; updates only in the `set base_shield` form, raise + restore both required); 13 new assert-form pins; a committed-base_shield=0 honesty check after the local run.
- **Docs:** `FULL_CAPACITY_PLAN §C P13` (SHIELD-2 shipped; P13 complete dark; the ACT-SHIELD flip script charter written — preconditions/monotonic backfill/instance copy/knobs 0.02 combat + 0.10 idle [D] Sparrow 100 / Mule 130 / Talon 85 [D]/smoke) + queue row 18; `SYSTEM_BOUNDARIES` (`shield = TWO` runtime writers disjoint by the exclusion predicate; `max_shield` = the two creators' inserts exactly; the direct-set-statement-not-leaf [D] recorded); `DEV_LOG`.
- **Collision notes (all landed-fact now):** (a) 0196 → DECKS-3 (merged), 0197 taken. (b) `port_entry_commission_build`: PR #138 (SHIPYARD-2, mig 0194) merged and re-signed it (`uuid`, `text`); this slice carries the merged 0194 BODY + the shield hunk with order-robust drops (`drop function if exists (uuid) then create-or-replace (uuid, text)`, ACL re-asserted). The apply chain 0193 → 0194 → 0196 → 0197 lands every accumulated hunk in the last body (extract-and-diff verified byte-for-byte against merged 0194). (c) The proof marker slot: DECKS-3 landed as the 23rd, SHIELD-2 appended as the 24th.

Verification. `team-command-proof.sh selftest` green (24 markers as-built); 3 mutation guttings (zero-writes guard, regen exactness, exclusion predicate) each fail-then-restore; migration bodies extract-and-diffed against their heads (reconciler/build/ensure — only the marked hunks differ); `tsc -b + vite build` green; the full pure spec battery by explicit list green (incl. the new `shipMeterPair.spec.ts`); eslint clean on every touched src file (`tests/harness/galaxyCoordHarness.tsx` carries 3 PRE-EXISTING errors, not introduced here).

2026-07-13 — DECKS-3: station affinity bonuses — the adapter's knob-gated matched-station multiplier (mig 0196, inert)

Request. The DECKS charter, slice 3: when a captain's specialization matches their held station's `affinity_specialization` (`ship_stations`, 0189 — Gunnery=combat, Engineering=mining, Logistics=trade, Sensors=exploration, Medbay=support, Bridge=NULL), scale THAT captain's `stats_json` contribution in `calculate_expedition_stats` by $(1 + \text{station_affinity_bonus})$ — EXACTLY the shape of the shipped 0180 captain-level fold. Dark/inert behind the zero-seeded knob; proposed flip value 0.15 [D owner-tunable — documented, never seeded].

Work done

- **True head (grep-verified, re-verified after the SHIELD-1/SOUL-2/SHIPYARD-3 rebase):** `calculate_expedition_stats` → **0193** (SOUL-1: creates at 0044 → 0115 → 0122 → 0170 → 0180 → 0193; nothing later re-creates it — SHIELD-1's 0195 re-creates the 0168/0169 combat engine heads, never the adapter). Re-created from THE 0193 BODY with marked `--` DECKS-3 (0196) hunks only — extract-and-diff run during the build: the diff is exactly the marked hunks, every accumulated hunk (0115/0122/0170/0180/0193) carried.
- **mig 20260618000196_decks3_affinity.sql** (0194 promised to SHIPYARD-2's renumber; 0195 landed as SHIELD-1 — this slice renumbered to 0196) — (a) knob `station_affinity_bonus` seeded '0' (byte-inert: $x(1+0) = x1.0$ exactly — the 0180 double-inertness law, second arm = no-match $x1.0$ regardless of the knob; the 0180 guard SHAPE + floor-at-0 posture mirrored byte-for-byte, read ONCE at entry — never per row. **HONESTY NOTE (hostile review M1):** the inherited `x <> x` NaN-detect arm is a NO-OP in PostgreSQL — `'NaN'::float8 = 'NaN'::float8` is TRUE, PG deviates from IEEE 754 — so that arm is unreachable and a knob mis-set to "NaN" WOULD poison matched captains' stats; kept byte-for-byte anyway (0180 parity outweighs a mid-slice idiom fork), documented in-body with a marked note, and the paired fix is QUEUED as `FULL_CAPACITY_PLAN` row 20 **NANGUARD** — both adapter sites to the working = `'NaN'::float8` idiom + their pinned

self-asserts, one slice). (b) The adapter delta: ONE added LEFT join (`ship_captain_assignments.station → ship_stations.affinity_specialization`; LEFT deliberately — an unstationed row must KEEP folding at $\times 1.0$, an inner join would silently drop that captain's whole contribution; post-0189 no writer can even produce a station-NULL row — defense-in-depth), ONE no-match CASE (`v_aff_mult := case when affinity = specialization then 1 + knob else 1 end` — `NULL = x` is `NULL` → the ELSE, so unstationed and the NULL-affinity Bridge take the same literal-1 branch as a plain mismatch), and `* v_aff_mult` composed at the EXISTING eight 0180 scale sites (`contribution × v_lvl_mult × v_aff_mult` — commutative arithmetic, pinned token order; the captain's specialization was ALREADY in the loop via `captain_types` 0122, no new read). Tradeoffs stay affinity-flat (never a stealth cost raise). No second captain loop. (c) Self-asserts: knob committed + reads exactly 0 (deploy-inert or refuse); the 0189 six-station affinity mapping re-asserted verbatim; prosrc pins — 1 once-at-entry knob read, the 0180 guard/floor shape, 1 LEFT station join (and exactly 1 join `ship_stations` token total), 1 no-match CASE, exactly `8 * v_lvl_mult * v_aff_mult` composed sites and exactly `8 * v_aff_mult` total (no ninth site), tradeoffs unscaled, no `random()`; EVERY 0180 level pin + 0193 trait pin + 0115/0122/0170 hunk token RE-RUN on the re-create (the accumulated-hunk law); ACL server-only re-asserted.

- **Proof** — `TEAMCMD_PASS_DECKS3` appended to `team-command-proof` (the **23rd** marker, reconciled after SHIELD1's 22nd at the rebase — both blocks kept, the established idiom): committed seed '0' pinned; KNOB-0 PARITY — c1 with the CAPLEVEL level-2 pair exactly as the 0189 auto-assign stationed them (capa BRIDGE/NULL, capb GUNNERY/combat — a stationed MATCHING captain aboard) answers the PRE-DECKS3 expectation to the byte (`hull + Σ attack × lvl_mult`, derived independently — the CAPLEVEL arithmetic re-stated); LIT (knob 0.15 in-txn via the real `set_game_config`, the numeric-knob idiom) — `combat_power = baseline + knob × the MATCHED share EXACTLY` (`attack × lvl_mult × knob`: the bonus provably COMPOSES with a real level-2 multiplier, 1.1×1.15 — and the matched share is guarded REAL and guarded PARTIAL, so a bridge that wrongly matched would overshoot), every other envelope key byte-identical; NOMATCH — a fresh `medbay(support≠combat)+bridge(NULL)` board via the real sole writer's explicit-station arm answers `hull + Σ attack` exactly and is byte-identical between knob 0.15 and knob 0; UNSTATIONED — pinned STRUCTURALLY (the SHIELD0 prosrc-pin precedent: LEFT join + the literal-1 ELSE), honest reason recorded: no writer can produce a station-NULL row post-0189 (writer always resolves a station, the backfill nulled none, unassign deletes the row) and the harness may not fixture one (the Captain sole-writer negative grep). The .sh gains the DECKS3 assert-form pins, the 23-marker list, and the committed-knob honesty check (`station_affinity_bonus` still '0' post-run — the SHARDDROP posture). DECKS3 runs AFTER SHIELD1 (verified: SHIELD1 touches no captain/growth/trait state — its knob writes are restored in-txn — so the DECKS3 preconditions hold).
- **Docs**: `FULL_CAPACITY_PLAN` (queue row 19 — DECKS-3 shipped inert; **ACT-DECKS3 = ONE knob write**, `set_game_config('station_affinity_bonus', '0.15')` [D proposed, owner-tunable], rollback = the same write back to '0'; knob indexed; NEW row 20 **NANGUARD** — the paired NaN fix), `SYSTEM_BOUNDARIES` (§1 `ship_stations` — `affinity_specialization` went functional, the adapter is its downward reader; §8 the adapter's new LEFT-join edge into Captain's own catalog).
- **Collision notes (resolved at the rebases)**: SHIPYARD-2 landed as 0194 (PR #138 — its function set, `port_entry_commission_build`, is disjoint from this slice's and it carries its own proof file: the rebase over it changed nothing of this slice); SHIELD-1 landed FIRST (mig 0195, PR #139, its proof marker the 22nd) → this slice renumbered 0195→0196 and its proof block reconciled to the 23rd slot, both-blocks-kept (the SOUL-1/TEAMMOVE routine); no migration-body collision anywhere (the adapter's 0193 head re-grepped after every rebase). CI CATCH after the first push: the SHIELD-1 proof-merge resolution left the superseded pre-SHIELD1 final PASS-select's line-HEAD dangling on the DECKS3 banner line — its `select` ' opened a string that swallowed the block's comment lines until the next quote (a parse error the static selftest cannot see: grep pins don't parse quote balance); fixed by deleting the dangling fragment — the one TRUE final select, carrying both the SHIELD-1 and DECKS-3 clauses, was already correct.

Verification. Hostile review: APPROVED (extract-and-diff clean, arithmetic re-derived independently; the one MEDIUM — the inherited NaN-guard no-op — documented honestly + NANGUARD queued, never fixed mid-slice). `team-command-proof.sh selftest` green (23 markers post-rebase); mutation guttings (affinity exactness / knob-0 parity / mismatch-stays-1.0) each fail-then-restore; the migration's prosrc pin counts verified statically against the extracted body; `tsc/vite` n/a — server-only slice, zero client files touched (confirmed via git status).

2026-07-13 — SHIPYARD-2: hull-build DELIVERY (mig 0194, dark) — all pre-flip seams closed

Request. The SHIPYARD charter, slice 2: make the 0038 queue engine DELIVER hulls and close ALL THREE recorded pre-flip requirements from the 0188 seam list — the engine's hull arm (activation with recipe `build_seconds` + completion), delivery through the ONE commission build core, and hull-aware `cancel_build_order` refunds. Dark behind `shipyard_enabled` + double-inert. Hardened against the hostile review (B1 proof-constraint blocker, H1 cron-wedge, M1 promotion race, L1/N1 wording) and RENUMBERED 0192 → 0194 after SOUL-1 (mig 0193, PR #136) landed first — exactly the choreography 0193's collision note prescribed.

Work done

- **True heads (grep-verified before writing; re-verified after #134/#135/#136 merged):** `production_start_next` / `production_complete_order` / `process_build_queue` / `cancel_build_order` — 0038 (0036/0038 the only create sites; every later mention is a grant/comment; 0190–0193 touch none); `port_entry_commission_build` — **0193** (0080 → 0184 NAME hunk → 0193 SOUL-1 gated trait-roll hook; grep-verified only callers = `port_entry_commission_writer` 0080 + `commission_additional_main_ship` 0091, both single-arg); `wallet_credit` — 0093; `inventory_deposit` — 0039. `train_units` / `production_create_order` untouched (still head 0038).
- **mig** 20260618000194_shipyard2_delivery.sql — five marked-hunk parity re-creates, each extract-and-diff-verified against its head: (a) `production_start_next` + the HULL arm: the oldest waiting hull row (strict `hull_build_recipes` join — the unit arm's `unit_types` join byte-intact) competes with the oldest waiting unit by `queued_at` (strictly-older hull wins, ties to the unit arm); hull duration = `greatest(min_build_seconds, recipe_build_seconds)` — no `build_time_scale` [D: recipe seconds are owner-tuned directly; quantity always 1]; PLUS the per-player 'production_start' advisory xact lock at the top (review M1 — two concurrent `start_next` calls could both read `v_active=0` and promote TWO rows past `max=1` through diverging skip-locked picks; the 0078 lock idiom closes it; the ONE deliberate unit-path delta, strictly a race-closure); (b) `process_build_queue`: kind-dispatched completion loop — the UNIT arm keeps the exact 0038 statements ungarded, the HULL arm runs in its OWN begin/exception subtransaction (review H1, the EV-1 per-publication precedent, `query_canceled` re-raised: a permanently-failing delivery — reachable via a disabled commission-port docking service — must never wedge the whole cron; the failed order stays 'active', a NOTICE logs it, other players' completions proceed) — plus the hull-only PROMOTER SWEEP (hull orders enqueue with NO immediate start — the 0188 order RPC is untouched [D: one fewer live re-create; ≤30s cron latency; self-healing]; zero hull rows → the sweep selects nobody → byte-identical cron); (c) `production_complete_order`: the 0038 completion UPDATE + RETURNING, then hull delivery through the ONE commission core under the per-player `main_ship_commission` lock (the 0184 naming-race law); atomic per order, the raise confined by (b)'s guard; (d) `cancel_build_order`: unit arm byte-identical; hull refund arm MATCHING the unit semantics [D: waiting → 100%, active → 50% floored — credits via `wallet_credit` (NO idempotency key; the FOR UPDATE status flip is the sole and sufficient credit double-refund guard — review L1), ingredients via `inventory_deposit`, each DEPOSIT keyed `hull_cancel:<order>:<item>`, the bill read from the durable `hull_build_receipts.ingredients_json` (missing receipt = hard raise — impossible by 0188 atomicity; the receipt is NEVER rewritten, replay stays verbatim post-delivery AND post-cancel); (e) `port_entry_commission_build` re-signed (uuid) → (uuid, text default 'starter_frigate') from the **0193 head** (old signature dropped — single-arg calls would be ambiguous; both existing callers ride the default to the byte-inert 0193 starter behavior — 'Sparrow' literal AND the SOUL-1 gated trait-roll hook carried verbatim, so a delivered hull is born with its soul when lit, the P12 "every new ship" reading); a non-starter hull rides the same insert path with its catalog stats and the 0184 name idiom — class name from the hull row, the numeral counting ALL of the player's ships across classes (a third ship that is a first Mule = 'Mule-class Hauler III', the per-player-ordinal law, NOT a per-class counter — review N1); delivery lands docked at Haven Reach [D: the charter's "ordering port" is unrecorded on 0188 orders — 0184 behavior kept, documented]; delivery does NOT check `max_main_ships_per_player` [D: the cap guards commission RPCs; a built hull is bought and paid — recorded for the flip review]. Self-asserts: dark + zero hull orders (double-inert), unit-arm byte tokens, hunk tokens (M1 lock, H1 guard + `query_canceled` re-raise + token order), old-signature-gone + starter default + callers-still-single-arg + the SOUL-1 hook surviving with EXACTLY one gated roll call, fleets insert unmoved, determinism (no `random()`), ACLs, the 0188 receipt-only replay grounding, the shared `max_build_orders` predicate at both cap sites (cap coherence verified — promotion only moves rows between the two counted states).
- **Fleets-shim retirement EXPLICITLY DEFERRED [D]** (the charter had sketched it here): build is NOT md5-pinned (pins = writer / `commission_first` / `normalize` — grep-verified; 0184 precedent), all three pinned bodies byte-untouched, the fleets insert byte-identical; full retirement would force a hygiene-only re-create of the FROZEN pinned `normalize_main_ship_dock` — the exact live-path risk the exception note forbids. \$1 `fleets` row updated with the recorded deferral; retirement stays a named follow-up (NOT an ACT-SHIPYARD pre-flip requirement).
- **Proof** `scripts/shipyard-proof.{sql,sh}` extended (same standalone workflow): P6 honestly rewritten (the invisibility seam is CLOSED — prosrc both-arms pins; kind-coherence + shared-cap unchanged) + 4 new markers: `SHIPYARD_PASS_PROMOTE` (cron-sweep promotion, recipe `build_seconds` exact, serial one-slot law across kinds, order RPC still enqueues waiting), `SHIPYARD_PASS_DELIVER` (two-timestamp EXACT-DURATION fast-forward — review B1: the proof is one txn with `now()` frozen at T and `queued_at` = T, so a shift past the duration would land `complete_at` < `queued_at` and trip the 0038 `build_orders_complete_after_queue` CHECK; the exact-duration shift lands `complete_at` = T, due AND constraint-legal → exact stats/name delivery — 'SY1 Test Dreadnought II' + 'Mule-class Hauler III' — docked present/location fleet at the commission port, canonical at `location`; unit order promotes/completes IDENTICALLY: the exact 0038 formula + `base_merge_units` +2, no ship; post-delivery replay verbatim), `SHIPYARD_PASS_DELIVERY_GUARD` (review H1's pin: poison the commission port's docking service in-txn → a second player's co-tick due unit completion PROCEEDS, the poisoned hull order stays active and uncounted, no ship minted; restore the fixture → the next tick self-heals and delivers exactly one ship), `SHIPYARD_PASS_CANCEL_REFUND` (exact full refund from the receipt bill 16/4/6/8/2 + the 400 credits; double-cancel rejected, no double refund; post-cancel replay verbatim; active-cancel floor-half 8/2/3/4/1 + 200 exact). 12 markers total; provisioning stays real-leaf-only (`bootstrap_me` + `production_create_order` added for the unit-parity/co-tick fixtures, selftest-pinned).
- **Verification:** selftest green (12 markers); 4 mutation guttings fail-then-restore (deliver pin / refund exactness / unit-parity pin / poison-guard pin — each selftest-FAILED when gutted, restored sha-verified byte-identical); every re-created body extract-and-diffed against its head — marked hunks only (build diffed against the 0193 head: signature + name-source +

hull-select hunks, the SOUL-1 hook byte-identical). HONESTY NOTE: no local Docker/Supabase stack exists on this machine, so the disposable CI job (shipyard-proof.yml, full-chain apply + the 12-marker real run) is the FIRST real execution of the extended harness — every P8/P9/P9G/P10 fixture timestamp was desk-checked against the 0038 (complete_after_queue, status domain, kind coherence) and 0188 (receipts) CHECKs after the B1 fix. No client code touched (SHIPYARD-3 owns UI).

- Docs synced: FULL_CAPACITY_PLAN §C P6 (seam list → RESOLVED; SHIPYARD-2 shipped entry + the deferral; ACT-SHIPYARD still needs flag flip + blueprint knob + mining/trade preconditions), SYSTEM_BOUNDARIES (fleets / build_orders / Production / Main Ship rows), this DEV_LOG.

Bugs / fixes

- `_(pre-flip fixes by design)_` the 0038 engine could never start a hull timer (unit_types-join pick) and would have eaten a cancelled hull order's credits/items (metal-only refund) — both closed here before ACT-SHIPYARD, exactly as the 0188 seam list recorded.

Open follow-ups

- ACT-SHIPYARD flip script (SHIPYARD-3 shipped client-only via PR #137 while this slice was in review; + the recorded [D]s to review: Haven-only delivery, no ship-cap check at delivery, active-cancel floor-half).
- The deferred fleets-shim retirement (Fleet-exposed commission/dock repoint + pin re-derivation).
- MERGE-RACE: **RESOLVED** — SOUL-1 (0193) landed first and this slice followed its recorded choreography exactly (renumber to 0194 + re-create build from the 0193 head so the roll hook and the delivery hunks coexist; the 0194 self-assert pins the hook's survival). Any FUTURE build re-create must now start from the 0194 head — note it changed the SIGNATURE to (uuid, text default 'starter_frigate') (a stale (uuid)-shaped re-create would reintroduce ambiguity and fail the 0194-era asserts loudly).

2026-07-13 — SHIELD-1: the shield enters the live combat engine (mig 0195, inert while all pools are 0/NULL)

Request. The SHIELD charter, slice 1: the tick/creator parity re-creates — member encounters snapshot the pool, in-combat regen, shield-absorbs-first damage, the 0191 leaf wired. The highest-parity-discipline slice of the train: two LIVE combat functions re-created.

Work done

- **True heads re-verified by grep over ALL migrations:** `combat_create_group_encounter` ← 0168:359 (its ONLY create site); `process_combat_ticks` ← 0169:93 (0017→0022→0023→0030→0032→ 0041→0046→0167→0169, nothing later). The solo `combat_create_encounter` (0168:481) verified member-row-free — its legacy path inserts catalog `fleet_units` rows only, and the 0168:446 group-creator insert is the ONLY `combat_units` insert carrying `main_ship_id` — so it is deliberately NOT re-created (documented in the 0195 header).
- **mig 20260618000195_shield1_tick.sql** (0193 SOUL-1 merged; 0194 promised to SHIPYARD-2's renumber — this slice took 0195) — both bodies copied verbatim from the heads, every delta a `-- SHIELD-1 (0195):` marked hunk, extract-and-diff verified:

`<font name="Sym"> </font>` **Creator:** `msi.shield/msi.max_shield` join `msi.hp` in the SAME manifest read (the 0168 one-read law); a live member with `max_shield > 0` snapshots `shield_max` frozen + `shield_current` = the CURRENT pool; a shieldless ship (every ship until ACT-SHIELD) and a degraded member snapshot **NULL/NULL, not 0/0** — the write-count-parity decision (NULL keeps regen/absorb/leaf provably skipped; the 0191 pairing CHECK's optional IFF tightening declined: a shieldless member's honest snapshot is "none").

`<font name="Sym"> </font>` **Tick:** (a) in-combat regen at the top of the per-row loop — `least(shield_max, shield_current + shield_max × knob)`, NULL-propagating (the D1 0167:12-14 idiom; the pairing CHECK guarantees the pair is never mixed); (b) THE ONE ABSORB POINT — `v_absorb := least(coalesce(pool, 0), damage)`, hull takes ONLY the overflow, no second damage path; (c) `mainship_sync_combat_shield(ship, round(pool))` beside its hp sibling, gated on a non-NULL pool (the leaf's FIRST caller; both leaves SECURITY DEFINER + service-role-only — the 0167:559-564 precedent); (d) UNTOUCHED: hull-only integrity accounting (Σ hp_max), hull-only defeat (a shielded ship at hull 0 is dead), report/tick jsonb keys; (e) the knob HOISTED to one `cfg_num` read per invocation, in the existing pre-loop knob block.

`<font name="Sym"> </font>` **Self-asserts:** marked-hunk tokens present in prosrc; old-head fingerprints absent (the pre-shield select/roster/insert/damage/update shapes); hull-only integrity + defeat expressions token-pinned; legacy jsonb key idiom pinned; RNG unchanged (creator 0, tick exactly the head's 1 variance call); knob read exactly once and before the loop; client ACL closed; knob '0', all combat rows shield-NULL, all instances 0/0, missing-row leaf inert.

`<font name="Sym"> </font>` **Scope note:** the commission `base_shield` → `max_shield` copy DEFERRED to SHIELD-2 (a provable no-op while every hull is 0; keeps this slice's re-create surface minimal).
- **Proof — new TEAMCMD_PASS_SHIELD1 block in team-command-proof.{sql,sh}** (22nd marker, reconciled after SOUL-1's 21st at the rebase — **collision resolved:** both blocks kept and the MARKERS list merged to 22, the established idiom): ZERO state

(member rows exist from the earlier blocks, none carries a snapshot, no fought ship's instance shield ever moved — and the earlier COMBATPARITY/TEAMHUNT/TEAMSETTLE exact-damage pins ran against THIS tick with all pools 0, which IS the zero-parity proof); LIT arm in-txn (max_shield 40 / shield 3 by sanctioned direct update — shields have no writer besides the tick-called leaf — then a REAL 1-ship team hunt): snapshot 40/3 carries the CURRENT pool, knob-'0' tick drains min(pool, damage) exactly with the hull taking only the overflow (vs the block's OWN independent damage derivation, variance 0), knob-'1' regen climbs 0→40 exactly then CAPS at max the next tick (non-degenerate under the $3 < \text{damage} < 40$ guard), the leaf mirrors round(pool) each tick with hp consistent, combat_ticks integrity stays hull-only at a nonzero pool, and the one-step-wipe defeat kills the fully-shielded ship at hull 0 (D1 terminal). Knob + enemy_attack_base restored in-txn. .sh: 16 new assert-form pins; the combat knob moved from the never-touch posture to the raised-and-restored-in-txn knob idiom (SHIELD-1 gave it its consumer) while the idle knob keeps never-touch; committed-'0' honesty checks retained for both.

- **Docs:** FULL_CAPACITY_PLAN §C P13 (SHIELD-1 shipped; SHIELD-2 next = regen home + UI + commission copy; ACT-SHIELD after) + queue row 18; SYSTEM_BOUNDARIES (combat tick = the in-combat shield writer; the leaf's caller list = process_combat_ticks exactly).

Verification. team-command-proof.sh selftest green (22 markers post-rebase); 3 mutation guttings (absorb-arithmetic pin, zero-parity pin, defeat-hull-only pin) each fail-then-restore; migration bodies diffed against the extracted heads — only the marked hunks differ; hostile-review blocker fixed (undeclared v_hull0 in the proof lit arm — CI's disposable chain is the lit arm's first runtime execution); tsc/vite n/a (no client change — server-only migration + proof + docs, confirmed via git status).

2026-07-13 — SOUL-1: ship traits go functional — commission roll hook + adapter trait fold (mig 0193, dark)

Request. The SHIP-SOUL charter (P12), slice 1: every new ship is born with its soul WHEN LIT (the commission path calls soul_roll_traits_for_ship), and calculate_expedition_stats folds each rolled trait's stats_json at the 0170 seam — all still dark behind ship_traits_enabled.

Work done

- **True heads (grep-verified, re-verified after the 0190/0191 rebase):** port_entry_commission_build → 0184 (0080 → 0184; 0185/0188 only mention it, 0190/0191 don't touch it); ensure_main_ship_for_player → 0078 (0043 → 0077 → 0078); calculate_expedition_stats → 0180 (0044 → 0115 → 0122 → 0170 → 0180; 0181..0191 never re-create it). Both commission WRITERS delegate the insert to build (0080 §B / 0091), so ONE hook in build covers first + additional ships.
- **mig 20260618000193_soul1_fold.sql** — (a) build re-created (0184 head) with ONE marked hook after the ship insert: if cfg_bool('ship_traits_enabled') then perform soul_roll_traits_for_ship(v_ship) — double-gated (call site + the roll fn's own gate-first), dark = ZERO calls, byte-parity; definer-to-definer works with no ACL widening (the 0169 team_hunt_send → 0152 leaf pattern). (b) ensure re-created (0078 head) with the SAME hook inside its CREATE BRANCH only (returning ... into v_new captured; P12 reads "every new ship" — a starter Sparrow deserves a soul; a replay must never roll an existing ship — that retroactive roll is ACT-SOUL's, behind the catalog-freeze count=8 precondition). (c) adapter re-created (0180 head) with the ONE knob-gated trait fold: v_traits_enabled read once at entry (the 0180 v_growth posture), the loop over main_ship_traits × ship_trait_types feeds the SAME accumulators in the module loop's 8-key vocabulary, placed adjacent to the 0170 hull fold (traits are the ship itself; additive accumulators — order can't change the sum); speed_mult_bonus joins the ONE final-speed multiplier (the only modified pre-existing line, `<font name="Sym"> 0` exactly while dark); NO tradeoff CASE (trait costs live in their minus keys); hp_mult NEVER read — applied once at roll time (0186), prosrc-pinned. Self-asserts: deploy-time double inertness (gate dark + zero rolled rows + catalog frozen at 8), exactly one gated hook per creator (ensure's inside the create branch), ONE trait read site / 8 key reads / no hp_mult read / speed inside the multiplier / traits unscaled, every 0180 captain-level pin re-verified on the re-create, ACLs server-only across the board.
- **Proof** — TEAMCMD_PASS_SOUL1 appended to team-command-proof (the 21st marker, after main's 20 — ...SHIELD0, TEAMMOVE): dark commission = zero trait rows + adapter byte-identical before/after a direct roll with the gate re-darkened (the knob-gated read); lit fold = dark baseline + the STORED traits' stats_json sums exactly per key (independent catalog join, 0=0 false-green guarded), speed = round(greatest(0.2, base_speed × (1 + Σ speed_mult_bonus)), 3), no non-stat key moves; a LIT commission births exactly 2 derivation-matching traits (pg_temp.soul_expect); the veteran-arm draw pins hp_mult ONCE at commission (max_hp = round(base × mult)) with the adapter never re-scaling; ensure creates WITH soul lit / without dark, a lit ensure REPLAY never rolls an existing unrolled ship, and lit + zero rows is byte-identical to dark (the empty-loop arm). **RECONCILE:** the SOUL0 block now commissions its roll fixtures DARK before its in-txn flip (under the hook a lit commission pre-rolls, which would break its inserted = 2 asserts) and the gate is re-darkened before TEAMMAP — every pre-SOUL-1 block (incl. SHIELD0's leaf-smoke commission and TEAMMOVE's docked hop) keeps byte-identical behavior (COMBATPARITY/TEAMSTATS pins run dark, where the fold is skipped entirely).
- **Docs:** FULL_CAPACITY_PLAN P12 + queue row 17 (SOUL-1 shipped; SOUL-2 + ACT-SOUL next); SYSTEM_BOUNDARIES §1 rows (ship_trait_types consumer pin; main_ship_traits callers wired) `<font name="Sym">` the §8 adapter edge (DOWNWARD read-only into Ship Soul, knob-gated).

- **Collision notes:** 0190 (TEAMMOVE) + 0191 (SHIELD-0) landed on main mid-slice → rebased, proof pair reconciled both-blocks-kept (SOUL1 21st), neither re-creates a SOUL-1 head; 0192 is claimed by in-flight SHIPYARD-2 → 0193 taken. **SHIPYARD-2 choreography (both directions, explicit — SOUL-1 is landing FIRST):** SHIPYARD-2 (in flight, currently numbered 0192, with its own `port_entry_commission_build` re-create from the 0184 head) MUST at its rebase (a) RENUMBER to ≥ 0195 — a 0192 applying after this 0193 on prod would silently erase the SOUL-1 hook (last create-or-replace wins), and on fresh chains 0193 applying after a 0192 would erase its delivery hunks — and (b) re-create build from THIS migration's head (0193 = the 0184 body + the SOUL-1 hook) so both hunks coexist. (Had SHIPYARD-2 landed first, this slice would have re-applied its minimal hook hunk on SHIPYARD-2's head instead — recorded in the 0193 header.)

Verification. `team-command-proof.sh selftest` green (21 markers); 3 mutation guttings (fold exactness / dark parity / commission-hook pin) fail-then-restore; `tsc -b + vite build` green (no client changes — server-only slice).

2026-07-13 — TEAMMOVE-1: a docked team moves onward as one (mig 0190, LIVE gate)

Request. The owner directive's second half ("the team should also be able to be docked or move as a whole"): the dock half worked (team send → members arrive docked, 0153; visible since TEAMMAP, 0187), but BOTH team sends require every member `status='home'` while a docked member is `'stationary'` — a docked team could arrive together but never leave together (gap G-B).

Work done

- **mig** `20260618000190_teammove_group_move.sql` — NEW `move_ship_group_to_location(p_group_id, p_location_id)`: the 0163-shaped wrapper composing the UNCHANGED live per-ship `move_main_ship_to_location` (TRUE head 0156, grep-verified 0053 → 0152 → 0156; its 'present' departure branch IS a docked member's state) once per member. Gate-first on the LIVE `team_command_enabled` (no dark posture — stated in the header), group FOR SHARE → ships FOR UPDATE (the 0163 lock order), docked-together preconditions UNDER the locks (every member in the 0153 stationary/at_location pair + exactly one owned 'present' fleet each, all at ONE location → else `member_not_ready`, the 0168 phrasing), the destination deliberately never read (the per-ship RPC's own rejects surface through the ONE all-or-nothing subtransaction → `member_send_failed`, the 0163 posture), and the 0187-idiom informational `fleets.group_id` tag off the collected envelopes on success. Lock-order inversion vs the standalone per-ship move (ship→fleet here, fleet→ship there) documented and accepted in the header: deadlock-detected (40P01), both sides fail closed, availability noise on a self-race — never corruption.
- **UI** — `teamApi.moveShipGroup` wrapper; NEW pure `teamMove.ts` (`groupMoveAvailability` server-mirror + `teamMapSendAction`, the ONE expedition-arm classifier) + `tests/teamMove.spec.ts` (incl. the property law: a team with ANY docked member never classifies as an enabled Send); `TeamMapSend` folds the present-fleet read through the REUSED `deriveDockedTeamRollups` and each team row renders by the classifier — "Move team here" (`team-move-<groupId> testid`) / muted "Docked here" badge / disabled Send + gather hint / the original Send. `member_not_ready` / `member_send_failed` copy generalized to cover both hunt and move.
- **Proof** — new `TEAMCMD_PASS_TEAMMOVE` block in `team-command-proof.{sql,sh}` (20th marker after the SHIELD0 reconcile), reusing TEAMMAP's docked end-state: empty/foreign rejects; mid-flight member and REAL split-port team (t2 genuinely settled at Driftmarch) both `member_not_ready` with zero departures; all-or-nothing pinned on real tables (t2's presence fixture-completed → t1 departs then rolls back, both fleets still docked); happy path Slagworks → Driftmarch (2 present-departure envelopes over the members' OWN fleets, moving + tagged, no strays) and the onward dock with the tag surviving. 12 assert-form selftest pins; Driftmarch added to the fixture ports.

Verification. selftest green (20 markers), `tsc -b, vite build`, 498 pure specs by explicit list, eslint clean on touched files.

2026-07-13 — SHIELD-0: the shield schema foundation (mig 0191, deploy-inert, data-gated)

Request. The SHIELD charter, slice 0 (owner directive: ships get a SHIELD that regenerates during and outside combat): schema + the one sync leaf + the two regen knobs ONLY — NO engine change (SHIELD-1 owns the tick/creator parity re-creates; SHIELD-2 the regen home + UI). Everything provably inert while every `base_shield = 0`, every instance 0/0, both knobs '0'.

Work done

- **The commission decision (the reason this slice re-creates ZERO functions):** both live insert heads — `port_entry_commission_build` (TRUE head 0184, grep-verified: creates at 0080→0184 only) and `ensure_main_ship_for_player` (TRUE head 0078: 0043→0077→0078) — ENUMERATE their insert columns, so default 0 not null shield columns ride the defaults and every newly commissioned ship is born 0/0 with no marked-hunk re-create. The `base_shield` → `max_shield`

commission copy lands with SHIELD-1/2's engine re-creates; ACT-SHIELD's flip script will do the monotonic backfill for pre-flip ships.

- **mig 20260618000191_shield0_foundation.sql** (0188–0190 claimed by in-flight slices — numbering coordinated):
 - `main_ship_hull_types.base_shield` integer not null default 0 (>= 0 — 0-legal, unlike `base_hp` > 0: shieldless is a permanent legal state) + `main_ship_instances.shield/ max_shield` (integer default 0; `shield` >= 0, `max_shield` >= 0, `shield` <= `max_shield` — the 0043:53-54 hp shape with the max relaxed to 0-legal). All CHECKs named for pinnability.
 - `combat_units.shield_max/shield_current` double precision NULL (the 0167:69-70 snapshot shape) + `combat_units_member_shield_pairing` — the 0167:85-88 pairing law ADAPTED with one deliberate, documented weakening: paired-together + member-row-only instead of the strict IFF, because the member-row writer (D2's hunt send) went LIVE at the 2026-07-12 activation and does not write shields until SHIELD-1 — a strict IFF would fail every live team-hunt insert on deploy. Catalog rows can never carry a stray shield (the half the engine's future coalesce reads depend on); SHIELD-1's writer re-create may tighten to the full IFF.
 - `mainship_sync_combat_shield(p_main_ship_id, p_shield)` — the `mainship_sync_combat_hp` (0167:129-145) sibling posture exactly (SECURITY DEFINER, set `search_path=public`, service-role-only per 0167:559-564, missing row = zero rows, one-leaf-one-concern) plus BOTH clamps — `least(max_shield, greatest(0, ...))` — because regen pushes UP where combat only ever lowered hp. Body kept free of the hp token so writes-shield-only is prosrc-pinnable. NO caller this slice (SHIELD-1 wires the tick).
 - Knobs `shield_regen_combat_pct/shield_regen_idle_pct` seeded '0' (the 0170/0180/0185 double-inert idiom); NO `shield_enabled` flag — the system is DATA-gated like the 0170 hull stats. Partial index `main_ship_instances (main_ship_id) where shield < max_shield` = the future regen pass's scan surface; matches zero rows while everything is 0/0.
 - **Self-asserts:** column shapes via `information_schema` (3 × integer/not-null/default-0; 2 × nullable double precision no-default); the 5 named CHECKs; leaf ACL (`has_function_privilege: service_role` yes, `authenticated/anon` no) + prosrc pins (both clamps, set `shield`, no hp, no `random()/setseed`); both knobs '0'; every hull 0, every instance 0/0, every combat row NULL/NULL; the missing-row leaf call moves nothing; the regen index exists and its predicate matches zero rows.
- **Proof — new TEAMCMD_PASS_SHIELD0 block in** `scripts/team-command-proof.{sql,sh}` (19th marker as this branch stands — see the reconcile note): committed knobs '0' asserted and NEVER touched, even in-txn (no consumer exists — the SHIPYARD0 never-flip posture applied to knobs, negative-grepped in the selftest for BOTH the raw-update and `set_game_config` idioms); the schema/CHECK/index pins; total inertness over every earlier block's fixtures; the leaf smoke on a fresh commissioned fixture (`max_shield` fixture-set 50 in-txn): floor clamp → 0, ceiling clamp → 50, in-range → 30 exact, missing ship = zero rows, `shield_le_max` trips on a direct over-max write, and `hp/max_hp` BYTE-EXACT before/after every call. `.sh` extends the local-mode committed-value honesty checks to pin both knobs still '0' after the run.
- **Docs:** `FULL_CAPACITY_PLAN` (new §C P13 — the SHIELD charter; queue row 18; [D] owner-tunable flip numbers: Sparrow 100 / Mule 130 / Talon 85 `base_shield`, 0.02 combat / 0.10 idle regen), `SYSTEM_BOUNDARIES` (§1 `main_ship_instances` disjoint-writers-by-column note + `combat_units` shield columns; §2 Main Ship gains the leaf).
- **Verification:** `team-command-proof.sh selftest` green with the new marker; 3 mutation guttings (marker / hp-untouched pin / committed-knob pin) each fail-then-restore; `tsc -b + vite build` green (no client change — additive server columns only).
- **Numbering/reconcile note:** written against origin/main's 18 proof markers; DECKS (0189, own proof) and TEAMMOVE (0190, appends the 19th team-proof marker) are in flight — if TEAMMOVE merges first this block becomes the 20th and the merge keeps both blocks (the established both-blocks-kept routine).

2026-07-13 — SHIPYARD-1: the hull-build ORDER command (mig 0188, dark)

Request. The SHIPYARD charter, slice 1: `start_hull_build` on the REUSED M4.5 `build_orders` serial queue — validate recipe + progression gates + balances, spend ingredients (`inventory_spend`)

- credits (`wallet_debit`) under lock, enqueue the build, receipt it. ORDER side ONLY — SHIPYARD-2 owns completion → delivery (the commission-core generalization). Dark behind `shipyard_enabled`.

Work done

- **True-head grounding (grep-verified before writing a line):** every `build_orders` queue function (`train_units/production_create_order/production_complete_order/production_start_next/process_build_queue/cancel_build_order`) is created ONLY at 0036/0038 → the 0038 bodies are the live heads; `inventory_spend` head = 0039 (sole creator); `wallet_debit/wallet_credit` head = 0093 (re-created over 0089/0090); the commission core `port_entry_commission_build` head = 0184 (over 0080) — hardcodes 'starter_frigate', SHIPYARD-2 territory, untouched; `captain_instances.level` = the 0177 additive column. **ZERO live functions touched this slice** — the clean shape the charter seam allows.

- **THE SEAM (load-bearing):** a hull order lands 'waiting' with unit_type_id NULL and is INVISIBLE to the 0038 engine by construction — production_start_next picks through join unit_types ut on ut.id = bo.unit_type_id (a NULL-unit row never joins → never promoted, never gets complete_at; unit training unaffected) and process_build_queue loops status='active' only (a never-promoted row can never hit its unit-shaped base_merge_units delivery). Both prosrc facts PINNED by the 0188 self-assert at apply time. Honest-dark: no hull order can exist in prod while the flag is false, and ACT-SHIPYARD requires SHIPYARD-2 (which ships the engine's hull arm: activation with recipe build_seconds, completion → commission delivery, AND hull-aware cancel_build_order refunds — today's cancel would eat a hull order's credits/items at metal_spent=0; recorded in the plan + SYSTEM_BOUNDARIES).
- **mig 20260618000188_shipyard1_order_rpc.sql** (0186/0187 are claimed by in-flight slices) — (a) ADDITIVE build_orders generalization: unit_type_id + base_id drop NOT NULL, new hull_type_id FK → hull_build_recipes (the STRICT FK: only recipe-carrying hulls orderable — T0 starter_frigate is deliberately recipe-less and can never be enqueued), new credits_spent (≥ 0, default 0), build_orders_kind_coherent CHECK (unit+base+no-hull XOR hull+no-unit+no-base; validates every legacy row at apply); (b) hull_build_receipts — the 0109/0126 per-player idempotency ledger point-for-point (unique (player_id, request_id) uuid, owner-read RLS, no client write path, replay-verbatim); (c) private production_start_hull_build (SECURITY DEFINER, service_role-only): gate-FIRST reject-before-any-read → request → per-player advisory lock (hull_build domain, before the replay check) → replay → catalog (unknown_hull vs no_recipe — the 0109 distinct-truthful posture) → the 0185 progression gates (required hull owned non-destroyed / any owned captain at required_captain_level — both dormant-NULL on T1) → the SHARED max_build_orders cap (waiting+active, the train_units predicate verbatim — one queue, one cap) → ingredient pre-check (friendly envelope before ANY write; integral-qty hard-abort) → wallet_debit (first write; false → insufficient_credits, nothing else spent — the 0089 posture) → inventory_spend per ingredient (exceptions roll back debit+spends+everything — all-or-nothing, failure writes NO receipt) → the 'waiting' hull-row insert (quantity 1, NO timestamps — the M4.5 serial law) → receipt. Deterministic (no random(); order by item_id — the 0041 law); (d) public start_hull_build(request_id, hull_type) wrapper (0109 idiom: anti-probe gate — dark answer identical for real vs garbage hulls — + reason→code/message map with per-reason context pass-through); (e) two-layer ACL (wrapper authenticated-only; writer off-client); (f) self-asserts: flag dark, schema shape exact, engine-invisibility prosrc pins, gate-first token-order pin over every read surface, determinism pin, ACL pins, receipts RLS posture, integral recipe quantities, the 0185 T1 catalog re-pinned intact.
- **Proof** scripts/shipyard-proof.{sql,sh} + .github/workflows/shipyard-proof.yml (standalone — team-command-proof is contended by two in-flight slices; modeled on trade-v1-proof.yml, REUSING scripts/lib/trade-proof-lib.sh): ONE rolled-back txn, flag raised in-txn only via the raw-update idiom, provisioning via the REAL leaves (reward_grant items, captains_mint_instance the gate captain, wallet funded by the sanctioned tm1 insert). P0 dark gate (wrapper envelopes BYTE-IDENTICAL for real vs garbage hull — no existence oracle; private writer dark on its own authority; zero writes) · P1 exact-spend order (hauler: wallet 500→100 exact, all 5 ingredients →0 exact, ONE waiting hull-shaped row with no timestamps, ONE receipt with the 5-element bill) · P2 replay (same request_id → same order/receipt verbatim, flagged, no double spend) · P3 shortfalls (blueprint_fragment 0/2 → insufficient_items exact; credits 100<400 → insufficient_credits with ingredients+wallet untouched — the all-or-nothing ordering) · P4 gates on an in-txn synthetic T2 fixture (unknown_hull/no_recipe truthful; hull-prereq reject; captain-level reject on BOTH arms — no captain, then a REAL level-1 captain vs required 2; boundary pass queues the 2nd order) · P5 self-prereq impossibility (hull_recipe_no_self_prereq check_violation) · P6 the engine seam for real (production_start_next + process_build_queue invoked: hull rows stay waiting, no timestamps, NO ship delivered; bare/hybrid rows check-rejected; shared cap queue_full at knob 2, zero writes). Selftest (hardened, the 0185-era pattern): self-rolling-back + flag-in-txn + set_game_config('shipyard_enabled'...) evasion catch + sole-writer bans (no direct player_inventory/captain_instances/hull_build_receipts inserts, no direct wallet/inventory updates) + exact-economics pins + full envelope/oracle/replay/seam token coverage.
- **Verification:** selftest GREEN; 4 mutation guttings each caught then restored byte-identical (PASS-marker gut → caught; flag-flip idiom swapped to set_game_config → caught; replay pin gut → caught; trailing COMMIT → caught). No Docker on this machine — the real-chain matrix runs in the new shipyard-proof.yml on push (selftest job + disposable local Supabase job). No client files touched (server-only slice — tsc/vite n/a).
- **Docs synced (the \$E same-PR law):** SYSTEM_BOUNDARIES — build_orders row rewritten (two row kinds, the writer, the seam + cancel note), NEW hull_build_receipts row, the two 0185 recipe rows' "NOTHING reads it today" replaced (0188 is their first reader), Production \$2 contract gains the two-layer command + the wallet/engine forbidden columns; FULL_CAPACITY_PLAN — P6 SHIPYARD-1 → shipped with the SHIPYARD-2 seam items, \$D trailing note updated.

Review fixes (hostile review 2026-07-13 — approved, one HIGH fixed in-slice while 0188 unmerged)

- **H1 — receipts vs the 0047 reaper (REAL BUG, fixed):** maintenance_cleanup_runtime_data (0047 \$10, live cron) deletes terminal build_orders >30d; the original order_id ... on delete cascade would have cascade-deleted the receipt — silently expiring the replay guarantee (a day-32 stale retry = a SECOND full-price order) and destroying the audit bill; the justifying "no delete path exists" comment was false (the 0109/0126 cascade precedent FKs durable property tables, not reaped runtime rows). Fixed: order_id now nullable with FK **ON DELETE SET NULL** — receipts are the durable idempotency+audit ledger and outlive the reap; replay reads ONLY the receipt row (verified: the writer's step 4 never selects through the FK); comment rewritten to the true lifetime story; new self-assert pins the FK confdeltype ('n') + the player_id cascade ('c'); NEW proof block **P7 SHIPYARD_PASS_RETENTION** (8th marker) — flips the P1 order terminal, backdates it 31d, runs the REAL reaper wet

(maintenance_cleanup_runtime_data(false, 5000)), asserts the order purged + the receipt survived (order_id NULL, bill intact) + the same-request_id stale retry replays the original envelope verbatim (order_id null) with no re-debit / no second order / no second receipt; selftest pins added (reaper-wet-run + survival + no-second-order tokens). Docs: the lifetime line added to the SYSTEM_BOUNDARIES hull_build_receipts row; the plan's P6 seam list marks this risk RESOLVED IN-SLICE (unlike cancel-refunds/delivery, which stay pre-flip requirements).

- **N1:** the gate-first self-assert's anchor now sits on the actual `cfg_bool('shipyard_enabled')` call token, not the first comment mention. **N2:** the no-random() determinism assert now also covers the wrapper's prosrc. (N3–N5 skipped per review.)
- Re-verified: selftest GREEN with 8 markers; retention-pin mutation gutting FAILS then restored byte-identical.

2026-07-12 — SOUL-0: the per-ship traits foundation (mig 0186, doubly dark)

Request. The SHIP-SOUL packet, slice 0 (owner directive: "each ship has its own ORIGINAL stats, skills — like Uncharted Waters"): the trait catalog + the per-ship instance table + the deterministic roll writer ONLY — no commission hook, no adapter change, no read RPC, no frontend (SOUL-1 owns the parity re-creates of the commission path + the adapter fold). All dark behind NEW `ship_traits_enabled=false`.

Work done

- **Stat-key verification (the 0183 lesson):** the adapter's TRUE head re-confirmed as 0180 (grep: creates at 0044→0115→0122→0170→0180; 0181..0184 never re-create it); its module loop reads stats_json keys attack/defense/repair/cargo/scan/mining/evasion/speed_mult_bonus (0180:212–219). Every trait seeds ONLY these input keys. HONESTY DEVIATION from the 0183 prosrc pin, deliberate: the adapter reads that vocabulary from MODULES/captains/hulls — the TRAITS fold lands in SOUL-1, so there is no trait read edge to prosrc-pin yet; the 0186 self-assert instead pins every seeded key against the HARDCODED vocabulary list (kept in lockstep with 0180, commented as such) — the 0111/0117 nothing-reads-this-yet posture.
- **mig 20260618000186_soul0_traits_foundation.sql** (0185 is claimed by the in-flight slice-shipyard0 — numbering coordinated):


```
<font name="Sym"> </font> ship_trait_types — the 0117 captain_types catalog posture verbatim (RLS public-read, no write policy/grant, migration-seeded only): trait_type_id/name/description/stats_json/ hp_mult (numeric, check (hp_mult >= 1.0) — a trait never lowers hp). 8 seeds [D], every magnitude below the same-stat module band (0111/0183) and at-or-below the captain band (0117), five of eight carrying a minus key (the law-4 tradeoff posture): veteran_frame {defense 5, hp_mult 1.08 — the SOLE hp carrier} · tuned_thrusters {speed +0.08, cargo -3} · reinforced_plating {defense 8, speed -0.04} · smugglers_holds {cargo 8, scan -2} · keen_arrays {scan 5} · hungry_guns {attack 6, defense -3} · steady_rigger {mining 4, repair 2} · ill_omened {evasion 6, attack -2}. Flavor copy in the 0042/0107/0117 register.
```

```
<font name="Sym"> </font> main_ship_traits — 2 slots per ship (slot in (1,2), unique (main_ship_id, slot)), FK cascade to the ship, owner-read RLS via the 0074 EXISTS-on-the-ship idiom (authenticated only, never anon), default table grants stripped (0176/0177 revoke posture), ZERO client write privilege. INSERT-ONLY IMMUTABLE: no update/delete path exists anywhere; the extra unique (main_ship_id, trait_type_id) makes slot-distinctness a schema fact (fail-closed).
```

```
<font name="Sym"> </font> soul_roll_traits_for_ship(p_main_ship_id) — SECURITY DEFINER, service_role only (the 0145/0176 private-writer posture), gate-FIRST on the flag (envelope reject, never a raise). Hash technique: traits are a PURE FUNCTION of the ship id — the 0041 determinism law via the 0176 generator's shipped idiom (the packet's notional md5 normalized to the repo's hashtextextended, which 0176 already proved GUC-stable): slot 1 = hashtextextended('<ship_id>soul:1', 0) mapped ((h % n + n) % n) into the catalog's order by trait_type_id total order; slot 2 re-salts deterministically (:soul:2, then :soul:2:<k>) until distinct — a bounded loop (64) that raises rather than silently falling back. NO random()/setseed anywhere (prosrc-pinned). Idempotent: both inserts on conflict (main_ship_id, slot) do nothing — a re-call computes the same traits and lands zero rows, so a RE-ROLL CANNOT EXIST. hp_mult applies ONLY when both rows landed this call (a replay can never re-raise hp): max_hp := round(base * mult), hp scaled proportionally, both >-guarded monotonic (the 0171 never-lower posture); ship row locked FOR UPDATE first (concurrent rolls serialize). DOUBLY DARK: flag false + NOTHING calls it.
```

```
<font name="Sym"> </font> Self-asserts: 8 catalog rows pinned verbatim (id + stats_json + hp_mult) with no strays; veteran_frame the only hp_mult carrier; every seeded stats key ∈ the hardcoded 0180 vocabulary; main_ship_traits empty; the flag dark; the roll fn prosrc carries the :soul: salt + hashtextextended + the idempotent insert and carries NO session-RNG token and NO update/delete against the traits table; ACLs (writer service-only via has_function_privilege; both tables zero client write privilege via has_table_privilege; catalog public-read, traits authenticated-only).
```
- **Proof — new TEAMCMD_PASS_SOUL0 block in scripts/team-command-proof.{sql,sh}** (17th block, appended after SHIPYARD0): committed flag asserted dark + the roll writer's gate-first reject probed with a RANDOM uuid (a reject-after-read regression would answer ship_not_found — no existence oracle) and zero rows written; flag flipped in-txn only; catalog pinned verbatim (the VALUES-join form). **Determinism by inline re-derivation** (the D0/D1 independent-computation idiom): `pg_temp.soul_expect` re-implements the salt/hash/order mapping and is computed BEFORE each roll; fresh fixture users are commissioned (the real RPC) until one derived pair CONTAINS veteran_frame and one does NOT (E[draws]≈3, bound 40) —

so the hp_mult arm and the plain arm are both exercised every run despite fresh uuids. Each roll must land EXACTLY the derived pair (else "the roll is not the pinned pure function"), slots distinct; veteran arm `max_hp = round(base × 1.08)` exactly with hp scaled (full-hp stays full); plain arm hp/max_hp byte-untouched; second roll = idempotent replay (inserted 0, same 2 rows/traits, max_hp NOT re-raised — the double-apply hazard pin). Selftest greps added in assert form + the Ship-Soul sole-writer negative grep (no direct insert/update/delete/copy on either table anywhere in the harness); `ship_traits_enabled` added to the flags-inside-txn assert and the post-run committed-false honesty loop. **Mutation-tested: 7 deliberate guttings** (marker, flag flip removed, hp_mult pin, the derivation salt, a smuggled direct traits insert, the replay pin, the catalog pin) — each fails the selftest; restored green.

- **Verify:** team selftest green (17 markers); all other proof selftests green (`osn-hub1a-production-catalog-verify.sh` selftest fails pre-existing on origin/main — "single object rejected" — untouched by this slice); `tsc -b && vite build` green (no src change). The real-chain arm rides `team-command-proof.yml`'s disposable-matrix job (no local Docker on this machine).
- **Docs:** `FULL_CAPACITY_PLAN` §C gains P12 SHIP-SOUL (SOUL-0 shipped dark; SOUL-1/2 + ACT-SOUL remain) + queue row 17; `SYSTEM_BOUNDARIES` §1 gains the two rows (`ship_trait_types` Reference/Config; `main_ship_traits` sole writer = the roll fn, insert-only). NO §2 Ship-Soul system row yet — the roll fn exists but nothing calls it until SOUL-1 (the 0111/0117 no-live-system-row posture), deferral recorded here.

Coordination note (RESOLVED at rebase). This slice was cut from a pre-#127 origin/main; slice-shipyard0 (PR #127, mig 0185 + the `TEAMCMD_PASS_SHIPYARD0` team-proof block) merged while it was in flight. Reconciled onto the new origin/main exactly as planned: BOTH team-proof blocks kept (SHIPYARD0 from main, SOUL0 appended after it — fixture-independent: SOUL0 uses only fresh users, and SHIPYARD0's in-txn shard/blueprint knob carries do not touch the roll path), MARKERS merged to **17**, the marker-count comment + both selftest/local echo lines merged to carry both slices, both sets of selftest greps preserved, and the final PASS select carries both phrases. Migration numbering holds (0185 shipyard on main; 0186 SOUL-0 next — no renumber needed). Docs union-merged (plan P6 SHIPYARD + P12 SHIP-SOUL; both `SYSTEM_BOUNDARIES` row sets; both `DEV_LOG` entries). Team selftest re-run green with 17 markers + a post-merge mutation spot-check.

Hostile-review absorption (APPROVED; 3 MEDIUMs fixed pre-merge while 0186 was unapplied). M1 — the derivation order is now COLLATION-PINNED: `trait_type_id text collate "C"` on the column AND order by `trait_type_id collate "C"` in the writer + the proof's `soul_expect` (lockstep), self-asserted both ways (attcollation + the prosrc token) — the DB default collation can never re-order the catalog and shift unrolled derivations. M2 — catalog-growth honesty: the plan's ACT-SOUL charter now carries the FIXED-CATALOG qualifier + the catalog-freeze precondition (assert count = 8 before the backfill), and the roll envelope now reports the STORED rows (traits + the ship's real hp_mult product — also the N3 fix: a replay on a veteran ship reports 1.08, not a default), proof-pinned. M3 — the selftest's gate-flag hardening generalized: ANY `set_game_config('<gate>'...)` touch of the 8 boolean gates fails closed (the shipyard_enabled review lesson applied to all — knobs stay `set_game_config`, gates ride the raw in-txn update only). Nits: `soul_expect`'s re-salt loop bounded at 64 (mirrors the writer), explicit `||` in the 0186 config description literal, this entry's block/marker counts corrected to 17th/17.

Bugs / fixes

- `_(none — pure additive foundation; no live surface is touched.)_`

2026-07-12 — SHIPYARD-0: the ship-production foundation (mig 0185, dark)

Request. The SHIPYARD charter, slice 0 (owner directive: "ships must be made through mining, production, level requirement and more") — the catalog + faucet foundation ONLY: hull build recipes, the two T1 hulls dark, the blueprint combat faucet inert. NO build_orders changes, NO RPC, NO cron edits (SHIPYARD-1/2 own the queue + commands). Supersedes the plan's old P6 HULLS-2 line.

Work done

- **Hull-column verification (the load-bearing check):** `main_ship_hull_types` = the 0043 column set + ONE later alter — 0075 added `base_cargo_capacity_m3 numeric NOT NULL CHECK (> 0)` (the abstract→volume 1:1 law), grep-verified sole ALTER site; 0170 only UPDATES `base_stats_json` ({attack, defense} input vocabulary), 0171 only UPDATES `base_captain_slots` (the 2→6 bump), 0184 only UPDATES `name`. So the T1 seeds carry ALL of: hp/speed/cargo + m3 (140/140.0, 20/20.0) / support 10 (starter parity [D] — support craft are deprecated scaffolding, the column is NOT NULL, the value cosmetic) / captains 6 DIRECTLY (the 0171 law — `activate-captains.sql` and the team proof both assert EVERY hull row at 6) / modules 2 and 4 / `base_stats_json` exact.
- **mig 20260618000185_shipyard0_foundation.sql** — (a) `shipyard_enabled='false'` (the 0107 config+flag idiom; every SHIPYARD RPC must gate-first on it) + `blueprint_fragment_drop_rate='0'` (the 0171 knob idiom); (b) `hull_build_recipes` (hull PK→`main_ship_hull_types`, `credits_cost ≥ 0`, `build_seconds > 0`, T2+ gates `required_hull_type_id/required_captain_level` — T1 seeds both NULL, honest: captain levels are dark) + (c) `hull_recipe_ingredients` (FK-normalized, qty 0 — the 0107/0125 recipe posture), both Reference/Config public-read, migration-seeded only;

(d) the two T1 hulls DARK — **bulk_hauler** 'Mule-class Hauler' 650/0.8/140/{5,15} + **strike_corvette** 'Talon-class Corvette' 420/1.3/20/{30,10} (the 0184 EVE register, exactly as 0184 pre-registered Mule/Talon; inert: every commission path hardcodes 'starter_frigate'); (e) recipes [D]: hauler = ore 24 + crystal 6 + engine_parts 6 + scrap 12 + blueprint_fragment 2, corvette = ore 16 + crystal 4 + weapon_parts 6 + pirate_alloy 8 + blueprint_fragment 2, both 400 credits / 3600s.

- **The blueprint faucet (parity discipline):** `pirate_loot_for_wave` re-created from its TRUE head — **0171** (grep-verified: 0041 + 0171 are the only create sites; 0046/0167/0169 call it, 0174/0176/0183 only cite it) — with ONE marked SHIPYARD-0 (0185) hunk: `waves ≥ 8 roll random() < coalesce(cfg_num('blueprint_fragment_drop_rate'), 0)` for exactly 1 blueprint_fragment, appended AFTER the 0171 shard hunk (rate-0 output byte-identical INCLUDING element order; strict-`<` against a [0,1) draw → rate 0 NEVER fires). Extract-and-diff verified: 0185 minus the marked hunk == the 0171 body byte-for-byte. WAVE THRESHOLDS (the packet design): shards $w \geq 2$, blueprints $w \geq 8$ — the deep-run gate (engine_parts depth); wave 1 stays deterministic scrap-only at ANY rate (the live verify-phase5 exact pin can never flake — its other wave pins are inclusion-based, so a lit faucet stays green there too). blueprint_fragment now has TWO live sources: the 0098 exploration one-shot + this repeatable deep-combat faucet — recipes needing 2 are farmable, not dead-ended (the F5 lesson applied forward).
- **Self-asserts (the migration refuses to land wrong):** gate dark + knob 0 at seed time; 2 hulls exact on EVERY gameplay column (+ the 0170 stats / 0171 slots laws re-checked across ALL hull rows); 2 recipe headers exact (400/3600/NULL gates) + 10 ingredient rows exact, stray-free; every ingredient item-typed + drop-grounded (F4: loot prosrc tokens, mining-field bundles at `qty > 0`, the blueprint dual-source incl. the 0098 site bundle); the loot hunk tokens pinned in prosrc + behavioral endpoint pins (wave-1 scrap-only at any knobs — shard-tolerant by design since ACT-CAPTAINS may have raised that knob; rate-0 wave-10 carries NO blueprint); craftability shape (no headerless ingredients / ingredientless headers).
- **Proof:** new `TEAMCMD_PASS_SHIPYARD0` block in `scripts/team-command-proof.{sql,sh}` (after MOD2): committed `shipyard_enabled false` + knob 0 asserted FIRST (the flag is NEVER flipped, even in-txn — no shipyard RPC exists; pinned by a selftest negative grep); the hull + recipe catalog pinned verbatim; the faucet at its deterministic endpoints (the SHARDDROP technique, riding its in-txn shard-rate-1 fixture carry, asserted explicitly): rate-0 wave-8/wave-10 byte-parity with the 0171 head, rate-1 (real `set_game_config`, rolled back) wave-8 gains EXACTLY one appended blueprint qty 1 (additive-only: bundle minus blueprint == the 0171 bundle), wave-7 + wave-1 thresholds hold. Selftest greps in assert form + sole-writer negative greps (no direct `hull_build_recipes/hull_recipe_ingredients` writes anywhere in the proof); post-run honesty checks extended (committed blueprint knob still 0, shipyard flag still false). **Mutation-tested: 9/9 caught** (marker drop, exactly-one/parity/threshold/hull/ingredient pin guttings, the `set_game_config` raise drop, a smuggled direct recipe-table write, an in-txn shipyard flag flip); restored green.
- **Docs:** `FULL_CAPACITY_PLAN P6` → the SHIPYARD charter (T0/T1/T2 ladder, SHIPYARD-0..3 + ACT-SHIPYARD with 0 marked shipped; ACT-SHIPYARD preconditions: mining lit for the ore faucet + trade/salvage lit for credits); `SYSTEM_BOUNDARIES $1` rows for both new tables (Reference/Config, migration-seeded only, NO runtime writer — SHIPYARD-1's future build command reads them downward and spends ONLY via `inventory_spend/wallet_debit`).

Bugs / fixes

- `_(none — pure additive catalog + one parity-disciplined re-create; no engine, RPC, cron, or frontend surface touched.)_`

2026-07-12 — MOD2-1: the shield line + mining rig module seeds (mig 0183, dark)

Request. Plan §C P7 queue #11: seed `shield_lattice` (the defense line — the packet-F2 fix: `survival` is hull-only 10 today, the defense curve near-degenerate) + `mining_rig_extension`, a PURE catalog seed — new `module_types` + recipes, ZERO engine edits; recipes from live drops only (the F4 lesson); prove the stat keys against the adapter's ACTUAL reads.

Work done

- **Stat-key verification (the load-bearing check):** the adapter's TRUE head is 0180 (creates at 0044→0115→0122→0170→0180, grep-verified); its module loop reads stats.json keys `attack/defense/repair/cargo/scan/mining/evasion/speed_mult_bonus` coalesce-0 into the shared accumulators — `defense` → `a_survival` → output `survival`; `mining` → `a_mining` → `mining_yield`. So the shield seeds `{"defense": 12}` (input vocabulary — NOT "survival", which is the output key; hull base_stats.json uses the same {attack, defense} form, 0170) and the rig seeds `{"mining": 8}`. Migration self-asserts pin both: every seeded key must appear in the deployed prosrc in the exact (`m.stats_json->>'<key>'`) read form, plus the `survival/mining_yield` output keys — a typo'd key (= a dead module) cannot land.
- **mig 20260618000183_mod2_shield_line.sql** — idempotent seeds on the real unique keys (module_types PK id; module_recipe_ingredients PK (module_type_id, item_id)); slot_type archetypes `defense/mining` are NEW and additive (0107 (b) posture); both take the adapter tradeoff CASE's `else 0` arm — the engine posture ("the cost is the slot"), deliberate: taxing defense with attention would require an adapter re-create, out of a seed slice's scope. Numbers **[D owner-tunable]**: defense 12 = the plan's number (band: slot-1 autocannon attack 10; hull defense 10 → the shield more than doubles the curve input);

mining 8 (band: mining_drone capacity-2 → mining 8; the shipped slot-1 sensor precedent seeds exactly the cap-2 drone's value — deep_scan scan 8 = survey_drone's 8).

- **Recipes + grounding (live drops only, self-asserted):** shield = repair_parts 4 + pirate_alloy 3 + scrap 8 (plan verbatim; pirate_loot_for_wave head 0171: scrap $w \geq 1$, alloy $w \geq 3$, repair_parts $w \geq 10$ — the deep-run gate is INTENDED: the shield is a mid-game unlock, earned where defense matters (packet §1.1: $w_{10+} \approx$ a 4-ship kitted team's farm depth), precedent expanded_cargo_lattice's repair_parts 2). Rig = crystal 2 + ore 6 + scrap 4 [D] — crystal/ore are mining-field drops (0103 bundles: ore in all 5 fields at 2–3, crystal in 3 of 5 at 1–2): the cross-activity mine-to-mine-better loop; magnitudes match the thruster's crystal 2 / scrap 4. Self-asserts query the REAL sources (loot prosrc tokens; field bundles at qty > 0) — F4 is enforced, not just documented. HONESTY NOTE: mining_yield has no engine consumer yet (0103: weighted/depleting yields are a later additive change) — the rig's stat surfaces in previews/group totals until then; a stat gap, not an F4 item gap.
- **Proof (judged: the team proof IS the right home — fixture players, funded wallets, the real reward_grant path, and the adapter already under exact pins there; a standalone modules proof would re-build all of that for one block):** new TEAMCMD_PASS_MOD2 block in scripts/team-command-proof.{sql,sh} — fresh fixture user (free first commission → canonically docked = settled-SAFE for fitting 0114; captain/module-free so the baseline decomposes to hull defense exactly), both module gates asserted COMMITTED-dark then flipped in-txn only; catalog + recipes pinned verbatim; ingredients granted at EXACTLY the recipe quantities via the real reward_grant sole writer; craft_module spends the exact price to zero (the craftability math: the listed items ARE the entire price), a second craft → insufficient_items, replay verbatim with ONE minted instance; fit_module_to_ship → adapter survival = baseline +12 exactly (hull 10 → 22 — the defense stat's FIRST end-to-end pin) and mining_yield +8 exactly, with the minus-key isolation compare (nothing else moves but module_slots_used 1 → 2). Selftest greps added in assert form + sole-writer negative greps (no direct writes to any Modules/Fitting/Inventory/Reward table anywhere in the proof); post-run honesty checks extended to both module flags. Mutation-tested: 6 deliberate guttings (delta values, flag flip, marker, boundary assert, a smuggled direct module-table write) each fail the selftest; restored green.
- **Verifier contracts extended (they pin exact catalog counts):** verify-modules.mjs 4→6 types / 12→18 recipe rows + both recipes; verify-fitting.mjs EXPECTED_FITTING_CATALOG + the two new (slot_cost, stats) rows. Docs: FULL_CAPACITY_PLAN P7 + queue row 11 → shipped (dark); MOD2-2 Mk-II remains queued.

Balance note — the survival curve (what +12 does to the packet's enemy-damage math). Incoming damage rides the 0023 curve (combat_per_unit_and_pacing) $\text{enemy_attack} \times 100 / (100 + \text{survival}) \times \text{variance}$; survival is hull-only 10 today, so every ship eats $\times 0.909$ of raw enemy attack. One shield (10 → 22) moves the factor to $\times 0.820$ — **–9.8% incoming damage everywhere**. At Snare (bd 10 → 12.5 raw @ danger 1) that is ~ 1.1 hp/tick — real but marginal, and the shield occupies the slot an autocannon (+10 attack) would fill, so shields rarely pay at starter zones (slower clears eat the saving). At Ember Reach's hidden sites (bd 40/50/60 → 50/62.5/75 raw @ danger 1) the same –9.8% is **4.5–6.7 hp/tick**, compounding over long deep-run tick counts where danger growth already outruns any fixed team — the shield's value scales with exactly the content its $w \geq 10$ repair_parts recipe is gated behind. A full 3-shield fit reaches survival 46 ($\times 0.685$, –24.7%) at the price of EVERY attack slot — the tradeoff rides the Σ slot_cost cap, as designed. The axis this opens: HULLS2's bulk_hauler (defense 15) and MOD2-2's shield Mk-II now land on a live curve instead of a degenerate one.

Bugs / fixes

- _(none — pure additive seeds; the adapter, craft, and fit surfaces are untouched.)_

2026-07-12 — EV-1: the world-events producer — worldstate_tick narrates threshold events (dark)

Request. Plan §C P8 first slice (queue #16, EV-1 only — EV-2 event sites + EV-3 feed UI are later): worldstate_tick publishes threshold events through the EXISTING world_events_publish — pressure crossings, depletion warnings, drift extremes — dedup-keyed; publish failure never aborts the tick; zero events while dark; no new cron.

Work done (shipped shape includes the same-day hostile-review rework: state-detection everywhere, per-publication failure isolation, gated+guarded knob reads; rebased onto main @ 938a70c — #117 merged 0181 haul_read_surface, so this slice is 0182 as planned.)

- **Migration** 20260618000182_ev1_worldstate_events.sql —
`<font name="Sym"> </font> True head found = 0137` (grep-verified: the tick's only create sites are 0032 → 0034 → 0135 → 0136 → 0137; 0138..0181 never re-create/alter/drop it — 0172 re-created mining_extract + worldstate_deplete_field, NOT the tick; post-rebase re-verified: 0181 touches neither). Re-created VERBATIM with four fenced EV-1 (0182) hunks (declares; the gated+guarded knob loader; the post-UPDATE location-event block; the per-field depletion block). Extract-and-diff verified after the rebase: 0182 minus the fenced hunks == the 0137 body byte-for-byte.
`<font name="Sym"> </font> Gate shape (verified, stated honestly):` the tick does NOT globally no-op while dark — the pre-P19 decay/rollup always runs (the 0034 live behavior); every P19 dynamic gates per-block on world_balance_enabled. EV-1 follows exactly that shape: every publication sits inside `if v_wb_enabled` (location events also inside the existing

`v_drifted` guard) → a dark tick publishes ZERO events, reads ZERO EV-1 config, and its writes stay byte-identical to 0137. DOUBLE-DARK: the 0140 publisher itself no-ops (NULL) while `phase20_polish_enabled=false` — events flow only when BOTH flip (exactly the Rung-7 order).

The events — ALL STATE-detected (hostile-review HIGH fix: an edge, once consumed, has advanced even if its publish failed — the event would be lost forever; a state still holds next minute, so a failed publish genuinely retries; per-day dedup keeps it exactly-once). All `event_type='world_state'`, all through `world_events_publish` — NEVER a direct insert, pinned: `pressure_high` while pressure sits at/above `event_pressure_high_threshold=75` (critical $\geq$ threshold + half the remaining headroom = 87.5 at defaults, else warning; a PARKED-high location re-announces once per UTC day — documented as intended pressure-nagging [D]) / `pressure_eased` (the EXACT complement on the threshold, info, NOISE-SUPPRESSED: only when TODAY's `pressure_high` was announced — a read-only lookup of the tick's own published rows, so a perpetually-calm location never "eases"; the daily re-announce keeps the same-day pairing coherent across midnight) — location scope; `field_depleting` (post-regen reserve $< \text{event_depletion_warn_fraction}=0.25$; global scope — fields are hidden spatial sites, not Map locations; the title carries the field NAME only, a deliberate mild narration reveal since a depleting field is by definition actively worked); `price_surge/price_crash` (outside `event_drift_extreme_band_low/high = 0.6/1.4`). Dedup keys `<family>:<subject_id>:<UTC day>`.

Thresholds grounded [D owner-tunable]: pressure 0..100 baseline 50 (0031) → 75 $\approx$ a sustained 7-defeat/hour danger target (0135 term). Reserve 1.0→0.1-floor in 0.1 steps (0137) → 0.25 $\approx$ 8 extractions deep. Drift: the brief's notional 1.6 high is UNREACHABLE — the 0136 target caps at 1.0 + $\text{coeff}(0.5) = 1.5$ — so 1.4 (80% of the max premium, needs pressure > 90 sustained ~ 16 ticks); 0.6 is symmetric defensive cover (target never < 1.0 under default tuning).

D2 cron-safety (per-publication isolation): EVERY publication is its OWN begin/exception subtransaction — pressure-high / eased / drift each isolated per location, depletion one subtransaction PER FIELD — so one failure can neither abort the tick (the location UPDATE sits outside the guards) nor roll back a sibling's success nor skip the remaining fields; `query_canceled` is re-raised in every guard (a cancel is never neutered); everything else logs a WARNING. State-detection makes the next minute's retry genuine. NO new cron — the 0033 60s heartbeat, self-asserted unchanged.

Knob safety (hostile-review MEDIUM fix): the four EV-1 knobs are NOT read in DECLARE (which would run unguarded every 60s, dark included) — they load in a gated (`v_wb_enabled` only) + guarded loader: a bad cast aborts only the loader subtransaction (→ seeded defaults + WARNING) and a NaN backstop catches values that cast fine but compare as the greatest double. A mis-set owner-tunable can never kill the live decay/rollup heartbeat, and the config descriptions' "only consumed when `world_balance_enabled=true`" is literally true.

Self-asserts: knobs at the seeds; BOTH gates still dark (lit-early deploy fails closed — the 0180 precedent); parity spot-pins (0135/0136/0137 head tokens survive; exactly 5 `world_events_publish` call sites; all 5 dedup families; exactly 5 failure guards + exactly 5 `query_canceled` re-raises; NO direct `world_events` insert in the prosrc); a dark dry-run (real tick → zero events; publish probe → NULL); cron unchanged (exactly the one 0033 job, no second schedule drives the tick).

- **Proof** `scripts/ev1-proof.{sql,sh}` (lib-based, one BEGIN..ROLLBACK; both gates + every transient config write inside the txn only): dark tick + dark publish → zero rows; state highs via the REAL decay math (transient baseline/decay-rate tuning: 60→80 warning, 60→92 critical ≥ 87.5); eased 80→50 with today's high announced → one info event AND a never-high location below the threshold stays silent (the suppression pin); same-day still-high re-tick → zero new rows (the dedup pin); depletion via the REAL sole writer `worldstate_deplete_field` $\times 8$ → one global warning naming the field, second tick deduped; drift fixtures 1.45→1.405 ≥ 1.4 surge / 0.52→0.568 ≤ 0.6 crash; **publish-failure injection** (a BEFORE INSERT trigger blowing up every `world_events` insert → the tick completes and its `location_state` write survives) **plus the genuine-retry pin** (trigger dropped → the very next tick finds the state still holding and LANDS the lost event — the property edge semantics could never satisfy); **knob guard** (uncastable "not-a-number" AND "NaN" threshold values → tick completes, falls back to the default 75, still narrates); **narrate-never-mutate** (direct publishes change no `location_state/mining_field_state` byte — md5-snapshot compare). Mutation-tested: 8 mutations (COMMIT swap, post-ROLLBACK flag toggle, direct `world_events` insert, retry-marker gut, real-writer bypass, dedup-family gut, NaN-arm gut, suppression-assert gut) each individually FAIL the selftest; restored → green.
- **Workflow** `world-events-proof.yml` (NEW; selftest + disposable-matrix on `slice-ev** + autopilot/**`). HOST DECISION: a new family-pure workflow, NOT a 7th trade-v1-proof step — trade-v1 is the trade-family host (name, triggers, and matrix are trade-branded) and team-command the team/captain host; grafting worldstate coverage onto trade-v1 would misfile it, make every trade branch pay for it, and pollute its branch filters. Many small family-pure proof workflows is the repo convention.
- **Docs:** SYSTEM_BOUNDARIES (World State exposes + forbidden rows, the `world_events` matrix row, a new acyclic-edge note World State → World Events, the 60s-cron section — narrate-never-mutate restated everywhere); FULL_CAPACITY_PLAN (P8 header + queue #16: EV-1 shipped dark; EV-2/EV-3 + ACT-PHASE20 remain).

Verify. ev1-proof selftest green + 8/8 mutations caught; all other proof selftests green (osn-hub1a-production-catalog-verify fails identically on the clean tree — pre-existing, environment-dependent, unrelated); parity diff re-verified byte-clean after the rebase onto main @ 938a70c (#117 merged; 0181 touches neither the tick nor `world_events`); `tsc -b + vite build` green

(untouched). Real-chain run = `world-events-proof.yml` disposable-matrix on the `slice-ev1` push (no local Docker/psql — the established norm).

Open follow-ups

- EV-2: event sites (time-activated `event_site` locations, 48h lifecycle, retired hidden).
- EV-3: feed UI (WorldEventsPanel + map badges via `ui_asset_catalog`).
- ACT-WORLDBAL + ACT-PHASE20 (queue #15/#16): the Rung-7 flips — world balance first, then phase20; the EV-1 knobs (75 / 0.25 / 0.6–1.4) are the [D] owner-tunables to revisit at flip time.

2026-07-12 — ACT-HAUL: the delivery-contracts activation script

Request. The HAUL closer (plan §C P2; ROADMAP phase-22): `scripts/activate-haul.{sql,sh}` in the proven `activate-*` idiom. NO migration — the contracts stack is fully built dark (0176 templates + schema + flag + the deterministic hourly generator; 0179 accept/deliver + receipts + `deliver_by` + the cap; 0181 the `get_port_contracts` bulletin read + the server-lit HaulBoardPanel, PR #117). The script IS the flip: `haul_contracts_enabled` → `true` + one sanctioned generator invoke.

Work done

- `scripts/activate-haul.sql` — one all-or-nothing `BEGIN..COMMIT` (txn-local UTC — the generator's `offer_day` anchor), zero psql meta-commands. `PRECONDITIONS` (read-only): `head ≥ 0181 + 0176/0179/0181` recorded; the 5 tables; the 12-function surface via `to_regprocedure` (the haul four + the deliver charter leaves + `market_buy` — the sourcing faucet — + the config leaves); `DEPLOYED`-body prosrc pins — the generator must be the 0179 re-create (the (a2) `deliver_by ≤ now()` → 'cancelled' hunk the stale 0176 body lacks) with the 0176 pins intact (offered-only expiry, `hashtextextended` determinism, natural-key on-conflict mint); accept pins (gate, `too_many_active`, the `hashtext('haul_accept')` per-player advisory serialization, `already_accepted_other`); deliver pins (`trade_cargo_consume` + `wallet_credit` — the charter fan-out — `deadline_passed`, `idempotent_replay`); read pins (gate, `expires_at > now()`, `accepted_by = v_player`); EXACTLY 10 active templates with the 0176 §4 WORTH-TAKING invariants re-derived from the LIVE market rows at both qty endpoints; the 3 starter ports generator-eligible; knobs sane READ-ONLY (`haul_offers_per_port 1..10`, `cap ≥ 1` — 0 is a legal owner freeze but not a launch state); the cron exactly once; the flag key exists (value not asserted — re-runs supported); ACLs intact (generator service-role-only, client RPCs authenticated-only).
- **THE TRADE PRECONDITION (decided: hard gate, not a warning).** `trade_market_enabled` must be COMMITTED true (`raw + cfg_bool`). THE FINDING THAT FORCED IT: accepting is trade-independent (a claim moves nothing — 0179 §1), but delivering consumes `ship_cargo_lots` via `trade_cargo_consume`, and the SOLE producer of `ship_cargo_lots` is `market_buy` (grep-verified: `trade_cargo_add_lot`'s only call sites are `market_buy` and its re-creates — 0089/0092/0136/0138) — which server-rejects while trade is dark. **The mining "alternative" was checked and does NOT exist in this codebase:** mining rewards land as ITEM-inventory rows ('ore'/'crystal' `item_types` via `reward_grant` — the 0102 decision (1)), never a cargo lot; the `trade_goods` 'ore' is a separate catalog. So with trade dark there is NO sourcing path at all — a lit board would be a 100% dead affordance (every accept ends in the (a2) cancel at `deliver_by`). Hence PRECONDITION with the full reasoning in the failure hint. FLIP ORDER: ACT-TRADE first.
- **STAGE 1 — the ONE flag write** (`set_game_config`), ORDERED BEFORE THE GENERATOR INVOKE: the generator dark-gates on the flag (feature_disabled while false — it physically cannot mint dark); both stages in the one txn, so an offer-stage failure rolls the flag back too.
- **STAGE 2 — INSTANT OFFERS (decided: YES, invoke in-stage).** One `public.haul.generate_offers()` call — the SANCTIONED sole mint entrypoint (the script never writes haul tables directly). Rationale: the hourly '7' cadence means the first lit firing lands up to ~1h post-flip — an empty bulletin hour for no reason; invoked in-txn (the txn sees its own stage-1 write, so the gate is open) offers exist at all 3 ports the moment the flip commits. Asserts: `envelope {ok:true}`; `ports == 3`; `minted > 0` with the ONE tolerated exception — a SAME-DAY RE-RUN mints 0 by the (origin, `offer_day`, slot) natural key, accepted only if today's rows already exist; and every starter port carries `≥ 1` today-row (run-time-independent). RE-RUN SEMANTICS: no-op success (flag upsert + idempotent mint; a later-day re-run mints the new day's board — exactly what the cron does anyway).
- **SMOKE — the authed-RPC decision:** activate-captains stopped at `to_regprocedure` existence; activate-ranking invoked its client RPCs directly (they need no auth subject). `get_port_contracts` needs `auth.uid()` and the script runs privileged — so it is called FOR REAL under a TRANSACTION-LOCAL fake JWT (`set_config('request.jwt.claims', ..., true)` with a random sub — the exact technique the 0179/0181 self-asserts and haul-proof.sql use; the sub owns nothing so mine must be [], nothing is written, claims cleared after and txn-local anyway), asserted `{ok:true}` + `max_active` mirroring the cap knob + the offered array EQUAL to the direct fresh-offered count at Haven Reach (RPC == table truth). Plus: flag committed (`raw + cfg_bool`), per-port fresh counts (FYI), cron intact once, `haul_receipts` selectable. Markers `ACTIVATE_HAUL_PASS_*` + final PASS line.
- **MOUNT VERIFIED — NO client PR:** HaulBoardPanel is already mounted on the Port screen aside rail (PortScreen.tsx:80, docked branch), gated ONLY by `isServerLit(board)` (HaulBoardPanel.tsx:120); no `HAUL_*` compile constant in `osnReleaseGates.ts` or anywhere in src (grep-verified). At flip time players docked at any starter port see the Contracts bulletin

with ~2 fresh offers (haul_offers_per_port=2) INSTANTLY; accept at the origin, "Your contracts n/3", deliver with any owned ship at the dest before deliver_by, credits via Wallet with idempotent receipts; the hourly cron keeps the board rolling.

- **ROLLBACK (commented, flag-only + THE EXPIRY FREEZE — the 0176 forward note landed, as required, extended by 0179):** darkening freezes BOTH generator passes — stale 'offered' rows persist un-expired (raw-RLS-readable, but the CLIENT board vanishes instantly because it rides the gated read RPC — the 0181 rationale, honored) and 'accepted' rows past deliver_by stay un-cancelled holding cap slots. Running the generator once post-dark does NOT sweep (gate-first no-op — the "optionally run the generator" idea is explicitly rejected in the section). Choice documented: ACCEPT the frozen state (recommended — unreachable through every gated surface; on re-light the first cron firing sweeps both passes, no gap) or the commented manual service-role sweep (the generator's own (a)/(a2) predicates verbatim).
- **scripts/activate-haul.sh** — selftest/run wrapper (confirm token ACTIVATE_HAUL). The DB-free selftest adapts the established write-counting to this script's TWO sanctioned mutations: exactly ONE set_game_config(' call site (→ true on the one approved key; never a knob or another window's key — the forbidden list now includes the haul knobs + captain_growth_enabled) AND exactly ONE := public.haul_generate_offers() invocation (the to_regprocedure signature strings count as existence, not call sites); NO direct table DML, no DDL, no meta-commands; one timed UTC BEGIN..COMMIT; the migration + prosrc + worth-taking + cron + trade-precondition greps; the fake-JWT smoke greps (call + offered-vs-table match + mine leak-check + claims cleared); the ordering/instant-offers/re-run/mount/rollback-freeze doc tokens.

Verification. bash scripts/activate-haul.sh selftest green; **mutation-tested 12/12 caught (reviewer added occurrence-count hardening for same-line/perform-form evasions — M6)** (second flag write, uncommented rollback false-write, gutted trade cfg_bool precondition, gutted (a2) prosrc pin, direct table UPDATE, TRUNCATE DDL, psql meta-command, dropped generator invoke, DOUBLED generator invoke, dropped fake-JWT clear, gutted ports=3 envelope assert, renamed STAGE2 marker — plus control green between each). All 8 existing activate/reveal selftests untouched-green (trade, ranking, captains, exploration, mining, team-command, coordinate-travel, ember-reach) + haul-proof selftest green; tsc --noEmit + vite build untouched-green (no src change). Docs: plan §C P2 + queue row 12 + the Then-line → fully stocked / script shipped; ROADMAP phase-22 row annotated. Nothing committed/pushed/applied — the flip stays human-gated, ordered AFTER ACT-TRADE.

2026-07-12 — C2-2: captain level → stats adapter fold (dark)

Request. The CAPXP continuation (plan §C P5, after the 0177 foundation): the level-curve → adapter parity delta — calculate_expedition_stats scales each assigned captain's stats contribution by $(1 + (\text{level} - 1) \times \text{level_bonus})$, byte-inert at level 1 AND while captain_growth_enabled is dark. C2-3 XP bars + C2-4 the 6→8 slot raise = later slices.

Work done

- **Migration 20260618000180_c2_2_captain_level_fold.sql** —
`<font name="Sym"> </font> True head found = 0170` (grep-verified: the only create sites are 0044 → 0115 → 0122 → 0170; 0171..0179 never re-create it — 0159/0165/0166/0168 only call it). Re-created VERBATIM with the marked C2-2 (0180) delta: three added declares (flag + knob read ONCE at entry — never per-captain, so a mid-scan config write can't split one ship's roster across regimes), i.level added to the captain-loop join select (additive column), and the ONE hunk where captain stats fold in: `v_lv1_mult := case when v_growth then 1 + (c.level - 1) * v_lv1_bonus else 1 end`, with `* v_lv1_mult` appended to the EIGHT stats_json contribution reads (7 accumulators + speed_mult_bonus). Extract-and-diff verified: the declares + the join column + the hunk only. Stated honestly in the header: unlike 0170's add-only shape, a multiplicative scale must MODIFY those eight lines.
`<font name="Sym"> </font> THE DOUBLE GATE:` while dark the multiplier is exactly 1.0 regardless of level; at level 1 it is exactly 1.0 regardless of the flag — so the delta is DOUBLY inert today (flag committed false AND every captain level 1 — the accrual, its sole writer, has only ever run dark). Knob floored at 0 and level ≥ 1 by the 0177 CHECK → the multiplier is never < 1 (leveling is never a nerf; the tradeoff CASE lines — attention/speed penalty — stay level-flat: growth is never a stealth cost raise).
`<font name="Sym"> </font> Knob captain_level_bonus_per_level seeded 0.10 [D owner-tunable]` — with the 0177 curve (level = 1 + floor(sqrt(xp/100))); level 2 (100 xp) = +10% on the captain-contributed portion (a level-2 gunnery_veteran contributes 4.4 attack instead of 4), level 3 = +20%, ...
`<font name="Sym"> </font> Self-asserts:` knob at 0.10; flag still dark + zero captains above level 1 at migration time (a lit-early deploy fails closed, forcing a human decision); prosrc pins — the gated-multiplier token verbatim, captain_level_bonus_per_level read, i.level joined, EXACTLY 8 * `v_lv1_mult` scale sites (fewer = a dropped stat, more = a leak onto another pipeline), tradeoffs unscaled; adapter ACL still server-only. (No adapter spot-call in the migration — no fixture exists there; the prosrc pins + the CI proof carry the behavior.)
- **Proof:** TEAMCMD_PASS_CAPLEVEL block appended to scripts/team-command-proof.sql (after CAPXP, consuming its outcome AS the level-2 fixture — capa+capb at exactly level 2 on c1, capd at level 1 on the grantless b1, flag lit in-txn). All expectations derived INDEPENDENTLY from catalog/instance joins (the TEAMSTATS style): (1) LIT + LEVEL 2 — c1's combat_power exceeds the level-1 baseline by exactly $\text{round}(\text{knob} \times \sum(\text{level} - 1) \times \text{attack}, 2)$ (23 → 23.8 at the seeds) with every other key

byte-identical (isolation pin); (2) FLAG OFF + LEVEL 2 — the level-1 world exactly (combat_power = hull attack + Σ captain attack, absolute); (3) FLAG ON + LEVEL 1 — b1 byte-identical lit or dark. Plus a real-bonus guard ($\Sigma(\text{level-1}) \times \text{attack}$ must be > 0 — a $0=0$ compare can only false-green) and the committed-knob-untouched pin.

Reconciliation (the C2-2 READ check): the CAPXP block flips the flag and raises captains to level 2, but NO existing block calls the adapter after that point (grep: zero adapter/preview/totals calls past the TEAMSETTLE block; the flag flip postdates every adapter-calling block) — so every existing pin runs dark at level 1, where the 0180 multiplier is exactly 1.0 twice over, and stays byte-valid with NO fixture surgery.

- `scripts/team-command-proof.sh`: `TEAMCMD_PASS_CAPLEVEL` marker + 7 CAPLEVEL selftest greps in assert form. Mutation-tested: each of the 7 pins individually gutted → selftest FAILs; restored → green.

Verify. team-command selftest green; all other proof selftests green (osn-hub1a-production-catalog-verify selftest fails identically on the clean tree — pre-existing, environment-dependent, unrelated); `tsc -b + vite` build green (untouched). Real-chain run = `team-command-proof.yml` disposable-matrix on the `slice-c2-2` push (no local Docker/psql — the established norm).

Open follow-ups

- C2-3: XP bars in TeamMemberCaptains (reads `captain_instances.xp/level` — first client consumer).
- C2-4: the 6→8 captain-slot raise (separate additive migration, lit-time decision).
- ACT-CAPXP flip still owns the dark-backlog decision (0177 header) — and now also lights the level fold: flipping `captain_growth_enabled` activates BOTH accrual and the adapter bonus.

2026-07-12 — HAUL-2: the contract accept/deliver RPCs (dark)

Request. The HAUL continuation (plan §C P2, after the 0176 foundation): the accept/deliver state-transition RPCs — `haul_accept_contract` + `haul_deliver_contract` — with receipts, the delivery deadline, the active cap, and the expiry of accepted-but-undelivered contracts. Everything stays DARK behind the same `haul_contracts_enabled` flag (no flag write). HAUL-3 UI + ACT-HAUL = later slices.

Work done

- **Migration** `20260618000179_haul_accept_deliver.sql` — the semantics decisions, each documented in the header:
 - Accept is a CLAIM, not a transaction:** moves NO cargo and NO credits — the player sources the goods themselves (market buy at origin / existing cargo); exactly what the 0176 reward math prices in (reward $>$ qty $\times$ origin.sell always — the worth-taking invariant covers the buy).
 - Origin-port accept:** the bulletin is per-port — the resolved ship must be DOCKED (the ONE resolver 0092/0138) at the contract's origin; a contract elsewhere folds into `contract_not_found`. Offer expiry is ALSO checked at accept (fail-closed — never waits on the hourly cron's flip).
 - Any-owned-ship deliver:** the contract is player-scoped (`accepted_by`); ANY owned ship docked at the DEST holding the goods delivers. `accepted_ship` = provenance only.
 - deliver_by reconciliation:** 0176's `duration_seconds` anchored the OFFER pickup window and shipped NO delivery window. Decision: reuse the same knob as the delivery window — `deliver_by` = `accepted_at` + `duration_seconds` (the contract's TEMPO: 6h staples, 12h premium runs; one owner-tunable number, no reseed; a future wave can split it with a new column). A new CHECK pins every 'accepted' row to a non-null `deliver_by` (else it would hold a cap slot forever).
 - Expiry shape (chosen):** the generator re-created from its 0176 head under PARITY DISCIPLINE — diff-verified byte-identical except THREE marked [HAUL-2 hunk] groups: the `v_cancelled` local, the (a2) pass ('accepted' past `deliver_by` → 'cancelled'; no penalty v1 [D]; frees the cap slot), and the `accepted_cancelled` envelope field. Chosen over a second cron function: one scheduled entryptpoint, one gate, one envelope — the cron job untouched. Pass (a) stays offered-only (accepted rows are never 'expired'); (a2) is the SINGLE writer of accepted→cancelled — the deliver RPC rejects `deadline_passed` but never flips the row.
 - haul_receipts** (0176 shipped NO receipts table — checked; explicitly deferred): the salvage_receipts 0174 shape point-for-point — unique (main_ship_id, request_id), ship-keyed, replay-verbatim; ONE table for BOTH actions (action discriminator) so a ship's request namespace is one idempotency domain (trade semantics, no payload-conflict check). `reward_credits` = credits actually moved by the action (0 on accept). Owner-read via the owning ship; sole writers = the two RPCs. New knob `haul_max_active_per_player` seeded 3 [D] (counts 'accepted' only, EXCLUDING the contract being re-accepted — so a replay at the cap works; 0 = freeze).
 - Reject orders** (gate-first envelopes, the 0174 template): `accept_not_authenticated` → `haul_contracts_disabled` → `invalid_request` → `ship_not_found` → per-ship lock → `not_docked` → `contract_not_found` (row FOR UPDATE — cross-player accept races serialize on the row lock; ship-lock-then-contract-lock everywhere, no deadlock cycle) → `already_accepted_other` → `too_many_active` → `idempotent_replay` → `already_accepted` (mine, fresh request) → ok. Deliver: ... `not_docked` → `contract_not_found` → `idempotent_replay` **BEFORE the**

state/port/deadline/cargo guards (a successful delivery flips the status AND consumes the cargo — a retry must replay, never bounce; the documented 0174-delta posture) → `contract_not_found` (not mine/not accepted) → `wrong_port` → `deadline_passed` → `insufficient_cargo` (inline lot-sum — the `market_sell 0090` read idiom) → ok: `trade_cargo_consume` (the ONE FIFO debiter; **cost basis consumed and LOST — the reward covers it**, reported as `cost_basis_consumed`) + `wallet_credit` + delivered flip + receipt, atomic. ACLs: both RPCs authenticated-only; every internal leaf stays client-revoked.

Self-asserts: `deliver_by` + CHECK; receipts RLS/grants exact; RPC identities + ACLs; internals still private; knob 3; flag STILL dark; generator parity spot-pins (offered-only expiry + salts + natural key + the (a2) predicate) + ACL; cron still exactly once; dark dry-runs — generator no-op envelope zero-delta, RPCs `not_authenticated` with no subject and `haul_contracts_disabled` gate-first under a transient fake JWT, zero receipts.

- **Proof** `scripts/haul-proof.{sql,sh}` **extended** (same one-BEGIN..ROLLBACK harness; fixture ships via the real `commission_first_main_ship`, travel via the real move command + rewind + the REAL arrival processor, wallets pre-set to a known 100 — the `tm1/sv1` funding precedent): P0 + dark RPC rejects (`haul_contracts_disabled`, zero receipts) → P1 also pins `deliver_by` NULL pre-accept → P4 accepted fixture now carries a future `deliver_by` (the 0179 CHECK) and pins `accepted_cancelled=0` — immunity of within-deadline accepted rows to BOTH passes → P5 + `haul_receipts` policy/grants + RPC ACLs + internal-leaf privacy → **P6 ACCEPT** (origin-port happy path: `deliver_by` = `accepted_at` + duration EXACT recompute, zero cargo/credit movement, receipt exact, replay verbatim) → **P6b guards** (`already_accepted` self, `already_accepted_other`, `wrong_port` + fixture-aged stale-offer `contract_not_found` fail-closed, `too_many_active` at a transient cap 1 WITH replay-at-cap intact — all zero-write) → **P7 DELIVER** (`wrong_port` at the origin, foreign `contract_not_found`, empty-hold `insufficient_cargo`; fixture cargo `qty+2` through the REAL `trade_cargo_add_lot` leaf at the origin's live sell price — direct `ship_cargo_lots` writes selftest-BANNED; happy path: `wallet` +reward EXACT on the known 100, cargo `-qty` exact (remainder 2), FIFO `cost_basis_consumed` = `qty` × `basis` exact, delivered flip + receipt; replay verbatim, no cost field, no doubles) → **P8 DEADLINE+CANCEL** (data-driven second accept at the ship's current port; fixture-aged `deliver_by`; `deadline_passed` reject-only zero-write; generator run: `accepted_cancelled=1`, the slot FREED (active count 0), the within-deadline accepted row untouched again, zero expiries/mints). New markers: `HAUL_PASS_ACCEPT` / `_ACCEPT_GUARDS` / `_DELIVER_GUARDS` / `_DELIVER` / `_DEADLINE_CANCEL`. Selftest greps extended: RPC invocations, dark rejects, `deliver_by` anchor recompute, claim-moves-nothing pins, all guard tokens, exact wallet delta, cost-basis pin, cancel-frees-slot pin, receipts-insert + `ship_cargo_lots`-write bans, cargo-leaf requirement.
- **Docs:** `SYSTEM_BOUNDARIES` §1 `haul_contracts` row (writers COMPLETE and closed: the three functions)

new `haul_receipts` row; §2 Haul Contracts system row (both RPCs' full reject orders + the (a2) pass

updated never-list). `FULL_CAPACITY_PLAN` §C P2 + queue row 12 → HAUL-2 shipped (dark); trailing resequence note → HAUL-3 + flip remain.

Verification. `bash scripts/haul-proof.sh selftest` green; **mutation-tested 14/14 caught** (commit-instead-of-rollback, flag-enable dropped, direct `haul_contracts` insert, dark-RPC-reject pin gutted, cargo-leaf call renamed away, direct `ship_cargo_lots` write, `deliver_by` anchor gutted, replay-at-cap pin dropped, deadline-cancel marker dropped, exact-wallet-delta gutted, SQL after the final rollback, direct `haul_receipts` insert, accepted-immunity pin dropped, (a2)-envelope pin dropped), control green. All seven lib-consumer selftests green (six trade-family + team-command). Generator parity diff-verified: 0176 → 0179 differs by EXACTLY the three marked hunks. `tsc/vite` build untouched-green (no client code). Real-chain run = the `trade-v1-proof.yml` disposable matrix on push (`slice-haul**` trigger covers `slice-haul2`). NOT committed/pushed (per instruction).

Bugs / fixes

- `_(none)_`

2026-07-12 — COORD-GUARD (queue #14.5): the A0-fix + the coordinate-travel flip script

Request. The pre-activation slice that makes the free-coordinate-travel flip safe: resolver-guard the raw coordinate command (the standing "A0-fix" debt, `docs/TEAM_COMMAND.md`), then ship the activate-family flip script for `mainship_coordinate_travel_enabled`. Motivation (`WORLD_RECON_F1` §7): exploration/mining sites sit out to ±4200 and need a settled in_space ship within 750 units, while the only dockable anchors are the three 0066 port anchors `(-50,-30)/(70,-10)/(10,80)` — computed from the seeds, the NEAREST site ('Derelict Listening Post' `(-1200,850)`) is ~1,434 units from the nearest anchor `(10,80)`, nearly double the radius, so those rungs are UNREACHABLE until this flag flips; the farthest site from its nearest anchor ('Precursor Vault Signal' `(-4100,3600)`) is ~5,411 units ≈ 5,411 s at `base_speed 1.0` — one legal jump under the 86,400 s cap. But the flag's command, `command_main_ship_space_move` (head 0070), still derived the ship with the pre-multi-ship unguarded `where player_id = v_player` (deferred by OC's §2.5 [C] row because the coordinate gate rejected before the read). Multi-ship is LIVE (24-ship cap, commissioning lit) — flipping without the guard lights an arbitrary-ship read.

Work done

- **Migration** 20260618000178_coord_guard_a0fix.sql — parity re-create from the grep-verified TRUE head (create-sites: 0060 → 0070; nothing later): trailing `p_main_ship_id` `uuid` `default` `null` (the 0081/0159 idiom; drop + recreate) and step 3's unguarded read swapped for `mainship_resolve_owned_ship(v_player, p_main_ship_id)` — explicit id → ownership-asserted, `null` → sole-ship shim, `N≠1` → fail-closed `no_ship` (verbatim shape). Diff-verified: the function body differs from 0070 by EXACTLY the step-3 comment + resolver swap; every single-ship player is byte-identical end to end. ACLs re-asserted on the new identity (authenticated-only). Self-asserts: 4-arg identity exists, 3-arg gone (no overload), resolver CALL in `prosrc`, unguarded read gone, both flag gates still precede the resolution, ACL, resolver head present. Backward compatible: id-less callers resolve via the trailing default → shim. Signature-pin repoints (the pins §#1 had recorded as "remain valid") appended COMPLETE to `docs/TRADE_FLEET_OC_VERIFIER_REPOINT.md` §#1 for the deploy-time human gate — the `regprocedure/privilege` pins PLUS the arg-type-OID census (`osn-postenable-verify.sql:176-181` `COORD_SURFACE_COUNT`, `pronargs` 3→4 + `proargtypes[3]=uuid`; `.sh:89/135` consumers) and the exact-args + SECURITY-INTENT asserts (`osn3-s6a-realchain-perm.sql:28, :34` and `osn3-s6a-live-check.sh:62;65` — the `:34/65` "no ship id param" intent is now deliberately false and gets a SEMANTIC rewrite: 'player' still absent, the sole ship arg is the trailing ownership-resolved `p_main_ship_id`). No verifier file edited. Scope note: this closes the last unguarded read on the OSN MOVEMENT surface; a separate read-only pre-multi-ship read remains in `commission_first_main_ship` (0072:117,129 — arbitrary-row CLASSIFICATION at `N>1`, not exploitable; tiny follow-up queued in `docs/TEAM_COMMAND.md`).
- **Client passthrough (same slice — the multi-ship UX half of the guard):** the S6C tap flow now sends the SELECTED ship. Without it, a multi-ship player would get a LIT coordinate UI (readiness is ship-scoped) whose every move answers `no_ship`. Following the Stop pattern exactly: `buildSpaceMoveRpcArgs` gains `p_main_ship_id` (null-defaulted; `spaceMoveCommand.ts`), `commandMainShipSpaceMove` gains the trailing `mainShipId` param (`mainShipApi.ts`), `useSpaceMoveCommand` gains the `mainShipId` override + `[mainShipId]` controller-recreation key (`useSpaceMoveCommand.ts` — the stop hook's exact lifecycle), and `GalaxyMap` threads `mainShip?.main_ship_id ?? null` — the SAME source as the stop command and the readiness read, so the move targets the ship the readiness was scoped to. `tests/spaceMoveCommand.spec.ts` extended: exactly FOUR payload keys, the only ship-matching key is `p_main_ship_id`, no `player/location/ target_kind` key, omitted/null ship id → explicit `p_main_ship_id: null` (the shim wire shape). Dark until the flip → zero live-behavior change today.
- `scripts/activate-coordinate-travel.{sql,sh}` (activate-captains idiom) — PRECONDITIONS: head ≥ 0178 recorded; the guarded body live via the `prosrc` pin PAIR (the resolver-CALL assignment token + the unguarded-read-GONE negative check — the pair is the teeth — plus old-3-arg-gone); stop guarded (0083); `get_osn_movement_readiness(uuid)` present with the `coordinate_travel_available` derivation; flag key exists; site/field counts + an anchors-nonempty assert (a NULL distance record must not pass as proof) + the LIVE-computed min anchor→site distance printed as the reachability record. STAGE 1 (the ONE write): `mainship_coordinate_travel_enabled` → true via `set_game_config`. SMOKE (read-only): committed value + `cfg_bool` + the movement-domain flag still true, readiness capability, move+stop guarded `prosrc`, ACL sanity, every site/field inside the ±10000 envelope. Markers + PASS line. Commented flag-only ROLLBACK with the verified in-flight-settle citation: `process_mainship_space_arrivals` (head 0064:95) reads NO flag; `command_main_ship_settle_arrival` (0150:66) gates only on `mainship_space_movement_enabled`; the stop writer only branches on a flag when NOT in transit (0064:250-259); grep-verified the coordinate flag's ONLY readers are the move command + the readiness projection. **NO client PR** (verified): coordinate targeting is driven SOLELY by the server-derived `coordinate_travel_available` — `osnReleaseGates.ts:5-16` (compile const RETIRED), `osnReadiness.ts:63` + `isCoordinateTargetingActionable` (81-91), `GalaxyMap.tsx:93-101` (`canTarget`). Selftest mutation-tested: 9 mutations (flag→false, second write, marker drop, pin drop, direct table write, uncommented rollback, precondition drop, `psql` meta-command, no-client-PR-note drop) — all 9 flip it to FAIL; the clean file passes.
- **Docs:** `FULL_CAPACITY_PLAN` — Rung 0.5 inserted in the ladder (exploration/mining rungs now marked REQUIRES Rung 0.5) + queue row #14.5 shipped (+ #14 marked shipped); `TEAM_COMMAND` A0-fix debt note marked RESOLVED; `TRADE_FLEET_OC_VERIFIER_REPOINT` §#1 converted with the repoint table.

Verification (this sandbox, honest). New selftest green + all 9 mutations fatal; every existing `activate-*/proof/frozen-verifier` selftest still green; the full pure spec battery green including the extended `spaceMoveCommand` spec (the 4-key payload proofs); `npm run build` (`tsc` + `vite`) green. No local DB (Docker down) — the migration apply remains the human deploy gate, per house convention.

2026-07-12 — CAPXP-0/1 (queue #13): captain-XP foundation (dark)

Request. Queue slice #13 of the full-capacity plan (§C P5): the captain-XP foundation — C2-0 (additive `captain_instances.xp/level` + `captain_growth_enabled` dark flag) + C2-1 (XP accrual as a downward reader of `FINALIZED reward_grants`, the commit-safe 0144/0145 anti-join idiom with a `captain_counted_grants` ledger). The level-curve adapter delta (C2-2), the XP-bar UI (C2-3), and the 6→8 slot raise (C2-4) are LATER slices. Everything dark; no client code.

Work done

- **Migration** 20260618000177_captain_xp_foundation.sql:

- `captain_instances` gains additive xp numeric not null default 0 + level integer not null default 1 — **read by NOTHING yet** (no adapter/RPC/client touches them until C2-2/C2-3); the mint (0118) and assignment (0119) writers ride the defaults and never touch either column.
- **THE SHIP-LINKAGE FINDING** (`reward_grants` carries NO ship column — the linkage is per-source via `source_id`: **combat** = encounter id → `combat_encounters.fleet_id` → the frozen `group_sortie_members` manifest (0168 — the charter's "sorties whose manifest included the captain's ship") ∪ the `fleets.main_ship_id` tag (0050) for solo sorties; **exploration** = `exploration_discoveries.main_ship_id` (0146/0172 securing link; nullable — legacy null-scanner rows underivable); **mining** = `mining_extractions.main_ship_id` (nullable — 0103 ON DELETE SET NULL; a destroyed-ship extraction consumes as a sentinel; review M1 corrected the earlier NOT-NULL claim); **trade** = NO `reward_grants` producer exists today (trade pays through Wallet) — if one appears its grants consume as sentinels until a linkage+knob slice defines them. Grants with no derivable ship (legacy unit fleets, retention-cleaned encounters) are consumed as SENTINELS, never credited.
- `captain_counted_grants` — the 0144 consumption-ledger idiom, captain-shaped: keyed unique nulls not distinct (`grant_id`, `captain_instance_id`) (one grant can feed MULTIPLE captains on one manifest — each credit its own exactly-once row) with a **NULL-captain SENTINEL** marking "consumed, no credit". The accrual anti-joins per GRANT (no ledger row at all), so every grant is examined EXACTLY ONCE ever: bounded scans, and **no retroactive XP backfill** — a grant's captain set is fixed the first time the accrual sees it. Server-only (RLS on, no client policy/grant — the 0144 posture).
- `captain_xp_accrue()` (SECURITY DEFINER, service-role-only): gate-FIRST → {ok:false, code:'feature_disabled'} while dark (a cron-safe no-op, NEVER a raise — the D2/0145 lesson) → global advisory lock → ONE commit-safe statement (the 0145 data-modifying-CTE shape): unconsumed grants → derivable ships → captains **currently assigned** (`ship_captain_assignments`) → ledger insert (credits + sentinels, on conflict do nothing) → per-captain Σ xp folded into `captain_instances.xp` with `level = 1 + floor(sqrt(xp/100))` maintained inline (coherent from day one; the C2-2 adapter ignores it until its own slice).
- **THE DESIGN-HONESTY NOTE (current-assignment vs at-sortie)**: XP goes to captains assigned AT ACCRUAL TIME, not at sortie time — the manifest freezes at-send SHIP membership, but **captain-at-sortie-time is recorded nowhere** (nothing snapshots the roster at send), so current-assignment is the only derivable semantic. Practical skew ≈ one 5-min cron window; a D-family manifest extension (per-member captain snapshots written by `send_ship_group_hunt`) would enable true at-sortie attribution — noted as a future refinement, deliberately not built in a dark XP slice. Corollary documented for the future ACT-CAPXP flip: dark-era grants are all unconsumed, so the FIRST lit run folds that whole backlog into current assignees — the activation script must accept the one-time backfill or pre-seed sentinels (flip-time decision).
- **XP formula [D, owner-tunable]**: flat XP per grant per source — a grant IS the finalized unit of "a sortie that came home with something" (UNIQUE (`source_type`, `source_id`)); knobs `captain_xp_per_combat_grant=10`, `captain_xp_per_exploration_grant=6`, `captain_xp_per_mining_grant=4`; every assigned captain on a linked ship gets the FULL amount (no split). Curve [D proposed]: `level = 1 + floor(sqrt(xp/100))` — 100→2, 400→3, 900→4.
- **Cron** `captain-xp-accrue`, **every 5 min** (the 0147 idiom + cadence rationale verbatim: freshness only, never correctness — the ledger anti-join folds any backlog next firing). Scheduled NOW, zero live effect while dark.
- Self-asserts: columns exist NOT NULL with defaults 0/1 and every instance rides them; ledger exists server-only; accrual exists service-role-only; cron scheduled exactly once; flag dark + knobs 10/6/4; and the cron-safety pin — a dark dry-run returns the no-op envelope, zero ledger rows, zero xp movement.
- **Proof** — `TEAMCMD_PASS_CAPXP` **block appended to** `scripts/team-command-proof.{sql,sh}` (captain/team-family; consumes the TEAMSETTLE fixture AS the captained team sortie — the settled uC sortie's grant, manifest {c1,c2}, with the TEAMHUNT roster still assigned: 2 captains on c1, 1 on c2, 1 on the grantless b1): committed seeds pinned (flag 'false', knob '10'); additive defaults on every instance; DARK accrue = clean no-op envelope with ZERO writes **while grants already exist** (a gate-after-read regression would fold them); flag on in-txn + knob raised to 100 via the real `set_game_config` (100 = the exact level-2 boundary) → ONE run credits the 3 currently-assigned manifest captains EXACTLY knob × 1 qualifying grant each, level lands exactly 2 at the boundary; a captain on a grantless ship and a freshly-minted UNASSIGNED captain gain NOTHING; an ORPHAN grant (minted via the REAL `reward_grant` sole writer with a random `source_id` — the retention-cleaned-encounter shape) consumes as a NULL-captain sentinel; zero grants left unconsumed; the curve recomputed independently over every instance; RE-RUN = all-zero envelope, ledger unchanged, total xp unchanged (the anti-join exactly-once pin). The `.sh` gains the marker, `captain_growth_enabled` in the flags-inside-txn list, 16 assert-form selftest greps, `captain_counted_grants` in the Captain-table negative grep (sole-writer law), and post-run committed-value honesty checks for the flag AND the xp knob (must stay 'false'/'10').
 - `SYSTEM_BOUNDARIES.md` — §1 row for `captain_counted_grants` (sole writer = the accrual; server-only) + the `captain_instances` row now states the disjoint-by-column writer split (mint owns row creation, `captain_xp_accrue` owns xp/level — mint/assign never touch them) + the §2 Captain row gains the accrual, its downward edges (Reward/Combat/Movement/Team-Command/ Exploration/Mining reads), and the forbidden edges (no XP writes outside the accrual; Combat never writes captain XP mid-tick).

- **Docs** — plan queue row 13 + ROADMAP phase row 24 → shipped (dark), both carrying the current-assignment note and the ACT-CAPXP dark-backlog decision.

Verification. `bash scripts/team-command-proof.sh selftest` green (the hardened lib); **mutation-tested 7/7 caught** (flag-flip dropped, boundary assert gutted, PASS marker dropped, direct ledger insert (sole-writer law), curve recompute gutted, re-run pin gutted, sentinel pin gutted), control green. All six trade-family selftests green (untouched consumers of the shared lib). `tsc/vite build` untouched-green (no client code). Real-chain run = the `team-command-proof.yml` disposable matrix on push (`slice-**` trigger; this machine has no Docker/psql — the established loop).

2026-07-12 — HAUL-0/1 (queue #12): delivery-contracts foundation (dark)

Request. Queue slice #12 of the full-capacity plan (\$C P2): the retention loop's foundation — HAUL-0 (`haul_contracts` schema + migration-seeded templates + `haul_contracts_enabled` dark flag) + HAUL-1 (the cron-driven, seeded-deterministic offer generator). The accept/deliver RPCs (HAUL-2) and the bulletin UI (HAUL-3) are LATER slices. Everything dark; no client code.

Work done

- **Migration** `20260618000176_haul_contracts_foundation.sql`:
`<font name="Sym"> </font> haul_contract_templates` — Reference/Config (migration-seeded ONLY, public read-only): 10 v1 templates over the 0173 three-port economy carrying origin/dest (fixed uuid or NULL='any'), good, qty range, reward params, duration (6–12h), and generation weight. **Reward math [D, owner-tunable]:** $\text{reward} = \text{round}(\text{reward_base} + \text{qty} \times (\text{LIVE dest market_offers.buy_price} + \text{reward_premium_per_unit}))$ — beats the best same-haul self-trade ($\text{qty} \times (\text{dest.buy} - \text{origin.sell})$) by exactly $\text{base} + \text{qty} \times \text{premium} > 0$, always, and MODESTLY (premiums $\approx 15\text{--}25\%$ of the route's per-unit market profit; the full 10-row table with profit ranges lives in the 0176 header). Two Drift-origin BACKHAULS ride market-dead 0173 routes (Drift is a pure importer) — the port pays above market to move goods raw arbitrage won't; smallest payouts on the board. Every `qty_max` fits the 50 m³ starter hold (self-asserted). Scale: a full day's board = 6 contracts $\approx 300\text{--}900$ profit — a retention garnish beside the repeatable +200/trip guaranteed routes, not a dominant faucet.
`<font name="Sym"> </font> haul_contracts` — the live offers: status domain offered/accepted/delivered/expired/cancelled, natural key unique (`origin_location_id`, `offer_day`, `slot`), HAUL-2 columns (`accepted_by`/ `accepted_ship`/ `accepted_at`/ `delivered_at`) ready but unwritten. RLS: 'offered' rows PUBLIC (per-port bulletins, no player data); accepted rows OWNER-only (`accepted_by = auth.uid()`); no client write path. Sole writer today = the generator; the only future writers = HAUL-2.
`<font name="Sym"> </font> haul_generate_offers()` (SECURITY DEFINER, service-role-only): gate-FIRST → `{ok:false, code:'feature_disabled'}` while dark — a cron-safe no-op, NEVER a raise (the D2/0145 lesson). **Determinism = the 0041 technique** (pure function of inputs, no session RNG), extended with a pure hash: `hashtextextended('haul:'||day||':'||port||':'||slot, 0)` → uniform → weighted template pick (cumulative weights in `template_id` order); a second salt (`haulqty`;) → quantity. Same (day, port, slot) ⇒ byte-identical offer; idempotency is structural (the natural key + `on conflict do nothing` — re-runs and racing double-fires mint nothing). Generates N per active port per day (N = new knob `haul_offers_per_port`, seeded 2; active = status active + docking active + an active market — so ECON-SEED feeds both the pricing and the port filter). The expiry pass flips 'offered' → 'expired' past `expires_at`; the predicate is `status='offered'` ONLY — **accepted contracts are never touched** (an accepted contract's delivery deadline is HAUL-2's business; design decision, proof-pinned).
`<font name="Sym"> </font> Cron haul-generate-offers, hourly ('7 * * * *')` — cadence decision: offers are daily-deterministic (generation does real work once per day; every other firing is an idempotent no-op), but offer durations are 6–12h (sub-day), so the hourly firing is what keeps the bulletin honest — expiry latency $\leq 1\text{h}$; daily would strand dead offers for hours, faster than hourly buys nothing. The 0033/0147 idiom verbatim; minute 7 avoids the top-of-hour herd. Scheduled NOW with zero live effect while dark.
`<font name="Sym"> </font> Self-asserts:` exactly the 10 approved templates; per-port pools non-empty; starter-haulable; worth-taking recomputed from the LIVE market rows at BOTH qty endpoints; generator exists + service-role-only ACL; cron scheduled exactly once; flag dark + knob 2; and the cron-safety pin — a dark dry-run of the generator returns the no-op envelope and leaves zero rows.
- **Proof** `scripts/haul-proof.{sql,sh}` (trade-family, lib-based, one BEGIN..ROLLBACK; wired as the 6th selftest + matrix step in `trade-v1-proof.yml`, trigger `slice-haul**`): P0 dark cron-safe no-op (envelope, zero rows, no raise) → flag on in-txn → P1 exactly $N \times 3$ offers, all 'offered', slots 1..N per port, template bounds + LIVE reward recompute + expiry anchor exact → P2 worth-taking (contract profit beats the recomputed same-haul self-trade AND is absolute-positive — covers the backhauls) → P3 determinism (same-day re-run creates 0 rows, offer-set signature unchanged; the Haven slot-1 identity RE-DERIVED from the raw hash technique, not by calling the generator) → P4 expiry (a fixture-aged offered row flips 'expired'; a past-deadline 'accepted' fixture is NEVER touched; zero new mints) → P5 RLS/ACL shape (2 SELECT-only policies pinned by name+qual, templates public-read, zero client write grants, generator ACL, cron exactly once). The harness never INSERTs into either haul table (all rows come from the real generator; two marked fixture UPDATES only) — selftest-pinned.

- **SYSTEM_BOUNDARIES.md** — §1 rows for both tables (owners, sole writers, read surfaces) + a §2 Haul Contracts system row (the charter guard: Contracts owns ONLY its table (+ future receipts); cargo moves only through Trade-Cargo, credits only through Wallet — HAUL-0/1 writes neither). ROADMAP phase row 22 + plan queue row 12 → shipped (dark).

Hostile-review fixes (applied same slice; cron-safety/determinism/economics/schema reviewed CLEAN):

- **M1 (shared lib scripts/lib/trade-proof-lib.sh — hardens ALL SEVEN lib consumers):** a flag toggle appended AFTER a proof's final rollback; AUTOCOMMITS in psql and previously SURVIVED the selftests — `tp_assert_flags_inside_txn` checked only the FIRST occurrence per flag and `tp_assert_self_rolling_back` only that the last txn VERB was rollback. Fixed both: EVERY occurrence of each flag toggle must now sit strictly inside `begin;..rollback;` (loop over all `grep -n` matches), and NOTHING but comments/whitespace may follow the final ROLLBACK (on its own line or any later one — comment-stripped check). Reconciliation: all seven lib-sourcing selftests (six trade-family + team-command) re-run GREEN unchanged — no existing proof carried trailing SQL or an out-of-txn toggle, so no proof needed adjustment.
- **L1 (0176 GUC-stable determinism):** the hash salts rendered the day via `format('%s', date)` (DateStyle-dependent) and the day boundary used `now()` (TimeZone-dependent). Now: the day is the UTC calendar day (`now() at time zone 'utc'::date`) and enters both salts via `to_char(day, 'YYYY-MM-DD')` — no session GUC can shift an offer's identity between cron, psql, and the proof. The proof's P1/P3/P4 day derivations + the P3 inline re-derivation updated in exact lockstep; new selftest greps pin the `to_char` rendering + the UTC boundary.
- **L3 (0176 templates):** added check (`reward_base + reward_premium_per_unit > 0`) — a future 0/0 template would make the worth-taking invariant a tie and could round a sub-0.5 reward to 0 → a lit-cron insert raise on `reward_credits > 0`.
- **Forward notes (0176 header + here):** (1) PRICE DRIFT — when `world_balance_enabled` lights, the P19 drift multiplier composes into the player's ACTUAL buy/sell prices at read time but NOT into contract rewards (priced at generation from the static `market_offers` rows); HAUL-2/HAUL-3 must surface the EFFECTIVE profit, not the static header math. (2) EMERGENCY DARKENING — a lit→dark flip freezes the expiry pass too (gate-first returns before it), so stale 'offered' rows persist publicly readable; the eventual ACT-HAUL rollback section must run one manual expiry pass (the generator's expiry UPDATE, as service role) or explicitly accept the frozen bulletin.

Verification. `bash scripts/haul-proof.sh selftest green`; **mutation-tested 14/14 caught** (commit-instead-of-rollback, flag-enable dropped, direct `haul_contracts` insert, `feature_disabled` envelope gutted, hash re-derivation removed, self-trade recompute gutted, accepted-row pin dropped, PASS marker dropped, `pg_policies` asserts dropped, migrations-path reference; post-lib-fix: the M1 survivor post-rollback flag toggle, post-rollback arbitrary SQL, same-line post-rollback SQL, and the `to_char` GUC pin gutted — the survivor also re-tested CAUGHT on salvage, proving the lib-wide fix), controls green. All seven lib-consumer selftests green (six trade-family + team-command). `tsc/vite` build untouched-green (no client code). Real-chain run = the `trade-v1-proof.yml` disposable matrix on push (slice-haul trigger).

Bugs / fixes

- `_(none)_`

2026-07-12 — RANK-SEASON + ACT-RANKING (queue #9): the ranking activation script

Request. Queue slice #9 of the full-capacity plan (§B rung 5): `scripts/activate-ranking.{sql,sh}` in the proven `activate-*` idiom. NO migration — the ranking stack is fully built dark (0127–0131 schema + `ranking_season_open` + accrual + read surface; 0144/0145 the commit-safe `ranking_counted_grants` ledger fold; 0147 the 5-min ranking-accrue-standings cron, already scheduled as a dark no-op; RankingPanel server-lit). The script IS the flip: `seasons + ranking_enabled → true`.

Work done

- `scripts/activate-ranking.sql` — one all-or-nothing `BEGIN..COMMIT` (txn-local **UTC** — the window anchors are UTC-defined), zero psql meta-commands. `PRECONDITIONS` (read-only): `head ≥ 0147` + all 8 ranking migrations recorded; the 3 tables + both season-invariant indexes; the whole function surface via `to_regprocedure` with the REAL signatures — `ranking_season_open(text, timestamptz, timestamptz, text)` (explicit bounds + label; it does NOT compute windows), `ranking_accrue_standings()`, `ranking_score_delta(jsonb)`, and the two client RPCs `rankingApi.ts` actually calls, `get_ranking_seasons()` + `get_ranking_leaderboard(uuid, text, int)` (there is NO `get_my_rankings/get_my_standing` — own standing is client-derived by design); the DEPLOYED accrual body prosrc-pinned TWICE to the 0145 commit-safe ledger fold (`ranking_counted_grants` + the `on conflict (season_id, grant_id) do nothing` exactly-once guard — the stale-0130 timestamp-cursor body carries neither); the cron scheduled EXACTLY once (jobname `ranking-accrue-standings`, command invoking the accrual); the `ranking_enabled` key exists (value NOT asserted — re-runs are supported); `reward_grants` readable with per-dimension counts **FYI only, never a gate** (ranking lights before grants flow; trade has no depositor yet — 0128).
- **STAGE ORDER FINDING — flag BEFORE seasons (inverts the task sketch, forced by the code):** `ranking_season_open` dark-gates on `cfg_bool('ranking_enabled')` FIRST (0129:71) and answers `feature_disabled` while false — a season physically cannot be opened dark. So STAGE 1 = the one `set_game_config('ranking_enabled', 'true')` write; STAGE 2 = the seasons; both in the ONE txn so a season failure rolls the flag back too (never a lit flag without seasons).

- **Season windows [D proposed, owner-tunable], computed from now() at run time:** weekly = ISO-Monday-anchored `date_trunc('week', now()) .. +7 days`, label `IYYY-"W"IW` (e.g. 2026-W28); monthly = calendar month .. +1 month, label `YYYY-MM` (e.g. 2026-07). Future rolls must reuse the convention so windows tile and the (cadence, starts_at) replay-idempotency (0129) holds.
- **RE-RUN SEMANTICS (decided): no-op success, and a later re-run IS the manual roll.** Each cadence is created ONLY if no CURRENT active season exists (active AND window containing now()): same-window re-run skips with a notice; a later-window re-run proceeds and `ranking_season_open` closes the expired active season while opening the new window. Envelopes asserted `ok + status='active'` — the edge where a CLOSED identical window replays verbatim (0129 never reactivates) fails LOUDLY instead of committing a lit flag with a dead board.
- **AUTO-ROLL FINDING: none exists.** The sole `ranking_seasons` writer is `ranking_season_open` (closes the prior active only when opening the next); the only ranking cron is the 5-min ACCRUAL. A season stays active past `ends_at` and the accrual's `granted_at` between `starts_at` and `ends_at` join simply stops folding — the board silently FREEZES. Season rolling is MANUAL for now (Monday 00:00 UTC / 1st of month; a script re-run is the roll) — noted as an operational item + a future **RANK-ROLL** automation slice (safe: the opener is idempotent per (cadence, starts_at)). **Edges the roller must close (review M1/L1): fold recently-closed seasons for a grace tick (the status=active join drops post-roll late commits into NEITHER season), and make the window join half-open (BETWEEN double-counts an exact-boundary grant into both).**
- **SMOKE (read-only):** flag committed (`raw + cfg_bool`); exactly one active season per cadence with a window containing now(); cron intact exactly once; standings + counted-grants selectable (0 rows FYI); the CLIENT surface answers lit — `get_ranking_seasons()` {ok, ≥2} and `get_ranking_leaderboard(weekly, 'overall')` {ok} (empty rows = the honest "No standings yet" state). Markers `ACTIVATE_RANKING_PASS_*` + final `PASS` line.
- **MOUNT VERIFIED — NO client PR:** `RankingPanel` is already mounted on the post-R3 `CommandScreen` aside rail (`CommandScreen.tsx:83`, inside `screenRailClass('aside')`), gated ONLY by `isServerLit(seasons) && litSeasons.length > 0` (`RankingPanel.tsx:87`); no `RANKING_*` compile constant exists in `osnReleaseGates.ts` or anywhere in `src` (grep-verified). At flip time players see the `Leaderboard` card with both seasons and "No standings yet"; boards fill as the 5-min cron folds finalized `reward_grants` (combat/exploration/mining deposit as those systems light; the trade dimension stays 0 until a trade activity deposits grants).
- **ROLLBACK (commented, flag-only — recommended and reasoned):** `ranking_enabled` → false; standings freeze intact (every ranking RPC + the cron reject-before-read). Seasons are deliberately LEFT ACTIVE: closing is effectively one-way (the 0129 replay returns a closed row verbatim, never reactivates), so a re-light inside the same window would fold nothing; active-but-dark seasons are unreachable and the ledger anti-join BACKFILLS in-window grants on re-light — no gap, no double-count.
- `scripts/activate-ranking.sh` — selftest/run wrapper (confirm token `ACTIVATE_RANKING`). The DB-free selftest asserts: exactly ONE `set_game_config` invocation (→ true on `ranking_enabled`; the signature string in the preconditions is counted as existence, not a call site), never another window's key, NO direct table DML (seasons only via `ranking_season_open` — both cadence calls present, conditional, envelope-asserted), no DDL, no meta-commands, one timed UTC `BEGIN..COMMIT`, the 0145 prosrc pins + cron jobname pin, markers/`PASS/rollback-commented`, and the flag-first/no-auto-roll/mount documentation tokens.

Verification. `bash scripts/activate-ranking.sh selftest` green; **mutation-tested 10/10 caught** (sneaked second flag write, uncommented rollback, dropped weekly season call, gutted conditional guard, direct table `UPDATE`, `TRUNCATE` DDL, `psql` meta-command, dropped 0145 prosrc pin, retargeted cron jobname, dropped status-active envelope assert), control green. All 6 existing activate/reveal selftests untouched-green (trade, captains, exploration, mining, team-command, ember-reach); `tsc --noEmit + vite build` untouched-green (no src change). Docs: plan queue row 9 → script shipped; ROADMAP phase-17 row annotated. Nothing committed/pushed/applied — the flip stays human-gated (plan §B rung 5 recommends flipping after rungs 1–3 so ≥3 dimensions accrue).

2026-07-12 — ZONES2-1+2 (queue #7+#8): "Ember Reach" content expansion — seeded HIDDEN + the reveal script

Request. Queue slices #7+#8 of the full-capacity plan (§C P4, the packet §1.3 Option-C seeds): the first content expansion — new higher-difficulty hunt zone "Ember Reach", seeded `status='hidden'`, plus the reveal operation script (reveal IS the cadence mechanism). PURE DATA slice: zero engine edits, zero flag flips; nothing player-visible until the human runs the reveal.

Work done

- **Migration** `20260618000175_zones2_ember_reach.sql` — additive, idempotent (fixed literal UUIDs embedding 0175, the 0066 identity idiom; on conflict (id) do nothing, any other unique-key collision aborts). Seeds: sector **Ashen Frontier** (index 3, danger_tier 3, ACTIVE) + zone **Ember Reach** (ACTIVE) + three HIDDEN `pirate_hunt` sites — **Ember Gate** (100,90 · bd 40 · tier 4 · min_power 150), **Cinder Maw** (125,110 · bd 50 · tier 4 · 220), **The Furnace** (150,130 · bd 60 · tier 5 · 300). Containers are ACTIVE because `get_world_map` filters all THREE levels on `status='active'` (0002:121/125/129) and the reveal flips only the location rows; an active zone with zero active locations renders nothing (the client flattens locations only — verified). Gates

are packet-derived [D owner-tunable]: 38 combat_power per kitted+captained ship (\$0.3) → 150/220/300 ≈ **4/6/8 ships** (≈3/5/6 after the 6-captain bump). Coordinates continue the 0154 distance-orders-by-difficulty ray NE beyond Blackden (85.1 → 134.5/166.5/198.5 from home), outside the legacy bbox (x -50...70, y -30...80), ≥16%-of-span marker separation (0154's ~9% threshold), deep inside the OSN envelope. Deliberate omissions, each justified in the header: no safe waypoint (keeps the reveal exactly-3), no anchors/services (not dockable), no location_state seed (worldstate lazily upserts, 0032:37-41); physical_role='activity_site' (first honest user of the 0065 value; excluded from every port predicate). Self-assert DO block: exact 3-row read-back (identity/stats/HIDDEN), monotonic gates+distance, game-wide name uniqueness, coordinates in-envelope/outside-bbox, and the **hidden-invisibility pin BOTH ways** — structural (get_world_map prosrc carries the three status = 'active' filters) AND behavioral (the migration calls get_world_map() — a STABLE table-reading SQL function — and asserts none of the 3 names/ids appear).

- **reward_tier finding (verified against the TRUE combat head 0169:354):** reward_tier multiplies the METAL reward per cleared wave — `round(10 × greatest(tier,1) × (1+0.25×danger))` → tier 4/5 pays 4×/5× tier-1 metal. ITEM loot (0041 pirate_loot_for_wave) keys on WAVE NUMBER only — tier has NO loot-table meaning; drops keep riding wave/danger progression as today (defining tier-keyed loot would be an engine edit, out of charter). Documented in the migration header.
- `scripts/reveal-ember-reach.{sql,sh}` — the reveal operation (activate-family wrapper pattern ADAPTED to a reveal; reveal-starter-ports txn semantics). One timed BEGIN..COMMIT, zero psql meta-commands. PRECONDITIONS: 0175 recorded by version; the 3 canonical rows exist ALL hidden (a rerun fails closed — already-revealed is an error); exact seeded identity (refuses drifted content); monotonic gates; parents ACTIVE; get_world_map three-level filter prosrc-pinned + a behavioral no-leak pre-check. STAGE — **the ONLY write:** ONE update `public.locations set status='active'` scoped to the 3 fixed uuids, `row_count=3` asserted. SMOKE: 3 active; net active-locations exactly +3; every non-canonical location status-invariant (the offsetting-proof digest); **game_config digest unchanged** (a content reveal must never move a flag); behavioral post-check (all 3 names now in the map read). Commented manual ROLLBACK: re-hiding IS supported for hunt sites (unlike the port reveal) — sends validate status at SEND time only (0019:39/0168:196), settle/combat/return key on location_id, so in-flight/in-combat fleets resolve normally and nothing strands; documented in the script. Selftest asserts the write-shape inversion (exactly ONE locations UPDATE, zero set_game_config, no insert/delete/DDI, rollback stays commented) — **mutation-tested: 8/8 mutations caught** (sneaked set_game_config, second-table update, uncommented rollback, retargeted uuid, DDL, dropped 0175 gate, dropped prosrc pin, sneaked insert), control green.
- **No client change needed:** the galaxy map is data-driven (markerStyle/labelTier derive from the location row; useGalaxyMapData flattens whatever get_world_map returns; camera content-fit computes its own bbox; grep: nothing keys on location count/sectors — one test fixture uses a synthetic Snare).

Verification. New selftest green + mutation-tested (8/8); all existing selftests untouched-green

(activate-team-command/captains/exploration/mining/trade, reveal-starter-ports, team-command-proof, postreveal-verify); `tsc -b + vite build` untouched-green (zero client changes in this slice). The migration self-asserts at apply time (fail-closed; aborts the deploy on any drift).

Follow-up (human-gated). Deploy 0175, then — deliberately later, once teams have kitted up (the gates assume ≈4/6/8 kitted ships; modules/captains need to be flowing first) — run `bash scripts/reveal-ember-reach.sh run REVEAL_EMBER_REACH`. Rows 7+8 marked shipped in the plan queue; ROADMAP phase 23 row added.

2026-07-12 — SALVAGE-0/1 (queue #6): the combat-loot → port-economy feed (dark)

Request. Queue slice #6 of the full-capacity plan (plan \$C P3): ports gain item buy-lists and the first item→credits path (closes gap G3). One PR = SALVAGE-0 (schema + seed + flag) + SALVAGE-1 (the sell RPC) + proof; the UI (SALVAGE-2) and the flip (ACT-SALVAGE) are later slices. Everything DARK behind the new `salvage_market_enabled` flag, seeded `false`.

Work done

- **Migration** `20260618000174_salvage_market.sql` — `port_item_demand (location_id, item_id, unit_price > 0, active, PK (location_id, item_id))`: Reference/Config posture, MIGRATION-SEEDED ONLY, no runtime writer (the market_offers 0085/0173 posture); buy-side only, so no pump loop can exist by construction. Seeded 3 ports × the FIVE items that actually drop from combat (grep-verified against `pirate_loot_for_wave` — head 0041, 0171 the only re-create): scrap 5/8/6, pirate_alloy 10/16/12, repair_parts 20/12/16, engine_parts 16/14/24, weapon_parts 15/13/22 (Haven/Slagworks/Driftmarch — Slagworks the industrial recycler tops scrap+alloy, Haven the city tops repair_parts, Driftmarch the frontier tops engine+weapon parts; all **[D] owner-tunable**, math in the migration header: a typical 3-wave Snare run = 3 scrap + 1 alloy → 40 credits at the best port, mid of the 30–80 target; module ingredient baskets sell for ≤168 — selling is the floor, crafting the ceiling). Progression items (`captain_memory_shard`, `blueprint_fragment`, `artifact_core`) are NEVER sellable — excluded BY OMISSION, pinned by the migration self-assert (by id AND by 0039 category) and the proof. `salvage_receipts` = the 0086 `trade_receipts` idempotency shape point-for-point (unique (main_ship_id, request_id), replay-verbatim) as a NEW table — `trade_receipts.good_id` FKs `trade_goods`, structurally trade-goods-specific, so item sales cannot reuse it without weakening a live FK. `sell_item_at_port(ship, item, qty numeric, req)` — the 0090/0138 sell shape on items: SECURITY DEFINER +

search_path, atomic under the per-ship lock; reject order not_authenticated → salvage_market_disabled (gate FIRST) → invalid_request/invalid_item/invalid_quantity (items are INTEGER quantities — fractional rejects, never rounds) → ship_not_found → not_docked (the ONE resolver mainship_resolve_docked_location) → no_demand → idempotent_replay → insufficient_items (pre-check inventory_get_balance; enforcement = inventory_spend's own FOR UPDATE — the 0109 crafting posture) → ok: inventory_spend + wallet_credit(qty × unit_price) + receipt, all in-function. ACL: revoke public/anon, grant authenticated. Pipeline law unchanged: SALVAGE consumes inventory exactly like crafting; combat never grants credits directly.

- **Proof** scripts/salvage-market-proof.{sql,sh} (trade-family — the trade-econ-seed lib-based pattern; one BEGIN..ROLLBACK, selftest + local): dark reject with ZERO writes; flag on in-txn only → full happy path (players via commission_first_main_ship, items via the REAL pipeline leaf public.reward_grant — never a direct player_inventory insert; sell 3 repair_parts at Haven @20 → wallet +60 EXACT, inventory 4→1 EXACT, one receipt with exact fields); idempotent replay (no double credit/spend/receipt); guards (invalid_quantity 0/-3/2.5, in-transit not_docked, crystal no_demand, over-held insufficient_items — all zero-write); the exact 15-row price table pinned by full-outer-join; the never-sellable pin (progression absence by id + category, AND a HELD shard still → no_demand). Wired as the 5th selftest + matrix step of .github/workflows/trade-v1-proof.yml; slice-salvage** added to its branch triggers.
- **Docs:** SYSTEM_BOUNDARIES rows for port_item_demand (Reference/Config) + salvage_receipts (Salvage Market, sole writer sell_item_at_port) + the Salvage Market system contract (\$E law, same PR); FULL_CAPACITY_PLAN queue #6 → shipped (dark); ROADMAP phase row 21 (P3 economy wave).
- **No client code** in this slice; tsc + vite untouched.

2026-07-12 — ACT-TRADE (queue #5): the trade-market activation script + the proof chain finally green

Request. Queue slice #5 of the full-capacity plan (plan \$B rung 3): the human-run flip tool for the trade market — scripts/activate-trade.{sql,sh} in the activate-team-command / activate-captains / activate-exploration idiom. NO migration in this slice; NO flag flips at merge time — the script is the recorded human go mechanism, run deliberately later.

Work done

- scripts/activate-trade.sql — one all-or-nothing BEGIN..COMMIT, zero psql meta-commands (management-API-runner compatible). PRECONDITIONS (read-only; any failure rolls everything back): migration head ≥ 20260618000173 AND 0173 recorded by version (head alone proves nothing — 0171/0172 are lower numbers); each starter port carries exactly 6 ACTIVE offers over the six 0073 goods (18 rows); the anti-pump spread recomputed over every row; the 3 flagship routes re-derived profitable from the LIVE rows (ore Slagworks→Haven, provisions Haven→Slagworks, machinery Slagworks→Driftmarch — the econ-seed proof's independent-recompute style, cheap form); the DEPLOYED get_market_offers / market_buy / market_sell bodies prosrc-pinned to the **0138 re-creates** via two positive tokens — public.mainship_resolve_docked_location(v_ship) (absent from the 0136 stale bodies, comments included) AND trade_effective_price((absent from the pre-drift 0092 bodies) — so the flip physically cannot run against a regressed body; relief knobs sane and READ-ONLY (relief_credits 250 within 0 < x ≤ starting_credits — relief must never out-pay the wallet seed, the 0095 farming-hole reasoning; relief_max_lifetime_claims 3 in 1..10; relief_cooldown_seconds 86400 ≥ 3600); starting_credits and main_ship_price present (price already 250 on prod runtime since the team activation — never rewritten here). STAGE 1 (the ONLY writes): trade_market_enabled → true AND trade_relief_enabled → true — they light TOGETHER (a lit market with dark relief reopens the softlock the floor exists to close). SMOKE (read-only): both flags committed (raw + cfg_bool); to_regprocedure over the 4 client RPCs (get_market_offers(uuid), market_buy(uuid,text,numeric,uuid), market_sell(uuid,text,numeric,uuid), market_claim_relief(uuid)) + the 8 internal leaves (resolve-owned/docked, trade_effective_price, cargo add/consume, wallet ensure/debit/credit); exactly 18 active offers across exactly 3 ports; the anti-pump CHECK constraint pinned by definition text (0085 left it unnamed); ship_cargo_lots selectable. Markers ACTIVATE_TRADE_PASS_{PRECONDITIONS,STAGE1,SMOKE} + a final PASS line naming the remaining step. Commented flag-only ROLLBACK (wallets/cargo/receipts/relief claims persist inert; the anti-pump constraint makes single-station pumping structurally unprofitable while lit).
- scripts/activate-trade.sh — selftest/run wrapper (confirm token ACTIVATE_TRADE). The DB-free selftest pins: exactly 2 set_game_config call sites, both → true, exactly the two approved keys; no knob rewrite (relief knobs / starting_credits / main_ship_price / caps), no other-window flag, no direct table write, no DDL, no meta-command; one timed txn; rollback stays commented; the 0173 gate, the 0138 prosrc pins, the route recomputes, the full smoke list, and the client-PR mount docs all present. **Mutation-tested:** flipping a write to 'false' fails the selftest; injecting a direct update game_config set ... (qualified or not, with or without WHERE) fails the selftest.
- **Client-mount verification (the captains-slice lesson — claim only what mounts).** Repo-verified: the one-line client PR flipping TRADE_MARKET_ENABLED mounts exactly (a) MarketPanel on the Port screen main rail (PortScreen.tsx:61) — the ONLY newly visible surface — and (b) completes the ShipSwitcher OR-gate (ShipScreen.tsx:62, TRADE_MARKET_ENABLED || MAINSHIP_ADDITIONAL_ENABLED) — the switcher is ALREADY mounted because MAINSHIP_ADDITIONAL_ENABLED has been true

since the 2026-07-12 team launch, so it shows no new change. No other consumer of the constant exists (tradeApi/MarketPanel/ShipSwitcher reference it in comments only). The panel's wallet/cargo reads are owner-read table reads (player_wallet, ship_cargoLots ↯ trade_goods), not RPCs; prices ride get_market_offers.

- **Docs:** FULL_CAPACITY_PLAN queue row 5 → script shipped (file brought over from the unmerged docs-full-capacity-plan branch — this slice carries it if that branch has not merged first); ROADMAP phase-10 row annotated.

Trade-proof-chain archaeology (this branch, earlier this wave). CI history showed trade-v1-proof.yml had NEVER been green — every historical run failed identically, masked by path-filtered triggers. Four rot layers, all fixed on this branch before the activation script landed:

1. **Stale OC fixtures (wallet)** — the TRADE-FLEET-OC proof was authored at chain 0084 (pre-wallet) and went stale the day 0091 (ships debit main_ship_price 1000) + 0093 (lazy wallet seed 1000) landed: the 3rd commissioned ship always died insufficient_credits. Fixed by funding the proof wallet in-txn (rolled back), citing 0091/0093 — the pricing itself is the designed credit sink.
2. **Cap pin** — 0160 raised max_main_ships_per_player 3→24; the proof's cap-enforcement stage was about to go latently stale, so the cap is now pinned back to 3 transient-in-txn (the property under test is enforcement, not the prod value).
3. **PROP2 non-deterministic ship picker** — all in-proof ships share transaction-frozen now() as created_at, so order-by-created_at asc/desc was plan-dependent and 0160's group_idx shifted the plan until both picks returned the SAME row; now ordered by main_ship_id (unique PK).
4. **Latent k-variable ambiguity** — plpgsql variables named k collided with the temp tables' k column (variable_conflict=error) in blocks that had never executed because the OC step always failed first; renamed to sk (the team-command-proof convention). After the fixes: all five proof selftests (OC, TM1, bootstrap, econ-seed, team-command) green locally; the real-chain run happens in CI's disposable Supabase.

Explicitly NOT in this slice: running the flip (human-gated, owner-run), the one-line client PR (TRADE_MARKET_ENABLED → true — ships only AFTER a green prod run of the script), any migration, any knob retune.

Verification. bash scripts/activate-trade.sh selftest green + both mutation tests fail as designed; the four trade proof selftests + team-command selftest untouched-green; npx tsc -b + npx vite build untouched-green (no client code changed).

2026-07-12 — ECON-SEED-1: differentiated three-port economy seed (migration 0173, DARK)

Request. Queue slice #4 of the full-capacity plan: seed the owner-approved multiport price table (master plan \$C P1) so the trade-market activation (ACT-TRADE, queue #5) lights a real buy-low/travel/sell-high economy instead of the single-port Haven placeholder. Zero live impact — trade stays dark (trade_market_enabled=false); no flag flip, no client change, no activation script.

Work done

- **Migration** 20260618000173_econ_seed_multiport_offers.sql (numbered 0173 to stay clear of in-flight 0171/0172; depends only on 0066 ports + 0073 goods + 0085 market_offers): idempotent upsert (on conflict (location_id, good_id) do update) of all 18 rows — 3 starter ports × 6 goods. Port roles: **Haven** (city/consumer — pays well for ore 16 + provisions 12), **Slagworks** (industrial — sells ore cheapest at 12, pays for provisions 19 + machinery 70 inputs), **Driftmarch** (frontier — pays premiums, top machinery payer at 120, sells everything dear). Haven reconciliation: the 0085 placeholder prices ALREADY equal the approved table — zero deltas; the do update arm just guarantees convergence on re-apply.
- **Self-assert DO block** in the migration: each port carries exactly 6 active offers; the anti-pump spread (sell_price >= buy_price) holds on every seeded row; 3 CONCRETE profitable routes recomputed from the rows (direction law: player buys at origin sell_price, sells at destination buy_price → profit = dest.buy - origin.sell): ore Slagworks 12 → Haven 16 (+4), provisions Haven 15 → Slagworks 19 (+4), machinery Slagworks 88 → Driftmarch 120 (+32).
- **Proof** scripts/trade-econ-seed-proof.{sql,sh} (sources scripts/lib/trade-proof-lib.sh; read-only over Reference/Config, still begin→ROLLBACK per the trade idiom; toggles NO flag): ES1_PASS_MULTIPORT (all 18 rows active at exactly the approved prices, matched against an inline expected table), ES1_PASS_ROUTES (the 3 route profits recomputed from the seeded rows — never hardcode-trusted — and pinned to +4/+4/+32), ES1_PASS_ANTIPUMP (every row), ES1_PASS_ROLES (Slagworks is the cheapest ore seller; Driftmarch pays the most for machinery). Wired into .github/workflows/trade-v1-proof.yml (selftest step + a fourth real-chain matrix step; slice-econ-** added to the push triggers).
- **Docs:** ROADMAP trade row annotated (seed shipped, awaiting the ACT-TRADE flip).

Explicitly NOT in this slice (ACT-TRADE, queue #5): flipping trade_market_enabled / trade_relief_enabled / client TRADE_MARKET_ENABLED, and any activation script.

Verification. bash scripts/trade-econ-seed-proof.sh selftest green; all pre-existing trade + team-command proof selftests untouched-green; npx tsc -b + npx vite build untouched-green. Real-chain run happens in CI's disposable Supabase (no Docker on this machine).

2026-07-12 — ACTIVATION PREP: exploration + mining flip scripts + the 0172 writer reconcile (Phases 11/12; no flip)

Request. Full-capacity plan queue slices #2+#3: ship the two HUMAN-run activation scripts for the fully-built-dark exploration (0097-0101 + 0146 dup-guard) and mining (0102-0106 + 0143 double-extract guard) systems, in the proven `activate-team-command` / `activate-captains` idiom. The adversarial review of the slice then UPGRADED a "noted in passing" finding to a launch-blocker and found its twin (both the same failure class), so the slice grew ONE fix migration. NO flag flipped by this slice — each script run is a later, deliberate human go decision.

Bugs found (both pre-existing; both the SAME mistake — a "verbatim" re-create copied a STALE body)

- **H1 (HIGH — launch-blocker for the exploration flip).** 0146's re-create of `exploration_scan` copied the 0099 body, silently reverting 0100's insert change: new discoveries recorded NO `main_ship_id`. The securing fallback `mainship_resolve_owned_ship(player, null)` resolves ONLY when the player owns EXACTLY one ship (0081:47-51) and returns null otherwise, making `process_exploration_securing` continue (0100:223-228) — FOREVER. Multi-ship commissioning is LIVE since the team flip, so the day exploration flipped, any 2+-ship player's discovery rewards would have stranded permanently. (Initially logged here as "functionally safe" — WRONG: the fallback is not a safety net for multi-ship owners, it is a dead end.)
- **H2 (HIGH — same class).** 0143's re-create of `mining_extract` copied the 0104 body, clobbering 0137's P19 field-depletion integration (the reserve-scaled bundle at step 11.5 + the `worldstate_deplete_field` call at step 12.5): the deployed writer had ZERO depletion hooks and `worldstate_deplete_field` had NO caller. Latent while `world_balance_enabled=false`, but it would have silently killed the P19 depletion subsystem the day world balance lights.
- **Systemic lesson (recorded in the 0172 header too):** a CREATE OR REPLACE re-create must start from the TRUE head body — grep ALL later migrations for the function name before copying. The parity-re-create law exists for exactly this; both regressions came from copying a stale body.

Work done

- **Migration** `20260618000172_exploration_mining_writer_reconcile.sql` — re-creates BOTH writers from their true current heads with the dropped features merged back (diff-verified, code-only): `exploration_scan` = the 0146 head body + ONE marked hunk (the discoveries insert records `main_ship_id` again — the exact 0100:170 column set; the 0146 `unique_violation` handler KEPT verbatim); `mining_extract` = the 0143 head body (advisory lock KEPT) + the 0137 hunks re-merged verbatim (depletion locals, step 11.5 reserve scaling, `v_bundle` in the insert + result envelope, step 12.5 deplete call). Both writers are dark (flags false) so the re-creates are not live-risky; ACLs re-asserted per precedent. The header documents the PROSRC-ASSERT COUPLING: the activation scripts require the literal tokens `unique_violation` + `pending_bundle_json`, `main_ship_id` (exploration) and `pg_advisory_xact_lock` + `worldstate_deplete_field` (mining) in the deployed bodies — future re-creates must keep them (the in-body comments deliberately avoid the exact tokens so a comment can never satisfy the assert).
- `scripts/activate-exploration.{sql,sh}` — one all-or-nothing BEGIN..COMMIT, zero psql meta-commands (management-API/Dashboard-paste compatible). PRECONDITIONS: head ≥ **0172** + 0172 recorded (the H1 fix is a HARD gate); the 5 seeded sites present+active; every active bundle catalog-closed against `item_types`; the MERGED writer LIVE (the 0098 `exploration_discoveries_player_id_site_id_key` unique constraint by name + prosrc of the deployed `exploration_scan` containing BOTH the 0146 `unique_violation` handler AND the 0172-restored `pending_bundle_json`, `main_ship_id` insert column list — the flip physically cannot run against the broken writer); keys exist; the `exploration_scan_radius` knob asserted sane ($0 < r \leq 20000$) and NEVER rewritten. STAGE 1: the ONE write — `exploration_enabled` → true via `set_game_config`. SMOKE (read-only): committed flag (raw + `cfg_bool`); `to_regprocedure` over the whole surface (`command_exploration_scan(uuid, uuid)`, `get_my_exploration_discoveries()`, `exploration_scan(uuid, uuid, uuid)`, `process_exploration_securing()`, `osn_distance(dp×4)`, `reward_grant(text, uuid, uuid, uuid, jsonb)`); ≥ 5 active sites; EXACTLY ONE `process-exploration-securing` cron job. `ACTIVATE_EXPLORATION_PASS_*` markers + final PASS line; flag-only commented ROLLBACK (packet §7: discoveries persist harmlessly; the 0100 processor ignores the flag and — because the flip preconditions on 0172 — every post-flip discovery records its ship, so securing never depends on the multi-ship-ambiguous resolver fallback; rows can still legitimately wait for a safe settle or an active home base).
- `scripts/activate-mining.{sql,sh}` — same shape: head ≥ **0172** + 0172 recorded (the H2 fix is a HARD gate); 5 fields seeded+active; bundles catalog-closed; the MERGED writer LIVE (prosrc of the deployed `mining_extract` containing BOTH `pg_advisory_xact_lock` AND `worldstate_deplete_field`) + the 0103 `mining_extractions_cooldown_idx`; `mining_extract_radius` / `mining_extract_cooldown_seconds` asserted sane, never rewritten → `mining_enabled` → true → smoke (`command_mining_extract(uuid, uuid)`, `get_my_mining_extractions()`, `mining_extract(uuid, uuid, uuid)`, `process_mining_securing()`, the shared leaves, plus `worldstate_field_remaining(uuid) / worldstate_deplete_field(uuid)`; ONE `process-mining-securing` cron). The 0137 hooks stay dormant behind `world_balance_enabled=false` — the script never touches that flag. `ACTIVATE_MINING_PASS_*` markers; flag-only ROLLBACK + the documented (not asserted) POST-FLIP WATCH: `mining_extractions` rows with `secured_at` NULL are pending yields securing on the next safe settle.

- **NO client PR needed for either flip (verified this slice):** ExplorationPanel/MiningPanel are server-lit — `if (!isServerLit(result)) return null` (ExplorationPanel.tsx:70 on `get_my_exploration_discoveries`; MiningPanel.tsx:77 on `get_my_mining_extractions`) — and both are already mounted unconditionally on MapScreen's top-left OverlayRail (MapScreen.tsx:141/148); no exploration/mining compile constant exists in `osnReleaseGates.ts`. The server flip alone mounts both.
- **Run order (documented in both headers):** exploration first (packet \$7 "can go first"), mining a few days later; the scripts are fully independent (neither touches the other's key — selftest-enforced).
- **Proof coverage note:** no disposable SQL proof pins these two writer bodies today (`team-command-proof` covers combat; `verify-exploration.mjs/verify-mining.mjs` probe dark-reject envelopes + function existence, not bodies; `mining-p12-double-extract-concurrency.sh` pins the 0143 advisory-lock behavior, which 0172 keeps, so it stays valid). The activation preconditions now BIND both fixes via the prosrc asserts — the flips cannot run against a regressed writer. A lit-path depletion proof remains deferred to the P19/world-balance activation checklist (0137's own note).
- Docs: ROADMAP Phase 11/12 rows annotated "activation script shipped — awaiting human flip".

Verification. Both `.sh` selftests green (DB-free static safety: exactly 1 `set_game_config` call site on the 1 approved key, no meta-commands, no knob/foreign-key writes, no DDL/table writes — including unqualified `UPDATE ... SET` (the write-net regex covers `update (public.)?<table> set`, not just `update public.`), rollback commented, markers + prosrc-bindings + no-client-PR + run-order documented; mutation-tested — a false write, a removed H1/H2 prosrc assert, and an injected unqualified `update ... set` are each caught). 0172's re-created bodies diff-verified code-only against their heads: `exploration_scan` vs 0146 = ONLY the 2-line insert (adds `main_ship_id/p_main_ship_id`); `mining_extract` vs 0143 = ONLY the 0137 hunks (4 locals, step 11.5, `v_bundle` swap $\times 2$, step 12.5); `mining_extract` vs 0137 = ONLY the 0143 step-10b advisory lock. `npx tsc -b + npx vite build` untouched-green; `scripts/team-command-proof.sh selftest` still green.

Open follow-ups

- Deploy 0172 (with the normal migration pipeline) BEFORE the exploration flip — the activation script hard-fails without it, by design.
- The flips themselves are HUMAN-gated: `bash scripts/activate-exploration.sh run ACTIVATE_EXPLORATION` (then `node scripts/verify-exploration.mjs` + the packet \$7 manual smoke), and days later `bash scripts/activate-mining.sh run ACTIVATE_MINING` (then `node scripts/verify-mining.mjs`).

2026-07-12 — DEPLOYS: prod migration head 0169; team-command roadmap merged (main @ 9a292ed)

Merged. The whole team-command roadmap (A → D4) + the Mission Control UI renewal (R0 → R4) are on `main @ 9a292ed` (stacked slice branches landed via PRs #86/#87 and #97). **Zero open PRs.**

Deployed. GitHub Pages auto-deploy live (players see the Mission Control UI). Migrations: the owner approved the pending production deployments — 0165 (C0), then 0166–0169 (D0–D3) — **prod head = 20260618000169**. Everything team-shaped stays DARK: `team_command_enabled=false` (server, reject-before-read on every team RPC) + `TEAM_COMMAND_ENABLED=false` (compile-time; TeamRosterPanel not mounted). No flag flipped.

Next. Activation is human-gated, never a slice: `docs/TEAM_COMMAND.md` "ACTIVATION CHECKLIST" — decision support in the new `docs/TEAM_ACTIVATION_PACKET.md`.

2026-07-12 — TEAM-COMMAND Slice D complete (D0–D4): team combat, dark (migrations 0166–0169)

One line per sub-slice (full detail: `docs/TEAM_COMMAND.md`):

- **D0 (0166, RPC-only).** `calculate_group_expedition_stats` — service_role-only team-stats authority: per-member delegation to the unmodified `calculate_expedition_stats`, team-level folding ONLY (the eight additive 0122 keys summed, `speed = min`; STRICT refuse-don't-clamp — any member raise refuses the team) + the thin gated wrapper `get_my_group_expedition_totals` (any raise → one opaque `stats_invalid`).
- **D1 (0167, schema + parity re-creates).** `combat_units` widened to carry a member main ship (`main_ship_id` XOR `unit_type_id`, frozen attack/defense snapshots); the LIVE `process_combat_ticks` (head 0046) + `report_create` (head 0026) re-created with provably-inert deltas only (coalesce-first reads / member-row-gated branches — member rows have NO writer until D2). Three new internal leaves: `combat_fleet_return_speed`, `mainship_sync_combat_hp`, `mainship_mark_combat_destroyed`.

- **D2 (0168).** The team enters the engine: `send_ship_group_hunt` (ONE fleet per team over the 0050 narrow bridge; power = Σ member `combat_power`, speed = min — the D0 folding law), the `group_sortie_members` manifest (manifest-wins routing law), and `combat_create_group_encounter` — the FIRST member-row writer (raise-free by construction: a bad member degrades to an inert `alive_count=0` row), routed by a single manifest-gated branch in the re-created `combat_create_encounter` (head 0023).
- **D3 (0169).** Sortie settle: the escape/forced-extract branch marks surviving members `'returning'` via the ONE 0152 leaf; the 0050 reconciler re-homes members once their manifest fleet finishes (exact- complement race guard); manifests RETAINED (die with their fleet via the 0047 retention cascade); retreat verified verbatim; **the M1 activation blocker FIXED** (the live single send now re-claims the ship row under `FOR UPDATE` re-verifying `status='home'` — a lost `'hunting'→'traveling'` update now rejects).
- **D4 (frontend-only).** The dark Hunt UI (hunt-zone select + two-click confirm in `TeamRosterPanel`), the authoritative "Server totals" beside the C1 estimate, and combat-panel member-unit null-safety (`combatUnitLabel`: uuid-shaped key → "Team ship xxxxxxxx"). **Roadmap A → D4 COMPLETE, fully dark.**

Proof. `scripts/team-command-proof.{sql,sh}` (disposable write-then-ROLLBACK, CI workflow) gained the `TEAMSTATS` / `COMBATPARITY` / `TEAMHUNT` / `TEAMSETTLE` blocks — incl. the delegation pin (totals == the proof's own independent per-member sums), the tick parity pin (`player_damage == \Sigma` attack), the manifest-wins pin (mid-flight unassign leaves the sortie intact), and the M1 race pin.

2026-07-12 — UI RENEWAL R0–R4: Mission Control (PRs #86, #87–#91; frontend-only)

The full visual renewal, shipped in five slices with no migration and no behavior change:

- **R0** — Mission Control design-system foundation (tokens + primitives).
- **R1** — Mission Control galaxy map (marker hierarchy, starfield, overlay slots).
- **R2** — legacy panels converted to the Mission Control tokens + primitives (panel skins).
- **R3** — screen composition: the Ship / Port / Command two-rail split.
- **R4** — first-run, loading, and combat polish (Mission Control complete).

Verification: `tsc -b + vite build` green per slice; pure-logic specs green; dark surfaces byte-identical.

2026-07-12 — TEAM-COMMAND Slices C0 + C1: captains into teams, dark (migration 0165)

- **C0 (0165, RPC-only, ZERO data change).** `get_my_group_expedition_preview(p_group_id, p_activity)` — dark read-only group preview; per member it calls the unmodified `calculate_expedition_stats` in a per-member exception scope (`valid:false` + error, never a team 500); zero stat arithmetic in SQL. **Both** captain-slot bumps (hull `base_captain_slots` 2→6 AND the instance backfill) are DEFERRED TO ACTIVATION — the Ship screen renders both values ungated, so either bump would be player-visible while dark.
- **C1 (frontend-only).** Captain roster inside the team roster: per-member assign/unassign via the existing `CAPTAIN-P15` commands verbatim (no new server authority), rendered only while `isServerLit(get_my_captain_instances)`; plus the per-team expedition-preview UI (`aggregateTeamStats` — display-only estimate; the authority is D0's).

Proof: the team-command proof gained `TEAMCMD_PASS_CAPTAINS` (captain seed-bonus delta over an uncaptured baseline, uncaptured byte-parity with the solo preview, unassign-reverts; captains provisioned only via the sole writers).

2026-07-11 — TEAM-COMMAND Slice B complete: send/stop BY TEAM, dark (migrations 0161–0164 + UI + proof)

- **B0 (0161).** The dark group write path: `upsert_ship_group` / `assign_ship_to_group` + `mainship_resolve_owned_group` (explicit-only; dual same-auth.uid() resolution closes the same-player gap).
- **B1 (0162).** `delete_ship_group` (lock + revalidate; members un-grouped by the 0160 `ON DELETE SET NULL`)
 the interactive (still dark) roster UI: create/rename/delete/assign, no optimistic UI.
- **B-send (0163).** `send_ship_group_expedition` — loops the unmodified live send once per member inside ONE all-or-nothing subtransaction; writes no movement row directly (no second movement engine).

- **B-stop (0164).** `stop_ship_group_transit` — best-effort twin (per-member subtransactions; skip is legitimate): halt every halttable via the unmodified live `STOP=HOLD`, aggregate `{stopped, skipped, failed}`.
- **B-ui.** Send (destination select) + Stop controls in the roster, behind the compile-time gate.
- **B-verify (PR #84).** The disposable write-then-ROLLBACK proof `scripts/team-command-proof.{sql,sh}` + `.github/workflows/team-command-proof.yml` (a `.sql/.sh` proof, not the once-planned `.mjs`).

2026-07-09 — A0 FOUNDATION FIXUP + TEAM-COMMAND Slice A: team model (migrations 0159, 0160)

- **A0 (PR #77, 0159).** The pre-team audit blockers closed: `get_my_docked_store` (0158) + `get_my_expedition_preview` (0049) converted from unguarded arbitrary-ship reads to the `mainship_resolve_owned_ship` pattern (trailing `p_main_ship_id`); the 3 client `.maybeSingle()` ship reads made plural + selection-aware; ship selection lifted into ONE `shellState.selection` (was 3 disagreeing instances).
- **Slice A (PR #78, 0160).** `ship_groups` (one row per (`player_id`, `group_index` 1..3), owner-select RLS, created empty) + `main_ship_instances.group_id`; `max_main_ships_per_player` raised 3 → 24 (inert while commissioning stays gated); `team_command_enabled` seeded `false`; dead `get_main_ship` (0043) dropped — the last arbitrary-ship reader. Read-only `TeamRosterPanel` behind `TEAM_COMMAND_ENABLED = false`.

2026-07-09 — PR #75 merged: recall/return-home removed + stop-zoom fix; session handoff (#76)

#75 `fix-remove-recall`. Recall/return-home removed end to end (port-centric: there is no home base to return to), the (0,0) "home" ship marker suppressed, full spaghetti cleanup + unit-test fixes, and the stop-zoom camera fix (single focus point → gentle neighbourhood zoom, not MAX_K). 214 pure-logic specs pass. **#76** — the session HANDOFF doc (machine setup, live state, team roadmap).

2026-07-08 — PORT-CENTRIC ACTIVATED on prod + station storage (migrations 0157/0158)

Activation (human-gated flag flips, per `docs/PORTCENTRIC_DECISION_PACKET.md`). The game is now port-centric on the LIVE prod DB: `mainship_space_movement_enabled=true` (OSN port-to-port travel + docking), 3 starter ports REVEALED (Haven / Slagworks / Driftmarch — reveal is one-way), and `station_storage_enabled=true`. Utility: `scripts/activate-port-centric.mjs` `--confirm`; `rollback = flags only` (reveal persists).

Station storage (PR #73, 0157/0158). Per-port, per-player storage — the docked-port "Hangar": P1 foundation (tables + sole writer) + P2 docked store/read surface (`get_my_docked_store`).

Port-centric UI (PR #74). The home base removed from the app; map readability fixes (#72 wheel-zoom no longer scrolls the page; bigger location dots/labels).

2026-07-07 — STOP = HOLD, Slice D2: client copy/comments now match the hold semantics

Request. Replace the now-false "return home" wording in the visible Stop UI (and the stale in-code comments) so the client matches the server hold behavior — the last frontend piece of the goal.

Change (frontend copy + comments only — no logic). Legacy Stop CTA (`MapScreen.tsx`): `stopLabel` "Stop — return home" → "Stop — hold here"; `stoppedMessage` "Turning around — returning home." → "Holding position in open space."; its block comment rewritten from "0149: halt → symmetric return home" to "0155: halt an in-transit legacy move and HOLD the ship in open space; re-depart to a new leg from the held position via `MainShipCommand`". Comment/doc sync in the stop wrappers: `spaceStopCommand.ts` (`STOP_TRANSIT_RPC` header → `hold`; `parseStopTransitResult` no-op list `already_returning` → `already_held`, the 0155 reason rename — the `{stopped:false}→'arrived'` mapping is unchanged and still correct), `mainshipApi.ts` (`commandMainShipStopTransit` 0149 → 0155, "symmetric return time" → "holds the ship in open space", reason example → `already_held`), `SpaceStopControls.tsx` (header example → "Stop — hold here"), `useSpaceStopCommand.ts` (0149 → 0155, hold

semantics, and corrected the stale "each legacy trip is a fresh fleet" note — hold/resume reuses the SAME fleet, and the per-leg reset comes from the CTA's outbound-only mount dropping the fleet id to null and back). The held-send success copy in `MainShipCommand.tsx` was already the refined "Main ship departing to ..." from Slice D1 (matches the siblings moving/dispatched) — no change needed.

Deliberately UNCHANGED (out of scope): the OSN stop copy defaults, the stop-controller logic, and the still-valid **recall** feature (`request_main_ship_return`, the Ship screen's "Return home" button) — recall genuinely returns a present ship home and is not the removed stop-return-home.

The server before/after (why return-home was removed) is in the Slice A (0155) / Slice B (0156) entries — not duplicated here. With D2 the Stop=HOLD goal is implementation-complete on this branch (server 0155/0156 + verifier + frontend wiring D1 + copy D2), PR-ready for human merge review.

Verification (this sandbox, honest). `npm run build` → green. `npm run lint` → the pre-existing 22-error baseline, zero new on touched files. `grep -rniE 'return home|returning home|symmetric return'` over `src/` shows NO remaining Stop wording (only the unrelated recall button/screen, correctly left).

2026-07-06 — STOP = HOLD, Slice C: verifier proves the legacy hold round-trip

Request. Make `scripts/verify-stop-roundtrip.mjs` prove the NEW legacy Stop=HOLD contract (Slices A/B), replacing its stale legacy section which still asserted the removed return-home behavior.

Change (verifier only — no migration, no client runtime). Rewrote the LEGACY family (L1–L10) to prove **send** → **stop(HELD)** → **send-from-held** → **stop(HELD)**: leg 1 departs present; **stop 1** halts and holds (`{stopped:true, held:true}` + halt coords; leg-1 `fleet_movements` row → terminal 'cancelled'; ship stationary/in_space at its own coordinates; fleet completed with `active_movement_id` cleared); a **duplicate stop** is an idempotent `{stopped:false, reason:'already_held'}` no-op with no drift; **leg 2 RESUMES** from the held point via `move_main_ship_to_location` on the SAME fleet (new movement, `origin_type='space'`); **stop 2** holds the resumed leg again. Also fixed the OSN re-dock bridge: the ship now ends HELD (not home) after the legacy family, so the anchored OSN origin is re-established by sending the HELD ship to a dockable port via the one legacy held-departure path (was `send_main_ship_expedition`, which requires home). New helpers: `active_movement_id` added to the fleet read, `origin_type` to the movement read, and `isLegacyHeldFleet`; removed the now-unused `pollShipHome` (a held ship never goes home).

The OSN family (O1–O10) and the consumed-key regression probe are UNCHANGED — they already prove the OSN true-hold + the receipt-replay pin, and stay valid.

Assertion-shape note (honest): the 0155 hold envelope carries `{ok, stopped, held, main_ship_id, space_x, space_y}` — deliberately NO `movement_id` (Slice A spec). So the stop assertions check those fields, and per-leg identity is proven the stronger way — the specific `mvL1/mvL2 fleet_movements` row going 'cancelled' in the DB — rather than a self-reported envelope id. No migration was changed for this (Slice C is verifier + re-dock only).

The before/after Stop semantics and why return-home was removed are recorded in the Slice A (0155) and Slice B (0156) entries below — not duplicated here.

Verification (this sandbox, honest). `node --check scripts/verify-stop-roundtrip.mjs` → OK. `npm run build` → green (no `src/` touched). The end-to-end DB run **defers here by design** (needs `SUPABASE_SERVICE_ROLE_KEY` + egress — the 0148–0154 precedent; the script exits 2 without the key). It runs at the human apply gate.

2026-07-06 — STOP = HOLD, Slice B: RESUME from the held point (migration 0156)

Request. Complete the Stop-hold fix: Slice A (0155) made Stop park the ship in open space, but nothing could then Send it onward — recon confirmed no legacy Send accepts a `spatial_state='in_space'` fleet (`send_main_ship_expedition` needs ship `status='home'`; `move_main_ship_to_location` / `request_main_ship_return` need fleet `status='present'` + active presence). This slice adds the resume.

Design decision (self-approved): ONE send path, not a parallel resume RPC. Extend `move_main_ship_to_location` to depart from EITHER validated departure state — "present at a location" OR the new "held in open space" — deriving the movement ORIGIN accordingly. "Depart from a held point" is just a different origin for the same location-target rally movement, so the one RPC keeps ONE movement path and reuses its destination-validation / speed / `movement_create` / in-flight-marking machinery.

Change (Slice B — migration 20260618000156_mainship_legacy_depart_from_hold.sql):

1. `fleet_movements.origin_type` **domain extended** (forward-only, additive): drop + re-add the inline column CHECK → `origin_type` in ('base', 'location', 'zone', 'space'). A 'space' origin = departed from a held open-space coordinate with NO base/location/zone anchor (`origin_*_id` all NULL, `origin_x/y` = the held coordinates). **Safe because origin_type is provenance metadata only** — settlement branches solely on `target_type` (`process_fleet_movements` 0030:53,60,74; `movement_settle_arrival` 0151/0153); nothing anywhere reads `origin_type` to decide (verified across migrations + `src/`). Same table, same sole writer (Movement), same `target_type`-driven arrival — **not** the OSN coordinate domain (`main_ship_space_movements` untouched).
2. `move_main_ship_to_location` **generalized** (0152 body): present-departure path preserved; a **held-departure branch** is selected when the fleet is not present — it requires the exact 0155 held shape (`main_ship_instances` `status='stationary'`, `spatial_state='in_space'`, `space_x/y` NOT NULL) and departs from those raw coordinates (`origin_type='space'`, no presence to close). Destination validation (active, `activity_type='none'`, non-combat) + speed + `movement_create` + the in-flight mark are **shared** — the ONLY per-branch difference is how the origin is derived (and `presence_complete` on the present branch). The fleet flip to `moving` is ONE shared guarded UPDATE (present: `status='present'`; hold: `status='completed'`; both `active_movement_id` IS NULL), raise+rollback on miss (0053 discipline). The ship is marked in-flight via the **same 0152 helper** the present branch already uses (`mainship_mark_legacy_in_flight(ship, 'traveling')`) — no inline duplication; for a held departure that same write clears the held coordinates as the ship leaves the hold point.

Accepted micro-deltas on the present path (documented; observable behavior preserved — 0153 precedent): the shared fleet-flip guard adds a NO-OP `active_movement_id` IS NULL for present and generalizes the race-only raise message; `presence_complete` now runs just after `movement_create` (independent tables, same atomic txn); the success envelope gains an additive from marker ('present' | 'hold') with existing keys unchanged (`from_location_id` NULL for a held departure).

No flag flip, no OSN touch, no new table, no new RPC, no `send_main_ship_expedition` change. Movement stays sole writer of `fleet_movements`; Main Ship sole writer of `main_ship_instances`; non-combat destinations only. **Slice C** updates the client (find/select the held fleet + the from marker) and rewrites the stop-roundtrip verifier's legacy section to `send→stop(held)→send→stop(held)`.

Verification (this sandbox, honest). `npm run build` → green. No JS touched (node --check n/a). `docs/SYSTEM_BOUNDARIES.md` synced same step (`origin_type` domain + `move_main_ship_to_location` departs present OR held). DB-dependent verifiers defer here by design (no service-role key + blocked egress — the 0148–0154 precedent); the `send→stop(held)→send→stop(held)` run is Slice C's rewritten verifier at the human apply gate.

2026-07-06 — STOP = HOLD-IN-OPEN-SPACE (legacy), Slice A: stop settlement (migration 0155)

Request. "Stop must HOLD, not return home." The visible Stop button drove `command_main_ship_stop_transit`, whose head-of-branch body (0149, re-created verbatim in 0152) HALTED an outbound legacy main-ship transit and then flew the ship **all the way back to its origin base** — it transformed the moving `fleet_movements` row in place into the `return_home` shape (symmetric turnaround), stepped the fleet moving→returning, and set the ship 'returning'. That is the wrong semantic: the player asked to stop where they are, not to abandon the trip and travel the whole way home. (The prior 2026-07-06 "stop fix" was a client-only idempotency-key fix — orthogonal; return-home was still live.)

Change (Slice A — migration 20260618000155_mainship_legacy_stop_holds_in_space.sql). Re-create `command_main_ship_stop_transit` with everything up to the halt-point interpolation UNCHANGED (auth · the EXISTING `mainship_send_enabled` gate, unchanged · owned-main-ship-fleet checks · claim the `status='moving'` `fleet_movements` row FOR UPDATE in the cron's lock order · the arrived due-row no-op that leaves settlement to the cron · the `v_frac/v_turn_x/v_turn_y` math). Only the **settlement** changes, to a true HOLD:

- `fleet_movements` **row** → `status='cancelled'`, `resolved_at=now()` (a first-class terminal in the 0007 status domain `moving|arrived|cancelled|failed`). 'cancelled' is the honest terminal for a player halt — the ship did NOT reach `target_x/target_y` (so not 'arrived'), and it is not an error ('failed'). Once non-'moving', the cron's `status='moving'` scan skips it forever; no arrival, no base/location deposit (main-ship legacy targets carry no reward — 0149:35-37).
- `fleets` **row** → `status='completed'`, `location_mode='movement'`, `active_movement_id=NULL`, `current_*=NULL` — the movement-less settled terminal, guarded where `status='moving'` and `active_movement_id=m.id` (raise + roll back on miss, 0149's discipline). This is the EXACT shape the OSN stop leaves a fleet held in open space (0064:312-317); it mirrors a normal legacy settlement (`fleet_complete`→'completed') but points at no base/location because the ship stopped out in space.
- `main_ship_instances` **row** → the held-in-open-space shape `status='stationary'`, `spatial_state='in_space'`, `space_x=v_turn_x`, `space_y=v_turn_y` (was `status='returning'`). **This reuses the EXISTING legal (stationary, in_space) representation** — legal under the 0055 lifecycle CHECKs (`in_space`⇒`stationary` 0055:143-144; `stationary`⇒`in_space` at `location` 0055:159-161) + the 0054 coordinate rule (`in_space` REQUIRES both finite coordinates). It is the SAME canonical shape OSN's stop writes (0064:319-322). Written inline to match 0149/0152's established style; the shared invariant is the constraint set (single source of truth for what "held in open space" legally is), not a helper — each domain settles its OWN movement inline.

Idempotency / replay. A stop grants nothing → no receipt. The old `already_returning` no-op is gone (Stop no longer produces a `return_home` row): a duplicate Stop now finds no moving row and, since the ship is already (`stationary`, `in_space`), returns `{ok:true, stopped:false, reason:'already_held'}`; `already_settled` (no moving row, not held) and `arrived` (due row left for the cron) are unchanged.

Response shape changed: a successful hold returns `{ok:true, stopped:true, held:true, main_ship_id, space_x, space_y}` — NO `arrive_at` (the ship is parked, not arriving). **Slice C** updates the client copy / handling to this shape.

No constraint edit, no new table, no flag change, OSN untouched. Movement stays sole writer of `fleet_movements`; Main Ship stays sole writer of `main_ship_instances`; the OSN coordinate domain and its `mainship_space_movement_enabled` flag are not touched; `mainship_send_enabled` is unchanged. **Slice B** (next) adds legacy **re-departure (Send) from the held position** — recon confirmed no existing legacy Send can resume from a `spatial_state='in_space'` fleet (`send_main_ship_expedition` needs `status='home'`; `move_main_ship_to_location/request_main_ship_return` need `status='present'` + active presence).

Verification (this sandbox, honest). `npm run build` → green. `docs/SYSTEM_BOUNDARIES.md` synced in the same step (Movement matrix cell + the `command_main_ship_stop_transit` flow section rewritten from `return-home` → `hold-in-open-space`; the 0152 helper-callers blockquote corrected — Stop is no longer a `mainship_mark_legacy_in_flight` caller; ownership rows unchanged). No JS touched (`node --check` n/a). DB-dependent verifiers defer here by design (no service-role key + blocked egress — the 0148–0154 precedent).

Known stale test, retired in Slice B (flagged, not fake-fixed): `scripts/verify-stop-roundtrip.mjs`'s LEGACY section (L3/L4/L9/L10) still asserts the OLD `return-home` shape (`mission_type='return_home'`, `ship returning/spatial_state` NULL) and its leg-2 re-departure relies on the ship settling HOME between trips — a path 0155 removes. It is NOT rewritten here: the correct legacy assertions are the held-in-space + re-departure pattern the verifier's OSN section already uses (O4/O5/O10), and leg-2 re-departure only becomes reachable once Slice B adds legacy Send-from-held. Slice B rewrites the legacy section to `send→stop(held)→send→stop(held)` and it runs at the human apply gate. (Retirement condition: Slice B lands the resume path + the rewritten verifier.)

2026-07-06 — GOAL WRAP-UP: stop fix + UI rebuild + map declutter — final verification & the ONE apply-time checklist

Scope of the goal (all three items implementation-complete on this branch; docs-only final pass): (1) the Stop bug, (2) the full UI rebuild, (3) the map declutter. Whole-branch verification ran clean — no remnant found, nothing fixed in this slice.

Final verification results (honest; this sandbox):

- `npm run build` → green. `npm run lint` → exactly the 22 pre-existing errors (all in files this goal never touched; ZERO on any goal surface). `node --check scripts/verify-stop-roundtrip.mjs` → OK. Controller unit specs (`spaceStopCommand` / `portMoveCommand` / `spaceMoveCommand`) → **40/40 green** (includes the consumed-key regression cases).
- DB-dependent verifiers **defer here by design** (no service-role key + blocked egress — the 0148–0154 precedent): `verify:stop-roundtrip` exits 2 without the key; `verify:m2..m5,m45`, `verify:mainship-legacy-dock` need the target DB. They are the human apply gate below.
- **Remnant sweep clean:** no live import/JSX reference anywhere in `src/` to any deleted surface (`ExpeditionLauncher`, `FleetStatusPanel`, `Dashboard`, `GalaxyMapScreen`, `CombatReportPage`, `CombatReportsView`, `MainShipPanel`, `MainShipPreview`, `DockServicesPanel`, `fleetGuards`, `lib/location` — historical comments only); exactly ONE galaxy route (`/map`; `/`, `/galaxy`, `/reports`, * all resolve into the shell) and NO fleets UI (`features/fleets` holds only the `api/types` the data layer reads); ZERO raw palette literals across the shell, all four screens, and the shared primitives (full numeric-scale + hex grep). Jargon spot-check: player copy is humanized on all four destinations; `MainShipCommand`'s "currently {status}" interpolates plain English words (`traveling/present/returning/home` — fine); noted, not changed: `ActiveCombatPanel` (visible only during a live battle) kept its pre-goal interior — a candidate for a future combat polish slice, not a goal remnant.

Goal exit criteria — met, with evidence:

1. **Stop works on every leg.** Root cause: the consumed idempotency key survived success and replayed the first receipt (recon §D). Fix: `requestId: null` on the success branch of all three controllers (`spaceStopCommand/portMoveCommand/spaceMoveCommand` — slice 1, unit-proven); end-to-end proof authored: `verify:stop-roundtrip` (`send→stop→send→stop` on BOTH families + the consumed-key replay probe asserting the server settles nothing on a stale key).
2. **Navigation genuinely restructured.** ONE persistent mobile-first four-destination shell (`Map · Ship · Port · Command`; `AppShell` + nested routes; shared state fetched once; the arrival settle consolidated to one mount); the duplicate map path (`ExpeditionLauncher`) and ALL legacy fleets UI (`FleetStatusPanel` + the client legacy-leave affordance) DELETED with server plumbing untouched; all four interiors rebuilt in one design language (`identity` → `right-now` → `details`, `StatRow/Meter`, plain player language, no-softlock chain preserved, dark panels server-lit only).

3. **Map readable.** Migration 20260618000154 relocates exactly the five clustered waypoints to ≥ 29.2 -world-unit pairwise separation ($\approx 24\%$ of the content span vs the $\sim 9\%$ no-overlap threshold), waypoints-only: ports/anchors/snapshots untouched, Dock-0 exact-match holds by construction (recon-proven; the migration guards it).

THE CONSOLIDATED HUMAN APPLY-TIME CHECKLIST (in order — the owner's gate, never this loop):

1. Apply migration 20260618000154_map_declutter_waypoints.sql forward-only, after 0153 (0152/0153 themselves are still pending apply per their own entries — apply 0152 → 0153 → 0154).
2. Run the engine suite: `npm run verify:m2 && npm run verify:m3 && npm run verify:m4 && npm run verify:m5 && npm run verify:m45`.
3. Run `npm run verify:mainship-legacy-dock` (the 0152/0153 round-trip + constraint guards), then `npm run verify:stop-roundtrip` (send→stop→send→stop on both families + the consumed-key regression probe; a family whose flag is dark on that DB skips loudly with exit 2).
4. Visual map-readability pass at default zoom: five separated, labeled waypoints + three ports + home — no overlapping labels/markers.
5. Mobile-width (~ 390 px) walkthrough of the four destinations: **Map** — select a destination → send; Stop visible mid-transit; dock at a port; **Ship** — repair reachable (disable via the dev path if desired), countdown while traveling, Return home when away; **Port** — docked shows the port card, undocked shows the friendly empty state; **Command** — base + battle history (reports), sign-out in the footer.

docs/SYSTEM_BOUNDARIES.md — confirmed unchanged across the WHOLE goal: client presentation + a data-only waypoint relocation (Map stays sole writer of `locations`) + a client-only stop fix; no server writer/table/constraint/cross-system call changed anywhere in these slices.

2026-07-06 — UI REBUILD (2b): Map interior — detail panel humanized, overlays organized, selector dedup

The Map destination's interior rebuilt — the galaxy canvas stays the hero; the location detail panel and the feature overlays now speak the shared design language (identity → right-now → details, `StatRow`, tokens only, plain player language):

- **Detail panel hierarchy:** IDENTITY (location name + a humanized kind + its zone, with one Badge: Port / Safe / Hostile) → RIGHT NOW (MainShipCommand — THE pick-a-destination → send flow, unchanged logic/testids, full-width primary CTA; flag-dark → omitted entirely) → DETAILS (humanized `StatRows`). Phone-friendly: the aside is now a capped, scrollable bottom sheet (max-h-[45dvh]) below md. The local `Row` component is deleted (the shared `StatRow` rule).
- **The dev-jargon → player-language mapping (design decision, lives ONLY in MapScreen):** `location_type` → "Trade port / Pirate hunting ground / Pirate den / Safe waypoint / Mining site / Derelict station / Rally point / Event site"; `base_difficulty` → Danger "None — safe space / Low (≤ 10) / Moderate (≤ 20) / High"; `reward_tier` → Rewards "None / Modest / Good / Rich"; zone + sector shown as plain words (subtitle + a "Region" row). **DROPPED as dev-internal noise:** raw coordinates, raw status (`get_world_map` returns only active rows — the field could never read anything else), `pressure/danger_modifier` decimals, and the active-fleets debug count. The map data layer is untouched (`locationStates` stay polled; presentation simply no longer surfaces them).
- **Overlay organization (no logic/wiring/gating change):** PortNavPanel (top-left) and the stop CTAs (bottom-right) keep their existing token-styled overlay positions; the three server-lit feature panels (Exploration / Mining / WorldEvents) now ride ONE bottom-left overlay rail (positioned, scrollable, `pointer-events-none` shell) so that WHEN a capability lights they read as coherent map overlays instead of raw flow cards breaking the canvas layout — dark today, the rail renders empty and never intercepts map gestures. All server-lit `return null` gates verbatim. **No-softlock preserved verbatim:** legacy transit Stop, PortNav's OSN stop + the held-in-space re-departure surface, and GalaxyMap's coordinate-transit Stop all stay mounted on this destination, flag-independent by their own state predicates exactly as before.
- **Reviewer-flagged duplication fixed:** the "active legacy movement row of the main-ship fleet" derivation, previously computed in BOTH `AppShell` (settle wiring) and `MapScreen` (stop CTA), is now ONE shared selector — `selectActiveLegacyMovement` in `spaceStopCommand.ts` (the pure map-logic module) — called from both sites. Pure refactor, identical behavior; `spaceStopCommand.spec.ts` re-run green (10/10).

Verification (honest): `npm run build green`; `npm run lint` at the exact 22-error pre-existing baseline (zero on touched files); zero raw palette literals on all touched surfaces (grep-verified); preserved test ids (`galaxy-map-screen`, `galaxy-map-loading/-error`, `galaxy-location-detail-panel`, all `mainship-*` command ids). The dark Map panels can't be exercised live from this sandbox (server-lit; no service key + blocked egress) — their gates were not modified.

docs/SYSTEM_BOUNDARIES.md unchanged (client-only presentation over unchanged server ownership).

2026-07-06 — UI REBUILD (2b): Command interior — home base in the shared design language

The Command destination rebuilt (identity → right-now → details, `StatRow` rows, tokens only, plain player language, mobile-first single column):

- **RIGHT-NOW focus rule (one focus per state, top-down):** pending onboarding first (`PortEntryPanel` — server-authoritative self-hide kept verbatim; its accent card is the screen's focus when the server says an action is needed) → any LIVE battle (`ActiveCombatPanel`, wiring untouched) → otherwise the base card's quiet all-clear line ("All quiet — nothing here needs your attention" + a set-out-from-the-Map hint), which `CommandScreen` suppresses while a battle is live so the combat panels hold the focus alone. No wall of equal-weight cards.
- **BasePanel interior rebuilt** (same file/props + a new `quiet` flag; presentation only, no RPC — the panel never had one): IDENTITY (base name + "Your home base"; the dev-jargon "(0, 0)" coordinate label dropped) → the right-now all-clear → DETAILS ("Stored resources" and "Garrison" as `StatRow` lists with mono tabular numbers; plain empty-states). **Honest scope note:** NO client production/build surface exists today — `train_units/cancel_build_order` have zero client call sites (the training UI was retired with the legacy fleet surfaces) — so no build section was invented (a new command surface is a capability decision, not presentation); the right-now third state is therefore the quiet state.
- **ReportsSection** adopted `StatRow` for its report facts (the local `Fact` label/value row deleted — the no-local-row rule); list/expand/round-log behavior unchanged. Dark `RankingPanel` keeps its server-lit `return null` gate verbatim — omitted while dark, never a placeholder.
- **Sign-out re-placed** as a quiet account footer (email + small ghost button) — a secondary affordance that no longer competes with base actions; behavior unchanged (no test id existed).
- **Dead code removed with its last caller:** `src/lib/location.ts` (`FormatLocationLabel`) — its final imports died with `FleetStatusPanel` (nav-shell slice) and `BasePanel`'s dropped coordinate label; zero call sites remained.

Verification (honest): `npm run build green`; `npm run lint` at the exact 22-error pre-existing baseline (zero on touched files); zero raw palette literals on all touched surfaces (grep-verified). The dark Command surface (`Ranking`) can't be exercised live from this sandbox (server-lit; no service key + blocked egress) — its gate was not modified. `docs/SYSTEM_BOUNDARIES.md` unchanged (client-only presentation over unchanged server ownership).

2026-07-06 — UI REBUILD (2b): Port interior — one docked-services surface, `DockServicesPanel` folded

The Port destination rebuilt in the Ship-established design language (identity → right-now → details, `StatRow` rows, tokens only, plain player language, mobile-first single column):

- **NOT DOCKED:** one clear, friendly empty state ("Not docked / Dock at a port to access its services" + a travel-via-Map hint; testid `port-not-docked`) — keyed off the SAME server-authoritative dock projection as everything else (`useDockServices` → `isDocked`), no second source of docked truth, never a broken/blank screen.
- **DOCKED:** the new `src/features/port/DockedPortCard.tsx` — IDENTITY (the port's name as the title + "Docked" badge), RIGHT NOW ("Berth secured..." + the leave-via-Map hint; docking is a passive service, so the port's action surfaces are the server-lit panels below), DETAILS (each ACTIVE service as a plain-language `StatRow`: Docking → "Berth secured", Market → "Buy & sell goods", ...; only what the server reported — never an inactive service).

`DockServicesPanel` FOLDED and deleted: its presentation became `DockedPortCard` (the old absolute map-overlay styling died with the overlay mount — it had been floating wrongly inside the Port flow since the shell slice); its dock read is now `PortScreen`'s single `useDockServices` call — this also retires the shell-slice double-read debt (the screen no longer reads the projection once for the branch and again inside the panel). All test ids preserved (`dock-services-panel` / `-title` / `-list` / `dock-service-<s>` / `-none`); the fail-closed `isDocked` render gate is kept verbatim inside the card. `StatRow` gained rest-prop passthrough (the Card convention) so rows can carry test ids — no new primitive added.

Rendered-proof suite kept honest: `tests/harness/dockServicesHarness.tsx` now mounts the REAL composition `PortScreen` uses (`useDockServices` → `DockedPortCard`, same injected fetcher + `__fail` path), and `tests/dockServicesUi.uispec.ts`'s copy assertions track the new presentation (the port name IS the title; the old map-overlay half-width comment corrected). Dark panels (`InvestmentPanel`, `MarketPanel` behind `TRADE_MARKET_ENABLED`) keep their server-lit gates verbatim — surfaced only when lit, omitted otherwise. No flag, no command logic, no RPC change.

Verification (honest): `npm run build green`; `npm run lint` at the exact 22-error pre-existing baseline (the harness's two immutability errors are pre-existing, line-shifted); zero raw palette literals on all touched surfaces (grep-verified). The `.uispec.ts` rendered suites are deliberately outside the default Playwright testMatch and need the CI browser runner (documented precedent — this sandbox lacks it); attempted anyway ("No tests found" under the default config), so the harness was additionally TYPE-CHECKED standalone (clean — only the expected standalone-tsc `import.meta.env` vite-types gap, unrelated).

Dark Port panels can't be exercised live (server-lit; no service key + blocked egress). docs/SYSTEM_BOUNDARIES.md unchanged (client-only presentation over unchanged server ownership).

2026-07-06 — UI REBUILD (2b): Ship interior — the MainShipPreview + MainShipPanel MERGE

The audit-mandated collapse, done: MainShipPreview (card + repair + the only recall) and MainShipPanel (derived status + destination countdown) are MERGED into ONE surface — src/features/ship/ShipStatusCard.tsx — and both old files are DELETED (they had no other mount after the shell slice). The union of capabilities is preserved: repair, recall ("Return home"), live travel countdown + progress, hull integrity, cargo/fittings, the no-ship starter-hull teaser. Same RPCs verbatim (repair_main_ship / request_main_ship_return), same double-submit guards, same testids (mainship-repair / mainship-recall / error notes) — presentation restructure only.

The hierarchy (the design language the other destinations will reuse): (1) IDENTITY — ship name + hull subtitle + one state Badge + a hull-integrity Meter; (2) RIGHT NOW — one prominent primary-action block for the current state (Repair when disabled · the live countdown + progress when under way, with a "use Stop on the Map" hint · Return-home when away · a quiet "ready to fly" line at home); (3) DETAILS — plain-language stat rows (Cargo hold / Speed / Captain seats / Module slots). Dev-jargon labels replaced ("Readiness (HP)" → hull integrity; raw status words → player sentences). Mobile-first: single column at ~390px, full-width ≥44px action buttons.

Data/wiring: the card is fed from the shell's already-pollled state (game.mainShip + map.mainShipFleet/movements) — the old preview's self-fetch existed only because the pre-shell overlay had no shared state; no new fetch, no polling change, no command-logic change. **No-softlock:** Repair now renders whenever the ship is disabled, INDEPENDENT of the send flag — matching the server's deliberately ungated repair safelock (0052:120); previously the preview's repair block sat inside its send-flag branch. Return-home stays send-flag-gated exactly as before (its RPC is flag-gated server-side). Dark Ship panels (Modules / Captains / Recruit / ShipSwitcher) keep their server-lit return null gates verbatim — surfaced only when lit, omitted otherwise.

New shared primitive (ONE, needed now): src/components/ui/StatRow.tsx — the label/value stat row (inside a <d1>), exported from the ui index. Ship uses it first; Port/Command/Map detail lists adopt it in their interior slices (each still carries a local Row/Fact copy of this exact pattern — to be replaced, not duplicated). No other abstraction added.

Dead code removed with its last caller: src/features/fleets/fleetGuards.ts (isMainShipFleet) — its final import died with MainShipPanel; the Phase-10E legacy/main-ship UI isolation it guarded is now STRUCTURAL (no legacy fleet surface exists in the client at all; the server RPCs and their guards are untouched).

Verification (honest): npm run build green; npm run lint at the exact 22-error pre-existing baseline (zero on touched files); zero raw palette literals on all touched surfaces (grep-verified). Dark Ship panels can't be exercised live from this sandbox (server-lit; no service key + blocked egress) — their gates were not modified. docs/SYSTEM_BOUNDARIES.md unchanged (client-only presentation over unchanged server ownership).

2026-07-06 — UI REBUILD (2b): the persistent four-destination nav shell (structure + navigation)

The restructure (not a re-skin): ONE persistent, mobile-first bottom tab bar — Map · Ship · Port · Command — replaces the old link-hopping between three sibling routes. Audit + locked target: UIREBUILD_AUDIT.local.md. This slice is structure/navigation + the two deletions; each destination's interior redesign is the following per-screen slices (panels were RELOCATED unchanged).

BEFORE → AFTER screen inventory (the 2e before→after record):

- **Routes before:** / Dashboard (base + port-entry + main-ship status + combat + expedition launcher + fleets list + inline reports + dark ranking), /galaxy GalaxyMapScreen (map + preview overlay + port-nav + stops + dock services + 8 dark panels + detail/send), /reports CombatReportPage, /auth, *→/. Navigation was three header links; no persistent nav.
- **Routes after:** /map, /ship, /port, /command under the ONE AppShell (bottom tab bar, ≥44px targets, tokens only, active tab from the router); / → /map (the primary play surface); legacy /galaxy → /map and /reports → /command redirects keep old bookmarks working; /auth + * fallback unchanged.
- **Map** (src/features/map/MapScreen.tsx): galaxy canvas + location detail with the ONE in-map send flow (MainShipCommand) + PortNavPanel (travel + OSN stop + the held-in-space re-departure surface) + the legacy transit Stop CTA + dark coordinate targeting + dark Exploration / Mining / WorldEvents (server-lit gates verbatim).

- **Ship** (src/features/ship/ShipScreen.tsx): MainShipPreview (card + repair + the ONLY recall) and MainShipPanel (status + destination countdown) relocated side by side — their MERGE into one surface is the Ship interior slice; dark Modules / Captains / RecruitCaptain / ShipSwitcher (server-lit gates verbatim; omitted while dark, never dead panels).
- **Port** (src/features/port/PortScreen.tsx): docked-only — DockServicesPanel + dark Investment / Market, keyed off the SAME server docked projection (isDocked); when not docked, a friendly "Not docked — dock at a port to access its services" empty state (never a broken screen).
- **Command** (src/features/command/CommandScreen.tsx): BasePanel + PortEntryPanel onboarding + ActiveCombatPanel(s) + the MERGED reports section + dark RankingPanel + sign-out.
- **DELETED (the two user-reported failures)**: ExpeditionLauncher (the duplicate map path — a Card that only linked to /galaxy; the send flow already lives IN the map, so nothing to fold) and FleetStatusPanel (ALL legacy fleets UI, including the client legacy-leave affordance fleetApi.requestLeaveLocation — no client call path to request_leave_location remains). The server-side fleets rows, RPCs, and movement plumbing are UNTOUCHED (load-bearing main-ship plumbing; fleetGuards.isMainShipFleet stays, used by MainShipPanel).
- **MERGED**: the /reports CombatReportPage + the inline CombatReportsView → ONE ReportsSection in Command (list + on-expand round-log fetch, fed from the shell's polled combat state instead of its own triple fetch). Empty shells deleted: Dashboard.tsx, GalaxyMapScreen.tsx, CombatReportPage.tsx, CombatReportsView.tsx.

Shared state lifted (fetched once): the three polled hooks (useGalaxyMapData, useGameState, useCombat) mount exactly once in AppShell and reach destinations via useShellState (src/app/shellState.ts) — no destination mounts its own copy.

Consolidated arrival settle: the old Dashboard mounted useSettleDueArrival for the legacy leg and GalaxyMapScreen for the OSN leg — safe only while those routes were mutually exclusive; with a persistent shell that invariant is gone, so the hook now mounts EXACTLY ONCE in AppShell covering BOTH legacyMovement and the OSN movement, and both per-screen mountings are removed.

Dark stays dark: every dark panel keeps its server-lit return null gate verbatim and is surfaced only when already lit — no flag flipped, no capability activated, no server change. **No-softlock preserved**: all three Stop CTAs + PortNav re-departure live on Map (mounted flag-independent, state-predicated as before); repair (MainShipPreview) mounts UNGATED on Ship.

Verification (honest): npm run build green (bundle -7 kB from the deletions); npm run lint back to the exact 22-error pre-existing baseline (zero errors in any new/touched file; one new-file react-refresh hit was fixed by moving the context/hook into shellState.ts). Zero raw palette literals on all new/kept surfaces (grep-verified). The deployed-site browser smoke (tests/galaxy.spec.ts) was updated to the new flow (sign-in lands directly on Map; the "Galaxy map" link/heading assertions are gone) — it runs against the DEPLOYED site, so it passes only once this UI deploys; dark panels/flows could not be exercised live from this sandbox (server-lit; no service key + blocked egress). docs/SYSTEM_BOUNDARIES.md unchanged — client-only navigation over unchanged server ownership (no table/writer/constraint/cross-system call changed).

2026-07-06 — MAP DECLUTTER: waypoint relocation (migration 0154, data-only)

Problem (root cause, full trace in MAP_DECLUTTER_RECON.local.md): the 0002 waypoints were seeded 1–3.6 world units apart on the tiny legacy map scale, while the 0066 starter ports sit on the OSN scale (–50...80). The content-fit camera (galaxyCamera fitCameraToWorldPoints, MAX_K=1024) frames the 120-unit port spread, compressing the two waypoint clusters to ~8–20 screen px — with counter-scaled constant-size markers/labels that means overlapping halos and unreadable labels at default zoom (min pairwise separation 1.2% of the content span vs the ~9% no-overlap threshold).

Migration 20260618000154_map_declutter_waypoints.sql (**forward-only; no shipped file touched**): relocates ONLY the five waypoint locations rows, matched by their post-0148 one-word zone-scoped (zone_id, name) key — the exact 0148 idiom (single fail-closed atomic do-block; presence check, GET DIAGNOSTICS exactly-5-rows guard, exact-coordinate read-back, ports-untouched guard; idempotent re-run — a same-value UPDATE still matches all five and the read-back accepts already-at-target):

waypoint	zone	before	after
Refuge (safe)	Wreck Belt	(11, 5)	(–30, 15)
Snare (pirate d10)	Wreck Belt	(12, 6)	(–15, 40)
Reaver (pirate d15)	Wreck Belt	(9, 4)	(–45, 40)
Lull (safe)	Ion Storm Route	(31, 22)	(40, 30)
Blackden (pirate d25)	Ion Storm Route	(33, 23)	(65, 55)

The content bbox (x –50...70, y –30...80) is unchanged, so the default zoom is unchanged; min pairwise separation over all nine map points (8 locations + the (0,0) home base) becomes **29.2 world units ≈ 24% of span** (≈163 px at default fit vs the ≈60 px label requirement — >2.5× margin, viewport-independent). Distance-from-home now orders by difficulty (Refuge 33.5 < Snare

42.7 < Lull 50 < Reaver 60.2 < Blackden 85.1 — the old seed had the d15 site as the CLOSEST point of all at 9.85u; zone geography preserved (Wreck Belt trio west with Haven, Ion Storm pair east with Slagworks/Driftmarch).

Deliberately untouched (the recon's blast-radius proof): NO port row, NO `space_anchors` row (the waypoints have none; the unmoved ports keep the 0066 anchor==location alignment, guarded in-migration), NO `fleet_movements/main_ship_space_movements` snapshot backfill (per-trip snapshots settle by IDs; rewriting in-flight geometry would teleport moving ships), NO function, flag, config, or grant. Dock-0's exact-match compares the ANCHOR to the movement snapshot (0067:564-572) and `locations.x/y` is consulted NOWHERE in the OSN domain — so docking holds by construction. Legacy sends read 1.x/1.y LIVE at send time, so future waypoint trips get ~2.8× longer on average; `travel_scale / min_travel_seconds` remain the human-owned pacing knobs (no value changed here). **STANDING INVARIANT for any FUTURE port relocation:** move `locations.x/y` and retire+insert the port's anchor in ONE migration (0063 lifecycle), same values both places (0066 invariant), accepting `target_anchor_changed` terminal failures for routes in flight at apply time — deliberate, never a silent redirect.

Verification (honest; environmental precedent unchanged — 0148-0153 "authored, reviewed, NOT applied"): no service-role key in this sandbox and network egress is blocked, so the migration was verified statically: one balanced do \$\$... \$\$; block; the five-row GET DIAGNOSTICS guard; exact-coordinate read-back for all five; the three fixed-UUID port rows asserted still at their 0066 coords; idempotent-re-run semantics reasoned through (same-value update → count 5 → read-back passes). `verify:m2` asserts waypoint NAMES + types only (`scripts/verify-m2.mjs:77-93`), never coordinates — it stays green post-apply. `docs/SYSTEM_BOUNDARIES.md` unchanged (Map remains the sole writer of `locations`; no `writer/table/constraint/cross-system` call changed). **HUMAN CHECKLIST (the owner's gate):** (1) apply 0154 after 0148-0153, forward-only; (2) re-run `verify:m2` + `verify:mainship-legacy-dock` + `verify:stop-roundtrip` (ports/anchors unmoved — both stop families unaffected); (3) visual pass of the galaxy map at default zoom (five separated, labeled waypoints); (4) optionally retune `travel_scale/min_travel_seconds` if the ~2.8× longer legacy waypoint trips should keep their old wall-clock feel.

2026-07-06 — STOP/MOVE FIX, slice 2: the send→stop→send→stop verifier

New verifier `scripts/verify-stop-roundtrip.mjs` (+ `package.json` script `verify:stop-roundtrip`) — proves goal item (1) end-to-end: Stop works on EVERY in-transit leg, not just the first. It proves the SERVER-side contract the slice-1 client fix relies on (each stop sent with a FRESH request id halts exactly ITS OWN leg); slice-1's controller unit tests prove the client now emits that fresh key per leg — the two layers together close the goal. Covers BOTH stop families plus the regression probe:

- Legacy family** (`fleet_movements / command_main_ship_stop_transit`): commissioned-docked departure (`move_main_ship_to_location`) → stop 1 transforms THE leg-1 row in place to `mission_type='return_home'` (target = home base), fleet returning, ship returning/NULL → settles home (on-demand 0151 settle + cron-poll backstop, ship reconciled home by the 0050 cron) → `send_main_ship_expedition` leg 2 (asserted a FRESH fleet row) → stop 2 transforms the leg-2 row identically — each stop owns exactly its own trip, twice over.
- OSN family** (the live-reachable defect; `main_ship_space_movements / command_main_ship_space_stop`): anchored departure via `command_main_ship_space_move_to_location` → stop 1 with a fresh key (outcome: 'stopped', movement stopped/player_stop, ship HELD stationary/in_space at its own coordinates) → leg 2 re-departs FROM the held-in-space state as a NEW movement → stop 2 with a second fresh key halts the SECOND movement (its own `movement_id`) — the exact reported "second leg" scenario, proven to stop. Wrapper calls `try` the 0083 (`p_main_ship_id`) shape first and fall back to the pre-0083 shape (schema-cache-miss fallback).
- Regression probe (documents WHY slice 1 was required):** on the fresh in-transit leg 2, a stop submitted with the PREVIOUSLY-CONSUMED leg-1 key is asserted to REPLAY the leg-1 receipt verbatim (the OLD `movement_id` in the envelope) and to settle NOTHING — leg 2 stays moving. That replay was the live "second Stop no-ops" bug; the probe pins the server contract (receipts are correct-by-design idempotency) so the fresh-key-per-leg client fix is provably the right layer. The probe is OSN-only by nature: the legacy stop carries no request key (idempotent by state).

Idiom (mirrors `verify-mainship-legacy-dock-travel.mjs` verbatim): `loadEnv/admin/newUser/` `poll/setCfg`, up-front capture of `travel_scale/min_travel_seconds` (set fast for the run, restored in `finally`), shared `teardownVerifier` for user cleanup, \$11-\$13 SKIP-loudly probes (commissioning absent, target-legal probe absent, no second dockable port). **NO capability flag is toggled — stricter than the sibling:** `mainship_send_enabled` and `mainship_space_movement_enabled` are READ ONLY; a family whose flag is dark on the target DB is SKIPPED loudly instead of force-enabled (`teardownVerifier` is passed `flag: null`). Exit contract: 1 on any failed assertion; 2 when anything was skipped (required capability absent / a family dark — "not fully proven"); 0 only when both families ran green.

Verification of this step (honest): `node --check scripts/verify-stop-roundtrip.mjs` → OK. `npm run build` green. `npm run lint` → the same 22 pre-existing errors in untouched files (the new `.mjs` sits outside ESLint's `**/*.ts,tsx` coverage; no `ts/tsx` file touched this slice). **DB execution deferred:** no `SUPABASE_SERVICE_ROLE_KEY` in this sandbox AND network egress is blocked (the 0148-0153 precedent) — the verifier exits 2 by design without the key. Authored + statically reviewed only. `docs/SYSTEM_BOUNDARIES.md` unchanged (verifier-only slice — no table, writer, constraint, or cross-system call changed).

MIGRATION-APPLY / ENABLE-TIME CHECKLIST ADDITION (the human owner's gate): after applying the pending migrations (0152→0153) and/or whenever the stop families are enabled on the target DB, ALSO run `npm run verify:stop-roundtrip` — confirms `send → stop → send → stop` lands on both families and the consumed-key replay settles nothing. (Supplements the slice-3 checklist in the MAINSHIP LEGACY SPATIAL-STATE FIX entry below; a family dark on that DB skips loudly with exit 2.)

2026-07-06 — STOP/MOVE FIX, slice 1: consumed idempotency keys cleared on success (client-only)

Bug (LIVE — the reported "second Stop no-ops"): the three pure OSN command controllers kept their idempotency `requestId` after a SUCCESS. The server receipt idempotency is correct-by-design (`mainship_space_stop` replays the stored `result_json` verbatim for a matching (`main_ship_id`, `request_id`) — 0067:695-704 — and Stop's canonical payload hash is CONSTANT, so a stale key can't even conflict), and the controller instances survive across trips (memoized on `[mainShipId]`; `PortNavPanel/GalaxyMap` stay mounted, returning `null` between trips). So the second Stop on a NEW transit resubmitted the FIRST stop's consumed key → the server replayed trip 1's success envelope → the new movement kept flying while the UI showed "Stopped in open space." — a silent no-op. Full root-cause trace: `STOP_UIRESTRUCTURE_RECON.local.md` §D. Same class, siblings: `portMoveCommand` (re-travel to the SAME destination after a success replayed the old receipt → no new movement; live-reachable) and `spaceMoveCommand` (identical idiom; dark behind the 0070 coordinate gate). The legacy stop (`useLegacyStopTransitCommand` → 0149) does NOT share the class: per-trip fleet keying + idempotent-by-state server (no receipts) — clean post-0152, untouched.

Fix (client-only; NO migration, NO server code, NO flag — the server behaves as designed): each controller's `submit()` success branch now ALSO sets `requestId: null` — the key is consumed by the success. Error/catch branches still keep the key, so a retry-after-error stays idempotent (same key), while the NEXT command after a completed one always generates a fresh key.

- `src/features/map/spaceStopCommand.ts` — `createSpaceStopController.submit res.ok` branch (+ the now-corrected key-lifecycle comment). ONE shared controller serves BOTH `useSpaceStopCommand` (the live-reachable OSN stop — `PortNavPanel/GalaxyMap`) and `useLegacyStopTransitCommand`, so this single change repairs the reported second-leg OSN stop.
- `src/features/map/portMoveCommand.ts` — `createPortMoveController.submit res.ok` branch (+ the `PortMoveState.requestId` comment).
- `src/features/map/spaceMoveCommand.ts` — `createSpaceMoveController.submit res.ok` branch, preserving the existing `serverTarget` reconciliation (+ the `SpaceMoveState.requestId` comment).

Deliberate omissions (considered, NOT done): (1) movement-id re-keying of the OSN stop hook — redundant once the consumed key is cleared on success (every trip already gets a fresh key); speculative plumbing out of this slice's scope. (2) A shared "idempotent submit" helper across the three controllers — they are pre-existing independent surfaces with distinct state shapes; a one-field `requestId: null` in each controller's own success branch is not a duplicated non-trivial block.

Tests (updated to the corrected contract): `tests/spaceStopCommand.spec.ts` / `tests/portMoveCommand.spec.ts` / `tests/spaceMoveCommand.spec.ts` — after a successful `submit()` `state.requestId` is `null`, and the NEXT submit (new trip / re-selected SAME destination) calls `genRequestId` again and sends a DIFFERENT key; the pre-existing error/catch retry cases (same key reused) are unchanged and still pass.

Verification (honest): `npm run build` (tsc + vite) green; `npm run lint` green on the five touched files (the suite's 22 pre-existing errors in untouched files are unchanged); `verify:osn:osn4` + `verify:osn:port` + `verify:osn:s6c` (the three controller spec files) green. No DB needed — this slice is pure client logic. `docs/SYSTEM_BOUNDARIES.md` unchanged (no table, writer, constraint, or cross-system call changed). **NEXT SLICE:** the end-to-end `send → stop → send → stop` verifier (`verify:stop-roundtrip`, both families — see the recon §D.2 assertion list).

2026-07-06 — MAINSHIP LEGACY SPATIAL-STATE FIX, slice 3: the end-to-end round-trip verifier

New verifier `scripts/verify-mainship-legacy-dock-travel.mjs` (+ `package.json` script `verify:mainship-legacy-dock`) — proves the EXACT live scenario the hotfix targets, end-to-end:

1. `commission_first_main_ship` (NOT `ensure_main_ship_for_player`) → asserts the canonical DOCKED start (`status='stationary'`, `spatial_state='at_location'`, `space_x/y` NULL) — the state the live bug fired from.
2. Picks destinations from the world map via `mainship_space_location_target_legal` (admin RPC): a DOCKABLE port D (`ok:true`, distinct from the current dock) AND a NON-dockable active 'none' safe-zone N (`ok:false` — Safe Rally Point / Quiet Drift); dies loudly if either kind is missing.

3. **Regression guard 1 (the reported live bug):** `docked → move_main_ship_to_location(fleet, D)` returns NO error (pre-0152: `ss_at_location_status` violation) and the ship drops to legacy in-flight (`traveling / spatial_state NULL / coords NULL` — `mainship_mark_legacy_in_flight`).
4. Settles the arrival (on-demand legacy settle + cron-poll backstop) → fleet present at D, active presence at D, and the SHIP re-docked canonically (0153's shared `mainship_mark_docked_at_location`) — the `docked→send→travel→arrive→docked` loop closed with zero constraint violation.
5. **Non-dock fallback:** `D→N` settles with the fleet present at N and the ship staying `spatial_state=NULL` (coherent `legacy_present`; nothing writes the ship).
6. **Regression guard 2 (the second live bug):** re-docks at D, then `request_main_ship_return` from the DOCKED ship returns NO error → ship returning / `spatial_state NULL` → settles home (fleet completed).
7. **Constraint guard (BEHAVIORAL — documented in the script header):** PostgREST exposes no `pg_constraint` path and the repo ships no introspection RPC, so instead of a metadata query the guard attempts each illegal direct write (service-role — bypasses RLS, never CHECKS) and asserts Postgres REJECTS it naming the constraint. Each probe runs from a ship state where EXACTLY ONE lifecycle constraint is violated, so the reported constraint name is deterministic (a docked ship's `spatial_state=IN_TRANSIT/home/` destroyed would violate `stationary_spatial_state` too — ambiguous which fires): from DOCKED — `ss_at_location_status` (the verbatim pre-0152 live write `status→traveling`) and `stationary_spatial_state` (`spatial_state→NULL`); from in-flight TRAVELING — `ss_home_status`, `ss_destroyed_status`, and `ss_in_space_status` (`in_space` carries coords 1,1 so the 0054 `space_coords` rule is satisfied and only the lifecycle rule fires); from RETURNING — `ss_in_transit_status` (from traveling that write would be the LEGAL OSN pair and would succeed). All SIX covered — proving they still EXIST and ENFORCE (strictly stronger than presence). The fix corrected the WRITERS, never the constraints.

Proof principle (script header): every RPC returning without error across steps 3–6 IS the constraint-never-violated proof — a violating write raises inside the RPC and fails it (that raise WAS the live bug). **Idiom:** mirrors `verify-mainship-move.mjs` (`loadEnv/admin/newUser/poll/setCfg`, up-front capture of `mainship_send_enabled/travel_scale/min_travel_seconds`, shared `teardownVerifier` restore in `finally` — no re-implemented `teardown`; NO OSN flag is toggled). Deployment probes SKIP loudly (§11–§13 idiom) when 0152/0153's helpers or commissioning (starter ports) are absent.

Verification of this step (honest): `node --check` → OK. `npm run lint` → 22 errors, ALL pre-existing in `src//tests/ ts+tsx` files this step never touched (ESLint's config lints `**/*.ts,tsx` only — the new `.mjs` is outside its coverage; `git status` shows zero `src/tests` modifications, so HEAD lints identically). DB execution deferred: no `SUPABASE_SERVICE_ROLE_KEY` in this sandbox AND network egress is blocked (the 0148–0153 precedent) — the verifier exits 2 by design without the key.

MIGRATION-APPLY VERIFICATION CHECKLIST (the human owner's gate — supersedes the slice-1/2 lists):

1. Apply migrations **0152 → 0153** (forward-only, in order, after 0148–0151).
2. `npm run verify:m2 && npm run verify:m3 && npm run verify:m4 && npm run verify:m5 && npm run verify:m45` (suite stays green), plus `npm run verify:mainship-send / npm run verify:mainship-move`.
3. `npm run verify:mainship-legacy-dock` — confirms a docked ship departs/returns and re-docks with no CHECK violation (regression guards for both live bugs), the non-dock fallback, and all six 0055 constraints still enforcing.
4. Confirm no client execute grants changed (the four legacy RPCs keep `authenticated`; both 0152/0153 helpers and the re-created internals stay client-revoked).

2026-07-06 — MAINSHIP LEGACY SPATIAL-STATE FIX, slice 2: legacy arrival docks the ship (migration 0153)

Gap closed: `movement_settle_arrival`'s location branch (0151) settled the FLEET (`fleet_set_present`

- `presence_create`) but never the SHIP — a legacy-arrived main ship sat `status='traveling', spatial_state=NULL` while "present" (decision-doc §2 writer #6). Post-0152 a `docked→send→arrive` trip would end in `legacy_present` instead of returning to the canonical docked pair.

Migration `20260618000153_mainship_legacy_arrival_docks_ship.sql` (forward-only; no shipped file touched).

1. `mainship_mark_docked_at_location(p_main_ship_id)` — THE one canonical docked-ship write (`status='stationary', spatial_state='at_location', space_x/y=NULL`; Main-Ship-owned leaf; SECURITY DEFINER; service_role-only, clients revoked) — the arrival-side mirror of 0152's `mainship_mark_legacy_in_flight`. Shared by BOTH docking routes (the OSN Dock-0 writer AND the legacy arrival settle); the docked-pair write now exists in exactly ONE place. **RETIREMENT:** when the legacy `fleet_movements` main-ship family is replaced by the OSN coordinate domain, Dock-0 becomes the sole caller and the write may fold back inline (same condition as 0152's helper).
2. `mainship_space_dock_at_location` re-created from its LATEST shipped body (0067:499 — the anchor-backed Dock-0 re-creation; the 0061 birth body is superseded — the 0152 latest-body precedent) with ONLY the dock-branch inline ship write (0067:618–621) swapped for the helper call; the terminal-failure `in_space` write and everything else are byte-identical (scripted diff: two hunks — the swap and the honestly amended settlement-timestamp comment). **Accepted micro-delta**

(documented in-file): the dock branch's SHIP `updated_at` is now stamped `now()` by the shared helper instead of `v_settled_at` — bookkeeping-only; the settlement record (movement `resolved_at` + fleets stamps) keeps `v_settled_at` exactly.

3. **movement_settle_arrival re-created from 0151 body-verbatim** (scripted diff: two hunks — the `v_main_ship` uuid declare and the new block) with the location branch gaining, after `presence_create`: look up `fleets.main_ship_id`; if non-NULL AND `mainship_space_location_target_legal(target)` passes → `mainship_mark_docked_at_location(ship)` (coherent `at_location`: `fleet_set_present` already set `present/location-mode/active_movement_id=NULL`, `presence` matches); otherwise write NOTHING — a main-ship fleet at an active 'none' but NON-dockable target (seed safe-zones Safe Rally Point / Quiet Drift — the reachable §3 case) stays in the constraint-legal `legacy_present` NULL representation from its 0152 departure write, and ordinary unit fleets (`main_ship_id` NULL) are untouched. The `target_type='base'` branch, `process_fleet_movements`, `process_mainship_space_arrivals`, and both on-demand settle RPCs are UNTOUCHED — they delegate here and inherit the fix.
4. **Execute surface:** CREATE OR REPLACE preserves grants on the two re-created internals (both were client-revoked at creation); only the NEW helper is locked (revoke public/anon/authenticated → grant `service_role`). `movement_settle_arrival`'s new call to the `service_role` predicate runs as function owner inside SECURITY DEFINER — NO client grant surface changes anywhere.

Verification (honest; environmental precedent unchanged — 0148–0152 "authored, reviewed, NOT applied"): no `psql/docker/supabase` CLI in this sandbox and network egress is blocked (supabase host probe → HTTP 000), so `verify:m*` cannot reach any database from here. Statically verified: the two scripted per-function diffs above (only the intended hunks); 3 `create or replace / 3 $$`; / SECURITY DEFINER + `set search_path = public` on all 3; exactly TWO update `main_ship_instances` in the file (the helper's docked write + the verbatim terminal-failure `in_space` write); the docked-pair write appears exactly ONCE; grant statements touch ONLY the new helper; no blanket re-lock. **HUMAN CHECKLIST (the owner's gate — never this loop):** (1) apply 0153 after 0152, forward-only; (2) re-run `verify:m2.m5,m45` + `verify:mainship-send` / `verify:mainship-move` + the OSN settle verifier, and run `npm run verify:mainship-legacy-dock` (slice 3 — the round-trip + regression-guard verifier; see the slice-3 entry's canonical checklist); (3) confirm no client execute grants changed post-apply (the four legacy RPCs keep authenticated; the two re-created internals and both helpers stay client-revoked); (4) confirm a commissioned ship's docked→send→travel→arrive-at-port trip ends `status='stationary'`, `spatial_state='at_location'` with no CHECK violation, and an arrival at Safe Rally Point / Quiet Drift ends `legacy_present` (`spatial_state=NULL`) — item (4) is exactly what the slice-3 verifier automates.

2026-07-06 — MAINSHIP LEGACY SPATIAL-STATE FIX, slice 1: departure/halt pair-writes (migration 0152)

Bug (LIVE): every legacy main-ship status writer was `spatial_state`-blind. Commissioned ships are canonically docked (`status='stationary'`, `spatial_state='at_location'` — 0072, ungated/live) with their fleet 'present', which is exactly what the legacy send surface accepts — so `move_main_ship_to_location` (0053:105, sets 'traveling') and `request_main_ship_return` (0051:213, sets 'returning') left `spatial_state='at_location'` behind and tripped the 0055 `ss_at_location_status` CHECK, aborting the whole RPC. Full recon/audit + design decision: `docs/MAINSHIP_LEGACY_SPATIAL_STATE_FIX.md` (the constraints are CORRECT; the writers were the defect).

Migration `20260618000152_mainship_legacy_in_flight_spatial_state.sql` (forward-only; no shipped file touched).

1. `mainship_mark_legacy_in_flight(p_main_ship_id, p_status)` — THE one legacy in-flight ship write (Main-Ship-owned leaf; SECURITY DEFINER; `service_role`-only, clients revoked): guards `p_status ∈ ('traveling', 'returning')` (raises otherwise), then one statement sets `status = p_status`, `spatial_state = NULL`, `space_x = NULL`, `space_y = NULL`, `updated_at = now()`. The legacy family lives entirely in the `spatial_state=NULL` domain (decision doc §5) — it never claims `in_transit/in_space`. **RETIREMENT CONDITION:** the helper retires together with its four callers when the legacy `fleet_movements` main-ship family is replaced by the OSN coordinate domain.
2. **Four writers re-created body-VERBATIM with ONLY the bare status UPDATE swapped for the helper call** (scripted extraction + diff against 0051/0053/0149 shows exactly one hunk per function — the swap itself; every gate, precondition, comment, and signature is byte-identical): `send_main_ship_expedition` + `request_main_ship_return` (0051 bodies), `move_main_ship_to_location` (0053 body), `command_main_ship_stop_transit` (0149 body). The `mainship_send_enabled` gate lines are verbatim-unchanged (no flag created/read differently/flipped).
3. **Execute surface:** CREATE OR REPLACE on an existing function PRESERVES owner + grants, so the four RPCs keep their authenticated EXECUTE automatically — deliberately NO blanket revoke `execute` on all functions re-lock (that idiom is for migrations adding NEW client RPCs). Only the NEW helper is locked: revoke from public/anon/authenticated, grant to `service_role`.

Verification (honest; environmental precedent unchanged from 0148–0151 "authored, reviewed, NOT applied"): no `psql/docker/supabase` CLI in this sandbox AND network egress is blocked (`verify:m2/m3` fail with `fetch failed` on plain world-map reads; `example.com` is equally unreachable — the suite cannot reach ANY database from here, and the handful of `m2` "

write blocked ✓" lines are fetch-failure false positives, not assertions). The migration was therefore verified statically: the per-function diff proof above; 5 create or replace / 5 \$\$; terminators / SECURITY DEFINER + set search_path = public on all 5 / exactly 4 helper call sites / exactly ONE update main_ship_instances in the file (inside the helper); grant statements touch ONLY the helper. **HUMAN CHECKLIST (the owner's gate — never this loop):** (1) apply 0152 after 0148–0151, forward-only; (2) re-run verify:m2..m5,m45 + verify:mainship-send / verify:mainship-move; (3) confirm the four RPCs still hold authenticated EXECUTE post-apply (ACL query or an authenticated call probe) — expected preserved by CREATE OR REPLACE semantics; (4) confirm a commissioned (at_location) ship can now depart/return via the legacy surface without a CHECK violation. **NEXT SLICE (not in 0152):** the ARRIVAL half — movement_settle_arrival's location branch settling the ship (docked pair via a transition shared with the OSN dock writer; legacy fallback for non-dockable 'none' targets) + the docked→send→travel→arrive→docked verifier.

2026-07-06 — VISUAL FOLLOW-ON item 4 (final sweep): palette-literal inventory; last live straggler converted

Sweep: grepped src/ (*.ts/*.tsx) for white/, black/, plain text/bg/border-white|black, slate-, indigo-, rose-, red-, emerald-, amber-, cyan-, sky-, violet-, zinc-, gray-, and raw hex in JSX attributes. Every hit classified live-visible vs. dark before touching anything (each dark panel's fail-closed server-lit gate re-verified at its return null site).

Converted (the ONLY live-visible straggler): src/app/RequireAuth.tsx — the auth-gate loading screen's text-white/40 → text-ink-muted. One class; no logic touched.

ACKNOWLEDGED DEBT (standing note — intentionally NOT converted): the dark, flag-gated panels retain incidental palette literals because they render null in production (server-rejected capability + client fail-closed gate), so they have zero visual surface today:

- investment/InvestmentPanel.tsx (22 hits; location_investment_enabled dark)
- modules/ModulesPanel.tsx (20; module_crafting_enabled/module_fitting_enabled dark)
- map/SpaceMoveTarget.tsx (18; mounts only behind canTarget — server coordinate_travel_available + mainship_space_movement_enabled, both dark)
- mining/MiningPanel.tsx (11; mining_enabled dark)
- exploration/ExplorationPanel.tsx (10; exploration_enabled dark)
- captains/CaptainsPanel.tsx (9; captain_assignment_enabled dark)
- events/WorldEventsPanel.tsx (8; phase20_polish_enabled dark — feed empties, panel nulls)
- captains/RecruitCaptainPanel.tsx (7; captain-system server-lit visibility, dark)
- map/DevFixedSpacePreview.tsx (3 hex SVG strokes; import.meta.env.DEV-only — statically compile-time eliminated from vite build, never shipped at all) **RETIREMENT CONDITION:** each panel is converted to the design system in the SAME change that lights its capability flag (the "lit-path" work for that feature) — a panel must not go live wearing off-system chrome. Until lit, do not restyle them speculatively.

Presentational-only: no behavior/handler/data-testid/route/flag/backend change anywhere in the sweep; src/components/ui/** and @theme tokens untouched. Post-sweep grep: the only remaining palette-literal files are exactly the nine listed above. npm run build (incl. tsc -b) green.

2026-07-06 — VISUAL FOLLOW-ON item 3: CombatReportPage restyled onto the shared design system

Done: src/features/combat/CombatReportPage.tsx (/reports) now composes the design-system primitives — PageHeader (title/subtitle + buttonClasses('ghost', 'sm') back link, matching the Dashboard idiom), one Card per battle row, Badge win/loss pill (success won / danger lost), Notice for loading/empty (neutral) and error (danger) callouts, Button ghost sm round-log toggle, and a single local Fact helper for the repeated "Label: value" detail lines. Nested round-log container on the surface-2/edge layer ramp. Zero raw palette literals remain (grep-verified).

Presentational-only: toggle() per encounter_id, ticks load into RoundLog on expand (RoundLog itself untouched — out of scope), won-detection (escaped/completed), ships/metal/locName formatters, formatDateTime/formatDuration, and loading/empty states all preserved; no data-testid existed in the file. No route, backend, RPC, flag, or src/components/ui/** change. npm run build (incl. tsc -b) green.

2026-07-06 — VISUAL FOLLOW-ON item 2: legacy /map list view retired (fully superseded by /galaxy)

Decision (human design authority): the M2-era read-only list browser at /map is fully superseded — /galaxy (GalaxyMapScreen) shows the same world data plus the main ship, movements, dock services, stop controls, and a detail panel covering every metadata field /map showed (type/sector/zone/coordinates/ status/difficulty/reward tier/worldstate). Keeping a second, un-modernized map surface is split-brain debt; clean deletion (like the prior pass's retired legacy UI) over restyling a redundant page.

Deleted (whole dead-code chain, each importer-verified to zero remaining callers first):

- src/features/map/MapPage.tsx (the /map screen)
- src/features/map/LocationPanel1.tsx (imported ONLY by MapPage)
- src/game/worldstate/danger.ts (+ its now-empty worldstate/ dir) — its only importers were MapPage and LocationPanel; its former send-fleet consumer (ExpeditionCommand) was retired 2026-07-05.

Removed references: the /map route + MapPage import in src/app/App.tsx; the two "List view" links (Dashboard.tsx, GalaxyMapScreen.tsx headers — action rows remain well-formed).

Kept (still live): mapApi.ts (fetchWorldMap/fetchLocationStates — used by useGameState, useGalaxyMapData, CombatReportPage) and mapTypes.ts (used across map/dashboard/combat/fleets/portentry).

Presentational/dead-code removal only: no backend, RPC, migration, or flag change; /galaxy behavior untouched. Grep confirms zero remaining src/+tests/+scripts/ references to /map, MapPage, LocationPanel, or worldstate/danger. npm run build (incl. tsc -b) green; bundle 579.99 kB → 573.58 kB.

2026-07-06 — VISUAL FOLLOW-ON item 1: AuthPage restyled onto the shared design system

Request: follow-on to the 2026-07-05 visual-modernization pass (item 5) — convert the remaining un-modernized surfaces, starting with the sign-in/sign-up screen.

Done: src/features/auth/AuthPage.tsx now composes the design-system primitives per the src/components/ui/README.md rule — Card/CardHeader panel, Button (submit primary with busy/busyLabel; mode-toggle ghost sm), Notice (danger error / success notice), and token-only input chrome (bg-surface-2 / border-edge / text-ink / accent focus, touch-sized min-h-11). Zero raw palette literals remain in the file (grep-verified).

Presentational-only: no behavior change — mode toggle, handleSubmit, authStore signIn/signUp, signup notice + auto-switch, navigate('/', { replace: true }), busy disable, required/ minLength={6} all preserved. No flag, route, RPC, or backend change; src/components/ui/** and @theme tokens untouched. npm run build green.

2026-07-05 — UX CLEANUP PASS COMPLETE — final consolidation: full-suite check + PR-ready handoff (no product changes)

The goal, delivered on autopilot/20260703-064048 **(every slice individually reviewed and passed):**

1. **Legacy UI retired** — TrainShipsPanel / BuildQueuePanel / ExpeditionCommand removed with their whole dead-code chain; backend RPCs (train_units, cancel_build_order, send_fleet_to_location) + crons intentionally intact, now client-unreferenced.
2. **Honest docking UX** — waypoint-aware Finish-Docking affordance (at_waypoint), truthful ineligible_port copy; server target_legal untouched as sole authority (frontend-only).
3. **Consistent in-transit stop** — fleet-domain halt→symmetric-return-home (command_main_ship_stop_transit, 0149) + the OSN PortNav stop hardening; ONE shared stop UI/controller for all families.
4. **One-word location names** — forward-only data migration 0148 (Refuge/Snare/Reaver/Lull/Blackden Haven/Slagworks/Driftmarch); every caller/test/doc updated same-step; world-map name checks field-anchored against the Haven<"Outer Haven" substring hazard.
5. **Cohesive visual modernization** — the ONE design system (@theme tokens + src/components/ui/ primitives + README rule) across all four most-seen screens: Command Center · Galaxy Map (incl. the elevated map look: deep-space backdrop, semantic markers, legend) · dock/port · market (dark; incidental-styling exception).

6. **On-demand arrival settlement** — both movement families (`command_main_ship_settle_arrival 0150`; `movement_settle_arrival` extraction + `command_main_ship_settle_arrival_legacy 0151`), cron primitives reused verbatim, idempotent by state, unified client due-trigger (~34s → ~an RPC round-trip).

Migrations 0148–0151: authored, reviewed, NOT applied to any database (the human gate). No feature flag was flipped anywhere in the pass; every new capability gates on its domain's EXISTING flag. Nothing merged or deployed.

Final full-suite state (this consolidation run; exact counts).

- `npm run build` — green.
- `verify:m2` — **13/13 PASSED**.
- `verify:m3` — **13/13 PASSED**.
- `verify:m4` — **36/40**: the SAME four pre-existing combat-pacing failures as every run this goal (`wave pacing – max 0 ticks/wave · damage-no-loss – not observed · wave HP decreasing – no mid-wave tick found · not one-shot – wave HP <= player damage`). Pre-existing/out-of-locked-scope (combat/reward correctness untouched; 0148–0151 are additive AND unapplied, so the deployed engine is byte-identical to before the goal). **No NEW failure anywhere.**
- `verify:m45` / `verify:m5` — **NOT EXECUTABLE in this environment** (honest report, not a pass claim): both hard-require `SUPABASE_SERVICE_ROLE_KEY`, which `.env.local` here deliberately lacks (anon-only), and no CI workflow runs the engine suites (they have always run with the human's server-side key). **No-regression argument by construction:** `m45` exercises `train_units/process_build_queue/ cancel_build_order` and `m5` exercises `worldstate_tick` — this goal changed NEITHER surface (item 1 removed only client UI + the browser spec; the `m45` `NODE` engine script is untouched), and the live DB the suites run against is unchanged by the goal (all migrations unapplied). Their outcome today is therefore identical to before the goal; the human should run both once with their key as part of the merge check below.

REGRESSIONS ATTRIBUTABLE TO THIS GOAL: NONE (every runnable suite green or at its documented pre-existing baseline; the two non-runnable suites are argued unchanged by construction and delegated).

Human-gated remainder (the explicit handoff):

1. Run `verify:m45` + `verify:m5` once with the service-role key (pre-merge confirmation).
2. Apply migrations **0148 → 0151** (forward-only, in order) to the database; then re-run the DB-dependent proofs that probe-skip today — `verify-mainship-move.mjs` §11 (stop-transit), §12 (OSN on-demand settle), §13 (legacy on-demand settle) — and the name-asserting verifiers (`postreveal-verify`, `osn-postenable-verify`, `catalog verifiers`), which expect post-0148 names.
3. Retire the `verify:m2` rename-pair TRANSITIONAL (collapse each old/new name pair to the new name) once 0148 is applied to every verified environment — the retirement condition recorded in-line.
4. Visual smoke-check the four converted screens (Command Center, Galaxy Map, dock/port, market — the last needs a local-only trade-flag enable to see).
5. Decide the `verify:m4` combat-pacing baseline (out of this goal's locked scope by design).
6. The merge itself.

Docs. `docs/SYSTEM_BOUNDARIES.md` needs NO change in this step (verification + documentation only) — confirmed. Branch state: **SAFE FOR HUMAN MERGE REVIEW.**

2026-07-05 — UX CLEANUP (item 5, slice D) — market surface on the design system (COMPLETES item 5's screen coverage; presentational-only)

Request. Final item-5 slice: convert `MarketPanel` + `ShipSwitcher` to the shared tokens/primitives. Market is DARK behind `trade_market_enabled` (client `TRADE_MARKET_ENABLED` + server rejection), so this is the allowed incidental-styling application to a dark surface — the flag is NOT flipped; production stays byte-invisible until the human lights it. No RPC, data flow, gate, buy/sell/selection logic, or data-testid change.

Converted (the overlay-block idiom; warning tone = the trade identity).

- `MarketPanel` — token container; wallet/cargo readouts and all prices/quantities on `font-mono tabular-nums` (the numeric token); offers table on edge borders with `ink-faint` headers; the qty input on `surface-2/edge` tokens; **Buy → the shared Button success variant / Sell → primary** (testids preserved via prop spread; per-row disable while in flight unchanged); row errors `text-danger`; the fail-closed unavailable note `ink-faint`. The buy/sell column-naming comment kept verbatim.
- `ShipSwitcher` — token container; sole-ship entry as a soft `warning/15` chip; N-ship selection buttons (selected `bg-warning text-app`, idle `surface-2` with hover); `ShipMeta` now inherits the entry's text color at reduced opacity instead of a fixed gray (correct on both selected and idle backgrounds).

One shared addition, no duplication: `Button` gained the `success` variant (token-driven, `bg-success text-app` + hover step) — added ONCE in `src/components/ui/` for the Buy action; no one-off styles introduced. (An early draft overrode `Button` padding per-row; removed — conflicting utilities have no guaranteed order. Standard `sm` sizing used.)

Preservation (proof). `data-testid` counts byte-identical before/after (stash comparison: MarketPanel 6, ShipSwitcher 4); grep-proven zero old palette literals in both files; all conditional states/behavior unchanged.

ITEM 5 SCREEN COVERAGE COMPLETE: Command Center (A) · Galaxy Map + map-look elevation (B) · dock/port (C) · market (D) — all on the ONE design system (@theme tokens + `src/components/ui/`).

Doc-sync. `docs/SYSTEM_BOUNDARIES.md` needs NO change — confirmed: presentation only. This entry added.

Verify. `npm run build green`; `tradeReasonMessage` + `mainshipStatusLabel` specs 12/12 (the market surface's pure-logic suites); `verify:m2/verify:m3` unaffected — results in the step report. Visual drive not possible without a flag change (correctly not made) — build + specs are the machine proof. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — UX CLEANUP (item 5, slice C) — dock/port surfaces on the design system (presentational-only)

Request. Apply the slice-A tokens/primitives to the dock/port screens: `DockServicesPanel`, `PortEntryPanel`, `PortNavPanel`. Presentational ONLY — no RPC, data flow, flag, item-2 affordance logic, stop-control wiring, or `data-testid` change.

State correction against the slice plan (honest accounting). `PortEntryPanel` — including the removal of its duplicated local `CARD` constant in favor of the shared `Card` — was ALREADY fully converted in slice A (it is mounted on the Command Center, so slice A's "no half-converted screen" rule pulled it in; that entry records the `CARD` retirement). Re-verified clean this step (zero old literals; 20 testids intact). This slice therefore converts the remaining two panels.

Converted (the compact map-overlay idiom from slice B: token-styled container, primitives for interactive elements).

- `DockServicesPanel` — success-toned container (the "safely docked" state), title `text-success`, service chips `bg-surface-2 text-ink-muted`, empty-state `ink-faint`. Layout/truncation caps unchanged.
- `PortNavPanel` — accent-toned container (the OSN travel surface); destination list buttons on tokens (selected `bg-accent text-app`, idle `bg-surface-2` with hover); the confirm action is now the shared `Button` (primary/sm, busy state — `testid port-nav-confirm` preserved via prop spread); error `text-danger`; travel line `text-accent`. The item-3 stop mount and the `destName`-gated label are untouched.

No new primitive was needed — `Button/Notice/Card` from slices A/B cover these panels; the semantic tokens replace every raw rose/amber/emerald/sky literal.

Preservation (proof). `data-testid` counts byte-identical before/after (stash comparison: `DockServicesPanel` 5, `PortNavPanel` 6, `PortEntryPanel` 20); all conditional states/behavior unchanged; grep-proven zero old palette literals across the three panels. Only the MARKET slice (`MarketPanel` + `ShipSwitcher`, dark behind `trade_market_enabled`) remains for item 5.

Doc-sync. `docs/SYSTEM_BOUNDARIES.md` needs NO change — confirmed: presentation only. This entry added.

Verify. `npm run build green`; `verify:portentry` 32/32 (drives `PortEntryPanel`'s affordance logic); `verify:osn:port` 25/25 (readiness + port-move logic) and `verify:osn:osn4` 9/9 (stop surface) as regression; `verify:m2/verify:m3` unaffected — results in the step report. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — UX CLEANUP (item 5, slice B) — Galaxy Map on the design system + map-look elevation (presentational-only)

Request. Apply the slice-A tokens/primitives to the Galaxy Map (the goal's explicit "improve the galaxy-map look") — no new palette, no parallel styling system; presentational ONLY (no RPC, data flow, flag, movement/selection logic, or `data-testid` change).

Screen chrome → primitives/tokens. `GalaxyMapScreen` (header nav → `buttonClasses`, states/aside/ overlay bar → tokens), `MainShipPreview` (`Card/Badge/Notice/Button` — recall stays disabled-gated), `MainShipCommand` (`Button/Notice/SectionLabel`; confirm/cancel flow identical), `SpaceStopControls` (warning-toned surface + the shared warning `Button`). `Button` gained ONE size, `icon` (square), consumed by the map's zoom cluster — no other primitive additions were needed.

Map canvas elevation (all token-driven — SVG consumes the same @theme tokens via `var(--color-*)`, which Tailwind v4 emits as `:root` custom properties; NO new tokens were needed).

- Backdrop: subtle deep-space radial glow (`surface→app`) + a faint edge grid pattern — replaces the flat hex fill; the container wears the standard rounded-card border-edge shadow-card chrome.
- Location markers (`LocationMarker`): semantic identity — hostile (`pirate_hunt/pirate_den`) → danger, safe (`safe_zone`) → success, dockable port (`trade_outpost`) → accent plus a second "hub" ring so ports read differently from waypoints at any zoom; resource/event → warning; derelict → muted. Each node: soft always-on identity halo + core dot + app-colored stroke;

NEW hover halo (group-hover) and a solid selected ring; labels carry an app-colored paint-order halo for legibility over the grid.

- Routes: FleetMovementLine outbound → warning, return → accent (ETA labels haloed); SpaceRouteLine outbound token + haloed ETA. In-transit emphasis is consistent across both families.
- Main-ship marker: state-toned chevron (outbound warning / returning accent / settled success) inside an always-accent "this is YOU" halo ring — the player reads distinctly at any zoom. Home base diamond + label → accent tokens.
- NEW compact legend (bottom-left, pointer-inert) mirroring the marker semantics exactly: safe · hostile · port · home, merged with the existing hint line.

Behavior/structure preserved (proof). SVG coordinate math, camera, gesture/selection handlers, and every flag-gated mount untouched; data-testid counts byte-identical before/after per file (stash comparison: GalaxyMapScreen 5, LocationMarker 1, MainShipMarker 1, FleetMovementLine 1, SpaceRouteLine 2, SpaceStopControls 4, MainShipPreview 7, MainShipCommand 11). Grep-proven ZERO old palette/hex literals in any converted map file. The only spec-pinned visual attributes (fill="none" on the S6C crosshair + dev preview) are untouched. PortNavPanel / DockServicesPanel / MarketPanel / the dark activity panels stay on their current styling — the dock/port and market slices remain next.

Doc-sync. docs/SYSTEM_BOUNDARIES.md needs NO change — confirmed: presentation only, no table/writer/flag/RPC/edge. This entry added.

Verify. npm run build green; ALL map-related unit specs green — 129/129 across spaceRouteModel/spaceRouteLine/galaxyShipLayer/spaceStopCommand/resolveMainShipMarker/spaceMoveTarget/galaxyCamera/mainshipStatusLabel/devFixedSpacePreview/openSpaceTransform; verify:m2/verify:m3 unaffected (results in the step report). SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — UX CLEANUP (item 5, slice A) — the ONE shared design system + Command Center conversion (presentational-only)

Request. Final goal item, foundation slice: establish the single source of truth for styling (tokens + primitives) and prove it by fully converting the first real screen — the Command Center. Presentational ONLY: no RPC call, data flow, flag, migration, or data-testid changed; other screens untouched (they keep the default Tailwind palette until their slices).

Tokens (src/index.css @theme — Tailwind v4; the canonical source, documented in-line). Deep-space dark: color layers app (#0b1120, near-black blue — never pure black) < surface (#131c31) < surface-2 (#1c2742) + border edge; text ink / ink-muted / ink-faint (AA on the surfaces); ONE interactive accent (sky #38bdf8) + semantic success/warning/danger (light-400 weights, AA on dark; each with a -hover step so filled buttons run dark-text-on-bright-fill at ~8:1); --font-sans (system stack) + --font-mono (numeric readouts); --radius-card (1rem) + --shadow-card (soft elevation + hairline top highlight). Type conventions recorded (page text-2xl semibold, panel text-lg semibold, body sm, metadata xs, micro-labels via SectionLabel). body now consumes the tokens (the old hardcoded #070b16/#e6ecff removed).

Primitives (src/components/ui/ + barrel + README — "screens compose primitives, never re-define styles"). Built ONLY what the Command Center consumes (no speculative components): Button (primary/secondary/ghost/danger/warning · sm/md · busy/busyLabel; buttonClasses() for router Links), Card/CardHeader (the panel treatment; tone prop for feature identity tints; spreads data-testid/aria), Badge (semantic status pill), Meter (progress/integrity bar), Notice (inline tinted callout), SectionLabel, PageHeader. README documents tokens, primitives, conventions, and the single-source rule.

Command Center fully converted (no half-old/half-new). Dashboard.tsx (PageHeader + buttonClasses nav + Notice error), BasePanel, ExpeditionLauncher, MainShipPanel (Badge status, Notice warnings, warning-variant Repair, Meter progress), FleetStatusPanel (STATUS_STYLE class map → semantic STATUS_TONE Badge map), ActiveCombatPanel (danger-tone Card; its local Bar now wraps the shared Meter) + its children CombatEventLayer/RoundLog (token swap), CombatReportsView, PortEntryPanel (the local CARD const retired for the Card primitive), and the dark RankingPanel (incidental token pass — the locked-scope dark-feature styling exception; renders null in production regardless). Grep-proven: ZERO old palette literals (white/*, slate-*, indigo-*, emerald-*, amber-*, rose-*, red-*, sky-*) remain in any converted file; every data-testid preserved (counts re-checked per file); element roles preserved (panels stay <section> via Card; the PortEntry affordance wrappers keep their outer <div data-testid="port-entry-panel">).

Next slices. Galaxy map screen, dock/port surfaces, market — each converts to the same primitives; add primitives only as those screens need them.

Doc-sync. docs/SYSTEM_BOUNDARIES.md needs NO change — confirmed: no table, writer, flag, RPC, or cross-system edge changed; this is client presentation only. This entry added.

Verify. npm run build green; verify:portentry (drives the PortEntryPanel affordance logic) green; verify:m2/verify:m3 unaffected (backend suites) — results in the step report. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — UX CLEANUP — reconcile the stale `verify:m2 world-count pin` (test-only; suite green again, 13/13)

Request. `verify:m2` had been red (10/11) since the goal started: its "exactly 5 locations" pin predates the 3 starter ports (0066) and the human's production reveal. Test-only reconciliation — no migration, seed, or product code touched; `verify:m3/m4/m45/m5` untouched. No `SYSTEM_BOUNDARIES` change (no table/writer/system change).

What the pin now asserts (strict — never a loose count). `scripts/verify-m2.mjs` runs against the LIVE DB via `get_world_map()` (ACTIVE locations only), so the correct world shape is: the 5 waypoints (0002 — Refuge/Lull `safe_zone`, Snare/Reaver/Blackden `pirate_hunt`) always present with exact names+types; the 3 starter ports (0066 — Haven/Slagworks/Driftmarch `trade_outpost`) as an ALL-OR-NOTHING set (0 pre-reveal on a fresh seed, 3 post-reveal — the production state; a partial set always fails); total count = exactly 5 + revealed ports (no strays); type totals (3 `pirate_hunt` + 2 `safe_zone` + N `trade_outpost`).

TRANSITIONAL rename-pair matching (in-line note + retirement condition). The first green run exposed that production is post-REVEAL but pre-0148 (the item-4 rename is human-applied and still pending), so pinning only the new names would keep `m2` red until 0148 lands — and pinning old names would re-drift the moment it does. Each location is therefore matched under EXACTLY one of its two known names (the 0002/0066 seed name OR the 0148 one-word rename) with the type pinned; the output labels which era it saw. **Retirement:** collapse each pair to the new name only, once the human applies 0148 to every verified environment.

Verify. `verify:m2 13/13 PASSED` (live output labeled "pre-0148 seed names", 8 locations = 5 waypoints + 3 revealed ports); `npm run build green` (nothing else moved).

`verify:m4 36/40 baseline confirmed genuinely pre-existing (no action — combat correctness is out of locked scope).` The same four failures on every run this goal (steps 6, 8): `wave pacing — max 0 ticks/wave · damage-no-loss — not observed · wave HP decreasing — no mid-wave tick found · not one-shot — wave HP <= player damage`. Rationale: all four are combat-PACING observations against the live engine; no slice in this goal modified combat/reward code — the branch's only server-side artifacts are the additive, UNAPPLIED migrations 0148–0151 (rename data + stop/settle functions), so the deployed combat path is byte-identical to before this goal. The drift is live-environment tuning that predates the branch.

2026-07-05 — UX CLEANUP (item 6, part B) — on-demand LEGACY main-ship arrival settlement (migration 0151: helper extraction + RPC + unified client due-trigger)

Request. The legacy first-trip arrival (MainShipCommand home→port/waypoint, and every return leg) still waits up to ~30s of `process-fleet-movements` cron (0011) + a 3–4s poll. Highest-risk slice (it touches the core legacy movement cron): extract the cron's per-movement settle into ONE shared helper both the cron and a new narrow RPC call — no second settlement copy, combat path untouched, spine proven green.

Internals confirmed first (citations). The CURRENT `process_fleet_movements` body is **0096** (NOT 0030 — 0096 re-created it for the activity-agnostic `reward_source_type` deposit): `due scan status='moving' and arrive_at<now()` **FOR UPDATE SKIP LOCKED** (movement rows first, fleets updated after) → per-row: `location target → arrived + fleet_set_present + presence_create(activity)` (combat init is NOT a cron branch — it lives downstream in `activity_start`, 0008:52/0018, which routes `hunt_pirates` → Combat); `base target → arrived + unit merge + fleet_complete` (requires `status='returning'`, 0006:163) + reward deposit ONLY when payload non-empty AND source set; unknown → failed. Main-ship legacy movements can never reach the combat/reward parts: sends hard-reject non-'none' targets (0050:104/0053:71), the fleets carry zero units, and payload stays '{}'. Main-ship predicate = `fleets.main_ship_id is not null` (0050:185-187); `gate = mainship_send_enabled` (0050:73/0053:34).

Migration `20260618000151_legacy_settle_arrival_on_demand.sql` (3 parts).

- `movement_settle_arrival(p_movement)` (internal; revoked from all client roles) — THE extracted per-movement settle: the verbatim 0096 loop body behind a guarded locked re-read (`status='moving' AND arrive_at<now()` FOR UPDATE — a no-op re-take for the cron, which already holds the lock on a row it proved due (`now()` is `txn-constant`); the authoritative claim for the RPC).
- `process_fleet_movements()` **re-created** — the scan/locks/count are UNCHANGED; the loop body is now exactly `perform movement_settle_arrival(m.id)`. **Byte-equivalent:** identical writes in identical order, including `presence_create(activity)` (hunt arrivals still enter Combat exactly as before) and the 0096 reward deposit. The combat-triggering path is untouched by construction — it was never a cron branch to begin with.
- `command_main_ship_settle_arrival_legacy(p_fleet default null)` (SECURITY DEFINER, `search_path=public`; revoke `public/anon`, grant `authenticated`) — `gate mainship_send_enabled` (no new flag, no flip) → resolve the fleet (explicit owned id, or the sole in-flight main-ship fleet; fail-closed `ambiguous_fleet` otherwise) → main-ship-only → claim the movement FOR

UPDATE SKIP LOCKED (the cron's own lock order ⇒ no deadlock either direction; contention → {settled:false, reason:'busy'}, the cron wins, never blocks) → not_due when early → **NON-COMBAT SCOPING**: refuses any activity_type<>'none' location target (combat_target_unsupported) so the on-demand path can NEVER drive combat init (defense-in-depth over the structural unreachability) → settle via the SAME helper. Idempotent by state (no receipts — settling grants nothing): the helper's guard makes a cron-vs-RPC race exactly-once; the loser no-ops already_settled. NOT applied to any DB.

Client (ONE due-trigger for both families — no second loop, no cadence changes). useSettleDueArrival refactored around a shared inner useDueTimer (one timer at arrive_at+150ms, one fire per movement id, primitive-field deps so polls never reschedule) and now takes the optional legacyMovement/legacyFleetId pair alongside the OSN movement — part A behavior byte-identical. settleArrival.ts gains LEGACY_SETTLE_ARRIVAL_RPC and the legacy outcomes (present/completed/failed) in the shared fail-closed parser; mainshipApi.commandMainShipSettleArrivalLegacy is the thin wrapper. Wired in GalaxyMapScreen (reuses the item-3 legacyMove) AND in Dashboard (the first-trip player waits at the Command Center's MainShipPanel countdown; routes are exclusive so only one hook instance is ever mounted; OSN part stays inert there).

Tests. tests/settleArrival.spec.ts extended (legacy RPC pin + legacy outcomes in the parser truth table). scripts/verify-mainship-move.mjs §13 (same harness, probe-SKIPs until 0151 is applied): not-due no-op → due outbound arrival settles ON DEMAND to present at the destination (cron race tolerated as already_settled — still exactly-once) → repeat no-op → due return_home settles to completed at home → a non-main-ship (combat-target) unit fleet is REFUSED (not_main_ship_fleet).

Doc-sync. docs/SYSTEM_BOUNDARIES.md: the §2 Movement row gains movement_settle_arrival (the ONE settlement body, both callers) + command_main_ship_settle_arrival_legacy (full guards), and §3 gains the RPC's flow entry + the cron note — Movement remains the SOLE writer of fleet_movements, no new table, call graph unchanged/acyclic. This entry added.

Verify (real runs — spine regression proof). npm run build green; verify:osn:settle green; verify:m3 **must stay 13/13** and verify:m4 **must show no new failures vs the recorded 36/40 baseline** (failing names before: wave pacing · damage-no-loss · wave HP decreasing · not one-shot) — results recorded in the step report. Honest scope note: the live DB does not include 0151 (human gate), so live m3/m4 exercise the deployed cron unchanged; the extraction's equivalence is proven by the verbatim code diff above and asserted end-to-end by §13 the moment the human applies 0151. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — UX CLEANUP (item 6, part A) — on-demand OSN arrival settlement (migration 0150 + client due-trigger)

Request. Per the diagnosis §C: a due OSN movement waits up to ~30s of process-mainship-space-arrivals cron (0058:129) plus a 3–4s poll before the player sees the ship settle (~34s worst case, ship visibly floating "arrived but not settled"). Add a server-authoritative, idempotent on-demand settle that reuses the cron's exact primitives — no duplicated settlement body, no cron/poll cadence changes.

Internals confirmed first (citations). Current processor body = 0064:95-178: non-locking due scan (112-118) → SKIP-LOCKED ship claim via mainship_space_lock_context(ship, true) (120; 0056:37-44 returns 'skipped' on contention) → coherent in_transit validate (126-131) → cross-domain exclusion (134-139) → movement re-read under lock, still status='moving' AND due, + full fleet-linkage check (142-160) → primitives: target_kind='location' → mainship_space_dock_at_location (0061; current body 0067 §E1 — returns docked:true, or docked:false = deterministic TERMINAL settlement) else mainship_space_settle_space_arrival with ONE captured timestamp (0064:43-92). Both primitives guard every write with ... and status='moving'. Ownership resolution incl. sole-ship shim = mainship_resolve_owned_ship (0081:26-54). Every OSN command gates on mainship_space_movement_enabled (0083:127). OSN travel time uses the same travel_scale / min_travel_seconds config as legacy (0067 core).

Migration 20260618000150_osn_settle_arrival_on_demand.sql — ONE authenticated RPC command_main_ship_settle_arrival(p_main_ship_id default null) (SECURITY DEFINER, search_path=public; revoke public/anon, grant authenticated). Gate = the SAME existing mainship_space_movement_enabled (no new flag, no flip; dark envs reject feature_disabled before any read). Body mirrors the cron's claim sequence VERBATIM for the caller's own ship, then invokes the cron's OWN primitives — zero settlement logic duplicated. **Idempotent/race-safe by state, no receipts** (settling grants nothing — it advances an inevitable transition): SKIP-LOCKED claim → contention returns {settled:false, reason:'busy'} (the cron wins, never blocks); the primitives' status='moving' guards make a cron/on-demand race exactly-once (loser observes not-moving → already_settled); not-due → not_due no-op; settled/idle ship → already_settled/no_active_movement; any linkage mismatch → frozen-failure incoherent_state, touching nothing. **No reward/spend re-entry exists:** OSN settlement writes only movement/fleet/ship/presence state (rewards ride LEGACY fleet_movements only). The cron stays UNCHANGED at 30s as the backstop. NOT applied to any DB.

Client due-trigger (no new poll loop, no interval changes). settleArrival.ts (pure): RPC literal, fail-closed envelope parser, computeSettleDelayMs (0 when due; exact remaining delay otherwise; null = nothing to schedule — the cron backstops).

`useSettleDueArrival` (hook): arms ONE timer at `arrive_at` (+150ms so the server-side due check is already true), fires the RPC ONCE per movement id (ref-guard, the existing stop/recall idiom), then `refresh()`; effect deps are the movement's PRIMITIVE fields so the 3–4s polls never reschedule it. Wired in `GalaxyMapScreen` (where the OSN movement is already in scope); `mainshipApi.commandMainShipSettleArrival` is the thin §2.5 wrapper. Perceived settle latency drops from ~34s worst case to roughly the RPC round-trip after `arrive_at`.

Tests. `tests/settleArrival.spec.ts` (+ `verify:osn:settle` script): RPC-name pin, envelope parsing (all settled outcomes / all no-op reasons / rejection passthrough / malformed fails closed — never a fabricated settlement), and the due-trigger timing truth table. `scripts/verify-mainship-move.mjs` §12 (same harness, probe-SKIPs until 0150 is applied): not-due no-op → due location move settles ON DEMAND (docks; cron-race tolerated as `already_settled` — still exactly-once) → ship canonically at `location` → repeat call `already_settled` → due SPACE move settles `arrived/in_space`. Flag handling in §12 follows the script's established capture/restore pattern (send flag precedent); the DARK `mainship_coordinate_travel_enabled` gate is restored within ~a second of issuing the one test move — possible because that flag gates INITIATION only (settlement is flag-independent, the OSN-4 in-flight principle). Note: §12 runs only when the human applies 0150 and executes the script.

Doc-sync. `docs/SYSTEM_BOUNDARIES.md`: new "On-demand OSN arrival settle" blockquote (the doc's established OSN-note idiom, beside the geometry/settled-safe leaf notes) — no new writer, no new table, call graph unchanged/acyclic. This entry added.

Verify (real runs). `npm run build` green; `verify:osn:settle` green; `verify:m3` green — results in the step report. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — UX CLEANUP (item 3) — consistent in-transit "stop" for LEGACY main-ship moves (migration 0149 + shared stop UI reuse)

Request. Per the movement→arrival→dock diagnosis: the stop system existed only in the OSN `main_ship_space_movements` domain, while every visible send (`send_main_ship_expedition` 0050 / `move_main_ship_to_location` 0053 — the `MainShipCommand` surface) creates LEGACY `fleet_movements` with NO halt capability at all (`request_main_ship_return` requires `status='present'`, 0050:189). Design decision (human): a legacy in-transit stop is a Fleet/Movement-domain **halt** → **return-home** with a **symmetric turnaround** (arrives home after the time already spent outbound) — never a new "hold in space" state (that is OSN's concept; recreating it in legacy would be a parallel movement system).

Movement internals confirmed first (citations). `fleet_movements` 0007:9-52 (status moving/arrived/cancelled/failed; `mission_type` incl. `return_home`; `one_active_movement_per_fleet` partial unique; `arrive_at` > `depart_at`); `movement_create` 0007:68-125; the cron `process_fleet_movements` 0030:36-83 claims due rows FOR UPDATE SKIP LOCKED (movement row FIRST, fleets updated after) and its base-arrival branch calls `fleet_complete`, which REQUIRES `status='returning'` (0006:163) — so a halt must also step the fleet moving→returning; the generic state machine has no such edge (0053:92-102 is the exact precedent: a dedicated, main-ship-scoped inline transition for its missing present→moving edge). The visible send is gated on `mainship_send_enabled` (0050:73, 0053:34).

Migration `20260618000149_mainship_stop_transit.sql` — ONE new authenticated RPC `command_main_ship_stop_transit(p_fleet)` (SECURITY DEFINER, `search_path=public`; revoke public/anon, grant authenticated). Gate = the SAME existing `mainship_send_enabled` (no new flag, no flip — dark envs reject `feature_disabled`; the human's live env gets it). Validates owned + main-ship fleet (the 0050 return predicate), then claims the active movement FOR UPDATE in the cron's own lock order and:

- **outbound + not due** → transforms the SAME row in place to the `return_home` shape `request_main_ship_return` produces (target = origin base; `depart_at=now`, `arrive_at = now + elapsed-outbound` floored at 1s; `origin_x/y` = the interpolated halt point so the map shows the ship turning around in place; origin entity ids keep the halted destination as provenance; `travel_seconds` is the design-fixed symmetric time — documented in the header; `speed_used` unchanged), steps the fleet moving→returning (dedicated scoped edge, 0053 idiom) and the ship to 'returning' (0050:223 idiom). The one-active-movement invariant holds by construction (no second row) and the transformed row is settled by the NORMAL `process_fleet_movements` return branch — one settlement path.
- **idempotent no-ops by state** (no receipts — a stop grants nothing): cron settled first / no transit → `{ok:true, stopped:false, reason:'already_settled'}`; already `return_home` → `'already_returning'`; due-but-unsettled → `'arrived'` (LEFT for the cron). Every mutation is guarded `status='moving'` under the row lock; same lock order as the cron ⇒ no deadlock, SKIP LOCKED ⇒ the cron never blocks on us.
- **No reward interaction possible:** main-ship targets are `activity_type='none'` (0050:104/0053:71) so no combat ever attaches cargo; `reward_payload_json` stays `'{}'` and the deposit branch (0030:70-72) requires a non-empty payload — double-reward is structurally unreachable. NOT applied to any DB.

Client (reuse, no parallel stop system). `spaceStopCommand.ts`: `STOP_TRANSIT_RPC`, `isActiveLegacyOutboundTransit` (fleet 'moving' + non-return mission), and `parseStopTransitResult` mapping the server envelope onto the SHARED `SpaceStopResult` (halt→'stopped'; every no-op→'arrived'). `mainshipApi.commandMainShipStopTransit(fleetId)` (thin wrapper, the 0064 idiom). `useLegacyStopTransitCommand` (sibling of `useSpaceStopCommand`, SAME `createSpaceStopController` — only the wired RPC differs;

recreated per in-transit fleet). `SpaceStopControls` gained optional copy props (defaults = the original OSN strings byte-for-byte) so the ONE component also serves "Main ship in transit / Stop — return home / Turning around — returning home." — mounted in `GalaxyMapScreen` beside the other overlays (renders only for an outbound legacy transit of the main-ship fleet; refreshes after the command settles; mutually exclusive with the OSN stop mounts by one-movement-owner state). **PortNav hardening (diagnosis follow-through):** `PortNavPanel` now renders the OSN stop for ANY location-target transit; only the destination NAME stays behind the visible-map check (fail-closed — no name/id/coord leak).

Tests. `tests/spaceStopCommand.spec.ts` extended (same harness): RPC-name pin, the legacy-transit predicate truth table, and the envelope mapping (halt / all three no-op reasons / rejection passthrough + copy fallback / malformed fails closed). `scripts/verify-mainship-move.mjs` extended with section 11 (fresh user: mid-flight stop → in-place return_home transform + fleet/ship 'returning' + symmetric timing bound; duplicate stop → already_returning; normal-path completion; post-arrival stop → already_settled), behind a loud DEPLOYMENT PROBE that SKIPS when `command_main_ship_stop_transit` is not yet in the target DB — so the suite stays green before AND after the human applies 0149.

Doc-sync. `docs/SYSTEM_BOUNDARIES.md`: the §2 Movement row gains `command_main_ship_stop_transit` (full semantics + guards) and §3 gains its flow entry — Movement remains the SOLE writer of `fleet_movements`, no new table, call graph unchanged/acyclic. This entry added.

Verify (recorded honestly — real runs). `npm run build green`. `verify:osn:osn4` (the extended stop spec) **9/9 green**. Live DB: `verify:m3 13/13 PASSED` (movement spine unaffected — 0149 is additive and NOT applied). `verify:m4 36/40`: the 4 failures are combat PACING/TUNING assertions (wave pacing, damage-no-loss, mid-wave HP ticks, not-one-shot) — **PRE-EXISTING live-environment balance drift**, provably unrelated to this commit (zero server-side change is applied by it; the client diff cannot touch a server-driving node script). Same class as the `verify:m2 "5 locations"` pin already on record — flagged for the human alongside it. Section 11 of `verify-mainship-move.mjs` SKIPS live until 0149 is applied (by design; the probe prints the skip loudly). SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — UX CLEANUP (item 2) — honest docking UX at non-dock waypoints (frontend-only; server eligibility untouched)

Request. Fix the misleading docking rejection per the movement→arrival→dock diagnosis: a dockability-blind "Finish Docking" affordance was offered for ANY legacy-present ship, so arriving at a non-dock waypoint (Refuge, Lull) offered docking, the click hit `normalize_main_ship_dock` → `mainship_space_location_target_legal` failing at `target_unsupported_role` (0067:89 — waypoints are `physical_role='unclassified'`), and 0084 collapsed that to blanket `ineligible_port`, rendered as the red "This port is not accepting docking right now." Revealed real ports dock fine (OSN arrivals dock directly via Dock-0; legacy port arrivals pass `target_legal`) — the defect was purely the player-facing surface.

Fix (frontend-only; NO migration, RPC, eligibility, flag, or reveal change).

- `src/features/map/mapTypes.ts` — `isDockablePortForDisplay(locationType)`: the ONE client classifier (dockable port ⇔ `location_type==='trade_outpost'`; waypoints are `safe_zone/pirate_hunt`). **Authority/retirement note (also in-line):** this is a DISPLAY heuristic exploiting the seed's clean `location_type` ↔ dockability coupling (0066: role city/port + active docking service ⇔ `trade_outpost`); server `target_legal` remains the sole enforcement authority (UI fails closed — a wrong guess still gets the server rejection). If that coupling ever changes (a non-dockable `trade_outpost`, or dockability exposed via `get_world_map`), update/retire THIS function; the literal is deliberately not scattered.
- `src/features/portentry/portEntryApi.ts` — `PortEntryShipState.presentLocationId` from `fleets.current_location_id` on the ALREADY-fetched active-fleet row (zero new reads).
- `src/features/portentry/portEntry.ts` — new read-only affordance `{kind:'at_waypoint', locationName}`: a coherent legacy-present ship at a location classified NON-dockable gets an honest explanation instead of the doomed Finish-Docking button; `resolvePresentLocation(state, locations)` resolves the location from the threaded world map. UNKNOWN location (map not loaded / id not visible) deliberately keeps the pre-existing 'normalize' behavior — a classification gap can only show the old button (server still rejects), never hide docking at a real port. Finish Docking at dockable ports is byte-identical. Reworded `ineligible_port` copy (the fail-closed fallback for a display/server disagreement): "You can't dock here — this location has no docking service." — never claims a "port".
- `src/features/portentry/usePortEntry.ts` + `PortEntryPanel.tsx` — optional `locations` threading (the parent's already-pollled `get_world_map` list; no new fetch) + the `at_waypoint` card (reuses the existing CARD idiom; port names in the hint are DERIVED via the same classifier — no hardcoded names).
- `src/features/dashboard/Dashboard.tsx` — passes `game.locations` (already in scope from `useGameState`).

Tests (tests/portEntry.spec.ts). New: legacy-present at waypoint → `at_waypoint` (NOT `normalize`); legacy-present at dockable port → still `normalize`; unknown location / no map → pre-existing `normalize` fallback; classifier truth table; `ineligible_port` copy asserted to never say "not accepting" and to name the real cause. Existing affordance/parser/controller tests unchanged (fixture gains `presentLocationId: null`).

Doc-sync. docs/SYSTEM_BOUNDARIES.md needs NO change — confirmed: no table, writer, flag, RPC, or cross-system edge changed; this is client display logic over existing owner-reads.

Verify. npm run build green; verify:portentry (pure-logic unit spec, no DB) green — results in the step report. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — UX CLEANUP (item 4) — one-word location names (forward-only data migration 0148 + every caller updated same step)

Request. Player-facing UX/cleanup pass, slice 2: rename every seeded location to a single evocative, unique word (display data only), via ONE forward-only migration, keeping every name-referencing script, test, and doc consistent in the same step.

The mapping (all 8 seeded locations — 0002 waypoints by unique (zone_id, name) key, 0066 ports by fixed UUID).

old	new	key
Safe Rally Point	Refuge	Wreck Belt waypoint
Pirate Ambush Point	Snare	Wreck Belt waypoint
Raider Outpost	Reaver	Wreck Belt waypoint
Quiet Drift	Lull	Ion Storm Route waypoint
Pirate Den	Blackden	Ion Storm Route waypoint
Haven Reach	Haven	b1a00001-0066-...-000000000001
Slagworks Anchorage	Slagworks	b1a00002-0066-...-000000000002
Driftmarch Waypost	Driftmarc h	b1a00003-0066-...-000000000003

Migration 20260618000148_location_names_single_word.sql — data-only, forward-only (0002/0066 NOT edited; 0066 documents name as mutable display data — every functional lookup is UUID-keyed). One atomic fail-closed do-block: ports updated by fixed UUID, waypoints by zone-scoped current name; aborts with no partial rename if any row is missing, any old name survives, or ANY location keeps a multi-word name. Idempotent re-run tolerated. unique(zone_id, name) holds (new names distinct per zone). **NOT applied to any database** — the human applies it; the reveal gate (reveal_starter_ports, human-gated) is untouched.

Every caller updated (grep-proven; the substring hazard is the real finding). The new port name Haven is a SUBSTRING of the sector name Outer Haven, so every world-map presence/absence text check was converted from bare-word matching to FIELD-ANCHORED matching ("name" *: "Haven" against get_world_map()::text, which is jsonb): postreveal-verify.sql (MAP_PORT_NAMES), osn-hub1a-production-catalog-verify.sql + worldhub1b-a-production-catalog-verify.sh (MAP_LEAK), portlaunch2b-realchain-fixtures.sql (pre-reveal absence + post-reveal presence), worldhub1b-a-realchain-proof.sql (hidden-leak check). Other updates:

- **src (player-facing copy):** portEntryCommand.ts commission success copy, PortEntryPanel.tsx claim copy → "Haven". (Recon note correction: these two hardcoded strings existed after all.)
- **tests:** portEntry.spec.ts (asserts the copy), plus fixtures in dockServicesUi.uispec.ts, mainshipStatusLabel.spec.ts, osnPortNavUi.uispec.ts → new names.
- **scripts:** name literals/labels/comments in osn-enablement-1b-journey.sh, portlaunch1a-realchain-concurrency.sh, port-entry-1-proof.sh/.sql, reveal-starter-ports-operation.sql, trade-economy-bootstrap-proof.sql, trade-fleet-0c-proof.sql, trade-market-1-proof.sql, postreveal-verify.sh/.sql, osn-postenable-verify.sh/.sql, portlaunch2b-realchain-fixtures.sql, worldhub1b-a-realchain-proof.sql (incl. its name='Haven Reach' → 'Haven' functional lookup and expected-VALUES rows), osn-hub1a-production-catalog-verify.sql, worldhub1b-a-production-catalog-verify.sh (P*_NAME).
- **Self-binding catalog verifiers kept coherent:** worldhub1b-a-production-catalog-verify.sh and osn-hub1a-production-catalog-verify.sh grep their expected literals from the checked-out migrations ("derived-from-migration"). The 0066 grep lists KEEP the old seed names (they verify the historical file's verbatim content); the CURRENT display names now bind to 0148 (new MIG148 greps added), and the live-DB assertions use the new names.
- **workflows (dispatch-only; nothing dispatched):** name mentions in comments/messages of osn3-anchor1b-realchain-proof.yml, portlaunch2b-realchain-proof.yml (all UUID-keyed functionally).
- **docs (current-fact, dated per the established idiom):** README.md "Current status" port line (renamed + dated note), REVEAL_STARTER_PORTS_RUNBOOK.md (table + dated note), BYEHARU_PROJECT_GUIDE.md (dated parenthetical), TRADE_FLEET_0A_IMPACT_AUDIT.md (illustrative example). (Review fix 2026-07-05: README was initially missed because the sweep grep listed src/ scripts/ tests/ docs/ .github/ and never the REPO ROOT — the re-run sweep below greps the whole tree from . so root-level files can no longer escape.)

- **Intentionally NOT touched (reported):** shipped migrations 0002/0066/0068/0072/0077/0078/0080/0085 (forward-only; their old-name mentions are comments/exception text — functional lookups are UUID-keyed) and DEV_LOG history.
- **Whole-tree post-sweep (from repo root, excluding only node_modules/dist/.git/test artifacts, git-ignored *.local.md, DEV_LOG history, and the shipped 2026-06 migrations):** exactly FOUR old-name mentions survive, all intentional — the two dated "formerly ..." annotations themselves (README.md, REVEAL_STARTER_PORTS_RUNBOOK.md) and the two osn-hub1a-production-catalog-verify.sh lines that grep the HISTORICAL 0066 migration file for its verbatim seed literals.

Ordering note for the human. The updated name-asserting verifiers (postreveal-verify, osn-postenable-verify, the catalog verifiers) expect the POST-0148 names — apply 0148 before dispatching them; until then they would report the old names (and vice versa the pre-edit scripts would break after 0148). Same-step atomicity is the point: migration + all callers ship together.

Engine verify scripts are name-INDEPENDENT (grep-proven). verify-m2/m3/m4/m45/m5 select locations by location_type (safe_zone / pirate_hunt) and UUIDs — zero name references — so they stay green on both sides of 0148 by construction. docs/SYSTEM_BOUNDARIES.md needs NO change: no table, writer, flag, or cross-system edge changed (it contains no location names — confirmed by grep). Confirmed.

Verify (real runs, live DB — which still has pre-0148 names; applying 0148 is human-gated). npm run build green. verify:m3 13/13 PASSED (dispatched by location_type='safe_zone' — it found "Safe Rally Point" by TYPE, not name, proving the suites stay green on both sides of 0148; throwaway m3test...@example.com rows are covered by the established %test% cleanup path). verify:m2 10/11 with ONE **PRE-EXISTING, name-independent** failure: 5 locations — got 8 — the human has revealed the three starter ports in production, so get_world_map() now returns 8 active locations while m2 still pins the pre-reveal count (its type-counts line still passes). NOT caused by this rename and NOT silently patched here (changing the engine pin belongs to the OSN/port-subsystem diagnosis step, flagged for it). Local note: Node needed --use-system-ca for TLS on this machine (environmental only).

2026-07-05 — UX CLEANUP (item 1) — retire the legacy Train Ships / Training Queue / map ExpeditionCommand UI (frontend-only; backend RPCs intentionally untouched)

Request. Player-facing UX/cleanup pass, slice 1 of 6: remove the three superseded legacy UI surfaces (TrainShipsPanel, BuildQueuePanel in the Command Center; ExpeditionCommand in the Galaxy Map detail panel) plus everything that becomes dead SOLELY as a result. UI-only retirement — no migration, RPC, cron, or flag change.

Removed (each verified zero-reference by grep before deletion).

- Mounts + imports: Dashboard.tsx (TrainShipsPanel, BuildQueuePanel), GalaxyMapScreen.tsx (ExpeditionCommand); no wrapper markup existed beyond the JSX elements themselves.
- Component files: src/features/production/TrainShipsPanel.tsx, src/features/production/BuildQueuePanel.tsx, src/features/map/ExpeditionCommand.tsx.
- Dead-code chain orphaned by the above (all grep-verified zero remaining consumers): src/features/production/productionApi.ts (fetchBuildOrders/trainUnits/cancelBuildOrder) + productionTypes.ts (BuildOrder) → src/features/production/ now empty, directory removed; src/game/production/buildPreview.ts (preview helpers) → src/game/production/ removed; buildOrders state/fetch in useGameState.ts; baseUnits + unitTypes state/fetch in useGalaxyMapData.ts (only ExpeditionCommand consumed them); sendFleetToLocation + SelectedUnit in fleets/fleetApi.ts and DispatchResult in fleets/fleetTypes.ts.
- Kept (still consumed elsewhere, verified): lib/time formatDuration/formatCountdown, baseApi.fetchBaseUnits, catalog.fetchUnitTypes, ExpeditionLauncher (a nav link, no fleet API use), MainShipCommand (the deliberate Phase 10D/10H replacement surface — stale comment updated), fleetApi.requestLeaveLocation (FleetStatusPanel).
- Tests: tests/m45.spec.ts (M4.5 Train/Queue browser acceptance) and tests/galaxy9b.spec.ts (9B map expedition send) existed ENTIRELY to drive the removed UI → deleted, with their npm scripts (verify:m45:browser, verify:galaxy9b:browser). tests/galaxy.spec.ts (9A read-only smoke) adjusted: now asserts the legacy expedition surface is ABSENT and keeps the no-fleet-created invariants.
- Workflow dangling-reference cleanup (both dispatch-only, human-triggered; nothing dispatched): .github/workflows/browser.yml deleted (its only test step ran the removed verify:m45:browser); the 9B step removed from browser-galaxy.yml (its 9A step + cleanup step remain). NOTE: workflows were outside the locked MAY-touch list — flagged here for explicit human review; cleanup-m45-orphans.yml (DB row cleanup) kept, it does not reference the spec.

Backend intentionally intact. train_units, cancel_build_order, send_fleet_to_location, the process-build-queue + process-fleet-movements 30s crons, and all tables/writers are untouched and now simply unreferenced from the client. The backend verify:m45 (node) engine script and M2/M3/M4 suites are unaffected. docs/SYSTEM_BOUNDARIES.md needs NO change: no table, writer, flag, or cross-system edge changed — the ownership matrix documents the (unchanged) server surfaces, not client mounts. Confirmed.

Doc-sync (same step). Stale current-tense references to the retired surfaces annotated with dated notes (historical text preserved, not rewritten): docs/MAINSHIP_TRANSITION.md §2 frontend-touchpoints ("the only send surface") + its tests-pin line + §7 ("keeps ... galaxy9b green") + a new **10D (2026-07-05 update)** bullet in the implemented-vs-planned reconciliation note; docs/ARCHITECTURE.md §16 M7 row ("Train Ships + Training Queue UI" → client UI retired, server RPCs/cron remain). Repo-wide doc grep confirms no other doc states the removed files/specs as current fact (remaining mentions are DEV_LOG history and dated recon snapshots).

Verify. npm run build (tsc -b + vite) green after the removals; 160 modules, no unused-import or type errors. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — RANKING-P17 POST-AUDIT FIX (item 5) — schedule the ranking-accrual cron; world-tick already scheduled, no redundant cron (migration 0147)

Request. Item (5), the deferred background schedulers: add the ranking-accrual cron, and document that world-balance is already scheduled (no redundant cron). NEW forward-only migration mirroring the 0033 cron idiom EXACTLY; same-step doc-sync. No flag flip, no shipped-migration edit, no new function/table, not applied/dispatched to any DB.

The resolution. Investigation confirmed only ONE background job genuinely lacked a scheduler:

- **Ranking-accrual** (ranking_accrue_standings, 0130/0144/0145) had NO cron (shipped as a safe dark no-op, "deferred"). → **This migration schedules it.**
- **World-balance world-tick** is ALREADY scheduled. Phase-19 folded ALL its dynamics INTO worldstate_tick() itself — pirate-pressure (0135), price-drift (0136), field-depletion (0137) are create or replace extensions of that ONE function — and worldstate_tick() is already driven every 60s by the pre-existing process-location-state-ticks cron (0033 → process_location_state_ticks() → worldstate_tick()). → **No second cron added: a redundant schedule would DOUBLE-TICK every world-balance dynamic** (double pressure decay / price drift / field regen-and-depletion). World-balance stays gated by its own dark no-op flag world_balance_enabled.

Work done (migration 0147 — 20260618000147_ranking_p17_accrue_cron.sql). Mirrors 0033 verbatim: create extension if not exists pg_cron; → the idempotent unschedule do-block for jobname 'ranking-accrue-standings' (the exception when undefined_table then null guard copied verbatim) →

```
select cron.schedule(
  'ranking-accrue-standings',
  '* /5 * * * *',
  $$select public.ranking_accrue_standings();$$
);
```

Scheduled DIRECTLY — no process_* wrapper: ranking_accrue_standings is already a self-contained, service-role-granted entry point with its own dark gate, so a wrapper would be dead abstraction (unlike 0033, which wraps worldstate_tick()).

Cadence rationale (self-approved) — every 5 minutes (* /5 * * * *). Standings are a SLOW incremental aggregate and the fold is idempotent + commit-safe (the 0144/0145 per-(season, grant) ledger anti-join), so cadence affects ONLY leaderboard freshness, never correctness — a missed/late run simply folds the backlog on the next firing (no grant is ever skipped). 5 minutes keeps boards fresh at negligible load versus the 60s world heartbeat.

Dark-no-op-until-flag (why scheduling now is safe). ranking_accrue_standings self-checks ranking_enabled FIRST (0145: dark gate before any read/write) and returns feature_disabled without folding/writing anything while the flag is 'false'. So installing the schedule changes NOTHING observable today — every firing is an instant no-op — and it begins accruing the moment the owner flips ranking_enabled true (no further migration needed).

Doc-sync (same step). docs/SYSTEM_BOUNDARIES.md: the §1 ranking_standings row, the §2 Ranking accrual description, and the Ranking acyclic-edge blockquote all changed from "no cron scheduled yet (deferred)" to "cron-driven every 5 minutes (ranking-accrue-standings, 0147) — dark no-op while ranking_enabled='false'"; the §3 cron topology section gains a ranking_accrue_standings() (cron 5min) entry and the process_location_state_ticks() entry now notes the world-balance world-tick is this SAME 60s cron (no cron of its own — a second would double-tick). This DEV_LOG entry added.

Preserved human gates. ranking_enabled stays 'false' (dark) — the scheduled job is a no-op until the human flips it; world_balance_enabled untouched (its 60s tick stays a dark no-op). No flag flipped, no shipped migration (0001–0146, incl. mining 0143, ranking 0144/0145, exploration 0146) edited, no new function/table, and this migration is NOT applied or dispatched to any database. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — CAPTAIN-P16 POST-AUDIT UI (panel 4 of 4) — the dark Captain Progression (recruit) screen, EXTENDING `src/features/captains/` (frontend-only; no server change)

Request. Item (4), panel 4 of 4: build the dark Captain Progression (recruit) screen by EXTENDING the existing `src/features/captains/` feature (reuse `captainsApi.ts/captainsTypes.ts` — no parallel dir, no duplicated roster read/types), reading the roster via the existing `get_my_captain_instances`, recipes via the existing public-read catalogs by direct select, and submitting ONLY the existing `recruit_captain` command — NO new server authority. Frontend-only.

Files touched (2 extended + 1 new + a mount edit).

- `src/features/captains/captainsTypes.ts` (EXTENDED) — added `RecipeIngredient`, `CaptainRecipe` (assembled client-side, with display names), `RecruitCaptainResult`, and `recruitCaptainErrorMessage`.
- `src/features/captains/captainsApi.ts` (EXTENDED) — added `recruitCaptain(requestId, captainType)` (thin `supabase.rpc`, fail-closed, `request_id` a `crypto.randomUUID()` string — TEXT param) and `getCaptainRecipes()` (three DIRECT public-read selects joined client-side; fail-closed to `[]`).
- `src/features/captains/RecruitCaptainPanel.tsx` (NEW) — the dark recruit panel (props `lifecycleKey`; no ship id — recruit is inventory→captain).
- Mounted `<RecruitCaptainPanel/>` in `src/features/map/GalaxyMapScreen.tsx` adjacent to `<CaptainsPanel/>`.

Enumerated REAL client codes (read from 0126 — not invented; and it is CODE-keyed, not reason-keyed). Unlike the assign/unassign wrappers (reason-keyed), the `recruit_captain` wrapper is CODE-keyed (the 0109 craft-command mirror). The real client codes: `feature_disabled` (there is NO `captain_progression_disabled` — the wrapper returns `feature_disabled`), `not_authenticated`, `invalid_request` (mapped from the internal `invalid_request_id`), `unknown_captain`, `no_recipe`, `insufficient_items` (+ `item_id`/have/ need payload), and the `unavailable` fallback. `recruitCaptainErrorMessage` prefers the server message, then maps the code, then appends the `insufficient_items` shortfall (the `ModulesPanel` craft-error idiom).

Recipe catalog read approach. `getCaptainRecipes()` does three DIRECT public-read selects — `captain_recipe_ingredients` (`captain_type_id`, `item_id`, `qty`; 0125), `captain_types` (`id`, `name`, `specialization`), `item_types` (`item_id`, `name`) — all grant `select to anon`, `authenticated` catalogs, the shipped direct-select convention (no RPC). It joins them client-side into per-type `CaptainRecipe` rows with item display names; any select error fails closed to `[]`.

Affordability NOT annotated (honest, no new authority). The only inventory-balance function `inventory_get_balance` (0039) is `service_role`-only (no client grant), and item (4) forbids adding a client read RPC — so the panel shows recipe COSTS only and relies on the server's `insufficient_items` {`item_id`, `have`, `need`} payload on attempt (surfaced by the decorator). No new RPC added.

Fail-closed + THE VISIBILITY-GATE DECISION (documented honestly in the panel header). `captain_progression_enabled` (0124) is gated ONLY in the recruit COMMAND (0126) — there is NO existing read RPC gated on it, and item (4) forbids adding one. So this panel derives VISIBILITY from the captain system's existing gated roster read `get_my_captain_instances` (gated on `captain_assignment_enabled`) — progression is the recruitment face of the captain system — rendering `null` unless `isServerLit(roster)`. The recruit COMMAND remains the AUTHORITATIVE `captain_progression_enabled` gate: while progression is dark it returns `feature_disabled`, surfaced inline on click (never a false success). The server is the sole control; no client flag enables recruiting. CAVEAT recorded: if `captain_assignment_enabled` is lit but `captain_progression_enabled` is not, the panel shows recipes with a Recruit affordance the server rejects `feature_disabled` on click — a dedicated progression-gated read surface (to also hide the affordance on the progression flag) is the clean future follow-up, out of this fix pass's no-new-authority scope.

Per-type recruit wiring. Each recipe row keys its own `pending/rowNote` by `captain_type_id` (the `ModulesPanel` Record-keyed idiom) and submits `recruitCaptain(crypto.randomUUID(), captain_type_id)` via `runGuardedCommand`, refreshing the roster on success and showing `recruitCaptainErrorMessage(res)` on failure.

Mount point. `GalaxyMapScreen`, immediately after `CaptainsPanel` — no ship id needed, just a `lifecycleKey`; non-spatial, placed at `left-[66.5rem]` in the bottom-left overlay row (after captains at `left-[50rem]`), overlapping none. Renders `null` while dark, so production is byte-unchanged.

Build. `npm run build (tsc -b && vite build)` GREEN — no type or build errors (the >500 kB chunk-size note is a pre-existing vite advisory, unrelated).

Boundaries. `docs/SYSTEM_BOUNDARIES.md` needs NO change — FRONTEND-ONLY: no table, writer, RPC, or cross-system edge. It consumes the existing 0125/0126/0123 surface already recorded in the \$2 Captain / Production contracts; a read/command UI consumer (incl. direct public-read catalog selects, already the shipped convention) adds nothing to the ownership matrix or call graph.

Preserved human gates. `captain_progression_enabled` stays 'false' (dark) — the recruit command is server-rejected and, via the roster-gate reuse, the panel renders `null` while the captain system is dark; no flag flipped, no migration/RPC/server-file changed, no new authority, nothing merged/deployed. This completes item (4) — all 4 read-surface UI panels (ranking, investment, captains-assign, captains-recruit) now exist dark. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — CAPTAIN-P15 POST-AUDIT UI (panel 3 of 4) — the dark src/features/captains/ assign/unassign screen (frontend-only; no server change)

Request. Item (4), panel 3 of 4: build the dark Captains (assign/unassign) screen as a new `src/features/captains/` feature, mirroring `src/features/investment/` + `src/features/mining/` (the `runGuardedCommand` guarded-submit) and the `ModulesPanel` per-row Record-keyed idiom, reading ONLY `get_my_captain_instances` (0123) and submitting ONLY the existing `assign_captain_to_ship` / `unassign_captain_from_ship` commands (0120/0121) — NO new server authority. Frontend-only.

Files added (3 + a mount edit).

- `src/features/captains/captainsTypes.ts` — pure discriminated-union types matching 0123/0120 jsonb exactly. NOTE these envelopes are REASON-keyed (not code): `GetMyCaptainInstancesResult = {ok:true, captains?: CaptainInstance[] | {ok:false, reason?}}` with `CaptainInstance = {instance_id, captain_type_id, name, specialization, stats_json, main_ship_id (assigned ship | null), created_at}`; `AssignCaptainResult / UnassignCaptainResult = {ok:true, action, captain_instance_id, main_ship_id, idempotent_replay?} | {ok:false, reason?, message?}`. Plus `captainCommandErrorMessage(res)` — prefers the server `message`, else maps the enumerated `reason` (mirrors `miningExtractErrorMessage`).
- `src/features/captains/captainsApi.ts` — thin `supabase.rpc` wrappers `getMyCaptainInstances`, `assignCaptainToShip(requestId, captainInstanceId, mainShipId)`, `unassignCaptainFromShip(requestId, captainInstanceId)`, each catching transport errors → fail-closed. `request_id` passed as a `crypto.randomUUID()` STRING (the wrapper param is TEXT).
- `src/features/captains/CaptainsPanel.tsx` — dark, server-driven roster panel (props `mainShipId`, `lifecycleKey`): renders `null` unless `isServerLit(roster)`; when lit shows each captain (name, specialization, key stats from `stats_json`) with its assignment state and ONE per-row action — an unassigned captain shows "Assign to ship" (guarded behind `mainShipId != null`), an assigned captain shows "Unassign" — both via `runGuardedCommand` with a per-captain key (the `ModulesPanel` Record-keyed `pending/rowNote` idiom), a fresh `crypto.randomUUID()` per submit, refreshing on success and showing `captainCommandErrorMessage` on failure. `data-testid="captains-panel"`.
- Mounted `<CaptainsPanel/>` in `src/features/map/GalaxyMapScreen.tsx` after `ModulesPanel`.

Enumerated REAL client reason codes (read from 0120 + 0121 — not invented). The `captain_command_client_envelope` mapper (0121) + the wrapper gates can return, to the client: `captain_assignment_disabled` (the dark visibility signal, no message), `not_authenticated`, `invalid_request` (the internal `invalid_request_id` is mapped to this), `ship_not_settled` (the 0121 settled-safe rule), `captain_not_owned`, `ship_not_owned`, `already_assigned`, `captain_slots_full`, `not_assigned`, and the `unavailable` fallback. Every non-dark failure carries a server message, which the decorator prefers.

Fail-closed logic. Visibility is 100% server-driven — NO client flag constant. While `captain_assignment_enabled='false'` `get_my_captain_instances` returns `{ok:false, reason:'captain_assignment_disabled'}` → `isServerLit(roster)` false → the panel returns `null`. Transport errors collapse to `{ok:false}` the same way. The commands are also server-rejected while dark; the UI is never the control.

Per-row assign/unassign wiring. Each captain row keys its own `pending/rowNote` by `instance_id` (the `ModulesPanel` `Record<string, ...>` idiom). Assign submits `assignCaptainToShip(crypto.randomUUID(), instance_id, mainShipId)`; Unassign submits `unassignCaptainFromShip(crypto.randomUUID(), instance_id)`. The server stays authoritative on ownership / slot cap / the settled-safe rule; the panel just reflects the roster and surfaces the reason on failure.

Mount point + rationale. `GalaxyMapScreen`, immediately after `ModulesPanel` — the player's `mainShip?.main_ship_id` is already in scope there (captains are assigned to that ship). Non-spatial like `Modules`, so it sits at `left-[50rem]` in the bottom-left overlay row (after exploration/mining/modules), overlapping none. Renders `null` while dark, so production is byte-unchanged.

Build. `npm run build (tsc -b && vite build)` GREEN — no type or build errors (the >500 kB chunk-size note is a pre-existing vite advisory, unrelated).

Boundaries. `docs/SYSTEM_BOUNDARIES.md` needs NO change — FRONTEND-ONLY: no table, writer, RPC, or cross-system edge. It consumes the existing 0120/0121/0123 surface already recorded in the §2 Captain contract; a read/command UI consumer adds nothing to the ownership matrix or call graph.

Preserved human gates. `captain_assignment_enabled` stays 'false' (dark) — the panel is server-rejected and renders `null`; no flag flipped, no migration/RPC/server-file changed, no new authority, nothing merged/deployed. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — LOCATION-INVEST-P18 POST-AUDIT UI (panel 2 of 4) — the dark `src/features/investment/` Port Investment screen (frontend-only; no server change)

Request. Item (4), panel 2 of 4: build the dark Port Investment screen as a new `src/features/investment/` feature, mirroring the just-built `src/features/ranking/` (leaderboard read + own-standing derivation) AND `src/features/mining/` (the `runGuardedCommand` guarded-submit + client `request_id` generation), reading ONLY the three existing 0134 RPCs and submitting ONLY the existing 0133 `invest_in_location` command — NO new server authority. Frontend-only: no migration/RPC/flag/server change.

Files added (3 + a mount edit).

- `src/features/investment/investmentTypes.ts` — pure discriminated-union result types matching 0134/0133 jsonb exactly: `GetLocationDevelopmentResult` (all_time_total, contributor_count, season_total, window_index, window_start, window_end), `GetLocationInvestmentLeaderboardResult` (rows: {rank, player_id, season_score}), `GetMyLocationInvestmentsResult` (rows: {investment_id, location_id, location_name, amount, invested_at}), `InvestInLocationResult` ({investment_id, location_id, amount, invested_at, idempotent_replay?} | {ok:false, code?}) — all {ok:true,...}|{ok:false,code?} so `isServerLit` narrows cleanly. Plus `investErrorMessage(code)` mirroring `miningExtractErrorMessage`.
- `src/features/investment/investmentApi.ts` — thin supabase.rpc wrappers `getLocationDevelopment`, `getLocationInvestmentLeaderboard`, `getMyLocationInvestments`, `investInLocation`, each catching transport errors → {ok:false} EXACTLY like `miningApi.ts` (fail-closed).
- `src/features/investment/InvestmentPanel.tsx` — dark, server-driven panel (props `locationId`, `mainShipId`, `lifecycleKey` like `MiningPanel`): on mount/refresh reads development + leaderboard + own history; renders null unless `isServerLit(development)`; when lit shows persistent development (all_time_total, contributor_count), the seasonal score (season_total + window bounds), the seasonal leaderboard with the own row highlighted, the caller's own history, and ONE Invest action (amount input → `investInLocation(mainShipId, amt, crypto.randomUUID())` via `runGuardedCommand`). `data-testid= "investment-panel"`.
- Mounted `<InvestmentPanel/>` in `src/features/map/GalaxyMapScreen.tsx` beside `DockServicesPanel`.

Enumerated REAL writer codes (read from 0133 — not invented). The `invest_in_location` wrapper + private writer return exactly: `feature_disabled`, `invalid_request` (null request_id), `not_docked`, `invalid_amount`, `insufficient_credits` (the real wallet code — NOT the instruction's placeholder "insufficient_funds"), `not_authenticated`, `ship_not_owned`; success {ok:true, investment_id, location_id, amount, invested_at} (+ idempotent_replay on a same-(player, request_id) replay). `investErrorMessage` maps these exact codes with an unavailable fallback.

Fail-closed logic. Visibility is 100% server-driven — NO client flag constant. While `location_investment_enabled='false'` every RPC returns {ok:false, code:'feature_disabled'} → `isServerLit(development)` false → the panel returns null. An undocked ship (`locationId` null → skipped read / `unknown_location`) and transport errors collapse to null the same way. The board appears only when the human lights the flag AND the ship is docked at a port.

Own-standing derivation (client-side only — no new RPC). The panel reads the signed-in user id from `authStore` and, among the returned leaderboard rows, highlights the row whose `player_id` matches ("you") and summarises "Your standing: #rank · score". Absent → "Unranked — outside the top N". Computed purely from the already-returned rows, never a server call.

Mount point + rationale. `GalaxyMapScreen`, immediately after `DockServicesPanel`. Chosen over the Dashboard because BOTH the server-reported docked location (`mainShipPresence?.location_id` — the same id `PortNavPanel` consumes as `currentDockedLocationId`) and the player's `mainShip?.main_ship_id` are already in scope there: the reads are location-scoped and the invest command uses the ship whose docked location the server derives, so the docked-port context is the natural home. Chosen over mounting INSIDE `DockServicesPanel` because that panel does not expose its resolved `location_id` to children — mounting in the map screen (which already holds both ids) is the smaller change (no prop-drilling / `DockServices` refactor). It renders null while dark and when not docked, so production is byte-unchanged.

Build. `npm run build (tsc -b && vite build)` GREEN — 161 modules transformed, no type or build errors (the >500 kB chunk-size note is a pre-existing vite advisory, unrelated).

Boundaries. `docs/SYSTEM_BOUNDARIES.md` needs NO change — FRONTEND-ONLY: no table, writer, RPC, or cross-system edge. It consumes the existing 0133/0134 surface already recorded in the §2 Location Investment contract; a read/command UI consumer adds nothing to the ownership matrix or call graph.

Preserved human gates. `location_investment_enabled` stays 'false' (dark) — the panel is server-rejected and renders null; no flag flipped, no migration/RPC/server-file changed, no new authority, nothing merged/deployed. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — RANKING-P17 POST-AUDIT UI (panel 1 of 4) — the dark src/features/ranking/ leaderboard screen (frontend-only; no server change)

Request. Item (4), panel 1 of 4: build the dark Ranking screen as a new `src/features/ranking/` feature mirroring `src/features/events` and the shared fail-closed idiom, reading ONLY the two existing 0131 RPCs (`get_ranking_seasons / get_ranking_leaderboard`) — NO new server authority. Frontend-only: no migration/RPC/flag/server-file change.

Files added (3 + a mount edit).

- `src/features/ranking/rankingTypes.ts` — pure discriminated-union result types mirroring the 0131 jsonb exactly: `RankingSeason` (`season_id`, `cadence`, `label`, `starts_at`, `ends_at`, `status`), `RankingRow` (`rank`, `player_id`, `score`, `events_counted`), `RankingDimension` (`'overall'|'combat'|'trade'|'exploration'|'mining'`), and `GetRankingSeasonsResult / GetRankingLeaderboardResult` as `{ok:true, ...} | {ok:false, code?}` so `isServerLit` narrows the success member cleanly (the `eventsTypes.ts` idiom; arrays optional for defensive `?? []`).
- `src/features/ranking/rankingApi.ts` — two thin `supabase.rpc` wrappers `getRankingSeasons()` + `getRankingLeaderboard(seasonId, dimension, limit?)`, each catching transport/DB errors and collapsing to `{ok:false}` EXACTLY like `eventsApi.ts` (fail-closed; a denied/failed call is not server-lit).
- `src/features/ranking/RankingPanel.tsx` — dark, server-driven leaderboard: on mount / lifecycle change reads `getRankingSeasons()`; renders null unless `isServerLit(seasons)` AND ≥ 1 season exists; when lit, default-selects the active season (else the first) + dimension `'overall'`, fetches `getRankingLeaderboard(season_id, dimension)`, and renders ranked rows (`rank`, short `player_id`, `score`, `events_counted`) with minimal season + dimension selectors (the 0131 domain). Uses the shared `useActivityPanelGuards` mounted-guard + a `lifecycleKey` refetch trigger exactly like `WorldEventsPanel`. `data-testid="ranking-panel"` mirrors the events panel.
- Mounted `<RankingPanel lifecycleKey={user?.id ?? 'anon'} />` in `src/features/dashboard/Dashboard.tsx` as a dark server-lit section (a season leaderboard is a top-level standing surface, not a map overlay; the Dashboard is the post-auth landing). It renders null while dark, so the Dashboard is byte-unchanged in production.

Fail-closed logic. Visibility is 100% server-driven — there is NO client flag constant. While `ranking_enabled='false'` both RPCs return `{ok:false, code:'feature_disabled'}` → `isServerLit` is false → the panel returns null. Transport/DB errors collapse to `{ok:false}` in the API layer and fail closed identically. The board only appears once the human lights the flag AND opens a season.

Own-standing derivation (client-side only — no new RPC). There is no `get_my_standing` RPC by design. The panel reads the signed-in user id from `authStore` and, among the returned leaderboard rows, highlights the row whose `player_id` matches (marked "(you)") and summarises "Your standing: #rank · score · events". If the player is absent from the returned rows it shows a small "Unranked — outside the top N" line — computed purely from the already-returned rows, never a server call.

Build. `npm run build (tsc -b && vite build)` GREEN — 158 modules transformed, no type or build errors (the >500 kB chunk-size note is a pre-existing vite advisory, unrelated).

Boundaries. `docs/SYSTEM_BOUNDARIES.md` needs NO change — this is FRONTEND-ONLY: no table, no writer, no RPC, no cross-system edge. It consumes the existing 0131 read surface already recorded in the §2 Ranking contract; a UI consumer adds nothing to the ownership matrix or call graph (the established precedent that read-only UI over an existing RPC is not a boundary fact).

Preserved human gates. `ranking_enabled` stays `'false'` (dark) — the panel is server-rejected and renders null; no flag flipped, no migration/RPC/server-file changed, no new authority, nothing merged/deployed. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — EXPLORATION-P11 POST-AUDIT FIX — convert the racing duplicate-scan insert into a clean `already_discovered` (migration 0146)

Request. Post-audit fix pass, item 3: `exploration_scan`'s step-11 discovery insert has no `unique_violation` handler, so a racing duplicate surfaces a raw SQL error instead of the clean `already_discovered` envelope the pre-check already returns. Fix as a NEW forward-only migration CREATE OR REPLACE-ing the writer, changing exactly ONE thing (wrap the insert); same-step doc-sync. No flag flip, no shipped-migration edit, no new table/RPC.

The gap. `exploration_scan` (0099) selects the nearest UNDISCOVERED site via a `not exists` pre-check (0099:181-182), then at step 11 does a BARE insert into `exploration_discoveries` (0099:200-202) with NO exception handler. The unique (`player_id`, `site_id`) constraint keeps a player's discovery of a site to exactly one row, but if two scans of the SAME player race past the pre-check for the SAME site (a TOCTOU window between the step-10 check and the insert), the second insert raises a raw `unique_violation` that propagates UNCAUGHT — instead of the clean `{ok:false, reason:'already_discovered'}` the settled-duplicate path returns (0099:192).

The fix (exactly ONE change vs 0099). Wrap the step-11 insert in a begin ... exception when unique_violation then return jsonb_build_object('ok', false, 'reason', 'already_discovered'); end; sub-block.v_now := clock_timestamp(); stays OUTSIDE/BEFORE the block (unchanged position); the success path (v_result + the step-12 receipt insert + return v_result) stays AFTER the block so it runs only on a successful insert. Verified by diff: the ONLY change is the wrap — signature, dark-flag gate, ship-lock/ownership/validation/cross-domain order, receipt lookup, site selection, the pre-check already_discovered/no_site_in_range branch, and the public wrapper command_exploration_scan are byte-identical to 0099 (CREATE OR REPLACE preserves the 0099 ACL, so it and osn_distance and the wrapper are not re-run). The wrapper already maps already_discovered → code/message (0099:283/296), so the caught path flows through it unchanged.

This is POLISH, not a double-discovery fix (conservation was already protected). The unique (player_id, site_id) constraint is the sole authority and already guaranteed at-most-one discovery per (player, site) — the losing insert was always rejected. The only defect was the SHAPE of that rejection (a raw error vs the truthful already_discovered); this hardens the error handling.

Honest reachability (defense-in-depth today). The two racing scans must be DIFFERENT commands (distinct request_id, so the 0055 receipt replay does not absorb them) on the SAME player at the SAME site — which needs two in-space ships of one player. A player holds >1 main ship only via commission_additional_main_ship (0080), DARK behind mainship_additional_commission_enabled='false', so the racing path is LATENT today (defense-in-depth), becoming live only if/when multi-ship-per-player is activated — mirroring the mining 0143 posture. PERMANENT guard, not a shim (no retirement; multi-ship activation is its relevance trigger).

Doc-sync (same step). docs/SYSTEM_BOUNDARIES.md §2 Exploration contract note UPDATED — it documented the scan's idempotency posture INACCURATELY (claimed a race-guarded on conflict (player_id, site_id) do nothing that was never in 0099); corrected to the real mechanism: the not exists pre-check PLUS the 0146 unique_violation catch → already_discovered, guarded by the unique (player_id, site_id) constraint, with the honest latent-two-ship reachability. No table/writer/edge changes, so the §1 matrix is untouched (the behavior-refinement precedent). This DEV_LOG entry added.

Preserved human gates. exploration_enabled stays 'false' (dark) — every call is still server-rejected feature_disabled before this code is reached; no flag flipped, no shipped migration (0001–0145, incl. mining 0143 and ranking 0144/0145) edited, no new table/RPC, nothing merged/deployed/applied to production. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — RANKING-P17 POST-AUDIT FIX — the deferred commit-safety PROOF scripts/ranking-p17-commit-safe-accrual-proof.sh (no migration/flag change)

Request. The "Verify" for item (2): a dynamic proof that the slice-B commit-safe fold actually counts a reward that COMMITS after an overlapping accrual run — the exact scenario the old timestamp cursor skipped forever — mirroring the deferred-proof idiom used for item (1). Only changes this slice: the new script + this note.

The proof (scripts/ranking-p17-commit-safe-accrual-proof.sh). Mirrors scripts/mining-p12-double-extract-concurrency.sh / scripts/osn3-s3-realchain-concurrency.sh point-for-point: a FIFO-driven held-open psql session (distinct application_name), pg_stat_activity state polling, a trap that restores ranking_enabled to 'false' and asserts it + cleans all fixtures, \$DB_URL-gated, never touches a shared/live DB.

Scenario staged (the exact skip case). Under ranking_enabled toggled true ONLY in the disposable stack: (1) one ACTIVE ranking_seasons row whose window spans the test + one throwaway player; (2) session A begins and inserts a reward_grants row so its granted_at is stamped at A's txn START (T1), and HOLDS the txn open (uncommitted ⇒ invisible); (3) grant B is inserted AND committed for a later granted_at (T2 > T1) in the same window; (4) ranking_accrue_standings() runs once — sees only B (A invisible): asserts B folded into ranking_standings (score/events 5/1) and B in ranking_counted_grants, A absent from the ledger; (5) session A commits (commit time T3 > T2, but A.granted_at is still the older T1) — the script asserts A.granted_at < B.granted_at AND A.granted_at < the run-1 watermark (last_counted_at ≈ T2), pinning that this is precisely the row the OLD 0130 granted_at > last_counted_at cursor would exclude forever; (6) ranking_accrue_standings() runs again and asserts the 0145 anti-join COUNTS A — A now in the ledger, standings rise to 8/2 (exactly A's +3 score / +1 event), exactly TWO grants folded for (season, player), and B still counted exactly once (no double-count — the ledger unique (season_id, grant_id) + anti-join).

Why this proves "no finalized reward is ever missed." A is visible to a run only once committed; whenever it first becomes visible — however late, whatever its granted_at — it is absent from the ledger, so the visibility-based anti-join includes it and folds it exactly once. The counterfactual (the old watermark dropping A because T1 < the advanced watermark) is documented in the header and pinned by the T1-below-watermark assertion, but not executed (the old function no longer exists).

Run instructions (DEFERRED to the human activation checklist). DB_URL=postgres://... bash scripts/ranking-p17-commit-safe-accrual-proof.sh. NOT wired into package.json's dark verify:* block — it needs a LIT DB (it flips ranking_enabled true INSIDE its disposable stack and restores it) and so cannot run in the flag-off sweep; referenced

only from its own header and this note. This environment has no local DB, so the lit run is deferred; static-checked green here with `bash -n`.

Preserved human gates. No migration edited, no committed file flips a flag — the script's `true` toggle is a runtime `psql` update inside a disposable `$DB_URL` stack, restored to the captured original (`'false'`) and asserted in the trap. `ranking_enabled` stays `'false'` in every committed file; no `package.json` entry added; nothing merged/deployed/applied to production. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — RANKING-P17 POST-AUDIT FIX, SLICE B — ranking_accrue_standings made COMMIT-SAFE by folding through the ranking_counted_grants ledger (migration 0145)

Request. Post-audit fix pass, item 2, slice B: rewrite `ranking_accrue_standings` so correctness no longer depends on the `granted_at` timestamp watermark. NEW forward-only migration `CREATE OR REPLACE`-ing the accrual writer, changing ONLY the fold; preserve everything else verbatim; same-step doc-sync + verifier update. No flag flip, no shipped-migration edit, no writer for any other table, `ranking_score_delta` unchanged.

The bug (recap). 0130's fold is INCREMENTAL by `ranking_standings.last_counted_at`: it counts only grants with `granted_at > last_counted_at` and advances the mark. `reward_grants.granted_at` defaults to the inserting txn's START time (20260616000015...:14), but a row is VISIBLE to the reader only at COMMIT. A grant whose txn started before a run yet commits after that run advanced the watermark past its `granted_at` is PERMANENTLY skipped — silently dropped points under concurrent finalization.

Before → after (the fold is the ONLY change).

- BEFORE (0130): a pure folded CTE selects grants with a HIGH-WATER FILTER (`((st.last_counted_at is null and rg.granted_at >= s.starts_at) or (rg.granted_at > st.last_counted_at))`) via a LEFT JOIN to the existing standings row, groups by (season, player, dimension), then upserted reads folded. Correctness depends on the timestamp watermark.
- AFTER (0145): a data-modifying newly_counted CTE insert into `ranking_counted_grants` (...) `select ... from reward_grants rg join ranking_seasons s on s.status='active' and rg.granted_at between s.starts_at and s.ends_at where rg.source_type in (...4...)` and not exists (`select 1 from ranking_counted_grants c where c.season_id=s.season_id and c.grant_id=rg.id`) on conflict (season_id, grant_id) do nothing returning season_id, player_id, dimension, score, granted_at. Then aggregated groups the RETURNING rows by (season, player, dimension) (`score=Σ, events_counted=count, last_counted_at=max granted_at`), and upserted reads aggregated with the SAME on conflict (season_id, player_id, dimension) do update set `score = t.score + excluded.score, ..., last_counted_at = greatest(...), updated_at = now()` shape as 0130.

Why the anti-join is commit-safe (and never skips a finalized reward). The fold no longer asks "is this grant newer than the watermark?" (a time question, defeated by commit-after-start visibility); it asks "is this grant already in the ledger for this season?" (a VISIBILITY question). A grant becomes visible to a run only once committed; whenever it first becomes visible — however late, whatever its `granted_at` — it is not yet in the ledger, so the `not exists` anti-join includes it and the run marks

- folds it exactly once. `unique (season_id, grant_id) + on conflict do nothing` guarantee at-most-once even under a raced run (belt-and-braces with the global advisory lock); a re-run with nothing unmarked inserts nothing and upserts nothing (idempotent). No `granted_at` ordering assumption remains, so no late-committing grant is ever dropped.

`last_counted_at` is now INFORMATIONAL. It is still written (`max granted_at` among grants folded this run, kept via `greatest`) for audit/display, but is NEVER read back as a cursor — the ledger's anti-join is the correctness cursor. The COLUMN is intentionally NOT dropped (a forward-only column drop is out of scope for this fix); the migration/function comments state this explicitly.

Preserved VERBATIM from 0130 (verified by code-only diff). The signature `ranking_accrue_standings()` returns `jsonb`, language `plpgsql` security definer set `search_path = public`, the declare block, the DARK-GATE-FIRST `cfg_bool('ranking_enabled')` reject, the `pg_advisory_xact_lock(hashtext('ranking_accrue_standings'), 0)` serialize, the three summary aggregate expressions (`count(distinct season_id)`, `count(*)`, `coalesce(sum(events_counted),0)` — only the FROM source changed folded→aggregated), the `jsonb_build_object('ok', true, 'seasons_scored', ..., 'rows_upserted', ..., 'events_folded', ...)` result shape, and the service-role-only ACL block are all byte-identical. `ranking_score_delta` is UNCHANGED and NOT redefined here (0130 owns it and its grants).

Sole-writer status. `ranking_accrue_standings` remains the SOLE writer of `ranking_standings` and is now the REALIZED sole writer of `ranking_counted_grants` (0144's deferred writer). No second write path to either table.

Doc-sync (same step). `docs/SYSTEM_BOUNDARIES.md`: §1 `ranking_standings` fold note rewritten to the commit-safe ledger anti-join with `last_counted_at` informational; §1 `ranking_counted_grants` row now shows the REALIZED sole writer `ranking_accrue_standings` (0145); §2 Ranking contract's Owns entry, accrual description, and Role fold note all updated to the

anti-join (present tense). Edges unchanged: Ranking → Reward (reward_grants read) + Reference/Config (cfg_bool), DOWNWARD, acyclic; nothing calls into Ranking. **Verifier:** scripts/verify-ranking.mjs extended to assert ranking_counted_grants is SERVER-ONLY — authenticated SELECT denied, anon SELECT denied, and a valid-shaped authenticated INSERT denied — mirroring the existing table-denial assertions; no lit path, no flag flip.

Preserved human gates. ranking_enabled stays 'false' (dark) — the writer still rejects feature_disabled before any read; no flag flipped, no shipped migration (0001–0144, incl. mining 0143 and the slice-A schema 0144) edited, no new table, no cron, nothing merged/deployed/applied to production. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — RANKING-P17 POST-AUDIT FIX, SLICE A — the commit-safe consumption-ledger SCHEMA ranking_counted_grants (migration 0144; schema only, no writer)

Request. Post-audit fix pass, item 2: ranking_accrue_standings's timestamp high-water cursor is commit-unsafe. Build the fix in slices, schema first. Slice A = a NEW forward-only migration adding the per-(season, grant) consumption ledger table ONLY (no writer — that is slice B), plus same-step doc-sync. No flag flip, no shipped-migration edit (0001–0142 or the mining 0143), no new RPC/writer.

The bug (commit-unsafe cursor). ranking_accrue_standings (0130) is INCREMENTAL by a TIMESTAMP high-water cursor, ranking_standings.last_counted_at (0128 decision 3): each run folds only grants with granted_at > last_counted_at and advances the mark. But reward_grants.granted_at defaults to now() = the inserting transaction's START time (20260616000015_reward_system.sql:14), while a row becomes VISIBLE to the accrual reader only at COMMIT. A grant whose txn started BEFORE an accrual run (small granted_at) but COMMITS AFTER that run advanced the watermark past its granted_at is then PERMANENTLY SKIPPED — the next run's granted_at > last_counted_at filter excludes it forever. Under normal concurrent finalization this silently drops points. reward_grants has a stable id uuid PK (0015:8), so a per-row consumption marker is the correct, ordering-independent key.

The fix (per-grant consumption marker; the securing secured_at idiom). Replace timestamp-cursoring with a visibility-based per-(season, grant) marker. ranking_counted_grants records, EXACTLY ONCE, that a specific reward_grants row has been folded into a specific active season. Slice B's accrual will select grants by an ANTI-JOIN against this table (grants NOT yet marked for the season) rather than by a granted_at comparison — so a late-committing grant is simply ABSENT from the ledger and picked up on the next run, regardless of granted_at ordering. Commit-safe by construction, exactly-once (the unique (season_id, grant_id) key), idempotent. This is the SAME per-row consumption-marker idiom the codebase already uses for commit-safe idempotency — the securing processors' secured_at mark (0100/0105) that the 0128/0130 accrual comment itself cites as its analogue — here materialized as its own ledger row (one grant folds once PER active season it belongs to, so the marker is per-(season, grant), not a single column on the grant). PERMANENT correctness structure, NOT a shim; it does not retire — the timestamp cursor is simply what slice B stops depending on.

Work done (migration 0144 — 20260618000144_ranking_p17_counted_grants_schema.sql). Created public.ranking_counted_grants (schema/RLS/index/comments only, mirroring the 0128 standings style): id uuid PK, season_id → ranking_seasons(season_id) ON DELETE CASCADE, grant_id → reward_grants(id) ON DELETE CASCADE, player_id → auth.users(id) ON DELETE CASCADE, dimension text with the IDENTICAL 0128 closed-set CHECK ('combat', 'trade', 'exploration', 'mining') (reused, not re-spelled), score numeric, granted_at timestamptz (informational snapshot, NOT a cursor), counted_at timestamptz default now(), and unique (season_id, grant_id) (the exactly-once key; its index also serves the anti-join lookup). Added ranking_counted_grants_fold_idx (season_id, player_id, dimension) for the per-standings-row aggregation. SERVER-ONLY posture (the 0103 securing-table stance, NOT the 0128 public-read stance): RLS enabled, NO policy, NO anon/authenticated grant → clients can neither read nor write; the SECURITY DEFINER writer (slice B) reaches it as definer-owner. NO writer created this slice.

Boundaries / doc-sync (same step). docs/SYSTEM_BOUNDARIES.md §1 matrix gains the ranking_counted_grants row (sole writer = ranking_accrue_standings (slice B); server-only, no client read; DARK behind ranking_enabled), adjacent to ranking_standings; the §2 Ranking row's Owns is extended and its accrual note now flags the last_counted_at timestamp cursor as commit-unsafe and being replaced by this ledger's anti-join. Ranking stays a READ-ONLY downward leaf: reads Reward's reward_grants DOWNWARD, writes only its own tables, nothing calls into Ranking — call graph unchanged and acyclic. No new table adds any cross-system edge.

Preserved human gates. ranking_enabled stays 'false' (dark) — no reader or writer references the new table yet; no flag flipped, no shipped migration (0001–0143, incl. mining 0143) edited, no new RPC/writer, nothing merged/deployed/applied to production. Verified: with RLS on and no policy/grant, no anon/authenticated client can read or write ranking_counted_grants. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — MINING-P12 POST-AUDIT RECONCILIATION — the 0143 double-extract race is NOT reachable today; honest reframing + deferred concurrency proof (no code/migration change)

Request. Before proving item (1), reconcile its reachability honestly against the code, correct the 0143 doc wording that overstated the race as reachable-today, and author the deferred concurrency proof — WITHOUT editing migration 0143 (the guard is correct defense-in-depth and the explicit audit ask).

Reachability verdict — I PARTIALLY DISAGREE with the stated premise (one factual correction).

- **CONFIRMED:** `mainship_space_lock_context(p_main_ship_id, false)` does `SELECT ... FOR UPDATE` on `main_ship_instances` (20260618000056_osn3_s2_transition_core.sql:46), so two concurrent `mining_extract` calls on the SAME main ship serialize on the ship row lock — the second reads the first's committed extraction and is cooldown-rejected.
- **CORRECTION:** `main_ship_instances.player_id` is **NOT UNIQUE today**. The inline `UNIQUE` at 20260617000043_main_ship_instance.sql:47 (`main_ship_instances.player_id_key`) was **DROPPED** in 20260618000079_trade_fleet_0c_drop_player_id_unique.sql. So "one main ship per player" is a DARK-GATE / runtime invariant, not a schema constraint.
- The path that COULD create a 2nd ship — `commission_additional_main_ship()` (20260618000080...) — exists and is real, but is DARK behind `mainship_additional_commission_enabled='false'` (cap `max_main_ships_per_player=3`); the first-ship writer is zero-ship-guarded. So every player holds ≤ 1 ship at runtime, and the "two ships of one player at one field" double-extract variant is **NOT constructible today**.
- **NET:** the `(player, field)` no-double-extract invariant IS already held today by [ship `FOR UPDATE` lock + cooldown read + the dark additional-commission gate keeping ≤ 1 ship/player]. The 0143 advisory lock is **defense-in-depth**, inert today, becoming **LOAD-BEARING** the moment `mainship_additional_commission_enabled` is flipped true (multi-ship-per-player). This makes the guard MORE relevant than the premise implied — its trigger is an already-built dark capability, not a hypothetical future schema change.

Doc corrections (this same step). Reframed the prior-slice wording that implied reachability today:

- `docs/DEV_LOG.md` (the 0143 entry below): title changed from "close the double-extract race ..." → "add a per-(player, field) advisory lock ... as defense-in-depth ..."; the "**The bug.**" paragraph reframed to "**The modeled race — reachability CORRECTED ...**" stating the two-ship variant is not constructible today and why (dark commission gate; 0043 `UNIQUE` dropped in 0079); the design- decision paragraph now records the **LOAD-BEARING TRIGGER** (`mainship_additional_commission_enabled true`).
- `docs/SYSTEM_BOUNDARIES.md` (§2 Mining note): the phrase "the lock closes that window" (which implied an active window today) replaced with a reachability statement — invariant already held by the ship `FOR UPDATE` + cooldown + ≤ 1 -ship dark gate; the advisory lock is defense-in-depth, load-bearing only if/when multi-ship is activated; permanent guard, no retirement.

Deferred concurrency proof (new artifact). `scripts/mining-p12-double-extract-concurrency.sh`, mirroring `scripts/osn3-s3-realchain-concurrency.sh` point-for-point (real concurrent FIFO-driven `psql` sessions, distinct `application_name`, `pg_stat_activity` wait-state, a trap that restores every flag/tunable it toggled — `mining_enabled`→`false` asserted, `mining_extract_cooldown_seconds`→captured original — plus fixture cleanup; `$DB_URL`-gated; never touches a shared/live DB). It proves the **REACHABLE** invariant: fixtures = one user + one settled `in_space` main ship + one active `mining_fields` row at the ship's coordinates (within `mining_extract_radius`) + a large cooldown, with `mining_enabled` flipped true **ONLY** inside the disposable stack; two sessions issue `mining_extract` for that ship with two **DISTINCT** `request_ids` → assert A succeeds (one extraction), B blocks on the ship `FOR UPDATE`, and after A commits B returns `reason='cooldown'` (not a second extraction); final assert = exactly **ONE** `mining_extractions` row for `(player, field)`. The two-ship variant is documented in the header as not constructible today (dark commission gate; 0043 `UNIQUE` dropped in 0079), so the proof covers the reachable surface; the 0143 advisory `(player, field)` lock is additionally verified **STRUCTURALLY** (present and ordered immediately before the cooldown read) as defense-in-depth (this check runs without a DB). Static-checked green with `bash -n`. The LIT run is **DEFERRED** to the human owner's activation checklist (this environment has no local DB, and no flag may be flipped in a committed artifact). It is deliberately **NOT** added to the `dark verify:*` block in `package.json` (it needs a lit DB); referenced only from its own header and this log.

Preserved human gates. No code or migration changed (0143 and all shipped migrations 0001–0142 untouched); the new script sets `mining_enabled='true'` **ONLY** at runtime inside a disposable `$DB_URL` stack and restores it to `false`. `mining_enabled` stays `false` in every committed file. No flag flipped, no `package.json` verify entry added, nothing merged/deployed/applied to production. **SAFE FOR HUMAN MERGE REVIEW.**

2026-07-05 — MINING-P12 POST-AUDIT FIX — add a per-(player, field) advisory lock to mining_extract as defense-in-depth for the double-extract race (migration 0143)

Request. Post-audit fix pass, item 1: the mining extract command had a read-then-insert double-extract race. Fix it as a NEW forward-only migration that CREATE OR REPLACES the writer, reproducing the 0104 body verbatim and changing exactly ONE thing — adding a per-(player, field) advisory lock — then sync the law docs the same step. No flag flip, no shipped-migration edit, no new RPC/table.

The modeled race — reachability CORRECTED 2026-07-05 (see the reconciliation note above; this paragraph was initially overstated as reachable-today and is now accurate). mining_extract (0104, step 11) reads the latest mining_extractions.created_at for (player, field) and, if older than mining_extract_cooldown_seconds, inserts a new extraction. The S2 canonical ship lock (mainship_space_lock_context, 20260618000056...:46 SELECT ... FOR UPDATE on main_ship_instances) serializes commands on the SAME ship only. IF one player could hold TWO ships both settled within mining_extract_radius of the SAME field, those two mining_extract calls would lock distinct ship rows, never contend, both pass the read-then-insert cooldown check, and double-extract — a double-reward window once process_mining_securing (0105) deposits both bundles. **That two-ship configuration is NOT constructible at runtime today:** the ONLY additional-ship path commission_additional_main_ship (0080) is DARK behind mainship_additional_commission_enabled='false' (cap max_main_ships_per_player=3), and the first-ship writer is zero-ship-guarded, so every player holds ≤ 1 main ship — two concurrent extracts therefore contend on the SAME ship row, and the second reads the first's committed row and is cooldown-rejected. **NOTE (premise correction):** the original 0043:47 player_id UNIQUE (main_ship_instances_player_id_key) was DROPPED in 0079, so ≤ 1-ship is a DARK-GATE / runtime invariant (the dark additional-commission flag), NOT a schema constraint. The advisory lock is thus DEFENSE-IN-DEPTH — inert today, load-bearing the moment multi-ship-per-player is activated.

Work done (migration 0143 — 20260618000143_mining_p12_extract_double_extract_guard.sql). CREATE OR REPLACE FUNCTION public.mining_extract(...), body copied byte-for-byte from 0104 with a SINGLE inserted block (new step "10b") between field resolution (step 10) and the cooldown read (step 11): perform pg_advisory_xact_lock(hashtext('mining_extract'), hashtext(p_player::text || ':' || v_field.id::text));. Two commands for the same (player, field) now serialize there — the second blocks until the first COMMITS, then reads the first's now-committed extraction at step 11 and is correctly cooldown-rejected. Verified by diff: the ONLY change vs 0104 is the 10b comment + the perform line; the signature, dark-flag gate, ship-lock/ownership order, receipt/idempotency logic, cooldown math, selection rule, accrual/reward math, the public wrapper command_mining_extract, and all grants are unchanged (CREATE OR REPLACE preserves the 0104 ACL, so the revoke/grant block is not re-run).

Idiom reused (not re-invented). The established two-arg advisory-lock pattern pg_advisory_xact_lock(hashtext('<domain>'), hashtext('<scope>')) already used at 0078 (commission), 0113 (fitting), 0126 (recruit), 0133 (location investment). Domain = 'mining_extract', scope = the combined (player, field) key.

Design decision — PERMANENT guard, no retirement condition; recorded load-bearing trigger. Xact-scoped advisory locks auto-release at commit/rollback (no cleanup path, no softlock risk — NO-ACCOUNT-SOFTLOCK holds) and are reentrant within the transaction (harmless alongside the existing S2 row locks). This is a correctness invariant of the writer, not a shim/compat path — it stays for the life of the function (so "no retirement condition"). Its LOAD-BEARING TRIGGER is recorded: today it is inert defense-in-depth (≤ 1 main ship per player ⇒ the ship FOR UPDATE lock already serializes the (player, field) invariant); it becomes load-bearing the moment mainship_additional_commission_enabled is flipped true (multi-ship-per-player), when two ships of one player at one field could otherwise race the cooldown read.

Boundaries / doc-sync (same step). No new table, writer, or cross-system edge — this refines the concurrency discipline of an EXISTING sole-writer (mining_extract remains the sole insert path of mining_extractions). docs/SYSTEM_BOUNDARIES.md §2 Mining contract row updated to document the new per-(player, field) extract serialization lock in its pacing/concurrency note; this DEV_LOG entry added.

Preserved human gates. mining_enabled stays 'false' (dark) — every call is still server-rejected feature_disabled before this lock is ever reached; no flag flipped, no shipped migration (0001–0142) edited, no new RPC/table, nothing merged/deployed/applied to production. SAFE FOR HUMAN MERGE REVIEW.

2026-07-05 — PHASE20-POLISH CLEANUP — independent re-audit of the closed polish milestone; ALL THREE audits CLEAN, ZERO remediation (docs-only close-out)

Request. The Phase-20 (Polish/expansion — map UI, portraits, icons, world events) milestone was already CLOSED (see the SLICE 7 entry below) and its three cleanup audits reported clean. Rather than trust that premise, re-establish the baseline and

INDEPENDENTLY re-verify each audit against the actual code, then record a durable close-out. Read-only audit throughout; this entry is the only artifact.

Honest premise correction (found at STEP 1). The three named audits were NOT previously persisted as `*_RECON.local.md` recon docs — the only Phase-20 recon in the tree is the pre-build STEP-0 recon (`WORLD_POLISH_P20_RECON.local.md`), and every `CLEANUP_*_RECON.local.md` covers an EARLIER phase (11–19). So the milestone's "clean" verdict had been resting on the goal's premise plus the SLICE 7 close-out, not on persisted per-audit evidence. This entry is therefore the DURABLE record: the three audits were re-run inline against `docs/DEV_LOG.md` + `docs/SYSTEM_BOUNDARIES.md` + the on-disk source (`migrations 0139–0142`, `src/features/{events,assets}`, `scripts/verify-phase20-polish.mjs`).

Audit 1 — World-Events / UI-Assets migrations 0139–0142 — CLEAN (all six properties PASS).

- **Dark-gate-first** — `cfg_bool('phase20_polish_enabled')` is the FIRST executable statement in all four functions (`world_events_publish 0140:87–89`, `world_events_set_active 0140:167–169`, `get_world_events 0141:48–50`, `get_ui_asset_catalog 0142:109–111`); no table is touched before it. `cfg_bool 0046:17–24` is a `stable`, read-only SQL select from `game_config`.
- **Exactly one sole-writer per table, no second writer anywhere** — tree-wide grep for `insert/update/delete` on either table returns only: `world_events` ← `world_events_publish 0140:130,146` + `world_events_set_active 0140:176`; `ui_asset_catalog` ← the `0142:80` seed insert only (no runtime writer). Zero writes elsewhere; no `DELETE` (retire is a status flip).
- **Pure downward LEAF** — the writers touch only `world_events`; their sole cross-reference is a read-only `EXISTS` against the static Map (zones `0140:118–119`, locations `0140:122–123`) for FK validation. Grep for the four function names finds callers ONLY in their defining migrations — zero inbound edges, acyclic, no write into another system's table.
- **ui_asset_catalog seed-only static Reference/Config** — RLS on, all client grants revoked (`0142:60–61`); rows come only from the `0142:80–90` seed.
- **Grants** — writers service-role-only (`0140:186–190`); read RPCs authenticated-only (`0141:92–93`, `0142:136–137`).
- **Doc-sync accurate** — `docs/SYSTEM_BOUNDARIES.md` (:64–65, :107, :109) documents both tables, both sole-writers, the downward-leaf boundary, the seed-only nature, and the grants with NO contradiction of the migration source.

Audit 2 — Frontend `src/features/{events,assets}` — CLEAN (all four properties PASS).

- **Reuses the shared helpers** — `WorldEventsPanel.tsx:2` imports `isServerLit` + `useActivityPanelGuards` from `src/lib/useActivityPanelGuards.ts` and calls them (:34, :49, :61); the icon resolver reads `get_ui_asset_catalog('icon')` through the shared `supabase.rpc` wrapper (`assetsApi.ts:17`), not an ad-hoc fetch. `runGuardedCommand/rewardBundle` are CORRECTLY omitted — a read-only presentational panel ("no actions/buttons", `WorldEventsPanel.tsx:15`) has no command-submit or reward surface to use them on; importing them would be dead code.
- **No duplicated guard/command/lit/reward logic** — the only new code is presentational (a 3-entry `SEVERITY_BADGE` map, a 5-entry `assetGlyphs asset_ref → emoji` map, thin per-RPC api wrappers, discriminated-union types shaped so the shared `isServerLit` narrows cleanly). Guard/lit logic lives once in the shared lib and is called, never inlined.
- **Fails closed with the server as sole control** — `WorldEventsPanel.tsx:61` renders `null` unless `isServerLit(result)` AND events exist; transport errors collapse to `{ok:false} → also null`. The mount (`GalaxyMapScreen.tsx:167–169`) passes ONLY a `lifecycleKey` re-fetch trigger — no client-side feature flag. Dark → the server empties the feed → the panel renders nothing (production UI unchanged).
- **Zero shims** — grep for `shim|compat|TODO|FIXME|HACK|temporary|transitional` across both feature dirs returns no matches. (The one sanctioned port-entry shim belongs to a different feature, not this surface.)

Audit 3 — Verifier `scripts/verify-phase20-polish.mjs` + `package.json` — CLEAN (props 1/2/4 PASS, prop 3 correct N/A).

- **Imports the shared harness, zero inline copies** — `:28–29` import `teardownVerifier` (`verifier-teardown.mjs:18`) + `Abort/createReporter/createUserFactory/resolveEnv` (`verify-harness.mjs:46,50,62,36`). Grep of the verifier for locally-redefined harness functions → none; the only local helper is the one-line read-only `cfgVal` (:44).
- **Strictly dark-posture** — never writes `game_config`, never flips `phase20_polish_enabled`, and exercises NO lit path; assertions prove only the LOCK (gate reads `'false'` :57–58; read surfaces return `ok:true` + empty while dark :68–70, :78–80; writers denied to authenticated + anon with VALID-shaped args :91–106; table SELECT/INSERT denied :113–147). Lit-path checks are explicitly deferred to the human owner's activation checklist (:17–20).
- **Single clean npm entry** — `package.json:51` `"verify:phase20-polish": "node scripts/verify-phase20-polish.mjs"` (grep count = 1; no duplicate/stale line).
- **No Phase-20 shell proof exists** — grep for `phase20|world_event|ui_asset` across `scripts/*.sh` finds nothing; none was invented and `trade-proof-lib.sh` is untouched.

Outcome — NO change warranted. The tree is clean; NO code, migration, flag, or `src/` behavior change was made.

`docs/SYSTEM_BOUNDARIES.md` is INTENTIONALLY UNTOUCHED because this cleanup changed no architectural fact — no table, writer, constraint, or cross-system edge was added, dropped, or altered (the slice-verifier precedent: a doc-sync is required only when an architectural fact changes). This entry is the sole artifact of the milestone.

Preserved human gates (nothing activated by this loop): `phase20_polish_enabled` and every Phase-11–20 master flag remain seeded `'false'` (untouched); migrations `0001–0142` are unedited (forward-only); NO `game_config` write; NO lit-path DB run; NO

cron scheduled; main untouched; no merge / deploy / production-apply / workflow-dispatch. **SAFE FOR HUMAN MERGE REVIEW** — activation (flag flip, deploy, event publish/cron) remains the human owner's decision.

2026-07-04 — PHASE20-POLISH SLICE 7 (FINAL) — the dark-posture verifier verify-phase20-polish.mjs + verify:phase20-polish; Phase 20 CLOSED

Request. Phase 20 final slice: ONE new verify script (the verify-world-balance.mjs analogue) + one package.json line + same-step doc-sync + phase close. NO migration change, NO flag write, NO lit-path DB run, NO src/, NO new RPC, NO cron, no git.

Work done — scripts/verify-phase20-polish.mjs (mirrors verify-world-balance.mjs point-for-point; ZERO inline harness copies — imports the shared Abort/resolveEnv/createReporter/ createUserFactory from scripts/lib/verify-harness.mjs + teardownVerifier from scripts/lib/verifier-teardown.mjs, the same admin/anon/throwaway-user/cfgVal scaffold, the same .catch/.finally teardown with NO flag entry passed — this verifier touches no flag; emailPrefix 'phase20'). Proves migrations 0139–0142 ship exactly as built and fully dark, with anon/authenticated clients only. Five assertion groups (String() storage-form-tolerant compares; VALID-shaped uuid/arg sets so a denial proves the LOCK, not argument validation):

1. **Config presence** (READ-ONLY) — phase20_polish_enabled='false' (the dark master gate).
2. **Read surfaces dark + ACL-correct** — as AUTHENTICATED: get_world_events({p_location_id:null, p_zone_id:null}) → ok:true with an EMPTY events array, and get_ui_asset_catalog({p_asset_kind: null}) → ok:true with an EMPTY assets array (flag-gated fail-closed → empty while dark). As ANON: BOTH RPCs DENIED (granted to authenticated only).
3. **World Events writers locked** — world_events_publish (full valid-shaped arg set) and world_events_set_active({p_event_id:randomUUID(), p_is_active:false}) DENIED to BOTH authenticated and anon (service-role-only ACL, 0140).
4. world_events **server-only** — authenticated + anon SELECT DENIED (no client read policy/grant); a direct authenticated INSERT DENIED (sole writers are its two owner functions).
5. ui_asset_catalog **server-only, still static** — authenticated + anon SELECT DENIED; a direct authenticated INSERT DENIED (static Reference/Config, seed-migration-only, no runtime writer added).

package.json — ONE line added adjacent to verify:world-balance: "verify:phase20-polish": "node scripts/verify-phase20-polish.mjs".

NO-FLAG-WRITE / NO-LIT-PATH stance (verbatim from the verify-world-balance precedent): the script NEVER writes game_config and NEVER flips phase20_polish_enabled; it exercises NO lit path. Lit-path verification (flag on a DEV DB → world_events_publish a scoped event → get_world_events returns it with its resolved severity icon; retire via world_events_set_active) is DEFERRED to the human owner's activation checklist.

Doc-sync (this step). This DEV_LOG entry (incl. the phase-close summary below). docs/SYSTEM_BOUNDARIES.md is INTENTIONALLY UNTOUCHED — a verifier script + a package.json line add no table/writer/function/edge (the Phase-15–19 slice-verifier precedent). Per the dark-phase convention (dark phases 11+ carry NO ROADMAP marker; DEV_LOG is authoritative), docs/ROADMAP.md gets NO Phase-20 status marker.

Verify. node --check scripts/verify-phase20-polish.mjs → parses OK. NOT executed against a DB — 0139–0142 are dark/undeployed, so a lit run is deferred (above). The ONLY changes this slice are the new script + the package.json line + this DEV_LOG entry. The M2/M3/M4/M4.5 engine tests are unaffected.

Phase 20 (Polish / expansion — map UI, portraits, icons, events) — CLOSED

Deliverables (all DARK behind phase20_polish_enabled='false'; migrations 0139–0142, forward-only):

- **World Events triad** — 0139 schema (world_events, server-only, scope↔target CHECK) → 0140 service-role idempotent writers (world_events_publish / world_events_set_active, nullable-unique dedup_key, retire-not-delete, both dark-gate-first no-op) → 0141 flag-gated fail-closed read surface (get_world_events, authenticated-only, live+in-scope filter). World Events is a NEW downward-LEAF system: writes ONLY world_events, grants nothing, reads only the static Map for FK validation — no second writer, acyclic.
- **UI asset vocabulary** — 0142 ui_asset_catalog (ONE table discriminated by asset_kind portrait/icon; static Reference/Config, seed-migration-only, NO runtime writer) + get_ui_asset_catalog (flag-gated fail-closed, authenticated-only).
- **Fail-closed frontend** — src/features/events/ (the World Events panel on the galaxy map, top-center, read-only, renders nothing while dark) + src/features/assets/ (the icon resolver + client glyph registry) consuming get_ui_asset_catalog('icon') for severity icons — so the 0142 catalog has a live consumer. Server (flag gate + live-window filter) is the sole visibility control.
- **Portraits** — delivered as DARK seed-ahead vocabulary (ui_asset_catalog portrait rows) pending their live host (captains, itself dark/unsurfaced) — the accepted Phase-6 support_craft_types seed-ahead pattern, NOT speculative UI.
- **Verifier** — verify-phase20-polish (dark posture, shared harness, no lit path).

Preserved human gates (nothing activated by this loop): phase20_polish_enabled stays 'false'; every Phase-11–20 capability flag remains 'false'; migrations 0001–0138 untouched (forward-only — Phase 20 is 0139–0142); NO game_config write; NO lit-path DB run; NO cron scheduled; main untouched; NO merge / deploy / production-apply / workflow-dispatch. Activation (flag flip on a DEV DB, lit verification, deploy) is the human owner's decision. **SAFE FOR HUMAN MERGE REVIEW.**

2026-07-04 — PHASE20-POLISH SLICE 6 — icon resolver (src/features/assets/) + severity icons on the World Events panel, fail-closed

Request. Wire `ui_asset_catalog` into the World Events panel as SEVERITY ICONS — delivering the "icons" polish with a real consumer so `get_ui_asset_catalog/ui_asset_catalog` (0142) are not a dead backend surface. Frontend-only: new `src/features/assets/*` + the extended `WorldEventsPanel.tsx`

- a DEV_LOG entry. No migration, no flag, no cron, no git.

Design decision (self-approved). The STEP-4 split holds: the SERVER owns the icon VOCABULARY (`severity_info/severity_warning/severity_critical` keys + display metadata + stable `asset_ref`); the CLIENT owns the rendered GLYPH per `asset_ref` (a tiny inline-emoji registry — the "files" side, ZERO binary assets). This is the intended architecture, not duplication. The resolver is generic (`asset_kind`-parameterized) but this slice only CONSUMES 'icon'; the seeded PORTRAIT rows stay a server-side vocabulary whose live host (captains) is still dark — a portrait UI consumer remains a documented seed-ahead deferral (the accepted Phase-6 `support_craft_types` pattern), not speculative UI.

Work done — new `src/features/assets/`.

- `assetsTypes.ts` — `UiAsset` (`asset_kind` 'portrait' | 'icon', `asset_key`, `display_name`, `asset_ref`, `category`, `sort_order`) + `GetUiAssetCatalogResult` as a DISCRIMINATED union (`{ok:true; assets?}` | `{ok:false}` — the `isServerLit`-compatible idiom, same reason SLICE 5 used it).
- `assetsApi.ts` — thin `supabase.rpc('get_ui_asset_catalog', { p_asset_kind })` wrapper (the `explorationApi.ts` / `eventsApi.ts` convention): error → `{ ok:false }`, never throws into render.
- `assetGlyphs.ts` — the client "files" side: a registry mapping each SEEDDED icon `asset_ref` (0142) → a tiny inline emoji glyph, with an in-file comment stating the split (server owns the key vocabulary; this file owns the rendered glyph per `asset_ref`). An unrecognized `asset_ref` → undefined → no glyph (fail-safe).

Work done — extended `WorldEventsPanel.tsx` (**NOT a new panel**). `refresh()` now fetches both surfaces together (`Promise.all([getWorldEvents(), getUiAssetCatalog('icon')])`) and stores both; a `useMemo` builds a `Map<asset_key, UiAsset>` from the returned 'icon' rows (empty while dark / on a failed read). Per event, it resolves `severity_${event.severity}` → its `UiAsset` → the glyph via `assetGlyphs[asset.asset_ref]`, rendered next to the title with the asset's `display_name` as `title/aria-label`. The existing severity badge stays; the icon augments it. FAIL CLOSED is unchanged (if `(!isServerLit(result) || (result.events?.length ?? 0) === 0) return null`); any icon miss (dark/empty catalog, unseeded key, unregistered `asset_ref`) renders the event with NO glyph — never breaks the feed.

Doc-sync (this step). This DEV_LOG entry. `docs/SYSTEM_BOUNDARIES.md` needs NO change — another client read-only consumer adds no table, no writer, no cross-system call-edge (it just calls the already-documented `get_ui_asset_catalog` RPC). Stated explicitly so the omission is intentional.

Verify. `npm run build` (`tsc -b + vite build`) — **GREEN** (156 modules, +2 for `src/features/assets/*`; typecheck clean; only the pre-existing >500 kB chunk-size advisory, unrelated). Touched ONLY `src/features/assets/*` (new) + `WorldEventsPanel.tsx` (extended) + this DEV_LOG entry — no migration, no flag, no cron, no git. `phase20_polish_enabled` remains 'false' and untouched. The M2/M3/M4/M4.5 engine tests are backend and unaffected by a fail-closed presentational panel. No lit DB run — while dark the icon catalog is empty (no glyph resolves) and the feed is empty anyway (panel renders nothing); the 0142 catalog now has a live consumer only once the human lights the flag and publishes events. Nothing deployed, main untouched.

2026-07-04 — PHASE20-POLISH SLICE 5 — the World Events display feature (src/features/events/) wired into the galaxy map, fail-closed

Request. The "events polish" — a read-only World Events overlay on the galaxy map, fail-closed, build-verified. Frontend-only: new `src/features/events/*` + the `GalaxyMapScreen.tsx` mount + a DEV_LOG entry. No migration, no flag, no cron, no git.

Design decision (self-approved). The panel renders NOTHING when there are no active events. Because `get_world_events` (0141) returns `{ok:true, events:[]}` while dark (the flag gate empties the feed server-side), an empty feed → nothing rendered → today's production UI is byte-unchanged; when the human later lights `phase20_polish_enabled` AND publishes events, they appear. The server (flag gate + live-window filter) is the SOLE control; the client never decides visibility. This slice builds ONLY the events display (single responsibility). Icons-from-the-asset-catalog is the NEXT slice; portraits stay a server-side vocabulary

with no speculative UI (their live host — captains — is itself dark/unsurfaced, so building portrait UI now would be speculative — deferred).

Work done — new `src/features/events/`.

- `eventsTypes.ts` — `WorldEvent` (`id`, `event_type`, `scope`, `zone_id`, `location_id`, `title`, `body`, `severity` `'info'|'warning'|'critical'`, `starts_at`, `ends_at`) + `GetWorldEventsResult`.
- `eventsApi.ts` — thin `supabase.rpc('get_world_events', { p_location_id: null, p_zone_id: null })` wrapper (the `explorationApi.ts` convention): on a transport/DB error resolves to the normalized fail-closed `{ ok:false }` — never throws into the render path. This minimal cut requests GLOBAL-scope events only (nulls) — always map-relevant, no coupling to selected-location state.
- `WorldEventsPanel.tsx` — mirrors `ExplorationPanel`: a `lifecycleKey` re-fetch trigger, `useActivityPanelGuards()` mounted guard, `refresh()` on mount / `lifecycleKey` change. FAIL CLOSED: `if (!isServerLit(result) || (result.events?.length ?? 0) === 0) return null` — renders nothing while dark (empty) or with no live events. When events exist, a compact READ-ONLY overlay (no actions/ buttons — purely presentational) lists each event's `title`, optional `body`, and a severity-styled badge (`info/warning/critical` color classes — the `ExplorationPanel` badge idiom). Positioned TOP-CENTER, clear of the four existing overlays (`PortNav` top-left, `DockServices` top-right, `Exploration/Mining` bottom-left, `Stop` bottom-right). `data-testid="world-events-panel"` + per-event/-badge testids.
- Wired into `GalaxyMapScreen.tsx` in the same overlay block as `ExplorationPanel/MiningPanel`, with the SAME `lifecycleKey` expression the siblings use, and the same server-driven-visibility comment.

Deviation from the brief (reported). `GetWorldEventsResult` is a DISCRIMINATED union (`{ok:true; events?}` | `{ok:false}`) rather than the brief's flat `{ ok:boolean; events? }`. A flat shape makes the shared `isServerLit()` guard's `Extract<T,{ok:true}>` narrow to never (fragile render types); the union is exactly the `explorationTypes.ts` idiom the brief told me to mirror, so the exact fail-closed line compiles to clean types. Runtime shape and behavior are identical.

Doc-sync (this step). This `DEV_LOG` entry. `docs/SYSTEM_BOUNDARIES.md` needs NO change — a client read-only consumer adds no table, no writer, and no cross-system call-edge (it just calls the already- documented `get_world_events` RPC). Stated explicitly here so the omission is intentional, not missed.

Verify. `npm run build (tsc -b + vite build)` — **GREEN** (154 modules transformed, typecheck clean; only the pre-existing >500 kB chunk-size advisory, unrelated). Touched ONLY `src/features/events/*` (new) + `GalaxyMapScreen.tsx` (mount) + this `DEV_LOG` entry — no migration, no flag, no cron, no git. `phase20_polish_enabled` remains `'false'` and untouched. The M2/M3/M4/M4.5 engine tests are backend and unaffected by a fail-closed presentational panel. No lit DB run — the panel renders nothing until the human lights the flag and publishes events; nothing deployed, `main` untouched.

2026-07-04 — PHASE20-POLISH SLICE 4 — the UI asset-key vocabulary `ui_asset_catalog` (static reference table + seed) + its flag-gated read surface `get_ui_asset_catalog(...)` (0142)

Request. The portrait/icon Reference catalog — the server-authoritative asset-key vocabulary the Phase-20 frontend polish will render. ONE forward-only migration + same-step doc-sync. No flag flipped, no `src/`, no `verifier/package.json` change, no cron, no shipped-migration edit, no git.

Design decision (self-approved). ONE static reference table `ui_asset_catalog` discriminated by `asset_kind` (`'portrait'|'icon'`), NOT two near-identical parallel tables — portraits and icons share the same shape (`key` → `display metadata` → `asset_ref`), so a single leaf catalog avoids a duplicated parallel system (DRY / no-spaghetti; a future third kind is an additive CHECK change, not a new table). Server-owned VOCABULARY, frontend-owned FILES: server rows reference a stable `asset_key` (e.g. `world_events.severity 'critical'` → an icon key; a future captain → a portrait key); the image files + `key`→`file` resolution live in the FRONTEND (`asset_ref` = the stable identifier the client resolves). Pure static leaf: SEED-ONLY, NO runtime writer (edited only by forward-only seed migrations, like the static Map — no sole-writer function, no second writer anywhere), references nothing, exposed only through a flag-gated fail-closed read RPC so the ENTIRE Phase-20 surface stays uniformly dark.

Work done — 0142_phase20_ui_asset_catalog.sql (forward-only; edits NO shipped migration 0001-0141).

- **(a) Table.** `ui_asset_catalog`: `asset_kind` (`portrait/icon` CHECK), `asset_key`, PK (`asset_kind`, `asset_key`), `display_name`, `asset_ref` (stable frontend identifier — not a file path/binary), `category`, `sort_order` default 0, `is_active` default true (retire without deleting), `created_at/updated_at`. SERVER-ONLY (the 0103 `mining_fields` / 0139 `world_events` posture): RLS enabled, revoke all ... from public, anon, authenticated — no client read, no client write, NO runtime writer.
- **(b) Seed** (on conflict (`asset_kind`, `asset_key`) do nothing; minimal, no bloat): five icons — `severity_info/severity_warning/severity_critical` (pairing with `world_events.severity`) + `event_notice/event_world_state`; three portraits — `captain_default/captain_veteran/ faction_pirate`. `asset_ref` values are stable frontend identifiers (e.g. `icon.severity.critical`), not file paths.

- **(c)** `get_ui_asset_catalog(p_asset_kind text default null) → jsonb, stable security definer, set search_path = public,` reusing the exact 0141 fail-closed envelope: while `phase20_polish_enabled=false → return {ok:true, assets:[]}` WITHOUT reading the table; enabled → active rows (`is_active`), optionally filtered by `p_asset_kind` when non-null, ordered (`asset_kind, sort_order, asset_key`). ACL revoke ... from public, anon; grant ... to authenticated — read-only, no write path.

Boundary discipline. `ui_asset_catalog` is Reference/Config (seed-migration only, no runtime writer, no sole-writer function) — no second writer anywhere. `get_ui_asset_catalog` reads ONLY its own table + the master flag and references nothing → a pure downward leaf, no new cross-system call-edge, acyclic; grants nothing, writes nothing.

Doc-sync (this step). `docs/SYSTEM_BOUNDARIES.md`: §1 gains `ui_asset_catalog` under Reference/Config (seed-migration only, NO runtime writer; server-only read); §2 gains a **UI Assets** read-leaf exposing `get_ui_asset_catalog` (flag-gated → empty while dark; authenticated-only; reads only its own table + the flag; references nothing → pure leaf). Folded in the one-line doc-consistency fix noted last review: the World Events §2 version tag bumped (`0139/0140`) → (`0139/0140/0141`), and the §1 `world_events` read-access cell updated from "future read RPC" to the now-shipped `get_world_events` (0141). This DEV_LOG entry.

Retirement. None — a permanent static catalog + permanent read surface. `phase20_polish_enabled` remains the permanent Phase-20 master gate (retires only on human activation).

Posture / gates. Flips NO flag — `phase20_polish_enabled` still 'false' and untouched (the read RPC only reads it to gate); edits no `0001-0141`; table has NO client policy and NO runtime writer (seed block only); the read RPC is granted to authenticated only (no anon/public execute, no write/insert path); no `src/`, no cron, no git. Backend-only and dark/undeployed, so **no lit DB run** — a lit apply proving dark→empty and enabled→seeded-vocabulary is the human owner's activation-checklist job (run with the flag flipped on a DEV DB). The M2/M3/M4/M4.5 engine tests are unaffected — no engine path reads `ui_asset_catalog` or calls `get_ui_asset_catalog`.

2026-07-04 — PHASE20-POLISH SLICE 3 — the World Events flag-gated READ surface `get_world_events(...)` (fail-closed, authenticated-only) (0141)

Request. The consumer of `world_events` — the flag-gated, fail-closed client READ surface — after the producer (0140), mirroring the command→read-surface order. ONE forward-only migration + same-step doc-sync. No flag flipped, no `src/`, no verifier/package.json change, no cron, no shipped-migration edit, no git.

Design decision (self-approved). `get_world_events(p_location_id, p_zone_id)` takes the display context as PARAMETERS the client already holds from the map, rather than resolving the player's ship position server-side. World events are PUBLIC presentational world info (no per-player secret, no cheat vector — unlike the hidden `exploration_sites/mining_fields` that MUST resolve server-side), so a parameterized read keeps World Events a PURE downward LEAF: it reads ONLY its own `world_events` table + the `phase20_polish_enabled` master flag, adding NO cross-system call-edge to Main-Ship/Presence. The server stays authoritative over WHAT IS SHOWN (the flag gate + `is_active` + the active-time-window), the only authority that matters for presentational info.

Work done — `0141_phase20_world_events_read_surface.sql` (forward-only; edits NO shipped migration `0001-0140`).

`get_world_events(p_location_id uuid default null, p_zone_id uuid default null) → jsonb, stable security definer, set search_path = public` (reads the RLS-locked, client-revoked `world_events` and returns curated rows). Reuses the exact `0087/0101/0106` read-surface convention (`jsonb {ok, events:[...]}` envelope — no new convention):

- **Fail-closed FIRST.** `if not coalesce(cfg_bool('phase20_polish_enabled'), false) → return {ok:true, events:[]}` immediately, WITHOUT reading the table (the server-rejected-while-dark proof; the read-side consumer of the master flag). World events are public presentational info, so the dark answer is an empty list (frontend renders nothing), not a reject envelope.
- **Live + in-scope filter (enabled).** Returns only rows that are BOTH currently LIVE (`is_active AND starts_at <= now()` AND (`ends_at is null OR ends_at > now()`)) AND IN SCOPE (`scope='global'` always; `scope='zone'` only when `zone_id = p_zone_id`; `scope='location'` only when `location_id = p_location_id`). Presentational columns only (`id, event_type, scope, zone_id, location_id, title, body, severity, starts_at, ends_at`). Deterministic order: severity rank (critical → warning → info) then `starts_at desc`.
- **ACL.** revoke execute ... from public, anon; grant execute ... to authenticated; (the map/dashboard are behind auth — the `0087/0101/0106` auth-guarded read idiom). No write path is exposed.

Boundary discipline. World Events stays a downward LEAF: `get_world_events` reads ONLY `world_events`

- the master flag and writes nothing; the client passes its display context, so NO new cross-system call-edge is introduced (no Main-Ship/Presence ship-position resolve). Acyclic; grants nothing.

Doc-sync (this step). docs/SYSTEM_BOUNDARIES.md: §2 World Events row gains `get_world_events` under "exposes" (read-only, flag-gated → empty while dark; reads only `world_events` + the master flag; client passes display context so no new call-edge; still a downward leaf). This DEV_LOG entry.

Retirement. None — a permanent read surface. `phase20_polish_enabled` remains the permanent Phase-20 master gate (retires only on human activation).

Posture / gates. Flips NO flag — `phase20_polish_enabled` still 'false' and untouched (this RPC only reads it to gate); edits no 0001-0140; granted to `authenticated` only (no anon/public execute, no write/insert path); no `src/`, no cron, no git. Backend-only and dark/undeployed, so **no lit DB run** — a lit apply proving dark→empty and enabled→live+in-scope filtering is the human owner's activation-checklist job (run with the flag flipped on a DEV DB). The M2/M3/M4/M4.5 engine tests are unaffected — no engine path calls `get_world_events`.

2026-07-04 — PHASE20-POLISH SLICE 2 — the World Events sole-writer functions `world_events_publish` / `world_events_set_active` (service-role, idempotent) + the `dedup_key` idempotency column (0140)

Request. Give `world_events` its promised sole writer (the producer) BEFORE the read surface (the consumer), mirroring the established command→read-surface order. ONE forward-only migration + same-step doc-sync. No flag flipped, no `src/`, no verifier/package.json change, no cron, no shipped-migration edit, no git.

Design decision (self-approved). The writer is **service-role-only** (SECURITY DEFINER, client-revoked, granted only to `service_role` — the 0021/0135 lockdown). That keeps World Events server-authoritative and structurally forbids any player-to-player event injection — there is NO client publish path, so events can never be a PvP / player-interaction vector (Online Presence & Visibility v1 stays deferred). Idempotent via a nullable-unique `dedup_key`: a retried publish with the same key returns the EXISTING event id, never a duplicate (the idempotent-command law); a NULL key = an ad-hoc, non-deduplicated event (a permanent optional key, not a shim). Retirement is a status flip (`is_active=false`), never a delete (no destructive cleanup).

Work done — `0140_phase20_world_events_writer.sql` (forward-only; edits NO shipped migration 0001-0139).

- **(a) Idempotency storage.** `alter table public.world_events add column dedup_key text; + a partial unique index world_events_dedup_key_idx ... (dedup_key) where dedup_key is not null` (idempotency ONLY over non-null keys — unlimited ad-hoc events coexist).
- **(b) `world_events_publish(...)` → `uuid`** — SECURITY DEFINER, `set search_path = public;` THE sole insert path. Validates (raises on violation, the leaf-writer exception idiom since it returns a bare uuid) `event_type/scope/severity` membership + the `scope↔target` invariant, mirroring the 0139 CHECKs exactly, and that a supplied `zone_id/location_id` exists (a DOWNWARD read of the static Map — the already-noted relationship, no new edge). Idempotent: a non-null `dedup_key` uses `insert ... on conflict (dedup_key) where dedup_key is not null do nothing` returning `id` with a fallback `select` of the existing `id`; a NULL key always inserts a fresh event.
- **(c) `world_events_set_active(event_id, is_active)` → `void`** — SECURITY DEFINER; flips `is_active` + bumps `updated_at`; the retire/reactivate path, NEVER a delete.
- **(d) ACL lockdown.** `revoke execute ... from public, anon, authenticated; grant execute ... to service_role;` for BOTH functions — `service_role` only, never clients.

Deviation from the STEP-2 brief (reported). Both writers gate on `phase20_polish_enabled` FIRST and no-op while false (publish → returns NULL; `set_active` → returns without writing), BEFORE any validation or write. The brief did not enumerate this gate; I added it because SLICE 1's shipped 0139 flag description commits that "any future World Events writer/processor must no-op" while false — omitting the gate would leave that shipped law text contradicting the code (a defect per the engineering principles). It is also the pervasive reject-before-any-read idiom (`location_investment_invest:74-78`), is strictly more conservative (darker), and does not change the enabled-path publish/dedup/set_active behavior the brief specified. The planner's "service-role-only alone keeps it dark" rationale (forbidding the PvP-injection vector) is fully compatible with also gating.

Boundary discipline. `world_events` now has its concrete sole writers (`world_events_publish` / `world_events_set_active`) — one write path per table, no second writer. World Events stays a downward LEAF: the ONLY cross-system access is a DOWNWARD read of the static Map (`zones/locations`) to validate a supplied FK target — no NEW call-edge, acyclic; it writes ONLY `world_events` and grants no rewards (one-directional pipeline law).

Doc-sync (this step). docs/SYSTEM_BOUNDARIES.md: §1 `world_events` sole writer updated from "its own future function" to the concrete `world_events_publish` / `world_events_set_active` (service-role only; idempotent via `dedup_key`; retire-not-delete); §2 World Events row records both functions, the dark-gate-first no-op, the Map-FK downward read, and the no-client-path / no-delete forbiddens. This DEV_LOG entry.

Retirement. None new. `dedup_key` NULL-means-ad-hoc is a permanent optional key, not a shim. `phase20_polish_enabled` remains the permanent Phase-20 master gate (retires only on human activation).

Posture / gates. Flips NO flag — `phase20_polish_enabled` still `'false'` and untouched; edits no `0001-0139`; no client grant on either function (service-role only); no `src/`, no cron, no git. Backend-only and dark/undeployed, so **no lit DB run** — a lit apply proving publish-then-dedup (same key → same id, no duplicate) and `set_active` (retire/reactivate, no delete) is the human owner's activation-checklist job (run with the flag flipped on a DEV DB). The M2/M3/M4/M4.5 engine tests are unaffected — no engine path calls these service-role functions or reads `world_events`.

2026-07-04 — PHASE20-POLISH SLICE 1 — the Phase-20 dark master flag `phase20_polish_enabled` + the World Events schema `world_events` (0139)

Request. Phase 20 (Polish / expansion — map UI, portraits, icons, events; ROADMAP :95) first build slice: ONE forward-only migration seeding the dark master flag + creating the World Events foundation table, with same-step doc-sync. No flag flipped, no `src/`, no verifier/package.json change, no cron, no shipped-migration edit, no git.

Design decision (self-approved, grounded in the docs). **World Events is a NEW server-authoritative downward-LEAF system**, the sole writer of its own `world_events` table — presentational, timed world happenings (a "pirate surge in Zone X" notice, a seasonal banner, a world-state highlight) that the map / dashboard will READ (via a later flag-gated read RPC) to satisfy Phase 20's "events" polish goal. It is a PURE leaf honoring the one-directional pipeline law (ROADMAP standing law 3): it NEVER writes `zone_state/location_state` (World State's tables — so it is NOT a second writer to World State), `fleets/combat_*/reward_grants`, or any other system's table, and it grants no rewards; it only READS the static Map (zones/locations) for FK integrity. Nothing depends on writing it. It is fail-closed and server-only — no client read/write path this slice; a later slice adds ONE flag-gated read RPC (the only client path) and later still a service-role writer to publish/expire events.

Work done — `0139_phase20_world_events_flag_and_schema.sql` (**forward-only; edits NO shipped migration 0001-0138**).

- **(a) Config.** Seeds the Phase-20 dark master flag `phase20_polish_enabled='false'` (on conflict (key) do nothing). Every Phase-20 read surface must gate on this FIRST and return nothing while false; any future writer/processor no-ops while false. NOT flipped true.
- **(b) Schema.** Creates `public.world_events`: `id` uuid pk, `event_type` (notice/world_state/ seasonal CHECK), polymorphic scope (global/zone/location CHECK) with nullable `zone_id`→ `zones(id)` / `location_id`→`locations(id)` (Map FK targets, on delete cascade) and a CHECK enforcing the scope↔target invariant (global ⇒ both null; zone ⇒ `zone_id` set & `location_id` null; location ⇒ `location_id` set & `zone_id` null), `title/body`, `severity` (info/warning/critical default info), `is_active` default true (retire without deleting — no destructive cleanup), `starts_at/ends_at` (null = open-ended), `created_at/updated_at`. Indexes: active-window (`is_active`, `starts_at` desc) + partial `zone_id/location_id` scoped-lookup indexes.
- **Fail-closed posture (the 0103 mining_fields / market_offers server-only idiom).** RLS enabled with NO client policy and revoke all ... from public, anon, authenticated — no client read, no client write. Sole writer will be World Events' OWN future service-role function; no runtime writer exists this slice and no other system writes this table.

Boundary discipline. One sole writer per table (`world_events` → World Events, deferred to its own future writer — the 0128/0103 schema-first idiom). Downward LEAF: no new cross-system CALL edge exists yet (no function is created here); the only relationship is the read-only Map FK. Acyclic and no second writer to any table — World Events never writes World State's `zone_state/location_state`.

Doc-sync (this step). `docs/SYSTEM_BOUNDARIES.md`: §1 ownership matrix gains the `world_events` row under a NEW **World Events** system (sole writer = its own future service-role writer; server-only, no client surface; DARK behind `phase20_polish_enabled`); §2 gains the World-Events contract row (owns `world_events`; a downward leaf — reads only the static Map for FK integrity; writes nothing else; grants nothing; no client surface yet). No call-edge invented (none exists yet). This DEV_LOG entry.

Retirement / activation. `phase20_polish_enabled` is a permanent capability gate (the Phase-20 master flag), not a transitional shim — it retires only when the human owner activates Phase 20.

Posture / gates. Ships the flag `'false'` — NOT flipped; edits no `0001-0138`; touches no `src/`, no other flag, no cron, no git. Backend-only and dark/undeployed, so **no lit DB run** — a lit run (apply 0139 on a DEV DB, confirm the flag is false, the table exists server-only, no client read/write) is the human owner's activation-checklist job. The M2/M3/M4/M4.5 engine tests are unaffected by a new dark, server-only table (no engine path reads or writes `world_events`).

2026-07-04 — WORLD-BALANCE-P19 CLEANUP — restore the 0092 docked-resolve dedup that 0136 regressed; re-route all three Trade Market RPCs through mainship_resolve_docked_location (0138)

Request. Act on the highest-priority world-economy cleanup finding (F1 from the baseline audit) with ONE new forward-only migration + same-step doc-sync. No flag write, no src/, no verifier/package.json change, no shipped-migration edit, no git.

Bug (F1 — duplication regression in 0136). Migration 0092 (trade_market_1) had extracted the copy-pasted ~10-line "resolve docked location" block (mainship_space_validate_context → require_at_location → read the present/location fleet's current_location_id) into ONE shared read-only helper public.mainship_resolve_docked_location(ship) and repointed all three Trade Market RPCs to it. But 0136 (price drift) rebuilt get_market_offers / market_buy / market_sell from the STALE pre-0092 bodies (0087/0089/0090) to add the trade_effective_price price composition — and in doing so **re-inlined** the docked block into all three (0136:295–305, 376–384, 472–480) and re-declared the v_ctx jsonb local 0092 had dropped. That silently reverted the dedup (the SAME non-trivial logic in three places again) and orphaned the helper from the trade path (it stayed in use only by 0133).

Fix — 0138_world_balance_p19_trade_docked_helper_reuse.sql (forward-only; edits NO shipped migration). create or replaces the three functions to the EXACT 0136 bodies, changing ONLY: (a) each re-inlined docked block → v_loc := public.mainship_resolve_docked_location(v_ship); followed by the SAME if v_loc is null then ... 'not_docked' ... end if; each already had; and (b) drop the now-unused v_ctx jsonb; local. A line-for-line diff of each function region (0136 → 0138) shows ONLY those two changes per function and nothing else. **BEHAVIOR-IDENTICAL:** both inline null-paths (not at_location; no matching fleet row) already collapsed to one not_docked reason, and the helper returns NULL for both, mapped to the same not_docked. Everything else is byte-for-byte 0136: the dark trade_market_enabled server-reject, mainship_resolve_owned_ship, the per-ship mainship_space_lock_context, the idempotency replay, the trade_effective_price composition on EVERY price, the receipt writes, and the same revoke ... from public, anon / grant ... to authenticated ACLs.

Posture / gates. Adds NO table / column / writer / flag / cross-system edge — the helper and the Trade Market → Main-Ship read edge already existed and were already documented. The feature stays **DARK** behind trade_market_enabled='false'; this migration flips NO flag and edits no 0001–0137.

Doc-sync (this step). This DEV_LOG entry. docs/SYSTEM_BOUNDARIES.md needs **no edit:** the Trade-Market row (line ~89, "the docked-location context (via the shared Main-Ship helper mainship_resolve_docked_location)") and the Main-Ship row (line ~86, "mainship_resolve_docked_location ... called DOWNWARD by the Trade Market RPCs") described the INTENDED end-state — 0136 was the drift, and 0138 makes both statements true of the shipped code again. No remaining contradiction found.

Retirement. None — this removes a regression and adds no temporary code. mainship_resolve_docked_location is the permanent single source for docked-location resolution across Trade Market and Location Investment.

Verify. Line-for-line diff of each of the three function regions (0136 → 0138) confirms the ONLY differences are the three helper-call substitutions and the three dropped v_ctx locals; no other change leaked in. Not executed against a DB (dark; a lit run is the human owner's activation-checklist job).

2026-07-04 — WORLD-BALANCE-P19 SLICE 4 (FINAL) — the dark-posture verifier verify-world-balance.mjs + verify:world-balance; Phase 19 CLOSED

Request. Phase 19 final slice: ONE new verify script (the verify-location-investment.mjs analogue)

- one package.json line + same-step doc-sync. NO migration change, NO flag write, NO lit-path DB run, NO src/, NO new RPC, NO cron, no git.

Work done — scripts/verify-world-balance.mjs (mirrors verify-location-investment.mjs point-for-point; ZERO inline harness copies — imports the shared Abort/resolveEnv/createReporter/ createUserFactory from scripts/lib/verify-harness.mjs + teardownVerifier from scripts/lib/verifier-teardown.mjs, the same admin/anon/throwaway-user/cfgVal scaffold, the same .catch/.finally teardown with NO flag entry passed — this verifier touches no flag). Proves migrations 0135–0137 ship exactly as built and fully dark, with anon/authenticated clients only. Five assertion groups (CODE/lock-keyed, String() storage-form-tolerant compares):

1. **Config presence (READ-ONLY)** — world_balance_enabled='false' + all eight tunables at their seeded values (world_balance_defeat_window_seconds='3600', world_balance_price_pressure_coeff='0.5', world_balance_price_drift_rate='0.1', world_balance_price_multiplier_min='0.5', world_balance_price_multiplier_max='2.0', world_balance_field_depletion_per_extract='0.1', world_balance_field_regen_rate='0.02', world_balance_field_reserve_min='0.1').

2. **Internal World-State functions locked** — `worldstate_current_price_multiplier`, `worldstate_field_remaining`, `worldstate_deplete_field`, and `worldstate_tick` are DENIED to BOTH anon and authenticated (service-role-only, 0135-0137); VALID-shaped uuid args so the denial proves the lock, not argument validation.
3. `mining_field_state` **server-only** — anon + authenticated SELECT DENIED (no client policy/grant — the `mining_fields` posture), and a direct authenticated INSERT DENIED (no client write path).
4. `location_state.price_multiplier` **dark no-op** — public-readable; every existing row equals 1.0 (composition inert while dark); a fresh DB with 0 rows does not fail (the column being selectable is the proof).
5. **Static catalogs — no second writer** — `market_offers` + `mining_fields` keep NO client write path (direct authenticated INSERT/UPDATE DENIED), confirming Phase 19 added no runtime writer to either (drift/depletion live on the World-State-owned `location_state/mining_field_state`).

`package.json` — one line added adjacent to `verify:location-investment`: `"verify:world-balance": "node scripts/verify-world-balance.mjs"`.

Lit-path DEFERRED (the verify-location-investment stance verbatim). The script NEVER writes `game_config` / NEVER flips `world_balance_enabled`; it exercises NO lit path. Lit-path verification — flag on → the tick raises pressure at recently-defeated locations and decays it; drifts `location_state.price_multiplier` toward the danger-premium target so `trade_effective_price` moves the charged/paid price in lockstep with the displayed price; depletes `mining_field_state.reserve_fraction` on each extraction (bundle yield thins, floored) while the tick regenerates it toward 1.0 — is deferred to the human owner's activation checklist (flip the flag on a DEV database and run the lit checks there, never here). Because 0135-0137 are not deployed, local verification is `node --check scripts/verify-world-balance.mjs` only (**parses OK**).

Doc-sync (same step). `docs/SYSTEM_BOUNDARIES.md` INTENTIONALLY UNTOUCHED — a verifier script + a `package.json` line add no table, writer, function, or cross-system edge (the Phase-15/16/17/18 slice-verifier precedent). Only this DEV_LOG entry is the doc change.

Phase 19 (World balance / living economy) CLOSED — backend + verifier deliverables:

- 0135 — PIRATE PRESSURE: wires the `defeat_pressure` seam in `worldstate_tick()` — the pressure decay TARGET gains a flag-gated danger term from recent `combat_reports` defeats (read DOWNWARD); the dark master flag `world_balance_enabled='false' + world_balance_defeat_window_seconds='3600'`.
- 0136 — PRICE DRIFT: the World-State-owned `location_state.price_multiplier` (tick-driven, gated) + the read helper `worldstate_current_price_multiplier` + the ONE composition helper `trade_effective_price`, routed through all three Trade Market functions (`display == charged/paid`); no runtime writer added to `market_offers`.
- 0137 — FIELD DEPLETION: the World-State-owned `mining_field_state` reserve (lazy rows) + `worldstate_field_remaining` (read) + `worldstate_deplete_field` (sole reserve write, NO-SOFTLOCK floor) + a gated tick regen, composed into `mining_extract` (bundle scaled by reserve, depleted once per real extraction); no runtime writer added to `mining_fields`.
- `verify-world-balance.mjs` — the dark-posture verifier + `verify:world-balance`.

Human gates preserved. `world_balance_enabled` stays 'false' and ALL `world_balance*` dynamics remain behind it; ALL Phase 11-18 flags remain 'false'; migrations 0001-0134 untouched (forward-only — Phase 19 is 0135-0137); backend-only (no `src/**`); no `game_config` write; no lit-path DB run; no cron scheduled (the 60s `process_location_state_ticks()` → `worldstate_tick()` path is reused); no main touch; no merge / deploy / production apply / workflow dispatch — activation is the human owner's decision. SAFE FOR HUMAN MERGE REVIEW.

Verify. `node --check scripts/verify-world-balance.mjs` → parses OK. `git status --porcelain` shows the ONLY changes are the new script + `package.json` + this DEV_LOG entry; no migration, `SYSTEM_BOUNDARIES.md`, flag, `src/`, or main touched. The verifier was NOT executed against a DB (the gates forbid a lit/production DB run) — dark-posture proof is by `node --check` + the mirrored precedent.

2026-07-04 — WORLD-BALANCE-P19 SLICE 3 (FINAL) — RESOURCE-FIELD DEPLETION (dark): World-State `mining_field_state` reserve, composed into `mining_extract`, regenerated by the tick (0137)

Request. Phase 19 third/final mechanic: resource-field depletion, dark-gated, as ONE coherent producer+consumer vertical slice — one forward-only migration 0137 + same-step doc-sync, one revertible commit, nothing dead. No flag flip, no new cron, no `src/`, no git, no edit to any shipped migration.

Design (self-approved; "world-state owns world-state" + no-second-writer / no-cycle / NO-SOFTLOCK). `mining_fields` stays static server-only Reference/Config (NO runtime writer). Depletion is NEW World-State-owned state — a LAZY per-field reserve — composed with the field's yield at extraction time and REGENERATED over time by the tick.

Work done — 0137 (producer + consumer together):

- `mining_field_state` — NEW World-State-owned table (`field_id` PK → `mining_fields`, `reserve_fraction` numeric default 1.0 check [0,1], timestamps). Rows created LAZILY on first depletion (upsert) — no seeding, no dead rows; an un-mined field has NO row and reads as full (1.0). Server-only (RLS on, no client policy/grant — the `mining_fields` posture). World State is the SOLE writer.
- `worldstate_field_remaining(field)` — flag-gated read (internal/service-role): 1.0 while dark OR when no row, else `reserve_fraction`.
- `worldstate_deplete_field(field)` — flag-gated writer (internal/service-role): no-op while dark; else upserts `reserve_fraction` down by `world_balance_field_depletion_per_extract`, hard-floored at `world_balance_field_reserve_min` (a depleted field never fully dies — NO-SOFTLOCK). THE sole reserve write on extraction.
- `worldstate_tick()` **re-created** = the 0136 body verbatim except a gated field-regen pass (step 5): when `world_balance_enabled=true`, nudge every `mining_field_state.reserve_fraction` toward 1.0 by `world_balance_field_regen_rate` per tick, clamped ≤ 1.0 (touching only not-yet-full rows). While dark the block is skipped.
- `mining_extract` **re-created** = the 0104 body verbatim except ONE gated block: inside if `cfg_bool('world_balance_enabled')`, read `worldstate_field_remaining(field)` and scale each item qty by it with a per-item floor of 1 (`greatest(1, round(qty × reserve))`) BEFORE snapshotting the bundle (into a new local `v_bundle`, used in BOTH the row insert and the result envelope), then call `worldstate_deplete_field(field)` once. The wrapper `command_mining_extract` is UNCHANGED (it passes the writer's `pending_bundle` through).

Deplete-once placement (verified against the real idempotency structure). In `mining_extract` the receipt lookup (step 6) RETURNS on a replay of (ship, request_id) BEFORE the extraction-row insert (step 12) and the receipt insert (step 13). The reserve read + bundle scale (step 11.5) and the `worldstate_deplete_field` call (step 12.5) sit in the success path AFTER the row insert — unreachable on a replay — so depletion fires EXACTLY ONCE per REAL extraction and NEVER on replay (no double-deplete). Reserve is read BEFORE this extraction's depletion, so the bundle reflects the pre-extraction reserve (first extraction from a full field yields full, then the field drops to 0.9, etc.).

Reused vs new config. Reused (NOT re-seeded): `world_balance_enabled` (0135). New this slice, all consumed: `world_balance_field_depletion_per_extract='0.1'` (~10%/extraction), `world_balance_field_regen_rate='0.02'` (~full in ~45 ticks), `world_balance_field_reserve_min='0.1'` (floor — worked fields thin out but recover and never die).

Dark-identical invariant (reserve = 1.0 → bundle verbatim + tick untouched; gated in tick, extract, AND both functions). With `world_balance_enabled='false'`:

1. **Tick** — the regen pass is entirely inside if `v_wb_enabled`; while dark `mining_field_state` is untouched and the `location_state/zone_state` logic is byte-for-byte 0136 → a dark tick is byte-identical.
2. `worldstate_field_remaining` — returns 1.0 while dark regardless of any stored row.
3. `mining_extract` — the scale + deplete are entirely inside if `v_wb` (read once via `cfg_bool('world_balance_enabled')` only on the success path after the cooldown check); while dark `v_bundle = v_field.reward_bundle_json` verbatim and `worldstate_deplete_field` is never called → the stored `pending_bundle_json` and the returned envelope are identical to 0104. All early-reject paths (dark mining, no field, cooldown) never even read `world_balance_enabled`.
4. `worldstate_deplete_field` — no-ops while dark (defense in depth: a stray caller can't deplete).

New downward edges (acyclic). Mining → World State: read `worldstate_field_remaining` + call the writer-function `worldstate_deplete_field` — both DOWNWARD (an activity depending on the world-state leaf). ACYCLIC: World State never reads or calls Mining. NO new edge into `mining_fields` (still static, no runtime writer, no second writer). World State stays the SOLE writer of `mining_field_state`; Mining writes only `mining_extractions` and still deposits ONLY via `Reward.grant('mining', ...)`.

Doc-sync (same step). docs/SYSTEM_BOUNDARIES.md: §1 new `mining_field_state` row (World State sole writer, server-only); §2 World State contract (the table + the two functions + tick regen; "must NOT add a runtime writer to `market_offers` or `mining_fields`"); §3 Mining row (reads `worldstate_field_remaining` + calls `worldstate_deplete_field` DOWNWARD, scales the bundle while on, verbatim while dark, never writes `mining_fields/mining_field_state`, still deposits only via `Reward.grant`); §4 a new "Mining → World State field-depletion edges are acyclic" note. docs/ROADMAP.md: Phase-19 row carries no per-phase status marker (dark phases 11+), left UNTOUCHED (noted per the instruction). **Phase 19's three mechanics (pirate pressure 0135, price drift 0136, field depletion 0137) are now all implemented DARK.**

Retirement / activation. `world_balance_enabled` is the permanent Phase-19 gate. Lit-path verification (flag on a DEV DB → extract repeatedly → the bundle thins toward the floor while the reserve upserts down; idle ticks regen it back toward 1.0; a replay never double-depletes) is deferred to the human's activation checklist. This slice flips NO flag.

Human gates preserved. `world_balance_enabled` stays 'false'; ALL Phase 11–18 flags remain 'false'; migrations 0001–0136 untouched (forward-only — 0137 is new); no new cron (reuses the 60s `process_location_state_ticks()` → `worldstate_tick()` path); backend-only (no `src/**`); no runtime `game_config` write; no lit-path/production DB run; no `main` touch; no merge/deploy/workflow dispatch. SAFE FOR HUMAN MERGE REVIEW.

Verify. Forward-only: 0137 is a new file; the only changes are 0137, docs/SYSTEM_BOUNDARIES.md, and this DEV_LOG entry — no shipped migration edited. No second writer to `mining_fields` (grep-confirmed: 0137 never writes it; the only new table writer is World State on `mining_field_state`). The dark-identical property + the deplete-once-never-on-replay property are established

by the logic walk above against the real 0104 idempotency structure. **The M2–M5 / mining verify suites could NOT be run locally:** they connect to a live Supabase (service-role key in `.env.local`), which the human gates forbid and where 0137 is not deployed — so NO green/red claim is made; the dark-safety argument rests on the logic walk + forward-only proof (the 0132–0136 dark-slice precedent).

2026-07-04 — WORLD-BALANCE-P19 SLICE 2 — PRICE DRIFT (dark): World-State price multiplier folded into `location_state`, composed into all three Trade Market prices (0136)

Request. Phase 19 second mechanic: price drift, dark-gated, as ONE coherent producer+consumer vertical slice — one forward-only migration 0136 + same-step doc-sync, one revertible commit, nothing dead. No flag flip, no new cron, no `src/`, no git, no edit to any shipped migration.

Design (self-approved; "world-state owns world-state" + no-second-writer / no-cycle). `market_offers` stays STATIC Reference/Config (NO runtime writer). Price drift is NEW World-State-owned state FOLDED into the existing `location_state` (the tick already iterates it — no parallel table), COMPOSED with the static base price at read/transaction time.

Work done — 0136 (producer + consumer together):

- `location_state.price_multiplier` — ONE new column, numeric not null default 1.0 check (> 0). add column ... default 1.0 not null backfills every existing row to a no-op 1.0. World State stays the SOLE writer — only `worldstate_tick` writes it.
- `worldstate_tick()` **re-created** = the 0135 body verbatim except a flag-gated multiplier drift. When `world_balance_enabled=true`: the multiplier nudges toward $\text{target} = 1.0 + \text{world_balance_price_pressure_coeff} \times \text{clamp}((\text{pressure} - \text{baseline}) / (\text{max} - \text{baseline}), 0, 1)$ by `world_balance_price_drift_rate` per applied tick, hard-clamped to `[world_balance_price_multiplier_min, world_balance_price_multiplier_max]` — the STEP-1 target-based / self-correcting / bounded philosophy, reusing the SAME baseline/max pressure config (not duplicated).
- `worldstate_current_price_multiplier(loc)` — the World-State read helper (internal/service-role). Flag-gated: returns 1.0 while dark regardless of the stored column (the provable dark guarantee), else the row's `price_multiplier` (1.0 if no row).
- `trade_effective_price(base, loc)` = $\text{greatest}(1, \text{round}(\text{base} \times \text{worldstate_current_price_multiplier}(\text{loc})))$ — the ONE shared composition helper (integer credits, ≥ 1 floor; the round/floor rule decided here once). Internal.
- **All three trade functions re-created** = 0087/0089/0090 verbatim EXCEPT the price read: `get_market_offers` composes BOTH displayed prices; `market_buy` composes the charged `sell_price`; `market_sell` composes the paid `buy_price`. So DISPLAYED == CHARGED/PAID always — no drift-vs-transaction exploit — and the composition lives in exactly ONE place. Docking resolution, dark gate, locks, idempotency, and grants are all preserved verbatim.

Reused vs new config. Reused (NOT re-seeded): `world_balance_enabled` (0135, the master gate) and the pressure `baseline/max` (0032). New this slice, all consumed: `world_balance_price_pressure_coeff='0.5'` (up to +50% at max danger), `world_balance_price_drift_rate='0.1'` (10%/tick toward target), `world_balance_price_multiplier_min='0.5'`, `world_balance_price_multiplier_max='2.0'` (a bounded premium that breathes, never runs away).

Dark-identical invariant (multiplier = 1.0 while dark, gated in BOTH the tick and the read helper). Two independent guards make the whole slice a no-op while `world_balance_enabled='false'`:

1. **Tick:** the multiplier column is written `price_multiplier = case when v_wb_enabled then v_new_mult else price_multiplier end` (the 0135 `last_tick_at` self-assign idiom), and ALL normalized/premium math is inside `if v_wb_enabled` — so while dark the column is left untouched at 1.0 and no drift math runs. The pressure/danger-modifier/zone-rollup logic is byte-for-byte 0135. So a dark tick's writes are identical to pre-slice.
2. **Read helper:** `worldstate_current_price_multiplier` returns 1.0 while dark REGARDLESS of any stored value, so `trade_effective_price = round(base × 1.0) = round(base)` = the base integer price. Every composed price (display, charged, paid) equals the pre-slice price. (The base `market_offers` prices seed as integers, so `round(base)` is a no-op.)

New downward edge (acyclic). Trade Market → World State (read `worldstate_current_price_multiplier`). ACYCLIC: World State reads only its OWN `location_state` + `combat_reports` (0135) and never reads Trade Market → no cycle, no two-way dependency. NO new edge into `market_offers` (still static, no runtime writer, no second writer). World State stays the SOLE writer of `location_state`; Trade Market writes only `trade_receipts`.

Doc-sync (same step). `docs/SYSTEM_BOUNDARIES.md`: §1 `location_state` row (the new column is tick-sole-written, dark/no-op); §2 World State contract (the price multiplier + `worldstate_current_price_multiplier` helper; "must NOT add a runtime writer to `market_offers`"); §3 Trade Market row (base price is now COMPOSED via the one `trade_effective_price` helper, the "source of ALL offer prices" phrasing corrected to "BASE offer prices", the new downward read edge, still never writes `market_offers/location_state`); §4 a new "Trade Market → World State price-composition edge is acyclic" note. `docs/ROADMAP.md`: Phase-19 row carries no per-phase status marker (dark phases 11+), left UNTOUCHED (noted per the instruction).

Retirement / activation. `world_balance_enabled` is the permanent Phase-19 gate (same as 0135). Lit-path verification (flag on a DEV DB → drive the tick under danger → the multiplier breathes toward the bounded target → composed buy/sell prices track it → display == charged/paid) is deferred to the human's activation checklist. This slice flips NO flag.

Human gates preserved. `world_balance_enabled` stays 'false'; ALL Phase 11–18 flags remain 'false'; migrations 0001–0135 untouched (forward-only — 0136 is new); no new cron (reuses the 60s `process_location_state_ticks()` → `worldstate_tick()` path); backend-only (no `src/**`); no runtime `game_config` write; no lit-path/production DB run; no main touch; no merge/deploy/workflow dispatch. SAFE FOR HUMAN MERGE REVIEW.

Verify. Forward-only: 0136 is a new file; the only changes are 0136, `docs/SYSTEM_BOUNDARIES.md`, and this DEV_LOG entry — no shipped migration edited. Single-sourced composition: all three trade functions call `trade_effective_price` (grep-confirmed). The dark-identical property is established by the two-guard logic walk above. **The M2–M5 / trade verify suites could NOT be run locally:** they connect to a live Supabase (service-role key in `.env.local`), which the human gates forbid and where 0136 is not deployed — so NO green/red claim is made; the dark-safety argument rests on the logic walk

- forward-only proof (the 0132–0135 dark-slice precedent).

2026-07-04 — WORLD-BALANCE-P19 SLICE 1 — PIRATE PRESSURE (dark): wire the `defeat_pressure` seam in `worldstate_tick()` (0135)

Request. Phase 19 first mechanic: pirate pressure, dark-gated, by EXTENDING the existing World State tick — ONE forward-only migration 0135 + same-step doc-sync. No flag flip, no new cron, no `src/`, no git, no edit to any shipped migration.

Design (self-approved, grounded in the STEP-0 recon seam). Pirate pressure is NOT a new system or a new column — it is a living reaction on the EXISTING `location_state.pressure` field, delivered by re-creating the one World State writer `worldstate_tick()`. It finally wires the long-standing -- `defeat_pressure` TODO (M5+): add recent-defeat reads from `combat_reports` seam left in the tick since 0032.

Work done — 0135:

- `world_balance_enabled` — NEW dark master flag, seeded 'false' (on conflict do nothing), the Phase-19 gate. CONSUMED this slice by the tick (not a dead flag): the danger term is gated on it.
- `world_balance_defeat_window_seconds` — NEW tunable, seeded '3600' (a one-hour rolling danger memory), consumed this slice.
- **Reused, NOT re-seeded:** `worldstate_pressure_defeat_increase` (from 0032) scales the danger term.
- `worldstate_tick()` **re-created** = byte-for-byte the 0034 body EXCEPT the decay TARGET. Old: decay toward `baseline`. New: decay toward `baseline + danger_term`, where `danger_term = 0` unless `cfg_bool('world_balance_enabled')` is true, in which case `danger_term = count(combat_reports at this location with result='defeat' within the window) * worldstate_pressure_defeat_increase`.

Join key (verified from the real schema, not invented). `combat_reports` (0016) carries `location_id` uuid references `public.locations (id)` and `result` text (report_create copies the encounter status — 'defeat' on fleet loss, 0032 — into `result`). So a defeat attributes directly to its location: `combat_reports.location_id = location_state.location_id`, filtered `result='defeat'` and `created_at >= now() - window`. Only DEFEATS raise pressure; victories/escapes/completions do not.

Preserved-while-dark invariant (byte-identical output). With `world_balance_enabled='false': v_wb_enabled=false` → the danger-term read is skipped entirely (no `combat_reports` query) → `v_danger_term=0` → `v_target = v_baseline` → the decay expression becomes $(v_baseline - v_pressure) * v_decay_rate$ — the EXACT 0034 line — minus the identical fleet-relief term, under the IDENTICAL `least(v_max, greatest(v_min, ...))` cap. Reconcile/danger-modifier/zone-rollup are untouched from 0034. So a dark tick produces the same `pressure/danger_modifier/active_fleets` writes as today: self-correcting toward baseline, no accumulation, pressure never exceeds `worldstate_pressure_max`. When the flag is on, the term is a decay TARGET (not an accumulator), so it is self-correcting too — as defeats age out of the window the target falls back to baseline and pressure decays back down, always bounded by the same cap.

New downward read edge (acyclic). `worldstate_tick()` now READS `combat_reports` DOWNWARD (history, read-only) — a NEW edge World State → Report. ACYCLIC: Report writes only `combat_reports` and calls nothing (0016), so it cannot call back. World State still writes ONLY `location_state/zone_state` and never fleets/combat/rewards. This mirrors Combat's pre-existing downward READ of `location_state.danger_modifier` (0032), just the other direction into finalized history — not a read of active state (Report stays never-a-source-of-truth-for-active-state).

Doc-sync (same step). `docs/SYSTEM_BOUNDARIES.md`: §2 World State contract row updated (target = baseline + gated danger term; the downward `combat_reports` read; dark/no-op posture; "must NOT write `combat_reports`"); §3 `process_location_state_ticks()` cron note + a new World State forbidden-edges line recording the acyclic downward read. `docs/ROADMAP.md`: the Phase-19 row carries no per-phase status column (it is a plain scope cell), so it is left UNTOUCHED this slice (noted here per the instruction).

Retirement / activation. `world_balance_enabled` is a permanent capability gate (the Phase-19 master flag), not a transitional shim — it "retires" only when the human owner activates Phase 19. Lit-path verification (flag on a DEV DB → seed a 'defeat' report at a location → run `worldstate_tick()` → pressure rises toward baseline + term, bounded by the cap → age the report out of the window → pressure decays back to baseline) is deferred to the human's activation checklist. This slice flips NO flag.

Human gates preserved. `world_balance_enabled` stays 'false'; ALL Phase 11–18 flags remain 'false'; migrations 0001–0134 untouched (forward-only — 0135 is new); no new cron (the existing 60s `process_location_state_ticks()` → `worldstate_tick()` path is reused verbatim); backend-only (no `src/**`); no `game_config` runtime write; no lit-path/production DB run; no main touch; no merge/deploy/workflow dispatch — activation is the human owner's decision. SAFE FOR HUMAN MERGE REVIEW.

Verify. Migration is forward-only: 0135 is a new file; `git status` shows the only changes are the new 0135, `docs/SYSTEM_BOUNDARIES.md`, and this DEV_LOG entry — no shipped migration edited. The byte-identical-while-dark property is established by the logic walk above. **The M2–M4.5 / M5 verify suites could NOT be run locally:** they connect to a live Supabase (service-role key in `.env.local`), which the human gates forbid (no production/lit-path DB run) and where 0135 is not deployed anyway — so no green/red claim is made on them; the dark-safety argument rests on the logic walk + forward-only proof, exactly as the prior dark slices' local verification was doc-level only (0132–0134 precedent).

2026-07-04 — LOCATION-INVEST-P18 SLICE 3 (FINAL) — the dark-posture verifier `verify-location-investment.mjs` + `verify:location-investment`; Phase 18 CLOSED

Request. Phase 18 final slice: ONE new verify script (the `verify-ranking.mjs` analogue) + one `package.json` line + same-step doc-sync. NO migration change, NO flag write, NO lit-path DB run, NO frontend, NO new RPC.

Work done — `scripts/verify-location-investment.mjs` (mirrors `verify-ranking.mjs` point-for-point; ZERO inline harness copies — imports the shared `resolveEnv/createReporter/createUserFactory/Abort` from `scripts/lib/verify-harness.mjs` + `teardownVerifier` from `scripts/lib/verifier-teardown.mjs`, same `admin/anon/throwaway-user/cfgVal` scaffold, same `catch/.finally` teardown with NO flag entry passed — this verifier touches no flag). Proves migrations 0132–0134 ship exactly as built and fully dark, with `anon/authenticated` clients only. Five assertion groups:

- Dark rejection** — `invest_in_location(<uuid>, 1, <uuid>)`, `get_location_development(<uuid>)`, `get_location_investment_leaderboard(<uuid>, 10)`, and `get_my_location_investments()` all return `{ok:false, code:'feature_disabled'}` while `location_investment_enabled='false'`; VALID-shaped args are passed precisely so the identical dark answer proves the anti-probe gate fires BEFORE any validation (`ship_not_owned` / `not_docked` / `unknown_location` are NOT reached). CODE-keyed, matching the 0133/0134 read/write envelopes.
- Owner-read posture (NOT public — the Phase-18 divergence from Ranking's public tables)** — the authenticated own-set of `location_investments` reads back empty (0 rows) on a fresh DB (RLS `player_id = auth.uid()`), and `anon SELECT` is DENIED (no `anon` grant — 0132 grants to authenticated ONLY). **Deviation noted:** the instruction's Group-2 wording said "anon returns 0 rows", but the shipped 0132 grant excludes `anon`, so `anon` is DENIED — the stronger, truthful proof of owner-read (NOT public). The verifier asserts `anon-DENIED` + `authenticated-0-rows`.
- No client write path** — a direct authenticated-client insert into `location_investments` is denied (no insert policy / no write grant — 0132).
- Internal surface locked** — the private sole-writer `location_investment_invest` AND the internal helper `location_investment_current_window` are BOTH denied to the authenticated client and to `anon` (service-role-only — 0133/0134).
- Config presence** (READ-ONLY; `String()` storage-form-tolerant compares) — `location_investment_enabled='false'`, `location_investment_min_amount='1'`, `location_investment_season_seconds='604800'`, `location_investment_season_epoch_seconds='1767225600'`.

`package.json` — one line adjacent to `verify:ranking`: `"verify:location-investment": "node scripts/verify-location-investment.mjs"`.

Lit-path DEFERRED (the verify-ranking stance verbatim). The script NEVER writes `game_config` / NEVER flips `location_investment_enabled`; it exercises NO lit path. Lit-path verification — flag on → a docked ship → `invest_in_location` debits credits via `wallet_debit` and appends exactly one ledger row → a replay of the same `request_id` is a no-op (no double debit) → `get_location_development` reflects the new `all_time_total/season_total` → `get_location_investment_leaderboard` ranks the contributor within the current window → crossing into the next window resets `season_total` while `all_time_total` + the ledger persist → withdrawal/payout is impossible (one-way sink) — is deferred to the human owner's activation checklist (flip the flag on a DEV database and run the lit checks there, never here). Because 0132–0134 are not deployed, local verification is `node --check scripts/verify-location-investment.mjs` only.

Doc-sync (same step). docs/SYSTEM_BOUNDARIES.md INTENTIONALLY UNTOUCHED — a verifier script + a package.json line add no table, writer, function, or cross-system edge (the Phase-15/16/17 slice-verifier precedent). Only this DEV_LOG entry is the doc change.

Phase 18 (Location Investment) CLOSED — backend + verifier deliverables:

- 0132 — the dark flag `location_investment_enabled='false'` + the `location_investments` append-only, monotonic per-contribution ledger (owner-read, client-unwritable).
- 0133 — the `location_investment_invest` SOLE writer via the public `invest_in_location` wrapper: a docked-gated, ledger-row-as-receipt idempotent, strict ONE-WAY credit sink (wallet_debit down, no withdrawal/no payout) + the `location_investment_min_amount='1'` floor.
- 0134 — the ONE season-window helper `location_investment_current_window()` + the three dark read RPCs (`get_location_development`, `get_location_investment_leaderboard` public; `get_my_location_investments` own-history): persistent-vs-seasonal as TWO derived reads over the one ledger, window derived deterministically from config (no season table, no coupling to Ranking).
- `verify-location-investment.mjs` — the dark-posture verifier + `verify:location-investment`.

Human gates preserved. `location_investment_enabled` stays 'false'; ALL Phase 11–18 flags remain 'false'; migrations 0001–0134 untouched (forward-only); backend-only (no `src/features/**`); no `game_config` write; no lit-path DB run; no cron scheduled; no `main` touch; no merge / deploy / production apply / workflow dispatch — activation is the human owner's decision. SAFE FOR HUMAN MERGE REVIEW.

Verify. `node --check scripts/verify-location-investment.mjs` → parses OK. `git status --porcelain` shows the ONLY changes are the new script + `package.json` + this DEV_LOG entry; no migration, `SYSTEM_BOUNDARIES.md`, flag, or `main` touched.

2026-07-04 — LOCATION-INVEST-P18 SLICE 2 — the dark PUBLIC read surface (development vs seasonal score) + the ONE season-window helper

Request. The Phase-18 read surface: ONE new forward-only migration exposing the persistent state vs the seasonal score, plus the caller's own history, plus the ONE shared season-window helper (window math in exactly one place), with same-step doc-sync. Reuse the Ranking read surface (0131 — `stable` security definer, dark-gate FIRST, code-keyed envelopes, anon+authenticated PUBLIC leaderboards, limit-clamp, `row_number()`, the own-history idiom (0106/0110), and `cfg_num`. Still NO writer, NO frontend, NO cron, NO flag flipped true.

Self-approved locked design (this slice).

- **Config-derived WEEKLY window with a fixed epoch.** `location_investment_season_seconds = '604800'` (7-day weekly cadence) + `location_investment_season_epoch_seconds = '1767225600'` (2026-01-01T00:00:00Z anchor). Both are numeric unix-seconds so the window helper computes purely via the existing `cfg_num` — NO new `cfg_text` helper.
- **The single window helper.** `location_investment_current_window()` returns (`window_index`, `window_start`, `window_end`) via `k = floor((now - epoch)/period)`, `window_start = to_timestamp(epoch + k * period)`, `window_end = window_start + period`. It is THE ONE definition of "the current season window" — every windowed read calls it, so no season table exists and no window arithmetic is duplicated (does NOT re-create Ranking's `ranking_season_open` machinery).
- **Persistent-vs-seasonal = TWO reads over ONE ledger.** Persistent development (all-time SUM + distinct-contributor count per location) and seasonal score (windowed SUM) are both DERIVED at read time from `location_investments` — never a stored denormalized row (the 0131 law). SECURITY DEFINER aggregates across owners but exposes ONLY totals/ranked scores; individual rows stay behind the 0132 owner-read RLS and surface only via the own-history RPC.
- **Public leaderboards vs owner-read rows.** The location/leaderboard reads are PUBLIC (anon + authenticated — the 0131 public-aggregate posture; an aggregate leaks no raw rows). The own-history read is authenticated-only and query-scoped `player_id = auth.uid()`.

Work done — `supabase/migrations/20260618000134_location_invest_p18_read_surface.sql`:

- (a) seeded the two season-window tunables (on conflict do nothing; consumed this slice — no dead config).
- (b) `location_investment_current_window()` — `stable`, language `sql`, INTERNAL (client-revoked).
- (c1) `get_location_development(uuid)` — PUBLIC; dark gate → `unknown_location` → window → `{ok:true, location_id, all_time_total, contributor_count, season_total, window_index, window_start, window_end}`.
- (c2) `get_location_investment_leaderboard(uuid, int default 100)` — PUBLIC; dark gate → `unknown_location` → clamp [1,500] → window → ranked rows: `[{rank, player_id, season_score}]`.
- (c3) `get_my_location_investments()` — authenticated; dark gate → auth → own rows joined to `locations` for the name, newest first → rows: `[{investment_id, location_id, location_name, amount, invested_at}]`.
- (d) ACL (the 0131/0106 posture): the two location/leaderboard reads granted to anon + authenticated; own-history to authenticated only; the window helper revoked from all clients (internal).

Doc-sync (same step). docs/SYSTEM_BOUNDARIES.md §1 matrix UNCHANGED (read-only functions add no table/writer — stated in the migration header per the 0131/0106 precedent). §2 Location Investment row gained the three RPCs + the window helper, recorded the now-realized DOWNWARD **Map** read edge (reads `locations` for validation/identity) alongside the existing Reference/Config edge, and noted persistent-vs-seasonal as two derived reads over the one append-only ledger, public leaderboards vs owner-read rows, all dark-gated FIRST, still acyclic (nothing calls into Investment).

Human gates preserved. `location_investment_enabled` stays 'false' (dark — every read gate rejects today). No flag flipped true; migrations 0001-0133 untouched (forward-only, new file only); backend-only (no `src/features/**`); no cron scheduled; no main touch; no merge/deploy. SAFE FOR HUMAN MERGE REVIEW.

Verify. SQL (no node `--check`): inspected against the 0131 read idiom (stable security definer, dark-gate-first code envelopes, clamp, `row_number()`, anon+authenticated ACL) and the 0106 own-history idiom; `cfg_num` returns double precision, so the window arithmetic and `to_timestamp` compose without a cast beyond the explicit `extract(epoch ...)::double precision`. `git status --porcelain` shows the ONLY changes are the new migration + docs/SYSTEM_BOUNDARIES.md + docs/DEV_LOG.md; no shipped 0001-0133 migration edited, no flag flipped.

2026-07-04 — LOCATION-INVEST-P18 SLICE 1 — the sole-writer invest command (`invest_in_location` → `location_investment_invest`) + min-amount tunable

Request. The Phase-18 core write path: ONE new forward-only migration adding the SOLE-writer invest command (the only path that writes `location_investments`) + its one consumed tunable, with same-step doc-sync. Reuse the established idioms — the Trade Market `market_buy` command (0089) + the shared docked-location resolver `mainship_resolve_docked_location` (0092) for ownership + dock resolution, the Wallet sink `wallet_debit/wallet_ensure` (0093) for the one-way debit, and the two-layer wrapper→private + advisory-lock-before-replay + verbatim-replay idempotency of `craft_module/production_craft_module` (0109). Still NO read surface, NO frontend, NO cron, NO flag flipped true. Also fold in the STEP-1 reviewer nit: move the §1 `location_investments` matrix row to sit after `ranking_standings` (restore Ranking's contiguous two-row group).

Self-approved locked design (this slice).

- **DOCKED-LOCATION-GATED.** An investment targets the ship's CURRENTLY DOCKED location, resolved server-side via `mainship_resolve_docked_location` (never a client-supplied location) — not docked → `not_docked`. The resolved id is a real `locations(id)` (from the present/location fleet), so the ledger FK is satisfied by construction.
- **LEDGER-ROW-AS-RECEIPT idempotency.** No separate receipts table — the `location_investments` row IS the receipt. A per-player advisory lock ('`location_investment`', `player`) is taken BEFORE the replay check (the 0109/0078 idiom); a replayed (`player`, `request_id`) returns the original row's envelope verbatim (0089/0109 trade-receipt semantics, no payload-conflict check). A raced unique (`player_id`, `request_id`) trip is caught in a sub-block: the savepoint rolls back that call's debit (NO double-charge) and replays the now-existing row.
- **ONE-WAY SINK.** `wallet_debit(player, amount)` DOWNWARD (false → `insufficient_credits`, no row written); on success exactly ONE append. There is NO `wallet_credit` / NO withdrawal / NO payout anywhere in Investment — score/development can never be farmed (ROADMAP :93 guard "no infinite exploit").
- **MIN-AMOUNT FLOOR.** `location_investment_min_amount = '1'` (anti-dust/spam), consumed THIS slice (no dead config); `amount <= 0` or `< floor` → `invalid_amount`, nothing spent.
- **REQUEST_ID TYPE BRIDGE (deviation, reported).** Per the directive, the command's `p_request_id` is `uuid` (same type as `market_buy`'s, 0089:66 — intrinsically bounded, null-only check). The shipped ledger column `location_investments.request_id` is `text` (0132, the `module_craft_receipts` idiom, which pairs with a text-param command). 0132 is forward-only / not editable, so the command bridges at the single ledger boundary with an explicit `p_request_id::text` cast (`uuid`→`text` is canonical + deterministic, so the idempotency key is preserved). Documented in-line.
- **ENVELOPES** are code-keyed (`{ok:false, code:'...'}`) — the 0131 posture; no localized message layer this slice (presentation belongs to the read/UI slice). The private writer returns well-formed code envelopes; the wrapper passes them through verbatim.

Work done — `supabase/migrations/20260618000133_location_invest_p18_invest_command.sql`:

- **(a)** seed `location_investment_min_amount = '1'` (on conflict (key) do nothing). No season-window tunables (those belong to the read slice).
- **(b1)** private `location_investment_invest(uuid, uuid, numeric, uuid)` — SECURITY DEFINER, the SOLE writer: dark gate FIRST → `request_id` null-check → per-player advisory lock → verbatim replay → `mainship_resolve_docked_location` → amount/min-floor validation → `wallet_debit` → ONE ledger row → success envelope; unique-violation sub-block backstop replays without double-debit.
- **(b2)** public `invest_in_location(uuid, numeric, uuid)` — authenticated wrapper: auth → dark gate (anti-probe) → `mainship_resolve_owned_ship` (`ship_not_owned`) → delegate → pass through.

- **(c) ACL** per the 0109 targeted idiom: private revoked from public/anon/authenticated + granted to service_role; public revoked from public/anon + granted to authenticated. No blanket relock (the 0064 default-privileges revoke already denies new functions; recent migrations use per-function ACL).

Doc-sync (same step). docs/SYSTEM_BOUNDARIES.md §1 — the `location_investments` sole writer is now the concrete `location_investment_invest` (via `invest_in_location`), and the row was MOVED to sit after `ranking_standings` (the STEP-1 reviewer nit; Ranking's two rows are contiguous again). §2 Location Investment — recorded both functions and replaced "no cross-system edge yet" with the now-REALIZED DOWNWARD edges (Wallet `wallet_debit` · Main Ship `mainship_resolve_owned_ship` + `mainship_resolve_docked_location` · Reference/Config `cfg_bool` + `cfg_num`); nothing calls into Investment → acyclic. Added the "mutate `location_investments` outside `location_investment_invest`" Must-NOT (the sole-writer law).

Human gates preserved. `location_investment_enabled` stays 'false' (dark — both the wrapper gate and the writer's first check reject every call today). No flag flipped true; migrations 0001-0132 untouched (forward-only, new file only); backend-only (no `src/features/**`); no cron scheduled; no `main` touch; no merge/deploy. SAFE FOR HUMAN MERGE REVIEW.

Verify. SQL (no node --check): inspected against the 0089 (`market_buy` ownership + `docked` + `wallet_debit` + code envelopes) and 0109 (two-layer wrapper, advisory-lock-before-replay, verbatim-replay, per-function ACL) idioms — signatures verified against source (`mainship_resolve_owned_ship(uuid,uuid)` 0081, `mainship_resolve_docked_location(uuid)` 0092, `wallet_debit(uuid,numeric)` 0093, `market_buy request_id uuid` 0089). `git status --porcelain` shows the ONLY changes are the new migration + `docs/SYSTEM_BOUNDARIES.md` + `docs/DEV_LOG.md`; no shipped 0001-0132 migration edited, no flag flipped.

2026-07-04 — LOCATION-INVEST-P18 SLICE 0 — dark flag + the `location_investments` root table (foundations only)

Request. Begin Phase 18 (Location investment — ROADMAP :93, "seasonal investment score vs persistent state", guard "no infinite exploit"), mirroring the proven RANKING-P17 slice-0 shape (migration 0127) exactly. ONE new forward-only migration establishing the dark capability gate + the Location-Investment-owned persistent root table, with same-step doc-sync. NO writer function, NO invest command, NO read RPC, NO aggregate read surface, NO frontend, NO cron, NO tunables beyond the flag, NOTHING client-writable, NO flag flipped true.

Self-approved locked design (owner-directed, recorded here so later slices are grounded). Location Investment is a NEW, DARK, downward **LEAF** owning exactly ONE persistent table — the append-only, monotonic per-contribution ledger `location_investments`.

1. **Persistent state** (a location's "development") = the all-time SUM of contributions per `location_id`, DERIVED from the ledger — never a denormalized column (the 0131 "derived, never stored" stance).
2. **Seasonal score** = a player's contributions SUMMED within the CURRENT season WINDOW, the window derived DETERMINISTICALLY from config (a period length + epoch tunable) in the consuming slice — NO season table, NO season-open writer, so it does **not** duplicate Ranking's `ranking_season_open` machinery (the no-duplication hard rule) and adds **no** season coupling to Ranking. Reset-by-season (ROADMAP law) is honored STRUCTURALLY: a new window resets the windowed SCORE read while the ledger (persistent state) is never touched. Those tunables are NOT seeded this slice (no dead config) — they land in the slice that consumes them.
3. **No infinite exploit** (the ROADMAP guard) = investment is a strict ONE-WAY SINK: `amount` CHECK (>0); the future invest command debits credits via `wallet_debit` DOWNWARD then appends a row; NO withdrawal path, NO payout returning value → score/development can never be farmed in a loop.
4. Edges (all DOWNWARD, acyclic, realized in later slices): Investment → Wallet (`wallet_debit` sink) · Map (read `locations`) · Reference/Config (`cfg_bool/cfg_num`). Nothing calls into Investment.

Work done — `supabase/migrations/20260618000132_location_invest_p18_flag_and_ledger.sql` (mirrors 0127 structure + header discipline):

- **(a) dark gate** — `insert into game_config ('location_investment_enabled', 'false', ...)` on conflict (key) do nothing (the exact 0097/0102/0107/0117/0124/0127 slice-0 flag idiom); the description records the server-authoritative reject-before-any-read posture every future Investment RPC must adopt and that the UI stays hidden independently (fails closed both sides). No other config value seeded this slice.
- **(b) `location_investments`** — the append-only per-contribution ledger. Receipt idiom matched POINT-FOR-POINT to `module_craft_receipts` (0109): `player_id` uuid not null references `auth.users(id)` on delete cascade, `request_id` text not null, unique (`player_id`, `request_id`) (player-scoped, non-spatial — NOT the ship-scoped trade keying). Plus `investment_id` uuid pk default `gen_random_uuid()`, `location_id` uuid not null references `locations(id)` (Map target; `locations.id` confirmed PK), `amount` numeric not null check (`amount > 0`), `invested_at/created_at` timestampz default `now()`. Two supporting indexes: (`location_id`, `invested_at`) (sum-by-location = persistent development) and (`player_id`, `invested_at`) (sum-by-player-within-window = seasonal score). RLS enabled; ONE owner-read select policy (`player_id = auth.uid()`) + grant select to authenticated ONLY — NO insert/update/delete policy, NO write grant → clients cannot

mutate; sole writer is Investment's own future SECURITY DEFINER command. comment on table captures append-only/monotonic, the persistent-vs-seasonal split, the one-way-sink rationale, and the dark gate. No function created → no execute-surface relock needed (0054/0127 precedent). The table is inert — no RPC/reader/writer references it yet.

Doc-sync (same step). docs/SYSTEM_BOUNDARIES.md §1 matrix gains the location_investments row under the NEW Location Investment system (sole writer = future invest command, owner read, DARK, append-only, persistent-vs-seasonal derivation), and §2 gains the Location Investment system row (a NEW DARK downward LEAF: owns location_investments; edges all DOWNWARD/acyclic — Wallet · Map · Reference/Config; Must-NOT includes withdraw/pay out, write another system's table, store a denormalized aggregate, delete/truncate to reset, duplicate Ranking's season machinery, or invest while the gate is off; explicitly notes this slice adds NO cross-system edge yet).

Human gates preserved. location_investment_enabled stays 'false' (dark). No flag flipped true; migrations 0001-0131 untouched (forward-only, new file only); backend-only (no src/features/**); no cron scheduled; no main touch; no merge/deploy. SAFE FOR HUMAN MERGE REVIEW.

Verify. SQL (no node --check): inspected against the 0127 idiom — flag insert, table shape, RLS posture, grants, and comment on table all match the slice-0 precedent; receipt columns match module_craft_receipts (0109) exactly. git status --porcelain shows the ONLY changes are the new migration + docs/SYSTEM_BOUNDARIES.md + docs/DEV_LOG.md; no shipped 0001-0131 migration edited, no flag flipped.

2026-07-04 — RANKING-P17 CLEANUP — correct two stale figures in the verify-harness.mjs header (docs-in-code only)

Request. Final Phase-17 Ranking auto-cleanup pass. The STEP 1 read-only audit (RANKING_CLEANUP_RECON.local.md) found the milestone clean on every claim EXCEPT one narrow docs-in-code defect: the shared harness's in-file ADOPTION / RETIREMENT PLAN header carried two stale figures. This slice corrects ONLY those figures — no logic change, no migration, no flag change, no frontend; ranking_enabled stays 'false'.

Defect (audit Claim 4). scripts/lib/verify-harness.mjs header said (a) "the 31 sibling verify-*.mjs scripts still carry inline copies" and (b) cited the osn_distance adopt-on-next-real-change precedent as docs/SYSTEM_BOUNDARIES.md:75-78. Both were wrong: there are only 27 scripts/verify-*.mjs files, of which 7 already import the harness (captain, captain-progression, exploration, fitting, mining, modules, ranking) → 20 remaining (not 31); and lines 75-78 are the Fleet/Movement/Presence/Activity matrix rows — the actual "OSN geometry leaf" note is at docs/SYSTEM_BOUNDARIES.md:101-104 (the "should adopt the helper whenever they are next re-defined" sentence is :103-104).

Work done — scripts/lib/verify-harness.mjs (header comment ONLY): rewrote the ADOPTION / RETIREMENT PLAN paragraph to state the self-checking accounting — 27 total / 7 adopters (named) / 20 remaining — and corrected the precedent citation to docs/SYSTEM_BOUNDARIES.md:101-104. Added an explicit retirement condition (plan discharged when all 27 import the harness). NO exported function, no loadEnv/resolveEnv/createReporter/createUserFactory logic, and no other line changed — the harness code and its behavior are untouched. Re-verified the 27/7/20 split by counting scripts/verify-*.mjs and grep -l "lib/verify-harness" this step.

Verify. node --check scripts/lib/verify-harness.mjs → parses OK. node --check scripts/verify-ranking.mjs → parses OK (unchanged, sanity re-check). No migration, package.json, verify-ranking.mjs, flag, or main touched; ranking_enabled remains 'false' (dark). SAFE FOR HUMAN MERGE REVIEW.

2026-07-04 — RANKING-P17 SLICE 5 — the dark-posture verifier scripts/verify-ranking.mjs + verify:ranking (FINAL Phase-17 slice)

Request. Phase 17 Slice 5 (final): ONE new verify script (the verify-captain-progression.mjs analogue) + one package.json line + same-step doc-sync. NO migration change, NO flag write, NO lit-path testing, NO frontend, NO new read RPC.

Work done — scripts/verify-ranking.mjs (mirrors verify-captain-progression.mjs point-for-point; ZERO inline harness copies — imports the shared resolveEnv/createReporter/createUserFactory/ Abort from scripts/lib/verify-harness.mjs + teardownVerifier from scripts/lib/ verifier-teardown.mjs, same admin/anon/throwaway-user/cfgVal scaffold, same . catch/.finally teardown with NO flag entry passed — this verifier touches no flag). Proves migrations 0127-0131 ship exactly as claimed and fully dark, with anon/authenticated clients only. Five assertion groups:

1. **Dark rejection** — get_ranking_seasons() and get_ranking_leaderboard(<valid uuid>,'combat',10) both return {ok:false, code:'feature_disabled'} while ranking_enabled='false'; a valid uuid + real dimension are passed precisely so the identical dark answer proves the anti-probe gate fires BEFORE any validation (unknown_season / invalid_dimension are NOT reached). CODE-keyed, matching the 0131 read surface.

2. **Public-read posture** — anon can SELECT `ranking_seasons` and `ranking_standings` (permitted, 0 rows on a fresh DB — reading the public tables back IS the assertion, the catalog-table precedent).
3. **No client write path** — direct authenticated-client inserts into `ranking_seasons` AND `ranking_standings` are denied (no insert policy / no write grant — 0127/0128).
4. **Internal surface locked** — `ranking_season_open`, `ranking_accrue_standings`, and `ranking_score_delta` are denied to the authenticated client, and `ranking_accrue_standings` is denied to anon (service-role-only — 0129/0130).
5. **Config presence** — `ranking_enabled` reads 'false' (READ-ONLY; storage-form-tolerant `String(v)=='false'` compare).

No-flag-write / no-lit-path stance (verbatim to `verify-captain-progression.mjs`): the script NEVER writes `game_config` and NEVER flips `ranking_enabled`. Lit-path verification — flag on → `ranking_season_open` opens an active season → deposit finalized `reward_grants` → `ranking_accrue_standings` folds them once → a re-run is a no-op → `get_ranking_leaderboard` ranks them (overall = sum of per-dimension scores) → opening a new season closes the prior active one while PRESERVING the closed season's standings rows — is DEFERRED to the human owner's activation checklist (flip the flag on a DEV database and run the lit checks there, never here). Because 0127-0131 are not deployed, local verification is `node --check scripts/verify-ranking.mjs` only (**parses OK**); the script is NOT executed against a live DB this slice (its execution belongs to the owner's post-apply checklist, exactly as the prior dark verifiers).

`package.json`. Added one line adjacent to `verify:captain-progression`: `"verify:ranking": "node scripts/verify-ranking.mjs"`.

Doc-sync (same step).

- `docs/SYSTEM_BOUNDARIES.md` — **intentionally UNTOUCHED**. A verifier script + a `package.json` line add NO table, writer, function, or cross-system edge (the Phase-15/16 slice-verifier precedent — the law doc describes architectural facts, and nothing architectural changed). Stated here explicitly rather than editing it.
- `docs/DEV_LOG.md`: this entry.

Phase 17 status — CLOSED (backend + verifier deliverables). The Ranking milestone is complete and PR-ready on the feature branch, fully dark and server-rejected:

- 0127 — `ranking_enabled='false'` dark flag + `ranking_seasons` root table.
- 0128 — `ranking_standings` per-(season, player, dimension) schema (dimension = the `reward_grants`. `source_type` domain 1:1; overall derived at read time).
- 0129 — `ranking_season_open` (sole writer of `ranking_seasons`; natural-key idempotent; close-prior-active = reset by season, not deletion).
- 0130 — `ranking_score_delta` + `ranking_accrue_standings` (sole writer of `ranking_standings`; incremental high-water fold reading `reward_grants` DOWNWARD — the acyclic Ranking→Reward edge).
- 0131 — `get_ranking_seasons` + `get_ranking_leaderboard` (public, dark-gated read surface).
- `verify-ranking.mjs` — the dark-posture verifier.

Human gates preserved. `ranking_enabled` stays 'false' (no flag flipped); every Phase 11–17 flag remains 'false'. NO migration changed (0127-0131 and all of 0001-0126 untouched). NO `game_config` write. NO lit-path DB run (deferred to the owner's activation checklist). Backend-only (no `src/features/**`). No merge/deploy/production apply/workflow dispatch. Activation (flipping `ranking_enabled` true + scheduling the accrual cron) is the human owner's decision, not this loop's.

2026-07-04 — RANKING-P17 SLICE 4 — the dark read surface `get_ranking_seasons` + `get_ranking_leaderboard` (migration 0131)

Request. Phase 17 Slice 4: ONE new forward-only migration (the two public leaderboard/season read RPCs) + same-step doc-sync. Still NO frontend, NO flag flipped, NO cron, NO new table/writer.

Work done — migration `20260618000131_ranking_p17_read_surface.sql` (**forward-only; 0001-0130 unedited**). Two READ-ONLY RPCs mirroring the 0123/0116 read-surface idiom (`jsonb` envelope · `stable` · `security` definer · dark-gate FIRST · SELECT only Ranking's own public tables · no write anywhere):

- `get_ranking_seasons()` → `jsonb` — dark-gate FIRST (`cfg_bool('ranking_enabled')` false → `{ok:false, code:'feature_disabled'}`). When enabled: `{ok:true, seasons:[...]}` selecting `season_id`, `cadence`, `label`, `starts_at`, `ends_at`, `status` from `ranking_seasons`, ordered `cadence`, `starts_at` desc (active naturally sorts to the top within a cadence).
- `get_ranking_leaderboard(p_season_id uuid, p_dimension text, p_limit int default 100)` → `jsonb` — dark-gate FIRST (same disabled envelope). When enabled: validate `p_dimension` in ('combat', 'trade', 'exploration', 'mining', 'overall') else `invalid_dimension`; validate the season exists else `unknown_season`; clamp `p_limit` to [1, 500] (default 100). For a concrete dimension: the `ranking_standings` rows for (season_id, dimension), ranked score desc, `player_id` with a 1-based `row_number()` rank, limited. For 'overall': **derived at read time** — `sum(score)`, `sum(events_counted)` grouped by `player_id` across the season, same ranking/limit — NEVER a stored row (the slice-1 Must-NOT). Returns `{ok:true, season_id, dimension, rows:[{rank, player_id, score, events_counted}, ...]}`.

- **ACL** (the 0123 idiom, adjusted for PUBLIC leaderboards): revoke `execute ...` from `public`, `anon`; grant `execute ...` to `anon`, authenticated. Granted to **anon + authenticated** (not authenticated-only like the captain own-roster surface) because leaderboards/season info are PUBLIC — the 0127/0128 `ranking_seasons/ranking_standings` public-read posture. No `auth.uid()` check / no per-player scoping — the exposed data is already public.

Deliberate divergences from the captain read surface (grounded).

- **Public, not own-data**: grant `anon + authenticated`; no auth check.
- **Envelope key** code (not `reason`) — consistent with the Ranking writers (0129/0130).
- **'overall' derived at read time** — sums per-dimension scores on the fly, no stored denormalized row (upholds the slice-1 standings Must-NOT).

Boundary placement (same-step doc-sync).

- **\$1 matrix UNCHANGED** — read-only functions add NO new table and NO new writer, so the sole-writer matrix is untouched (the 0101/0106/0110/0116/0123 precedent: read surfaces are recorded in the \$2 system row, not the matrix). Stated here explicitly rather than editing \$1.
- `docs/SYSTEM_BOUNDARIES.md` \$2 **Ranking** row: recorded `get_ranking_seasons + get_ranking_leaderboard` (READ-ONLY, dark-gated, PUBLIC `anon+authenticated`, overall derived at read time, no writer added); confirmed NO new cross-system edge (they read only Ranking's own tables + `cfg_bool`) — call graph still ACYCLIC, nothing calls into Ranking.
- `docs/SYSTEM_BOUNDARIES.md` \$2 "Ranking read-edge is acyclic" note: extended the `cfg_bool` edge list to include the two read RPCs; noted they SELECT only Ranking's own public tables and write nothing (overall derived, never stored).
- `docs/DEV_LOG.md`: this entry.

Human gates preserved. `ranking_enabled` stays `'false'` (no flag flipped; both RPCs' dark gate rejects every call today with the identical anti-probe envelope); every Phase 11–17 flag remains `'false'`. No existing migration edited (0001–0130 untouched, forward-only). No `game_config` value changed. Backend-only (no `src/features/**`). No cron. No merge/deploy/production apply/workflow dispatch. Surface is inert while dark: the RPCs exist and are grant-scoped, but server-reject every call until the human activates.

2026-07-04 — RANKING-P17 SLICE 3 — the core standings-scoring accrual `ranking_accrue_standings` (the **SOLE** writer of `ranking_standings`, reading `reward_grants` **DOWNWARD**; migration 0130)

Request. Phase 17 Slice 3: ONE new forward-only migration (the standings accrual writer) + same-step doc-sync. Still NO read RPC, NO frontend, NO flag flipped, NO cron scheduled.

Metric lock (read first, no guessing). Re-confirmed the EXACT `reward_grants.rewards` bundle shape from 0040/0015: { `"metal": <number>`, `"items": [ { "item_id": "...", "quantity": <int> }, ... ]` } — the item quantity key is `quantity` (0040:64 (`e1->'quantity'`):`numeric`). The per-event score value is defined in ONE place, `ranking_score_delta(p_rewards jsonb)` returns `numeric` (IMMUTABLE, single-source, testable): `coalesce(metal,0) + coalesce(sum of item quantities,0)`. Rationale: standings are PER-DIMENSION separate leaderboards (slice 1), so absolute scale is irrelevant within a board; a reward-magnitude metric is uniform across dimensions, deterministic, and computed purely from the finalized event. The items sum is guarded to a real `jsonb` array (the 0040 "fail safely" ethos — a malformed row can never abort the batch; metal-only combat bundles simply have no `items` key → 0).

Work done — migration 2026061800130_ranking_p17_accrue_standings.sql (forward-only; 0001–0129 unedited).

- `ranking_score_delta(jsonb) → numeric` — IMMUTABLE, service-role-only (client-revoked). The ONE place the per-event score is defined.
- `ranking_accrue_standings() → jsonb` — PRIVATE, SECURITY DEFINER, service-role-only, THE sole writer of `ranking_standings`. Batch accrual over ALL active seasons (cron/admin-style, no player input):
 1. **DARK GATE FIRST** — `if not cfg_bool('ranking_enabled') → {ok:false, code:'feature_disabled'}` before any read/write (folds nothing while dark).
 2. **Advisory lock** `pg_advisory_xact_lock(hashtext('ranking_accrue_standings'), 0)` — concurrent accruals serialize.
 3. **Incremental, idempotent fold** — one statement (with folded `...`, upserted as `(insert ... on conflict ... do update) ...`). Source: `reward_grants rg` joined to each `ranking_seasons s` where `s.status='active'` ON `rg.granted_at` between `s.starts_at` and `s.ends_at`, LEFT JOIN the existing `ranking_standings st` ON `(season_id, player_id, source_type)`. **High-water filter:** fold only `(st.last_counted_at is null and rg.granted_at >= s.starts_at)` or `(rg.granted_at > st.last_counted_at)` — NULL high-water counts from season start inclusive; strict `>` afterward makes re-runs a no-op. group by `(season_id, player_id, source_type) → score = sum(ranking_score_delta(rewards))`, `events_counted = count(*)`, `last_counted_at = max(granted_at)`. on conflict `... do update set score = score + excluded.score, events_counted = events_counted + excluded.events_counted, last_counted_at = greatest(...), updated_at = now()`. `dimension = rg.source_type` directly (the slice-1 1:1 domain lock — no translation); the fold is scoped `rg.source_type` in

('combat','trade','exploration','mining') so an out-of-domain source (none exist today) is skipped rather than aborting the batch / tripping the dimension CHECK.

4. Returns {ok:true, seasons_scored, rows_upserted, events_folded} (a summary for the future cron/verifier).

 ACL: revoke execute ... from public, anon, authenticated; grant ... to service_role (0129 private-writer block). **No public wrapper, no cron scheduled** — accrual is a server/cron/admin op; scheduling is deferred, and the dark-gated fn is a safe no-op until the human activates it.

Reset-by-season semantics. The fold's window is bounded by each ACTIVE season's [starts_at, ends_at]; a CLOSED season (closed by ranking_season_open, 0129) is no longer joined so it stops accruing, but its standings rows remain intact. A "reset" is a new active season scoping a fresh standings set — NEVER a delete of any standings or reward_grants event data.

Boundary placement (same-step doc-sync).

- docs/SYSTEM_BOUNDARIES.md §1: updated the ranking_standings row — sole writer is now the CONCRETE ranking_accrue_standings (0130) (was "future scoring fn"); service-role-only + DARK + incremental-by-last_counted_at; the ranking_score_delta metric; no-cron-yet.
- docs/SYSTEM_BOUNDARIES.md §2 **Ranking** row: recorded ranking_accrue_standings + the ranking_score_delta helper as the standings writer, and added the concrete **Ranking → Reward** (reward_grants read) DOWNWARD edge — the FIRST realized cross-system read.
- docs/SYSTEM_BOUNDARIES.md §2 notes: added a "**Ranking read-edge is acyclic**" note (the "Trade fan-out is acyclic" precedent — the home for dark-phase cross-system edge facts, since §3 is the fixed MVP-5-entry-points snapshot the activity securing processors also don't touch): confirms Ranking → Reward + Ranking → Reference/Config are the only edges, both DOWNWARD reads; Reward never reads Ranking; nothing calls into Ranking → **ACYCLIC**, one sole-writer per Ranking table.
- docs/DEV_LOG.md: this entry.

Human gates preserved. ranking_enabled stays 'false' (no flag flipped; the writer's dark gate folds nothing today); every Phase 11–17 flag remains 'false'. No existing migration edited (0001–0129 untouched, forward-only). No game_config value changed. Backend-only (no src/features/**). No player wrapper / no client execute grant. No cron scheduled. No merge/deploy/production apply/workflow dispatch. Surface is inert: dark-gated + service-role-only, no caller/schedule exists.

2026-07-04 — RANKING-P17 SLICE 2 — the season-management writer ranking_season_open (the SOLE writer of ranking_seasons; migration 0129)

Request. Phase 17 Slice 2: ONE new forward-only migration (the season-lifecycle writer) + same-step doc-sync. Still NO standings scoring, NO read RPC, NO frontend, NO flag flipped.

Work done — migration 20260618000129_ranking_p17_season_open.sql (forward-only; 0001–0128 unedited).

- Idempotency natural key.** New index ranking_seasons_cadence_start_uidx = unique (cadence, starts_at) on ranking_seasons (a NEW index in a NEW migration — 0127 is never edited). A season window is uniquely identified by its (cadence, starts_at), so season-open is idempotent WITHOUT a receipts table (the 0126 receipt ledger was per-(player, request_id); a lifecycle op is keyed by its window, not a client request).
- ranking_season_open(p_cadence text, p_starts_at timestampz, p_ends_at timestampz, p_label text) → jsonb — PRIVATE, SECURITY DEFINER, service-role-only, THE sole writer of ranking_seasons. Body mirrors the 0126 production_recruit_captain writer idioms:
 - DARK GATE FIRST** — if not cfg_bool('ranking_enabled') → {ok:false, code:'feature_disabled'} before any read/write (anti-probe; identical answer while dark; cfg_bool (0046) coalesces a missing key to false).
 - Validation** (no reads): p_cadence in ('weekly','monthly') else invalid_cadence; p_ends_at > p_starts_at (both non-null) else invalid_window; non-empty trimmed label with a sanity length cap (the 0126:121 text-bound hygiene) else invalid_label. Codes returned directly — no reason→code translation layer (there is no client wrapper).
 - Advisory lock** pg_advisory_xact_lock(hashtext('ranking_season_open'), hashtext(p_cadence)) — concurrent opens of the SAME cadence serialize, so the replay check and the close→insert window cannot be raced by another open of this cadence.
 - Idempotent replay** from the natural key: if a season exists for (cadence, starts_at), return it VERBATIM ({ok:true, idempotent:true, season_id, cadence, label, starts_at, ends_at, status, created_at}) — NO second insert, NO status churn (a re-open of an already-closed window does NOT reactivate it).
 - Open new active window** — in the same tx update ... set status='closed' where cadence=... and status='active' (closing the prior active season — reset by season, NOT deletion: its standings rows remain under the closed season_id, then insert ... status='active'. The close-prior step makes room for the partial unique active index (0127); a raced unique_violation on either index is caught into a clean {ok:false, code:'conflict'} rather than a raw exception.

 ACL: revoke execute ... from public, anon, authenticated; grant ... to service_role (the 0126:273–274 private-writer block). **No public wrapper** — season management is a server/cron/admin operation, not a player command, so it stays service-role-only and dark.

Design rationale (locked; grounds later slices).

- **Sole writer, concrete.** This is the function the 0127 §1/§2 "future season fn" note promised — no second write path to `ranking_seasons`, ever.
- **Reset by season, never by deletion** (ROADMAP :92). Opening a new active window CLOSES the prior active one; nothing is deleted, and standings accrued under the closed `season_id` remain intact (a closed season is queryable history; a "reset" is the NEW active season scoping a fresh standings set). No DELETE of any season or event data anywhere.
- **One active per cadence** — the close-prior step + the partial unique active index (0127) + the per-cadence advisory lock together guarantee exactly one active window per cadence.

Boundary placement (same-step doc-sync).

- docs/SYSTEM_BOUNDARIES.md §1: updated the `ranking_seasons` row — sole writer is now the CONCRETE `ranking_season_open` (0129) (was "future season fn"); service-role-only + DARK; recorded the natural-key idempotency, the close-prior-active semantics, and the conflict guard.
- docs/SYSTEM_BOUNDARIES.md §2 **Ranking** row: recorded `ranking_season_open` as the season-lifecycle writer; added the ONE cross-system edge (Ranking → Reference/Config `cfg_bool` read — a DOWNWARD leaf read, acyclic, nothing calls into Ranking); the standings-scoring writer stays a later slice; Must-NOT now also forbids deleting closed-season standings.
- docs/DEV_LOG.md: this entry.

Human gates preserved. `ranking_enabled` stays 'false' (no flag flipped; the writer's dark gate rejects every call today); every Phase 11–17 flag remains 'false'. No existing migration edited (0001–0128 untouched, forward-only). No `game_config` value changed. Backend-only (no `src/features/**`). No player wrapper / no client execute grant. No merge/deploy/production apply/workflow dispatch. Surface is inert: dark-gated + service-role-only, no caller exists.

2026-07-04 — RANKING-P17 SLICE 1 — the standings (leaderboard) schema `ranking_standings` (migration 0128)

Request. Phase 17 Slice 1: ONE new forward-only migration (the per-player leaderboard score schema) + same-step doc-sync. Still NO scoring function, NO season-management writer, NO read RPC, NO frontend, NO flag flipped.

Domain lock (read first, no translation layer). Re-confirmed the EXACT literal set the standings dimension maps to: the `reward_grants.source_type` domain is the closed activity-source set ('combat', 'exploration', 'mining', 'trade') established by the 0096 `fleet_movements_reward_source_type_domain` CHECK (the carrier feeding `reward_grant(source_type,...)`). `reward_grants.source_type` itself is text not null (0015, no CHECK), constrained upstream by that carrier + the direct depositor calls. Live depositors today: combat/exploration/mining call `reward_grant` directly; trade **is in the domain but has NO depositor yet** — Trade V1 banks via the Wallet path, not `reward_grants` — so its standings stay 0 until a trade activity deposits a grant. `dimension` uses these four literals verbatim (1:1, no lookup/mapping).

Work done — migration 20260618000128_ranking_p17_standings_schema.sql (forward-only; 0001–0127 unedited).

- `ranking_standings` — the per-(season, player, dimension) score row:


```
<font name="Sym"> </font> season_id uuid not null references ranking_seasons(season_id) on delete cascade;
  player_id uuid not null references auth.users(id) on delete cascade; dimension text not null check (dimension in
  ('combat','trade','exploration','mining')); score numeric not null default 0; events_counted integer not null
  default 0; last_counted_at timestamptz (nullable); updated_at timestamptz not null default now();
```
- **PK** (season_id, player_id, dimension).
- **Leaderboard index** `ranking_standings_leaderboard_idx` (season_id, dimension, score desc) — the future read RPC's access path (a season's dimension board, best first).
- **RLS ON + ONE public-read select policy + grant select to anon, authenticated; NO insert/update/delete policy and NO write grant** — clients cannot mutate.
- Table + `last_counted_at` comments record the sole-writer, derived-overall, and high-water-mark facts below.

Self-approved locked design decisions (owner-directed; grounds later slices).

1. **Dimension = the read source, 1:1, no translation.** `dimension` is exactly the `reward_grants.source_type` domain (the 0096 closed set). The scoring fn folds a grant into the row of its own `source_type` — no lookup. A future activity source is an additive forward-only CHECK change here + at 0096, in lockstep.
2. **One score row per (season, player, dimension); OVERALL is DERIVED at read time** (sum across dimensions), NEVER a stored denormalized row — so there is no second write path to keep in sync.

3. **Incremental, idempotent accrual via `last_counted_at`** — the high-water mark of the latest `reward_grants.granted_at` already folded in (NULL until first count). The future scoring fn accrues only grants with `granted_at > last_counted_at` and advances the mark in the same write, so a re-run never double-counts and never re-reads old events (the 0100/0105 `secured_at` idempotency analogue).
4. **No writer this slice.** The **SOLE** writer of `ranking_standings` is Ranking's OWN future season-scoring function (a later slice: `SECURITY DEFINER`, client-revoked). No RPC reads/writes it yet — dark, inert.
5. **Reset by season, never by deletion.** A reset is a **NEW** `season_id` scoping a fresh standings set; old standings rows and the `reward_grants` behind them are never deleted. The `on delete cascade on season_id` is a schema-integrity guard for an intentionally-removed season, NOT the reset mechanism.

Boundary placement (same-step doc-sync).

- `docs/SYSTEM_BOUNDARIES.md` §1 matrix: added the `ranking_standings` row under the **Ranking** owner (sole writer = future scoring fn; public read-only; DARK), adjacent to `ranking_seasons`.
- `docs/SYSTEM_BOUNDARIES.md` §2 **Ranking** row: extended Owns to `ranking_seasons`, `ranking_standings` (0128); kept the READ-ONLY-downward-leaf role accurate (reads `reward_grants` DOWNWARD, overall derived at read time, edges Ranking → Reward read · Reference/Config flag read — acyclic, nothing calls into Ranking). Added the explicit Must-NOT "store a denormalized OVERALL score".

Human gates preserved. `ranking_enabled` stays 'false' (no flag flipped); every Phase 11–17 flag remains 'false'. No existing migration edited (0001–0127 untouched, forward-only). No `game_config` value changed. Backend-only (no `src/features/**`). No merge/deploy/production apply/workflow dispatch. Slice is inert: no reader/writer references `ranking_standings` yet.

2026-07-04 — RANKING-P17 SLICE 0 — dark flag `ranking_enabled` + the Ranking-owned root table `ranking_seasons` (migration 0127)

Request. Begin Phase 17 (Ranking / competition — ROADMAP :92 "weekly/monthly seasons; combat/trade/explore/mine · reads finalized events; reset by season, not deletion"). Slice 0 only: ONE new forward-only migration (the dark flag + the seasons foundation) + same-step doc-sync. NO scoring/season/read function, NO standings table, NO frontend, NO flag flipped true.

Work done — migration `20260618000127_ranking_p17_seasons_and_flag.sql` (**forward-only; 0001–0126 unedited**).

- **(a) Dark flag.** Seeded `game_config('ranking_enabled', 'false')` with the exact 0097/0102/0107/ 0117/0124 slice-0 flag idiom (on conflict (key) do nothing, inherits the table-wide `game_config_public_read` posture). Description records that it gates ALL future Ranking scoring/read/season RPCs, each of which must check it FIRST and reject-before-any-read while false (fails closed both server + UI). No flag flipped true.
- **(b) `ranking_seasons` — the NEW Ranking-owned root table.** Columns: `season_id` uuid pk default `gen_random_uuid()`, cadence text check in ('weekly','monthly'), label text, `starts_at/ends_at` timestampz, status text default 'upcoming' check in ('upcoming','active','closed'), `created_at`, plus a table-level check (`ends_at > starts_at`). Integrity: a **partial unique index** `ranking_seasons_one_active_per_cadence` (`unique (cadence) where status = 'active'`) — AT MOST ONE active season per cadence. RLS ON + ONE public-read select policy + grant select to anon, authenticated; **NO insert/update/delete policy and NO write grant** — clients cannot mutate. A table comment records: Ranking is the sole writer, seasons are the reset-by-season scoping mechanism (never delete event data), DARK behind `ranking_enabled`.

Self-approved locked design decisions (owner-directed; grounds later slices).

1. A **season is a named scoring WINDOW per cadence**. A weekly AND a monthly season may run CONCURRENTLY over the same finalized events (independent leaderboards) — cadence is part of identity, not mutually exclusive. Direct reading of "weekly/monthly seasons".
2. **Reset-by-season, NEVER by deletion** (ROADMAP law). A reset is a NEW season row scoping a new [`starts_at`, `ends_at`] window; the finalized event ledger (`reward_grants`) is never deleted or truncated. Scores partition by the window without touching any event.
3. **At most one active season per cadence** (the partial unique index) — the "one active window" invariant; `upcoming/closed` seasons are unconstrained (history + scheduling).
4. **No writer this slice.** The **SOLE** writer of `ranking_seasons` is Ranking's OWN future season-management function (a later slice: `SECURITY DEFINER`, client-revoked; rows are runtime-created, NOT migration-seeded). No RPC reads or writes the table yet — dark, inert.

Finalized-event source (later slices, not built here). Ranking's read source is the idempotent reward ledger `reward_grants` (0015): `UNIQUE (source_type, source_id)` = one row per SECURED activity result; `source_type` ('combat','exploration','mining','trade' — the closed 0096 activity domain) is the activity dimension, `player_id` the leaderboard subject, `granted_at` the season-window field. A per-player, per-season, per-activity score is fully derivable from a plain DOWNWARD read — no writer to `reward_grants` or any activity table, ever.

Boundary placement (same-step doc-sync — the 0098/0103/0117 table-creating-slice precedent).

- docs/SYSTEM_BOUNDARIES.md §1 matrix: added the `ranking_seasons` row under a NEW **Ranking** system (sole writer = Ranking's future season fn; public read-only; DARK behind `ranking_enabled`).
- docs/SYSTEM_BOUNDARIES.md §2: added the **Ranking** system row — Owns `ranking_seasons` (0127); role = a READ-ONLY downward leaf consumer of finalized events (reads `reward_grants` DOWNWARD in later slices; writes only its own tables); Must-NOT = write any other system's table, be written by any non-Ranking function, or reset scores by deleting event data. Edges (later slices) are all DOWNWARD (Ranking → Reward read · Reference/Config flag read) — nothing calls into Ranking, so the call graph stays **acyclic**, one sole-writer per table, and Ranking is a NEW dark **leaf** owner.

Human gates preserved. `ranking_enabled` stays 'false' (no flag flipped); every Phase 11–17 flag remains 'false'. No existing migration edited (0001–0126 untouched, forward-only). No `game_config` value changed except ADDING the new dark key. Backend-only (no `src/features/**`). No merge/deploy/production apply/workflow dispatch. Slice is inert: no reader/writer references `ranking_seasons` or the flag yet.

2026-07-04 — CAPTAIN cleanup audit — SYSTEM_BOUNDARIES §1-matrix doc-sync (rows `captain_types` / `captain_recipe_ingredients` / `ship_captain_assignments`)

Request. Cleanup/audit pass over the Captain milestone (Phase 15 assignment 0117–0123 + Phase 16 progression 0124–0126, backend-only). Read-only recon (`CLEANUP_CAPTAIN_RECON.local.md`) + the narrowest same-step law-doc sync for the ONE defect it found. NO migration change, NO flag write, NO code/frontend change.

Defect found (F-1, LOW, docs-only). docs/SYSTEM_BOUNDARIES.md §1 (the sole-writer matrix) had three rows frozen at their creation-slice, contradicting the as-built 0120–0126, the fully-current §2 Captain row, and the §4 adapter note (a law doc that contradicts the code is a defect — the doc-sync law): `captain_types` said "nothing reads them yet", `captain_recipe_ingredients` said "nothing reads it yet", and `ship_captain_assignments` said "NO caller exists yet" / the settled-SAFE rule "lands in the later command slice". The parallel rows `ship_module_fittings` (kept current — "called today ONLY by `fitting_execute_command`"), `captain_instances`, and `captain_assignment_receipts` proved the matrix was half-updated, not a deliberate convention.

Fix (docs-only, same step — the TRADE_ECONOMY_CLEANUP_RECON doc-only-defect precedent). Rewrote the three §1 rows to present tense matching the as-built surface, mirroring the current-row phrasing:

- `captain_types` → "read today by the Phase-15 stats-adapter `calculate_expedition_stats` (0122 ...), the read surface (0123), and the captains_mint_instance/production_recruit_captain type-existence checks (0118/0126)".
- `captain_recipe_ingredients` → "Read DOWNWARD today by its consumer, the Phase-16 Production-owned recruit command `production_recruit_captain` (0126), for the recruit cost".
- `ship_captain_assignments` → the settled-SAFE rule "lives in the command `captain_execute_command` (0121 ...)"; "the command `captain_execute_command` translates" its exception-style reasons; "called today ONLY by that command (0120/0121)"; "the adapter (0122)" (was "the future adapter").

The SOLE-WRITER / ownership facts in all three rows were already correct and are unchanged; only the stale reader/caller/rule-timing prose was corrected. §2, §4, docs/ARCHITECTURE.md, and every migration are untouched (they were already in sync). No flag flipped; 0001–0126 unedited; no `game_config` value touched.

2026-07-04 — CAPTAIN-P16 SLICE 4 — the dark-posture verifier scripts/verify-captain-progression.mjs (the verify-captain.mjs analogue) + the self-approved "no new read RPC" decision

Request. Phase 16 slice 4, the final implementation slice: ONE new verify script proving migrations 0124–0126 ship exactly as claimed and fully dark, mirroring `verify-captain.mjs` point-for-point for the recruitment surface, plus one `verify:captain-progression` package.json line and same-step doc-sync. NO migration change, NO flag write, NO lit-path testing, NO frontend, NO new read RPC.

Work done — NEW `scripts/verify-captain-progression.mjs` (mirrors `verify-captain.mjs`; ZERO inline harness copies — imports `resolveEnv/createReporter/createUserFactory/Abort` from `scripts/lib/verify-harness.mjs` + `teardownVerifier` from `scripts/lib/verifier-teardown.mjs`; the same `admin/anon/throwaway-user/cfgVal` scaffold and the same `.catch/.finally` teardown, passing NO flag entry since this verifier touches none). Assertions, all with `anon/authenticated` clients only:

- **§1 Dark rejection** — with a throwaway authenticated user, a syntactically valid `p_request_id` AND a REAL captain type id ('gunnery_veteran'), `recruit_captain` returns `{ok:false, code:'feature_disabled'}` — the anti-probe gate fires BEFORE any

validation while `captain_progression_enabled='false'`. CODE-keyed (like `craft_module` 0109), NOT the reason-keyed assignment surface — the 0126 wrapper envelope matched exactly.

- **\$2 Recipe catalog contract** — reads `captain_recipe_ingredients` publicly and asserts the exact 0125 seed set verbatim: the five recipes' (`captain_type_id`, `item_id`, `qty`) rows (gunnery_veteran shard1/weapon_parts3/pirate_alloy2 · trade_broker shard1/scrap8/repair_parts2 · survey_cartographer shard1/scan_data4/anomaly_shard2 · extraction_foreman shard1/ore6/crystal2 · fleet_quartermaster shard1/repair_parts3/engine_parts2), and every `qty > 0`. Reading the public seeds back IS the public-read posture assertion.
- **\$3 Player-state RLS + no client write path** — `captain_recruit_receipts` returns 0 rows for a fresh user, and a direct client insert is denied (no insert policy / no write grant — 0126).
- **\$4 Internal surface locked** — `production_recruit_captain` denied to the authenticated client; `recruit_captain` denied to anon.
- **\$5 Config presence** — `captain_progression_enabled` reads `'false'` (READ-ONLY; the storage-form-tolerant `String(v)=== 'false'` compare).

No-flag-write / no-lit-path stance (verbatim to `verify-fitting.mjs/verify-captain.mjs`): the script NEVER writes `game_config` and NEVER flips `captain_progression_enabled`. Lit-path verification (flag on → recruit within balance → success + one new `captain_instances` row + one receipt → insufficient balance → `insufficient_items` → verbatim replay returns the original receipt without a second mint/spend → `unknown_captain` / `no_recipe` reasons) is DEFERRED to the human owner's activation checklist — flip the flag on a DEV database and run the lit checks there, never here.

`package.json` — one new line adjacent to `verify:captain`: `"verify:captain-progression": "node scripts/verify-captain-progression.mjs"`.

Self-approved read-surface decision (2026-07-04): NO new read RPC for Phase 16. Following the 0110 precedent (decision 2 — a read RPC that duplicates an already-available surface is not added), Phase 16 adds NO get-recipe/get-receipt RPC because every recruitment-relevant surface is ALREADY readable: (a) the recruitment RESULT — `captain_instances` — is exposed by `get_my_captain_instances()` (0123); (b) the recipe catalog `captain_recipe_ingredients` is public-read (clients select it directly, the `item_types/captain_types` stance); (c) `captain_recruit_receipts` is owner-read (direct select). A get-catalog/receipt RPC would duplicate an already-available surface — so none is added. (When the feature lights up, the client reads its recipes from the public catalog, submits `recruit_captain`, and sees the new captain via the existing 0123 read.)

Doc-sync — SYSTEM_BOUNDARIES intentionally UNTOUCHED this slice. A verifier script + a package.json line + a not-added read RPC add NO table, NO writer, NO function, NO cross-system edge — so no architectural fact changed (the slice-I precedent: the Phase-15 verifier left SYSTEM_BOUNDARIES untouched for the same reason). `docs/SYSTEM_BOUNDARIES.md` is deliberately not edited.

Human gates preserved. No flag flipped true, no migration change, no lit-path DB run (the verifier's execution belongs to the owner's activation checklist, exactly as the Phase-15 slice-I verifier), no frontend/client change, no production DB / merge / deploy / workflow dispatch. This CLOSES Phase 16's backend + verifier deliverables — dark, server-rejected, PR-ready on the feature branch.

2026-07-04 — CAPTAIN-P16 SLICE 3 — the dark recruit command:

`captain_recruit_receipts` + `private production_recruit_captain` + `two-layer recruit_captain` (the 0109 `craft_module` analogue, point-for-point)

Request. Phase 16 slice 3, the core: ONE new forward-only migration adding the Production-owned recruit command, mirroring 0109 `module_craft_command` POINT-FOR-POINT with the captain domain substituted (`craft a module` → `recruit a captain`). NO read surface, NO adapter change, NO frontend, NO flag flipped.

Work done — NEW `supabase/migrations/20260618000126_captain_p16_recruit_command.sql` (leaves 0001-0125 unedited; forward-only). Every idiom inherited from the 0109 mirror:

- `captain_recruit_receipts` — Production-owned per-player idempotency ledger, the `module_craft_receipts` (0109:55–84) shape verbatim: `receipt_id` uuid pk default `gen_random_uuid()`, `player_id` → `auth.users` on delete cascade, `request_id` text, `captain_type_id` → `captain_types(id)`, `instance_id` → `captain_instances(id)` on delete cascade, `created_at`, unique (`player_id`, `request_id`). No extra index (the unique index leads on `player_id`). RLS on, owner-read `captain_recruit_receipts_select_own` using (`player_id = auth.uid()`), grant select to authenticated (NOT anon), no write path. Table comment matches the 0109 shape (sole writer = `production_recruit_captain`; replay verbatim; DARK behind `captain_progression_enabled`).
- `production_recruit_captain(player, captain_type, request_id)` — PRIVATE writer, SOLE writer of `captain_recruit_receipts`, the `production_craft_module` (0109:86–195) body verbatim: (1) DARK GATE FIRST `cfg_bool('captain_progression_enabled')` → `feature_disabled` before any read; (2) `request_id` non-empty + `length ≤ 200` → `invalid_request_id`; (3) `pg_advisory_xact_lock(hashtext('captain_recruit'), hashtext(player))` before the replay

check; (4) idempotency replay — existing (player, request_id) receipt rebuilds the original success envelope verbatim (idempotent_replay:true, no re-spend/re-mint, no payload-conflict check); (5) catalog validation — type exists → else unknown_captain; recipe rows exist → else no_recipe (distinct truthful reasons); (6) ingredient PRE-CHECK loop over captain_recipe_ingredients (ordered by item_id) via inventory_get_balance → insufficient_items

 {item_id, have, need} BEFORE any spend; (7) SPEND loop via inventory_spend (any exception rolls back the whole tx — no receipt, nothing minted); (8) MINT exactly ONE via captains_mint_instance(player, captain_type, 'recruit:|player|'|':'|request_id) — namespaced per 0108, can never collide with craft::; (9) insert the receipt + return {ok:true, receipt_id, instance_id, captain_type_id, recruited_at}.

- recruit_captain(request_id, captain_type) — authenticated public wrapper, the craft_module (0109:197–263) idiom: auth.uid() null → not_authenticated; flag gate FIRST (anti-probe, identical answer while dark) → feature_disabled; delegate; on success re-emit; on failure map reason → code/message (invalid_request_id→invalid_request; unknown_captain; no_recipe→"This captain cannot be recruited yet."; insufficient_items→"Not enough materials to recruit this captain." + pass-through {item_id, have, need}; else unavailable).
- **ACLs** (0109:265–273): private writer revoked from public/anon/authenticated, granted service_role; wrapper revoked from public/anon, granted authenticated.

DARK — the whole surface ships server-rejected. captain_progression_enabled='false' (0124), so the wrapper's gate AND the writer's first check both reject every call; no receipt/instance row can exist today. No client UI (backend only, like Phase 15).

Boundary edges (all DOWNWARD, acyclic — no cycle). production_recruit_captain fans out one-directionally: Production → Inventory (inventory_spend, reusing 0109's EXISTING spend edge) · Production → Captain (captains_mint_instance mint — this is now that leaf's FIRST caller) · Production → Captain recipe read (captain_recipe_ingredients) · Production → Reference/Config (cfg_bool). Recruitment NEVER touches player_inventory/inventory_ledger/captain_instances directly — only through the two leaves (the forbidden-column law). Captain stays a pure instance-leaf: the recipe CONFIG is Captain's, the recruit COMMAND is Production's (which owns the Inventory spend), so there is NO Captain→Inventory edge and NO second writer to any table.

Doc-sync (SAME step).

- docs/SYSTEM_BOUNDARIES.md §1 matrix gains captain_recruit_receipts under **Production** (sole writer = production_recruit_captain; owner-read; DARK behind captain_progression_enabled), adjacent to module_craft_receipts.
- §2 **Production** row: Owns column gains captain_recruit_receipts *(0126)*; the row body adds the recruit_captain→production_recruit_captain two-layer command with its new DOWNWARD edges (Production → Captain mint + recipe read, reusing Production → Inventory spend + Reference/Config); the Must-NOT column adds "write captain_instances directly (recruit minting ONLY via Captain.captains_mint_instance)" and "recruit while the gate is off".
- §2 **Captain** row + §1 captain_instances row: the "NO caller exists yet" note on captains_mint_instance is REPLACED — its ONE caller is now production_recruit_captain (0126); the DARK gate reference updated to captain_progression_enabled (the recruit command's gate). The 0125 recruit-recipe-config note updated from "next-slice consumer" to "consumer is now 0126". This is the 0109-replaces-Modules-note precedent (a law doc must never contradict the code).

Human gates preserved. No flag flipped true (captain_progression_enabled and all captain flags stay 'false'), no read surface, no adapter change, no frontend/client change, no migration 0001–0125 edited, no production DB / merge / deploy / workflow dispatch. Dark, server-rejected, PR-ready on the feature branch.

2026-07-04 — CAPTAIN-P16 SLICE 2 — the recruitment recipe catalog captain_recipe_ingredients + seeds (the 0107 module_recipe_ingredients analogue)

Request. Phase 16 slice 2: ONE new forward-only migration adding the recruit recipe config table captain_recipe_ingredients (the captain analogue of Modules' module_recipe_ingredients, 0107) + five seed recipes, plus same-step doc-sync. NO command, NO writer, NO RPC, NO adapter change, NO frontend, NO flag flipped.

Recipe design decision (self-approved 2026-07-04). The recruit recipe is a normalized, items-only, existing-items-only catalog — the exact 0107 recipe posture, one implicit recipe per captain type:

- **Normalized-table encoding, not jsonb** (0107 decision 3): FK to captain_types + item_types, qty > 0 CHECK, composite PK (captain_type_id, item_id) — real referential integrity, no parallel jsonb recipe vocabulary.
- **Items-only cost** — the pipeline law: progression consumes INVENTORY, so recruitment's cost lands ONLY in player_inventory (via the next-slice command's inventory_spend), NEVER in metal/credits/Base/Wallet.
- captain_memory_shard **the shared gating ingredient** — the 'progression/'rare' item (0039) seeded expressly for this, qty 1 on every recipe; each type adds two specialization-flavored materials from the existing catalog.
- **Existing item_types only** (0039 + 0097) — no new item invented; quantities in the 0107 1–8 band.

Work done — NEW supabase/migrations/20260618000125_captain_p16_recruit_recipes.sql (leaves 0001–0124 unedited; forward-only):

- create table public.captain_recipe_ingredients (captain_type_id text ref captain_types(id), item_id text ref item_types(item_id), qty integer check (qty>0), created_at, PK (captain_type_id, item_id)) — mirrors module_recipe_ingredients line-for-line.
- RLS/grants VERBATIM from 0107:96–98 — RLS enabled, ONE public-read select policy captain_recipe_ingredients_public_read using (true), grant select to anon, authenticated, NO insert/update/delete policy, NO write grant → clients cannot mutate; only migration/service_role write (the 0039/0107 catalog posture).
- Five seeds (on conflict (captain_type_id, item_id) do nothing), all item ids verified to exist and all captain_type ids verified against 0117:


```
<font name="Sym"> </font> gunnery_veteran (combat): captain_memory_shard 1 · weapon_parts 3 · pirate_alloy 2
<font name="Sym"> </font> trade_broker (trade): captain_memory_shard 1 · scrap 8 · repair_parts 2
<font name="Sym"> </font> survey_cartographer (exploration): captain_memory_shard 1 · scan_data 4 · anomaly_shard 2
<font name="Sym"> </font> extraction_foreman (mining): captain_memory_shard 1 · ore 6 · crystal 2
<font name="Sym"> </font> fleet_quartermaster (support): captain_memory_shard 1 · repair_parts 3 · engine_parts 2
```

INERT this slice. Nothing reads the table yet. Its FIRST consumer is the next-slice **Production**-owned recruit command (ROADMAP law 5 "Production = crafting"), which reads this config DOWNWARD — the acyclic 0109 fan-out: Production → Captain recipe read · Production → Inventory inventory_spend · Production → Captain captains_mint_instance mint. One sole-writer per table; this slice adds NO writer and NO cross-system edge, and NO Captain→Inventory edge (Captain stays a pure instance-leaf; the recipe CONFIG belongs to Captain, the recruit COMMAND to Production).

Doc-sync (SAME step — the 0107 catalog-table-creating precedent).

- docs/SYSTEM_BOUNDARIES.md §1 matrix gains captain_recipe_ingredients under **Captain** (catalog/config — seeded by migration only, NO runtime writer; public read-only), adjacent to captain_types, mirroring how module_recipe_ingredients sits under Modules.
- §2 Captain row: the Owns column gains captain_types, captain_recipe_ingredients *(catalog/config ... no runtime writer)* (the Modules-row idiom), and the row body notes the recruit recipe config now exists with its first consumer the next-slice Production recruit command — NO writer/edge added this slice.
- A pure catalog table under an existing system adds a table but NO new writer/function and NO cross-system edge, so the acyclic ownership graph is unchanged; documenting the new owned config IS the required sync (unlike 0124's pure flag seed, which added no table and so left SYSTEM_BOUNDARIES untouched).

Human gates preserved. No flag flipped true (all captain flags stay 'false'), no frontend/client change, no migration 0001–0124 edited, no production DB / merge / deploy / workflow dispatch. Backend catalog/config only, on the feature branch.

2026-07-04 — CAPTAIN-P16 SLICE 1 — the dark capability flag captain_progression_enabled = 'false' (flag-only, the 0107/0117 seed idiom) + the self-approved Phase-16 design decision

Request. Phase 16 slice 1: ONE new forward-only migration seeding exactly one game_config row captain_progression_enabled = 'false', mirroring the 0117 captain_assignment_enabled / 0107 module_crafting_enabled flag-seed idiom, plus same-step doc-sync. NO recipe table, NO command, NO writer, NO adapter change, NO frontend, NO flag flipped true.

Self-approved Phase-16 design decision (owner-directed 2026-07-04; recorded here so later slices are grounded). Phase 16 "Captain progression (consumes inventory)" (ROADMAP :91) is the captain analogue of module crafting (Phase 13 / 0109 craft_module):

- **Mechanism = captain RECRUITMENT via consuming inventory.** A dark, idempotent command spends a per-captain-type item recipe through the Inventory inventory_spend leaf (0039) and mints ONE captain_instances row through the already-built captains_mint_instance leaf (0118 — built explicitly for "Phase-16 progression consuming inventory"). This is the truest reading of ROADMAP law 3 (Progression = the inventory-consuming acquisition system): "inventory is the bridge".
- **Reuse, no new writer.** Recruitment NEVER touches player_inventory / inventory_ledger / captain_instances directly — ONLY through the two pre-built leaves. No second writer to captain_instances (sole writer stays captains_mint_instance), no schema change to it, no adapter change (the 0122 stats feed reads the minted rows unchanged). Edges all DOWNWARD (Progression command → Inventory inventory_spend · Captain captains_mint_instance · Reference/Config catalog + this flag), acyclic — the exact 0109 fan-out shape. Any recruitment bookkeeping that would otherwise need a second writer to an existing table lives in a NEW progression-owned table with its OWN sole writer, read DOWNWARD by the adapter.
- **Later slices, all gated on THIS flag** (reject-before-any-read while false): a Production-owned recruitment receipts table + its private writer + a two-layer public wrapper command (the 0109/0113 idiom, PLAYER-scoped request_id idempotency), plus a per-captain-type recipe catalog.

Work done — NEW supabase/migrations/20260618000124_captain_p16_progression_flag.sql (leaves 0001–0123 unedited; forward-only):

- Seeds exactly one `game_config` row: ('captain_progression_enabled', 'false', ...) with a description mirroring the 0117 wording — server-authoritative dark gate for Phase-16 captain progression (recruitment that consumes `player_inventory`); OFF until the owner explicitly enables it; every Phase-16 RPC must check it FIRST and reject-before-any-read while false; UI stays hidden independently (fails closed both sides).
- on conflict (key) do nothing — idempotent. Flips NO flag true, creates NO table/function.
- The migration header records the full self-approved Phase-16 locked design above.

FLAG-ONLY AND INERT. No RPC, recipe table, writer, or reader references the flag yet — the 0117 "flag exists dark, no RPC yet" posture. The row inherits the table-wide public-read `game_config_public_read` policy (0003:13-15). No execute-surface relock needed (no function created — 0054 precedent).

Doc-sync — SYSTEM_BOUNDARIES intentionally UNTOUCHED this slice. Re-read docs/SYSTEM_BOUNDARIES.md: `game_config` is already Reference/Config public-read (§1 matrix, "unit_types, game_config, ..."), and a pure key seed adds NO table, NO writer, NO cross-system edge — so no architectural fact changed. Per the 0117-analogue deferral (a doc must never describe state that isn't real — the 0111:57-58 no-row-yet posture), NO §2 Captain-progression system row is added until a writer/function actually exists in a later slice. SYSTEM_BOUNDARIES is deliberately not edited.

Human gates preserved. No flag flipped true (`captain_progression_enabled` and all captain flags stay 'false'), no frontend/client change, no migration 0001–0123 edited, no production DB / merge / deploy / workflow dispatch. Backend flag-seed only, on the feature branch.

2026-07-04 — CAPTAIN-P15 SLICE I — the DARK-posture verifier scripts/verify-captain.mjs (the verify-fitting.mjs analogue; the sole remaining Phase-15 deliverable)

Request. Phase 15 slice I: ONE new verify script proving migrations 0117–0123 ship exactly as claimed and fully dark, mirroring `verify-fitting.mjs` point-for-point for the captain surface, plus one `verify:captain` package.json line and same-step doc-sync. NO migration change, NO flag write, NO lit-path testing, NO frontend.

Work done — NEW scripts/verify-captain.mjs (mirrors `verify-fitting.mjs`; ZERO inline harness copies — imports `resolveEnv/createReporter/createUserFactory/Abort` from `scripts/lib/verify-harness.mjs` and `teardownVerifier` from `scripts/lib/verifier-teardown.mjs`):

- **§1 Dark rejection** — with a throwaway authenticated user and syntactically VALID uuids/ `request_ids` (so the identical answer proves the anti-probe gate fires BEFORE any validation): `assign_captain_to_ship` / `unassign_captain_from_ship` / `get_my_captain_instances` / `get_my_ship_captains` all return {ok:false, reason:'captain_assignment_disabled'} (the 0120 wrapper + 0123 read envelopes — reason-keyed, the ONE server-driven visibility signal, adapted from fitting's code-keyed `feature_disabled`).
- **§2 Catalog contract** — reads `captain_types` id/name/specialization/description/stats_json back verbatim for all five 0117 seeds (gunnery_veteran combat/attack 4 · trade_broker trade/cargo 8 · survey_cartographer exploration/scan 3 · extraction_foreman mining/mining 4 · fleet_quartermaster support/repair 3) and asserts every `specialization` sits in the CHECK set — reading the public seeds back IS the public-read posture assertion (the `item_types/module_types` posture).
- **§3 Player-state RLS + no client write path** — `captain_instances` / `ship_captain_assignments` / `captain_assignment_receipts` each return 0 rows for a fresh user, and direct client inserts are denied (no insert policy / no write grant — 0118/0119/0120).
- **§4 Internal surfaces locked** — `captain_assign_apply` / `captain_execute_command` / `captain_command_client_envelope` / `mainship_space_assert_settled_safe` denied to the authenticated client; the four public RPCs denied to anon.
- **§5 Config presence** — `captain_assignment_enabled` reads 'false' (READ-ONLY).
- **Deliberate no-flag-write / no-lit-path stance** (copied from `verify-fitting.mjs:20-28`): the script NEVER writes `game_config` and NEVER flips `captain_assignment_enabled`; every assertion runs with anon/authenticated clients only; `service_role` is used ONLY for teardown (delete the throwaway user via the shared `teardownVerifier`, no flag entry passed). Lit-path verification (assign within slots → success + adapter stats change with specialization tradeoffs → over-capacity `captain_slots_full` → settled-SAFE `ship_not_settled` → already/not-assigned → verbatim replay without double-assign → unassign reverts stats) is deferred to the human owner's activation checklist — flip the flag on a DEV database and run the lit checks there, never here.

package.json — one line added adjacent to `verify:fitting`: "verify:captain": "node scripts/verify-captain.mjs".

Doc-sync. docs/SYSTEM_BOUNDARIES.md **untouched** — a verifier adds NO table, NO writer, NO cross-system edge, so no architectural fact changed (the ownership matrix and Captain §2 row are already correct as of slices A–G).

Verify: `npm run build green` (confirms nothing else drifted). The verifier itself is dark-posture proof only — it is NOT run against production and requires no flag flip.

2026-07-04 — CAPTAIN-P15 SLICE G — the dark read surface:

`get_my_captain_instances()` + `get_my_ship_captains(ship)` (the 0110/0116 analogues)

Request. Phase 15 slice G: ONE new forward-only migration with exactly two read-only RPCs, each mirroring its own analogue's idiom precisely (0110 for the instances roster, 0116 for the per-ship roster — both read first per the slice spec), plus same-step doc-sync. NO frontend, NO verify scripts, NO adapter change, NO flag changes.

Work done — NEW `supabase/migrations/20260618000123_captain_p15_read_surface.sql` (0001–0122 unedited):

- `get_my_captain_instances()` — the 0110 shape with its check ordering copied exactly (auth → dark gate → query): `jsonb envelope · stable · security definer set search_path = public · dark gate on captain_assignment_enabled BEFORE` any row read, returning the identical literal `{ok:false, reason:'captain_assignment_disabled'}` for every caller (no probing while dark; the same envelope the 0120 wrappers emit — ONE visibility signal) · own-rows-only query (query-scoped `player_id = auth.uid()`, defense in depth over RLS) joining `captain_instances` to `captain_types` display identity (name / specialization / stats_json) · `jsonb_agg(... order by created_at desc)` newest-first · `{ok:true, captains:[...]}` — PLUS a per-row assignment indicator (the assigned `main_ship_id` or null) via LEFT JOIN to `ship_captain_assignments`, so the client renders roster state from one call. **LOCKED DECISION (header):** the left join is read-only display data — no new writer, no new dependency direction (a Captain-owned function reading Captain-owned tables + the public catalog).
- `get_my_ship_captains(p_main_ship_id uuid)` — the 0116 shape including its exact gate/auth ORDERING (0116:23–26/51–56: gate FIRST, then auth — copied verbatim as the slice spec requires): identical dark reject → auth → ship validated by the (**main_ship_id, player_id**) pair (the 0079 multi-ship posture; foreign = missing → `ship_not_owned`) → that ship's roster joined via `captain_instances` to `captain_types`, ordered `assigned_at desc`, `captain_instance_id` (the 0116 determinism idiom with the uuid tiebreak) → `{ok:true, captains:[...]}`. **NO COUNTS — 0116 returns none, so none were added** (the slice spec's mirror rule; no speculative surface). The 0116:18–22 deliberate-omission rationale transfers: limits come from the client's own `main_ship_instances` rows (the 0043 grant covers `captain_slots`) or `get_my_expedition_preview` (carries `captain_slots_used/limit` since 0122).
- **NO catalog RPC** — the 0110:9–15 stance: `captain_types` is a public-read Reference/Config catalog the client selects directly; a get-catalog RPC would duplicate an already-public surface.
- **ACLs** copied from the analogues (0110:72–75 / 0116:84–87): both RPCs revoke from public, anon; grant execute to authenticated. Dark today — the gates reject every call.
- `docs/SYSTEM_BOUNDARIES.md` (same step): the §2 Captain row gains the read-surface contract (two gated read RPCs, identical dark envelope, own-rows only, the assignment indicator's no-new-writer note, the no-counts and no-catalog-RPC stances). §1 matrix unchanged — read surfaces are recorded in the §2 system row, not the matrix (the 0101/0106/0110/0116 precedent).
- **Verify:** `npm run build green` (SQL-only slice — confirms nothing else drifted). §5 invariant checklist re-read: NO new table, NO new writer (both functions are pure reads — every sole writer unchanged); no new cross-system edge (Captain-owned functions reading Captain-owned tables + the public catalog — graph unchanged, acyclic); no client write path (read-only RPCs, authenticated-only); reward path and combat truth untouched; no flag flipped — the whole surface ships server-rejected.

2026-07-04 — CAPTAIN-P15 SLICE F — assigned captains feed

`calculate_expedition_stats` (the 0115 analogue; third feed block, headcount-capped)

Request. Phase 15 slice F, the 0115 analogue recon §5 fully determines: ONE new forward-only migration re-creating `calculate_expedition_stats` with EXACTLY one addition — a THIRD feed block for captains — the support-craft and module paths byte-identical. NO read surfaces, NO frontend, NO verify scripts, NO flag changes.

Work done — NEW `supabase/migrations/20260618000122_captain_p15_stats_adapter.sql` (0001–0121 unedited; a create or replace re-create, the 0044/0115 forward-only idiom):

- **The captain block, mirroring 0115:157–196 point for point:** new declares (`c / v_cap_used / v_cap_speed_bonus`); roster read `ship_captain_assignments` → `captain_instances` → `captain_types` filtered by `a.main_ship_id = v_ship.main_ship_id` with NO player filter (the 0115:47–50 rationale cited in the header: the (1)(2) ship read proves ownership; `captain_assign_apply`'s owner-consistency invariant (0119) covers the rest); contributions into the SAME nine accumulators from the exact `stats_json`

key list (attack/defense/repair/cargo/scan/mining/evasion, coalesced to 0) with `speed_mult_bonus` summed into `v_cap_speed_bonus` — ONE stat pipeline, no parallel vocabulary.

- **Tradeoffs (locked values, header-documented):** a specialization CASE, one slot each so no cost scaling — combat → attention +2, `spd_pen +0.02 · trade` → attention +1, `spd_pen +0.02 · exploration` → attention +1 · mining → attention +1, `spd_pen +0.02 · support` → 0 (a captain draws attention like crewed hardware; support-role captains are the low-profile option; magnitudes mirror 0115's per-slot numbers).
- **Headcount HARD cap:** `v_cap_used > v_ship.captain_slots → raise` — count-based (one captain = one slot), defense-in-depth over the 0119 assign-time gate; refuse, never clamp.
- **Speed:** `round(greatest(0.2, v_speed * (1 + v_mod_speed_bonus + v_cap_speed_bonus) * (1 - a_spd_pen)), 3)` — the captain bonus joins the module bonus ADDITIVELY inside the one multiplier; a zero-captain ship reduces EXACTLY to the 0115 expression.
- **Output:** exactly two added keys `captain_slots_used / captain_slots_limit`, mirroring the module pair; no existing key's value changes for a zero-captain ship.
- **Compatibility contract (header, the 0115:10–18 way):** `verify:phase8 / verify:mainship-preview / verify:fitting` assert field VALUES and list-membership, never an exact key SET → additive keys are safe; and no assignment row can exist until the owner flips `captain_assignment_enabled` (0117 flag dark; 0118 instances have no producer; 0119–0121 command chain `service_role-only/dark`) → live behavior is unchanged today.
- **Diff proof run against 0115's shipped body:** the ONLY changes are (a) the three captain declares, (b) the captain feed block + count hard cap inserted between the module cap and the speed expression, (c) the one speed-expression edit (+ its comment), (d) the two output keys. The support-craft and module paths are byte-identical.
- **ACL:** the 0115:232–233 targeted re-assert verbatim — revoke from public/anon/authenticated, grant `service_role` only.
- `docs/SYSTEM_BOUNDARIES.md` (same step): §4 item 8 now records the third read edge (Captain), the three hard caps (`support_capacity / module_slots / captain_slots` — captain cap count-based), the additive speed-bonus composition, the two new output keys, and the "(later +captains)" note reworded as REDEEMED — THE single source of expedition stats (still not wired into live combat; the proven fleet-stack path owns outcomes). The §2 Captain row's inbound note gains the adapter-read edge (the 0115 Fitting precedent) — still nothing writes through Captain but its own command.
- **Verify:** `npm run build green` (SQL-only slice — confirms nothing else drifted). §5 invariant checklist re-read: the adapter owns no table and mutates nothing (pure read/compute, `stable`); no writer changes anywhere; the new edge is DOWNWARD read-only (adapter → Captain — graph stays acyclic); no client write path (`service_role-only`); reward path and combat truth untouched; no flag flipped.

2026-07-04 — CAPTAIN-P15 SLICE E — the settled-SAFE rule for captain assignment + the `mainship_space_assert_settled_safe` shared-leaf extraction (fitting re-created behavior-identically)

Request. Phase 15 slice E, the 0114 analogue: read 0114 first, determine whether the settled-SAFE check is a callable helper or inline, then add the same rule to captains honoring the no-duplication HARD RULE. NO adapter change, NO read surfaces, NO frontend, NO verify scripts, NO flag changes.

What 0114 was read to contain: the check is `INLINE` in `fitting_execute_command` (0114:126–142) — no callable composite exists; 0114:41–44 explicitly recorded "no shared-helper extraction is needed" because the mechanism appeared ONCE. Captain is now the second consumer → the hard rule triggers the extraction in this same step (case (b) of the slice spec).

Work done — NEW `supabase/migrations/20260618000121_captain_p15_settled_safe_rule.sql` (0001–0120 unedited; 0114/0120 stay as history):

- **NEW shared leaf** `mainship_space_assert_settled_safe(p_main_ship_id uuid)` returns `boolean` — the 0114:126–142 composite VERBATIM: `mainship_space_validate_context ok AND validated state in ('home', 'at_location')` (the 0100/0105 SAFE set) AND `mainship_space_assert_cross_domain_exclusion ok`; fail-closed (legacy `NULL / in_space / in_transit / destroyed / incoherent` → false). Main-Ship-owned (`mainship_space.*` family), `service_role-only` (the 0056 family ACL posture). **Signature is ship-id-ONLY, deliberately without the `p_player_id` the slice spec's example sketched:** its family siblings (0056:91/224) take only the ship id, and ownership resolution is per-action per-system semantics that must stay in each command for the fitting re-create to be behavior-identical — a player param would be either dead or a behavior change.
- `fitting_execute_command` **re-created — a PURE refactor, ZERO behavior change** (the compatibility contract stated in the header the 0115:10–18 way). **Diff proof run against 0114's shipped body:** the ONLY changes are (a) two declare lines dropped (`v_val/v_excl` moved into the leaf), (b) the step-6 comment updated to name the leaf, (c) the inline two-step check block replaced by the single leaf call inside the same `if v_check_ship is not null` guard (short-circuit AND preserves skip-if-null). Same reads, same evaluation order, same single truthful `ship_not_settled`. Wrappers, writer, receipts, mapper untouched on the fitting side.

- **captain_execute_command re-created with the rule** — the 0120 header's promised forward-only amendment (the P14 0113→0114 split, delivered). LOCKED DECISIONS (header): placement AFTER the replay check and action-shape validation, BEFORE delegating to captain_assign_apply (game rule in the COMMAND, structure in the WRITER); applies to BOTH actions — assign checks the TARGET ship (owner-scoped, ship_not_owned at this layer — the 0114 fit-branch shape), unassign checks the ship the captain is CURRENTLY assigned to (owner-scoped read of ship_captain_assignments; an unassigned captain SKIPS the rule — the structural writer's truthful not_assigned handles it, the exact 0114 unfit-branch semantics) — because a loadout, captain roster included, is frozen mid-transit/in-space/mid-combat; reject reason = the same truthful ship_not_settled. Everything else byte-identical to 0120.
- **captain_command_client_envelope re-created ONLY to add the ship_not_settled mapping** (reason + player-facing copy — 0120 shipped without it because the rule did not exist yet; the 0114:164–165 re-create rationale).
- **ACLs:** the new leaf service_role-only; the three re-created functions' grants re-asserted (the 0114:217–224 idiom). No other grants touched.
- **Safe to ship dark:** captain_assignment_enabled and module_fitting_enabled are both 'false' — no caller could reach the rule-less 0120 command in the gap, and none can reach these; no flag touched.
- docs/SYSTEM_BOUNDARIES.md (same step): the §2 Captain row's deferral note replaced by the shipped settled-SAFE contract (rule position in the flow, both-action semantics, the skip-for-unassigned nuance) and its edges sentence now names the shared leaf; the §2 Fitting row's inline-check wording now records that the composite lives in the shared leaf since 0121 (pure refactor); a new **Settled-SAFE leaf** note block added beside the OSN-geometry-leaf note (the family's documentation home) — including why exploration/mining deliberately do NOT consume it (their accepted state is in_space, a different rule) — so no section contradicts the new leaf.
- **Verify:** npm run build green (SQL-only slice — confirms nothing else drifted). §5 invariant checklist re-read: no new table and no writer change (ship_captain_assignments / ship_module_fittings / both receipt ledgers keep their single writers); the leaf owns no table and adds no new cross-system edge (Fitting and Captain already read Main Ship downward — graph stays acyclic); no client write path (leaf + commands service_role-only); reward path and combat truth untouched; no flag flipped.

2026-07-04 — CAPTAIN-P15 SLICE D — the dark assign/unassign command + captain_assignment_receipts (settled-SAFE rule deliberately deferred, the 0113→0114 split)

Request. Phase 15 slice D, mirroring the P14 command slice (0113): ONE new forward-only migration adding the player-scoped receipts ledger, the ONE private command, the TWO thin authenticated wrappers, and the shared reason→envelope mapper, plus same-step doc-sync. NO adapter change, NO read surfaces, NO frontend, NO verify scripts in this slice.

Work done:

- **NEW** supabase/migrations/20260618000120_captain_p15_assign_command.sql (0001–0119 unedited):

```
<font name="Sym"> </font> captain_assignment_receipts — the 0113:69–91 posture verbatim: PK (player_id, request_id)
(captains are non-spatial → the PLAYER-scoped keying, not the ship-scoped space receipts), action CHECK
('assign','unassign'), request fingerprint columns for audit, result_json (the success envelope, verbatim replay truth),
created_at; RLS own-row select + grant select to authenticated, NO write policy/grant.
```

```
<font name="Sym"> </font> captain_execute_command(p_player_id, p_action, p_captain_instance_id, p_main_ship_id,
p_request_id) — service_role-only, THE sole writer of the receipts, the exact 0113 flow order: dark gate FIRST on
captain_assignment_enabled (reject before any read/lock/write) → request_id validation → per-player advisory lock
BEFORE the replay check (the SAME ('captain_assignment', player) key as captain_assign_apply — reentrant, so the
nested acquisition is safe) → verbatim replay of the stored result_json on (player, request_id) hit (trade semantics, no
payload-conflict check) → action-shape validation in ('assign','unassign') → delegate DOWNWARD to the slice-C sole
writer captain_assign_apply (assign passes the ship id, unassign passes null — ship_captain_assignments keeps ONE
writer; this command writes only its own receipts) → only a SUCCESSFUL mutation writes a receipt.
```

```
<font name="Sym"> </font> Exception→envelope translation (the 0119 header's promise fulfilled): the writer is
exception-style, so the delegate runs in a guarded block translating its reason-prefixed raises
(captain_not_owned/ship_not_owned/already_assigned/captain_slots_full/ not_assigned) into failure envelopes;
UNKNOWN exceptions RE-RAISE (never hide a bug). The writer returns void, so the command builds the success envelope
(ok/action/ids) and stores it verbatim.
```

```
<font name="Sym"> </font> assign_captain_to_ship(p_request_id, p_captain_instance_id, p_main_ship_id) /
unassign_captain_from_ship(p_request_id, p_captain_instance_id) — the 0113 two-wrappers-one-command shape:
auth check → anti-probe dark gate returning the identical literal {ok:false, reason:'captain_assignment_disabled'}
for every caller → delegate with the fixed action. Reason-keyed client envelopes throughout (locked adaptation of 0113's
code-keyed ones, matching the 0110/0116 read-surface signal convention).
```

- captain_command_client_envelope — 0113's fitting_command_client_envelope was READ FIRST per the slice spec: its map is coupled to fitting's reason vocabulary (0113:219–250), NOT feature-generic, so the captain analogue was created and is called from BOTH wrappers — never inlining the map twice (the exact 0113:33–35 extraction rationale). Its feature_disabled entry emits the same literal dark envelope as the wrapper gates.
- Targeted ACLs (the 0113:305–317 block verbatim): private command + mapper revoked from public/anon/authenticated, granted to service_role only; the two wrappers revoke from public, anon; grant execute to authenticated (dark: both gates reject today).
- **LOCKED DECISION (header): the settled-SAFE game rule (ship must be home/at_location) is NOT in this slice** — it lands NEXT slice as a forward-only amendment of this command, mirroring exactly how P14 shipped 0113 (command) then 0114 (settled-SAFE). Safe because captain_assignment_enabled is 'false': the gate rejects before any read, so no caller can reach the rule-less command in the gap.
- docs/SYSTEM_BOUNDARIES.md (same step): §1 matrix gains the captain_assignment_receipts row in the module_fitting_receipts row's exact shape (ONE private command for both actions so the ledger keeps ONE writer, PK (player_id, request_id), verbatim replay, DARK — rejects before any read); the §2 Captain row extended with the command/wrapper contract, the exception→envelope translation, the deferred settled-SAFE note, the new downward Reference/Config (cfg_bool) edge, and the inbound client edge (ONLY the two authenticated wrappers); captain_assign_apply's "called by NOTHING yet" replaced — this command is its ONE caller.
 - Verify:** npm run build green (SQL-only slice — confirms nothing else drifted). §5 invariant checklist re-read: captain_assignment_receipts has exactly ONE writing system and no client write path; ship_captain_assignments keeps captain_assign_apply as its only writer (the command only delegates); edges stay DOWNWARD/acyclic (new: Reference/Config cfg_bool read); Activity table-less; reward path and combat truth untouched; no flag flipped — the entire surface ships server-rejected.

2026-07-04 — CAPTAIN-P15 SLICE C — ship_captain_assignments schema + the ONE assignment writer (inert AND dark)

Request. Phase 15 slice C, mirroring the P14 fittings slice (0112): ONE new forward-only migration creating the assignment junction table + the ONE structural sole writer, plus same-step doc-sync. NO receipts, NO client commands, NO settled-SAFE rule, NO read surfaces, NO frontend, NO adapter change, NO verify script in this slice.

Work done:

- NEW** supabase/migrations/20260618000119_captain_p15_assignments_schema.sql (0001–0118 unedited):

 ship_captain_assignments — **the captain_instance_id PK IS the one-ship-per-captain invariant** (the exact 0112:54 shape): PK FK captain_instances on delete cascade · main_ship_id FK main_ship_instances cascade · player_id FK auth.users cascade · assigned_at timestampz default now(); index on main_ship_id (the headcount cap + the future adapter's per-ship read, 0115:162–167); RLS on with the own-row SELECT policy + grant select to authenticated ONLY — no write policy/grant.

 captain_assign_apply(p_player_id uuid, p_captain_instance_id uuid, p_main_ship_id uuid) — THE sole writer covering ALL mutations: p_main_ship_id NOT NULL = ASSIGN, NULL = UNASSIGN (the fitting_apply one-writer shape — two functions would be two writers). Structural invariants enforced in the writer, reject never clamp: captain ownership; ship read by the **(main_ship_id, player_id) pair** (never "the player's ship" singular — the 0079 multi-ship posture); truthful already_assigned NAMING the current ship (PK backstops — never silently re-homed); and the **HEADCOUNT hard cap** count(*) < captain_slots — the captain analogue of $\sum \text{slot_cost} \leq \text{module_slots}$, count because slice A locked one-captain-one-slot. Owner-consistency guaranteed: stored player_id = captain owner = ship owner (the 0115:47–50 guarantee that later lets the adapter join without a player filter). Race safety: the per-player pg_advisory_xact_lock(('captain_assignment', player)) taken FIRST (the 0112:99–103 idiom) — the count→insert window is single-writer by construction. Targeted ACL: revoke public/anon/authenticated, grant service_role only.

 Error style (locked, documented in the header): exception-style (the 0039/0108 internal-leaf idiom) with stable reason-prefixed messages (captain_not_owned / ship_not_owned / already_assigned / captain_slots_full / not_assigned) — a deliberate deviation from fitting_apply's envelopes; the future command slice translates raised reasons into client envelopes.

 LOCKED DECISION (header): the settled-SAFE game rule is deliberately NOT in this structural writer — the dark gate, the home/at_location spatial rule (the 0114 layer), and receipt idempotency all land in the later COMMAND slice, exactly as P14 split 0112 (structure) from 0113/0114 (command + game rule). Until then nothing can call this writer (service_role-only, no caller exists), so the system stays inert AND dark — no row can exist today.
- docs/SYSTEM_BOUNDARIES.md (same step): §1 matrix gains the ship_captain_assignments row in the ship_module_fittings row's exact shape (Captain, ONE sole writer captain_assign_apply covering assign AND unassign, PK = one-ship-per-captain,

headcount ≤ captain_slots hard cap — reject never clamp); the §2 Captain row extended to name the new table + writer, the game-rule/adaptor deferrals, and the system's FIRST cross-system edge: Captain → Main Ship (read-only ownership + captain_slots) — downward, acyclic, nothing depends on Captain.

- **Verify:** npm run build green (SQL-only slice — confirms nothing else drifted). §5 invariant checklist re-read: ship_captain_assignments has exactly ONE writing system and no client write path (select-only policy/grant); the one new call edge is a downward read (Captain → Main Ship — graph stays acyclic, no second writer to main_ship_instances or any table); Activity stays table-less; reward path and combat source-of-truth untouched; no flag flipped.

2026-07-04 — CAPTAIN-P15 SLICE B — captain_instances schema + the single Captain mint writer (inert AND dark)

Request. Phase 15 slice B, mirroring the P13 instances slice (0108): ONE new forward-only migration creating the instances table + the ONE internal-leaf sole writer, plus same-step doc-sync. NO assignment table, NO receipts, NO commands, NO read surfaces, NO frontend, NO verify script in this slice.

Work done:

- **NEW** supabase/migrations/20260618000118_captain_p15_instances_schema.sql (0001–0117 unedited):
` captain_instances — INDIVIDUAL rows, never counts (no quantity column by design): id uuid PK gen_random_uuid() · player_id FK auth.users on delete cascade · captain_type_id FK captain_types · mint_key text not null unique (the idempotency spine) · created_at; player index (player_id, created_at desc); RLS on with the own-row SELECT policy + grant select to authenticated ONLY — no write policy/grant (the 0108:42–63 shape exactly). NO assignment columns — assigned-ship/slot state belongs to the later assignment slice's own junction table (the ship_module_fittings shape), forward-only there.`
` captains_mint_instance(p_player_id uuid, p_captain_type_id text, p_mint_key text) — THE ONE writer of captain_instances (internal leaf, SECURITY DEFINER): validates the mint key + catalog id with exception-style errors (no envelopes — the 0039/0108 internal-leaf idiom), inserts on conflict (mint_key) do nothing, and on replay returns the EXISTING instance id for that key (the 0108:95–104 idiom — the same key can never mint twice). Targeted ACL: revoke from public/anon/authenticated, grant execute to service_role only.`
` LOCKED DECISION (recorded in the migration header): no acquisition path is built in this slice — nothing calls captains_mint_instance yet. It is the future downward leaf for whatever grants captains (Phase-16 progression consuming inventory, or a later dark grant command), exactly as modules_mint_instance (0108) predated its craft command (0109) by one slice. The system is therefore inert AND dark: no client-reachable surface exists, and captain_assignment_enabled (0117) stays 'false' besides — no row can exist today.`
- docs/SYSTEM_BOUNDARIES.md (same step): §1 matrix gains the captain_instances row in the module_instances row's exact shape (system = Captain, sole writer = captains_mint_instance, idempotent by the NOT NULL UNIQUE mint_key, service_role-only internal leaf); and the **§2 Captain system row is added NOW** — the system has its first writer, so the row is real (the 0108 precedent; slice A had deferred it). Contract stated: owns captain instance state + the FUTURE assignment state, reads only its own captain_types catalog (downward, intra-system — no cross-system edge exists yet), no inbound client surface, everything dark behind captain_assignment_enabled.
- **Verify:** npm run build green (SQL-only slice — confirms nothing else drifted). §5 invariant checklist re-read against the migration: captain_instances has exactly ONE writing system (the mint leaf; no client write path — select-only policy/grant); no new cross-system call edge (the helper reads only Captain's own catalog — graph stays acyclic); Activity stays table-less; reward path and combat source-of-truth untouched; no flag flipped.

2026-07-04 — CAPTAIN-P15 SLICE A — dark flag captain_assignment_enabled + the captain_types catalog (foundations only)

Request. Phase 15 "Captain instances + assignment" (ROADMAP :90) slice A, mirroring the 0107/0111 catalog+flag idiom: ONE new forward-only migration + same-step doc-sync. NO instances/assignment/receipt tables, NO commands, NO read surfaces, NO frontend, NO verify scripts in this slice.

Work done:

- **NEW** supabase/migrations/20260618000117_captain_p15_catalog_and_flag.sql (0001–0116 unedited):
`<font name="Sym"> </font> Dark flag captain_assignment_enabled='false' inserted into game_config on conflict (key) do nothing (the exact 0107:63–69 shape) — created FALSE, NOT flipped; every future Phase-15 RPC must check it FIRST and reject-before-any-read while false.`

 `captain_types` catalog (Reference/Config posture verbatim from 0039/0042/0107: RLS on, ONE public-read select policy, `grant select to anon, authenticated, NO write policy/grant`): text id PK · name · specialization with a CHECK ('combat','trade','exploration','mining','support') — deliberately UNLIKE 0107's unconstrained display-only `slot_type`, because specialization is the captain analogue of the module `slot_type` tradeoff CASE (ROADMAP law 4: never a plain sum), a constrained mechanism input the later adapter slice consumes · `description` · `stats_json` jsonb not null default ' {} ' in the ONE shared stat vocabulary (attack/defense/repair/cargo/scan/mining/evasion + optional `speed_mult_bonus` — 0115:173–180; no parallel captain vocabulary).

 NO `slot_cost` column (locked decision): every assigned captain occupies exactly ONE slot — `main_ship_instances.captain_slots` (0043:58; starter frigate seeds 2) is a HEADCOUNT, not a point budget; the later adapter cap is `count(*) <= captain_slots` (reject, never clamp).

 Five seeds, one per specialization, on `conflict (id)` do nothing, each clearly weaker than the same-role module in the 0111 band (attack 10 / cargo 25 / scan 8): combat → attack 4 · trade → cargo 8 · exploration → scan 3 · mining → mining 4 · support → repair 3. Captains complement fitting, never replace it; Phase 16 progression (consumes inventory) is the growth path. Conservative, not final balance.

- docs/SYSTEM_BOUNDARIES.md (same step): §1 matrix gains the `captain_types` row under the new **Captain** system in the `module_types` row's exact shape (catalog/config, migration-seeded only, NO runtime writer, public read-only). **The §2 Captain system row is deliberately DEFERRED** to the instances slice: no writer/function exists yet, and a doc must never describe state that isn't real (the 0111 no-Fitting-row-yet precedent). That slice adds it together with the first writer.
- Verify:** `npm run build green` (SQL-only slice — confirms nothing else drifted). §5 invariant checklist re-read against the migration: the new table is migration-seeded catalog/config with no runtime writer and no client write path; no new call edge (acyclic graph unchanged); no reward-path or combat-truth change; no flag flipped.

2026-07-04 — CLEANUP SLICE 4 (final, scripts-only) — auto-cleanup part 4: the trade proof scripts' five duplicated blocks extracted into the sourced `scripts/lib/trade-proof-lib.sh`

Request. Part 4 — the FINAL slice of the module-fitting-milestone auto-cleanup: the three trade proof orchestrators (`trade-economy-bootstrap-proof.sh` / `trade-fleet-0c-proof.sh` / `trade-market-1-proof.sh`) each carried near-byte-identical copies of five shell blocks. Extract them into ONE sourced library, adopted by all three in the same step. Scripts-only — NO migration, `src/`, `flag`, `CI/workflow`, or other-script change; NO change to what any proof proves.

Work done:

- NEW** `scripts/lib/trade-proof-lib.sh` (sourced, never executed; sited beside the existing shared mjs verifier libs `verify-harness.mjs/verifier-teardown.mjs`) exposing the five blocks as functions — the header states who sources it and that NEW trade-proof scripts must source it rather than re-copying: `fail + (1) tp_init` (arg/usage scaffold: shell opts, global MODE, `usage → exit 2`), **(2) `tp_assert_self_rolling_back`** (begin;/final-ROLLBACK/ no-COMMIT static checks — one implementation of the byte-identical block), **(3) `tp_assert_flags_inside_txn`** (ONE list/loop form; `trade-fleet-0c-proof.sh`'s single-flag inline spelling became a one-element list call — same logic per the recon), **(4) `tp_assert_out_of_scope`** (the identical `src/-and-migrations` guard), **(5) `tp_run_local`** (the local-mode `psql + PASS-line + per-marker greps`, on bootstrap's existing `$MARKERS/$PASS_LINE` interface). Feature-specific pieces stay in each caller as parameters/greps (SQL path, flag list, marker list, PASS line, provisioning/reject-token/ property asserts, the selftest summary echo) — the lib never forks per caller. The one-line : `"${DB_URL:?...}"` env contract stays in each caller so its diagnostic keeps naming the script, not the lib.
- All three scripts converted** to source the lib: 76→52 / 74→55 / 74→53 lines (net -69 across the three; the lib is 80).

Behavior identical — verified honestly within sandbox limits: `bash -n` clean on all four files (`shellcheck` is NOT available in this sandbox — stated plainly). Before/after outputs captured for EVERY DB-free path of all three scripts — no-arg usage (exit 2), `selftest` (DB-free static checks, exit 0), and `local` without `DB_URL` (exit 1): usage and selftest outputs are BYTE-IDENTICAL; the only diff anywhere is the bash-generated line NUMBER inside the `DB_URL` diagnostic (line 66 → line 51 etc. — the scripts got shorter; script name, message text, and exit codes unchanged). The lib's failure paths were exercised directly on doctored SQL files (missing begin; / missing rollback; / missing flag / src-reference) — each fires the exact pre-change `FAIL: ...` message with exit 1. The real `local psql` mode cannot be exercised here (no disposable DB; the documented environmental precedent) — it is the same `psql/grep` text verbatim, parameterized, and remains the owner/CI gate. One fail-path-only wording note: 0c's flag-assert failure copy now uses the shared loop form ("...the dark flag 'mainship_additional_commission_enabled'..." instead of "...the dark add-ship flag...") — unreachable on the green path and semantically identical.

- docs/SYSTEM_BOUNDARIES.md **explicitly needs NO change** — a shell-block extraction inside the proof harness adds no table, writer, flag, or cross-system edge; no architectural fact changed. **This completes the module-fitting-milestone auto-cleanup (parts 1–4).**

2026-07-04 — CLEANUP SLICE 3 (frontend) — auto-cleanup part 3: the four-way duplicated guard body extracted into `runGuardedCommand`

Request. Part 3 of the post-milestone auto-cleanup: the same ~20-line guarded command-submit body (`tryClaim` → `pending/note reset` → `try { await command; mounted guard; ok → note + refresh; else → mapped error note } finally { release; conditional pending clear }`) lived at FOUR call sites — `ExplorationPanel scan`, `MiningPanel extract`, `ModulesPanel craft` and `runFitting`. Extract it into ONE shared helper and adopt it at all four sites in the same step. NO behavior change, no copy change, no fail-closed render-guard change; no migration/flag/script change.

Work done:

- `src/lib/useActivityPanelGuards.ts` — new exported `runGuardedCommand<R>` extends `{ok: boolean}>` beside `tryClaim/release/activeRef` (the module IS the guard idiom's shared home; header comment extended). One options object: `{ key, guards, setPending, setNote, exec, successNote, errorNote, refresh }` — the body preserves the exact current semantics and ORDER (bail unless claimed · pending on · note cleared · exec once per accepted claim · mounted guard after the await · ok → success note + refresh, else → decorated error note · finally always releases, pending clears only while mounted). The doc comment carries the shared rationale (synchronous double-submit guard, mounted guard, finally-release); `Extract<R,...>` casts hand each callback the discriminated member (generic `R` does not narrow by `res.ok` alone — the `isServerLit` stance).
- **Four call sites converted to thin wrappers**; the site-specific pieces stay AT the sites as closures: the `!mainShipId` pre-guard (`Exploration/Mining`, before the helper); request-id minting moved INSIDE each site's `exec` thunk (`crypto.randomUUID()` — the `runFitting` idiom; still fresh per submit since `exec` runs once per accepted claim); boolean setters (`scan/extract`) vs per-row Record updaters over the fixed/per-row key (`craft/runFitting`); and each site's error decoration (`mining's cooldown ~Ns`, `craft's insufficient_items item/have/need`, `runFitting's insufficient_slots used/limit/needs`, `exploration plain`). `runFitting` keeps its `exec/verb` params from JSX and forwards through the helper. Per-site scaffold comments trimmed to the site-specific parts; the helper's doc comment carries the shared explanation.
- **MarketPanel intentionally NOT touched** (recon-note scope): its `submit` carries an extra synchronous `validate`→`release` step before the `await` — a different posture; it adopts the helper on its next real change, per the established adopt-on-next-real-change rule.
- `docs/SYSTEM_BOUNDARIES.md` **needs NO change** — stated explicitly: a frontend-only extraction adds no table, writer, flag, or cross-system edge; no architectural fact changed.

Verify (honest): `npm run build green (tsc -b + vite — typecheck included)`. `npm run lint`: the four touched files are CLEAN (`npm run eslint` on them exits 0); the repo-wide run reports 14 PRE-EXISTING errors, all in untouched files (`src/features/map/MainShipMarker.tsx` / `SpaceRouteLine.tsx` / `useSpaceMoveCommand.ts` and `tests/ harness/spec` files) — none introduced by this slice. Each converted site's diff visually confirmed to preserve the claim-key, pending/note targets, and decoration logic exactly.

Follow-up (separate slice, NOT this step): part 4 — shared `scripts/lib/trade-proof-lib.sh` for the three trade proof scripts' duplicated blocks.

2026-07-04 — CLEANUP SLICE 2 (docs-only) — auto-cleanup part 2: the fleets commission writes recorded as the sanctioned Main-Ship shim (with retirement condition)

Request. Part 2 of the post-milestone auto-cleanup: the Main-Ship port-entry commission path writes `fleets` directly — `port_entry_commission_build` (0080; called by `port_entry_commission_writer` and the dark `commission_additional_main_ship`, 0080/0091) inserts the commissioned ship's present/location fleet row, and `normalize_main_ship_dock` (0084) normalizes dock state on `fleets` — while \$1 named **Fleet** the sole writer with no recorded exception. An undocumented second writer on `fleets` is a LAW-DOC DEFECT (the doc contradicted shipped code). Docs-only — NO migration, code, script, or flag change in this step.

DESIGN DECISION (planner authority): DOCUMENT-THE-SHIM, not the repoint migration. Rationale: the commission path's bodies are guarded by the FROZEN md5-pinned PORT-ENTRY production verifiers (`normalize_main_ship_dock` and `port_entry_commission_writer` — the build core's caller — are two of the three prosrc-md5-pinned bodies; see 0084's header and `docs/TRADE_FLEET_0C_VERIFIER_REPOINT.md`), and the path is the ACTIVE first-ship onboarding writer. A behavior-identical CREATE OR REPLACE purely for boundary hygiene would invalidate deploy-gate md5 pins (a deploy-time human-gate concern) and add risk to a live path for zero functional gain. The honest, reversible fix today is to make the law doc match reality with an explicit retirement condition.

Retirement condition (recorded verbatim in the \$1 note): the exception retires when the port-entry path is next reworked for a FUNCTIONAL reason; at that point the `fleets` writes MUST be repointed through a Fleet-exposed commission/dock function

via a forward-only migration (re-deriving the PORT-ENTRY prosrc-md5 pins at that deploy gate) and the \$1 exception note deleted.

Work done (docs only):

- **\$1 fleets/fleet_units row** — owner cell amended in the matrix's existing long-parenthetical idiom: Fleet stays the sole writer EXCEPT the ONE sanctioned Main-Ship port-entry commission shim (the two functions above, writes confined to the calling player's OWN rows), with the retirement condition and the not-repointed-now rationale attached in place.
- **\$2 Main Ship row** — corrected ONLY the contradicting Must-NOT clause: "touch fleets" now carries the "(except ...)" parenthetical pointing at the \$1 exception note (the Combat row's existing "(except request return via Movement)" idiom). The \$2 Fleet row contradicts nothing and was not touched; verified the caller set against 0080/0084/0091 before wording.
- `npm run build green` (docs-only sanity).

Follow-ups (separate slices, NOT this step): part 3 — shared frontend guard helper for the four duplicated command-submit bodies; part 4 — shared `scripts/lib/trade-proof-lib.sh` for the three trade proof scripts.

2026-07-04 — CLEANUP SLICE 1 (docs-only) — module-fitting-milestone auto-cleanup part 1: `market_offers` law-doc sync

Request. Part 1 of the post-milestone auto-cleanup: fix two DOC DEFECTS in `docs/SYSTEM_BOUNDARIES.md` where the law doc contradicted shipped code (0085/0087/0089/0090). Docs-only — NO code, migration, script, or flag change in this step.

The defects (law doc contradicted shipped code):

- `market_offers` (shipped in 0085 as the Trade Market price catalog) had **no \$1 sole-writer row at all** — 0085's own header claimed a `SYSTEM_BOUNDARIES` ownership posture that the doc never actually recorded.
- The \$2 Trade Market row attributed the RPCs' reads to "`trade_goods` + the docked-location context", never mentioning `market_offers` — but ALL offer prices actually come from `market_offers` (`get_market_offers` projects the docked station's active offers (0087), `market_buy` takes its `sell_price` (0089), `market_sell` its `buy_price` (0090)); `trade_goods` genuinely provides only good identity/metadata (`unit_volume_m3` for the buy-side volume check, 0089 — `market_sell` reads it not at all).

Work done (docs only):

- **\$1 ownership matrix** — added the `market_offers` row directly after its sibling catalog `trade_goods`, following that row's exact Reference/Config idiom: owner **Reference/Config** (admin/migration; Trade Market price catalog — migration-seeded only (0085, idempotent seed), NO runtime writer) · read = public read-only (RLS public-read policy, no client write path).
- **\$2 Trade Market row** — corrected ONLY the reading clause: prices now attributed to `market_offers` (read by all three RPCs as above); `trade_goods` kept for what it still truly provides (good identity/metadata — `unit_volume_m3` for the buy-side volume check). Nothing else in \$2 touched.
- Verified against 0085/0087/0089/0090 before wording; `npm run build green` (docs-only sanity).

Follow-ups (separate slices, NOT this step): part 2 — commission-writer repoint decision (`port_entry_commission_build` / `normalize_main_ship_dock` write `fleets` directly while \$1 names Fleet its sole writer); part 3 — shared frontend guard helper for the four duplicated command-submit bodies (`ExplorationPanel scan` / `MiningPanel extract` / `ModulesPanel craft` + `runFitting`); part 4 — shared `scripts/lib/trade-proof-lib.sh` for the three trade proof scripts' duplicated blocks.

2026-07-04 — FITTING-P14 SLICE G (final) — `verify:fitting` dark-posture script. Phase 14 Module fitting — dark implementation complete (slices A–G)

Request. Implement slice G, the last Phase 14 slice: the dark-posture verify script + its `package.json` entry, mirroring the P13 slice-F verifier exactly (read end-to-end first: `verify-modules.mjs`, `scripts/lib/verify-harness.mjs`, `scripts/lib/verifier-teardown.mjs`, the `package.json` verify cluster). Touches ONLY `scripts/verify-fitting.mjs`, `package.json`, this file, and the recon scratch file. No migrations (head stays **0116**), no CI/workflow edits, no flags.

Flag-handling mechanism — the twins', stated and followed exactly. The script NEVER writes `game_config` and NEVER flips `module_fitting_enabled` — dark contracts only (the `verify-mining.mjs:16–20` mechanism; the `set_game_config` flip in `verify-mainship-send.mjs` is the explicitly-NOT-copied alternative). Lit-path behaviors live in the HUMAN ACTIVATION CHECKLIST below — run on a DEV database by the owner, never by this script. Teardown: the shared `teardownVerifier` deletes the throwaway user (the 0112/0113 player FKs cascade its rows away); no flag entry is passed — nothing to restore, `module_fitting_enabled` stays exactly as found.

Work done:

- **NEW** `scripts/verify-fitting.mjs` — shared harness imports from day one (`Abort/createReporter/createUserFactory/resolveEnv + teardownVerifier`) — ZERO inline harness copies. Service key OPTIONAL (teardown only); one throwaway signup. Asserts, in the twins' order/idioms: (1) **dark rejection** — authenticated `fit_module_to_ship` AND `unfit_module_from_ship` → `{ok:false, code:'feature_disabled'}` with syntactically VALID `uuids/request_ids` passed, so the identical dark answer proves the 0113 anti-probe gate fires BEFORE any validation; and `get_my_ship_fittings` → `{ok:false, reason:'module_fitting_disabled'}` (0116); (2) **catalog contract (0111, exact)** — `module_types.slot_cost/stats_json` publicly readable; the four archetypes' seeds verbatim (autocannon `1/{"attack":10} · thruster 1/{"evasion":3,"speed_mult_bonus":0.1} · cargo lattice 2/{"cargo":25} · sensor 1/{"scan":8}`) and every `slot_cost` `>= 1` — public-read IS the posture assertion (the P13 inversion note applies); (3) **player-state RLS + no client write path** — fresh user sees 0 rows in `ship_module_fittings` + `module_fitting_receipts`; inserts denied on both; (4) **internal surfaces locked** — `fitting_apply`, `fitting_execute_command`, and `fitting_command_client_envelope` denied to the authenticated client; the three public RPCs denied to anon; (5) **config presence (read-only)** — `module_fitting_enabled` = false via the same `jsonb-storage-tolerant` comparison.
- `package.json` — `"verify:fitting": "node scripts/verify-fitting.mjs"` added directly after `verify:modules`, same command shape.
- **CI note:** the exploration/mining/modules verifiers are wired into NO workflow file — nothing to mirror, and no workflow was created or modified. Wiring `verify:fitting` into CI, if desired, is a human / PR-review step.
- **Verify posture run honestly:** `node --check scripts/verify-fitting.mjs` parses clean; `npm run build` green. `node scripts/verify-fitting.mjs` in this sandbox aborts at the throwaway SIGNUP step with the environmental TLS failure ("fetch failed / unable to verify the first certificate") — `node scripts/verify-modules.mjs` aborts at the IDENTICAL point in the SAME run (the P12/P13 precedent), so this is the known environmental-fail-only posture and reaching that identical abort point proves the harness wiring. The assertions themselves run against a real DB in the owner's environment.
- `docs/SYSTEM_BOUNDARIES.md` needs NO change this slice — confirmed and stated (the P12/P13 verify-slice precedent): a read-only verifier script + one `npm` alias adds no table, writer, or cross-system edge.

Phase 14 Module fitting — dark implementation complete (slices A–G) — closing summary

- **Migrations** 0111–0116 (head 0110 → 0116; all forward-only; 0001–0110 never edited): 0111 config/flag + stats catalog (`module_fitting_enabled='false'` + `module_types.slot_cost/stats_json` with the four seeded archetypes) · 0112 the fittings table + THE ONE writer (`ship_module_fittings` with the `module_instance_id` PK-as-invariant + `fitting_apply`, service_role-only) · 0113 the two-layer command (`module_fitting_receipts` + `fit_module_to_ship/unfit_module_from_ship` → private `fitting_execute_command`; dark-gate-first, lock-before-replay, trade-semantics verbatim replay, failure-writes-no-receipt) · 0114 the settled-SAFE rule correction (`ship_not_home` → `ship_not_settled`) · 0115 the adapter integration (fitted modules feed `calculate_expedition_stats` under the `module_slots` hard cap; +`module_slots_used/module_slots_limit`) · 0116 the read surface (`get_my_ship_fittings`).
- **Frontend:** the fitting section EXTENDS `src/features/modules/` (`types/api/panel` + the `mainshipApi.ts` `fetchMyMainShips` list variant) — double-gated server-driven visibility, fails closed to nothing; per-instance fit/unfit controls with display-only slot arithmetic.
- **Verify:** `scripts/verify-fitting.mjs` + `npm run verify:fitting` (dark posture only, shared harness, never flips flags).
- **Locked design decisions (each with its one-line rationale):**
 1. **Fitting is a NEW leaf system** (ROADMAP law 5 "Fitting=modules") owning `ship_module_fittings` + `module_fitting_receipts` — never a second writer or new columns on `module_instances`.
 2. **ONE writer for BOTH mutations** (`fitting_apply`: ship = FIT, null = UNFIT) — one sole writer per table covers ALL its mutations; two functions would be two writers.
 3. **ONE private command for both actions** (`fitting_execute_command`) — so the receipts table keeps ONE sole writer.
 4. **Capacity hard-reject + slot_type tradeoffs, never a raw sum** — $\sum \text{slot_cost} \leq \text{module_slots}$ enforced at fit time AND re-checked in the adapter (raise, never clamp — the 0044 mechanism); weapon/cargo/sensor tradeoffs mirror the role rules.
 5. `stats_json` **reuses the base_stats_json idiom** (same seven keys through the SAME accumulators) + `speed_mult_bonus` applied before penalties — one stat pipeline.
 6. **Settled-SAFE rule (C2 correction)** — the 0113 'home' literal was dead-on-arrival (no writer produces it); 0114 ships the 0100/0105 SAFE state set + the 0099/0104 companion machinery (intent preserved, literal fixed, precedent reused).
 7. **Extend-not-duplicate frontend** — the fitting UI lives in `ModulesPanel` (which already lists instances); consequence: double-gated, renders nothing while either flag is dark.
 8. **Two deliberate read-surface omissions** (0116) — no catalog RPC (public-read direct selects) and no ship `module_slots` in the RPC (limits come from the client's own `main_ship_instances` rows / `get_my_expedition_preview`).
- **Ownership/edges recap:** all DOWNWARD/acyclic — Fitting → Modules (read) · Main Ship (read, incl. the OSN context helpers) · Reference/Config (read); inbound only the adapter's 0115 READ. Sole writers: `ship_module_fittings` = `fitting_apply` (called only by `fitting_execute_command`) · `module_fitting_receipts` = `fitting_execute_command` (via the two wrappers). `SYSTEM_BOUNDARIES` §1/§2 synced in the SAME step as every fact change.

- **HUMAN ACTIVATION CHECKLIST (the owner's gate — never this loop):** (1) apply migrations 0111–0116 to the target DB; (2) run `npm run verify:fitting` there — expect ALL dark-posture checks green; (3) optionally flip `module_fitting_enabled='true'` on a DEV database and exercise the lit path: craft (or service-role-mint) module instances, then fit within slots → success AND the adapter stats change with the tradeoffs visible in `get_my_expedition_preview` (attack/evasion/cargo/scan up; pirate_attention/speed per the slot_type rules; `module_slots_used/limit` correct); an over-capacity fit (e.g. $2 \times \text{cargo lattice} + \text{autocannon} = 5 > 3$) → `insufficient_slots` with `{used, cost, limit}` and NOTHING written; a fit/unfit while the ship is in_space/in_transit → `ship_not_settled`; `already_fitted` (naming the current ship) and `not_fitted` codes fire; REPLAY the same (player, request_id) → the verbatim envelope + `idempotent_replay:true` and provably NO double-fit; unfit → the adapter stats revert; `verify:phase8`, `verify:mainship-preview`, and `verify:m2/m3/m4/m45` all stay green throughout; then flip the flag back and decide production activation separately. The loop ships everything server-rejected; activation is exclusively the human's.

State. `npm run build` green; `node --check` clean on the new script. Migration head **0116**; `module_fitting_enabled='false'` everywhere; no flag flipped, no live DB write, no workflow touched. **Phase 14 Module fitting is implemented DARK end-to-end and PR-ready on** `autopilot/20260703-064048` — SAFE FOR HUMAN MERGE REVIEW; main untouched.

2026-07-04 — FITTING-P14 SLICE F — dark frontend: the fitting section EXTENDS `src/features/modules/` (fit/unfit controls inside `ModulesPanel`; renders nothing while either flag is dark). Frontend only — no migration

Request. Implement slice F: the dark fitting UI as a minimal extension of the existing `src/features/modules/` feature. NO migration (head stays **0116**), no config, no verify script. Read end-to-end first: `modulesTypes.ts` / `modulesApi.ts` / `ModulesPanel.tsx`, the shared `src/lib/useActivityPanelGuards.ts`, the `GalaxyMapScreen.tsx` mounting, the `mainshipApi.ts` ship-reading convention, and the twins' `crypto.randomUUID()` request-id idiom.

DECISION — EXTEND, don't duplicate (locked): the fitting UI extends `ModulesPanel` rather than adding a parallel panel, because the panel already lists the player's module instances and a second panel would duplicate that list (the no-duplication rule). **CONSEQUENCE (recorded honestly): the fitting section is server-gated TWICE** — it renders only when the CRAFTING read surface is lit (the panel's existing `isServerLit` gate on `get_my_module_instances`, `module_crafting_enabled`) AND `get_my_ship_fittings` answers ok (`module_fitting_enabled`) — it fails closed both ways and renders NOTHING today. With both flags 'false' the fittings RPC is not even called (it rides the lit branch); with crafting lit and fitting dark, every fitting element is behind a `litFittings &&` gate, so the rendered output is exactly the pre-slice-F markup.

Work done (4 files, all existing modules — no new feature dir, no `GalaxyMapScreen` change):

- `modulesTypes.ts` — added the fitting types: `ShipFittingRow` (the seven 0116 fields) + `GetMyShipFittingsResult`; `FittingCommandResult` (the 0113/0114 wrapper envelopes — success passes the writer's fitted/unfitted + slot facts through with the replay flag; failure carries code/message with the real `insufficient_slots {used, cost, limit}` and `already_fitted {main_ship_id}` context); and the `FITTING_ERROR_COPY` map + `fittingErrorMessage()` covering `feature_disabled` / `invalid_request` / `ship_not_settled` / `module_not_owned` / `ship_not_owned` / `already_fitted` / `not_fitted` / `insufficient_slots` / `not_authenticated` / `unavailable`. The read reason `module_fitting_disabled` is handled by fail-closed rendering, not copy (stated in the section header).
- `modulesApi.ts` — three thin wrappers in the existing envelope idiom (transport error → normalized failure, never a throw into the render path): `getMyShipFittings()`, `fitModuleToShip(moduleInstanceId, mainShipId, requestId)`, `unfitModuleFromShip(moduleInstanceId, requestId)`.
- `mainshipApi.ts` — minimal extension INSIDE the existing module (never a second ship-select elsewhere): the ship column list was extracted to `SHIP_COLS` (now used by both selects) and `fetchMyMainShips()` added — the multi-ship-ready LIST variant of `fetchMyMainShip` (same owner-read RLS, same columns incl. `module_slots`; [] on error, non-fatal).
- `ModulesPanel.tsx` — the fitting section: `getMyShipFittings()` rides the existing lit-branch read batch (mount + `lifecycleKey`); ships fetched only once fitting is lit. Per instance row (all double-gated): fitted state joined from the fittings result by `module_instance_id` ("Fitted → <ship name>" + an Unfit control), or a fit control — ship picker over the player's ships labeled name ($\Sigma \text{slot_cost used} / \text{module_slots}$) computed from the already-loaded fittings data (**display-only arithmetic**; `fitting_apply's hard cap` + **the 0114 settled-SAFE rule remain the enforcer**, commented in place) + a Fit button. Each command generates a fresh `crypto.randomUUID()` request id (the twins' idiom; the server dedups on (player_id, request_id)), claims the row synchronously via the shared `tryClaim(instance_id)` (craft rows key by catalog slug, fitting rows by instance uuid — the key spaces cannot collide, noted), disables the row while in flight, renders the server's message (falling back to the copy map, with the real `{used, cost, limit}` suffix on `insufficient_slots` — the `insufficient_items` idiom), and refetches instances + fittings on success. One shared `runFitting()` executes both commands (no duplicated submit block).

Doc-sync note. `docs/SYSTEM_BOUNDARIES.md` needs NO change — confirmed and stated (the MODULES-P13 SLICE E precedent): frontend-only; no table, no writer, no cross-system edge; the client reads/commands only through the shipped RPCs (0113/0114/0116) and the existing owner-read selects.

State. `npm run build green (tsc -b + vite, exit 0 — this slice DOES touch src; one TS error was caught and fixed during the slice: a double-wrapped Promise<ReturnType<...>> in the shared submit helper's type). Targeted eslint on all four touched files: exit 0.`

Dark-render trace (manual, per the request): with both flags 'false' the panel's first read returns `module_crafting_disabled` → `isServerLit false` → the panel renders null, byte-identical to before this slice (the fittings RPC is never called); with crafting lit + fitting dark every added element is behind `litFittings &&` → the instances list renders the exact pre-slice markup. Migration head stays **0116**; `module_fitting_enabled='false'`; nothing flipped, no live DB write, no workflow touched. PR-ready on `autopilot/20260703-064048`, main untouched. Next: slice G (`scripts/verify-fitting.mjs` + the `verify:fitting` package.json entry).

2026-07-04 — FITTING-P14 SLICE E — the dark read surface 0116 (`get_my_ship_fittings()`). Server side of Phase 14 complete, fully dark

Request. Implement slice E: ONE new forward-only migration with the read surface, mirroring the modules read surface (0110, the 0101/0106 family) — re-read end-to-end first. NO write path, NO new table, NO frontend, NO verify script this slice; flag stays 'false'.

Work done — NEW `supabase/migrations/20260618000116_fitting_p14_read_surface.sql` (migration head moves **0115** → **0116**; `0001-0115` unedited):

- `get_my_ship_fittings()` — the 0110 body idiom (`jsonb envelope · stable · security definer · set search_path = public · jsonb_agg row shape + coalesce-to-[] · {ok:true, fittings:[...]}` plural envelope), with ONE deliberate divergence recorded in the header: **the dark gate runs FIRST, then auth** (0110 checks auth first) per the slice spec — `{ok:false, reason:'module_fitting_disabled'}` identically for every caller while dark (the frontend's server-driven visibility signal; anon has no execute grant anyway), then the 0110 `not_authenticated` posture. Per row: `module_instance_id`, `main_ship_id`, `fitted_at`, `module_type_id`, plus the catalog display fields the future panel needs (`name`, `slot_type`, `slot_cost`), joined DOWNWARD via `module_instances` to `module_types`; ordered `fitted_at` desc **then** `module_instance_id` (determinism — the 0110 ordering idiom + a uuid tiebreak since several fittings can share a timestamp). Rows are scoped `player_id = auth.uid()` IN THE QUERY (defense in depth over the 0112 own-row RLS, as 0110 does).
- **NO catalog RPC** (the 0110 stance restated in the header): `module_types` (incl. the 0111 `slot_cost/stats_json`) is a public-read Reference/Config catalog read by direct client select — an RPC would duplicate an already-public surface.
- **DECISION — deliberately NO ship module_slots in this RPC** (recorded so the omission is never read as forgotten): the slot LIMIT belongs to the ship, not the fitting rows — the client reads its own `main_ship_instances` rows (the 0043 own-row grant covers `module_slots`) or `get_my_expedition_preview` (whose stats carry `module_slots_used/module_slots_limit` since 0115). The surface stays dumb — fitting rows only.
- **ACL (0110:72-75 verbatim):** execute revoked from public/anon, granted to authenticated only — and dark today: the gate rejects every call while `module_fitting_enabled='false'`. Table RLS unchanged.

Doc-sync (same step). `docs/SYSTEM_BOUNDARIES.md`: the §2 Fitting row gained `get_my_ship_fittings()` with its gate-first semantics, the no-catalog-RPC stance, and the no-ship-limit decision. The §1 **matrix is UNCHANGED — confirmed and stated**: no new table, no new writer (the 0101/0106/0110 precedent: read surfaces are recorded in the §2 system row, not the matrix).

State. `npm run build green` (no `src/` change was made — confirmed). Migration head **0116**; `module_fitting_enabled='false'` everywhere. **The server side of Phase 14 Module fitting is COMPLETE (slices A-E) and fully dark end-to-end:** the flag + stats catalog (0111), the fittings table + THE ONE writer (0112), the two-layer fit/unfit command + receipts (0113, settled-SAFE rule 0114), the adapter integration (0115), and the read surface (0116) — every client-reachable surface server-rejects while the flag is false; the writer/command internals are service_role-only. No flag flipped, no live DB write, no workflow touched. **DB-apply posture (honest, unchanged):** no `psql/docker/supabase` CLI in this sandbox — the migration was hand-verified line-by-line against 0110 at the idioms cited above; live assertions run in the owner's environment and will be covered by the slice-G `verify:fitting` dark-posture script. PR-ready on `autopilot/20260703-064048`, main untouched. Next: slice F (dark frontend `src/features/fitting/` or a minimal `src/features/modules/` extension), then slice G (`scripts/verify-fitting.mjs` + the `verify:fitting` entry).

2026-07-04 — FITTING-P14 SLICE D — stats integration 0115 (fitted modules feed `calculate_expedition_stats` under the `module_slots` hard cap; re-create of the 0044 adapter)

Request. Implement slice D: ONE new forward-only migration re-creating the stat adapter so fitted modules feed expedition stats via capacity/tradeoff — ROADMAP law 4's "replace the SOURCE of stats ... capacity + tradeoffs, never a plain sum", now

real for modules. NO frontend, NO verify script, NO wrapper/writer/receipts change this slice. Read first, end-to-end: 0044 (the function being re-created), 0049 (its only live caller), and BOTH pinning scripts.

What the pinning scripts actually assert (checked, not assumed):

- `verify-phase8.mjs` asserts specific field VALUES (empty loadout → `support_capacity_used 0, limit 10, combat_power 0, speed 1, cargo_capacity = ship's`; per-craft deltas; rejection/exception cases; determinism = two identical calls compared to each other; ship + inventory not mutated) and LIST-membership finiteness over its `NUM_FIELDS` array (`:38-39, .every(...)`) — it never asserts an exact key SET, so ADDING keys is safe.
- `verify-mainship-preview.mjs` (reported per the request): asserts envelope/value checks only — `has_ship/valid` flags, `stats.support_capacity_limit === 10/used === 0/combat_power === 0 (:52-54)`, `used === 3 + combat_power > 0` for one `missile_boat (:56-58)`, over-capacity → `valid:false` with an error message matching `/capacity/i (:60-62` — the support-capacity exception text is unchanged, so this still matches), unknown craft → `valid:false`, no-ship → hull-teaser fields, preview-writes-nothing, and the adapter still denied to clients. It also never asserts an exact key set — additive keys are safe here too.

Work done — NEW `supabase/migrations/20260618000115_fitting_p14_stats_adapter.sql` (migration head moves **0114** → **0115**; 0001-0114 unedited): create or replace `calculate_expedition_stats` — SAME signature, support-craft path byte-identical, and the module feed ADDED between the loadout capacity check and the final jsonb build:

1. **Read** the ship's fit set: `ship_module_fittings` (for `v_ship.main_ship_id`) → `module_instances` → `module_types` (`slot_cost/slot_type/stats_json` — the 0111 columns' FIRST code consumer). Pure downward read; no player filter needed — the existing owned-ship read plus `fitting_apply`'s owner-consistency invariant (0112) guarantee the fit set is the player's (commented in place).
 2. **Capacity** — 0044:112-115 verbatim: $\Sigma \text{slot_cost} > v_ship.module_slots \rightarrow \text{raise exception}$. Defense-in-depth: fit-time enforcement in `fitting_apply` is primary; the adapter still refuses to compute from an over-capacity state rather than clamp or trust it.
 3. **Contributions** into the SAME accumulators the loadout loop uses (`a_combat/a_survival/a_repair/a_cargo/a_scout/a_mining/a_retreat`), exact key list `attack/defense/repair/cargo/scan/mining/evasion`, coalesced to 0 — one stat pipeline, no parallel module pipeline.
 4. **Speed** — $\Sigma \text{speed_mult_bonus}$ applied BEFORE penalties (the slice-A locked model): `round(greatest(0.2, v_speed * (1 + v_mod_speed_bonus) * (1 - a_spd_pen)), 3)` — floor and rounding untouched; zero modules reduces the expression to 0044's exactly.
 5. **Tradeoffs (numbers + rationale, recorded)**: `slot_type CASE × slot_cost` (the module analogue of `×qty`) — **weapon** → attention +2·cost, speed_pen +0.03·cost; **cargo** → attention +2·cost, speed_pen +0.04·cost; **sensor** → attention +1·cost; **engine** → no tradeoff. Rationale: weapons/cargo mirror the 0044 `combat_damage/cargo` role tradeoffs — more firepower / a bigger hold draws pirates and slows the burn; active sensors emit (attention only); the engine's cost is the slot itself. Unknown/future `slot_types` contribute stats but no tradeoff (CASE else 0 — 0044's permissive unmatched-role posture). No activity-tag warning for modules — `module_types` has no `activity_tags` column.
 6. **Output** — exactly two added keys, `module_slots_used/module_slots_limit`, mirroring the support-capacity pair. **THE COMPATIBILITY CONTRACT**: a ship with no fitted modules returns today's values for every pre-existing key — which is what keeps `verify:phase8` / `verify:mainship-preview` green; and no fitted module can exist anywhere until the owner flips the dark flag.
- **ACL** re-asserted with the targeted idiom (0084/0113/0114 posture; same end state 0044 established: `service_role` only, never clients — `get_my_expedition_preview` (0049) remains the one client path and needs no change: it passes the adapter's jsonb through, so the two new keys simply appear in previews).

Doc-sync (same step). `docs/SYSTEM_BOUNDARIES.md`: §4 item 8 (stat adapter) now records the 0115 extension — reads `ship_module_fittings+module_instances+module_types` downward read-only, enforces BOTH hard caps (raise, never clamp), same accumulators + speed bonus, the two added keys, the zero-module compatibility contract; the §2 Fitting row's "future adapter edge" note became the real shipped edge (Expedition-stats → Fitting is a READ by the adapter; nothing writes through Fitting but its own command). Still acyclic; the adapter owns no table.

State. `npm run build green` (no `src/` change was made — confirmed). Migration head **0115**; `module_fitting_enabled='false'` — the adapter change is inert today (no fitting rows can exist while the command surface is dark, so every caller sees pre-0115 values plus `module_slots_used:0/module_slots_limit`). No flag flipped, no live DB write, no workflow touched. **DB-apply posture (honest, unchanged)**: no `psql/docker/supabase CLI` in this sandbox — the re-created function was mechanically diffed against 0044 (only the stated additions: declares, the module block, the two speed-line/output changes) and the pinning scripts were read end-to-end as reported above; live assertions run in the owner's environment (`verify:phase8` + `verify:mainship-preview` must stay green there) and the later `verify:fitting` covers the dark posture. PR-ready on `autopilot/20260703-064048`, main untouched. Next: the read surface + frontend + `verify:fitting`.

2026-07-04 — FITTING-P14 SLICE C2 — settled-SAFE ship-state rule 0114 (corrects the 0113 'home' literal; ship_not_home → ship_not_settled)

Request. Forward-only correction of the slice-C game rule, while everything is still dark: the strict `spatial_state = 'home'` literal was confirmed dead-on-arrival (NO shipped writer ever produces 'home' — commissions land at_location, OSN writers produce in_transit/in_space/at_location, destruction/repair leave NULL), so even a flag flip would strand the feature behind another migration. **Rationale (one line): intent preserved, literal fixed, precedent reused** — the rule's INTENT ("loadout never changes mid-transit/in-space/mid-combat") stands; only the accepted-state literal was wrong; the codebase's authoritative "settled and safe to act on" definition is reused. No flag touched, nothing activated — a design correction within the loop's authority.

The shipped gating, as read first (transcribed in FITTING_P14_RECON.local.md 56b): the scan/extract COMMANDS (0099:151–167 / 0104:124–140) gate IDENTICALLY to each other — no stricter-of-two choice was needed: `mainship_space_validate_context` must be ok, its validated state must be 'in_space' exactly, then `mainship_space_assert_cross_domain_exclusion` (no active legacy movement / coordinate-pointer mismatch / presence conflict) must be ok. Their 'in_space' state exists because scan/extract ARE open-space actions — transcribing that literal into fitting would contradict the recorded intent, so it deliberately does NOT transfer. The settled-SAFE STATE SET is the securing processors' (0100:231 / 0105:69): `spatial_state in ('home', 'at_location')`. **What ships: the processors' state set verbatim + the commands' companion machinery verbatim** — `validate_context` ok AND state in ('home', 'at_location'), then `cross_domain_exclusion` ok — so fitting is gated AT LEAST as strictly as the shipped activity commands. Every non-settled outcome (legacy NULL, in_space, in_transit, destroyed, incoherent context, busy in either movement domain) collapses to ONE truthful reject `ship_not_settled` (the 0099:159 "one truthful reason" idiom). Satisfiable today: commissioned ships sit at_location in the canonical coherent shape.

Work done — NEW `supabase/migrations/20260618000114_fitting_p14_settled_safe_rule.sql` (migration head moves 0113 → 0114; 0001–0113 unedited; 0113 stays as history):

- `fitting_execute_command` **re-created** (the 0044-style `create or replace` forward-only idiom) changing ONLY the step-6 game rule: the affected ship is resolved per action first (fit → the owner-checked target, `ship_not_owned` unchanged; unfit → the currently-fitted ship, rule skipped when no fitting row exists so the writer still answers `module_not_owned/not_fitted` truthfully), then ONE shared settled-SAFE check block runs. Dark-gate order, `request_id` validation, per-player lock, verbatim replay, action-shape validation, delegation to `fitting_apply`, and failure-writes-no-receipt semantics are byte-identical to 0113 (the only other diff: the declare block swaps `v_state` for `v_check_ship/v_val/v_excl`). **NO-DUPLICATION NOTE (explicit, per the review):** the settled-SAFE mechanism appears ONCE (resolve-then-check), so no shared-helper extraction is needed — and the membership check itself is one line.
- `fitting_command_client_envelope` **re-created** only because it embeds the renamed code + copy: `ship_not_settled` with the message "The ship must be settled at home or docked at a location to change its module loadout." (matching the existing copy tone); every other line identical. Repo grep confirms NO other site references `ship_not_home` (0113 itself is history; no frontend/verify script exists yet).
- **ACL re-asserted** for both re-created functions exactly as 0113 (revoke public/anon/ authenticated + grant service_role — `create or replace` preserves grants, but the shipped re-create precedents re-assert explicitly). `fitting_apply`, `module_fitting_receipts`, both wrappers, and every exploration/mining object are NOT touched.

Doc-sync (same step). `docs/SYSTEM_BOUNDARIES.md` §2 Fitting row: the Δ unsatisfiable-rule note is replaced by the settled-SAFE rule as now shipped (citing the 0099/0104 machinery + 0100/0105 state-set precedents), and the edges line now records the OSN-context-helper reads (`mainship_space_validate_context` + `mainship_space_assert_cross_domain_exclusion` — read-only, downward, the exact 0099/0104 reuse-never-reinvent posture; still acyclic, still nothing depends on Fitting).

State. `npm run build green` (no src/ change was made — confirmed). Migration head 0114; `module_fitting_enabled='false'` — the surface stays server-rejected at every layer; no flag flipped, no live DB write, no workflow touched. **DB-apply posture (honest, unchanged):** no `psql/docker/supabase CLI` in this sandbox — the re-created functions were diffed line-by-line against 0113 (single-rule change + declares + the two mapper lines) and the new rule block against 0099:151–167/0104:124–140 (machinery) and 0100:231/0105:69 (state set); live assertions run in the owner's environment and will be covered by the later `verify:fitting` dark-posture script. PR-ready on `autopilot/20260703-064048`, `main` untouched. Next: the adapter slice and/or the read surface, then frontend + `verify:fitting`.

2026-07-04 — FITTING-P14 SLICE C — the dark two-layer fit/unfit command 0113 (`module_fitting_receipts` + `fit_module_to_ship/unfit_module_from_ship` → ONE private `fitting_execute_command`)

Request. Implement slice C of Phase 14: ONE new forward-only migration with the player-scoped fitting-receipt ledger and the dark two-layer fit/unfit command, delegating every mutation to the 0112 writer. NO frontend, NO adapter change, NO verify script this slice. Idioms matched by re-reading the shipped sources end-to-end first: 0109 (the two-layer craft command this slice mirrors verbatim — receipts posture, gate order, lock-before-replay, trade-semantics replay, failure-writes-no-receipt, envelopes, relock), 0112 (the writer being wired), 0054/0055 (the exact `spatial_state` values + constraints).

Work done — NEW `supabase/migrations/20260618000113_fitting_p14_fit_command.sql` (migration head moves **0112 → 0113**; 0001-0112 unedited):

- `module_fitting_receipts` — Fitting-owned per-player idempotency ledger: **PK (player_id, request_id)** (the locked keying — the idempotency key IS the row identity; 0109 used a surrogate receipt_id + a UNIQUE on the same pair, same semantics), `action` check in ('fit','unfit'), the request fingerprint (`module_instance_id` FK cascade, `main_ship_id` FK cascade nullable — as-requested, NULL on unfit; the 0088/0109 order-safe-cascade lesson), and `result_json` — the writer's success envelope stored VERBATIM. RLS = the 0109 posture verbatim: owner-read select only, no write path. No extra index (the PK leads on `player_id` — the 0086/0109 comment idiom).
- **DECISION — ONE private command for BOTH actions:** `fitting_execute_command(p_player, p_action, p_module_instance_id, p_main_ship_id, p_request_id)` (service_role-only) handles 'fit' AND 'unfit' precisely so the receipts table keeps **ONE sole writer** — a fit-command and an unfit-command each inserting receipts would be TWO writers on one table. Order of operations mirrors 0109 exactly: **dark gate FIRST** (`module_fitting_enabled` via `cfg_bool`, `reject-before-any-read`, `feature_disabled`) → request_id validation (text, non-empty, ≤200) → **per-player advisory lock BEFORE the replay check** using the SAME ('module_fitting', `player`) key as `fitting_apply` (documented: `pg_advisory_xact_lock` is reentrant within a transaction, so the writer's nested acquisition is safe and the replay check is serialized with ALL fitting mutations — a same-request_id race resolves to one mutation + one verbatim replay) → **verbatim replay** (an existing (player, request_id) receipt returns its stored `result_json` + `idempotent_replay:true`; NO payload-conflict check — the 0089/0095/0109 trade semantics: a reused request_id replays the original result even if the call names a different action/module/ship) → action-shape validation ('fit' requires a ship, 'unfit' forbids one → `invalid_request`) → **the GAME RULE this layer owns** (below) → delegate to `fitting_apply` (NEVER touching `ship_module_fittings` directly — the sole-writer law; writer reasons `module_not_owned/ship_not_owned/already_fitted/not_fitted/insufficient_slots` pass through) → **only a SUCCESSFUL mutation writes a receipt** (failures write nothing — the 0109 law).
- **THE HOME-ONLY GAME RULE** (`ship_not_home`). The affected ship — `p_main_ship_id` on fit; the currently-fitted ship (read from `ship_module_fittings`, owner-scoped) on unfit — must have `spatial_state = 'home'`. RATIONALE (recorded per the locked spec): constrained state transitions — a loadout must never change mid-transit / in-space / mid-combat; expedition stats are frozen for the duration of an expedition; refitting happens at home before departure. Fail-closed (is distinct from 'home'): NULL (legacy) and every other state reject. **AS-SHIPPED HONESTY NOTE (for the human activation review):** grep of all migrations shows NO shipped writer ever sets `spatial_state = 'home'` — commissions insert ships `at_location` (0072/0077/0078/0080), OSN writers produce `in_transit/in_space/at_location`, destruction/repair leave NULL (0059) — so with current writers EVERY existing ship answers `ship_not_home` even once the flag flips. Implemented as the strict locked reading; relaxing to the 0100/0105 settled-SAFE set (in ('home', 'at_location')) or adding a 'home' writer is a forward-only HUMAN decision.
- **TWO thin authenticated wrappers** (0109 wrapper idiom; named per ROADMAP :89): `fit_module_to_ship(p_module_instance_id, p_main_ship_id, p_request_id)` and `unfit_module_from_ship(p_module_instance_id, p_request_id)` — each does auth resolution + the anti-probe dark-gate-first check exactly like `craft_module`, then calls the private command with its fixed action. **Adaptation (the no-duplication hard rule):** 0109 inlined its reason→ code/message map in its single wrapper; two wrappers would duplicate that block, so it is extracted ONCE as `fitting_command_client_envelope(jsonb)` (pure jsonb→jsonb; service_role-only surface) and both wrappers call it. Codes covered: `feature_disabled`, `not_authenticated`, `invalid_request`, `ship_not_home`, `module_not_owned`, `ship_not_owned`, `already_fitted` (+`main_ship_id` context), `not_fitted`, `insufficient_slots` (+{`used`, `cost`, `limit`} context), `unavailable` fallback, and the `idempotent_replay` marker on replays.
- **ACL (0109:265–273 verbatim posture):** private command + shared mapper revoked from public/anon/authenticated + granted to service_role; both wrappers revoked from public/anon + granted to authenticated (dark: every layer's gate rejects today).

Doc-sync (same step). `docs/SYSTEM_BOUNDARIES.md`: §1 gained the `module_fitting_receipts` row (**Fitting**; sole writer = `fitting_execute_command` via the two wrappers; one-command-for-both- actions rationale; DARK, no row can exist today) and the `ship_module_fittings` row's "called by NOTHING yet" became "called today ONLY by Fitting's own command `fitting_execute_command` (0113)"; the §2 Fitting row now records the full command layer (wrappers → private command → `fitting_apply`), the home-only rule with its rationale and the as-shipped **AS** note, the new DOWNWARD reads (Reference/Config flag · Main Ship `spatial_state`), and the expanded forbidden column (no second receipt writer, no client exposure of command/writer/mapper, no fit/unfit while the gate is off). Still nothing depends on Fitting.

State. `npm run build green` (no `src/` change was made — confirmed). Migration head **0113**; `module_fitting_enabled='false'` — the entire command surface is server-rejected at every layer (both wrappers gate, the private command gates first, the writer stays service_role-only); no flag flipped, no live DB write, no workflow touched. **DB-apply posture (honest, unchanged from P11–P13):** no `psql/docker/supabase` CLI in this sandbox — the migration was hand-verified line-by-line against 0109 (order,

replay, receipts, ACL), 0112 (delegation contract), and 0054/0055 (state values); live assertions run in the owner's environment and will be covered by the later `verify:fitting` dark-posture script. PR-ready on `autopilot/20260703-064048`, main untouched. Next: the adapter slice (modules feeding `calculate_expedition_stats` under the slot cap) and/or the read surface, then frontend + `verify:fitting`.

2026-07-04 — FITTING-P14 SLICE B — `ship_module_fittings` + the single Fitting writer 0112 (`fitting_apply`; FIT and UNFIT through THE ONE writer)

Request. Implement slice B of Phase 14: ONE new forward-only migration with the fitting-state table and THE ONE Fitting writer. NO RPC wrapper, NO receipts table, NO adapter change, NO frontend, NO verify script this slice. Idioms matched by re-reading the shipped sources first: 0108 (module_instances schema + mint writer — the slice this one mirrors), 0109 (the per-player advisory-lock key derivation), 0043 (main-ship ownership shape + module_slots).

Work done — NEW `supabase/migrations/20260618000112_fitting_p14_fittings_schema.sql` (migration head moves 0111 → 0112; 0001-0111 unedited):

- `ship_module_fittings` — Fitting-owned junction state: `module_instance_id uuid PRIMARY KEY FK → module_instances` on delete cascade (**the PK IS the invariant**: one module instance is fitted to at most one ship, ever — a schema fact, not writer discipline), `main_ship_id FK → main_ship_instances` on delete cascade, `player_id FK → auth.users` on delete cascade, `fitted_at timestampz`, plus a (`main_ship_id`) index (the capacity sum + the future adapter read). RLS posture = 0108 verbatim: own-row SELECT only (`player_id = auth.uid()`), select granted to authenticated, NO write policy/grant — no client write path exists.
- `fitting_apply(p_player uuid, p_module_instance_id uuid, p_main_ship_id uuid)` returns `jsonb` — THE sole writer of `ship_module_fittings` (SECURITY DEFINER; service_role-only via the 0108 relock idiom). **DECISION — fit/unfit in ONE writer:** `p_main_ship_id NOT NULL = FIT`, `NULL = UNFIT`; one sole writer per table covers ALL mutations of that table (insert AND delete) — two writer functions would be two writers. The writer enforces the STRUCTURAL invariants itself so no future caller can violate them, in order: (1) per-player `pg_advisory_xact_lock(hashtext('module_fitting'), hashtext(player))` FIRST (the exact 0109 key-derivation idiom — serializes all of a player's fitting mutations; since every fitting on a ship belongs to the ship's owner, per-player IS per-ship-fit-set, so the capacity read cannot be raced); (2) module instance exists AND `module_instances.player_id = p_player` (`module_not_owned` — another player's instance answers like a nonexistent one); on FIT: (3) ship exists AND owned (`ship_not_owned`; also fixes owner-consistency — `row.player = module owner = ship owner`); (4) `already_fitted` reject NAMING the current ship — an already-fitted module is never silently re-homed (explicit unfit first; the PK backstops); (5) the CAPACITY HARD CAP — $\sum \text{module_types.slot_cost}$ currently fitted to the ship + the new module's `slot\_cost` ≤ `main_ship_instances.module_slots`, else `insufficient_slots + {used, cost, limit}` — a hard rejection mirroring 0044:112-115, NEVER a clamp; (6) the one insert. UNFIT of a non-fitted module → distinct `not_fitted` (idempotency ENVELOPES are the slice-C command's receipt-replay job, not the writer's). Envelopes are the 0104/0109 private-writer family (`{ok, reason, ...}` — the slice-C wrapper maps reasons to client codes); validation failures write nothing. **GAME-RULE checks deliberately live in the slice-C command layer, NOT here:** the `module_fitting_enabled` dark gate, the ship-must-be-home spatial rule, and receipt-keyed idempotency — this writer owns only table invariants and is unreachable by clients (service_role-only) until that gated command exists, so the feature stays fully dark.
- **ACL (0108:108-113 relock idiom verbatim):** execute revoked from public/anon/authenticated, granted to service_role only. No existing grant touched.

Doc-sync (same step). `docs/SYSTEM_BOUNDARIES.md`: §1 gained the `ship_module_fittings` row (**Fitting**, owner; sole writer = `fitting_apply`, service_role-only, FIT+UNFIT through the one writer, called by nothing yet); §2 gained the new **Fitting** leaf-system row (owns `ship_module_fittings`; the full writer semantics; edges all DOWNWARD — Fitting → Modules (read `module_instances`) · Main Ship (read ownership + `module_slots`) · Reference/Config (read `module_types.slot_cost`); no system depends on Fitting yet — the Phase-14 adapter slice will later add the Expedition-stats → Fitting downward READ edge; forbidden column bans a second mutation path, clamping, silent re-homing, client exposure, and gating game rules in the writer).

State. `npm run build green` (no `src/` change was made — confirmed). Migration head 0112; `module_fitting_enabled='false'` — still fully dark: the ONE writer is service_role-only with ZERO callers (dead-until-slice-C by design, documented as such), the table has no client write path, and no flag was flipped, no live DB write, no workflow touched. **DB-apply posture (honest, unchanged from P11-P13):** no `psql/docker/supabase` CLI in this sandbox — the migration was hand-verified line-by-line against the shipped idioms it copies (0108 table+RLS+ACL posture, 0109 advisory-lock key derivation, 0043 ownership reads, 0044 hard-cap semantics); live assertions run in the owner's environment and will be covered by the later `verify:fitting` dark-posture script. PR-ready on `autopilot/20260703-064048`, main untouched. Next: slice C (the dark two-layer fit/unfit command — `module_fitting_enabled` gate, player-scoped receipts, ship-must-be-home rule, delegating to this writer).

2026-07-04 — FITTING-P14 SLICE A — locked design decisions + dark flag/stats-catalog migration 0111 (module_fitting_enabled + module_types.slot_cost/stats_json)

Request. Begin Phase 14 "Module fitting" (ROADMAP :89 — "fit_module_to_ship | server-validated; feeds stats") with slice A: record the planner-approved LOCKED design decisions, then ONE new forward-only migration seeding the dark flag + the module stats/slot-cost catalog wiring. NO fittings table, NO RPC, NO adapter change, NO frontend, NO verify script this slice. Recon: FITTING_P14_RECON.local.md (scope locked 2026-07-04). Template: the 0107 slice-A idiom.

LOCKED DESIGN DECISIONS (planner-approved 2026-07-04):

1. **SYSTEM SHAPE** — Phase 14 creates a NEW leaf system **Fitting** per ROADMAP law 5 ("Fitting=modules"); fitting state will live in a NEW Fitting-owned junction table `ship_module_fittings` (arrives slice B — NOT this slice) with its own sole writer, never a second writer or new columns on `module_instances`; Fitting depends one-directionally DOWNWARD on Modules (read instances), Main Ship (read `module_slots`), and Reference/Config.
2. **CAPACITY/TRADEOFF MODEL** — mirrors the proven support-craft mechanism in 0044: each module type has an integer `slot_cost ≥ 1`; the adapter (extended in a later slice via `create or replace` in a new migration) will hard-REJECT when $\sum \text{slot_cost of fitted modules} > \text{main_ship_instances.module_slots}$ (exception, never a clamp — the 0044:112–115 idiom), and `slot_type`-based tradeoff rules (pirate_attention / speed penalty) will apply exactly like 0044's role-based rules — so module power is capacity-limited with tradeoffs, never a raw sum.
3. **STATS ENCODING** — reuse the `support_craft_types.base_stats_json` idiom: add a `stats_json jsonb not null default '{}'` column to `module_types`, using the SAME physical stat keys the adapter already reads (attack/defense/repair/cargo/scan/mining/evasion) plus one new key `speed_mult_bonus` (numeric fraction of hull base speed, applied before penalties — the engine archetype's positive effect; the adapter clamps total speed exactly as today: `round(greatest(0.2, ...), 3)` — 0044:117–118).
4. **FLAG** — `module_fitting_enabled` seeded 'false', the exact 0097/0102/0107 idiom; every Phase 14 RPC must check it FIRST and reject-before-any-read; this migration flips nothing.

Work done — NEW `supabase/migrations/20260618000111_fitting_p14_config_and_stats.sql` (migration head moves 0110 → 0111; 0001–0110 unedited):

- (a) `game_config` seed `module_fitting_enabled='false'` (on conflict (key) do nothing, the exact 0097/0102/0107 dark-gate idiom + description stating the reject-before-any-read law).
- (b) alter table `module_types` add column `slot_cost integer not null default 1 check (slot_cost >= 1)`, add column `stats_json jsonb not null default '{}'::jsonb` — Reference/Config CATALOG data exactly like `support_craft_types.capacity_cost/base_stats_json` (0042). Posture unchanged: the existing 0107 public-read policy + grants cover new columns (the 0075/0076 add-column precedent); still no client write path; no function created → no execute relock (0054 precedent). First code consumer arrives with the Phase 14 adapter slice — nothing reads them today.
- (c) Write-once per-id UPDATES seeding the four shipped archetypes, guarded on `stats_json = '{}'::jsonb` (the update analogue of the seeds' on conflict do nothing — a re-run or later owner rebalance is never clobbered). Magnitudes were read against the 0042 `base_stats_json` band (missile_boat cap 3 → attack 12 · cargo_drone cap 2 → cargo 20 · survey_drone cap 2 → scan 8 · decoy_drone cap 1 → evasion 6) so a full 3-slot fit is comparable to a similarly-sized support loadout: autocannon_battery (weapon) slot 1 → {"attack":10} · vector_thruster_kit (engine) slot 1 → {"evasion":3,"speed_mult_bonus":0.1} · expanded_cargo_lattice (cargo) **slot 2** (deliberately multi-slot so the $\sum \text{slot_cost cap math}$ is exercised) → {"cargo":25} · deep_scan_sensor_array (sensor) slot 1 → {"scan":8}.

Doc-sync (same step). `docs/SYSTEM_BOUNDARIES.md`: the §1 `module_types` row now records `slot_cost + stats_json` (still migration-seeded only, NO runtime writer; consumer = the Phase 14 fitting adapter, later slice). Deliberately NO Fitting system row yet — `ship_module_fittings` does not exist until slice B, and a doc must never describe state that isn't real.

State. `npm run build green` (no src/ change was made — confirmed). Migration head 0111; `module_fitting_enabled='false'` and `module_crafting_enabled='false'` — nothing client-writable was added (one dark flag + two inert catalog columns + write-once seeds; no RPC, no writer, no reader). No flag flipped, no live DB write, no workflow touched. **DB-apply posture (honest, unchanged from P11–P13):** no `psql/docker/supabase CLI` in this sandbox — the migration was hand-verified line-by-line against the idioms it copies (0107 flag seed, 0042 catalog stats shape, 0075/0076 add-column posture); live assertions run in the owner's environment and will be covered by the later `verify:fitting` dark-posture script. PR-ready on `autopilot/20260703-064048`, `main` untouched. Next: slice B (`ship_module_fittings` + the Fitting sole writer).

2026-07-04 — MODULES-P13 SLICE F (final) — verify:modules dark-posture script. Phase 13 Module crafting — dark implementation complete (slices A–F)

Request. Implement slice F, the last Phase 13 slice: the dark-posture verify script + its package.json entry, mirroring the exploration/mining verifier twins (read end-to-end first: verify-exploration.mjs, verify-mining.mjs, scripts/lib/verify-harness.mjs, scripts/lib/verifier-teardown.mjs). Touches ONLY scripts/verify-modules.mjs, package.json, this file, and the recon scratch file. No migrations (head stays **0110**), no CI/workflow edits, no flags.

Flag-handling mechanism — the twins', stated and followed exactly. verify-mining.mjs:16-20 records the mechanism verbatim: the twins **NEVER write game_config and NEVER flip their flag** — they exercise NO lit path at all (the set_game_config flip in verify-mainship-send.mjs is the explicitly-NOT-copied alternative; the self-rolling-back mechanism exists only in the separate trade-economy-bootstrap-proof psql/CI harness, which is workflow-wired and out of this loop's scope). So verify-modules.mjs proves the DARK contracts only, and the requested lit-path behaviors are recorded in the HUMAN ACTIVATION CHECKLIST below — run on a DEV database by the owner, never by this script. Teardown guarantee: the shared teardownVerifier deletes the throwaway user (the 0108/0109 player FKs cascade any of its rows away); no flag entry is passed because the script touches NO flag — nothing to restore, module_crafting_enabled stays exactly as found ('false').

Work done:

- **NEW** scripts/verify-modules.mjs — imports the shared harness from day one (Abort/createReporter/createUserFactory/resolveEnv from scripts/lib/verify-harness.mjs
`<font name="Sym"> </font> teardownVerifier from scripts/lib/verifier-teardown.mjs)` — ZERO inline harness copies (the harness header's law). Same posture as the twins: dark contracts only; NEVER writes game_config; service key OPTIONAL (teardown only); one throwaway signup. Asserts, in the twins' order/idioms: (1) **dark rejection** — `craft_module` → {ok:false, code:'feature_disabled'} (0109 gates before any validation — anti-probe) and `get_my_module_instances` → {ok:false, reason:'module_crafting_disabled'} (0110), both authenticated; (2) **catalog seeds (0107, exact contract)** — `module_types` = the 4 seeded archetypes with their slot types; `module_recipe_ingredients` = exactly 12 rows, all qty > 0, per-type ingredient maps equal to the seed verbatim, and every ingredient id present in `item_types` (the client-checkable form of FK validity). NOTE the deliberate inversion vs mining's "no field leak": these catalogs are PUBLIC-READ by design (the item_types posture), so reading them back exactly IS the posture assertion; (3) **player-state RLS + no client write path** — `module_instances` + `module_craft_receipts` own-row RLS (fresh user sees 0 rows) AND inserts denied to the authenticated client on all four Modules/Production tables (both state tables + both catalogs); (4) **internal surfaces locked** — `production_craft_module` + `modules_mint_instance` denied to the authenticated client; both public RPCs denied to anon; (5) **config presence (read-only)** — `module_crafting_enabled` = false, via the same jsonb-storage-tolerant comparison.
- package.json — "verify:modules": "node scripts/verify-modules.mjs" added in the verify cluster, directly after verify:mining, same command shape.
- **CI note:** grep confirms the exploration/mining verifiers are wired into NO workflow file — there is nothing to mirror, and no workflow was created or modified (dispatching/enabling workflows is outside this loop). Wiring `verify:modules` into CI, if desired, is a human / PR-review step.
- **Verify posture run honestly:** `node --check scripts/verify-modules.mjs` parses clean; `npm run build` green. `node scripts/verify-modules.mjs` in this sandbox aborts at the throwaway SIGNUP step with the environmental TLS failure ("signup failed: fetch failed") — `node scripts/verify-mining.mjs` aborts at the IDENTICAL point in the same run, so this is the known environmental-fail-only posture (the Phase 12 slice-G precedent), and reaching that identical abort point proves the harness wiring. The assertions themselves run against a real DB in the owner's environment.
- docs/SYSTEM_BOUNDARIES.md needs NO change this slice (checked the twins' verify slices: the Phase 12 slice-G entry recorded the same) — no table, writer, or cross-system edge (a read-only verifier script + one npm alias).

Phase 13 Module crafting — dark implementation complete (slices A–F) — closing summary

- **Migrations** 0107-0110 (head **0106** → **0110**; all forward-only; 0001-0106 never edited): 0107 config/flag + catalogs (`module_crafting_enabled='false'` + `module_types` + `module_recipe_ingredients` + 4 seeded archetypes whose recipes reuse EXISTING `item_types` rows) · 0108 instances schema + the ONE mint writer (`module_instances` with the `mint_key` idempotency spine + `modules_mint_instance`, service_role-only) · 0109 the craft command (`module_craft_receipts` + `craft_module` wrapper → private `production_craft_module`; dark-gate-first, per-player advisory lock, trade-semantics verbatim replay, items-only cost via `inventory_spend`, one craft = one instance via the namespaced `craft: mint` key) · 0110 the read surface (`get_my_module_instances`; the 0101/0106 idiom; no catalog RPC — public-read catalogs are direct client selects).
- **Frontend:** dark `src/features/modules/` (types/api/panel — the twins' structure; server-driven visibility, fails closed to null; MarketPanel per-row claims; direct catalog + own-inventory selects) wired beside MiningPanel in GalaxyMapScreen.tsx.
- **Verify:** scripts/verify-modules.mjs + `npm run verify:modules` (dark posture only, shared harness, never flips flags).
- **Design decisions (owner-directed, slice A):** Modules leaf system + Production-owned craft command; instant idempotent craft (player-scoped receipts; `build_orders` integration deferred with the mint-helper retirement note); normalized items-only recipes; one craft = one instance; `module_crafting_enabled` flag.
- **Ownership/laws:** SYSTEM_BOUNDARIES \$1 rows (`module_types/module_recipe_ingredients`, `module_instances`, `module_craft_receipts`) + \$2 Modules and Production rows — every doc synced in the SAME step as its fact; edges all

DOWNWARD/acyclic (Production → Inventory · Modules · Reference/Config); sole writers: catalogs none (migration-seeded), `module_instances = modules_mint_instance` (called only by 0109), `module_craft_receipts = production_craft_module`.

- **HUMAN ACTIVATION CHECKLIST (the owner's gate — never this loop):** (1) apply migrations 0107–0110 to the target DB; (2) run `npm run verify:modules` there — expect ALL dark-posture checks green; (3) optionally flip `module_crafting_enabled='true'` on a DEV database and run the lit path there: seed a test player's inventory (e.g. via secured mining/exploration deposits or a dev grant), then `craft_module` with sufficient balances → expect success AND (a) exactly the recipe quantities spent from `player_inventory` with matching negative `inventory_ledger` rows, (b) exactly ONE `module_instances` row minted with the namespaced `craft:<player>:<request_id>` key, (c) exactly one `module_craft_receipts` row; REPLAY the same (player, request_id) → same `instance_id + idempotent_replay:true`, and provably NO double-spend/double-mint; a shortfall craft → `insufficient_items` with {item_id, have, need}, nothing spent, no receipt; `unknown_module` and `no_recipe` codes fire; `modules_mint_instance` twice with the same key (service_role) → one row; `get_my_module_instances` returns the crafted instance newest-first for the owner and NOT for a second test player; then flip the flag back and decide production activation separately. The loop ships everything server-rejected; activation is exclusively the human's.

State. `npm run build` green; `node --check` clean on the new script. Migration head **0110**; `module_crafting_enabled='false'` everywhere; no flag flipped, no live DB write, no workflow touched. **Phase 13 Module crafting is implemented DARK end-to-end and PR-ready on** `autopilot/20260703-064048` — SAFE FOR HUMAN MERGE REVIEW; main untouched.

2026-07-04 — MODULES-P13 SLICE E — dark frontend `src/features/modules/` (catalog + craft + instances panel; renders nothing while the server says dark). Frontend only — no migration

Request. Implement slice E: the dark module-crafting frontend mirroring the post-cleanup exploration/mining twins exactly (read end-to-end first: both panels + `api/types` modules, the shared `src/lib/useActivityPanelGuards.ts`, the `GalaxyMapScreen` mounting, `MarketPanel`'s per-row state shapes, and the `mainshipApi.ts` direct-select convention). NO migration (head stays **0110**), NO verify script, no config.

Work done — NEW `src/features/modules/` (the twins' structure, adapted where crafting differs):

- `modulesTypes.ts` — pure types + copy (the `miningTypes.ts` idiom): `ModuleInstance` (the 0110 row), `GetMyModuleInstancesResult`, `CraftModuleResult` (0109 wrapper shape; `item_id/have/need` are REAL server data on the `insufficient_items` code), the public catalog row types, and the craft error-copy map (`module_crafting_disabled` read reason is handled by the fail-closed render, not copy; command codes covered: `feature_disabled`, `invalid_request`, `unknown_module`, `no_recipe`, `insufficient_items`, `not_authenticated`, `unavailable`) + `craftModuleErrorMessage`.
- `modulesApi.ts` — thin `supabase.rpc` wrappers for `craft_module` + `get_my_module_instances` (identical envelope-handling idiom: transport error → normalized failure, never a throw into the render path), PLUS two direct selects per the shipped conventions: `fetchModuleCatalog()` reads the PUBLIC-READ `module_types` + `module_recipe_ingredients` (0107) by direct table select — the `mainshipApi.ts` hull-types convention; deliberately NO catalog RPC exists (0110 header) — and `fetchMyItemBalances()` reads the caller's own `player_inventory` rows through the EXISTING 0039 own-row grant (the existing Inventory read path; no new server surface, no new cross-system edge).
- `ModulesPanel.tsx` — server-driven visibility: reads the instances on mount/lifecycle change and **fails closed to null on ANY non-ok envelope** — the Exploration/Mining twins' posture via `isServerLit` (the hook documents this server-lit stance as distinct from `MarketPanel`'s shell-with-unavailableNote, which is reserved for client-flag-mounted shells — the twins ARE the match here; while the server returns `module_crafting_disabled` the panel renders nothing, so production is unchanged; the panel never pretends the feature is on). Catalog + balances are fetched only after the server lights the surface.
Per-module-type claim keys (`tryClaim(entry.id)` — the `MarketPanel` per-row granularity, with its `pending/rowNote`: `Record<string, ...>` state shapes) since the catalog lists multiple craftable types; fresh `crypto.randomUUID()` request id per submit (the twins' idiom; the server dedups on (player_id, request_id)); craft buttons disable while in flight and on a client-side shortfall preview (server stays authoritative — `insufficient_items`); ingredient lines show `item × qty` (have N) with shortfalls flagged rose; the `insufficient_items` failure note appends the server's real `item_id`: `have/need` (the mining cooldown-suffix idiom); crafted-instances list (name, slot badge, timestamp), newest first. **Crafting is NON-SPATIAL** (player-scoped, 0109) — no ship/settled precondition, so unlike the twins the panel takes only `lifecycleKey` (no ship props; a deliberate, documented deviation). Sky styling vs violet/amber; positioned bottom-left BESIDE `MiningPanel` (`left-[33.5rem]` — the w-64 row continues; all three activity panels are server-lit, so overlap only ever involves lit surfaces).
- **Wiring:** `GalaxyMapScreen.tsx` — `ModulesPanel` imported and rendered directly adjacent to `MiningPanel`, same container, same comment convention, same `lifecycleKey` expression.

Doc-sync note. `docs/SYSTEM_BOUNDARIES.md` needs NO change (the MINING-P12 SLICE F precedent, stated explicitly per the request): frontend-only — no table, no writer, no cross-system edge; the server contracts the new files mirror (0107/0109/0110, the 0039 grant) are unchanged.

State. `npm run build green` (`tsc -b + vite`, exit 0). `npm run lint`: the 4 touched files (`modulesTypes.ts`, `modulesApi.ts`, `ModulesPanel.tsx`, `GalaxyMapScreen.tsx`) lint CLEAN (targeted eslint exit 0, incl. exhaustive-deps via the stable-ref dep); full-repo lint still FAILS with exactly the same 14 pre-existing out-of-scope errors recorded in MINING-CLEANUP SLICE 1 (`MainShipMarker.tsx`, `SpaceRouteLine.tsx`, `useSpaceMoveCommand.ts`, tests harnesses/spec — no new problems). Migration head stays **0110** (no migrations, no flags); everything still dark and server-rejected — the panel is wired but renders nothing while `module_crafting_enabled='false'`. PR-ready on `autopilot/20260703-064048`, `main` untouched. Next: slice F (`scripts/verify-modules.mjs` + the `verify:modules` entry).

2026-07-04 — MODULES-P13 SLICE D — the dark read surface 0110 (`get_my_module_instances()`). Server side of Phase 13 complete, fully dark

Request. Implement slice D: ONE new forward-only migration with the read surface, mirroring the exploration/mining read surfaces (0101/0106) exactly — re-read end-to-end first. NO frontend, NO verify script this slice.

Work done — NEW `supabase/migrations/20260618000110_modules_p13_read_surface.sql` (migration head moves **0109** → **0110**; `0001-0109` unedited):

- `get_my_module_instances()` — the 0101/0106 body step-for-step (line-level sources: envelope + auth + dark-gate order **0101:36–44** / **0106:38–46**; `jsonb_agg` row shape + desc ordering + coalesce-to-[] **0101:49–63** / **0106:51–65**; `stable/security definer/ set search_path = public posture 0101:26–29): auth.uid() → not_authenticated envelope; then the dark gate BEFORE any instance read — {ok:false, reason:'module_crafting_disabled'} (the 0101 exploration_disabled / 0106 mining_disabled envelope shape), identical regardless of caller state (no probing while dark); then the caller's OWN module_instances joined to their module_types catalog identity. Per row: instance_id, module_type_id, name, slot_type, created_at — newest first (created_at desc); response {ok:true, instances:[...]} mirroring {ok:true, discoveries/extractions:[...]}.`
- **Catalog surface decision — the precedent points AGAINST a catalog RPC, and was followed:** 0101/0106 exist because `exploration_sites/mining_fields` are HIDDEN (RLS, no client policy — reveal only through the player's own rows). The module catalog/recipe tables are the opposite posture by design (0107): public-read Reference/Config catalogs exactly like `item_types` (0039:23–25) / `support_craft_types` (0042:32–36) / `trade_goods`, which the client reads by DIRECT table select (the shipped convention — e.g. the hull-type selects in `src/features/map/mainshipApi.ts`). A `get_module_catalog` RPC would duplicate an already-public surface — NOT added.
- **No inventory-balance join:** no shipped read surface joins another system's balances (`inventory_get_balance` is an internal service_role-only leaf, 0039:156). The surface stays dumb; the client reads its own `player_inventory` through the existing Inventory read path (the 0039:50–52 own-row select policy + grant). No new cross-system read edge without precedent.
- **ACL verbatim from 0101:69–70 / 0106:71–72:** execute revoked from public/anon, granted to authenticated only — and dark today: the gate rejects every call while `module_crafting_enabled='false'`.

Doc-sync (same step). `docs/SYSTEM_BOUNDARIES.md`: the §2 Modules row gained `get_my_module_instances()` with its dark-gate semantics + the two recorded non-additions (no catalog RPC, no balance join, with the precedent reasons). The §1 matrix is UNCHANGED — mirroring the 0101/0106 precedent exactly: read surfaces add no writer and are recorded in the §2 system row, not the matrix (the mining slice-E entry did the same).

State. `npm run build green`. Migration head **0110**. **The server side of Phase 13 Module crafting is COMPLETE (slices A–D) and fully dark end-to-end:** the craft command (both layers) and the read surface all server-reject while `module_crafting_enabled='false'`; the mint writer is service_role-only; the catalogs are inert public-read reference data. No flag flipped, no live DB write, no workflow touched. **DB-apply posture (honest, unchanged from slices A–C):** no `psql/docker/supabase CLI` in this sandbox and `npm run fetch (UNABLE_TO_VERIFY_LEAF_SIGNATURE)` — the migration was hand-verified line-by-line against 0101/0106 at the sources cited above; live assertions run in the owner's environment and will be covered by the slice-F `verify:modules` dark-posture script. PR-ready on `autopilot/20260703-064048`, `main` untouched. Next: slice E (dark frontend `src/features/modules/`, consuming `useActivityPanelGuards`), then slice F (`verify:modules`).

2026-07-04 — MODULES-P13 SLICE C — the dark craft command 0109 (`module_craft_receipts` + `craft_module` → `private production_craft_module`)

Request. Implement slice C of Phase 13: ONE new forward-only migration with the player-scoped craft-receipt ledger and the two-layer craft command. NO read surface, NO frontend this slice. Idioms matched by re-reading the shipped sources end-to-end first: `0099` (scan command) + `0104` (extract command) for the two-layer/envelope/ACL shape, `0089` (`market_buy`) + `0095` (`market_claim_relief`) for the trade replay + insufficient-balance envelope, `0086/0094` for the receipts-table RLS posture, `0078/0080` for the per-player advisory lock.

Work done — NEW supabase/migrations/20260618000109_modules_p13_craft_command.sql (migration head moves **0108** → **0109**; 0001–0108 unedited):

- **module_craft_receipts** — Production-owned per-player idempotency ledger: `receipt_id` uuid pk, `player_id` (the 0108 `auth.users` on delete cascade FK shape), `request_id` text not null, `module_type_id` FK → `module_types`, `instance_id` FK → `module_instances` (on delete cascade — the instance only ever disappears via the `auth.users` cascade today; cascading the receipt keeps account deletion order-safe across the multi-path cascade graph, the 0088 child-FK lesson), `created_at`, **unique (player_id, request_id)**. RLS posture copied from the player-scoped receipts precedent `trade_relief_claims` (**0094:24–43**): owner-read select policy + grant select to authenticated, NO write policy/grant. No extra index — the unique index leads on `player_id` and covers idempotency probes + owner lookups (the 0086:53–55 comment idiom).
- **craft_module(p_request_id text, p_module_type text)** (authenticated wrapper — the 0099:221–300 wrapper idiom: `auth check` → anti-probe flag gate **FIRST** → `delegate` → `reason` → `code/message` map; the `insufficient_items` failure passes its `{item_id, have, need}` context through, the 0104 `retry_after_seconds` pass-through idiom) → `private production_craft_module(p_player, p_module_type, p_request_id)` (`service_role`):
 1. **Dark gate FIRST** (0107 law; **0099:108–113**): `module_crafting_enabled` false → `feature_disabled` before ANY other read.
 2. `request_id` validation — TEXT per the locked signature (the shipped receipt columns are uuid; text is validated non-empty + length-capped at 200 since it lacks uuid's intrinsic bound).
 3. **Per-player advisory lock BEFORE the replay check** — `pg_advisory_xact_lock(hashtext('module_craft'), hashtext(player))`, the shipped commission idiom (**0078:43/79**): the player-scoped analogue of `market_buy`'s per-ship lock (0089:104–106) and relief's wallet FOR UPDATE (0095:53–57), both taken before their idempotency checks for the same race-safety reason (a same-`request_id` race resolves to one craft + one verbatim replay; the pre-check→spend window can't be raced by another craft of the same player).
 4. **REPLAY — matched to the TRADE receipts semantics (0089:108–116 / 0095:60–66), stated explicitly**: an existing (player, `request_id`) receipt returns the ORIGINAL success envelope rebuilt verbatim from the receipt row, flagged `idempotent_replay` — **NO payload-conflict check** (a same-key-different-module_type replay returns the original receipt's data, exactly as `market_buy` replays a same-key-different-good call). The `request_id_payload_conflict` hash check (0099:140–148) belongs to the ship-scoped space receipts, which this player-scoped command does not use.
 5. Catalog validation: `unknown_module` (bad id) vs `no_recipe` (catalog row with zero `module_recipe_ingredients` rows — a distinct truthful reason so a seed gap is diagnosable).
 6. **Ingredient pre-check** via `inventory_get_balance` — shortfall returns `{ok:false, reason:'insufficient_items', item_id, have, need}` (the **0089:150–153** `insufficient_credits` + context shape) WITHOUT spending anything.
 7. **One transaction**: loop the recipe rows → `inventory_spend(player, item, qty)` each (its exceptions — 0039:113–121 → roll back everything; a failed craft writes NO receipt, the 0099/0104 law) → mint exactly ONE instance via `modules_mint_instance(player, module_type, 'craft:'||player||':'||request_id)` (the namespaced key per 0108's producer contract) → insert the receipt → success envelope with `instance_id/receipt_id/module_type_id/crafted_at`. Crafting never touches `player_inventory/inventory_ledger/module_instances` directly — only the two leaf functions. This is `inventory_spend`'s FIRST live caller.
- **ACL (0099:302–311 / 0104:291–299 verbatim)**: private writer revoked from public/anon/authenticated + granted to `service_role`; wrapper revoked from public/anon + granted to authenticated (dark: both its gate and the writer's first check reject today).

Doc-sync (same step). `docs/SYSTEM_BOUNDARIES.md`: §1 gained the `module_craft_receipts` row (**Production**, owner; sole writer = `production_craft_module` via the `craft_module` wrapper; DARK) and the `module_instances` row's "NOTHING calls it yet" became "called today ONLY by Production's craft command (0109)"; §2 Production row gained `module_craft_receipts` in its owns column, the full `craft_module` semantics in its functions column (dark-gated, idempotent by player+`request_id` with verbatim replay, items-only cost, downward `inventory_spend` + `modules_mint_instance` fan-out, one craft = one instance), and the direct-write bans in its forbidden column; §2 Modules row's "will belong to Production" note went present-tense ("SHIPPED as `craft_module` (0109)") and its "NOTHING calls it yet" was replaced by the caller fact. New edges all DOWNWARD (Production → Inventory · Modules · Reference/Config) — acyclic, no second writer anywhere.

State. `npm run build` green. Migration head **0109**; still fully dark — the wrapper and writer both server-reject while `module_crafting_enabled='false'`; no flag flipped, no live DB write, no workflow touched. **DB-apply posture (honest, unchanged from slices A/B)**: no `psql/docker/supabase` CLI in this sandbox and `npx` cannot fetch (`UNABLE_TO_VERIFY_LEAF_SIGNATURE`) — the migration was hand-verified line-by-line against the named idiom sources above; live assertions run in the owner's environment and will be covered by the slice-G `verify:modules` dark-posture script. PR-ready on `autopilot/20260703-064048`, main untouched. Next: slice D (the read surface, e.g. `get_my_module_instances()` — the 0101/0106 idiom).

2026-07-04 — MODULES-P13 SLICE B — `module_instances` schema + the single Modules mint writer 0108 (`modules_mint_instance`; idempotent by `mint_key`)

Request. Implement slice B of Phase 13: ONE new forward-only migration with the `module_instances` table and the ONE Modules mint writer. NO craft command, NO receipts table, NO read surface, NO frontend this slice. Idioms matched by re-reading the shipped sources first: 0098 (exploration_discoveries) + 0103 (mining_extractions) for the player-state schema/RLS posture, 0039 (Inventory) for the SECURITY DEFINER internal-writer + idempotency-key pattern, and 0104:291-299 for the function-ACL relock wording.

Work done — NEW `supabase/migrations/20260618000108_modules_p13_instances_schema.sql` (migration head moves 0107 → 0108; 0001-0107 unedited):

- `module_instances` — `id` `uuid` primary key default `gen_random_uuid()`; `player_id` `uuid` not null references `auth.users` (`id`) on delete cascade (the exact 0098/0103 player-FK shape); `module_type_id` `text` not null references `module_types` (`id`); `mint_key` `text` not null unique — the idempotency spine; `created_at` `timestamptz` not null default `now()`; plus the (`player_id`, `created_at` desc) player index (0098/0103 idiom). Instances are INDIVIDUAL rows, never counts (the Phase-13 law) — no quantity column by design. **NO fitting columns** (`fitted_ship_id`/slots/stats are Phase 14, forward-only).
- **RLS posture copied from the P11/P12 player-state tables exactly** (verified, not assumed — both 0098 `exploration_discoveries` and 0103 `mining_extractions` DO expose an owner-select policy): RLS enabled + `module_instances_select_own` (`player_id = auth.uid()`) + `grant select to authenticated`; NO insert/update/delete policy, NO write grant — no client write path exists.
- `modules_mint_instance`(`p_player` `uuid`, `p_module_type` `text`, `p_key` `text`) returns `uuid` — THE ONE writer of `module_instances`: `plpgsql` SECURITY DEFINER, set `search_path = public`; exception-style errors matching Inventory's internal-leaf idiom (`raise exception` on missing key / unknown module type — not a player envelope RPC); then `insert ... on conflict (mint_key) do nothing`, and on conflict returns the EXISTING instance id for that key — true idempotent replay mirroring `inventory_deposit`(`p_key`)'s ledger-insert-is-the-guard semantics (0039:85-90): the same key can NEVER mint twice. Key namespacing is the producer's contract (the slice-C craft command derives keys from its own player-scoped receipts). Header states the **sole-writer law**: every future producer — the Phase-13 craft command AND any future `build_orders` queue completion (the recorded M4.5 retirement path) — must mint through this function and nothing else.
- **ACL (0099/0104 relock idiom verbatim)**: execute revoked from public/anon/authenticated, granted to `service_role` only. No existing grant touched.

Doc-sync (same step). `docs/SYSTEM_BOUNDARIES.md`: §1 gained the `module_instances` row (**Modules**, owner-read; sole writer = `modules_mint_instance` (0108), idempotent by `mint_key`, `service_role`-only, nothing calls it yet) and the catalog row dropped its now-stale "mint writer arrives with `module_instances`" note; the §2 Modules row's function list gained the mint signature + semantics (replacing "none yet — no function exists") — the Production-will-own-the-craft-command note, the M4.5 retirement note, and the forbidden column are unchanged and still accurate. No new cross-system edge: the helper reads only Modules' own catalog (`module_types`), and nothing calls it yet — the graph stays acyclic.

State. `npm run build green` (`tsc -b + vite`, exit 0). Migration head 0108; `module_crafting_enabled='false'` — still fully dark: the mint writer is `service_role`-only with ZERO callers (dead-until-slice-C by design, documented as such), the table has no client write path, and no flag was flipped, no live DB write, no workflow touched. **DB-apply posture (honest, unchanged from slice A)**: no `psql/docker/supabase` CLI in this sandbox and `npx` cannot fetch (`UNABLE_TO_VERIFY_LEAF_SIGNATURE`) — the migration was hand-verified line-by-line against the shipped idioms it copies (0098/0103 table+RLS posture, 0039 writer/idempotency pattern, 0104 ACL block); live assertions run in the owner's environment and will be covered by the slice-G `verify:modules` dark-posture script. PR-ready on `autopilot/20260703-064048`, main untouched. Next: slice C (the craft command — Production system, player-scoped receipts, `inventory_spend` fan-out + this mint helper).

2026-07-04 — MODULES-P13 SLICE A — locked design decisions + dark flag/catalog migration 0107 (`module_types` + `module_recipe_ingredients`)

Request. Begin Phase 13 "Module instances + crafting" (ROADMAP :88 — "instances, not stack-only") with slice A: record the owner's LOCKED design decisions, then ONE new forward-only migration seeding the dark flag + the module catalog/recipe config tables + starter seeds. NO instances table, NO command, NO read surface, NO frontend this slice. Recon: `MODULES_P13_RECON.local.md` (scope locked 2026-07-04).

LOCKED DESIGN DECISIONS (owner-directed 2026-07-04 — not self-approved):

1. **System shape** (ROADMAP law 5: "Production=support craft/crafting · Fitting=modules"): a NEW leaf system **Modules** owns the module state tables (`module_types` catalog, `module_recipe_ingredients` config, and — in later slices — `module_instances`

- + a mint writer), while the craft COMMAND itself will belong to the existing **Production** system, depending DOWNWARD on Inventory (inventory_spend) and Modules (mint) — acyclic, one sole-writer per table.
- 2. **Crafting is INSTANT in Phase 13**: an idempotent dark command in the 0099/0104 two-layer idiom with a PLAYER-scoped receipts table (crafting is non-spatial, so trade_relief_claims-style (player, request_id) keying, NOT ship-scoped space receipts). The M4.5 "same queue" note is FUTURE meaning — integrating with build_orders would touch the shipped Production queue and risk the green M4.5 tests, so it is explicitly deferred with this RETIREMENT NOTE: when module production later moves onto the serial queue, the queued completion path must call the SAME Modules mint helper this phase creates.
- 3. **Recipe encoding is a normalized table, NOT jsonb**: module_recipe_ingredients (module_type_id, item_id, qty) with FKs to module_types and item_types and a qty > 0 check — referential integrity over blob parsing; one implicit recipe per module type (its ingredient rows); costs are ITEMS-ONLY in Phase 13 (no metal/credits — the pipeline law says crafting consumes INVENTORY; metal would drag in a Base edge the phase doesn't need and can be added forward-only later).
- 4. **One craft = one instance** (no batching), keeping idempotency trivial.
- 5. **Flag name** module_crafting_enabled, seeded 'false', following the exact 0097/0102 config+flag idiom including the server-side feature_disabled rejection posture for every future RPC.

Work done — **NEW** supabase/migrations/20260618000107_modules_p13_catalog_and_flag.sql (migration head moves **0106** → **0107**; 0001-0106 unedited):

- (a) game_config seed module_crafting_enabled='false' (on conflict (key) do nothing, the exact 0097/0102 dark-gate idiom + description stating the reject-before-any-read law).
- (b) module_types — minimal intrinsic catalog identity ONLY: id text primary key, name text not null, slot_type text not null (intrinsic archetype; display now, fitting validation in Phase 14; unconstrained text like item_types.category/support_craft_types.role — no code consumer yet), description text not null, created_at. **NO stats columns** — stats wiring is Phase 14's job, added forward-only there.
- (c) module_recipe_ingredients per decision 3: FKs to both catalogs, qty integer not null check (qty > 0), PK (module_type_id, item_id).
- (d) Seeds (on conflict do nothing): 4 starter module types spanning distinct slot archetypes, copy matching the 0042 catalog tone — autocannon_battery (weapon: weapon_parts ×4
` pirate_alloy ×2 + scrap ×6), vector_thruster_kit (engine: engine_parts ×4 + crystal ×2 + scrap ×4), expanded_cargo_lattice (cargo: scrap ×10 + pirate_alloy ×3 + repair_parts ×2), deep_scan_sensor_array (sensor: scan_data ×5 + anomaly_shard ×2 + blueprint_fragment ×1). Recipes consume ONLY EXISTING item_types rows (0039/0097 seeds REUSED — item_types is NOT touched; the 0097 reuse law).`
- (e) RLS/grants — verified against the sources, not assumed: both tables copy the Reference/Config catalog posture verbatim (item_types 0039:23–25 / support_craft_types 0042:32–36): RLS enabled, ONE public-read select policy, grant select to anon, authenticated, NO write policy/grant. No function created → no execute-surface relock needed (0054 precedent).

Doc-sync (same step). docs/SYSTEM_BOUNDARIES.md: §1 matrix gained the row module_types, module_recipe_ingredients → **Modules** (catalog/config — seeded by migration only, NO runtime writer yet; the mint writer arrives with module_instances in a later Phase-13 slice; public read-only); §2 gained the **Modules** system row recording the dark gate, the Production-will-own-the-craft-command note (with the downward Inventory+Modules fan-out, the player-scoped receipt keying, one-craft-one-instance, and the M4.5 retirement note) and the forbidden column (never write player_inventory/inventory_ledger/base_resources; never mint outside the ONE mint helper; fitting/module_slots is Phase 14).

State. npm run build green (tsc -b + vite). Migration head **0107**; module_crafting_enabled='false' — nothing client-writable exists (two public-read catalogs + one dark flag; no RPC, no writer, no reader). No flag flipped, no live DB write, no workflow touched. **DB-apply posture (honest)**: this sandbox has no psql/docker/supabase CLI and npx cannot fetch (the recorded UNABLE_TO_VERIFY_LEAF_SIGNATURE environmental posture) — the migration was hand-verified line-by-line against the shipped idioms it copies (0039/0042 table+RLS posture, 0097/0102 seed idiom, 0098 same-step boundaries sync), exactly the P11/P12 slice-B/C verification posture; the seeds/flag assertions run against a real DB in the owner's environment (and will be covered by the slice-G verify:modules dark-posture script). PR-ready on autopilot/20260703-064048, main untouched. Next: slice B (module_instances + the mint helper).

2026-07-04 — MINING-CLEANUP SLICE 2 (final) — MarketPanel migrated onto useActivityPanelGuards. Guard-hook extraction complete — no local copies remain

Request. Final slice of the mining-milestone cleanup: migrate MarketPanel1.tsx (the idiom's original reference copy) onto the shared hook, byte-equivalent behavior, and close the doc trail.

Work done — src/features/map/MarketPanel1.tsx **only**:

- Local mounted-guard block + per-row `inFlightRef` Set replaced by `const { activeRef, tryClaim, release } = useActivityPanelGuards()` — the per-row granularity maps directly onto the hook's Set-of-string keys (`tryClaim(goodId)` / `release(goodId)`).
- Submit handler: `!shipId` first, then `tryClaim(goodId)` early-return; the qty validation now sits AFTER the claim with `release(goodId)` before its early return. **Behavior-equivalent reordering (claim→validate→release vs check→validate→claim):** the whole sequence is synchronous with NO await in between, so a duplicate in-flight click still returns before any validation side effect, and an invalid qty still leaves no lasting claim. `finally` now calls `release(goodId)`; the `activeRef`-guarded pending reset is untouched.
- Idiom comments repointed: the guard scaffold's home is `src/lib/useActivityPanelGuards.ts`; `MarketPanel` is now a consumer like `Exploration/Mining`, not the reference copy.
- refresh deps: `[shipId]` → `[shipId, activeRef]` (the SLICE 1 exhaustive-deps posture; ref identity is stable so `refresh`'s identity is unchanged). NOT touched: `pending/qty/rowError` state shapes, `refresh()`'s `Promise.all` body, the `!selectedShip` → null check, the `shell-with-unavailableNote` posture (still NOT `isServerLit` — documented in the hook), error copy via `tradeReasonMessage`, render output.

Extraction promised in the SLICE F note is COMPLETE: all three activity panels (`Market/Exploration/Mining`) consume `useActivityPanelGuards`; `grep` confirms no file under `src/` declares a local `activeRef/inFlightRef` guard outside the hook itself.

Doc-sync note. `docs/SYSTEM_BOUNDARIES.md` needs NO change (frontend-only, SLICE F precedent).

State. `npm run build green (tsc -b + vite)`. `npm run lint`: the 2 touched files lint clean; full-repo lint still FAILS with exactly the same 14 pre-existing out-of-scope errors recorded in SLICE 1 (no new problems). Migration head stays **0106** (no migrations, no flags); everything still dark and server-rejected. Diff = exactly 2 files (`MarketPanel.tsx`, this file).

2026-07-04 — MINING-CLEANUP SLICE 1 — panel guard scaffold extracted to `src/lib/useActivityPanelGuards.ts`; Exploration + Mining migrated

Request. Extract the duplicated activity-panel guard pattern (the documented cross-panel "`MarketPanel` idiom") into ONE shared hook and migrate the two twin panels, byte-equivalent behavior. This is the sanctioned "**adopt-on-next-real-change**" the SLICE F entry recorded when it deliberately did NOT extract the scaffold (third copy landed → the change is now real).

Work done — NEW `src/lib/useActivityPanelGuards.ts` (frontend-only; `src/lib/` is the established shared home, concern-per-file):

- `useActivityPanelGuards()` → `{ activeRef, tryClaim, release }`. The mounted guard is the `MarketPanel` block verbatim (`useRef(true)` + one empty-deps effect; `StrictMode` re-arms). The in-flight guard is a `useRef<Set<string>>` with stable callbacks: `tryClaim(key)` claims synchronously BEFORE any await (false if already claimed — the same-tick double-submit killer), `release(key)` drops it in the caller's `finally`. A Set-of-string serves BOTH granularities — `MarketPanel`'s per-row `good_id` keys and `Exploration/Mining`'s fixed `'scan'/'extract'` key — so one hook covers all three panels with zero behavior change.
- `isServerLit(result)` — the shared form of the `!result || !result.ok` fail-closed check, ONLY for server-lit panels that render nothing until the server affirms (`Exploration/Mining` style); explicitly NOT for `MarketPanel`'s `shell-with-unavailable-note` posture. Typed `result` is `Extract<T, { ok: true }>` so the discriminated-union narrowing the inline checks gave callers is preserved (`result.discoveries/result.extractions` stay type-safe).

Migrated (behavior byte-equivalent): `ExplorationPanel.tsx` + `MiningPanel.tsx` — the local `activeRef` block and boolean `inFlightRef` replaced by the hook; guard ORDER preserved (`!mainShipId` first, then `tryClaim`); `finally` now calls `release(...)`; fail-closed render check now `if (!isServerLit(result)) return null` under the same FAIL CLOSED comments; the "`MarketPanel` idiom" comments repointed to the shared hook. NOT touched: `refresh()`, effect deps, `lifecycleKey`, state shapes, error/success copy (incl. the mining cooldown suffix), `MarketPanel`.

Doc-sync note. `docs/SYSTEM_BOUNDARIES.md` needs NO change (SLICE F precedent): frontend-only, no table, no writer, no cross-system edge; server contracts unchanged.

State. `npm run build green (tsc -b + vite)`. `npm run lint`: the 4 touched files lint clean (0 problems, incl. exhaustive-deps via the stable-ref dep); full-repo lint FAILS with 14 pre-existing errors in out-of-scope files (`MainShipMarker.tsx`, `SpaceRouteLine.tsx`, `useSpaceMoveCommand.ts`, `tests/ harnesses/spec`) that predate this slice and sit outside the locked scope — NOT fixed here (scope law), left for their own cleanup step. Migration head stays **0106** (no migrations, no flags); everything still dark and server-rejected. Diff = exactly 4 files (new hook, two panels, this file). Next slice: migrate `MarketPanel`'s per-row guards onto the same hook and retire its local copy.

2026-07-04 — MINING-P12 SLICE G (final) — verify: mining dark-posture script. Phase 12 Mining — dark implementation complete (slices A–G)

Request. Implement slice G, the last Phase 12 slice: the dark-posture verify script + its `package.json` entry, mirroring the exploration slice-H precedent (the post-cleanup, harness-importing form). Touches ONLY `scripts/verify-mining.mjs`, `package.json`, this file, and the recon scratch file. No migrations (head stays **0106**), no CI/workflow edits, no flags.

Work done:

- **NEW** `scripts/verify-mining.mjs` — imports the shared harness from day one (`Abort/createReporter/createUserFactory/resolveEnv` from `scripts/lib/verify-harness.mjs` `<font name="Sym">` `</font>` `teardownVerifier` from `scripts/lib/verifier-teardown.mjs`) — ZERO inline harness copies (the harness header's law). Same posture as `verify-exploration.mjs`: proves the DARK contracts only; NEVER writes `game_config`, NEVER flips `mining_enabled`; service key OPTIONAL (teardown only); one throwaway signup. Asserts, in the exploration script's order/idioms: (1) **dark rejection** — `command_mining_extract(ZERO, ZERO) → {ok:false, code:'feature_disabled'}` (0104 gates before ship resolution — anti-probe) and `get_my_mining_extractions() → {ok:false, reason:'mining_disabled'}` (0106), both authenticated; (2) **no field leak** — authenticated select on `mining_fields` → denied/0 rows (0103 posture) and `mining_extractions` → 0 rows for a fresh user (own-row RLS); (3) **internal surfaces locked** — `mining_extract` + `process_mining_securing` denied to the authenticated client, both public RPCs denied to anon (`osn_distance` deliberately NOT re-asserted — `verify:exploration` owns that slice's surface, no duplicate assertion); (4) **config presence (read-only)** — `mining_enabled = false`, `mining_extract_radius = 750`, `mining_extract_cooldown_seconds = 300`, via the same `jsonb-storage-tolerant` comparison.
- `package.json` — `"verify: mining": "node scripts/verify-mining.mjs"` added in the verify cluster, directly after `verify: exploration`. No CI/workflow edits.
- **Verify posture run honestly:** `node --check scripts/verify-mining.mjs` parses clean. `node scripts/verify-mining.mjs` in this sandbox aborts at the throwaway SIGNUP step with the environmental TLS failure (`UNABLE_TO_VERIFY_LEAF_SIGNATURE → "signup failed: fetch failed"`) — `node scripts/verify-exploration.mjs` aborts at the IDENTICAL point in the same run, so this is the known environmental-fail-only posture (DEV_LOG 2026-07-03 precedent), and reaching that identical abort point proves the harness wiring. The assertions themselves run against a real DB in the owner's environment.
- `docs/SYSTEM_BOUNDARIES.md` needs NO change this slice — no table, writer, or cross-system edge (a read-only verifier script + one npm alias).

Phase 12 Mining — dark implementation complete (slices A–G) — closing summary

- **Migrations** 0102–0106 (head **0101** → **0106**; all forward-only; 0001–0101 never edited): 0102 config/flag (`mining_enabled='false' + mining_extract_radius='750' + mining_extract_cooldown_seconds='300'`; NO new item_types rows) · 0103 schema (`mining_fields` hidden/server-only + repeatable `mining_extractions` with own-row RLS + cooldown/player indexes; 5 seeded fields, items-only bundles from the existing `ore/crystal/artifact_core`) · 0104 the extract command (`command_mining_extract` wrapper → private `mining_extract`; dark-gate-first, S2 lock, receipts idempotency, `osn_distance` radius, per-(player, field) cooldown with `retry_after_seconds`) · 0105 the securing processor (`process_mining_securing` + `pg_cron process-mining-securing @60s`; flag-ignoring in-flight safety; deposits via `reward_grant('mining', extraction_id, ...)` — the sole depositor — on safe settle) · 0106 the read surface (`get_my_mining_extractions`; reveal-after-extraction only).
- **Frontend:** dark `src/features/mining/` (types/api/panel twins of the post-cleanup exploration files; server-driven visibility, fails closed to null) wired beside `ExplorationPanel` in `GalaxyMapScreen.tsx`; **shared-lib extractions** `src/lib/rewardBundle.ts` (`PendingBundle`) + `src/lib/osnState.ts` (`isSettledInSpace`) with exploration repointed same-step (no second copy).
- **Verify:** `scripts/verify-mining.mjs` + `npm run verify: mining` (dark posture only, shared harness, never flips flags).
- **Design decisions** (recon §8, self-approved 2026-07-04): exploration-template OSN-native extract command; repeatable extraction + server-enforced cooldown (no unique pair); rewards land in item inventory ONLY via `reward_grant('mining', ...)` reusing existing catalog rows (never `base_resources`; `trade_goods 'ore'` untouched); hidden fields + reveal-after-extraction; forfeiture deferred (0100 posture); cooldown per-ship serialization accepted while multi-ship is dark (0105 header note).
- **Ownership/laws:** `SYSTEM_BOUNDARIES` §1/§2 rows + `ARCHITECTURE` §7-adjacent §14 processor row + `ACTIVITIES/ROADMAP` untouched where already accurate — every doc synced in the SAME step as its fact; edges all DOWNWARD/acyclic; sole writers: `mining_fields` none (Reference/Config), `mining_extractions` = Mining (0104 inserts · 0105 secures); `reward_grant` the only depositor.
- **HUMAN ACTIVATION CHECKLIST (the owner's gate — never this loop):** (1) apply migrations 0102–0106 to the target DB; (2) run `npm run verify: mining` there — expect ALL dark-posture checks green; (3) optionally flip `mining_enabled='true'` on a DEV database and run the lit path (settle in space near a seeded field → `command_mining_extract` → pending row → repeat → cooldown with `retry_after_seconds` → dock/return home → `process_mining_securing` deposits ore/crystal/core items via `reward_grant('mining', ...)` → `get_my_mining_extractions` shows Secured) — then decide production activation separately. The loop ships everything server-rejected; activation is exclusively the human's.

State. `npm run build green`; `node --check clean` on the new script. Migration head **0106**; `mining_enabled='false'` everywhere; no flag flipped, no live DB write, no workflow touched. **Phase 12 Mining is implemented DARK end-to-end and PR-ready on autopilot/20260703-064048** — SAFE FOR HUMAN MERGE REVIEW; `main` untouched.

2026-07-04 — MINING-P12 SLICE F — dark frontend `src/features/mining/` + shared `src/lib/rewardBundle.ts/osnState.ts` extraction (exploration repointed same-step)

Request. Implement slice F of the Phase 12 plan (recon \$9): the dark mining frontend mirroring the post-cleanup exploration frontend exactly, with the HARD-RULE duplication check. No server changes, no migrations, no config — migration head STAYS **0106**.

Duplication check first (the hard rule) — TWO extractions, exploration repointed in this same step:

- **NEW** `src/lib/rewardBundle.ts` — `PendingBundleItem` + `PendingBundle` (the 0040/0041 server bundle contract). Mining needed the identical types verbatim; the one copy moved out of `explorationTypes.ts` (which now imports it) and `miningTypes.ts` imports it from day one.
- **NEW** `src/lib/osnState.ts` — `isSettledInSpace()` (the 0055 settled-in-space predicate that drives the action button's enabled state; server stays authoritative). Same story: moved out of `explorationTypes.ts`; `ExplorationPanel.tsx` and `MiningPanel.tsx` both import it from here. `src/lib/` is the established shared home (catalog/location/time idiom, concern-per-file).
- **NOT extracted (below the bar, stated per the request):** the panel scaffold (mounted-guard `activeRef` + synchronous `inFlightRef` + `refresh` callback) is the documented cross-panel "MarketPanel idiom" already present in several panels — the exploration cleanup pass reviewed these exact files and did not extract it; a shared-hook refactor would touch `MarketPanel` and siblings, out of this slice's scope (adopt-on-next-real-change precedent). The API wrappers (2-line `rpc` calls with per-feature names/types), the per-feature error-copy maps (different strings), and the inline `toLocaleString()` one-liner are trivial per-feature glue.

Work done — NEW `src/features/mining/` (the exploration twins, post-cleanup state — NO speculative disabled-reason constant, exactly what the cleanup pass deleted from exploration):

- `miningTypes.ts` — `MiningExtraction` (the 0106 row: `field_name`, `space_x/space_y`, `extracted_at`, `secured_at`, `bundle`), `GetMyMiningExtractionsResult`, `CommandMiningExtractResult` (0104 wrapper success shape; failure envelope includes optional `retry_after_seconds` — REAL server data on the cooldown code), and the extract error-copy map (the 0104 code set) + `miningExtractErrorMessage`.
- `miningApi.ts` — thin `supabase.rpc` wrappers for `command_mining_extract` + `get_my_mining_extractions`, identical envelope-handling idiom (transport error → normalized failure, never a throw into the render path).
- `MiningPanel.tsx` — the `ExplorationPanel` structure verbatim: server-driven visibility (reads the extractions on mount/lifecycle change and **fails closed to null on ANY non-ok envelope without inspecting reason** — the documented deliberate posture; while the server returns `mining_disabled` the panel renders nothing, so production is unchanged); extract enabled only when settled in space; fresh `crypto.randomUUID()` request id per submit with the synchronous in-flight guard; extraction history list (field name, Pending/Secured badge, bundle contents as `item × qty`, coords + timestamp). Mining-specific glue: the cooldown failure note appends the server's `retry_after_seconds`; amber styling vs exploration's violet; positioned bottom-left BESIDE `ExplorationPanel` (`left-[17rem]`; both are server-lit so overlap only ever involves lit surfaces).
- **Wiring:** `GalaxyMapScreen.tsx` — `MiningPanel` imported and rendered directly adjacent to `ExplorationPanel`, same import style, same props (`lifecycleKey/mainShipId/shipStatus/ shipSpatialState`), same comment convention.

Doc-sync note. `docs/SYSTEM_BOUNDARIES.md` needs NO change: the extractions are client-side display types/predicates (no table, no writer, no cross-system edge); the server contracts they mirror are unchanged.

State. `npm run build green` (`tsc -b typecheck + vite`; standalone `tsc --noEmit` also clean). Migration head stays **0106**; everything still dark — the panel is wired but renders nothing (every mining RPC server-rejects while `mining_enabled='false'`), and the repointed exploration surface is behavior-identical. PR-ready on autopilot/20260703-064048, `main` untouched. Next: slice G (`scripts/verify-mining.mjs` + the `verify:mining` entry).

2026-07-04 — MINING-P12 SLICE E — the dark read surface `0106` (`get_my_mining_extractions()`). Server side of Phase 12 complete, fully dark

Request. Implement slice E of the Phase 12 plan (recon §9): ONE new forward-only migration with the read surface, mirroring the exploration read surface 0101 exactly (function shape, dark-gate behavior, reveal semantics, envelope, ACL). No frontend, no config changes, no other functions.

Work done — NEW `supabase/migrations/20260618000106_mining_p12_read_surface.sql` (migration head moves **0105 → 0106**; 0001-0105 unedited):

- `get_my_mining_extractions()` — the 0101 body step-for-step: `auth.uid()` resolution (not_authenticated envelope), then the dark gate BEFORE any extraction/field read — `{ok:false, reason:'mining_disabled'}` (the 0101 `exploration_disabled` envelope shape), identical regardless of caller state (no probing while dark) — then the caller's OWN `mining_extractions` joined to the hidden `mining_fields` rows. Per row it reveals exactly the 0101 attribute classes: field name + `space_x/space_y` (as 0101 reveals sites), the extraction's lifecycle fields (`extracted_at` = the row's `created_at`, `secured_at`), and `bundle` = the row's `pending_bundle_json` snapshot — 0101 exposes the discovery's pending bundle, so mining mirrors it; the field's own `reward_bundle_json` is never exposed directly. Ordering (`created_at desc`), response shape (`{ok:true, extractions:[...]}`) mirroring `{ok:true, discoveries:[...]}`, and posture (stable, security definer, set `search_path = public`) all verbatim from 0101. Repeatability nuance (header-documented): the history legitimately contains multiple rows per field — one per extraction — and extracted-then-disabled fields stay visible (the 0101 posture: the player's own history).
- **Reveal rule (header):** a field is revealed ONLY through the player's own extraction rows — no browse-all surface; the 0103 no-client-policy posture on `mining_fields` is untouched, so an un-extracted field stays unreachable by construction (identical anti-probe stance to exploration).
- **ACL verbatim from 0101:** execute revoked from public/anon, granted to authenticated only — and dark today: the gate rejects every call while `mining_enabled='false'`.

Doc-sync (same step). `docs/SYSTEM_BOUNDARIES.md`: the §2 Mining row dropped its last FORTHCOMING — the read surface is now named LIVE (read-only, dark-gated `mining_disabled`, the only client path to field data, strictly post-extraction). The §1 matrix rows need NO change — slice E adds no writer (`get_my_mining_extractions` is read-only; `mining_fields` still has no runtime writer, `mining_extractions` still has exactly the two 0104/0105 writer fns).

State. `npm run build` green. Migration head **0106**. **The server side of Phase 12 Mining is COMPLETE (slices A–E) and fully dark end-to-end:** the command wrapper + writer and the read surface all server-reject while `mining_enabled='false'`; the securing processor correctly ignores the flag but is inert (no extraction row can exist). No flag flipped, no live DB write; PR-ready on `autopilot/20260703-064048`, `main` untouched. Next: slice F (dark frontend `src/features/mining/`), then slice G (`verify:mining`).

2026-07-04 — MINING-P12 SLICE D — the securing processor 0105 (`process_mining_securing` + `pg_cron`; deposits via `reward_grant('mining', ...)`)

Request. Implement slice D of the Phase 12 plan (recon §9): ONE new forward-only migration with Mining's own securing processor, mirroring the exploration securing processor 0100 exactly — mining's as-built extraction rows already model the lifecycle the same way (`secured_at` NULL = pending, 0103), so the processor shape carries over verbatim with no structural deviation. No read surface, no frontend, no config changes.

Work done — NEW `supabase/migrations/20260618000105_mining_p12_securing_processor.sql` (migration head moves **0104 → 0105**; 0001-0104 unedited):

- `process_mining_securing()` — the 0100 body step-for-step: `FOR UPDATE SKIP LOCKED` sweep of `mining_extractions` where `secured_at` is null; carrying ship = the row's `main_ship_id`, else the player's canonical main ship via `mainship_resolve_owned_ship` (the 0100 NULL-fallback verbatim; unresolvable → the row waits); settled `SAFE = spatial_state` in ('home', 'at_location') per the 0055 state model (anything else waits); deposit target = the player's active home base (0050 idiom; null base → the row waits — guard kept verbatim from 0100 even though mining bundles are items-only, one pattern not two); then `reward_grant('mining', extraction_id, player, base, pending_bundle_json) + secured_at = now()` in the same transaction. Like 0100 there is NO per-row exception wrapper — skips are `continue` branches, and idempotency is DOUBLE-guarded (`secured_at` NULL filter + the `reward_grants` UNIQUE (source_type, source_id) law), so a re-run can never double-deposit.
- **Flag posture (0100 wording convention):** the processor deliberately IGNORES `mining_enabled` — in-flight safety: accrued pending value must never be stranded by an emergency flag-off. Naturally inert today: the 0104 writer rejects while the flag is false, so no extraction rows can exist and the processor sweeps an empty set.
- **Slice-C review note recorded in the header:** the 0104 cooldown is serialized per SHIP via the S2 lock, not per player — acceptable because the canonical model is one main ship per player (multi-ship stays DARK behind `mainship_additional_commission_enabled=false`) and no double-deposit is possible regardless (receipts + the `reward_grants` unique key); revisit if multi-main-ship ever activates.
- **ACL + cron verbatim from 0100:** execute revoked from public/anon/authenticated, granted to service_role; `create extension if not exists pg_cron`; idempotent unschedule guard (`undefined_table` swallowed);

`cron.schedule('process-mining-securing', '* * * * *', ...)` — every 60s (pg_cron's seconds form caps at 59s, so every-minute standard cron, the 0100 comment).

- NO forfeiture: a pending extraction simply WAITS (destroyed ships secure after recovery lands them home) — the 0100 posture, recon decision 4.

Doc-sync (same step).

- docs/SYSTEM_BOUNDARIES.md: the `mining_extractions` §1 row now reads "ONE owner system, two writer fns: `mining_extract` (0104) inserts · `process_mining_securing` (0105) sets `secured_at`" (both LIVE); the §2 Mining row's securing paragraph rewritten to present tense (0105 shipped — safe-settle definition, double-guarded idempotency, flag-ignoring in-flight safety) with only the slice-E read surface still FORTHCOMING; the Mining → Bases (deposit-target read) and Mining → Reward (grant) edges are now live and the edge list stays all-DOWNWARD, exactly the Exploration shape.
- docs/ARCHITECTURE.md §14 processors table: added the `process_mining_securing()` row exactly parallel to the exploration row (every 60s; deposits via `reward_grant('mining', extraction_id, ...)` once the ship settles safe; deliberately ignores `mining_enabled`; pg_cron job `process-mining-securing`; migration 0105).

State. `npm run build` green. Migration head **0105**; still dark END-TO-END — the command rejects while `mining_enabled='false'`, so the processor (which correctly ignores the flag) has nothing to sweep; no flag flipped, no live DB write; PR-ready on `autopilot/20260703-064048`, `main` untouched. Next: slice E (the read surface).

2026-07-04 — MINING-P12 SLICE C — the dark extraction command 0104 (`command_mining_extract` → `private mining_extract`)

Request. Implement slice C of the Phase 12 plan (recon §9): ONE new forward-only migration with the two-layer extraction command, mirroring the exploration scan command's AS-BUILT form (0099 body

- the 0100 changes) — same shape, envelopes, locking, and ACL — deviating only where the recon §8 decisions require (repeatability/cooldown instead of unique-discovery). No processor, no read surface, no frontend, no `game_config` changes.

Work done — NEW `supabase/migrations/20260618000104_mining_p12_extract_command.sql` (migration head moves **0103** → **0104**; 0001–0103 unedited):

- **Private** `mining_extract(p_player, p_main_ship_id, p_request_id)` — the 0099/0100 writer step-for-step: dark gate FIRST (`cfg_bool('mining_enabled')`) → `feature_disabled` before ANY read/lock/write) → request-id validation → S2 canonical lock context → ownership-from-locked-snapshot → canonical payload hash → receipt lookup (`main_ship_space_command_receipts`; replay returns the first committed result; `request_id_payload_conflict` on hash mismatch — the EXACT 0099 mechanism, no new receipt system) → `mainship_space_validate_context` (settled in_space required; destroyed / not_in_space reasons) → `mainship_space_assert_cross_domain_exclusion` → live position under lock → NEAREST active `mining_fields` row within `cfg_num('mining_extract_radius')` via `osn_distance` (deterministic tie-break distance-then-name; none → `no_field_in_range`). **Deviations (recon decisions 2/4, all header-documented):** no discovered-filter and no ON CONFLICT race guard (repeatable; the S2 ship lock serializes concurrency, receipts dedupe replays); NEW cooldown step — the latest `mining_extractions.created_at` for (player, field) (the 0103 (player_id, field_id, created_at desc) index) must be older than `cfg_num('mining_extract_cooldown_seconds')`, else {ok:false, reason:'cooldown', retry_after_seconds} (failure writes NO receipt — 0064 posture — so the same request_id retries cleanly after the cooldown). On success: ONE extraction row inserted with `pending_bundle_json` = the field's `reward_bundle_json` verbatim + the resolved `main_ship_id`; success envelope in 0099's shape; receipt finalised atomically.
- **Public wrapper** `command_mining_extract(p_main_ship_id, p_request_id)` — the 0099 wrapper verbatim: auth check → `mining_enabled` gate BEFORE any ship/argument resolution (anti-probe; {ok:false, code:'feature_disabled'}) → `mainship_resolve_owned_ship` → delegate → the same reason→code/message map with `no_field_in_range/cooldown` replacing `no_site_in_range/already_discovered`; the `cooldown` failure passes `retry_after_seconds` through.
- **ACL verbatim from 0099:** `mining_extract` revoked from public/anon/authenticated, granted to service_role; `command_mining_extract` revoked from public/anon, granted to authenticated.
- DARK today: both gates reject every call; a successful extraction would only sit pending anyway — the securing processor arrives in slice D (unreachable today).

Doc-sync (same step). docs/SYSTEM_BOUNDARIES.md: the `mining_extractions` §1 row now names `mining_extract` (0104) as the LIVE sole insert path (insert-only) with `process_mining_securing` still FORTHCOMING; the §2 Mining row rewritten to present tense for the shipped command (wrapper → private writer, receipts/S2/validate/exclusion reuse, cooldown + nearest-field rule) with slices D/E still marked forthcoming — edge list stays all-DOWNWARD (Mining → OSN geometry/locks · Main Ship read · Reference/Config reads · Bases/Reward deferred to slice D), exactly the Exploration shape.

State. `npm run build` green. Migration head **0104**; still fully dark — the wrapper and writer both server-reject while `mining_enabled='false'`; no flag flipped, no live DB write; PR-ready on `autopilot/20260703-064048`, `main` untouched. Next: slice D (`process_mining_securing`).

2026-07-04 — MINING-P12 SLICE B — mining schema migration 0103 (mining_fields + mining_extractions, dark, no writer exists)

Request. Implement slice B of the Phase 12 plan (recon §9): ONE new forward-only migration with the two mining tables + seeds + RLS, mirroring the exploration schema slice 0098 and deviating only where the recon §8 decisions require it. No functions, no cron, no game_config changes, no frontend.

Work done — NEW supabase/migrations/20260618000103_mining_p12_fields_schema.sql (0098 structure/idioms; migration head moves 0102 → 0103; 0001-0102 unedited):

- **mining_fields** — hidden static resource-field catalog, the 0098 exploration_sites shape verbatim: name-unique seed key; space_x/space_y with the finite-only CHECK idiom + the [-10000,10000]² envelope; deterministic reward_bundle_json (jsonb-object CHECK); is_active soft-disable. **RLS enabled with NO client policy and NO client grant** (anti-probe posture identical to 0098 — field coordinates/composition are never client-readable before extraction). Seeds 5 fields on the integer grid, near/far spread, distinct from the exploration sites, with ITEMS-ONLY bundles (decision 3) drawn from the EXISTING catalog rows: ore in every field (qty 2–3), crystal in three (qty 1–2), artifact_core in exactly one (qty 1) — per-item quantities in the 0098 magnitude; no metal scalar, so nothing can ever land in base_resources.
- **mining_extractions** — per-extraction state row: player + field FKs, main_ship_id (on delete set null, the 0100 resolver-fallback idiom), pending_bundle_json snapshot + secured_at (NULL = pending — the exploration as-built lifecycle verbatim; no '{}' default needed on a fresh table, unlike the 0099 migration-validity shim), created_at as the cooldown anchor. **THE deliberate deviation from 0098 (decision 2): NO unique (player_id, field_id)** — extraction is repeatable; one row per extraction; idempotency = the slice-C receipt convention; pacing = the per-(player, field) cooldown. Indexes: (player_id, field_id, created_at desc) (cooldown lookup) + (player_id, created_at desc) (the 0098 player-index idiom, serves the slice-E read surface). Own-row SELECT policy + grant select to authenticated only — NO write policy/grant; writers are the forthcoming slice-C command and slice-D processor.
- Forfeiture of pending bundles: DEFERRED with the documented 0100 posture (pending rows wait; destruction semantics are a future product decision) — stated in the migration header.

Doc-sync (same step). docs/SYSTEM_BOUNDARIES.md: §1 matrix gained the two rows — mining_fields = Reference/Config (static hidden world data, NO runtime writer, server-only read); mining_extractions = **Mining** (ONE owner system, two FORTHCOMING writer fns: slice-C mining_extract inserts · slice-D process_mining_securing sets secured_at; DARK behind mining_enabled=false, schema-only today) — and §2 gained the Mining system row (forthcoming surfaces named; bundles items-only; deposits ONLY via Reward.grant('mining', extraction_id, ...) — reward_grant remains the sole depositor; edges all DOWNWARD in the Exploration shape; forbidden column includes writing base_resources at all). No other section contradicts the new tables (the "OSN geometry leaf" note already anticipated Mining as a later downward consumer).

State. npm run build green. Migration head 0103; both tables exist DARK with NO writer anywhere (nothing can insert until slice C ships); no flag flipped, no live DB write; PR-ready on autopilot/20260703-064048, main untouched. Next: slice C (command_mining_extract).

2026-07-04 — MINING-P12 SLICE A — design decisions (self-approved) + dark config/flag migration 0102

Request. Record the Phase 12 Mining design decisions and implement slice A: ONE new forward-only migration seeding the dark gate + tunables. Nothing else — no tables, no functions, no grants, no catalog rows, no frontend, no flag flipped.

Design decisions (self-approved; full text + rationale in MINING_P12_RECON.local.md §8):

1. **Extract shape:** mirror the exploration OSN-native template as a single command_mining_extract (prospect+extract in one command) — settled in_space under the S2 lock + cross-domain exclusion, proximity via the existing osn_distance leaf against mining_extract_radius, pending extraction row secured later by a flag-ignoring securing processor when the ship settles safe (the 0099→0100 shape). Maximal reuse; no new engine surface.
2. **Repeatability:** mining is repeatable (no unique(player_id, field_id)); one extraction row per extraction, receipts idempotency, plus a server-enforced per-(player, field) cooldown from the latest extraction's created_at against mining_extract_cooldown_seconds.
3. **Reward landing zone:** item inventory via reward_grant('mining', extraction_id, ...) reusing the EXISTING item_types rows ore/crystal/artifact_core; NO new catalog rows, NO writes to base_resources (that would add a second landing path to the Base-owned economy scalars). The trade_goods 'ore' is a separate Trade Market catalog and is not touched.
4. **Field visibility:** hidden like exploration_sites (RLS, no client policy/grant); deterministic reward_bundle_json per field; the read surface reveals only fields the player has extracted from. Forfeiture of in-flight pending bundles DEFERRED with the exploration 0100 posture (pending rows wait; destruction semantics are a future product decision).

Slice plan (recon \$9): A=config/flag (this step) → B=mining_fields+mining_extractions schema+seeds → C=command_mining_extract → D=securing processor → E=read surface → F=dark frontend src/features/mining/ → G=verify: mining script (must import scripts/lib/verify-harness.mjs).

Work done — NEW supabase/migrations/20260618000102_mining_p12_config_and_flag.sql (0097 structure/idioms; migration head moves **0101** → **0102**; 0001-0101 unedited):

- `game_config` seeds (established on conflict (key) do nothing upsert idiom): `mining_enabled='false'` (the dark gate — every later mining RPC must check it FIRST and reject-before-any-read), `mining_extract_radius='750'` (matches the exploration radius default; retune via config, no redeploy), `mining_extract_cooldown_seconds='300'`.
- Deliberately NO `item_types` rows (decision 3 — unlike 0097, which needed two new item classes).
- No table, no function, no grant, no cron; the keys are inert until slice C reads them.

Doc-sync note: docs/SYSTEM_BOUNDARIES.md needs NO change this slice — no new table, writer, or cross-system edge exists yet (`game_config` stays Reference/Config, admin/migration sole writer); the Mining ownership rows + \$2 system row land with the slice-B schema, same-step (the 0097/0098 precedent).

State. npm run build green. Migration head **0102**; no flag flipped, no live DB write — everything dark and server-rejected (`mining_enabled='false'`); PR-ready on autopilot/20260703-064048, main untouched.

2026-07-04 — EXPLORATION CLEANUP step 4 (final) — delete dead EXPLORATION_DISABLED_REASON export; closes recon finding #3. Exploration cleanup complete (findings 1–3 all fixed)

Request. Fix finding #3 and close out the exploration cleanup pass. Repo-wide grep re-confirmed `EXPLORATION_DISABLED_REASON` (`explorationTypes.ts:48`) had zero references outside the recon scratch file; deleted the constant + its doc comment (2 lines) and nothing else. Rationale: the panel deliberately collapses ALL failure envelopes without inspecting `reason` (`ExplorationPanel.tsx:86`), so the constant was speculative dead code with no consumer and no planned consumer — the dark surface stays server-driven either way.

Cleanup pass summary (the trade-milestone-style audit of Phase 11, slices A–H / 0096–0101):

- **Audit verdict:** boundaries acyclic and all edges downward; sole writers hold everywhere (`exploration_discoveries` = one owner system, two writer fns; `exploration_sites` = no runtime writer; `reward_grant` the sole depositor; `osn_distance` a pure IMMUTABLE leaf); dark gates consistent (all three client-callable surfaces reject-before-any-read while `exploration_enabled='false'`; the securing processor's flag exception documented as in-flight safety); every shim carries its retirement condition. Three cleanup-class findings, none severe.
- **Finding #1 (step 2):** docs/ARCHITECTURE.md doc-sync — \$14 processors table gained the `process_exploration_securing()` row; \$7 gained the OSN-native as-built clarification (narrow self-approved scope amendment, recorded in the recon \$3).
- **Finding #2 (step 3):** shared verify harness extracted to `scripts/lib/verify-harness.mjs`; `verify-exploration.mjs` repointed (pure extraction, identical environmental abort point); retirement plan: the 31 sibling verifiers adopt on next meaningful touch (`osn_distance` precedent, `SYSTEM_BOUNDARIES.md:75–78`).
- **Finding #3 (this step):** dead `EXPLORATION_DISABLED_REASON` export deleted.

State. npm run build green; node --check scripts/verify-exploration.mjs parses clean. Migration head remains **0101**; no migration edited, no flag touched, `game_config` seeds untouched; every exploration surface still dark/server-rejected; everything PR-ready on autopilot/20260703-064048, main untouched. The exploration cleanup audit is **CLOSED** (recon: `CLEANUP_EXPLORATION_RECON.local.md`, findings 1–3 all FIXED).

2026-07-04 — EXPLORATION CLEANUP step 3 — extract shared verify harness (scripts/lib/verify-harness.mjs); closes recon finding #2

Request. Fix finding #2 of the exploration cleanup recon: `scripts/verify-exploration.mjs` had added the 32nd verbatim inline copy of the verify-script harness (the `loadEnv()` env loader + URL/key resolution, the `ok/bad/Abort/die` reporting harness, and the throwaway-signup `newUser()`) instead of extracting it to `scripts/lib/`. Touches ONLY the new module, the exploration verifier, this file, and the recon scratch file — no migrations, no flags, no sibling scripts, no `package.json`.

Work done — pure extraction, no behavior change:

- **NEW** `scripts/lib/verify-harness.mjs` (next to the existing `verifier-teardown.mjs`, same module style): exports `loadEnv()` + `resolveEnv()` (anon key required → exit 2; service key OPTIONAL at this layer — a verifier that requires it asserts that itself), `Abort/die`, `createReporter()` (`ok/bad` + shared pass/fail counts), and `createUserFactory({url, anonKey, emailPrefix,`

`createdUserIds}}`) → `newUser(tag)` (ids pushed immediately after creation for finally-teardown). Parameterized ONLY where the sibling comparison showed a variation point the exploration script actually relies on (email prefix, optional service key, caller-owned `createdUserIds`); no speculative knobs for sibling quirks it doesn't use.

- `scripts/verify-exploration.mjs` repointed at the module; its inline copies deleted. Every assertion, ordering, envelope check, and the teardown behavior are semantically identical.
- **RETIREMENT PLAN for the remaining duplication (stated in the module header):** the 31 sibling `verify-*.mjs` scripts still carry inline copies and MUST adopt `verify-harness.mjs` the next time each is meaningfully touched — the documented `osn_distance` adopt-on-next-real-change precedent (`docs/SYSTEM_BOUNDARIES.md:75–78`). New verifiers import from the harness from day one.

Verification. `node --check` parses both files clean. `node scripts/verify-exploration.mjs` reaches the IDENTICAL sandbox environmental abort as before the extraction (X ABORTED – signup failed: fetch failed, exit 1 — the same no-reachable-Supabase blocker the slice-H entry records), proving the harness wires up identically: `resolveEnv` resolved the keys, the shared `newUser`'s die threw Abort through the script's `instanceof Abort` catch, and the shared reporter printed the summary. `npm run build` green. Finding #2 of `CLEANUP_EXPLORATION_RECON.local.md` is FIXED (marked in the recon; finding #3 remains).

2026-07-04 — EXPLORATION CLEANUP step 2 — ARCHITECTURE.md doc-sync (docs-only; closes recon finding #1)

Request. Fix finding #1 of the exploration cleanup recon: `docs/ARCHITECTURE.md` contradicted the as-built Phase 11 code — §14's processors/cron table omitted `process_exploration_securing()` (a live 60s `pg_cron` job since 0100), and §7 still implied no exploration processor exists ("Later: `process_exploration_ticks()`"). Docs-only: no code, no migrations, no flags.

Scope amendment (self-approved, recorded in the recon §3). The cleanup's locked scope is amended to also allow `docs/ARCHITECTURE.md`, restricted to exactly these two out-of-sync spots — a law doc contradicting as-built code is a defect under the doc-sync principle; `ARCHITECTURE.md`'s exclusion from the original MAY-touch list was an oversight in the scope lock itself.

Work done — two surgical edits in `docs/ARCHITECTURE.md`:

- **§14 processors table:** added one row — `process_exploration_securing()` · every 60s · deposits pending exploration discovery bundles via `reward_grant('exploration', discovery_id, ...)` once the carrying main ship settles safe (home / at_location); deliberately ignores `exploration_enabled` (in-flight safety); `pg_cron` job `process-exploration-securing`, migration 0100.
- **§7 activity list:** added an as-built bullet — Phase 11 shipped exploration OSN-native, outside the presence dispatch, with its own securing processor (dark behind `exploration_enabled='false'`; mirrors the `docs/ACTIVITIES.md` §2 as-built clarification); the `explore_derelict` presence branch stays deliberately unwired. The "Later" line now names `process_mining_ticks()` / `process_trade_ticks()` and a presence-domain `process_exploration_ticks()` only as a hypothetical future form — it no longer implies the exploration processor is unbuilt.

State. `npm run build` green (docs-only sanity check). Migration head remains 0101; no flag touched; finding #1 of `CLEANUP_EXPLORATION_RECON.local.md` is FIXED (marked in the recon; findings #2–#3 remain for the next steps).

2026-07-04 — EXPLORATION-P11 SLICE H (final): `verify:exploration` dark-posture script + wiring. Phase 11 Exploration: dark implementation complete (slices A–H)

Request. The exploration verify script + `package.json` wiring + this closing entry. Touches ONLY `scripts/verify-exploration.mjs`, `package.json`, and this file; migrations 0001–0101 unedited; no CI/workflow files; no flags flipped.

Design decisions (self-approved).

1. `verify:exploration` wired in `package.json` exactly like the `verify:mainship-*` entries (one `node scripts/...` line in the verify cluster). **No CI/workflow edits** — the narrowest compliant wiring; the human can extend the existing CI pattern later; this loop does not touch workflows.
2. **The script proves the DARK POSTURE and contracts only — it activates nothing.** It never writes `game_config`, never sets `exploration_enabled`, and creates nothing beyond the sibling scripts' throwaway-test-player convention (one signup, tracked and deleted by the shared `scripts/lib/verifier-teardown.mjs` when a service key is present; without one, teardown is skipped with a note — the `verify-m3/m4` precedent). The sibling `verify-mainship-send.mjs` DOES flip its own flag via

`set_game_config` (its lines 49/98/105) — that part was **deliberately not copied**, and the script says so in its header: lit-path verification is deferred to the activation checklist below. `SUPABASE_SERVICE_ROLE_KEY` is OPTIONAL (teardown only); every assertion runs with anon/authenticated clients.

The script asserts, in order (idioms: env loader + ok/bad + Abort + exit codes from `verify-m45.mjs:12-41, 147-149`; throwaway `newUser` from `verify-m45.mjs:49-57`; client-role ACL-denial loop from `verify-m45.mjs:135-137`; read-only `cfgVal` query shape from `verify-mainship-send.mjs:50`, run as the client role; teardown from `verify-mainship-send.mjs:206-224` minus the flag entry):

- **(a) dark rejection** — `command_exploration_scan(ZERO, ZERO) → {ok:false, code:'feature_disabled'}` (0099/0100 wrapper envelope; the wrapper gates before ship resolution, so a zero id gets the same anti-probe answer) and `get_my_exploration_discoveries() → {ok:false, reason:'exploration_disabled'}` (0101), both for an authenticated throwaway user.
- **(b) no site leak** — authenticated `select on exploration_sites → denial or 0 rows` (0098 posture); `exploration_discoveries → 0 rows` for a fresh user (own-row RLS).
- **(c) internal surfaces locked** — client-role rpc calls to `exploration_scan`, `process_exploration_securing`, `osn_distance` all denied; plus anon denied on both public RPCs.
- **(d) config presence (read-only)** — `exploration_enabled` reads `false`, `exploration_scan_radius` reads `750`, compared tolerantly of the jsonb storage form (see Bugs below).

Phase 11 Exploration — dark implementation complete (slices A–H)

The six migrations (0096–0101; head 0095 → 0101, all forward-only, nothing shipped edited):

- **0096** — engine carrier made activity-agnostic: `fleet_movements.reward_source_type` (+ closed CHECK) and `movement_attach_cargo(..., source_type default 'combat')`; `process_fleet_movements` deposits under the carried type. Combat behavior unchanged.
- **0097** — the four-item exploration reward set (`scan_data` + `anomaly_shard` seeded; `blueprint_fragment` `<font name="Sym"> </font>` `artifact_core` reused from 0039) + the `exploration_enabled='false'` dark gate.
- **0098** — hidden `exploration_sites` (RLS, NO client policy/grant; OSN coordinate convention; deterministic `reward_bundle_json`; 5 seeds) + own-row `exploration_discoveries` with unique (`player_id`, `site_id`).
- **0099** — dark write path: `osn_distance` leaf + `exploration_scan` private writer (S2 locks, 0055 receipts idempotency, reject-before-any-read) + `command_exploration_scan` wrapper + `exploration_scan_radius='750'`; pending-bundle accrual columns.
- **0100** — securing: `exploration_discoveries.main_ship_id`, the race-guarded re-created writer, and `process_exploration_securing()` (60s cron) → `reward_grant('exploration', discovery_id, ...)` when the carrying ship settles safe (home / at_location); in-flight-safe (no flag check); double-guarded idempotency.
- **0101** — dark read surface `get_my_exploration_discoveries()` (reveal-after-discovery; the ONLY client path to site data).

Frontend surface (Slice G): `src/features/exploration/` (types + api + `ExplorationPanel`), mounted in `GalaxyMapScreen`'s OSN overlay stack; server-driven visibility — renders nothing while the server answers `exploration_disabled`.

The corrected securing law (Slice E re-decision): OSN-native activities never traverse `fleet_movements`, so Exploration secures via its OWN processor calling `reward_grant` directly — the same sole depositor the fleet return branch uses; `movement_attach_cargo` remains the fleet-domain carrier only (combat today). `docs/SYSTEM_BOUNDARIES.md` + `docs/ACTIVITIES.md` carry the as-built law.

ACTIVATION CHECKLIST — for the human owner only (nothing below is done by this loop):

1. **Flip** `exploration_enabled → 'true'` — **that is the ONLY switch**. The cron (`process-exploration-securing`), both RPCs, the read surface, and the panel are already in place and fail closed until the flip; the securing processor deliberately ignores the flag (in-flight safety) and is inert while no discoveries exist.
2. **Reposition the** `GalaxyMap.tsx:390` **bottom-left legend** ("N locations · M moving · drag to pan ...") — the `ExplorationPanel` also renders bottom-left when lit and will cover it. Cosmetic; deferred because a dark panel covers nothing today.
3. **Decide destruction-forfeiture semantics for pending exploration data BEFORE OSN main-ship combat ships**. v1 never forfeits: a pending discovery waits and secures after recovery. Fine while destruction is rare/dev-only; a real combat loop needs an explicit forfeit-or-keep rule (ACTIVITIES.md §2 note).
4. `activity_start/explore_derelict` **is deliberately unwired in v1** (OSN-native-only scope decision, Slice F). Do not "finish" that dispatch branch without a product decision.
5. **Before any flip: run** `verify:exploration` (dark posture) **against a dev DB, then the lit-path checks there** — flip the flag ON THE DEV DB only, scan from a settled in-space ship near a seeded site, watch the discovery appear pending, dock/return home, confirm the cron deposits (metal → base, items → inventory, `secured_at` set, `reward_grants` row ('exploration', `discovery_id`)), and re-run `verify:exploration` after re-darkening. Production flips remain a human production-gate action (`PROD_GATE_APPROVAL_POLICY`).

Nothing in slices A–H flipped a flag, merged anything, deployed anything, or touched production. All work sits PR-ready on autopilot/20260703-064048; every exploration surface is server-rejected while `exploration_enabled='false'`; main untouched.

State (this slice). `npm run build` green; `node --check scripts/verify-exploration.mjs` parses clean; `node scripts/verify-exploration.mjs` in this sandbox aborts at signup with `fetch failed` (no reachable Supabase) — the identical ENVIRONMENT blocker `verify:m3/m4` record, not a syntax/logic failure. Migration head remains **0101**.

Bugs / fixes

- **jsonb type mismatch in the step-4 config assertions (reviewer-caught; fixed).** `game_config.value` is `jsonb` (0003:8), so the seeded literals `'false'` (0097) / `'750'` (0099) store as JSON boolean/number and supabase-js returns JS `false` / `750` — the original string comparisons (`v === 'false'` / `v === '750'`) would have false-failed step 4 on exactly the healthy dev DB the activation checklist targets (masked here because the sandbox aborts at signup, before step 4; no sibling script string-compares a `cfgVal` result — they only capture-and-restore — so there was no precedent to copy). Fixed to storage-form-tolerant comparisons (`String(v) === 'false'` / `Number(v) === 750`), mirroring how the server's `cfg_bool/cfg_num` (0046:23 idiom) are storage-form-agnostic; noted in-line in the script.

2026-07-04 — EXPLORATION-P11 SLICE G: dark frontend surface src/features/exploration/ (scan control + discoveries panel; renders nothing while the server says dark). Frontend only — no migration

Request. The exploration client surface: scan control + discoveries panel in a new feature folder, integrated with the OSN in-space controls. NO new migration; nothing outside the feature folder + one integration point + docs.

Design decisions (self-approved).

1. **Server-driven visibility, no client flag constant.** The panel calls `get_my_exploration_discoveries()` on mount/lifecycle change and renders **nothing unless the server affirmatively answers** `{ok:true}` — the `exploration_disabled` dark envelope (and any transport failure) fails closed to `null`. This follows the fail-closed side of the trade client idiom (all `{ok:false, reason}` shapes collapse quietly; nothing throws into the render path — `tradeApi.ts/MarketPanel.tsx`) while deliberately NOT copying trade's compile-time `TRADE_MARKET_ENABLED` constant: visibility is the SERVER's answer alone. The UI is not the control anyway — the server rejects every exploration RPC while dark (fail-closed law, both sides).
2. **Placement with the OSN in-space controls.** The panel mounts in `GalaxyMapScreen`'s map-overlay stack (where the OSN command surfaces live: `PortNavPanel` top-left, `DockServicesPanel` top-right, `SpaceStopControls` bottom-right) at **bottom-left**, receiving the ship id + `status/spatial_state` the screen already threads to its OSN siblings. Single feature folder `src/features/exploration/`; no new route (matches how the trade/dock surfaces integrate — overlay panels, not routes).
3. **request_id idiom copied from the existing command clients:** one `crypto.randomUUID()` per intentional submit (the `MarketPanel` idiom; the server dedups on `(main_ship_id, request_id)`), a synchronous in-flight ref so a same-tick double-click can't mint a second id, and disabled buttons while in flight.

Work done — three new files + one integration point; no migration, `0001–0101` unedited:

- `src/features/exploration/explorationTypes.ts` — framework-free types mirroring the server contracts exactly (`CommandExplorationScanResult` from 0099/0100's wrapper; `ExplorationDiscovery` / `GetMyExplorationDiscoveriesResult` from 0101: `discovery_id`, `site_name`, `space_x`, `space_y`, `discovered_at`, `secured_at`, `bundle`), the `isSettledInSpace` predicate (0055 model: `in_space` $\Leftrightarrow$ `stationary`; drives only the button — the server stays authoritative), and the scan reason→message copy map in the `spaceStopCommand.ts` style (`feature_disabled`, `invalid_request`, `request_conflict`, `no_ship`, `ship_destroyed`, `not_in_space`, `busy_legacy`, `no_site_in_range`, `already_discovered`, `not_authenticated`, `unavailable`).
- `src/features/exploration/explorationApi.ts` — two thin `supabase.rpc` wrappers (`commandExplorationScan(mainShipId, requestId)`, `getMyExplorationDiscoveries()`); transport/DB errors normalize to `{ok:false, code/reason:'unavailable'}` (tradeApi idiom — never throw into render).
- `src/features/exploration/ExplorationPanel.tsx` — reads discoveries on mount + lifecycleKey change (the `useDockServices` re-fetch idiom); early-returns `null` unless `result.ok`; Scan button enabled only when settled in space (disabled with the truthful hint "Stop in open space to scan." otherwise); on success shows "Discovered <name>." and refreshes; failures show the server message (fallback: copy map). Discovery rows: name, rounded coordinates, local discovery time, and a Pending/Secured badge from `secured_at`. Styling/testids match the neighboring OSN overlay panels.
- **Integration (the ONE point):** `src/features/map/GalaxyMapScreen.tsx` — `ExplorationPanel` mounted directly after `DockServicesPanel` in the map-area overlay block, passing `mainShipId/shipStatus/shipSpatialState` + the lifecycle key the siblings already use.

Panel renders nothing in production today because the server returns `exploration_disabled` — no flag was touched.

`docs/SYSTEM_BOUNDARIES.md` NOT edited: verified it does not document client surfaces per system (the Trade Market row lists RPCs/flags only; trade's UI is not listed), so exploration's UI is not added either.

State. Frontend-only; migration head remains **0101**. `npm run build green (tsc -b && vite build, 144 modules` — up from 141 with the three new files); `npx eslint` on the new folder + the integration file: clean. `verify:m3/verify:m4` fail only on `fetch` failed (no reachable Supabase from this sandbox) and `verify:m45` needs `SUPABASE_SERVICE_ROLE_KEY` — the recorded environmental posture; no code/assertion failure. `main` untouched.

Bugs / fixes

- `_(none — additive dark UI; server rejects everything while dark, and the panel renders nothing on that answer.)_`

2026-07-04 — EXPLORATION-P11 SLICE F: dark read surface

`get_my_exploration_discoveries()` (reveal-after-discovery) + ACTIVITIES.md as-built reconciliation

Request. The exploration read surface: one server RPC exposing the caller's own discoveries (with the joined site data), dark-gated; plus the reviewer-flagged ACTIVITIES.md lifecycle reconciliation. No frontend yet.

Design decisions (self-approved).

1. **The client never reads `exploration_sites` directly.** Reveal-after-discovery goes through ONE server read RPC that joins the player's OWN `exploration_discoveries` to the site rows and returns only discovered sites. The 0098 no-client-policy posture on sites is untouched: an undiscovered site's existence/name/coordinates stay unreachable **by construction** (a site row is reachable exclusively through one of the caller's own discovery rows; where `d.player_id = auth.uid()`). Same spirit as the `get_my_current_dock_services` (0069) read surface — already-authoritative, player-scoped, everything derived server-side.
2. **Dark-gated FIRST, copying the 0087 `get_market_offers` read idiom exactly** (0087:46-50): auth check, then `if not cfg_bool('exploration_enabled') → {ok:false, reason:'exploration_disabled'}` BEFORE any discovery/site read — the identical envelope regardless of caller state, so nothing can be probed while dark (matches the `trade_market_disabled/trade_relief_disabled` reason-token style).
3. **Exploration v1 is OSN-native ONLY (explicit scope decision).** The `activity_start/explore_derelict` location-presence dispatch is deliberately NOT wired in Phase 11 (ROADMAP: "scan in OSN proximity ... where applicable"); `activity_start` still raises on `explore_derelict` — intended behavior, recorded in ACTIVITIES.md so nobody "finishes" the branch by accident.

Work done — one new migration `2026061800101_exploration_p11_read_surface.sql` (head **0100** → **0101**):

`get_my_exploration_discoveries()` — language `plpgsql` stable security definer; no arguments (`player = auth.uid()`); flag gate first; then one read-only aggregate: the caller's discoveries joined to `exploration_sites`, each as `{discovery_id, site_name, space_x, space_y, discovered_at, secured_at, bundle}` (`bundle` = the row's `pending_bundle_json` snapshot; `secured_at` null = pending, non-null = deposited), ordered `discovered_at desc`, `[]`-coalesced; envelope `{ok:true, discoveries:[...]}`. **No write anywhere** (single SELECT aggregate). Discovered-then-disabled sites stay visible — the discovery is the player's own history. ACL (0087 idiom): revoke from public/anon; grant execute to authenticated only. Dark today because the gate rejects while `exploration_enabled='false'`.

Doc sync (same step). (a) `docs/SYSTEM_BOUNDARIES.md` — the Exploration row's surface list now names `get_my_exploration_discoveries()` (read-only, dark-gated, the ONLY client path to site data, strictly post-discovery). (b) `docs/ACTIVITIES.md` — the reviewer-flagged reconciliation as a marked **"Phase 11 as-built clarification (not a new design)"** note in §2: OSN-native activities secure pending rewards when the carrying ship next settles SAFE (home or docked at `location`, 0055 model) via the activity's own processor + `reward_grant`; the "home arrival" wording is the `fleet_movements-domain` form (combat); destruction-forfeiture deferred; Exploration v1 OSN-native-only dispatch decision recorded.

State. Forward-only; migration head **0101**; `0001-0100` unedited. No flag flipped; no frontend (next slice). `main` untouched. `npm run build green`; `verify:m3/verify:m4` fail only on `fetch` failed (no reachable Supabase from this sandbox) and `verify:m45` needs `SUPABASE_SERVICE_ROLE_KEY` — the recorded environmental posture; no code/assertion failure.

Bugs / fixes

- `_(none — additive read-only surface; the ACTIVITIES.md fleet-domain wording ambiguity is reconciled by the marked as-built note.)_`

2026-07-04 — EXPLORATION-P11 SLICE E: dark securing/deposit path — `process_exploration_securing cron → reward_grant('exploration', ...)` on safe settlement. ⚠ DESIGN RE-DECISION: corrects the Slice-C carrier law

Request. Secure pending exploration discoveries into real rewards when the scanning ship next settles safe, via Exploration's OWN cron processor (the ACTIVITIES.md "own cron per activity" template). Everything stays dark.

⚠ **DESIGN RE-DECISION (self-approved) — the Slice-C carrier law was wrong for OSN-native scanning.** Slice C's SYSTEM_BOUNDARIES row said exploration deposits ride `movement_attach_cargo(..., 'exploration')`. That path is **UNREACHABLE** for OSN scanning: an OSN in-space ship never traverses `fleet_movements` — the S2 posture never locks legacy movements, `mainship_space_assert_cross_domain_exclusion` rejects a ship claimed by one, and OSN has no HOME leg (`origin_not_anchored` fails closed) — so the fleet carrier can never fire for it. The engine contract Exploration actually reuses is one level down: `reward_grant` is **THE sole secured-deposit owner and idempotency owner** (`reward_grants` UNIQUE (`source_type`, `source_id`)), and the activity accrues pending value on its own state until a safe arrival. Exploration's own processor therefore calls `reward_grant('exploration', discovery_id, player, base, bundle)` directly — exactly as `process_fleet_movements` calls it for fleet returns. `movement_attach_cargo` remains the carrier for `fleet_movements`-domain activities ONLY (Slice A stays correct and is used by combat today). Dependency direction stays acyclic and DOWNWARD: Exploration → {OSN geometry/locks (read), Main Ship (read), Bases (read: deposit target), Reward (grant)}; OSN and the arrival processors are NOT edited and never call into Exploration. SYSTEM_BOUNDARIES corrected in the SAME step (matrix + Exploration row).

Work done — one new migration `20260618000100_exploration_p11_securing_processor.sql` (head **0099** → **0100**):

- `exploration_discoveries.main_ship_id` — FK → `main_ship_instances` on delete set null; records WHICH ship holds the unsecured scan data. NULL only possible for legacy/deleted-ship rows; securing falls back to the player's canonical main ship (`mainship_resolve_owned_ship(player, null)`, the 0081 shared resolver).
- `exploration_scan` **re-created from the 0099 body with EXACTLY TWO changes** (diff-proven; ACL re-asserted verbatim): (a) the discovery insert records `main_ship_id`; (b) **race-guard fix** — the insert is now on `conflict (player_id, site_id) do nothing` and 0 rows inserted returns a truthful `already_discovered` instead of a raw unique-violation exception on a same-player concurrent scan (failure reasons write no receipt — the 0064 posture — so retries stay deterministic).
- `process_exploration_securing()` — Exploration's OWN cron processor (security definer; internal/service_role only, 0033 ACL idiom). `secured_at` is null rows via `FOR UPDATE SKIP LOCKED`; resolves the carrying ship (row's `main_ship_id`, else canonical); secures ONLY if settled **SAFE** per the 0055 state model — `spatial_state` in ('home', 'at_location') (constraints tie these to `status='home' / status='stationary'`, 0055:151–153 / 0055:145–147) — never in_transit/in_space/destroyed/legacy-NUL; resolves the deposit base with the 0050 idiom (from bases where `player_id=...` and `status='active'` order by `created_at` limit 1) and SKIPS (row stays pending) rather than granting with a null base — `reward_grant` would silently drop the metal half; then `reward_grant('exploration', d.id, ...) + secured_at = now()`. **Idempotency double-guarded:** the `secured_at` filter (fast path) + `reward_grants` UNIQUE (`source_type`, `source_id`) (the law — can never double-deposit). **No forfeiture in this slice:** pending rows simply wait (a destroyed ship secures after recovery lands it home); destruction semantics for pending scan data are a future product decision, deliberately not invented here.
- **IN-FLIGHT SAFETY (0064 precedent, stated in the header):** the processor does NOT check `exploration_enabled` — accrued pending value must never be stranded by an emergency flag-off. Naturally inert today: no discovery rows can exist while scan is dark.
- **Cron:** `process-exploration-securing` every 60s via the 0033 idiom (guarded `cron.unschedule` DO-block + `'* * * * *'` — `pg_cron` rejects '60 seconds'). Cadence rationale: securing is not latency-sensitive; matches the location-state tick's order of magnitude. Cadence summary: `movement` 30s · `combat` 3s · `worldstate` 60s · `space arrivals` (0058) · **exploration securing 60s**.

State. Forward-only; migration head **0100**; 0001–0099 unedited. No flag flipped (`exploration_enabled` stays 'false'; the scan writer + wrapper still dark-reject first). No frontend. docs/SYSTEM_BOUNDARIES.md corrected in the SAME step: the Exploration row now carries the securing law (own processor → `Reward.grant('exploration', discovery_id)`; `movement_attach_cargo` = fleet-domain carrier ONLY, with the unreachability rationale; edges listed, all downward, acyclic), and the matrix row records the writer SET (`exploration_scan` inserts · `process_exploration_securing` sets `secured_at`) under the ONE Exploration owner system. `main` untouched. `npm run build green`; `verify:m3/verify:m4` fail only on `fetch failed` (no reachable Supabase from this sandbox) and `verify:m45` needs `SUPABASE_SERVICE_ROLE_KEY` — the recorded environmental posture; no code/assertion failure.

Bugs / fixes

- **Slice-C carrier law unreachable for OSN scanning (doc/design defect).** Fixed by the re-decision above — corrected in SYSTEM_BOUNDARIES the same step; no shipped code implemented the wrong law (the deposit path did not exist until this slice), so this is a doc/design correction, not a behavior change.
- **Same-player concurrent-scan race (0099).** Two concurrent scans of the same site could raise a raw unique violation instead of a clean reason. Fixed in the re-created writer via `on conflict ... do nothing + already_discovered`.

2026-07-04 — EXPLORATION-P11 SLICE D: dark `command_exploration_scan` — OSN-proximity scan → pending discovery bundle (server-rejected while `exploration_enabled=false`; nothing deposits yet)

Request. The exploration write path: an OSN-proximity scan that records a per-player discovery with a PENDING (not yet deposited) reward bundle. Deposit wiring is deliberately NOT in this slice.

Design decisions (self-approved).

1. **Geometry is OSN's concern; Exploration depends on it DOWNWARD.** New pure IMMUTABLE leaf `osn_distance(ax,ay,bx,by) → double precision` — the exact euclidean formula the movement writers already use inline (`sqrt(power(bx-ax,2)+power(by-ay,2))`, verified at 0007:105, 0057:179, 0067:319). The shipped movement-writer bodies were NOT re-created just to swap their one-line inline `sqrt`: a single arithmetic expression is below the duplication bar, and re-creating proven critical writers for a cosmetic swap adds regression risk with zero behavior gain. **Future re-definitions of those writers should adopt the helper when next touched for real changes** (also recorded in SYSTEM_BOUNDARIES).
2. **Accrual law (ACTIVITIES.md):** the activity accrues pending rewards on ITS OWN state. The discovery row snapshots the site's bundle at scan time — new columns `pending_bundle_json` `jsonb not null default '{}'` (CHECK object; the default is a migration-validity shim only — retirement is behavioral: the sole writer ALWAYS snapshots a real bundle, so no row ever relies on it) and `secured_at timestampz` (NULL = pending; set ONLY by the deposit slice's securing path). **This slice mints nothing into inventory** — no inventory/base/reward/movement write anywhere in the scan path.
3. **Scan preconditions:** ship settled OSN in-space (`mainship_space_validate_context` state = `in_space`, which the 0055 constraints tie to `status='stationary'`; in transit / docked / home / legacy all reject), not claimed by another domain (`mainship_space_assert_cross_domain_exclusion` — the 0064 arrival-processor posture, reused not re-derived), within `exploration_scan_radius` (new `game_config` tunable, default **750** — same order as the world's port/proximity scales, tunable without redeploy) of an `is_active` site the player has not discovered. Nearest-first, deterministic tie-break (distance, then name).

Work done — one new migration `20260618000099_exploration_p11_scan_command.sql` (head bump **0098 → 0099**):

- `osn_distance` — `language sql immutable strict`; internal posture (no client grant; `service_role` for CI parity with the S2 helpers).
- `exploration_discoveries` + **pending columns** (decision 2 above).
- `exploration_scan(p_player, p_main_ship_id, p_request_id)` — PRIVATE service-role/internal writer; the **sole writer** of `exploration_discoveries`. Ordered body: (1) **DARK GATE FIRST** — `if not cfg_bool('exploration_enabled')` → `feature_disabled` BEFORE any other read/lock/write (0097 reject-before-any-read law, 0070 idiom); (2) null `request_id` → `invalid_request_id`; (3) S2 canonical blocking lock (ship → fleet → coordinate movement → presence) — `missing_ship` / lock status; (4) ownership from the LOCKED snapshot → `not_owned`; (5) canonical payload hash (no coordinate body — 0064 stop idiom); (6) **receipts idempotency REUSED EXACTLY** — `main_ship_space_command_receipts` (0055), lookup AFTER ship lock + ownership (0064 order): verbatim replay of the first committed `result_json`, or `request_id_payload_conflict`; (7) `validate_context` → `destroyed` / `not_in_space` unless settled `in_space`; (8) cross-domain exclusion → forwarded reason; (9) ship coords under lock; (10) nearest undiscovered active site within radius via `osn_distance` — else `already_discovered` (an in-range active site exists but all are this player's discoveries) or `no_site_in_range`; (11) insert the discovery with the bundle snapshot (`secured_at` NULL); (12) receipt insert atomic with the discovery (`movement_id` null).
- `command_exploration_scan(p_main_ship_id, p_request_id)` — authenticated public wrapper (0083 idiom): `auth` → **anti-probe flag gate** (while dark, identical answer regardless of input — no hidden-site probing) → `mainship_resolve_owned_ship` (selected owned ship or sole-ship shim; UI never trusted) → `delegate` → narrow reason→code/message map. Reason set: `feature_disabled`, `invalid_request_id`, `request_id_payload_conflict`, `missing_ship/not_owned`→`no_ship`, `destroyed`, `not_in_space`, `active_legacy_movement`→`busy_legacy`, `no_site_in_range`, `already_discovered`, else `unavailable`.
- `exploration_scan_radius = '750'` seeded on conflict (key) do nothing.
- **ACL (targeted 0083/0095 idiom):** `osn_distance` + `exploration_scan` revoked from public/anon/ authenticated, granted to `service_role` only; `command_exploration_scan` revoked from public/anon, granted to authenticated — dark today because both the wrapper gate and the writer's first check reject.

Nothing deposits yet — pending bundles only. The deposit-on-arrival wiring through the Slice-A activity-agnostic carrier (`movement_attach_cargo(..., 'exploration')`) is the NEXT slice.

State. Forward-only; migration head **0099**; 0001-0098 unedited. No flag flipped (`exploration_enabled` stays `'false'`; it is read, never written). No frontend change. `docs/SYSTEM_BOUNDARIES.md` synced in the SAME step: matrix + Exploration row now name `exploration_scan` as sole writer via the dark `command_exploration_scan`, enumerate the reused OSN machinery, and a new note records the `osn_distance` leaf ("pure/immutable, consumed downward by activities; movement writers adopt it when next re-defined for real changes"). `main` untouched. `npm run build green`; `verify:m3/verify:m4` fail only on `fetch failed` (no reachable Supabase from this sandbox) and `verify:m45` needs `SUPABASE_SERVICE_ROLE_KEY` — the recorded environmental posture; no code/assertion failure.

Bugs / fixes

- `_(none` — additive dark write path; reuses the proven receipts/lock/validation machinery unchanged.)_

2026-07-04 — EXPLORATION-P11 SLICE C: hidden exploration_sites + per-player exploration_discoveries (tables + seed + RLS only; no RPC, no client path, fully dark)

Request. Exploration domain schema: the hidden static site table and the per-player discovery ledger — tables + seed + RLS only; no RPCs, no processors, no client paths; everything stays dark.

Design decisions (self-approved).

1. **Sites are hidden — server-only read, fail-closed by construction.** exploration_sites is migration-seeded static world data with NO runtime writer and — unlike locations/item_types — NO public read: a hidden site's coordinates must never be client-readable before discovery. RLS is ENABLED with **no client policies at all and no anon/authenticated grant**; future SECURITY DEFINER exploration functions reach it as owner. There is nothing for the UI to hide — the client simply cannot see the table.
2. **Per-player discovery state in its own table** exploration_discoveries with unique (player_id, site_id) (+ a (player_id, discovered_at desc) index). **Sole writer = the Exploration system** (its future RPC/processor — nothing writes it yet). Own-row select only, copying the reward_grants_select_own idiom (0015:18–21); no insert/update/delete policy, no write grant; grant select to authenticated only.
3. **v1 reward semantics: deterministic** reward_bundle_json **per site** in the EXACT pending-bundle shape the carrier already transports ({ "metal": N, "items": [{item_id, quantity}] }, the 0040/0041 shape; CHECK jsonb_typeof = 'object') — reuses the Slice-A activity-agnostic deposit path byte-for-byte with zero new roll logic. Weighted "discovery rolls" are an additive later change and, if they come, must reuse/extract the combat loot-roll helper as ONE shared leaf, never a copy. is_active boolean not null default true lets a bad seed be disabled without deleting world data (no destructive cleanup).

Coordinate representation — copied from OSN, no second convention. Column names space_x/space_y from main_ship_instances (0054:33–36); double precision; finite-only CHECKs via the <> 'NaN':double precision idiom and the immutable world envelope [-10000,10000]^2, both verbatim from main_ship_space_movements (0055:56–63), matching the movement writer's inclusive bounds gate (0057:58–59, 95–96). Seeds use integer-grid values (the 0070 command canonicalizes targets to the integer grid) well inside the envelope — every site is a legal open-space target.

Work done — one new migration 20260618000098_exploration_p11_sites_schema.sql (head bump **0097 → 0098**): the two tables above + five idempotent seeds (natural name unique key + on conflict (name) do nothing — the 0002 world-seed idiom; NOT fixed uuids, matching how sectors/zones/locations seed). Seed inventory (bundles draw ONLY from the Slice-B reward set; metal calibrated to the 0041 combat scale of ~10–40/wave):

- Derelict Listening Post (-1200, 850) — 25 metal, scan_data ×3 (common)
- Shattered Survey Buoy (2100, -1400) — 30 metal, scan_data ×2 + anomaly_shard ×1 (common)
- Anomalous Debris Field (-2600, -1900) — 40 metal, anomaly_shard ×2 (uncommon)
- Silent Foundry Wreck (3300, 2500) — 60 metal, scan_data ×2 + blueprint_fragment ×1 (rare)
- Precursor Vault Signal (-4100, 3600) — 100 metal, anomaly_shard ×1 + artifact_core ×1 (epic)

State. Forward-only; migration head **0098**; 0001–0097 unedited. No function created → no execute-surface relock needed (0054 precedent). No flag added/read/flipped — the feature stays server-rejected behind exploration_enabled=false (0097) with no RPC even existing. docs/SYSTEM_BOUNDARIES.md synced in the SAME step: \$1 matrix gains exploration_sites (Reference/Config; NO runtime writer; **server-only** read) and exploration_discoveries (Exploration future writer; owner-read); \$2 gains the **Exploration** system row with the dark gate inline (like Trade Market) and the carrier-reuse law (movement_attach_cargo(..., 'exploration'), never a parallel deposit path). main untouched; no frontend, no workflow, no verifier change. npm run build green; verify:m3/verify:m4 fail only on fetch failed (no reachable Supabase from this sandbox) and verify:m45 needs SUPABASE_SERVICE_ROLE_KEY — the recorded environmental posture; no code/assertion failure.

Bugs / fixes

- _ (none — additive schema + seed; no writer, no reader, no behavior.)

2026-07-04 — EXPLORATION-P11 SLICE B: reward-item catalog entries + exploration_enabled=false dark flag (foundations only; nothing client-reachable, no behavior change)

Request. Exploration foundations: the reward item catalog entries and the dark capability flag. No gameplay logic, no RPC, no table, nothing reachable by clients.

Design decisions (self-approved).

1. **Reuse the existing item-catalog + player_inventory path** that pirate loot uses (0039/0040/0041) — no new inventory table, no new depositor. reward_grant stays the sole depositor; its item validation is catalog-driven (exists (select 1 from item_types where item_id = ...), 0040:78; same guard in inventory_deposit, 0039:81), so it recognizes the new ids with **zero code change** — seeding the catalog row IS the enablement.
2. **The ACTIVITIES.md §3 exploration reward classes** ("data / shards / blueprint fragments / artifact cores") **become exactly four catalog items** — the smallest closed set covering the documented classes; more variants are additive later. Two classes already had exact catalog matches seeded in 0039 and reserved by ACTIVITIES.md §5 for precisely these later progression drops — blueprint_fragment (progression, rare) and artifact_core (progression, epic) — so they are **REUSED, not re-added** (re-adding them under exploration-specific ids would duplicate catalog concepts — forbidden). Only the two missing classes are seeded: scan_data ('Scan Data', category data, common — the bulk "data" class; the category value is the class name the ACTIVITIES.md row uses; category is unconstrained Reference/Config metadata with no code consumer, grep-verified) and anomaly_shard ('Anomaly Shard', material, uncommon — the "shards" class, named from the exploration ownership row's "anomalies"; deliberately NOT captain_memory_shard, which is captain-progression material, a different concept). Exploration reward set = { scan_data, anomaly_shard, blueprint_fragment, artifact_core }.
3. **Capability flag** exploration_enabled = 'false' — the standard server-authoritative dark gate, copying the 0070/0071 reject-before-any-read idiom verbatim (same posture as trade_market_enabled/trade_relief_enabled). No RPC exists yet; the flag simply exists dark. The migration header states the law: every exploration RPC added in later slices **MUST** check it **FIRST** and reject-before-any-read while false — UI hiding is never the only control.

Work done — one new migration 20260618000097_exploration_p11_catalog_and_flag.sql (head bump **0096 → 0097**): two idempotent item_types rows + one idempotent game_config row, both via the established on conflict ... do nothing seeding idiom (0039 / 0070). No table, no function, no RPC, no frontend, no index — nothing else.

RLS/grants — verified, not assumed. New item_types rows inherit the table-wide public-read posture (item_types_public_read for select using (true) + grant select to anon, authenticated, 0039:23-25); the game_config row likewise (game_config_public_read, 0003:13-15). The items are inert without any exploration RPC. No function created → no execute-surface relock needed (0054 precedent). Also grep-verified: item_types is seeded **ONLY** in 0039 (no later migration adds items or a category constraint), and 0041 produces only 0039-seeded ids — so nothing can mint the new items yet; no loot source references them.

Flag seeded false, nothing client-reachable, no behavior change. No flag set true anywhere; no existing flag value touched. docs/SYSTEM_BOUNDARIES.md NOT edited — verified it enumerates dark gates only inline within per-system rows (Trade Market / Main Ship), and no Exploration system row exists yet (it arrives with the exploration tables in a later slice, which will add the matrix row + gate in the same step); item_types / game_config ownership (Reference/Config, §1) is unchanged by adding rows. docs/ACTIVITIES.md untouched. main untouched; migration head is now **0097**; 0001-0096 unedited. npm run build green; verify:m3/verify:m4 fail only on fetch failed (no reachable Supabase from this sandbox) and verify:m45 needs SUPABASE_SERVICE_ROLE_KEY — the recorded environmental posture; no code/assertion failure.

Bugs / fixes

- _(none — additive Reference/Config seed + dark flag; no writer, no reader, no behavior.)_

2026-07-04 — EXPLORATION-P11 SLICE A: activity-agnostic deposit-on-arrival carrier (reward_source_type; refactor only, combat behavior unchanged, everything dark)

Request. Prerequisite refactor for Phase 11 Exploration: make the pending-bundle → attach → deposit-on-arrival carrier activity-agnostic. No new feature, no behavior change, nothing activated.

Design decision (self-approved). reward_grant(source_type, ...) has been generic since 0015/0040 — the only combat coupling in the engine path (docs/ACTIVITIES.md §2) was at the CARRIER layer: fleet_movements had no source-type column and process_fleet_movements' return branch (latest shipped body: 0030:36) hard-coded reward_grant('combat', ...). **Why:** Exploration (and later Mining) must reuse the exact same pending-bundle → movement_attach_cargo → deposit-on-arrival path — one shared engine carrier, never a parallel deposit system — so the movement row now transports its reward source type instead of the engine assuming combat.

Work done — one new migration 20260618000096_engine_reward_source_type.sql (head bump **0095 → 0096**):

- fleet_movements.reward_source_type — text not null default 'combat' (existing rows backfill to 'combat': every payload-carrying return in flight today IS combat's) + closed domain CHECK ('combat', 'exploration', 'mining', 'trade') matching the docs/ACTIVITIES.md §3 activity ownership table (closed set now; a future activity is an additive constraint change in a new forward-only migration).
- movement_attach_cargo(movement, source, bundle, source_type default 'combat') — the old 3-arg signature is **DROPPED** first (the 0038/0081-0084 signature-evolution idiom; keeping both overloads would make existing 3-arg calls ambiguous),

- then re-created with the defaulted 4th param that writes the column. Every existing caller — `process_combat_ticks`, latest `0046:185`, a 3-arg call bound by name at runtime — keeps working verbatim via the default; combat callers are untouched.
- `process_fleet_movements` — re-created from its latest shipped body (`0030:36`, grep-confirmed: no later migration re-defines it or `movement_attach_cargo`; `0032/0041/0046` only re-define the CALLER `process_combat_ticks`) **byte-identical except** the deposit call — `reward_grant(m.reward_source_type, ...)` instead of the literal `'combat'` — and that call's two-line comment, which claimed combat-specificity and would otherwise contradict the code it annotates.
- **ACL preserved (anti-cheat; no new client execute grants):** the re-created 4-arg `movement_attach_cargo` gets the explicit internal revoke (from `public`, `anon`, `authenticated` — `0093` idiom; the DROP discarded the old signature's ACL); `process_fleet_movements` keeps its ACL through CREATE OR REPLACE with a defense-in-depth re-assert (`0070` idiom). Neither had — nor gains — any client or service_role grant (grep: no `src/`, no client RPC, no verify-runner grant; `cron + SECURITY DEFINER` orchestrators invoke them as owner).

Combat behavior unchanged. Same column default, same attach default, same deposit semantics and timing law (pending while out, secured once on home arrival, forfeited on destruction), same idempotency (`reward_grants` UNIQUE (`source_type`, `source_id`), `0040`). No flag added, read, or flipped; no activity enabled; exploration remains entirely unbuilt/dark after this slice.

State. Forward-only. Migration `0096` is now the highest-numbered file; `0001-0095` unedited. `docs/SYSTEM_BOUNDARIES.md` synced in the SAME step: the Movement row now names `movement_attach_cargo(..., source_type='combat')` as the internal shared carrier, and the §3 return-arrival edge now reads `Reward.grant(reward_source_type, bundle)` (activity-agnostic; today always `'combat'`). Sole writers unchanged: Movement remains the sole `fleet_movements` writer (`movement_attach_cargo` is Movement-owned; `process_fleet_movements` remains the only return-branch writer) and `reward_grant` remains the only depositor — no new cross-system edge, call graph unchanged and acyclic. `docs/ACTIVITIES.md` untouched (it already describes this target state). `main` untouched; no frontend, no workflow, no verifier change. `npm run build green (tsc -b && vite build, 141 modules)`; `verify:m3/verify:m4` fail only on `fetch failed` (no reachable Supabase from this sandbox) and `verify:m45` needs `SUPABASE_SERVICE_ROLE_KEY` — the same environmental posture recorded by the 2026-07-03 entries; no code/assertion failure.

Bugs / fixes

- `_(none — pure carrier refactor; the deposit path's combat literal was a latent Phase-11 blocker, removed before any exploration code exists.)_`

2026-07-03 — Trade-economy cleanup audit: ROADMAP Phase-10 doc-sync (docs-only; one stale figure fixed)

Request. Final auto-cleanup/audit pass for the trade-economy milestone: verify the branch is in the clean end-state the milestone claims (boundaries doc, single docked-location helper, dark-both-sides flags, CI proof wiring, law-doc sync) and fix — narrowly — anything that is not.

Audit result (full evidence trail in the untracked scratch file `TRADE_ECONOMY_CLEANUP_RECON.local.md`): claims (a)–(d) — `SYSTEM_BOUNDARIES` coverage + sole-writers/acyclicity, single `mainship_resolve_docked_location` (defined once in `0092`, no surviving inline copy in `0093-0095` or the proof SQL), dark both sides (`trade_market_enabled/trade_relief_enabled` seeded false + RPC dark-rejects + `TRADE_MARKET_ENABLED=false` client gate; relief has NO client UI — N/A by evidence), and the `trade-v1-proof.yml` posture (feature-branch triggers only, no `environment:`, `permissions:` `contents:` `read, if: always()`) teardown, proof SQL one begin...ROLLBACK txn, no COMMIT) — all ✓ CLEAN. ONE defect: claim (e) — `docs/ROADMAP.md` (Phase-10 cell, line 85) still said "migration head `0092`" and omitted the economy-bootstrap phase entirely, contradicting this log's own "migration head remains **0095**" (CI-wiring entry below).

Work done — exactly one doc-sync edit to the Phase-10 status sentence in `docs/ROADMAP.md`, nothing else:

- Stale "migration head `0092`" → "**0095**".
- The pipeline enumeration now includes the previously-missing 2026-07-03 work: cleanup helper `0092`, `ECONOMY-BOOTSTRAP 0093-0095` (seed capital via `wallet_ensure` + no-softlock relief `market_claim_relief`), and the disposable proof `scripts/trade-economy-bootstrap-proof.{sql,sh}` wired into `trade-v1-proof.yml`. The historical `0073-0084 / 0085-0091` ranges and the "implemented DARK & PR-ready ... all trade flags/gates OFF" meaning are unchanged.

State. Docs-only, forward-only. No code, migration, RPC, frontend, workflow, proof, flag default, or behavior changed — nothing activated; `main` untouched; migration head remains **0095** on `autopilot/20260703-064048`. No `SYSTEM_BOUNDARIES.md` change (no architectural fact changed; §1/§2 already document the `0092-0095` surface — re-verified in this audit). `npm run build green (tsc -b && vite build, 141 modules)`; the remote-DB `verify:m2/m3/m4` runs fail only on `fetch failed` (no reachable Supabase from this sandbox) and `verify:m45/m5` need `SUPABASE_SERVICE_ROLE_KEY` — environmental, no code/assertion failure; `scripts/trade-economy-bootstrap-proof.sh selftest` passes in-sandbox.

Bugs / fixes

- **ROADMAP Phase-10 status stale (doc/law mismatch).** The `0093-0095` + proof + CI slices updated this log but not the ROADMAP status figure written back at head-`0092` time. Fixed by the doc-sync above; no other contradiction found.

2026-07-03 — TRADE-ECONOMY-BOOTSTRAP proof wired into existing trade-v1-proof.yml (disposable-only; no new workflow)

Request. Wire the economy-bootstrap proof into CI by EXTENDING the existing trade-v1-proof.yml — which already spins up ONE disposable stack for all Trading-V1 proofs — rather than adding a parallel workflow that would redundantly start a second throwaway stack. This resolves the "CI wiring is a separate follow-up" note from the prior entry.

Work done — additive edits to .github/workflows/trade-v1-proof.yml only (no new workflow file):

- **selftest job** — added a third step - run: bash scripts/trade-economy-bootstrap-proof.sh selftest after the trade-market-1 selftest (DB-free static check).
- **disposable-matrix job** — added a TRADE-ECONOMY-BOOTSTRAP real-chain matrix step mirroring the trade-market-1 step's exact shape (set -a; . /tmp/sbenv; set +a; bash scripts/trade-economy-bootstrap-proof.sh local), placed AFTER the trade-market-1 matrix and BEFORE the if: always() "Stop disposable stack" teardown so the single throwaway stack is still up when it runs. The new proof is self-rolling-back and order-independent; ordering just keeps the file readable.
- **Truthful references** — the supabase start step name "applies migrations 0001..0092" → "0001..0095"; the top-of-file comment + workflow name: now enumerate the economy-bootstrap proof (seed capital + no-softlock relief floor, 0093..0095) alongside the existing two, keeping the "NEVER production / no environment: / flips no committed flag / disposable local Supabase only" language intact.

Preserved: permissions: contents: read, the concurrency block, the on: triggers (feature branches only — NOT main/any release branch), no environment: on any job, and the if: always() teardown. Reuses the single disposable stack — no second stack started.

State. CI-config only. No migration, no src/, no committed flag changed; dispatches no production/deploy/ verifier/sensitive workflow (dispatching this workflow is a human/CI action, not taken here). No SYSTEM_BOUNDARIES.md change — a CI workflow is not an architectural fact. main untouched; migration head remains **0095**. selftest re-run in-sandbox (DB-free) and passes; the disposable-matrix job needs GitHub-hosted Docker/Supabase (same limitation as the sibling matrix) and was not run here.

Bugs / fixes

- _(none — additive CI wiring; runs existing disposable proofs, changes no product code.)_

2026-07-03 — TRADE-ECONOMY-BOOTSTRAP proof: disposable, self-rolling-back seed + relief exercise (no CI yet)

Request. Add a disposable proof that actually exercises the seed-capital + no-softlock-relief SQL end-to-end — the only way this logic runs, since verify:m* can't reach a live DB in-sandbox. Mirror the trade-market-1-proof.{sh,sql} idiom. Touch no src/ and no migrations.

Work done — two new files under scripts/ (no migration, no committed flag change):

- **trade-economy-bootstrap-proof.sql** — one begin;...rollback; transaction that persists NOTHING (no COMMIT anywhere). Same idiom as the sibling: \set ON_ERROR_STOP on, the pg_temp.call_as(sub, fn) JWT-subject helper, a teb temp fixture table, teb.-prefixed fixture users, the "mirror production config a fresh chain lacks" setup (reveal_starter_ports() + transient mainship_space_movement_enabled='true'), real-RPC provisioning (commission_first_main_ship()), and owner-level insert into public.player_wallet (player_id, balance) / insert into public.ship_cargo_lots ... for state setup (harness runs as DB owner, bypassing RLS; all reverted by ROLLBACK). Both dark flags (trade_market_enabled, trade_relief_enabled) are toggled ONLY inside the txn. Asserts, each ending in a raise notice PASS marker:
 SEED: SEED_PASS_DARK (wallet-less buy while trade dark → trade_market_disabled, no wallet seeded), SEED_PASS_APPLIED (first buy seeds starting_credits=1000 once then debits → balance 1000-T), SEED_PASS_ONCE (2nd buy debits further; balance never returns to 1000 — wallet_ensure's on conflict do nothing is unfarmable).
 RELIEF anti-farm matrix: RELIEF_PASS_DARK (rock-bottom claim while relief dark → trade_relief_disabled, no claim, wallet 0), RELIEF_PASS_NO_WALLET (wallet-less → no_wallet, still no wallet — proves relief never calls wallet_ensure, closing the seed+relief double-grant hole), RELIEF_PASS_WALLET_NOT_EMPTY, RELIEF_PASS_CARGO_NOT_EMPTY, RELIEF_PASS_GRANT (0 → relief_credits=250, exactly one claim @ 250), RELIEF_PASS_IDEMPOTENT (replay → idempotent_replay, no 2nd claim/credit), RELIEF_PASS_COOLDOWN (relief_cooldown_active + next_eligible_at), RELIEF_PASS_CAP (cooldown transiently 0; 3 grants then 4th → relief_cap_reached). Ends with the TRADE-ECONOMY-BOOTSTRAP PROOF PASSED line, then rollback;.
- **trade-economy-bootstrap-proof.sh** — mirrors the sibling's two modes. selftest (DB-free): verifies the .sql is self-rolling-back (opens a txn, last verb is rollback; no COMMIT), toggles both dark flags strictly inside the txn, provisions

via the real RPCs (`commission_first_main_ship/market_buy/market_claim_relief`), sets up a wallet via an owner insert, contains every PASS marker, asserts the key reason tokens (`trade_market_disabled/trade_relief_disabled/no_wallet/wallet_not_empty/cargo_not_empty/ idempotent_replay/relief_cooldown_active/relief_cap_reached`), and references neither `src/` nor migrations/; prints an ALL-PASSED line. `local` (against a disposable `DB_URL`): `psql -X -v ON_ERROR_STOP=1 -f the .sql`, require the final PASS line + every marker, print `OVERALL_PASS`.

Self-rolling-back; persists nothing; flips no committed flag. The whole proof runs inside one rolled-back transaction — no wallet, lot, claim, ship, fixture user, or flag flip survives. The dark flags are enabled only transiently inside the txn to exercise the capabilities; production/committed defaults stay false. Relief credits are never injected directly — the GRANT case drives the real `market_claim_relief` RPC.

State. No migration added/edited; no `src/`; no committed flag changed. **CI wiring is a separate follow-up** (would mirror the existing `trade-*-proof` CI idiom). No `SYSTEM_BOUNDARIES.md` change — a proof script is not an architectural fact. `main` untouched. `selftest` was run in-sandbox (DB-free) and passes; `local` needs a disposable Supabase (same environmental limitation as `verify:m*`) and was not run here.

Bugs / fixes

- `_` (none — new disposable proof; exercises existing DARK logic, changes no product code.)

2026-07-03 — TRADE-MARKET-1 no-softlock floor: relief claim RPC `market_claim_relief` (DARK; server-rejected)

Request. The relief floor's writer: a Trade-Market orchestrator that grants `relief_credits` to a genuinely softlocked player and records the claim. Forward-only; ships DARK; no flag flipped true.

Work done — one new migration `20260618000095_trade_market_1_claim_relief.sql` (head bump **0094** → **0095**): `public.market_claim_relief(p_request_id uuid)` returns `jsonb` (`plpgsql` / security definer / `search_path = public;` market_buy idiom + ACL: revoke ... from public, anon; grant ... to authenticated). It is the **sole writer** of `trade_relief_claims`. Ordered body: (1) `auth.uid()` → `not_authenticated`; (2) **DARK reject before any read** if `not cfg_bool('trade_relief_enabled')` → `trade_relief_disabled`; (3) `p_request_id` is null → `invalid_request`; (4) **account lock + rock-bottom read** `select balance ... for update` on the EXISTING wallet row, NOT FOUND → `no_wallet`; (5) idempotency on (player, request_id) → verbatim replay, no re-grant; (6) `balance <> 0` → `wallet_not_empty`; (7) cargo sum across ALL the player's ships (`ship_cargo_lots` $\bowtie$ `main_ship_instances`) `<> 0` → `cargo_not_empty`; (8) lifetime cap count `>= cfg_num('relief_max_lifetime_claims')` → `relief_cap_reached`; (9) `cooldown last > now()` - `cfg_num('relief_cooldown_seconds')` → `relief_cooldown_active` (+ `next_eligible_at`); (10) grant `cfg_num('relief_credits')` through `wallet_credit`; (11) insert the claim, return `{ok, claim_id, amount, claimed_at}`.

Anti-farm design (the "no wallet_ensure in relief" rule). `wallet_credit` now routes through `wallet_ensure`, which seeds `starting_credits` (1000). If relief ensured a wallet, a rock-bottom player with NO wallet row would be seeded 1000 **plus** granted 250 relief — a farming hole. So relief **requires an EXISTING** `player_wallet` row (reason `no_wallet` when absent) and **never calls** `wallet_ensure`: a player with no row hasn't entered the economy and gets the normal seed on first trade, not relief. The rock-bottom read is `SELECT balance ... FOR UPDATE` on that existing row, giving a natural **per-account lock** — every check and the ledger write run under it, so distinct-request_id races cannot bypass the cap/cooldown. Relief fires only at exact rock-bottom (`balance = 0` AND zero cargo across all ships), bounded by the lifetime cap and cooldown.

Boundaries. Trade Market is the sole `trade_relief_claims` writer; the balance write flows only through `wallet_credit`, preserving `player_wallet`'s sole-writer invariant. All of `player_wallet` (FOR UPDATE), `ship_cargo_lots`, `main_ship_instances` are DOWNWARD reads — no new cross-system edge, no cycle, Wallet stays a downward leaf.

State. Forward-only. Migration `0095` is the highest-numbered file; `0001-0094` unedited. Ships **DARK/server-rejected** (`trade_relief_enabled=false`) — no flag default flipped. `docs/SYSTEM_BOUNDARIES.md` synced in the SAME step (Trade Market row names `market_claim_relief` as sole `trade_relief_claims` writer + records the downward reads; acyclic-invariant note updated). `main` untouched. No frontend, no workflow, no verifier, no engine (M2/M3/M4/M4.5) change.

Bugs / fixes

- `_` (none — additive DARK RPC; the seed+relief double-grant hole is closed by design via the existing-wallet-row requirement, not patched after the fact.)

2026-07-03 — TRADE-MARKET-1 no-softlock floor: relief ledger + tunables + dark flag (schema slice; NO RPC)

Request. Schema/config/flag slice of the no-softlock relief floor: add the relief ledger table, its tunables, and a dark gate — no RPC and no writer yet. Forward-only; ships DARK; no flag flipped true.

Ownership decision (planner authority). The relief ledger + orchestrator belong to **Trade Market**, not Wallet (overriding the scope-lock's tentative "Wallet-owned ledger" phrasing — table ownership is a design detail within scope). Trade Market is the economy orchestrator that ALREADY fans out downward to Wallet (credit), Trade Cargo (lots), and Main Ship (read); siting relief there introduces **zero new cross-system edges** and keeps Wallet a pure downward leaf. Making Wallet orchestrate relief would force Wallet to read Trade Cargo + Main Ship and stop being a leaf. Mirrors the existing `trade_receipts` table + `market_buy/market_sell` RPCs. The relief credit is granted THROUGH `wallet_credit`, so Wallet stays the sole `player_wallet` writer — Trade Market never writes `player_wallet` directly.

Work done — one new migration `20260618000094_trade_market_1_relief_claims.sql` (head bump **0093** → **0094**):

- `public.trade_relief_claims` — Trade-Market-owned, per-player idempotent relief ledger: `claim_id` (pk), `player_id` (fk → `auth.users`, on delete cascade), `request_id`, `amount` (check `>= 0`), `claimed_at`, unique (`player_id`, `request_id`) idempotency key, and a (`player_id`, `claimed_at`) index for the cooldown / lifetime-cap lookups the RPC will do. RLS enabled; owner-read policy `player_id = auth.uid()`; grant `select` to authenticated (NOT anon); **no** insert/update/delete policy and **no** write grant → Trade Market will be the sole writer via the forthcoming SECURITY DEFINER RPC. Account-scoped (keyed by `player_id`, not ship) because relief is account-level softlock recovery; RLS/comment idiom matches `trade_receipts` (0086).
- **Three tunables** (placeholders) via `on conflict (key) do nothing`: `relief_credits=250` (grant per claim), `relief_cooldown_seconds=86400` (24h minimum spacing — prevents rapid re-farming), `relief_max_lifetime_claims=3` (lifetime cap per player — bounds total relief while still guaranteeing genuine-softlock recovery).
- **Dark flag** `trade_relief_enabled='false'` via `on conflict (key) do nothing`. The relief RPC (next step) is server-rejected until this flips; it stays false here — no flag set true.

No writer exists yet. The table starts DARK with no writer — exactly as `player_wallet/trade_receipts` did in 0086. Nothing reads or writes `trade_relief_claims` yet; the sole writer arrives with the relief RPC in the next slice, itself gated by `trade_relief_enabled=false`.

State. Forward-only. Migration 0094 is the highest-numbered file; 0001-0093 unedited. No new cross-system edge, no cycle — Wallet remains a downward leaf and `player_wallet`'s sole-writer invariant is preserved (relief credits flow through `wallet_credit`). `docs/SYSTEM_BOUNDARIES.md` synced in the SAME step (ownership matrix + Trade Market section + acyclic-invariant note). `main` untouched. No frontend, no workflow, no verifier, no engine (M2/M3/M4/M4.5) change. No flag default flipped true.

Bugs / fixes

- `_(none — additive schema/config/flag slice; no writer, no behavior change.)_`

2026-07-03 — TRADE-MARKET-1 seed capital: `starting_credits` tunable + single shared `wallet_ensure` (DARK)

Request. Seed-capital slice of the Trading V1 economy bootstrap: add a `starting_credits` tunable seeded into a wallet on first creation, and collapse the two copies of the Wallet "lazy ensure" block (inline in `wallet_debit` 0089 and `wallet_credit` 0090) into ONE shared helper. Forward-only; ships DARK; no flag default flipped.

Work done — one new migration `20260618000093_trade_market_1_wallet_seed_capital.sql` (head bump **0092** → **0093**):

- `starting_credits = '1000'` added to `game_config` via the `on conflict (key) do nothing` numeric-seed idiom (0003). Placeholder economy value; an inert tunable until a wallet is actually created.
- `wallet_ensure(player)` — the ONE shared lazy-ensure + seed: `insert into public.player_wallet (player_id, balance) values (p_player, coalesce(cfg_num('starting_credits'),0)::numeric) on conflict (player_id) do nothing`. Seeds the starting balance exactly once on first creation; idempotent + **unfarmable** by the `player_id` primary-key conflict (a re-call is a no-op — the row is only ever inserted once). Internal (revoke execute ... from public, anon, authenticated), security definer, set `search_path = public`.
- `wallet_debit` **de-duplicated**: former inline `insert ... on conflict do nothing` → perform `wallet_ensure(...)`; the existing atomic conditional update ... where `balance >= p_amount` and return `found` are left exactly as-is. Behavior preserved: seed on first touch, then race-safe conditional debit that can never overdraw.
- `wallet_credit` **de-duplicated**: reworked from its upsert-add into **ensure-then-add** — perform `wallet_ensure(...)` then an unconditional `update ... set balance = balance + p_amount`. The ensure guarantees the row exists (seeded on first creation), then the amount adds on top — credit semantics preserved, second copy of the ensure logic removed. This is the de-duplication target: the ensure block now lives in exactly one place.

Ships DARK — no flag flipped. The seed only ever fires when a wallet is first created, and every wallet-creation path is already server-rejected: `market_buy/market_sell` under `trade_market_enabled=false`, and the additional-ship commission debit under

`mainship_additional_commission_enabled=false`. So no wallet — and thus no seed — occurs while trade/commission stay dark. No flag default changed.

State. Forward-only. Migration 0093 is now the highest-numbered file; 0001-0092 unedited. Wallet stays a **downward leaf** (reads `cfg_num('starting_credits')` from Reference/Config — a DOWNWARD read; no new cycle, no new writer to any non-Wallet table). `docs/SYSTEM_BOUNDARIES.md` Wallet row synced in the SAME step (names `wallet_ensure` as the shared lazy-ensure+seed, records the config read, drops the stale `wallet_credit` "lazy ensure" phrasing). `main` untouched. No frontend, no workflow, no verifier, no engine (M2/M3/M4/M4.5) change.

Bugs / fixes

- `_(none — clean de-duplication + additive tunable; behavior preserved on both wallet writers.)_`

2026-07-03 — Docs-only roadmap reconciliation: Phase-10 label + live migration head (no code/flag change)

Request. Reconcile the Phase-10 (row 10 [wait]) cell in `docs/ROADMAP.md` (line 85) with its own appended status: replace the stale leading label `**designed, NOT built.**` and bump the stale live "migration head" figure. Docs-only; touch nothing else.

Work done — exactly two edits to the Phase-10 cell, nothing else in it:

- **Label** `designed, NOT built` → `implemented DARK, NOT activated`. The cell's own appended note already reads **"implemented DARK & PR-ready ... all trade flags/gates OFF"**, so the leading "NOT built" clause was factually wrong (the pipeline IS built, only un-activated). The new label preserves the "not live" meaning while removing the contradiction.
- **Live "migration head 0091"** → **"migration head 0092"**. The docked-location-helper migration `20260618000092_trade_market_1_resolve_docked_location.sql` was added after that status note was written, so the live head figure was stale. The historical `TRADE-MARKET-1 0085-0091` range is **left untouched** — it correctly describes TRADE-MARKET-1's original migration set; 0092` is the later cleanup helper and only the live head figure was stale.`

State. Docs-only, forward-only. No code, migration, RPC, frontend, workflow, test, verifier, flag default, or behavior changed — nothing activated. Locked scope was `docs/ROADMAP.md` + this `docs/DEV_LOG.md` entry only. No `SYSTEM_BOUNDARIES.md` sync needed (no architectural fact changed — no table/writer/constraint/call-graph change). Migration head remains **0092** on `autopilot/20260703-064048; main` untouched. No build/test run is required (no runtime surface); the M2/M3/M4/M4.5 engine tests are unaffected.

Bugs / fixes

- `_(none — docs reconciliation only.)_`

2026-07-03 — Trading V1 cleanup: CI proof workflow `trade-v1-proof.yml` (disposable DB only; no production)

Request. Wire the two already-existing Trading-V1 proofs into CI. Add ONE workflow that runs both against a throwaway/disposable Supabase only — never production, flipping no flag. Reuse the `port-entry-1-proof.yml` idiom.

Work done

- **New workflow** `.github/workflows/trade-v1-proof.yml` (modeled on `port-entry-1-proof.yml`). One workflow, one disposable stack for both proofs:


```
<font name="Sym"> </font> selftest job — DB-free static checks: bash scripts/trade-fleet-0c-proof.sh selftest +
bash scripts/trade-market-1-proof.sh selftest.
<font name="Sym"> </font> disposable-matrix job — supabase start (applies the full local chain 0001..0092, incl. the new
shared docked-location helper), exports the disposable DB_URL via supabase status -o env into a tmp env file (no secrets),
then runs trade-fleet-0c-proof.sh local then trade-market-1-proof.sh local, and an if: always() supabase stop
--no-backup || true.
<font name="Sym"> </font> on: workflow_dispatch + push to autopilot/**, trade-**, trade-market-**, trade-fleet-** —
not main / any release branch. permissions: contents: read; concurrency on github.ref with cancel-in-progress: true.
No environment: on any job → no job can read production secrets.
```
- **Flips NO committed flag.** Both proofs are self-rolling-back: they enable the dark trade capabilities ONLY inside a txn that ends in ROLLBACK (no COMMIT), so the committed flag defaults (`trade_market_enabled`, `mainship_additional_commission_enabled = false`) are untouched. Disposable local Supabase only — never prod.

State. Additive CI wiring only. No proof `.sql/.sh`, migration, flag default, `MarketPanel`, or boundary-doc change (a CI workflow is not an architectural fact, so `SYSTEM_BOUNDARIES.md` needs none). Migration head unchanged at **0092**. Not dispatched/triggered

(a human/CI action); `main` untouched. Both `selftest` invocations pass locally (DB-free); the `disposable-matrix` job needs GitHub-hosted Docker/Supabase and was **not** run in-sandbox.

Bugs / fixes

- `_(none — additive CI wiring around existing proofs.)_`

2026-07-03 — Trading V1 cleanup: extract shared docked-location helper (migration 0092; behavior-identical)

Request. The identical ~10-line "resolve docked location" block was copy-pasted verbatim into `get_market_offers` (0087), `market_buy` (0089), and `market_sell` (0090). Extract ONE shared helper and repoint the three RPCs, in a NEW forward-only migration — never editing 0087/0089/0090; behavior-identical; DARK.

Work done

- **New migration** `20260618000092_trade_market_1_resolve_docked_location.sql`. Adds `public.mainship_resolve_docked_location(uuid)` returns `uuid` (security definer, `set search_path, stable, read-only`): calls `mainship_space_validate_context`, requires `ok + state='at_location'`, then reads the present/location fleet's `current_location_id` — returns that id or NULL. Both original "not docked" null paths collapse to one NULL, which each caller maps to the same `{ok:false, reason:'not_docked'}` → behavior-identical.
- **Repointed all three RPCs** via `create` or `replace` (supersedes 0087/0089/0090 forward-only; those files are untouched). Each body is byte-for-byte its original except (a) the inline block → the helper call, and (b) the now-unused `v_ctx jsonb`; local dropped (dead after extraction). Flag gate, `mainship_resolve_owned_ship` ownership assert, per-ship lock, request-id idempotency, offer/volume/cargo checks, and all wallet/cargo/receipt writes are unchanged.
- **ACL — INTERNAL (deviation from the step's suggested grant authenticated, on security grounds)**. The helper is revoked from `public/anon/authenticated` (no client grant), matching its true siblings `mainship_space_validate_context` / `mainship_resolve_owned_ship`. It does NOT assert ownership (the orchestrators do, before calling it); granting it to `authenticated` would create a new client-callable SECURITY DEFINER read that leaks any ship's dock. It is called only inside the SECURITY DEFINER trade RPCs (which run as owner), so the internal ACL changes no call path.
- **Law-doc sync (same step)**. `SYSTEM_BOUNDARIES.md`: named the helper in the Main Ship §2 row (shared read-only docked-location helper, internal, called DOWNWARD by Trade Market) and in the Trade Market row's docked-context read; extended the acyclic-fan-out note with the (pre-existing) Trade Market → Main-Ship-read edge, now a single named function.

State. Migration head now **0092**. No flag/behavior change; feature stays **DARK** (`trade_market_enabled`, `TRADE_MARKET_ENABLED`, `mainship_additional_commission_enabled`, `MAINSHIP_ADDITIONAL_ENABLED` all OFF). No migration ≤ 0091 edited; `main` untouched; not applied to production.

Bugs / fixes

- `_(none — pure de-duplication; three verbatim copies → one helper, behavior-identical.)_`

2026-07-03 — Trading V1 cleanup pass: SYSTEM_BOUNDARIES doc-sync (docs-only; no behavior/flag change)

Request. Bring `docs/SYSTEM_BOUNDARIES.md` back in sync with the actual schema after the TRADE-FLEET-0C / TRADE-MARKET-1 migrations (0073–0091). Docs-only; touch no code, migration, RPC, workflow, or flag.

Work done

- **Corrected the stale one-ship-per-player claim**. §4 item 7 (and the §2 Main Ship row) asserted `main_ship_instances` had one row per player via a `player_id` UNIQUE. That UNIQUE (`main_ship_instances_player_id_key`) was **dropped in migration 0079** — a player MAY now own multiple ships. Both spots now state multi-ship is structurally allowed but stays **DARK**: sole-ship is a runtime shim / dark gate (`mainship_additional_commission_enabled=false`), not a schema constraint.
- **Documented the four new tables in the §1 ownership matrix** with their real sole-writers: `trade_goods` = **Reference/Config** (Trade Market static catalog; admin/migration, seed-only), `ship_cargo_lots` = **Trade Cargo**, `player_wallet` = **Wallet**, `trade_receipts` = **Trade Market**.
- **Added the three new systems to the §2 contract: Wallet** (downward leaf; `wallet_debit/wallet_credit` — both Main Ship (add-ship `main_ship_price` debit) and Trade Market (buy debit / sell credit) depend DOWNWARD on it, Wallet depends on nothing above → acyclic, no mutual dependency); **Trade Cargo** (`trade_cargo_add_lot/trade_cargo_consume` — per-ship volume-keyed lots; a leaf Trade Market depends on); **Trade Market** (`trade_receipts`; orchestrates buy/sell fanning out

DOWNWARD to Wallet + Trade Cargo, reads `trade_goods` + docked context; DARK while `trade_market_enabled=false`).

Added an acyclic-fan-out note confirming exactly one sole-writer per table and no second writer anywhere.

State. Docs-only. **No** migration/RPC/MarketPanel/workflow/flag change; migration head unchanged at **0091**. The trade feature stays **DARK** (`trade_market_enabled`, `TRADE_MARKET_ENABLED`, `mainship_additional_commission_enabled`, `MAINSHIP_ADDITIONAL_ENABLED` all OFF); `main` untouched.

Bugs / fixes

- `_` (none — a law-doc that contradicted the schema was corrected; no behavior path changed.)

2026-07-03 — TRADE-UI-1 landed DARK + PR-ready (ship-switcher + buy/sell + \$2.5 sole-ship shim retirement)

Request. Complete **TRADE-UI-1** on autopilot/20260703-064048: the client trading surface (ship switcher, market buy/sell) and the **\$2.5 sole-ship shim retirement** (the UI passes an explicit `p_main_ship_id`). Additive, gated **OFF**, behavior-preserving; no migration/DB/verifier/workflow/flag change; `main` untouched.

Work done

- **Client trade surface (DARK).** Selected-ship model `useMainShipSelection` (owner-reads `main_ship_instances`, auto-selects the sole ship, N-ship-ready); `ShipSwitcher` (selection-only; a single ship renders as a non-interactive sole entry); `MarketPanel` read view (wallet, occupied cargo m³ vs capacity, station offers) **plus per-offer buy/sell** wired to `market_buy` / `market_sell` — each intentional click is one idempotent command keyed by a fresh `crypto.randomUUID()`, a **synchronous in-flight ref** guards against double-submit, and a success re-reads wallet/cargo/offers via `refresh()`. Fail-closed server reasons map through the pure `tradeReasonMessage`. Everything mounts only behind `TRADE_MARKET_ENABLED = false` and is **double fail-closed** against the server `trade_market_enabled` flag (also false — the trade RPCs reject before any ship read).
- **\$2.5 sole-ship shim retirement.** The client now sends an explicit `p_main_ship_id` at ⑤ port move-to-location, ④ space-stop, ③ movement-readiness, ② dock-services, ① repair, and ⑦ normalize-dock. Each is behavior-preserving; with one ship the sourced id equals the shim-derived sole ship; a transitional `null` still resolves via the server `count = 1` shim; ownership is server-asserted, so an explicit id can only ever act on the caller's own ship. ⑥ `command_main_ship_space_move` is **deferred by design** — its RPC intentionally never took `p_main_ship_id` in TRADE-FLEET-0C (it rejects at the coordinate gate before any ship read).
- Delivered as six small, independently-reviewable commits (map hooks/panels; plus `dashboard/MainShipPanel.tsx` for repair and `portentry/` for normalize under a deliberately-widened frontend scope, id-threading only).

State. Migration head **unchanged at 0091** — TRADE-UI-1 touched **no** migration/DB/verifier/workflow. The feature is **DARK and PR-ready** on autopilot/20260703-064048: buildable, **not deployed, not verified in production**. All trade / add-ship gates + flags remain **OFF**: `TRADE_MARKET_ENABLED`, `MAINSHIP_ADDITIONAL_ENABLED`, `trade_market_enabled`, `mainship_additional_commission_enabled`, `mainship_coordinate_travel_enabled`.

Human-gated follow-ups (NOT done, by design)

- **Activate trading:** flip `trade_market_enabled` + `TRADE_MARKET_ENABLED` (and, for the multi-ship add-ship path, `mainship_additional_commission_enabled` + `MAINSHIP_ADDITIONAL_ENABLED`).
- **Server-side removal of the sole-ship shim** — a future migration, only once the UI-explicit-id path is merged.
- **Run the rendered .uispec.ts suites in CI** — this sandbox lacks the browser binary (`chrome-headless-shell`).
- **Small react-hooks lint-debt cleanup** — documented pre-existing suppressions in `usePortEntry.ts` and `useDockServices.ts` (a `useState`-initializer refactor; out of scope for the id-threading commits).

Bugs / fixes

- `_` (none — additive dark UI + behavior-preserving id threading; no production code path changed.)

2026-07-03 — Repo/docs sync + PORT-ENTRY player UI landing recorded (no new build)

Request. Pull `main` current on the local machine and bring the project docs (log, guide, PDFs) up to date.

Work done

- Synced local `main` (fast-forward **22 commits** → `f48bc53`). No code written this session.
- Recorded that the **PORT-ENTRY player UI** (PR #65, `cb0d4fe`) is **merged** — the player-facing **Claim First Ship + Finish Docking (normalize)** panel (`src/features/portentry/PortEntryPanel1.tsx` + hooks) now exists, **frontend-only**, calling the migration-0072 RPCs; no new migration.

- Refreshed the guide **Current project snapshot** with a 2026-07-03 note (main head → f48bc53, PORT-ENTRY UI merged, Trading V1 FIXED to volume-only, TRADE-FLEET-0A audit recorded via PR #66).

State. Migration head **unchanged at 0072**; coordinate travel stays **DARK** (mainship_coordinate_travel_enabled = false). Next planned: **TRADE-FLEET-0B** (user-approved multi-ship + volume-cargo contract — design/approval only). Trading V1 not started.

Bugs / fixes

- `_`(none — docs/sync only; no code path touched.)

2026-07-02 — Trading V1 design record — FIXED product direction (volume-only per-ship cargo + multi-ship foundation) + TRADE-FLEET-0A read-only audit (DESIGN RECORD ONLY; nothing built)

Request. Do **not** begin Trading implementation. Fix the Trading V1 product direction (below) as binding for design, and produce **TRADE-FLEET-0A** — a strict read-only impact audit for introducing **multiple persistent main ships** and **ship-bound, volume-based cargo**. No branch, PR, migration, code, seed, workflow, deployment, or production-state change; PORT-ENTRY, coordinate-travel, flags, and movement are untouched (mainship_coordinate_travel_enabled stays **false**). Migration head remains 0072.

Supersession note. This direction **replaces** the earlier same-day draft that used **kilograms + cubic metres (dual mass+volume caps)** and allowed **same-port ship-to-ship transfer**. The **FIXED** model is **volume-only (m³)**, and **cargo transfer between ships is OUT of Trading V1 scope**. Mass / density / fuel / acceleration / handling are **future-only**, not part of this foundation.

Fixed direction (binding for design):

1. **Multi-ship from the start.** Multiple persistent main ships are a **Trading foundation**, not a later module/captain feature. A player may eventually own and operate several main ships **concurrently** (one docked & trading while another travels or docks elsewhere).
2. **Cargo is ship-bound.** Trade cargo is physically assigned to **one** ship; it moves only when that ship moves; it is **never pooled** across a player's ships. **No** account-level trade inventory. **No** remote buy/sell and **no** cargo teleportation.
3. **Volume-only capacity (m³).** Canonical storage + validation unit is **cubic metres**. Player-facing display may use m³ (and litres for small amounts). **No** abstract cargo units. **No** kilograms / mass / density / dual mass+volume in Trading V1 (those are explicitly future-only).
4. **Commodities have a defined physical volume.** Trade denominations (crate / pallet / tank / container / bundle...) each resolve to a **fixed canonical m³**; the capacity rule is **occupied volume only**.
5. **Every market action targets one selected ship** — owned by the player, physically **docked** at the relevant port, in an eligible state; buy/sell operate only on **that ship's** cargo.
6. **Coordinate travel stays dark.** Existing **port-to-port** travel is sufficient for the first economy; no coordinate-travel activation, change, or dependency is recommended.
7. **Out of V1 scope:** pooled fleet cargo; account-level trade inventory; remote market actions; **cargo transfer between ships**; port warehouses; automated trade routes; player-to-player trading; dynamic supply/demand; cargo loss / piracy / insurance / destruction economics; mass / density / fuel / acceleration / handling.

Implementation sequence (design-level; unchanged ordering, cargo model corrected to volume-only):

```
PORT-ENTRY (complete, mig 0072)
→ TRADE-FLEET-0A  read-only impact audit (this entry — design record only)
→ TRADE-FLEET-0B  explicit user-approved multi-ship + volume-cargo contract (design/approval only)
→ TRADE-FLEET-0C  coherent implementation slice (multi-ship + ship-bound volume-only m³ cargo, one slice)
→ TRADE-MARKET-1  server-authoritative market (offers, wallet, atomic volume-checked buy/sell vs a selected ship)
→ TRADE-UI-1      selected-ship market + fleet interface
```

TRADE-FLEET-0A audit (read-only). The full impact audit — every current one-main-ship assumption (DB / backend / frontend / verifier / onboarding) classified mandatory / compatibility-sensitive / optional / not-affected; cargo-locality guarantees; a minimal design-level data boundary; multi-ship concurrency & safety; compatibility/migration risks across all ship states; affected frontend surfaces; verifier implications; blockers; open decisions; and a recommended slice order — is recorded in [docs/TRADE_FLEET_0A_IMPACT_AUDIT.md](#). Key finding: the locking/idempotency substrate is **already ship-scoped** (mainship_space_lock_context(main_ship_id), no advisory/player lock; idempotency keyed (main_ship_id, request_id)); the only hard single-ship blockers are the `main_ship_instances.player_id` **UNIQUE** constraint and the uniform `where player_id = v_player` ship derivation.

Work done

- DEV_LOG (this entry) + ROADMAP Phase 10 row and Standing Law #1 annotated with the **FIXED** (volume-only) direction.
- New read-only audit doc `docs/TRADE_FLEET_0A_IMPACT_AUDIT.md` (replaces the superseded kg+m³ draft audit).

Bugs / fixes

- `_`(none — design record only; no code path touched)

2026-06-30 — OSN-COORD-ENABLE (dark) → PORT-ENTRY-1 first-ship commission/normalize → production verifier (head 0070 → 0072)

Since the entry below (head 0070, OSN port-to-port live, coordinate travel server-disabled) the project built the coordinate-travel capability **end-to-end and left it DARK**, then shipped the **first-ship / port-entry** backend (the Trading prerequisite), then added a dedicated production verifier for it. **Net production change:** migration head 0070 → 0072; **no flag flipped** — `mainship_coordinate_travel_enabled` stays **false**, coordinate UI hidden, raw coordinate command server-rejected, port-to-port unchanged/enabled. `main` head a947c8d.

Work done (in order):

- **OSN-COORD-ENABLE-1B (migration 0071, PR #57, deployed DARK).** Extended the authenticated read-model `get_osn_movement_readiness()` with one additive boolean `coordinate_travel_available = osn_available AND cfg_bool('mainship_coordinate_travel_enabled')` — derived from the existing anchored-origin decision, false for every caller while the gate is false. Disposable 2x2 truth-table proof; gated deploy.
- **OSN-COORD-ENABLE-1B-VERIFY (PR #58).** Repinned the read-only post-enable verifier to head 0071 + a single-RPC readiness-capability contract probe. Production read-only run: `OVERALL_PASS=true`.
- **OSN-COORD-ENABLE-1C (PR #59, Pages-deployed).** The frontend empty-space coordinate UI is now driven SOLELY by the server-derived `coordinate_travel_available` (strict fail-closed parser + `isCoordinateTargetingActionable`); the compile-time `OSN_COORDINATE_TRAVEL_ENABLED` constant is retired as the UI authority. **Effect:** when the server flag is later flipped true, the coordinate UI lights up with no redeploy; until then it stays dark. Live bundle independently verified dark.
- **PORT-ENTRY-1 (migration 0072, PR #61, deployed).** First-ship commissioning + same-location dock normalization — the Trading prerequisite. `port_entry_commission_writer(uuid)` (service-role-only) inserts a new player's ship DIRECTLY into canonical `at_location` at Haven Reach; `commission_first_main_ship()` (authenticated, zero-arg) outcome matrix A-F; `normalize_main_ship_dock()` (authenticated) upgrades a coherent legacy_present ship in place. Two-phase lock protocol; proven with a real two-session concurrency race (B blocks on the `player_id` unique conflict until A commits). Additive function-only; no flag/data/coordinate change. **No player-facing UI yet.**
- **PORT-ENTRY-1-VERIFY-1 (PR #62, merged — tooling only).** A dedicated, dispatch-only, production-gated read-only verifier proving production contains exactly the three PORT-ENTRY functions (signatures, bodies via raw `pg_proc.prosrc` md5, SECURITY DEFINER, search_path, ACLs) AND the **complete** authenticated client-RPC inventory (exact 20-RPC set by OID). Disposable proof passes + fails closed for 8 mutation cases. **Not yet run against production** (the gated run is the next human-approved checkpoint).

Current authoritative state (HELD): head 0072; `mainship_send_enabled=true`, `mainship_space_movement_enabled=true` (port-to-port enabled), `mainship_coordinate_travel_enabled=false`, `coordinate_travel_available=false`. Coordinate travel and Trading V1 are **not** started; PORT-ENTRY player UI is the next active development.

2026-06-29 — OSN enabled → Phase 9 docked-port surface → coordinate-gate hardening → Phase 10 Trading design (head 0068 → 0070)

Since the PORT-LAUNCH entry below (head 0068, ports public, OSN still dark) the project advanced through OSN enablement, a first player-facing port surface, a coordinate-travel security fix, and a full Trading V1 design pass. **Net production change:** migration head 0068 → 0070; **OSN port-to-port travel is now ENABLED; free arbitrary-coordinate travel is server-disabled by default.** Current live flags: `mainship_send_enabled = true`, `mainship_space_movement_enabled = true` (port-to-port ON), `OSN_COORDINATE_TRAVEL_ENABLED = false` (frontend) + `mainship_coordinate_travel_enabled = false` (server, new in 0070). `main` head 6e2a091.

Work done (in order):

- **OSN enablement (config-only; head stays 0068).** The dark OSN port-to-port path was turned on via the controlled one-shot enable operation (`mainship_space_movement_enabled false→true`), independently read-only verified against production, and a disposable authenticated port-to-port journey (depart → arrive → dock `at_location`) confirmed live behavior. A ship docked at a port can now travel port-to-port; arbitrary coordinate travel stayed off.
- **Phase 9 — docked-port read surface (PR #49 → migration 0069, deployed).** `get_my_current_dock_services()` (authenticated, read-only, zero-arg, SECURITY DEFINER): derives player → own ship → validated dock, and ONLY for the `at_location` state returns the port + its ACTIVE `location_services` (today: Docking). Frontend `DockServicesPanel` shows "

Main ship docked at <port>" + service chips only when docked. No buy/sell/market. Proven (disposable RPC matrix + rendered UI), deployed 0068→0069, read-only verified live (OVERALL_PASS=true).

- **Phase 9 closeout (PR #50, frontend/tooling only — no migration).** Dock-context hardening (stale-data protection on a lifecycle change, safe-failure, mobile width cap), the one stale player-facing string fixed, and the current-state verifier `osn-postenable-verify` repinned head 0068→0069 + dock-surface ACL assertions; the historical pre-enable verifiers were left untouched.
- **OSN-COORD-GATE-1 (PR #51 → migration 0070, deployed).** Closed a real gap: the public raw coordinate command `command_main_ship_space_move` was guarded only by `mainship_space_movement_enabled` (true for the enabled port-to-port path), while the "free coordinate travel OFF" control was **frontend-only** — so a direct authenticated API caller could request arbitrary coordinates. Fix: a server-owned key `mainship_coordinate_travel_enabled` (default **false**); the raw command now returns `coordinate_travel_disabled` BEFORE any ship read / lock / writer call (no side effect) while the key is false. The location-target command `command_main_ship_space_move_to_location` is **unchanged** (still governed by `mainship_space_movement_enabled`; port-to-port unaffected). Disposable matrix ok[1..7] green; deployed 0069→0070. Gate ships **false**.
- **Phase 10 Trading V1 — design & calibration (DESIGN ONLY; nothing built).** A full pass produced the Trading V1 contract: free-port model (trade eligibility = own ship's validated current dock + active market capability), a **HYBRID cargo** model (account loot stays in `player_inventory`; a per-ship trade-hold carries trade goods), a **lazy player wallet** (currency separate from items), server-owned `market_offers` (price/availability, never in `location_services`), `trade_receipts` whole-trade idempotency, a per-offer **purchase-allowance** throttle, 7 proposed original commodities + a capacity-accurate 3-port matrix, and a route/balance simulation (no same-port profit; no unbounded reinvestment). Two hard findings: (1) a brand-new player has **no main ship** today (`bootstrap_me` makes only a base; `ensure_main_ship_for_player` is service-role-only with no player path) — so **main-ship provisioning** is the gating prerequisite; (2) trading needs the OSN `at_location` state, which neither `repair_main_ship` (→home) nor the legacy `send_main_ship_expedition` (→legacy_present) produces, while `command_main_ship_space_move_to_location` refuses a home origin by design — so a canonical **port-entry transition** is needed. Cargo-loss-on-destruction is deferred (free instant repair makes any recovery grant farmable). **No migration / seed / RPC / wallet / market / UI was created.**

Bugs / fixes

- Phase-9 dock proof: the `in_transit` fixture inserted the movement before its fleet (FK order) — fixed.
- Coord-gate proof: the disposable chain defaults `mainship_space_movement_enabled=false` (production's `true` is runtime, not a migration), so the first gate fired before the new gate — the proof now enables the movement domain on the disposable stack.

FORWARD PLAN (approved direction; not started):

1. **Main-ship provisioning — the prerequisite that gates all of Trading.** A one-time authenticated "Commission Your First Ship" claim that atomically creates ship + fleet + presence + an `at_location` dock at one designated **starting port** (a spawn placement, **not** a home port; `player_home_port` stays unused), plus a canonical OSN **port-entry transition** so existing `home/legacy_present` ships can reach a tradeable `at_location` state.
2. **Trading V1 implementation** (only after the open decisions below are approved): read model (`trade_goods` / `market_offers` / `player_wallet` / `ship_trade_cargo` / `trade_receipts` / allowance) → market capability + catalog seed → atomic idempotent buy/sell write path → Market UI from the Phase-9 dock seam → disposable proofs → gated deploy → read-only verifier.
3. **Then** Exploration (Phase 11) → Mining (Phase 12) → Modules/Captains (13–16) → Ranking (17) → economy/polish (18–20).
4. **Cross-cutting, deferred:** the `world_sites` canonical identity layer (build only when its F2 trigger fires), Online Presence & Visibility, main-ship combat, and a cargo-loss / repair-cost redesign.

Open product decisions (need user approval before any Trading build): cargo model (hybrid), currency (lazy wallet, start 0), first commodities + price matrix, per-offer allowance + reset window, starting spawn port, first-voyage starter cargo, and credit purpose (proof loop accumulating toward a future ship/captain/module sink).

2026-06-27 — PORT-LAUNCH: public port launch (foundation → reveal → independent verification)

The OSN-HUB-1A line (head 0067, prior entry) advanced through the full **PORT-LAUNCH** epic: the dark public-launch back end + front end were built and production-verified, then the three starter ports were **revealed** in a single controlled, human-gated operation, and the result was **independently, read-only verified** against production. Net production change: migration head 0067 → 0068; authenticated client-RPC surface 16 → 17; the three starter ports **hidden** → **active/public**. **OSN port-to-port movement stays dark** — `mainship_send_enabled` = true, `mainship_space_movement_enabled` = false, `OSN_COORDINATE_TRAVEL_ENABLED` = false (frontend) — all unchanged by this epic.

Requests / work done (in order):

- **ENABLEMENT-1 (PR #36 → 3b5e6ce)**. Re-pinned scripts/osn-enablement-preflight.sql to head 0067 / surface 16, widened space|location target checks, mirrored the function inventory into the DOCK-0 / HUB-1A allowlists. Tooling/gate only — no gameplay, no flag flip.
- **Fixture maintenance (PR #37 → 83d44e6)**. Replaced a global "anchors empty" assumption with an exact identity baseline (the three 0066 starter-port anchors). Housekeeping; depended on #36 landing first.
- **Enablement preflight (run 28253259301)**. Read-only production check → `OVERALL_PASS=true` at 0067/16.
- **PORT-LAUNCH-1A (PR #38 → 122374f, migration 20260618000068)**. Added `reveal_starter_ports()` (service-role-only, one-way, all-or-nothing, never auto-invoked; locks the full sector→zone→location→anchor →service hierarchy before validating) and `get_osn_movement_readiness()` (authenticated, read-only; reports `osn_available=false` while the flag is off). Surface 16→17.
- **Deploy 0068 (run 28281667811)**. Human-gated deploy; head 0067→0068; functions + surface re-lock only, **zero data change** (no reveal, no flag, no row touched).
- **Catalog-verifier refresh (PR #39 → 27df8e8) + production verify (run 28288983383, OVERALL_PASS=true)**. Re-aimed the read-only catalog verifier at 0068/17; proved production still dark (ports hidden, flags off).
- **PORT-LAUNCH-1B (PR #40 → ab07f14)**. Dark port-to-port travel UI (PortNavPanel / osnReadiness / portMoveCommand / osnReleaseGates); shows nothing while the flag is off; in-transit keeps route/ETA/Stop.
- **PORT-LAUNCH-2A (assessment) + 2B (PR #41 → 589abb9)**. Read-only onboarding-readiness recon, then a disposable full-chain proof: reveal → real send_main_ship_expedition accepts Haven Reach → real arrival settles → resolver returns anchored → readiness anchored (flag off) → world reverted. Added the verifier's A9 STP_* fail-closed pre-reveal checks.
- **PORT-LAUNCH-2C (PR #42 → 33af7e8)**. The controlled one-shot reveal workflow: workflow_dispatch only, main-only, typed REVEAL_THREE_STARTER_PORTS confirmation before any DB connection, production environment gate, pinned-CA verify-full, one transaction (lock → preconditions → reveal ×1 → postconditions incl. an **identity-level non-canonical digest** → commit-only-on-pass), rerun/uncertain fail-closed, no retry. Disposable proof ok[1..6].
- **PORT-LAUNCH-2D (run 28294311791)**. Dispatched + approved; reveal executed once: `REVEAL_FUNCTION_CALLS=1` · `STARTER_PORTS_ACTIVE_AFTER=3` · `FLAGS_UNCHANGED=true` · `REVEAL_OPERATION_PASS=true`. Three ports hidden → active. One-way.
- **PORT-LAUNCH-2E (PR #43 → 00dfdd2, run 28295627367)**. New independent read-only post-reveal verifier (scripts/postreveal-verify.{sql,sh} + .github/workflows/postreveal-verify*.yaml) — leaves the dark-state verifier untouched; checks the server catalog **and** the authenticated `get_world_map()` boundary. Live run returned `MIGRATION_HEAD=0068` · `CANONICAL_PORTS_ACTIVE=3` · `CANONICAL_PORTS_HIDDEN=0` · `UNEXPECTED_PORT_STATE_CHANGES=0` · `AUTHENTICATED_MAP_PORTS_VISIBLE=3` · `MAINSHIP_SEND_ENABLED=true` · `MAINSHIP_SPACE_MOVEMENT_ENABLED=false` · `OVERALL_PASS=true`.

Bugs / fixes:

- **1A lock-order TOCTOU** — reveal first locked only the three port rows; hardened to lock the full hierarchy (sector→zone→location→anchor→service) in a fixed order before validating; proven with concurrent psql sessions.
- **1A duplicate-insert proof premise** — the real block is a synchronous unique-constraint violation, not an FK lock-wait; proof corrected to assert the actual mechanism.
- **2B forced arrival** — back-dating only `arrive_at` violated `fleet_movements` (`arrive_at > depart_at`); fixed by moving the whole travel window into the past.
- **2C postcondition** — net "+3 active" could be fooled by an offsetting change; added an md5 digest of every non-canonical (id,status) to prove identity-level invariance.
- **2E test-harness** — a `emit_markers | grep -qx happy-path` assertion was fragile under `pipefail`; switched to reconcile + direct `mval` spot-checks (verifier logic itself was correct on first run).

State after this epic (all on main, head 00dfdd2): production head 0068, surface 17, three starter ports **active/public** (independently verified), flags unchanged (send true, space false). The in-game OSN travel panel is built but dark. The only remaining arc item is the separate, optional, future OSN flag-enable decision (`mainship_space_movement_enabled = true`) — **not started, not needed, not urgent**.

2026-06-26 — Session wrap-up + FORWARD PLAN (notes/design only; nothing started)

Closing-session record. **No product code / migration / workflow / verifier / flag / production change** in this entry. Captures where things stand after OSN-HUB-1A and the deliberately-gated next steps, so the next session can resume without re-deriving.

State at this wrap (all on main): product/production migration head 0067; main is the OSN-HUB-1A closure + verifier-tooling line (PRs #31 product, #32/#33 read-only verifier tooling, #34 closure record). **OSN is DARK** and stays dark: `mainship_send_enabled = true` (legacy named-location travel LIVE), `mainship_space_movement_enabled = false`. Hidden starter ports remain hidden/ineligible/unassigned; no home-port assigned; no base anchor; no public OSN enablement. OSN-HUB-1A

was merged → deployed (0067) → read-only verified (production catalog verifier run 28229418325 = OVERALL_PASS=true) → formally closed (prior entry). The legacy bases.x/y / locations.x/y coordinate path is frozen; the OSN coordinate domain resolves origins/targets through canonical space_anchors.

Reusable asset created this line of work: a dispatch-only, production-environment-gated, **strictly read-only** production catalog/ACL/configuration verifier (scripts/osn-hub1a-production-catalog-verify.{sql,sh})

- .github/workflows/osn-hub1a-production-catalog-verify.yml, disposable proof ...-proof.yml). It answers "does production still match the approved dark state at head 0067?" via one REPEATABLE READ ONLY snapshot + rollback (pinned CA / verify-full / session-pooler). Model future "is prod still in the approved known state?" checks on it. **Lesson encoded in it:** Supabase hosted **default privileges** grant EXECUTE-to-service_role on public functions that a migration doesn't explicitly revoke for service_role — so a public RPC granted only to authenticated still has service_role EXECUTE on prod but not on the disposable local stack; assert such platform-default ACLs as an **explicit production policy**, not reference-vs-local parity (this was PR #33 "correction A").

FORWARD PLAN — NOT STARTED. Each item needs its own separately-approved owner charter; do not begin on your own. No flag flip / port reveal / home-port assignment / anchor seed as a side effect. Ordered by readiness:

1. **ENABLEMENT-1 (tooling/gate maintenance — no gameplay, no flag flip).** Re-pin scripts/osn-enablement-preflight.sql from migration head 0064→0067 and the authenticated client-RPC surface 15→16 (it currently fails-closed on the new head/surface — that is why it was deferred). Update the **DOCK-0 perm allowlist** (scripts/osn3-dock0-realchain-perm.sql, exact-15 client-RPC list) to add command_main_ship_space_move_to_location (the same maintenance OSN-4 did for its Stop wrapper). Preserve the read-only / fail-closed / verify-full / pinned-CA contract. This unblocks a green enablement preflight; it does NOT enable anything.
2. **OSN flag-enable go/no-go (the first player-facing OSN change).** Only after ENABLEMENT-1 + a green production enablement preflight + an explicit owner decision: flip mainship_space_movement_enabled=true via the controlled dev-mainship-space-movement-flag.yml workflow. Reversible (single config key).
3. **Port-centric world build-out (heavy, charter-gated).** Reveal/seed real ports; seed canonical space_anchors for reachable locations; assign home-ports (assign_home_port); per the **F2 Option C** packet add the canonical world_sites identity layer (G1+) with the strict 1:1 immutable locations bridge; geographic-zones layer; possibly the **World Workbench** authoring plane first (PORTCENTRIC_DECISION_PACKET.md / F2_COMPATIBILITY_MODEL_DECISION_PACKET.md are the approved sources).
4. **Baseline activities & beyond (depend on OSN live + ports).** Exploration / Mining / Trading → Online Presence v1 → player interaction; Repair & Recovery (replace the instant-Home safelock); main-ship combat; captains / modules / rankings. Long-order rationale: docs/BYE HARU_PROJECT_GUIDE.md §10–11 and docs/ROADMAP.md.

Ship discipline that produced this line (keep using it): one owner-authorized step per message (build → disposable CI proof → PR → pre-merge integrity review → admin-override no-ff merge → deploy → read-only verify); the human owner approves every environment: production gate; never flip a flag / reveal a port / dispatch or approve a workflow as a side effect; work in a throwaway worktree off origin/main and never touch the stale osn3-dock0-location-arrival checkout.

2026-06-26 — OSN-HUB-1A FORMALLY CLOSED — dark canonical location-target navigation, deployed + verified (flag OFF)

Administrative closure record (notes only; no product code / migration / workflow / verifier / flag / production change in this entry). **OSN-HUB-1A is formally closed.**

- **What shipped (PR #31, merge 09f8ba6, migration 0067).** The dark, additive **canonical location-target navigation** foundation: the OSN coordinate domain now resolves a docked **origin** and a named-location **target** through canonical space_anchors (NOT legacy locations.x/y / bases.x/y). One discriminated core writer mainship_space_begin_move_core (the deployed 5-arg mainship_space_begin_move preserved as a space-only delegate); the single canonical target-legality rule mainship_space_location_target_legal (active sector/zone/location + role city|port + activity_type='none' + one active docking service + one active in-bounds anchor); anchored origin resolution (HOME stays fail-closed origin_not_anchored, no base anchor); Dock-0 (mainship_space_dock_at_location) re-pointed from locations.x/y to the canonical anchor with **full arrival-time revalidation** under target-hierarchy FOR SHARE locks and a clock_timestamp() settlement time (resolved_at >= arrive_at); OSN-4 **Stop compatibility** for location routes (mid-flight interpolated stop / at-or-after-arrival settles via the SAME Dock-0 decision; mainship_space_settle_space_arrival stays strict space-only); and the one new public authenticated wrapper command_main_ship_space_move_to_location(uuid, uuid) (flag-gated before target resolution; hidden-port UUID = nonexistent → generic invalid_target; **authenticated surface stays exactly 16**). Frontend is read-only/dark (target_location_id read-model; location routes render only to VISIBLE destinations).
- **Deployed.** Production migration head 0067 (Deploy Supabase migrations run 28219980298, approved production gate). OSN remains **DARK**: mainship_send_enabled = true, mainship_space_movement_enabled = false. **No port reveal, no home-port assignment, no base anchor, no flag flip, no player/world mutation.**

- **Verified (read-only).** Final corrected production catalog/ACL/configuration verifier run 28229418325 → OVERALL_PASS=true at verified main 30e5a36 (verifier tooling commits; product head 0067 unchanged). One REPEATABLE READ READ ONLY snapshot + ROLLBACK; **no production write**. All assertions passed: dark-state (head 0067, flags dark, zero active coordinate movement, no incoherent pointer, empty `player_home_port`, no base anchor); hidden-world (3 hidden ports hidden/ineligible/absent from `get_world_map`, one anchor + one docking service each, original five intact); RPC surface **exactly 16** + anon limited to `get_world_map`; the **13 internals service_role-only** + catalog tables locked down; **6/7 function bodies + descriptors byte-identical** ref↔prod; and the public wrapper's explicit hosted-production `service_role EXECUTE = true` policy.
- **Verifier tooling PRs.** PR #32 (merge 09f8ba6→... on main) added the dispatch-only, production-gated, strictly read-only verifier. PR #33 (merge 30e5a36) was a **verifier-only correction**: the public wrapper is granted only T0 authenticated in 0067, so its `service_role EXECUTE` is governed by Supabase hosted DEFAULT PRIVILEGES (allowed) which the disposable reference does not reproduce; PR #33 replaced that accidental local-reference dependence with an **explicit, testable hosted-production** `service_role EXECUTE = true contract` (strict parity preserved for the body hash + args + lang + owner + SECDEF + search_path + anon/authenticated/PUBLIC, and full SRVX parity for the six internals). Both PRs were **verifier tooling only** — no migration, no production data/ACL change.

NEXT: the next product step (e.g. ENABLEMENT-1 / the OSN enablement preflight re-pin to head 0067 + surface 16, the DOCK-0 perm allowlist update, then any controlled OSN flag-enable go/no-go) requires a **separately approved charter**. None is started. OSN remains dark.

2026-06-23 — ANCHOR-2 P0-A census closed + PORT-CENTRIC direction (durable handoff; design/ops only)

Cross-computer handoff record. **No code/schema/migration/anchor/resolver/flag/production change** — this entry makes the current direction recoverable from main.

1. **ANCHOR-2 P0-A census — CLOSED.** One authorized, production-Environment-gated, **read-only** count-only census ran and succeeded — workflow `osn3-anchor2-p0a-homebase-census.yml`, run 28061856879, source commit a12743f4829782530fc05015af509135886f8bf3, one BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ READ ONLY snapshot then ROLLBACK (no write). Result: TOTAL_SHIPS=72, ELIGIBLE=72, UNRESOLVED=0; the one-ship-per-owner invariant held (72 = DISTINCT_NON_NULL_SHIP_OWNER_IDS); zero null-owner/orphan/no-base/ inactive-only/multi-base anomalies. This closes **only** the old-data ambiguity prerequisite (legacy base records are clean). **The census must not be rerun without explicit authorization.**
2. **PORT-CENTRIC direction (supersedes the home-base P0 plan).** Byeharu is a **multi-port navigation world**, not a permanent-main-base game. A ship's meaningful normal location is its **current docked port**. Normal loop: Dock at Port A → depart → travel/act → dock at Port B → depart from Port B. The permanent `main_ship_instances.home_base_id` / ship-to-owner-base P0 plan is **CANCELLED** (no FK / NOT NULL / backfill / creation-path change). Legacy bases are **bootstrap / starter / registration / possible-recovery records only**, never operational homes. "Return home" is **not** ordinary navigation; emergency recovery is separate future work.
3. **Technical boundary.** The existing dark `at_location` state (ship `spatial_state='at_location'` + the fleet's `current_location_id` + an active `location_presence`) is the **proto current-dock model**. `space_anchors` (migration 0063, empty/dark) remains the **future fixed-coordinate foundation**. Future port docking/departure must resolve through **location identity + the eligible port's canonical `space_anchors (kind='location')` coordinate** — not legacy `locations.x/y`. The current dark DOCK-0 exact-match against `locations.x/y` (migration 0061) is proto behavior only and **remains unchanged**.
4. **Map-growth policy.** The open-space boundary stays $\approx [-10000, 10000]^2$ — a **temporary technical frontier**, not a permanent world/lore edge; no final map size is chosen. Future expansion grows **outward** and **preserves all existing coordinates** (do not remap/move existing ports, anchors, ships, or players); keep the initial central region **dense** and reserve outer space for later. **No map-size expansion is authorized now.**
5. **Exact next project gate.** *Next work is a **port-centric product-decision packet**: port eligibility, dock identity, legacy-base/recovery role, recovery model, anchor-seeding scope, and initial central-region layout policy. No ANCHOR-2 implementation, anchor seeding, resolver change, map work, coordinate command work, migration, or flag change is authorized before those decisions.*

Live baseline unchanged at this handoff: production migrations end at 0063; `mainship_send_enabled=true`, `mainship_space_movement_enabled=false`; OSN paused.

2026-06-23 — MSP-0: Main Ship Progression ↔ Movement integration contract (design only)

Read-only reconnaissance + integration contract answering where future main-ship progression stats must live so the current named-location route and future OSN movement consume **one** server-calculated result. **No**

code/migration/workflow/flag/branch change — design packet only.

- **Speed-truth trace.** Both routes derive main-ship speed solely from `main_ship_hull_types.base_speed` (`starter_frigate=1.0`): `legacy_send_main_ship_expedition/move_main_ship_to_location/ request_main_ship_return` → `resolve_fleet_movement_speed` → `movement_create` (LIVE); OSN `mainship_space_begin_move` reads the hull inline + computes duration inline, with `resolve_fleet_movement_speed` only as an equality assert (DARK). Speed + `arrive_at` are snapshotted once at departure, never recomputed at arrival. Frontend submits **intent only** (no client speed/duration math; the one `previewTravelSeconds` is dead code).
- **Divergence to prevent (already nascent):** `calculate_expedition_stats` computes a support-craft speed penalty that live movement ignores.
- **Recommendation (Option B):** one private main-ship-keyed `mainship_effective_stats` resolver (`effective_travel_speed` first; empty loadout = raw hull base ⇒ current behavior byte-for-byte unchanged) that both movement adapters consume. First slice = **module-first**, first effect = travel speed on the live named-location route. Phases MSP-0..MSP-4 defined; `module/captain` schema is greenfield (only integer `module_slots/captain_slots` counts exist today).

No implementation started. Flags unchanged (`mainship_send_enabled=true`, `mainship_space_movement_enabled=false`); migrations end at **0063**. NEXT (needs approval): Option-B decision + **MSP-1** (additive, dark module-ownership schema only). ANCHOR-2 / seeding / resolver extension / S6B-PRES / coordinate enablement remain deferred.

2026-06-23 — OSN-ANCHOR-1B: empty canonical-anchor schema (space_anchors) — DEPLOYED & CLOSED (flag OFF)

Additive, EMPTY, server-only canonical-anchor foundation (branch `osn3-anchor1b-space-anchors`, PR #18 merge 7264f12, migration 0063). `public.space_anchors`: closed kind ∈ {base,location} with **exactly one real typed owner FK** (`base_id`→`bases` ON DELETE CASCADE, `location_id`→`locations` ON DELETE RESTRICT; no ownerless / all-null / polymorphic (`kind`, `owner_uuid`)); coords NOT NULL + finite + within $[-10000,10000]^2$ (rejects NULL/NaN/±Inf/oob); partial-unique **one active anchor per base & per location** (no (`space_x`,`space_y`) unique — intentional co-location stays possible); BEFORE-UPDATE immutability guard (SECURITY INVOKER, `search_path=public`: `active`→`retired` only; `kind/owner/x/y/created_at` immutable; `retired` terminal; DELETE unguarded so base CASCADE works); private RLS (no policy) + explicit revoke from `public/anon/authenticated` + grant **service_role-only**.

Seeds NOTHING; copies nothing from `bases.x/y/locations.x/y`; **NOT read by** `mainship_space_resolve_origin` (resolver **UNCHANGED** → `home/at_location/legacy_*` still resolve `origin_not_anchored`); **no flag/resolver/ docking/movement/UI change**. Proof: disposable real-chain `osn3-anchor1b-realchain-proof.yml` (all 17 points — `shape/types/RLS/indexes/checks/trigger`, `kinds/owners/coords/uniqueness/immutability`, `base-cascade`, `location-restrict`, `ACL`, `resolver-unchanged`; asserts table empty) + S1-S6A / DOCK-0 / ANCHOR-1A non-regression + Build, all GREEN. (Three proofs first failed on a transient Docker-pull 502 at `supabase start` — proof step skipped, not a defect — and reran green with no code change.)

Deploy: production-Environment-gated run 28025760972 (approved) applied exactly 0063 ("Finished supabase db push"); remote migration history now ends 20260618000063 (no 0064+). Live confirm: `anon REST GET /space_anchors` → HTTP 401 42501 permission-denied (table **exists** in prod, clients **denied**); flags `mainship_send_enabled=true`, `mainship_space_movement_enabled=false`. **OSN is now PAUSED at this boundary.** NEXT: **Main Ship Progression (MSP)** — not ANCHOR-2.

2026-06-23 — OSN-ANCHOR-1A: production catalog-parity verification — CLOSED

Verified the deployed truthful-origin resolver `mainship_space_resolve_origin` (migration 0062) is **byte-identical + semantically identical to source** in production, via a dedicated, strictly read-only catalog-parity spotcheck. Built across two PRs: **#16** (2b11f28) added the `osn3-anchor1a-catalog-spotcheck` workflow + script capability; **#17** (cb0219a) a CA-trust remediation after the first production run failed `sslmode=verify-full` against the shared IPv4 pooler — pinned the official **Supabase Root 2021 CA** (`scripts/supabase-prod-ca.crt`, cert SHA-256 807025ad50d4ed219d2c9c7d299c004f824eb00cf7f65afe607d07b72e6cafa)

- used the **session pooler (port 5432)**; kept `verify-full`, **no TLS downgrade**.

Production verification run 28022976137 (`workflow_dispatch` on `main`, production gate approved) PASSED: raw stored-body `prosrc` SHA-256 7d4548e64e2fca60a944fe2875c0b8e3e381c85bb0960f14bb8670d71d6038b0 identical reference-vs-production (exact, no normalization); 17-field descriptor parity identical; invariants OK (`plpgsql` / `owner=postgres` / `SECURITY DEFINER` /

search_path=public / service_role-only; anon/authenticated/PUBLIC denied); remote migration history ends exactly 0062; read-only gate proven before any catalog query; **no production write**. Equality source = raw p.prosrc (version-stable), NOT pg_get_functiondef. Flags unchanged (send=true, space=false). No resolver/data/flag change.

2026-06-22 — OSN-ANCHOR-1A: truthful-origin guard (dark) — DEPLOYED & CLOSED (flag OFF)

Migration 0062 (branch osn3-anchor1a-truthful-origin, PR #15 merge fb28481) re-creates mainship_space_resolve_origin(uuid) (CREATE OR REPLACE; signature / SECURITY DEFINER / search_path=public / service_role-only all preserved) so home / legacy_home / at_location / legacy_present now resolve {ok:false, reason:'origin_not_anchored'} instead of reading legacy bases.x/y / locations.x/y as a movement origin; in_space unchanged (origin = ship space_x/space_y); in_transit→must_stop; destroyed→destroyed. Closes the proven defect of legacy dynamic-map coordinates leaking into OSN movement origins. **NO anchor table, NO bases/locations column, NO seed/backfill, NO legacy fallback; both flags untouched** (send=true, space=false).

Proof: real chain 0001..0062 (osn3-anchor1a-realchain-proof.yml) — the four legacy/home states → origin_not_anchored (no movement / receipt / legacy-origin written); in_space success with origin == ship coord; rejected-request idempotency; resolver ACL/security/signature parity; cross-domain / destruction / DOCK-0 non-regression. Deployed via production-gated run 27988863386; production catalog-parity verification followed separately (see the 2026-06-23 catalog-parity entry). Coordinate movement stays dark.

2026-06-22 — OSN-3 S6C: flag-dark empty-space coordinate command surface — CLOSED (flag OFF)

Frontend-only coordinate-move command path (branch osn3-s6c-empty-space-coordinate-command, PR #14 merge 9ce5567, **no migration / RPC / flag / server change**). Empty-space map tap → screenToWorld → canonicalized target → existing S6A wrapper command_main_ship_space_move(p_target_x, p_target_y, p_request_id). Layered gating (feature flag mainship_space_movement_enabled read once; eligibility; controls/crosshair mount only when enabled + within bounds; tap qualifies only on empty SVG) → **production-dark**: flag false ⇒ wrapper returns feature_disabled and writes nothing. The client submits **intent only** (target coords + request_id); never a speed/duration/stat/ship-id. Build green; flags unchanged (send=true, space=false). NEXT then was the S6B presentation foundation + ANCHOR truthful-origin work.

2026-06-22 — OSN-3 S6B: fixed-space frontend coordinate foundation — CLOSED (flag OFF, read-only)

S6B closes the **read-only frontend coordinate-rendering foundation** for open space across four merged sub-slices. It is **not** a player-enabled movement feature: coordinate movement remains **production-dark** (mainship_space_movement_enabled=false; mainship_send_enabled=true), there is **no player command path, tap selection, selected-target persistence, or coordinate-movement enablement**, and **no migration / RPC / flag / server change** in any S6B slice (migrations remain through 0060).

- **S6B1** (merge 586d67c) — src/features/map/openSpaceTransform.ts: a **pure** fixed-domain transform — worldToViewBox/viewBoxToWorld over [-10000,10000]→[0,1000] (explicit Y-inversion), worldToScreen/ screenToWorld (camera + preserveAspectRatio letterbox), and a **separate** isWithinOpenSpaceBounds predicate (no hidden clamping; conversions never validate). Verifier verify:osn:s6b.
- **S6B2** (merge f7974ac) — a **mandatory discriminated** coordinateSpace: 'legacy_dynamic' | 'open_space_fixed' on the resolved ShipMarker; the ship's open-space states (in_space, coordinate in_transit) route through the fixed transform while legacy/named states keep buildNormalizer. Exhaustive switch + **never** guard, no silent legacy fallback. Verifier verify:osn:resolver.
- **S6B3** (merge e2de473) — a **development-only**, non-interactive fixed-space preview (DevFixedSpacePreview), gated **solely** by import.meta.env.DEV and **compile-time eliminated** from the production bundle — proven by vite build + a dist/ grep showing the s6b3-dev-preview sentinel and the component are **absent** (true removal, not runtime hiding). pointerEvents:none, aria-hidden, minimal hollow ring/crosshair. Verifier verify-s6b3.
- **S6B4** (merge adc7009) — behavior-preserving extraction of MainShipMarker's routing into a pure exported markerViewBoxPoint(marker, norm) that **the component and the tests both call** (no duplicate); proves a **resolved** open_space_fixed marker is projected through worldToViewBox (the dynamic norm is **never** called) and that the preview + a

distinct fixed-space ship point **co-move** under the camera (screen Δ = letterbox-zoom $\times$ viewBox Δ across zoom 0.4/1/2/8 $\times$ zero/nonzero pan $\times$ square/wide/tall/mobile viewports; pure geometry, no comparison to dynamic named-location coords).

Verifiers verify:osn:resolver

 verify:osn:s6b.

Acceptance (all green): verify:osn:s6b (transform) · verify:osn:resolver (provenance + S6B4 routing) · verify-s6b3 (dev preview + production-elimination) · build (tsc -b + vite build) · post-merge **Build + Pages** deploy. On production data the ship marker is always legacy_dynamic (open-space states are dark) and the dev preview is absent → **zero production visual change**.

Explicitly NOT done / still pending. Fixed-space markers and legacy named locations are **not yet an approved co-registered presentation**. **S6B-PRES is mandatory before any S6D enablement** — it must charter, implement, and prove **either** named locations rendered through a verified fixed-domain transform **or** a distinct coordinate-navigation map mode where legacy dynamic markers are hidden/non-spatial. No tap/mapToWorld wiring (S6C), no command/CTA/RPC, no flag flip.

NEXT: OSN-3 **S6B-PRES** reconnaissance — the fixed-space ↔ named-location presentation decision (the mandatory pre-S6D gate). S6C input wiring must **not** precede that decision.

2026-06-21 — OSN-3 S6A: public coordinate-command boundary (flag-dark) — CLOSED (flag OFF)

First **player-facing** coordinate-movement command surface (branch osn3-s6a-public-space-move-command, no-ff merge ac9230a, code commit 581dea9, migration 0060). A narrow, **authenticated**, SECURITY DEFINER wrapper

command_main_ship_space_move(p_target_x, p_target_y, p_request_id) that derives the caller from auth.uid(), derives the caller's **own** main ship server-side (**no client player/ship id**), defense-in-depth flag-gates, **canonicalizes** the target to the integer world-unit grid (round(numeric) — half **away from zero**, deterministic; non-finite rejected before the cast; bounds remain the writer's authority, so a raw value with |canonical| ≤ 10000 snaps inward and is accepted), **DELEGATES** to the existing private writer mainship_space_begin_move, and **maps** the result to a narrow player-safe payload. Canonicalization is a discrete-grid concern only — p_request_id **remains the idempotency key**. The private writer stays the **final authority** on flag/ownership/bounds/ state/exclusion/travel-cap/locking/idempotency/movement-creation and remains **service_role-only** (the client never gains it; the definer-owner postgres invokes it). **NO writer/processor/S2/S5 change, NO new table/cron, NO flag flip, NO UI/CTA**.

Dark in production: mainship_space_movement_enabled stays **false**, so the wrapper returns feature_disabled and writes nothing → **net player-visible effect: none**. mainship_send_enabled stays **true**; legacy named-location travel is untouched and **mutually exclusive** with coordinate movement (proven both directions: a coordinate-domain ship rejects legacy send/move by precondition; a legacy-busy ship rejects the coordinate command via cross-domain exclusion; the fleet active_movement_id XOR active_space_movement_id holds).

Also: sibling dev flag tool dev-mainship-space-movement-flag.mjs (+ workflow) for the coordinate flag (legacy send-flag tool untouched; **not** run against prod in S6A); fetchMainshipSpaceMovementEnabled() typed read in src/lib/catalog.ts (no UI wiring — an S6B seed). The migration re-locks the execute surface (canonical client RPCs + **the new wrapper**; writer/processor/destruction/S2 helpers stay service_role-only).

Authoritative proof (real chain 0001..0060, disposable Supabase; osn3-s6a-realchain-proof.yml). GREEN: permission/boundary (wrapper authenticated-only, owner postgres / SECURITY DEFINER / search_path public / no dynamic SQL / no player-or-ship param; private writer + S4 + S5 + four S2 helpers service_role-only; canonical client-RPC inventory = prior 13 + the wrapper); runtime **SET ROLE** (anon denied / authenticated allowed on the wrapper; writer client-denied, service_role-allowed); fixture matrix (dark→feature_disabled + no write; success from home/in_space/at_location; canonicalization half-away-from-zero + near-edge inward snap + out_of_bounds/non-finite reject; zero_distance; idempotency exact **and** equivalent-canonical replay + request_conflict + no duplicate; state matrix in_transit→must_stop_first / destroyed→ship_destroyed / legacy-busy→busy_legacy; legacy↔coordinate mutual exclusion both directions + fleet pointer XOR); REST boundary (private writer rejected for anon **and** authenticated; wrapper reachable for authenticated but dark → feature_disabled, no movement). Flags restored if: always().

Gates (all green): S6A real-chain proof; **S1–S5 real-chain regression**; Build (tsc -b + vite build); deploy-migrations (live db push of 0060); post-deploy integration **Verify**; live legacy regressions verify-mainship-send (send + **return/recall**), verify-mainship-move, verify-mainship-repair. **Live read-only spot check** (osn3-s6a-live-spotcheck.yml): 0060 applied; wrapper present **authenticated-only**; private engine **service_role-only**; canonical inventory intact; one S4 arrival cron @30s; mainship_send_enabled=true, mainship_space_movement_enabled=false, cap=86400; main_ship_space_movements=0, command_receipts=0 — **no game-state mutation by the deploy**. (An earlier batch of live runs was **cancelled** by the shared live-db-tests concurrency group — a workflow-concurrency incident, not a test failure; each was re-run serially to a real success.)

NEXT (not started, needs approval): OSN-3 S6B — the fixed-domain paired coordinate transform (worldToMap/mapToWorld over [-10000,10000], Y-inverting, pan/zoom-aware) + a **read-only target preview**, still flag-off. No map tap/CTA until S6C; no enablement until S6D; OSN-4 Stop remains deferred.

2026-06-21 — OSN-3 S5: coordinate-complete trusted destruction primitive — CLOSED (flag OFF)

Fifth **OSN-3** slice (branch `osn3-s5-destruction-hardening`, approved head `a7ab585`, normal **no-ff** merge `0d84256`, migration `0059`; final main == origin/main == `fda8778` after a read-only live-spot-check tooling commit). **Narrow hardening only — NO public RPC, NO UI, NO new processor/cron, NO Return/Stop, NO generic reconciliation, NO flag change.** Both flags untouched (mainship_send_enabled stays **true**, mainship_space_movement_enabled stays **false**).

The defect S5 fixes. `dev_set_main_ship_destroyed(p_player uuid)` — the **unique** trusted main-ship destruction writer (audited: the only fn that sets `main_ship_instances.status='destroyed'/hp=0`; combat destroys legacy unit-fleets via `fleet_destroy`, never main ships; `repair_main_ship` only recovers) — predated the coordinate domain and therefore could **not** destroy a ship in a valid coordinate state without violating a coordinate constraint (in_transit left `fleets.active_space_movement_id` set → violates `fleets_active_space_movement_requires_moving`; in_space/at_location left a non-null `spatial_state` → violates the `...ss_*_status` CHECKS). Latent (service_role-only path; coordinate movement dark), but closed before coordinate movement is ever enabled.

Migration 0059 re-creates **only** `dev_set_main_ship_destroyed` (same signature, SECURITY DEFINER, owner postgres, search_path=public, **service_role-only**, no player wrapper, no new cron). It: acquires `mainship_space_lock_context(id,false)` first (canonical order; never locks `fleet_movements`); requires `validate_context` ok — **any generic contradiction ABORTS atomically with all rows unchanged**; for a coherent in_transit cancels the active coordinate movement → `status='cancelled', terminal_reason='ship_destroyed', resolved_at` (history preserved); clears `active_space_movement_id`; preserves the existing legacy cleanup; and sets the ship `destroyed/hp=0/spatial_state=NULL/ space_x/space_y NULL` (NULL — not 'destroyed' — so `repair_main_ship`, which sets `status='home'` without resetting `spatial_state`, stays valid → a repaired ship is a clean `legacy_home`). The S3 command receipt is immutable; no history deletion. `repair_main_ship`, the S4 processor, the S3 writer, the S2 helpers, and all legacy writers are untouched; migrations `0052/0055/0056/0057/0058` are untouched.

Authoritative proof (real chain 0001..0059, disposable Supabase; osn3-s5-realchain-proof.yml). GREEN at `a7ab585`: coherent destruction of in_transit (movement→cancelled/ship_destroyed, receipt immutable), in_space, at_location, and preserved `legacy_present`; idempotent repeated destruction; **real** `repair_main_ship` **after destruction** → **clean** `legacy_home` with no coordinate residue; the full contradiction-abort matrix (active legacy movement, unexpected presence, pointer/ownership mismatch, multiple fleets, in_transit-without-movement, destroyed-plus-moving) each non-mutating; real concurrent-session races (arrival-wins-then-destroy-clears-in_space; destruction-wins-arrival-never- settles-cancelled; two destructions race → one terminal, second idempotent); runtime ACL + SET ROLE denial; REST/RPC denial of the primitive + processor + writer + S2 helpers for anon and a real authenticated JWT. Root-cause note: the first run was red only on a **proof-harness** transaction defect (concurrency sessB ran destruction in autocommit, never observed idle-in-transaction); fixed by holding sessB's destruction in a txn — **the migration/primitive needed no change** (no `0060`).

Gates (all green at a7ab585): S5 real-chain proof; S1/S2/S3/S4 real-chain regression; the Build gate via draft PR #3 (`npm ci, lint, tsc -b, vite build`); verify:osn:resolver; the legacy-send read-only verifier. **Live read-only spot check** (`osn3-s5-live-spotcheck.yml`, post-deploy): 0059 applied; primitive present with the approved signature (`p_player uuid`), owner=postgres, SECURITY DEFINER, search_path=public, no dynamic SQL, no player wrapper, service_role-only; canonical client-RPC inventory unchanged; `repair_main_ship` still authenticated-executable; S2 helpers + S3 writer + S4 processor non-client-executable; exactly one S4 arrival cron @ 30 seconds (cadence unchanged); `mainship_send_enabled=true`, `mainship_space_movement_enabled=false`, `max_coordinate_travel_seconds=86400`; `main_ship_space_movements=0`, `main_ship_space_command_receipts=0` — no game-state mutation by the deploy or verification.

Scope confirmation. S5 added **no** player coordinate RPC, UI, processor/cron, Return, Stop, generic reconciliation, history cleanup/retention, legacy-writer change, `repair_main_ship` change, S2/S3/S4 helper change, or feature enablement. The internal coordinate lifecycle is now complete and dark: **departure (S3) → arrival settlement (S4) → parked in_space → coordinate-complete destruction (S5).** **NEXT (not started, awaiting a separate explicit charter):** a PC-first coordinate command/map surface (public wrapper + UI, gated by `mainship_space_movement_enabled`), then **OSN-4 Stop**.

2026-06-21 — OSN-3 S4: coordinate-arrival processor — CLOSED (flag OFF)

Fourth **OSN-3** slice (branch `osn3-s4-arrival-processor`, approved head `33588e2`, normal **no-ff** merge `6b1a88e`, migration `0058`; final main == origin/main == `6b1a88e`). **One private, server-only background PROCESSOR — still NO public RPC, NO UI, NO**

Return/Stop, NO feature enablement, NO reconciliation/destruction. `mainship_space_movement_enabled` stays **false** (the processor does not gate on it); `mainship_send_enabled` stays **true** (untouched legacy path).

Migration 0058 — `public.process_mainship_space_arrivals()` returns integer. One SECURITY DEFINER, owner postgres, `search_path=public`, **service_role-only** processor (PUBLIC/anon/authenticated revoked; no player wrapper), driven by a **pg_cron** job `process-mainship-space-arrivals` at the established 30 seconds cadence (`command = select public.process_mainship_space_arrivals()`; idempotent `unschedule-by-name`). It settles each due, still-coherent S3 coordinate movement **exactly once**: non-locking candidate scan (`status='moving'` and `arrive_at<=now()`, ORDER BY `arrive_at,id` LIMIT 100) → per ship `mainship_space_lock_context(id, true)` skip-locked (S2 canonical order ship → fleet → coordinate-movement → presence; never locks legacy `fleet_movements`) → `validate_context` must be `in_transit` → `assert_cross_domain_exclusion` → re-confirm under lock → atomic settlement.

- **Arrival transition:** movement `moving` → `arrived` (`resolved_at=now()`, `terminal_reason='auto_arrival'`; immutable origin/target/speed/time history preserved); fleet `moving` → `completed` with `location_mode='movement'` and `active_space_movement_id / active_movement_id / current_*` cleared (truthful open-space terminal — verified legal once the space pointer is NULL; no base field set, `fleet_complete()` not used); ship `traveling/in_transit` → `stationary/in_space` at the movement's `target_x/target_y`.
- **Terminal history preserved** (the arrived row stays; existing FK CASCADE cleans it only on owner/ship deletion — no retention/cleanup job added). The S3 creation receipt is immutable; S4 writes no receipt and creates/leaves no `location_presence`.
- **Contradiction policy (frozen):** every contradiction / malformed / destroyed / legacy-conflict / presence-conflict / pointer-mismatch / ownership-mismatch / not-due / already-terminal case is left **untouched** with a concise log (no settle/fail/repair/normalize/delete; hardening deferred to S5).
- **Flag rule:** the processor never reads `mainship_space_movement_enabled` (so disabling it can't strand in-transit ships) and never touches `mainship_send_enabled`.

Authoritative proof (real chain 0001..0058, disposable Supabase; osn3-s4-realchain-proof.yml). GREEN at 33588e2: due settles exactly once; not-yet-due stays moving; second call settles 0 (idempotent); two concurrent processors settle once (loser skip-locked); skip-locked ship skipped then settles; settlement proceeds with the space flag FALSE; full arrival-state assertions (movement `arrived/auto_arrival/resolved`, ship `stationary/in_space` at exact target, fleet `completed/` movement with pointers+base cleared, no presence, no legacy mv, S2 `validate_context=in_space`, S3 receipt unchanged, terminal history present); the seven contradiction cases each proven non-mutating (per-ship state hash) as real due candidates; runtime ACL + SET ROLE denial; REST/RPC denial of the processor + writer + S2 helpers for anon and a real authenticated JWT; cron asserted present once @30s; cleanup + flags/cap/cron restored & asserted. Root-cause note: the first proof run was red on a **fixture** timestamp assumption only (transaction-scoped `now()` made `arrive_at < depart_at`), corrected by moving both timestamps into the past + asserting every fixture's precondition; **the processor/0058 needed no change** (no 0059).

Gates (all green at 33588e2): S4 real-chain proof; S1/S2/S3 real-chain regression; the Build gate via draft PR #2 (`npm ci, lint, tsc -b, vite build`); verify: `osn:resolver`; the legacy-send read-only activation verifier. **Live read-only spot check** (`osn3-s4-live-spotcheck.yml`, post-deploy): 0058 applied; processor present with the approved signature (no args), owner=postgres, SECURITY DEFINER, `search_path=public`, no dynamic SQL, no player wrapper, `service_role-only` (anon/authenticated/PUBLIC denied); S3 writer + four S2 helpers `service_role-only`; canonical client-RPC inventory unchanged; anon/authenticated cannot CREATE in public; **exactly one** cron job `process-mainship-space-arrivals @ 30 seconds` (no duplicate); `mainship_send_enabled=true`, `mainship_space_movement_enabled=false`, `max_coordinate_travel_seconds=86400`; `main_ship_space_movements=0` and `main_ship_space_command_receipts=0` — live deployment created zero coordinate movements and zero receipts; no game-state side effect (a natural cron tick that finds zero due movements is harmless).

Scope confirmation. S4 added **no** player coordinate RPC, UI, Return, Stop, reconciliation/auto- repair, destruction/repair behavior, history cleanup/retention, legacy-writer/processor change, S2/S3 helper change, or feature enablement. `mainship_send_enabled=true` remains the temporary playable legacy named-location path; `mainship_space_movement_enabled=false` remains dark. **NEXT (not started, awaiting a separate explicit S5 charter):** reconciler / destruction hardening (S5) → target UI (S7) → a public player wrapper for the writer → **OSN-4 Stop** (S8).

2026-06-21 — Legacy main-ship send: controlled production activation (config-only, reversible)

Enabled the **already-built legacy named-location** main-ship travel path on live by flipping **one** game-config key via the established controlled workflow `dev-mainship-flag.yml` → `scripts/dev-mainship-flag.mjs --enabled true` (writes only `mainship_send_enabled` via the owned `set_game_config`). **No migration, no code/UI change, no fixtures, no test users, no writer execution.**

Target/result live config: `mainship_send_enabled = true`, `mainship_space_movement_enabled = false` (untouched), `max_coordinate_travel_seconds = 86400` (untouched). The activation script logged Before: false → After: true.

Read-only preflight (osn3-s3-live-spotcheck, run 27899732391): confirmed the pre-state — send=false, space=false, cap=86400, main_ship_space_movements=0, main_ship_space_command_receipts=0, S3 writer + four S2 helpers service_role-only, canonical client-RPC inventory unchanged. **Read-only post-activation verification** (osn3-legacy-send-activation-check, run 27899841147): confirmed send=true, space_movement=false, cap=86400, main_ship_space_movements=0, command_receipts=0, and that mainship_space_begin_move + the four S2 helpers remain **service_role-only / non-client-executable** with the canonical client-RPC inventory unchanged and public-schema CREATE denied to anon/authenticated.

What this does / does not do. It re-exposes only the **legacy named-location** player capability (send_main_ship_expedition base→location, move_main_ship_to_location location→location, plus the always-available recovery paths request_main_ship_return and repair_main_ship). It does **not** enable coordinate movement or any OSN player command: the S3 coordinate writer stays service_role-only and flag-dark (mainship_space_movement_enabled=false), no coordinate UI/command surface exists, and no coordinate movement or command receipt was created (both counts remain 0). No game-state row was created or modified by the activation. **Rollback** is the same controlled workflow with mainship_send_enabled=false (single-key, instant, no migration). **S4 has not started.**

2026-06-21 — OSN-3 S3: first internal coordinate-movement writer — CLOSED (flag OFF)

Third **OSN-3** slice (branch osn3-s3-begin-move-writer, approved head e267eee, normal **no-ff** merge f4ba07e, migration 0057; final main == origin/main == f4ba07e). **One private, server-only WRITER — still NO public RPC, NO UI, NO processor, NO arrival/Return/Stop, NO feature enablement.** Both flags stay false on live.

Migration 0057 — public.mainship_space_begin_move(p_player uuid, p_main_ship_id uuid, p_target_x double precision, p_target_y double precision, p_request_id uuid) returns jsonb. One SECURITY DEFINER, owner postgres, search_path=public, **service_role-only** function (PUBLIC/anon/ authenticated revoked) that composes the deployed S2 boundary — mainship_space_lock_context → mainship_space_validate_context → mainship_space_assert_cross_domain_exclusion → mainship_space_resolve_origin — to begin exactly one coordinate move. Hard-gated on mainship_space_movement_enabled (stays false); mainship_send_enabled untouched. Adds one additive non-flag guard max_coordinate_travel_seconds=86400 (the [-10000,10000]² envelope is the distance bound; no MAX_COORDINATE_MOVE_DISTANCE).

- **Supported stationary origins:** home/legacy_home/in_space (materialise a new main-ship fleet in-txn) and at_location/legacy_present (reuse the present fleet, closing its active presence). **Space-only target contract** (target_kind='space' + p_target_x/p_target_y + p_request_id); the client never supplies origin/player/ownership/state/fleet/speed/ETA/status or screen coords.
- **One atomic transaction, canonical S2 lock order** (ship → fleet → coordinate-movement → presence); never locks legacy fleet_movements; never calls a frozen legacy writer. Creates one moving main_ship_space_movements row + coherent fleet pointer (active_space_movement_id, legacy active_movement_id stays NULL) + ship traveling/in-transit + finalised idempotency receipt.
- **Idempotency** via main_ship_space_command_receipts (main_ship_id, request_id): same id + same canonical payload hash → replays the committed result_json; same id + changed payload → request_id_payload_conflict; rejections write no receipt.
- **Validate-before-mutate:** every admission rejection (incl. travel_time_exceeds_limit) returns {ok:false,reason} before any write — no rejection leaves an orphan fleet/movement/ship/presence/ receipt; only a genuine integrity fault raises and rolls back.

Authoritative proof (real chain 0001..0057, disposable Supabase; osn3-s3-realchain-proof.yml). GREEN at e267eee: positives from all five origins (each asserting movement.origin == resolved origin, speed_used == resolve_fleet_movement_speed(fleet), coherent fleet/ship/receipt, presence closed once); the full rejection matrix each proven non-mutating; idempotent replay + payload conflict; real concurrent-session races (two distinct → one move, loser rejects after revalidation; two same-request retries → identical committed receipt); travel_time_exceeds_limit with explicit no-effect; runtime ACL + SET ROLE denial; REST/RPC denial of the writer + S2 helpers for anon and a real authenticated JWT; cleanup + flags/cap restored & asserted. Root-cause note: the first proof run was red on a **fixture** assumption only (the real chain auto-provisions a Home Base at (0,0) via initialize_new_player, so the zero-distance fixture's hard-coded target was wrong) — corrected by deriving every origin from mainship_space_resolve_origin; **the writer/0057 needed no change** (no 0058).

Gates (all green at e267eee): S3 real-chain proof; S1 trigger/FK + S2 real-chain regression; the Build gate via draft PR #1 (npm ci, lint, tsc -b, vite build); verify:osn:resolver (resolver unit suite). **Live read-only spot check** (osn3-s3-live-spotcheck.yml, post-deploy): 0057 applied; writer present with the exact approved signature, owner=postgres, SECURITY DEFINER, search_path=public, no dynamic SQL, acl={postgres,service_role} (anon/authenticated/PUBLIC denied); four S2 helpers service_role-only; canonical client-RPC inventory unchanged; anon/authenticated cannot CREATE in public; mainship_send_enabled=false, mainship_space_movement_enabled=false, max_coordinate_travel_seconds=86400;

`main_ship_space_movements=0` and `main_ship_space_command_receipts=0` — no coordinate movement created live, no game-state side effect. No fixtures/users/movements/receipts created by the deployment.

Scope confirmation. S3 added **no** public player RPC, UI, processor/cron, arrival settlement, Return, Stop, reconciler, repair/destruction, legacy-writer change, S2-helper change, or feature enablement. **NEXT (not started, awaiting a separate explicit S4 charter):** arrival processor (S4) → reconciler/destruction hardening (S5) → target UI (S7) → OSN-4 Stop (S8).

2026-06-21 — OSN-3 S2: internal transition boundary + validation core — CLOSED (flag OFF)

Second **OSN-3** slice (branch `osn3-s2-transition-core`, approved head `1f2c45d`, normal **no-ff** merge `93cb977`, migration `0056`).

Private, server-only transition boundary — NO movement writer, NO processor, NO Stop, NO UI, NO public RPC, NO flag change. Current `main == origin/main == a38247f` (the four commits after `93cb977` changed **only** read-only live-verification tooling: `.github/workflows/osn3-s2-live-spotcheck.yml`, `scripts/osn3-s2-live-spotcheck.sh`, `scripts/osn3-s2-live-inspect.sql`). No history rewrite / force-push / rebase / squash / reset.

Migration 0056 — four SECURITY DEFINER helpers (server-only), the locking/validation core for the future coordinate-move writer (S3+):

- `public.mainship_space_lock_context(uuid, boolean)` — acquires per-ship locks in the canonical order `main_ship_instances` → `fleets` → `main_ship_space_movements` → `location_presence`; never locks legacy `fleet_movements` (non-locking EXISTS read only); `boolean` = skip-lock (FOR UPDATE SKIP LOCKED at the ship row → returns skipped with no downstream locks).
- `public.mainship_space_validate_context(uuid)` — validates the full ship/fleet/pointer/presence state.
- `public.mainship_space_resolve_origin(uuid)` — resolves the move origin from current authoritative state.
- `public.mainship_space_assert_cross_domain_exclusion(uuid)` — enforces the legacy/coordinate domain mutual exclusion. All are owned by postgres, set `search_path = public`, and relocked so PUBLIC/anon/authenticated have **no** EXECUTE; `service_role` only. None is exposed as a player-facing PostgREST/RPC function. The canonical client-RPC grants survived the relock intact; anon/authenticated cannot CREATE in public.

Authoritative proof (real migration chain, off the shared DB). A disposable local Supabase stack applied the actual chain `0001..0056` (`osn3-s2-realchain-proof.yml`). Real concurrent psql sessions (FIFO-driven, `pg_stat_activity` wait-state + FOR UPDATE NOWAIT probes) proved the runtime lock sequence stage-by-stage (`osn3-s2-realchain-lockorder.sh`), plus blocking vs. skip-lock behavior, the legacy `fleet_movements` non-locking path, valid/contradictory state fixtures, cross-domain exclusions, ownership/pointer/presence mismatches, origin resolution, no-mutation (md5 before/after), fixture cleanup, and runtime REST/RPC denial under anon/authenticated (`osn3-s2-realchain-perm.sql`, `-fixtures.sql`, `-rest.sh`). The earlier reduced postgres:15 stub proof (`osn3-s2-transition-proof.yml`) is demoted to **supplementary / non-gating**.

Live read-only spot check (`osn3-s2-live-spotcheck.yml`, **run passed**). Method = `migration list` + `db dump` (corroboration) + an **authoritative direct catalog query** (pure SELECTs over `pg_catalog/game_config` + one count, via the Supabase pooler `aws-1-ap-southeast-1`) + REST reads. Confirmed on live: `0056` applied; all four helpers present with approved signatures, `owner=postgres`, `prosecdef=t`, `search_path=public`, `acl={postgres,service_role}` (no PUBLIC/anon/authenticated EXECUTE); canonical 13-RPC inventory preserved; anon keeps `get_world_map`; no helper authenticated-executable; schema CREATE denied to client roles; `mainship_send_enabled=false`, `mainship_space_movement_enabled=false`; `main_ship_space_movements` **row count** = 0. Note: supabase db dump is --no-owner and lossy for privileges, so owner/ACL facts come from the direct catalog query (authoritative); the dump only corroborates presence/SECURITY DEFINER/search_path. No test users, fixtures, game-state rows, coordinate movements, or flag changes were created during deployment or verification — strictly read-only.

Scope confirmation. S2 added **no** coordinate-movement writer, no coordinate return, no arrival settlement, no processor/cron, no Stop, no target UI, no public movement RPC, no public grant, no reconciler, no repair/destruction change, no legacy-writer change, no feature enablement, no frontend change. **NEXT (not started, awaiting a separate explicit S3 charter):** begin-move RPC (S3) → arrival processor (S4) → reconciler/destruction hardening (S5) → target UI (S7) → OSN-4 Stop (S8).

2026-06-21 — OSN-3 S1: coordinate-domain schema + invariants + read-model — CLOSED (flag OFF)

First **OSN-3** implementation slice (merge commit `90637d6`, branch `osn3-s1-schema-read`, migration `0055`). **Schema + read-model only — NO movement writers, NO processor, NO UI, NO Stop.** Both flags stay false (`mainship_send_enabled`, new `mainship_space_movement_enabled`). Builds on OSN-2 (the durable open-space position model). Five design gates (A → A3.2) preceded it; all blockers resolved.

Mandatory preflight (proven before deploy). A disposable postgres:15 CI container (scripts/osn3-s1-trigger-proof.sql + osn3-s1-schema-proof.sql, workflow osn3-s1-trigger-proof.yml) proved, on the real engine but off the shared DB: the `fleets.main_ship_id` **write-once** trigger (rejects reassignment / late-attach / ordinary detach), that `ON DELETE SET NULL` fires **after** the parent ship row is gone (so the trigger permits parent-deletion orphaning and existing user/ship hard-delete cleanup keeps working), and the full §5.1 constraint matrix. Bug found by the proof: the cyclic `fleets` $\rightleftharpoons$ `main_ship_space_movements` graph tripped a constraint mid-cascade on a direct ship delete → fixed by making `fleets.active_space_movement_id` FK DEFERRABLE INITIALLY DEFERRED.

Migration 0055 (additive, transactional).

- `main_ship_space_movements` — the coordinate route engine, **separate** from frozen `fleet_movements` so `process_fleet_movements` can never claim it. `target_kind` $\in$ `space|location|base` with an explicit `id-iff-kind` CHECK; all coords finite + within $[-10000, 10000]^2$; `speed_used` finite > 0; `arrive` > `depart`; `status/resolved_at` integrity; one-active partial-uniques per ship & per fleet; due-arrival index; owner-read RLS, no client write; FKs cascade on ship/fleet/user.
- `fleets.active_space_movement_id` (+ FK DEFERRABLE) — the honest moving-fleet pointer; mutual-exclusion with `active_movement_id` + requires-moving/movement CHECKS; one-fleet-per-movement unique.
- `main_ship_space_command_receipts` — `UNIQUE(main_ship_id, request_id)` + `canonical_payload_hash`; RLS on, **no** client read/write (server-only).
- `main_ship_instances.status` += 'stationary' + six legacy-safe forward lifecycle CHECKS (the reverse `stationary` rule uses `... IS TRUE` to reject `stationary+NULL`). No reverse rules for legacy statuses; no back-fill (existing rows stay `spatial_state=NULL`).
- write-once `fleets.main_ship_id` trigger; `mainship_space_movement_enabled=false`; execute relocl.

Read-model (the SINGLE resolver, extended — no second resolver). `resolveMainShipMarker` now reads the already-deployed coordinate states: `in_transit` (interpolate the active `main_ship_space_movements` row, fully validated against ship/fleet/pointer/timestamps/presence), `at_location` (validated present fleet + matching active presence), and `home` (base, no active state). Legacy NULL behavior unchanged; any contradiction → null. A new owner-read fetch of the active coordinate movement runs inside the existing 4s poll; the fleet read gains `location_mode/active_movement_id/active_space_movement_id`.

Verification (all green via CI; local toolchain unusable). Disposable trigger+schema proofs ✓; branch closure (`npm run lint + tsc -b + vite build + resolver unit tests 32/32`) ✓; migration deploy ✓; phase8 engine regression ✓; live verify:osn3:s1 **13/13** (both flags false, RLS owner-read, client writes denied, receipts unreadable, write-once trigger live, **0 coordinate rows**) ✓; live `spatial_state` distribution **56/56 NULL**. No writer/processor/UI/reconciler/repair/legacy change. **NEXT (not started):** shared transition boundary → begin-move RPC (S3) → arrival processor (S4) → reconciler/destruction hardening (S5) → target UI (S7) → OSN-4 Stop (S8). `MAX_COORDINATE_MOVE_DISTANCE` / `MAX_COORDINATE_TRAVEL_SECONDS` and the emergency processor-pause contract are deferred to those slices.

2026-06-21 — OSN-1 / OSN-2a / OSN-2b (Open-Space Navigation, read side) — CLOSED

Cross-cutting **Open-Space Navigation (OSN)** initiative (see `MAINSHIP_TRANSITION.md` §12). These stages add the main ship's single position model and a durable open-space coordinate — **read/schema only, no movement writers yet**. `mainship_send_enabled` stays **false**; engine + legacy paths frozen. (Builds on the earlier, separately-recorded main-ship transition 10C–10H + direct A→B move, which live in `MAINSHIP_TRANSITION.md` §7 rather than this log.)

OSN-1 — read-only main-ship map marker (commit 727388f). New pure resolver

`src/features/map/resolveMainShipMarker.ts` (single source of main-ship display position: `home`→`base`, `present`→`location`, `moving/returning`→`interpolate` active movement clamp 0..1, `destroyed`→`null`, `in-flight-without-movement`→`null` no-teleport) + `MainShipMarker.tsx` (pointer-transparent, 1s tick only while moving) + Playwright unit test. Flag-gated; camera/command paths untouched.

OSN-2a — durable open-space position SCHEMA (commits 1f844e9, 9534319; migration 0054). Added nullable-no-default `main_ship_instances.spatial_state` + `space_x/space_y` (double precision) as the single authoritative owner of a "stopped in open space" coordinate. CHECKS: domain `NULL|home|at_location|in_transit|in_space|destroyed`; coords both-null-or-both-set; coords present IFF `in_space`; finite-only (reject `NaN/±Inf`). **No back-fill** — existing ships stay `spatial_state=NULL` (legacy; position still from `base/fleet/movement/presence`). No functions → no relocl; RLS/grants unchanged (owner-read, no client write). `verify:osn2 23/23`. Bug fixed: ASI hazard (regex at statement start) in the verifier (9534319).

OSN-2b — resolver reads the new columns, read-model only (commits bfebb1f, 30289fe, f400ee4, 17ceb51, 8a9518d).

Extended the **single** resolver (no second resolver): `in_space`→`ship-owned space_x/space_y` (finite, no active fleet/presence); `NULL`→`legacy`, with the named-location path now deterministic (requires fleet present + `current_location_id` + matching `ACTIVE location_presence` + resolvable location, else null); `destroyed/contradiction/other`→`null`. Read-side plumbing only: `MainShipLite` + owner-read select gain the 3 columns; `fetchActiveMainShipPresence` (narrow: linked `fleet_id` +

`status='active', 3 fields, limit 1)` runs inside the existing poll; GalaxyMap threads presence into the marker. No writer/migration/RPC/flag/status/reconciler/destruction/lock change.

Closure verification (commit 8a9518d). `@playwright/test` pinned **exactly 1.61.0** (devDep + lockfile); resolver test runs via `npm ci` (dropped ad-hoc `npm install --no-save`); on-demand strict closure workflow runs **full** `npm run lint + tsc -b + vite build + resolver test`, all green; read-only verify: `osn2:distribution` confirmed the live distribution is **54/54** `spatial_state=NULL` (zero in_space/home/at_location/in_transit/destroyed — no live ship hidden by the resolver). Bugs fixed during closure: resolver workflow missing Playwright install → exit 127 (30289fe); violet in_space marker color reverted — it was LocationMarker's derelict-station color, not main-ship visual language (f400ee4); two pre-existing `Date.now()`-during-render eslint errors (MainShipPanel.tsx, MainShipMarker.tsx) fixed via the existing now-in-state tick so full repo lint is green (17ceb51).

Local toolchain note: the dev machine cannot run lint/tsc/build/playwright locally (OneDrive node_modules corruption + TLS-intercepting proxy); all verification runs in CI. Migrations through **0054. NEXT:** OSN-3 (arbitrary-coordinate movement) — Design Gate A produced; 4 open decisions before schema slice S1 (see the OSN-3 design report / MAINSHIP_TRANSITION.md).

2026-06-19 — Design correction: HIGH-STAKES ships (destructible) + emergency restart (docs)

Decision (replaces "never destroyed + self-repair"): main ships are persistent but **NOT immortal** — they **can be permanently destroyed** (gone/retired) for real strategic stakes. **Safelock rule: permanent ship loss is allowed; permanent account lockout is not.** When a player has **zero usable main ships**, grant **one weak emergency starter ship** (starter hull, **no modules, no captain bonuses, basic readiness, restart-only**) — does NOT restore the destroyed ship, does NOT refund resources, gated by **strict eligibility + cooldown** (no farming; a player with any usable ship is ineligible). Future defeat consequences: destroyed ship lost · cargo/rewards lost · modules lost/damaged/salvaged later · captains injured/rescued/captured later · surviving ships keep going.

Docs only (MAINSHIP_TRANSITION.md): rewrote §6 anti-softlock to the high-stakes destructible-ship + emergency-restart model; updated the fix-direction, softlock-coupling note, §5 model (defeat = possible permanent destruction; surviving ships remain), the ★ vision (persistent ≠ immortal), §8 residual-softlock (zero-ships → mandatory emergency replacement; airtight eligibility), §9 (no destruction/replacement in 10C; both ship together in 10E), and the phase table — **10C stays NON-COMBAT-only (no destruction); 10E renamed to destruction & safelock** (permanent destruction + emergency-replacement RPC).

Not implemented. No code, no migration, no combat change. 10C not started (awaiting separate approval). Backend unchanged.

2026-06-18 — Design correction: deprecate support capacity / support craft (UI + docs)

Decision: support capacity / support craft is **no longer part of the byeharu vision**. The core is **multiple persistent main ships + captains + modules + upgrades**. Remove support **safely** (hide → stop depending → delete), not by sudden deletion. This step: **hide from UI + mark deprecated in docs**. No backend change, no migration, no deletes.

Docs (MAINSHIP_TRANSITION.md): added a △ deprecation callout in the ★ vision (support is dormant scaffolding, not core; loadout = captains/modules/upgrades, no support craft, no capacity budget); revised the model + 10D wording; added a **"9b. Removing support — later"** safe-order section (hide → stop depending → deprecate fns → drop schema last).

UI (10B preview revised → "Main Ship" read-only view): MainShipPreview.tsx now shows the **main ship only** — name, hull, status, readiness (hp/max_hp), speed, cargo, captain slots, module slots. **Removed: support-craft picker, support-capacity bar, support-loadout wording, activity selector.** mainshipApi.ts rewritten to read `main_ship_instances` (owner-read) + `main_ship_hull_types` (public) directly — dropped `fetchSupportCraftTypes` / `fetchExpeditionPreview` (the support-laden client wrappers). Galaxy toggle relabeled "[sat] Main Ship". Still strictly read-only; no writes.

Backend: UNCHANGED. No migration. `get_my_expedition_preview`, `calculate_expedition_stats`, `support_craft_types`, and the `support_capacity` columns stay in place but **dormant + unused by the UI**. (verify:mainship-preview still exercises the dormant RPC — left as a backend regression.) **Remaining support dependencies (to remove in a later phase):** `support_craft_types` table (Phase 6 + verify-phase6); `calculate_expedition_stats` support math (Phase 8) + `get_my_expedition_preview` wrapper (Phase 10B); `support_capacity/base_support_capacity` columns; a non-displayed `support_capacity` read in `useGalaxyMapData` (Phase 9A). **Recommended later removal phase:** after the captain/module/upgrade stat source replaces the support layer and no live path calls it. **Docs + UI only; not pushed; no CI run.**

2026-06-18 — Phase 10B: read-only main-ship expedition preview (implemented; pending verify)

Scope: strict preview only — see what your main ship + a support-craft loadout WOULD bring. No writes, no sending, no combat/engine change; the Phase 9B send path is untouched. Per docs/MAINSHIP_TRANSITION.md (10B).

Migration 0049_mainship_preview.sql — `get_my_expedition_preview(p_loadout jsonb, p_activity_type text) → jsonb`.

STABLE (read-only), SECURITY DEFINER, `auth.uid()`-scoped, granted to **authenticated**. Reuses the **single stat source** `calculate_expedition_stats` (Phase 8, stays server-only — the wrapper calls it as the definer; not exposed to clients).

- Ship exists → `{has_ship:true, valid:true, ship, stats}`.
- Validation errors (over-capacity / unknown craft / bad qty) are **caught** → `{valid:false, error}` (a preview warning, not a client crash).
- No ship yet → `{has_ship:false, hull:...}` starter-hull teaser. **It does NOT commission a ship** (no write) — commissioning is a later phase.

Frontend (read-only): `mainshipApi.ts` (`fetchSupportCraftTypes`, `fetchExpeditionPreview`); `MainShipPreview.tsx` — a panel with an activity dropdown + capacity-limited support-craft picker + live stat grid + a `support_capacity` used/limit bar + warnings, labeled **"Preview only · does not send."** Wired into `/galaxy` behind a header toggle (`[sat]` Main Ship preview), **separate from the send command** (single send surface preserved).

Verify: `scripts/verify-mainship-preview.mjs` (`npm run verify:mainship-preview`, standalone — NOT wired into the chained verify): base stats · valid loadout (reuses adapter) · over-capacity → warning · unknown craft → warning · no-ship hull teaser · **wrote-nothing proof** (no-ship player still has none) · adapter still client-denied.

Untouched: combat/fleet/movement/reward/send/cleanup. No deletes, no renames. Known limit: to see loadout numbers a ship must exist; live players without one see the hull teaser (commissioning = later phase). Test main-ship rows for `mspreviewtest*` users persist (not a runtime table; tiny). **Pending build + verify (handed off to user)**.

2026-06-18 — Follow-up: M4.5 browser test self-cleaning + orphan cleanup (test hygiene)

Why The M4.5 browser test (`m45browser.*@example.com`, no "test", no cleanup step) left runtime orphans the guarded `cleanup_test_runtime` couldn't remove (3 rows: 1 fleet + 2 build_orders). Pre-existing, predates the Phase C `%test%` convention; not a 9C change.

Part A — prevent future: test email `m45browser.*` → `m45testbrowser.*@example.com`; `browser.yml` gains the shared `live-db-tests` **concurrency group** + an `if: always()` cleanup step `verify-cleanup --pattern '%m45testbrowser%@example.com'`. That pattern is unique: it can NOT match `verify` (`m45test.TAG` / `m*test` / `p*test` / `invtest`) or `galaxy` (`galaxytest*`).

Part B — remove existing orphans: one-time `scripts/cleanup-m45-orphans.mjs` (+ dispatch workflow, dry-run default). It collects runtime `player_ids`, **proves ownership via** `auth.admin.getUserById` (**email must match** `/^m45browser\./`), shows the rows, then deletes `child→parent` **only** those players' runtime rows. No TRUNCATE; no guard change; never touches `bases/inventory/main_ship/config/world`.

Result: dry-run proved 1 orphan player (`m45browser.1781756112790@example.com`) owning exactly 3 rows (1 completed fleet + 2 terminal build_orders); `--confirm` deleted them (`child→parent`). The renamed M4.5 browser run self-cleaned its own 3 rows via `%m45testbrowser%`. **verify:phase8 ✓ 21/21, galaxy 9A/9B ✓, M4.5 browser ✓** — all test data self-cleans to 0.

db:counts after = 360, and that is CORRECT (not test junk): a read-only owner diagnostic (`scripts/whoami-runtime.mjs` + `runtime-owners.yml`) showed all remaining runtime rows belong to **ONE REAL player** — `gkwnngns714@gmail.com` (the project owner's own manual galaxy-map test: an expedition to a pirate hunt → 1 fleet + 1 combat encounter, 88 ticks/264 events). **Not deleted — real player data**. Test infrastructure leftover = 0. (The 88 ticks mean a verify run had `combat_tick_logging` on during that combat; it's reset to false by m4/m5's finally and those ticks age out via Phase B 3-day retention.)

Net: M4.5 browser test is now self-cleaning + serialized; old orphans removed; no real/ config/permanent data touched; no TRUNCATE; no gameplay change. **Follow-up CLOSED**.

2026-06-18 — Phase 9C: Expedition UI Reframe (BUILD + VERIFY + BROWSER GREEN ✓)

Request Make the player understand: Galaxy Map = where you send expeditions; Command Center = status + shortcuts; fleet status area = active/returning/completed. Remove duplicate send controls. Frontend/copy only — no backend.

Duplicate removed: the old in-dashboard `SendFleetPanel` (list-based send) duplicated the Phase 9B map send → **deleted** (`src/features/fleets/SendFleetPanel.tsx`; only the Dashboard imported it). `/galaxy` is now the **only** send surface.

Reframe (frontend only):

- `ExpeditionLauncher.tsx` (new) — replaces the dashboard send panel with a pointer card: "Send your first from the Galaxy Map", a prominent **Open Galaxy Map** button, and the reward rule in plain words ("pending while out · secured only on return"). Empty-vs-active copy. `testid dashboard-expedition-launcher`.
- `Dashboard.tsx` — swaps `SendFleetPanel` → `ExpeditionLauncher`; header already links to the map. No other send control remains.
- `FleetStatusPanel.tsx` — kept the `Fleets` heading + "previous run(s)" wording (m45 selectors) but added a subtitle ("Active expeditions — travel, on-station, and returns"), reframed the empty state to **"No active expedition. Send your first from the Galaxy Map →"** (links to `/galaxy`), and made the status badge **activity-aware** (present + hunt → "Fighting", else "On station"). Existing reward wording kept ("rewards locked (secured on arrival)").

No backend touched — no migration, no RPC/combat/reward/return/cleanup change, no second map or send flow. Phase 9B send logic unchanged.

Tests: `galaxy9b.spec.ts` gains a check that the Command Center shows `dashboard-expedition-launcher` and has **no** "Send a fleet" control (single-surface proof). 9A/9B/m45 selectors preserved.

Result (commit aaea9d5): build/typecheck + lint ✓, Pages deployed. **verify:phase8 ✓ 21/21. Browser: 9A 1/1, 9B 1/1** (incl. single-send-surface assertion), **M4.5 1/1** (confirms the `FleetStatusPanel` reframe kept its selectors). **db:counts runtime = 0.** No backend/migration/table-write change; `/galaxy` is the only send surface. **Phase 9C CLOSED.**

2026-06-18 — Phase 9B: Map-based Expedition Send (BUILD + VERIFY + BROWSER GREEN ✓)

Backend path inspection (done before wiring — no backend change, no migration):

- **RPC used:** `send_fleet_to_location(p_base uuid, p_location uuid, p_units jsonb)` (migration 0019), via the existing wrapper `sendFleetToLocation(baseId, locationId, units)` in `fleetApi.ts`. This is the same path Phase 8's chain (verify-m4) drives — already verified.
- **Inputs:** `base id, location id, units = [{unit_type_id, quantity}]`.
- **Success:** `{ fleet_id, movement_id, arrive_at }`.
- **Failure:** raises → `supabase-js` returns `error.message`; the wrapper throws it.
- **Backend-authoritative validation (already present):** `base owned+active · location valid/ active · activity_type ∈ {none, hunt_pirates} · active-fleet-limit (max_active_fleets=3, counts moving/present/returning) · units non-empty · units available & positive (via base_reserve_units, which also reserves them so the same units can't be re-sent) · fleet power ≥ min_power_required. → it already blocks invalid sends, over-limit/duplicate active expeditions, insufficient units, and invalid/locked destinations. No second expedition system created.`

Implementation (frontend only):

- `useGalaxyMapData.ts` — additionally loads `unitTypes` (catalog, static) + `baseUnits` (polled) so the command area can offer a loadout. Still read-only fetches.
- `ExpeditionCommand.tsx` (new) — replaces the disabled 9A placeholder. Compact unit picker + Send → **confirmation step** → calls `sendFleetToLocation` **exactly once** (synchronous `sendingRef` guard + `sending` state → double-submit-proof). Shows sending / success / error states + a disabled reason. **No optimistic movement** — on success it calls the hook's `refresh()`; the movement line appears only from refetched `movements`.
- `GalaxyMapScreen.tsx` — wires the command area into the detail panel; passes `base/units/ unitTypes`; `onSent` → `refresh`.
- `LocationMarker.tsx` — adds `data-activity / data-location-id` (test selectors). `FleetMovementLine.tsx` — adds `data-testid="galaxy-movement-line"`.
- **Frontend-only checks (clarity, not authority):** no destination / non-dispatchable activity / no units selected / already sending → disabled with a reason. Everything real (ownership, limits, units, power, validity) stays backend-authoritative; backend errors are surfaced verbatim.

No direct table writes from the UI — the only write is the approved `send_fleet_to_location` RPC. No combat/reward/return/cleanup/logging logic touched.

Tests: `galaxy.spec.ts` updated (9A read-only smoke kept; send button asserted disabled before a loadout). `galaxy9b.spec.ts` (new) — select a dispatchable marker → pick units → send → confirm (double-clicked) → success → assert **exactly one** fleet+movement via backend read → movement line on map → no dup from double-submit → send disabled before units →

no console errors. `browser-galaxy.yml` runs both then `verify:cleanup` (test email contains `test` so `cleanup_test_runtime` removes its runtime rows → `db:counts` back to 0).

Result (commit `aefd5ea`): build/typecheck ✓ (lint clean after remount-via-key + CSS cursor fixes), Pages deployed.

verify:phase8 ✓ 21/21 ... M4 40/40 (run alone). **Browser: 9A smoke 1/1, 9B send 1/1. db:counts runtime = 0** (galaxy cleanup scoped to `%galaxytest%`). **Transient verify failure root-caused + fixed:** I'd dispatched the browser suite + verify concurrently; the browser workflow's broad `%test%` cleanup deleted verify's in-flight phase5 fleet mid-combat → "no wave cleared". Fixed: shared `live-db-tests` concurrency group on both workflows + galaxy cleanup narrowed to `%galaxytest%` (can't touch verify's `m*/p*` users). Re-run verify alone = 21/21. **No backend/migration/table-write/second- expedition-system added. Phase 9B CLOSED.**

2026-06-18 — Phase 9A: Read-only Visual Galaxy Map (BUILD + VERIFY GREEN ✓)

Request First visual galaxy map screen — read-only, using existing backend world data. See the world/locations/home/ship/active movements; select a location for details. No commands, no writes, no backend change.

No backend change needed. All data already exists: `get_world_map()` (sectors→zones→ locations with `x,y`), `bases` (`x,y,name`), `fleet_movements` (`origin_x/y` + `target_x/y` stored → paths drawable directly), `location_state` (pressure/danger), `main_ship_instances` (owner-read). Confirmed before building; **no migration added.**

Files (all `src/features/map/`, matching the existing feature structure):

- `useGalaxyMapData.ts` — read-only hook: world map + base once; polls movements + location states + a small `main_ship_instances` owner-read every 4s. Builds location→sector/zone meta.
- `GalaxyMap.tsx` — plain **SVG** 2D map (no canvas/WebGL). Normalizes world coords into a 0..1000 `viewBox`; transform group gives pan (drag) + zoom (wheel/+/-/reset). Renders movement paths, home/ship anchor, and location markers. Labels hidden when zoomed out.
- `LocationMarker.tsx` — colored marker + truncated label, counter-scaled to stay constant on-screen size; selecting only highlights.
- `FleetMovementLine.tsx` — dashed origin→target path (amber outbound / sky return) + ETA (`formatCountdown`). Purely visual.
- `GalaxyMapScreen.tsx` — page with loading / error / empty / selection states + a **read-only detail panel** (name, sector/zone, type, coords, status, difficulty/reward, live world state) + a disabled "Send expedition (Phase 9B)" button + "coming in Phase 9B" note.
- `fleetTypes.ts` — additive: `origin_x/y`, `target_x/y` (already returned by `select('*')`).
- `App.tsx` — new `/galaxy` route (`RequireAuth`). Nav links added from Dashboard + `MapPage`.

Read-only guarantees: the screen calls only read paths (`get_world_map`, table selects on `location_state/fleets/fleet_movements/main_ship_instances`). No RPC mutation, no `send_fleet`, no table writes. Action-implying controls are disabled/labeled Phase 9B.

Result (commit `c1de252`): frontend **build/typecheck** ✓ (`tsc -b && vite build`), **Pages deployed** (`/galaxy` live), **verify:phase8** ✓ (Phase 8 21/21 ... M4 40/40; frontend can't affect backend). `db:counts` unaffected (auto-cleanup ran). Manual interactive browser check not runnable from the dev sandbox (no GUI/network) — offered a Playwright smoke test as follow-up. **Phase 9A code complete + CI-green.** Phase 9B: click-to-select destination + expedition send.

Follow-up — Playwright `/galaxy` smoke (commit `6d84d19`): `tests/galaxy.spec.ts` signs in, opens `/galaxy`, asserts the map + ≥1 marker render, selecting a marker opens the read-only detail panel, the Send button is **disabled + Phase-9B**, and **no fleet/movement is created** (read-only proof), failing on serious console/page errors. Stable testids added (`galaxy-map-screen/-loading/-error`, `galaxy-location-marker`, `galaxy-location-detail-panel`, `galaxy-send-expedition-disabled`). `verify:galaxy:browser` script + `browser-galaxy.yml` dispatch. **Result: smoke 1/1 ✓, build ✓, verify:phase8** ✓ (21/21 ... M4 40/40), **db:counts runtime = 0.** No backend/migration/write change.

2026-06-18 — Prevention Phase C: self-cleaning verify runs (DEPLOYED + VERIFIED ✓)

Request Stop verify runs leaving runtime/test rows behind. Minimal + safe; no gameplay/ combat/reward/movement/report changes; no TRUNCATE; no real/config/permanent data touched.

How test data is identified (no `test_run_id` added): every verify script signs up throwaway users with emails matching `%test%@example.com` (`m4test/m5test/m45test/invtest/ p4test...p8test`), and every runtime table carries `player_id`. So

verify-created runtime rows = rows owned by a test-email player. The email pattern is the cleanup key — the existing convention made a schema column unnecessary.

Migration 0048 `cleanup_test_runtime.sql`: `cleanup_test_runtime(p_pattern default '%test%@example.com', p_dry_run default true) → returns (table_name, rows_matched, rows_deleted, cleanup_key)`. Deletes ONLY the 9 runtime tables (+ fleet_units) for test-email players, child→parent. Guards: pattern MUST contain test (else raises); **never** touches auth.users, bases, base_units, base_resources, player_inventory, inventory_ledger, main_ship_instances, *_types, game_config, or world tables. No TRUNCATE. SECURITY DEFINER, service_role only.

Files: migration 0048; scripts/verify-cleanup.mjs (verify:cleanup:dry-run / verify:cleanup --confirm, optional --pattern); package.json; verify-phase8.mjs prints a cleanup reminder at the end; verify.yml adds a final if: always() **auto-cleanup step** so every CI verify removes its own test data (even on failure).

Result (commit 2ac700f): migration 0048 deployed ✓. verify:phase8 ✓ (Phase 8 21/21 ... M4 40/40). Auto-cleanup deleted **728 runtime rows** (matched == deleted every table — the whole accumulated test backlog + this run). db:counts after: **all 10 runtime tables = 0**. No TRUNCATE; no auth.users/bases/inventory/main_ship/config/world touched. **Phase C CLOSED — prevention complete (A logging controls · B retention cleanup · C self-cleaning verify)**.

2026-06-18 — Prevention Phase B: safe retention cleanup (DEPLOYED + VERIFIED ✓)

Request Add a batched, dry-run-first retention cleanup. No TRUNCATE, no destructive reset, no active/seeded/player-owned data touched.

Schema reconciliation (inspected, not assumed) — reported deviations:

- combat_ticks has resolved_at, not created_at → index + rule use resolved_at.
- fleet_movements has no updated_at → index + rule use resolved_at (set on resolve).
- reward_grants has granted_at, not claimed_at → use granted_at. (There is no "pending"/"claimed" state in this table — a grant row IS an already-secured deposit; pending rewards live on combat_encounters/fleet_movements jsonb and are untouched.)

Cascade hazard (inspected): ON DELETE CASCADE roots everything at fleets (→ fleet_units, fleet_movements, location_presence, combat_encounters → ticks/events/reports; presence → encounters too). Since combat_reports (30d) hangs under encounters/presence/ fleets, deleting any ancestor would cascade-delete a still-retained report. So **encounters, location_presence, and fleets are additionally gated**: never deleted while they have an ACTIVE encounter or a RETAINED (<30d) report. Net: those three are effectively kept until their report expires; non-combat presence (no report) still cleans at 1 day.

Migration 0047 `runtime_retention_cleanup.sql`:

- 10 indexes (CREATE INDEX IF NOT EXISTS) on the real scan columns (3 substituted per above).
- maintenance_cleanup_runtime_data(dry_run boolean default true, batch_limit int default 5000) → returns (table_name, retention_rule, rows_matched, rows_deleted, dry_run) per table. Batched deletes via ctid in (... limit batch_limit) loops — never one-shot, never TRUNCATE. Kill-switch: if runtime_cleanup_enabled=false, forced to dry-run. SECURITY DEFINER, service_role only.
- Retention: ticks 3d, events 7d, reports 30d, encounters terminal>14d(+guard), presence terminal>1d(+guard), movements terminal>14d, fleet_units (parent terminal>14d), fleets terminal>14d(+guard), reward_grants 30d, build_orders terminal>30d.
- Safety: only TERMINAL statuses deleted (active/retreating/moving/waiting/queued never match); active encounters/fleets/movements/rewards/builds never deleted; no bases/ base_resources/base_units/inventory/main_ship/_types/game_config/world tables referenced.

Files: migration 0047; scripts/db-cleanup.mjs (db:cleanup:dry-run / db:cleanup --confirm); package.json; .github/workflows/db-cleanup.yml (dispatch; dry-run default, deletes only on confirm=true; shows size/counts before+after).

Fix during deploy: first push failed 42P13 — input param dry_run collided with the OUT column dry_run; renamed inputs to p_dry_run/p_batch_limit (project p_-convention), OUT column stays dry_run. **Result (commit dac35a1):** migration 0047 deployed ✓. **Dry-run: 0 matched across all 10 tables** (all data fresh — nothing past the 3/7/14/30-day cutoffs); **live run (confirm=true): 0 matched / 0 deleted** — delete path executes cleanly, nothing destructive. **verify:phase8 ✓ — Phase 8 21/21 ... M4 40/40** (indexes + function did not affect combat/regression). **Phase B CLOSED**. Next: Phase C (self-cleaning verify runs).

2026-06-18 — Prevention Phase A: combat logging controls + DB visibility (DEPLOYED + VERIFIED ✓)

Request Stop byebaru re-filling the disk: make high-volume combat logging opt-in and add size/row visibility. No deletes (that's Phase B), no combat-outcome changes.

Migration 0046_combat_logging_controls.sql:

- `cfg_bool(key)` accessor + `set_game_config(key, value jsonb)` (service_role/CI only).
- 7 `game_config` flags (insert-if-absent): `combat_debug_logging=false`, `combat_tick_logging=false`, `combat_event_logging=true`, `runtime_cleanup_enabled=true`, `combat_tick_retention_days=3`, `combat_event_retention_days=7`, `combat_report_retention_days=30`.
- `process_combat_ticks` (same combat math) now **gates logging**: all `combat_ticks` inserts behind `combat_tick_logging` (default OFF → no per-tick rows); per-unit `hull_damage` events behind `combat_debug_logging` (default OFF → kills the worst per-tick multiplier); other meaningful events behind `combat_event_logging` (default ON → UI animation + reports intact). `v_seq` still advances so display ordering is unchanged.
- `db_table_sizes()` (top-20 by `pg_total_relation_size`) + `db_runtime_counts()` (10 runtime tables) — service_role only.

Default logging after this change: per combat tick we now write **0** `combat_ticks` rows and **0** `hull_damage` events (was 1 tick + N `hull_damage`); only milestone/animation events (`wave_spawned`, `missile_salvo`, `laser_burst`, `unit_destroyed`, `explosion`, `retreat`) remain. `combat_reports` (player-facing summary) untouched.

Regression compatibility: verify-m4 + verify-m5 inspect `combat_ticks`, so they flip `combat_tick_logging` on via `set_game_config` at start and **restore it off in finally** (shared DB → production default stays off). Only those two scripts read ticks.

Visibility: `scripts/db-size.mjs` (`npm run db:size`) + `scripts/db-counts.mjs` (`npm run db:counts`), plus a `db-report.yml` dispatch workflow to run both in CI.

Restrictions honored: no TRUNCATE, no deletes, no seeded/config/world/player tables touched.

Result (commit e3d0ba4): migration 0046 deployed ✓. `db:size` + `db:counts` work (post-cleanup DB tiny — largest table 120 kB; 240 total runtime rows). **verify:phase8** ✓ — **Phase 8 21/21, Phase 7 18/18, Phase 6 10/10, Phase 5 25/25, Phase 4 16/16, Inventory 18/18, M4.5 27/27, M5 28/28, M4 40/40** (incl. "waves last 3+ ticks" — m4/m5 tick-toggle works). **Phase A CLOSED**. Next: Phase B (retention cleanup function, dry-run first).

2026-06-18 — Phase 8: calculate_expedition_stats() (DEPLOYED + VERIFIED ✓)

Request Build the deterministic stat ADAPTER that will eventually turn Main Ship + Support Craft (+ later Captains + Modules) + Activity into final expedition stats — the bridge between the new main-ship model and the proven engine. **Read/compute only**; no mutation; engine unchanged; live combat still uses the old fleet-stack path.

Migration 0044_calculate_expedition_stats.sql — one function, `calculate_expedition_stats(p_player, p_main_ship_id, p_loadout jsonb, p_activity_type)` returns `jsonb`. SECURITY DEFINER, **STABLE (read-only)**, **service_role only**:

- Reads the owned `main_ship_instances` row (+ `main_ship_hull_types` for `base_speed`); errors if the ship isn't found/owned. Validates `activity_type` ∈ {`pirate_hunt`, `trade_run`, `exploration`, `mining`, `none`}.
- Normalizes the support loadout: **combines duplicate** craft ids; **rejects** unknown types, and non-positive / non-integer / NaN / Inf quantities.
- **Enforces** `support_capacity` as a **HARD cap** — `used = Σ(qty × capacity_cost)`; over the ship limit → rejected. This is the anti-unlimited-stacking mechanism (never a plain sum).
- Effects (conservative, linear within the cap) derive from each craft's Phase-6 `base_stats_json` (attack→`combat_power`, defense→`survival`, repair, cargo, scan→`scouting`, mining→`mining_yield`, evasion→`retreat_safety`) plus role rules for `pirate_attention` (`combat_damage/cargo` +2, `heavy_cargo` +4) and a speed penalty (`combat_damage` 0.05, `heavy_cargo` 0.08, `extraction` 0.02). Non-useful-for-activity craft add a non-fatal warning.
- Returns normalized, **clamped (≥0)**, **rounded** stats: `support_capacity_used/limit`, `speed`, `cargo_capacity`, `combat_power`, `survival`, `retreat_safety`, `scouting`, `mining_yield`, `repair`, `pirate_attention`, `warnings[]` — **never NaN, never negative, deterministic**.

Read/compute only (verified): mutates nothing — ship row + inventory unchanged after many calls; no fleets/combat/rewards/ranking/reports touched. **NOT wired into live combat** — the fleet-stack path still owns outcomes (M2–M5 untouched). Support-craft OWNERSHIP isn't implemented yet, so Phase 8 validates **type/capacity/math** against `support_craft_types` only; ownership consumption comes when loadouts attach to real expeditions.

Anti-cheat: new function default-grants to PUBLIC on create → re-ran the lockdown (revoke; re-grant the 8 client RPCs; `calculate_expedition_stats` → service_role only). A client preview RPC (`auth.uid()`-scoped) will arrive with the Phase 9 UI.

Boundaries/docs: SYSTEM_BOUNDARIES decision #8 (table-less read/compute adapter, mutates nothing). ROADMAP Phase 8 ✓. ARCHITECTURE Phase 8 note. ACTIVITIES: documented which stats each activity will read from the adapter later.

Verify: `scripts/verify-phase8.mjs` — base stats on empty loadout (0/10, speed 1, cargo 50); mixed loadout capacity (7/10); reject over-capacity / unknown / zero / negative / non-integer; duplicate-combine; per-craft effects (missile_boat→combat+attention/speed, cargo_drone→cargo+attention, survey→scouting, mining→yield, decoy→retreat, repair→repair+ survival); no-NaN; determinism; ship + inventory not mutated; client-denied; then chains `verify-phase7` (full regression). CI runs `verify:phase8`.

Status (commit 5a4c954): Migration 0044 **deployed** ✓ (direct-Postgres push succeeded). **Verification BLOCKED by a Supabase infra issue, not code:** every REST/RPC request returns `upstream request timeout` — including a trivial read of the public `main_ship_hull_types` table (which touches no Phase 8 code), persisting 13+ min across two runs. The REST/PostgREST layer is globally unresponsive (DB accepts direct connections — deploy worked — but the API gateway times out). Needs the Supabase project checked/restarted (paused / compute-exhausted / stuck schema reload), then re-run `verify:phase8`.

Resolution: free-tier disk was full (combat-log churn from dozens of verify runs). Cleared via one-time migration `0045_dev_cleanup_churn` (TRUNCATE of 10 throwaway runtime/log tables over the working direct-Postgres connection — user-authorized), then a dashboard **project restart** bounced the stuck PostgREST. **Verify** ✓ — **Phase 8 21/21, Phase 7 18/18, Phase 6 10/10, Phase 5 25/25, Phase 4 16/16, Inventory 18/18, M4.5 27/27, M5 28/28, M4 40/40. Phase 8 CLOSED.** Follow-up: Phases A–C add logging controls + safe retention cleanup + self- cleaning verify so the disk can't fill again.

2026-06-18 — Phase 7: Main Ship Instance (DEPLOYED + VERIFIED ✓)

Request Create the player's ONE main ship — the player identity, not stackable, one active per player. Additive foundation only: no combat hook, no support-craft attachment, no `calculate_expedition_stats`, no capacity enforcement. Engine untouched.

Migration `0043_main_ship_instance.sql` — two tables + three server-only functions:

- `main_ship_hull_types` (Reference/Config, public-read): one starter hull `starter_frigate` — base_hp 500, base_speed 1.0, cargo 50, **support_capacity 10**, captain_slots 2, module_slots 3. (Conservative; not final balance.)
- `main_ship_instances` (Main Ship system, owner-read, **no client write**): `player_id` UNIQUE (one per player), hull FK, name default 'Byeharu', status CHECK in the 10 allowed states (default home), hp/max_hp/cargo_used/cargo_capacity/support_capacity/captain_slots/ module_slots with `>=0/>0` checks. Stats are copied from the hull on creation so the instance can later diverge (damage/upgrades) without mutating the template.
- `ensure_main_ship_for_player(player)` — idempotent, concurrency-safe via the `player_id` UNIQUE (on conflict do nothing → select) → one ship per player. `get_main_ship(player)` read helper. `rename_main_ship(player,name)` — trims, rejects empty + >40 chars, requires an existing ship. All SECURITY DEFINER, **service_role only** (clients read their ship via owner-read RLS; no client mutation/RPC path).

What did NOT change (by design): combat, fleets, `fleet_movements`, presence, production/ build queue, rewards, inventory, support_craft metadata. No fleet-table renames. Player- creation path (`initialize_new_player`) untouched — the ship is created on demand via `ensure_main_ship_for_player` (a future bootstrap/RPC will call it). Anti-cheat: new functions default-grant to PUBLIC → re-ran the lockdown (revoke; re-grant the 8 client RPCs; ensure/get/ rename → service_role). Prior service_role grants untouched.

Boundaries/docs: `main_ship_hull_types` added to Reference/Config; new **Main Ship** owner row (sole writer of `main_ship_instances`) + per-system contract + ownership decision #7. ROADMAP Phase 7 ✓. ARCHITECTURE Phase 7 note (ship exists, doesn't drive expeditions yet).

Verify: `scripts/verify-phase7.mjs` — starter hull public-read + client-write-blocked; ensure creates exactly one ship (idempotent, no dup); owner-read + cross-user RLS; client INSERT/UPDATE/DELETE + server-RPCs all blocked; stats valid & copied from hull; status defaults `home`; rename trims + rejects empty/overlong/no-ship; then chains `verify-phase6` (full regression) to prove the engine is unchanged. CI runs `verify:phase7`.

Result (commit 05b1cc5): Deploy ✓ · Build ✓ · Pages ✓ · Verify ✓ — **Phase 7 18/18, Phase 6 10/10, Phase 5 25/25, Phase 4 16/16, Inventory 18/18, M4.5 27/27, M5 28/28, M4 40/40** (M2 11 / M3 13 chained), 0 failed. Ship created hp 500/500, support 10, captain 2, module 3, status `home`; idempotent; client writes + server RPCs blocked. Migration 0043 live on `dlkbwztrdvnjlvaydut`. **Phase 7 CLOSED.**

2026-06-18 — Phase 6: Support Craft Reframe (DEPLOYED + VERIFIED ✓)

Request Reframe "build ships" toward the future "build support craft / expedition equipment" model — **metadata foundation only**, no engine change. Support craft must be **capacity-limited loadout choices, not unlimited additive power**. No instances, no expedition attachment, no `calculate_expedition_stats`, no capacity enforcement yet.

Migration `0042_support_craft_types.sql` — one Reference/Config table (mirrors `item_types`): `support_craft_type_id` PK, name, role, capacity_cost int check (>0), stackable, buildable, activity_tags jsonb, tradeoffs_json, base_stats_json.

Public-read RLS, **no write policy / no write grant** → **clients cannot mutate**. Seeds the **8 starter craft** with real roles + capacity costs + tradeoffs:

- scout_escort (light_escort, cap 1) · missile_boat (combat_damage, cap 3) · repair_drone (repair, cap 2) · cargo_drone (cargo, cap 2) · survey_drone (scanning, cap 2) · decoy_drone (retreat_safety, cap 1) · mining_drone (extraction, cap 2) · trade_barge (heavy_cargo, cap 5).
- base_stats_json is illustrative only — **nothing consumes it yet** (Phase 8).

What did NOT change (by design): combat (unit_types scout/corvette/frigate untouched, separate namespace), the serial build queue / build_orders / train_units, fleets, movement, rewards, inventory. No fleet-table renames. No new functions (so no execute-lockdown needed). Frontend wording left as-is to avoid risking the M4.5 browser acceptance; the build-queue reframe is conceptual/documented (M4.5 = Serial Build Queue Foundation; ARCHITECTURE

- SYSTEM_BOUNDARIES updated).

Boundaries/docs: support_craft_types added to the Reference/Config sole-writer row; new ownership decision #6 (metadata only, capacity enforced later by main ship + calculate_expedition_stats). ROADMAP Phase 6 ✓. ARCHITECTURE Phase 6 note.

Verify: scripts/verify-phase6.mjs — 8 definitions exist & public-read; capacity_cost > 0 matching documented costs; every craft has role + activity_tags + tradeoffs; zero overlap with combat unit_types (engine untouched); client INSERT/UPDATE blocked by RLS; then chains verify-phase5 (→ phase4 → inventory → m45 → m5 → m2/m3/m4) to prove combat + serial queue unchanged. CI runs verify:phase6.

Result (commit 4038209): Deploy ✓ · Build ✓ · Pages ✓ · Verify ✓ — **Phase 6 10/10, Phase 5 25/25, Phase 4 16/16, Inventory 18/18, M4.5 27/27, M5 28/28, M4 40/40** (M2 11 / M3 13 chained), 0 failed. 8 craft seeded, capacity 1–5, client writes blocked, zero overlap with combat unit_types. Migration 0042 live on d1kbwztrdvnjlvaydut. **Phase 6 CLOSED.**

2026-06-18 — Phase 5: Multi-Item Pirate Loot (DEPLOYED + VERIFIED ✓)

Request Pirate combat should accrue real item drops alongside metal — a controlled combat-reward DATA change, not an engine rewrite. Reuse the proven Phase 4 bundle; keep the reward timing law; metal stays in base_resources; server-authoritative loot only. No crafting/modules/captains/UI.

Migration 0041_pirate_loot.sql — two isolated, server-only helpers + a 3-line injection into the existing combat tick:

- pirate_loot_for_wave(p_wave int, p_danger numeric) — the loot table. **Deterministic** (no RNG → stable tests), small/clamped, **only Phase-3-seeded ids**: scrap (every wave), pirate_alloy (≥3), weapon_parts (≥5), engine_parts (≥8), repair_parts (≥10). p_danger reserved for future scaling; v1 keeps qty=1 so survival can't make loot explode. Returns [] below wave 1 (no NaN, no unknown ids).
- loot_merge_items(a, b) — combines two items[] by id (summed) to keep the accumulated bundle tidy across waves. (reward_grant also de-dups on deposit — belt & suspenders.)
- process_combat_ticks (copied verbatim from 0030) gains exactly three PHASE-5 lines: declare v_loot_items; on wave-clear set v_loot_items := pirate_loot_for_wave(wave,danger) and put it in reward_delta; merge items[] into total_rewards_json next to the accumulated metal. Everything else — carry-home, retreat, defeat-forfeit ('{}'), secured-on-arrival — is unchanged.

Reward flow (all unchanged from Phase 4): drops are pending in total_rewards_json → ride reward_payload_json home → reward_grant on arrival splits metal→base_resources, items→player_inventory (idempotent). Defeat clears the bundle → forfeits metal AND items. Retreat alone never secures.

Boundaries/docs: ACTIVITIES.md pirate_hunt loot section made concrete (server-side only; rare progression ids reserved). Combat still owns only its reward accrual — it writes the pending bundle, never Inventory/Base directly. Frontend unchanged (no client loot path).

Anti-cheat: new helpers default-grant to PUBLIC on create → 0041 re-runs the lockdown (revoke from public/anon/authenticated, re-grant the 8 client RPCs; loot helpers → service_role for CI only). reward_grant/inventory_* service_role grants untouched.

Verify: scripts/verify-phase5.mjs — (A) deterministic loot-table + merge helpers (positive ints, known ids, clamped, dedup), (B) **real combat**: items appear in total_rewards_json, stay pending through retreat, deposit to player_inventory + base_resources on home arrival, report keeps metal; (C) **defeat** forfeits metal+items; (D) chains verify-phase4 (→ inventory → m45 → m5 → m2/m3/m4). CI runs verify:phase5.

Result (commit bf32dbf): Deploy ✓ · Build ✓ · Pages ✓ · Verify ✓ — **Phase 5 25/25, Phase 4 16/16, Inventory 18/18, M4.5 27/27, M5 28/28, M4 40/40** (M2 11 / M3 13 chained), 0 failed. Real run banked metal +38 and scrap +1 on arrival; defeat forfeited the bundle. Migration 0041 live on d1kbwztrdvnjlvaydut. **Phase 5 CLOSED.**

2026-06-17 — Phase 4: Pending Loot Bundle (DEPLOYED + VERIFIED ✓)

Request Generalize the metal-only pending reward into a future-proof `PendingRewardBundle { metal?, items:[{item_id,quantity}] }`. Backend plumbing only — no new pirate drops, no trading/mining/crafting/UI. Keep the reward timing law exactly: pending while travelling · secured **once on home arrival** · forfeited on defeat · retreat doesn't secure. Metal stays in `base_resources`; items go to `player_inventory`.

Key finding (no schema change needed). The pending bundle already rides existing jsonb columns end-to-end: combat accrues → `combat_encounters.total_rewards_json` → (on exit) `fleet_movements.reward_payload_json` (via `movement_attach_cargo`) → (on arrival) `reward_grant('combat', encounter, player, base, bundle)`. So Phase 4 is a **single function change** — additive, no new column, no rename.

Migration `0040_pending_loot_bundle.sql` — rewrites `reward_grant()` (the secured- deposit owner) to **split the bundle**:

- metal (and any scalar resource) → `Base.base_add_resources(p_rewards - 'items')`. The - 'items' strip is essential: `base_add_resources` casts every jsonb value to double and would choke on the items array.
- `items[]` → `Inventory.inventory_deposit(player, item, qty, key)` with key `'<source_type>:<source_id>:<item_id>'`.
- **Idempotency**: metal guarded by `reward_grants` `UNIQUE(source_type,source_id)` (one grant/source, early-return on replay) **plus** the inventory ledger key — both metal and items double-deposit-proof across cron retry / reprocessing.
- **Fail-safe validation**: items deduped by id (quantities summed), filtered to positive integers < 1e9 (rejects negative/zero/NaN/Infinity); unknown item ids skipped with a logged `WARNING`; per-item + outer exception isolation so one bad entry never forfeits the metal or the valid items.
- Anti-cheat: create or replace preserves the 0039 client-revoke; added grant `execute ... reward_grant ... to service_role` (server/CI only, never clients) so the verifier can drive it — mirrors `inventory_deposit` / `process_build_queue`.

Boundaries: `SYSTEM_BOUNDARIES.md` — Reward now splits the bundle (sole caller of `base_add_resources`; calls `Inventory.inventory_deposit` for items). Call graph stays acyclic (Reward → Base, Reward → Inventory). Combat/movement unchanged; combat still accrues **metal only** (no new drops — that's Phase 5). Reports keep `total_rewards_json` (metal display intact; items ride along for free, display deferred).

Verify: `scripts/verify-phase4.mjs` — drives `reward_grant` directly as `service_role` (metal-only · metal+items · idempotent re-grant · per-source key · unknown-item-safe · duplicate-combine · negative/zero/NaN-skip · empty-bundle no-op · client-denied) then chains the regression (`verify-inventory` → m45 → m5 → m2/m3/m4) which proves the end-to-end timing law (defeat forfeits, retreat doesn't secure, reports keep metal). CI `verify.yml` now runs `verify:phase4`.

Result (commit 4e1d7eb): Deploy ✓ · Build ✓ · Pages ✓ · Verify ✓ — **Phase 4 16/16, Inventory 18/18, M4.5 27/27, M5 28/28, M4 40/40** (M2 11 / M3 13 chained), 0 failed. Migration 0040 live on `d1kbwztrdvnnjlvaydut`. **Phase 4 CLOSED.**

2026-06-17 — Phase 3: generic inventory foundation (DEPLOYED + VERIFIED ✓)

Request Clean generic item inventory for future rewards/materials. Metal stays in `base_resources` (untouched); a future loot bundle deposits metal → `base_resources`, items → `player_inventory`. No trading/mining/crafting/etc.

Migration `0039_inventory.sql`:

- `item_types` (Reference/Config, public read) + 10 starter items (scrap, ore, crystal, pirate_alloy, weapon_parts, engine_parts, repair_parts, captain_memory_shard, blueprint_fragment, artifact_core).
- `player_inventory` (PK (player_id,item_id), quantity >= 0) — **owner-read RLS, no client write**. `inventory_ledger` (audit + `unique(idempotency_key)`) — owner-read.
- Functions (SECURITY DEFINER, server-only): `inventory_deposit(player,item,qty,key?)` (validates item+qty, upserts, **idempotent via the ledger key**), `inventory_spend` (transactional FOR UPDATE, rejects insufficient, **never negative**), `inventory_get_balance`. Lockdown re-grant (clients unchanged; `inventory_*` → `service_role`).

Boundaries: new **Inventory** system owns `player_inventory+inventory_ledger`; `item_types` = Reference/Config. Metal/`base_resources` **untouched**; combat/movement/ world-state/reward unchanged.

Verify: `scripts/verify-inventory.mjs` (11 tests: seed, owner-read, cross-user RLS, client-cannot-mutate, deposit-adds, idempotent deposit, spend-subtracts, insufficient, no-negative, unknown-item, regression). CI `verify.yml` now runs `verify:inventory` (chains M4.5 → M5 → M2/M3/M4).

Result (commit 49cc946): Deploy ✓ · Build ✓ · Pages ✓ · Verify ✓ — **Inventory 18/18, M4.5 27/27, M5 28/28, M4 40/40** (M2/M3 chained green), 0 failed. Migration 0039 live on `d1kbwztrdvnnjlvaydut`. **Phase 3 CLOSED.**

2026-06-17 — Phase 2: Expedition Activity Architecture (design doc only)

Request Define the clean activity abstraction so future gameplay types plug into the Expedition Engine without spaghetti. Docs only — no code, no migrations, no src/.

Work: new docs/ACTIVITIES.md covering the 10 required items — ExpeditionActivityType (pirate_hunt / trade_run / exploration / mining, mapped to the existing activity_type enum placeholders); shared lifecycle owned by the Engine (travel · arrival · presence · dispatch · pending-reward accrual · return · secured-on-arrival deposit · status · reports); per-activity ownership table; the **Activity Handler contract** (<activity>_create + process_<activity>_ticks cron + optional _request_leave + Engine.finish) — grounded in the existing activity_start router + the Combat precedent; PendingRewardBundle ({ metal?, items[] }); history-only report/result shape; "add an activity = enum value + handler + one dispatch line + one panel" (no giant switch); the anti-spaghetti call graph (activity → pending → secure-on-return → inventory → progression → ranking); explicit non-goals; acceptance criteria.

No code / migrations / src/ changes. ROADMAP Phase 2 marked done → ACTIVITIES.md. M2 11/11 · M3 13/13 · M4 40/40 · M4.5 27/27 unaffected (nothing executable changed). **Next:** Phase 3 (generic inventory) when chosen.

2026-06-17 — Phase 1: roadmap / architecture reconciliation (docs only)

Request After M4.5, make the docs match the real game direction — **main-ship expedition game**. Documentation only; no gameplay code; M2/M3/M4/M4.5 stay green.

Work (docs only):

- **New docs/ROADMAP.md** — the authoritative forward direction: final identity (one main ship + captains + modules + support craft → expedition → activity → return → inventory → progression → ranking); reclassification (**M2–M4 = Expedition Engine**, **M4.5 = Serial Build Queue Foundation**); standing laws (support craft = capacity-limited loadout, not additive power; one-directional pipeline activity → pending → secure-on-return → inventory → progression → ranking; don't replace the engine, replace the source of expedition stats via calculate_expedition_stats); the Phase 1–20 plan.
- **README** — intro reframed to main-ship expedition; milestones reclassified (Engine + Build Queue Foundation done) + forward direction → ROADMAP; removed the stale "M7 not started" / combat-reward-only framing.
- **ARCHITECTURE §16** — direction-update note + reclassification + pointer to ROADMAP.

Not built (deferred to later phases): main ship · captains · modules · inventory · trading · exploration · mining · ranking. No migrations, no frontend behavior change. M2 11/11 · M3 13/13 · M4 40/40 · M4.5 27/27 unaffected. **Next:** Phase 2 (expedition activity architecture, design only) when chosen.

2026-06-17 — ✓ M4.5 CLOSED (browser acceptance passed)

The automated **Playwright browser acceptance** test passed against the live Pages site — M4.5's manual gate is met, so M4.5 is closed.

- **Browser test:** tests/m45.spec.ts (verify:m45:browser), CI workflow .github/workflows/browser.yml, run against <https://gkwns714-spec.github.io/byebaru/>. **1 passed (17.3s)**. Verified live: friendly coords (Sector 0:0, no raw "0, 0") · Train Scout ×5 active row (Per ship / Total order / Ship 1 of 5 / Remaining ticking / "delivered when full order completes") · Corvette ×2 waiting (no countdown, no Ship N) · cancel inline confirm (Refund + Penalty + Keep Building + Confirm Cancel) · Keep Building doesn't cancel · Confirm refunds **once** (+125 = 50%) and the next waiting starts · refresh = no duplicate refund, cancelled gone · completed-history fold/unfold. Screenshots + traces uploaded as the playwright-m45 CI artifact.
- **Backend:** verify:m45 27/27; regression M2 11/11 · M3 13/13 · M4 40/40; CI build green. No gameplay/migration changes for the test (test infra only).

M4.5 reframed for the future as the **Serial Build Queue Foundation** (see [[byebaru-final-direction]]) — Main Ship + Support Craft). **Next:** Phase 1 docs/roadmap reconciliation (docs only).

2026-06-17 — M4.5 Core UX + production queue law fix (CLOSED — see entry above)

Status: NOT closed. Fixes to the **M7 production queue** + two UI bugs (build_orders is the M7 system — M5/M6/M7 already done; full M2–M7 kept green). Migration 0038.

Production now SERIAL (was accidentally parallel — every order got `complete_at` on creation): `build_orders` gains `waiting/active` states, nullable `complete_at`, `started_at`; config `max_active_ship_production_slots=1` (designed to become N). `train_units` enqueues **waiting** then `production_start_next` promotes one to **active** (absolute `started_at` + `complete_at`). `process_build_queue` completes due **active** orders then starts the next. Waiting items have **no** `complete_at` and don't tick.

Cancellation: `cancel_build_order` RPC — server-authoritative; validates ownership + status; **waiting** → **100% refund**, **active** → **50%, completed/cancelled** → **rejected** (refund via `Base.base_add_resources`). Cancelling the active item starts the next waiting one.

UI: `BuildQueuePanel` shows active (countdown) vs waiting (no countdown) + Cancel buttons; `FleetStatusPanel` completed-history fold fixed (was an empty `<details>`) → controlled toggle "Show N previous run(s)" / "Hide previous run(s)" with real content; new `src/lib/location.ts` `formatLocationLabel` + `BasePanel` replace raw "0, 0" with "Sector 0:0" / friendly names.

Boundaries: Production-only; combat/movement/world-state/reward untouched; absolute timestamps (no per-tick decrement). `SYSTEM_BOUNDARIES` Production row already covers it.

Verify: `scripts/verify-m45.mjs` (serial · completion-starts-next · cancel waiting/active · cannot-cancel-completed · ownership · anti-cheat · regression) — **supersedes** `verify-m7` (parallel model; removed). CI `verify.yml` now runs `verify:m45`.

Closure gate (pending): deploy 0038 · `verify:m45` green · M2–M5 regression · CI build · browser check (serial countdown, cancel works, history folds, friendly coords).

2026-06-17 — M7 Ship Training (implemented; pending deploy/verify + click-through)

Status: NOT closed. Training-first ship production — the spending loop: **spend metal** → **queue training** → **cron completes ships into** `base_units`. Metal-only, timed queue, no buildings/shipyard/research/captains/trade/mining/multi-resource.

Migrations 0035–0037:

- `0035_unit_costs.sql` — `unit_types.metal_cost` (scout 50 / corvette 150 / frigate 400); config `build_time_scale=1.0`, `min_build_seconds=5`, `max_build_orders=5`.
- `0036_production_system.sql` — `build_orders` table (Production-owned, RLS owner-read, no client writes); `base_spend_resources` (Base fn); `production_create_order/complete_order`; `train_units` RPC (auth → validate ownership/unit/qty/metal/queue-cap → `Base.spend` → `Production.create`); `process_build_queue` cron fn (FOR UPDATE SKIP LOCKED; idempotent — only queued→completed, ships never double-added); lockdown re-grant (+`train_units` to authenticated, `process_build_queue` to service_role).
- `0037_cron_build_queue.sql` — `process-build-queue` every 30s.

Frontend: `features/production/{productionTypes,productionApi,TrainShipsPanel, BuildQueuePanel}`, `game/production/buildPreview` (cost+ETA preview, non-authoritative), `catalog.ts` +`metal_cost`, `useGameState` +`build_orders`, Dashboard composes. Player wording: **Train Ships / Training Queue / Not enough metal**. Only new action = `train_units`.

Boundaries: Production = sole writer of `build_orders` only; **never** writes `base_units/base_resources` (spends via `Base.base_spend_resources`, deposits via `Base.base_merge_units`). Acyclic Production→Base. Reward logic unchanged (only reads/debits metal). No combat/world-state/movement changes.

Verify: `scripts/verify-m7.mjs` (16 tests) + `verify:m7`; CI `verify.yml` now runs `verify:m7` (chains `m5` → `m2/m3/m4`).

Closure gate (pending): deploy 0035–0037 · `verify:m7` green · M2–M6 regression · CI build/typecheck · browser check (Train Ships + Training Queue render, train works, ships appear).

2026-06-17 — M5 balance correction: pressure decay toward baseline (follow-up #3, Option A)

Request Fix the M5 issue where, with no players, every `pirate_hunt` location drifted to pressure 100 / Severe and punished new players. **Option A only** (pure decay) — no newbie zones, no new columns, no Option B/C.

Change (migration 0034_worldstate_pressure_decay.sql): `worldstate_tick` passive pressure now **DECAYS toward baseline** instead of drifting up: `pressure += (baseline - pressure) * decay_rate - active_fleets * relief`. The step is a fraction of the gap, so it asymptotes to baseline and **never overshoots** (`decay_rate` in (0,1]). Empty locations return to **NORMAL** (baseline 50 → danger_modifier **exactly 1.0** ≈ M4); hunting still relieves below baseline; future defeat/event pressure can still raise it above baseline (defeat_pressure remains a TODO, unwired). New config key `worldstate_pressure_decay_rate = 0.1`. `danger_modifier` mapping unchanged.

M5 law preserved: World State still sole writer of `location_state/zone_state`; combat **reads** `danger_modifier` only; presence is source of truth; `active_fleets` stays a reconciled cache; cron unchanged (`process_location_state_ticks` → `worldstate_tick`). No new schema/columns, no newbie zones, no frontend / combat / reward / fleet / presence changes.

Verify: verify-m5 Test 2 changed from drift-up to decay (above→down, below→up, at-baseline stays + modifier exactly 1.0, no overshoot, clamped); Test 4 relief made deterministic. M2/M3/M4 regression unchanged.

2026-06-17 — ✓ M6 CLOSED (frontend depth / player clarity)

M6 browser re-test passed; milestone officially closed.

Closure evidence:

- M6 browser re-test passed.
- CI build/typecheck passed.
- Reports now show readable time.
- Round logs show per-round time.
- "en route" was removed from player-facing UI.
- Fleet wording is clearer: Traveling / Returning home.
- Dev-only ship grant script was added.
- No backend logic changed.
- No migrations changed.
- No combat math changed.
- No reward logic changed.
- M2–M5 backend systems remained untouched.

Open follow-ups (tracked separately — NOT part of M6): pre-existing react-hooks lint cleanup · stuck throwaway test users/presences cleanup · danger pressure balance / newbie-safe zones. **Next:** M7 (not started).

2026-06-17 — M6 Frontend Depth (implementation record — CLOSED above)

Status: CLOSED 2026-06-17 (see closure entry above). Implemented and CI-verified to compile; closure gate is a manual browser click-through (below). Player-clarity pass over the M2–M5 loop — **frontend only:** no migrations, no backend/combat/reward math, reads server truth only.

Created (5): `src/game/worldstate/danger.ts` (shared display labels + High/Severe warning), `src/features/map/LocationPanel.tsx`, `src/features/combat/RoundLog.tsx` (real `combat_ticks` fields only), `src/features/combat/CombatReportPage.tsx (/reports)`, `.github/workflows/build.yml`.

Modified (9): `combatApi.ts` (+read-only `fetchTicksForEncounter`, owner-RLS), `useGameState.ts` (+`location_state` poll), `MapPage.tsx` (clickable cards → panel), `SendFleetPanel.tsx` (pre-dispatch danger preview/warning), `FleetStatusPanel.tsx` (lifecycle wording), `ActiveCombatPanel.tsx` (RoundLog replaces debug table), `CombatReportsView.tsx` (link to `/reports`), `Dashboard.tsx` (pass states + nav), `App.tsx (/reports route)`.

CI build/typecheck — ✓ green (run 27656389298): `tsc -b` pass, `vite build` pass (92 modules). `lint` is **non-blocking** and flagged 3 **pre-existing** M3/M4 files (`useState(Date.now())`, `void refresh()` in effect — strict react-hooks v7); none of the new M6 files. CI frontend verification is required since local npm is unreliable (see `[[byecharu-build-onedrive-bug]]` equivalent note).

Backend untouched: zero migration/SQL/RPC changes; push did not trigger deploy/verify. M5-close verification (M5 25/25 · M4 40/40 · M3 13/13 · M2 11/11) stands.

M6 closure gate (manual browser click-through — all must pass):

1. Map card click opens LocationPanel.
2. LocationPanel shows correct danger/activity + warning.
3. SendFleetPanel shows pre-dispatch danger preview.
4. FleetStatusPanel lifecycle wording is clear.
5. ActiveCombatPanel shows RoundLog (not the debug table).
6. Retreat/return messaging is understandable.
7. `/reports` opens correctly.
8. A past report can load its RoundLog.
9. No obvious broken layout.
10. No frontend writes to World State.

Deferred (out of M6): pre-existing react-hooks lint cleanup; danger-decay balance (separate small migration only if the UI proves misleading — not rebalanced here).

2026-06-17 — ✓ M5 CLOSED (deployed + verified green in CI)

Migrations 0031–0033 deployed to the remote via the GitHub Action, and the new **Verify** workflow ran the full suite on CI (Node 22). All green:

- **M5: 25/25 · M4: 40/40 · M3: 13/13 · M2: 11/11** — 0 failures.
- M5 coverage proven: world-state rows seeded, passive drift, register/relief/ unregister edges, active_fleets reconciliation, double-tick idempotency, and combat safely reading danger_modifier at a high-pressure location.
- M4 balance confirmed untouched (baseline pressure → danger_modifier 1.0).

Bugs found + fixed during deploy/verify (couldn't surface without a live DB/CI):

1. **pg_cron** '60 seconds' **invalid** (SQLSTATE 22023) — sub-minute syntax is 1–59s; 60s must be standard cron '* * * * *'. Fixed in 0033. (031/032 had already applied; 033 rolled back cleanly and re-applied after the fix.)
2. **CI on Node 20 threw "Node.js 20 detected without native WebSocket support"** — supabase-js 2.108's realtime client needs native WebSocket (Node 22+). Bumped verify.yml to Node 22.

Verify CI: secrets VITE_SUPABASE_URL / VITE_SUPABASE_ANON_KEY / SUPABASE_SERVICE_ROLE_KEY configured; workflow auto-runs after each deploy and on manual dispatch. Verification no longer depends on the local toolchain.

Next: M6 (frontend depth) per docs/ARCHITECTURE.md §16.

2026-06-17 — M5 Living World (built; pending deploy + verify)

Request Build M5 per the "Living World Design Law": world-state pressure + danger drift + location dynamics via a 60s cron, **without rewriting** the M2–M4 loop. Strict ownership (World State sole writer of location_state/zone_state), combat may only read danger_modifier, acyclic cron, anti-cheat lockdown.

Step 0 inspection (key findings)

- No worldstate_* / location_state / zone_state existed — only deferred-stub comments (0002, 0008). Built fresh.
- **Single unregister seam:** every terminal presence transition (escape, defeat, safe-leave) funnels through presence_complete() → one hook, not six.
- **Combat touches one function:** combat_create_encounter starts enemy_integrity_current = 0, so wave 1 spawns inside process_combat_ticks; the danger read goes there only.

Work done (migrations 0031–0033)

- 0031_worldstate_tables.sql: location_state (pressure/danger_modifier/ active_fleets/last_tick_at) + zone_state rollup; public-read RLS, no client write; seeded one row per location/zone.
- 0032_worldstate_fns.sql: 10 game_config keys (no magic numbers); worldstate_register_presence / worldstate_unregister_presence (cache ±1) / worldstate_tick() (reconcile active_fleets from real presences → drift/relief if elapsed ≥ min → bounded danger_modifier → zone rollup); service-role-only dev_worldstate_prime test helper; **edges wired** by re-creating presence_create (→ register) and presence_complete (→ unregister), behavior otherwise identical; **combat read** added to process_combat_ticks (× a fallback-guarded danger_modifier, else 1.0); re-locked execute surface.
- 0033_cron_location_state.sql: process_location_state_ticks() → only worldstate_tick(); pg_cron every 60s. Cadences now 30s / 2s / 60s.

Balance safety (Rule F): danger_modifier is **piecewise with baseline → exactly 1.0**, and seed pressure = baseline = 50 → fresh locations multiply combat by 1.0, so M4 numbers are unchanged until pressure actually drifts.

Frontend (minimal, read-only): mapTypes.ts LocationState; mapApi.ts fetchLocationStates() (public read); MapPage.tsx shows "Pirate activity: Calm/Rising/Severe" + "Danger: Low/Medium/High" on pirate_hunt cards. No writes.

Verification: scripts/verify-m5.mjs + verify:m5 — Tests 1–9 (rows, drift, register, relief, unregister, reconcile, danger-feeds-combat, double-tick idempotency, M2/M3/M4 regression). Uses a **service-role key** to drive the locked worldstate_tick()/dev helper (clients stay denied), mirroring the dev_reset_player precedent.

Not yet run (gated on user): fresh clone has no .env.local and migrations aren't on the remote. Local npm install/build also blocked by a known npm bug on this OneDrive path (optional wasm deps @tailwindcss/oxide-wasm32-wasi etc. fail to reify → "Exit handler never called", no .bin shims). **To finish M5:** supabase db push, add SUPABASE_SERVICE_ROLE_KEY to .env.local, npm run verify:m5.

CI: added `.github/workflows/verify.yml` — runs `verify:m5` (chains M2/M3/M4) on ubuntu after the deploy workflow succeeds, or via manual dispatch. Sidesteps the local npm/TLS toolchain blockers. Needs repo secrets `VITE_SUPABASE_URL`, `VITE_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`.

Also: reconciled README milestone list to the real M1–M6 roadmap.

2026-06-17 — M4 cleanup (loose ends; verified 40/40)

1. Reward deposit → home arrival. Combat no longer deposits at escape. On escape/auto-extract the pending rewards are attached to the return movement (`fleet_movements.reward_grant_source + reward_payload_json` via new `movement_attach_cargo()`), and `process_fleet_movements()`'s **return-arrival branch** deposits them via `reward_grant` (idempotent unique source). Defeat → none (zeroed). Deferred so future en-route risk/cargo "just works." **2. Config extraction.** Added `reward_danger_scale=0.25`, `danger_time_divisor_seconds=180`, `combat_damage_variance_pct=0.10`, `defense_curve_base=100`; `process_combat_ticks` now reads them. No combat magic numbers remain in code. **3. Dead code.** Dropped `fleet_apply_losses()` (superseded by `combat_units + fleet_sync_quantities`; confirmed no live caller).

UI: combat pending note "secured only after your fleet returns to base"; returning fleet shows "[$\$$] rewards locked (secured on arrival)"; report "rewards secured when it reaches base".

Files: `0030_m4_cleanup_reward_on_arrival.sql`; `scripts/verify-m4.mjs`; `fleetTypes.ts`, `FleetStatusPanel.tsx`, `ActiveCombatPanel.tsx`, `CombatReportsView.tsx`; `SYSTEM_BOUNDARIES.md`. Backend: 1 migration. Frontend: wording/types only.

Verify: `verify:m4` **40/40** (escape: not deposited; return carries rewards; arrival deposits exactly once +metal; defeat/retreat-death: none; destroyed don't return), `verify:m2` 11/11, `verify:m3` 13/13.

M4 closed — no known loose ends.

2026-06-17 — M4 CLOSE (combined final pass; all verified)

Part 1 — retreat + wording

- Retreat delay **20s → 8s** (config `retreat_delay_seconds`; UI countdown reads it).
- Report wording "Return movement started." → "Fleet escaped — now returning to base." Banner → "fleet breaks away and heads home in Ns." Combat-state label friendly ("In combat" / "Next wave incoming" / "Retreating").

Part 2 — edge cases (verify:m4 37/37): destroyed-during-retreat → defeat (no reward/return); retreat spam → exactly one accepted; **destroyed ships do NOT return** (base = initial – lost, e.g. scout 98 after losing 2); one-encounter-per-fleet; reward once (idempotent); safe-zone & invalid-location rejected; defeat leaves no stuck presence. Browser-refresh/offline: all combat state is server-side (cron-driven), UI reloads from backend — survives refresh/close. M2 11/11, M3 13/13 (no regressions).

Part 3 — cleanup

- Dev helper `dev_reset_player(uuid)` added — SECURITY DEFINER, **not granted to clients** (SQL-editor/service-role only): clears stuck combat/movement/presence.
- Reward-securing rule: granted at **escape** (combat end). Return trip is uninterruptible (no en-route combat), so this == "secured on guaranteed return"; death only happens pre-escape → no reward. Kept as-is (would move to home-arrival only when en-route risk exists).
- Hard-coded values to extract in a future balance pass: reward danger factor 0.25, danger time-divisor 180s, $\pm 10\%$ variance, defense curve $100/(100+\text{def})$.

Files: migrations `0027` (wave HP), `0028` (retreat 8s), `0029` (`dev_reset`); `scripts/verify-m4.mjs`; `ActiveCombatPanel.tsx`, `CombatReportsView.tsx`.

M4 is safe to close. Remaining (low) risk: balance not tuned to fleet power; weapon cooldowns prepared not implemented; a few hard-coded balance constants.

2026-06-17 — M4 final checklist audit (all pass)

Request 22-point M4 final checklist before moving on.

Already passing (no change): combat start rules, ownership/RLS, one-active- encounter-per-fleet (partial unique indexes), fixed 3s tick, wave transition, pirate scaling (HP+attack+reward all danger-scaled), player damage (single aggregate, no double-count), per-group damage distribution, per-unit integrity, damage carryover, ship destruction, retreat behavior, defeat behavior, reward

behavior (idempotent), combat feed, debug (combat_ticks incl. unit_snapshot_json), processor idempotency (FOR UPDATE SKIP LOCKED), client/server authority, boundaries, final summaries.

Needed change: wave pacing was ~2 ticks for a modest fleet at low danger (undertuned vs the 3-6 target). Fix 0027: enemy_hp_base 6→14 → easy waves ~3+ ticks, scaling to normal/strong with danger. Added verify cases C (damage w/o loss), F (one encounter/fleet), G (safe zone starts no combat), pacing assert ≥3, defeat leaves no active presence.

Files: supabase/migrations/0027_wave_hp_pacing.sql, scripts/verify-m4.mjs. **Backend:** 1 config value (wave HP). **Frontend:** none.

Verification: verify:m4 33/33, verify:m2 11/11, verify:m3 13/13 — no regressions (checklist J). Wave 504→320 (dealt 185), 3+ ticks/wave; survivors report {scout:7,frigate:2,corvette:5}.

Remaining M4 risk (low): wave HP scales with danger, not fleet power → a massively-overpowered fleet still clears low-danger waves fast (acceptable/by design); weapon cooldowns prepared but not implemented; per-unit before/after captured in combat_ticks but not surfaced in the UI debug table. Deep balance deferred.

2026-06-17 — M4 combat clarity pass (verified 28/28)

Request Combat now feels like a survival loop. Small clarity improvements + fixed-interval tick confirmation.

Backend (0026)

- Combat tick **2/4s** → **fixed 3s** (cron + config; damage keeps ±10% variance, the interval is fixed/non-random per design). Confirmed fixed-interval model; per-group damage loop already structured for future weapon/unit cooldowns (not implemented yet).
- Added combat_reports.survivors_json; report_create now records exact survivors + losses from per-unit combat_units (drives the post-retreat summary).

Frontend (clarity)

1. Latest exchange while retreating: "Your fleet is retreating — weapons disengaged" + "Pirates dealt N damage during disengagement" (no more confusing "0 damage").
2. Pending rewards note: "Locked — secured only if your fleet returns home safely" (and not-secured warning while active).
3. Retreat banner: "Retreating — return movement starts in Ns" (ties to M3 spine).
4. Per-unit rows show "alive/original ships (N lost) · HP · %".
5. Post-retreat **result summary** in Combat reports: result, waves, ships returned, ships lost, rewards secured/forfeited, "Return movement started."
6. Top line: "Wave 3 · Danger 3 · 2 waves cleared · Retreating".

Verification — verify:m4: **28/28** (incl. report survivors {scout:7,frigate:2,corvette:5}). Boundaries intact: server-authoritative; client renders + retreat only; M3 movement used only after retreat succeeds; no captain/trading logic.

2026-06-17 — M4 combat overhaul: pacing + per-unit HP (verified 27/27)

Request Browser feedback: waves one-shot (HP 195 vs 385 dmg), no visible wave progress, only total fleet HP, unclear feed. Make combat readable + per-unit correct.

Root cause Wave HP and wave damage were the SAME number → a 385-attack fleet one-shot a 195-HP wave. Fixed by decoupling: wave **HP** scales large with danger; wave **attack** is a separate, smaller danger-scaled value.

Backend (migrations 0023–0025)

- 0023: tick 2s→**4s**; config knobs enemy_hp_base(6), enemy_hp_danger_scale(0.6), enemy_attack_base(1.0), enemy_attack_danger_scale(0.25), wave_transition_seconds(3). New table combat_units (per-unit-type combat HP: ship_hp, initial/alive count, hp_max/current, carries over between waves). combat_create_encounter snapshots it; process_combat_ticks rewritten: decoupled HP/attack, **server-side damage distribution across unit groups by ship count**, deterministic ship loss (alive = ceil(hp/ship_hp)), next_wave_at transition, richer event payloads, fleet_sync_quantities to write survivors back to Fleet. encounter wave_number; ticks wave_number + unit_snapshot_json.
- 0024: re-lock execute (also block anon/authenticated default).
- 0025: fleet_sync_quantities → **SECURITY INVOKER** (Supabase re-grants execute to authenticated on new fns and resists revoke; invoker means a client call runs as authenticated with no fleet_units UPDATE grant → denied; internal caller runs as owner → works). Grant-independent lockdown.

Frontend

- `combatTypes/combatApi/useCombat`: `CombatUnit` + fetch `combat_units`; encounter wave fields. `ActiveCombatPanel`: total + **per-unit-type integrity bars** (alive/initial ships, HP, %), wave-incoming display, "latest exchange", richer debug.
- `CombatEventLayer`: meaningful text ("Missile salvo hit the pirate wave for N damage", "Pirates damaged Corvette group for N hull", "N Scout destroyed", "Wave N cleared. +M metal pending", "Wave N incoming").

Verification — `verify:m4`: 27/27

- Lockdown: `process_combat_ticks` / `fleet_sync_quantities` / `base_add_resources` denied.
- A: multi-tick wave (HP 252→37, dealt 215; not one-shot), per-unit HP present + decreasing via distribution, metal accrued, retreat→escaped, reward once, +metal, return via M3.
- B defeat: 0 rewards, base unchanged, no return, destroyed. C retreat-death: same.

Remaining: wave pacing is multi-tick but on the short side (~2 ticks for a strong fleet at low danger); deeper balance deferred per request — tunable via `game_config` (`enemy_hp_base`, `enemy_hp_danger_scale`).

2026-06-16 — M4 fixes from browser feedback (verified 26/26)

Request Browser testing surfaced issues. Fix before M4 complete.

Fixes

1. **Reward-on-defeat bug (critical, backend)**: defeat kept accrued `total_rewards_json`, so the report/pending looked rewarded. Migration 0022: on defeat (both paths) `total_rewards_json = '{}'`, no `reward_grant`, no `base_add_resources`, no return. `reward_grant` only ever called on escaped/completed.
2. **Integrity model (backend)**: added `player_integrity_max/current`, `enemy_integrity_max/current` on encounters and `*_integrity_before/after` on ticks. Persistent integrity pool decreases each tick (visible HP), unit losses incremental-proportional → explains "hull damaged, no ships destroyed". Frontend: Fleet/Pirate-wave HP bars + "Latest exchange" (you dealt / they dealt / losses).
3. **Retreat reward-locking (backend)**: while `retreating`, fleet takes damage but deals none, clears no waves, accrues no rewards (locked at retreat). 0022 adds `retreat_started_at`; frontend shows "Retreating — escaping in Ns" countdown.
4. **Completed history**: collapsed into "Completed history: N previous run(s)".
5. **Wording**: "use the Retreat button in the combat panel" (non-positional).
6. **Balance**: left as-is per request (combat still easy; tune later).

Verification — `verify:m4`: 26/26 PASSED

- Anti-cheat lockdown (4 fns denied).
- A escape: integrity exposed, pending accrued, retreat → escaped, rewards locked (no farming), `reward_grants` ×1, base metal +once, return created.
- B defeat (1 scout): defeat, destroyed, report 0 rewards, 0 `reward_grants`, base unchanged, no return.
- C retreat-death (6 scouts): defeat, 0 rewards, base unchanged, no return.
- (verify script bug fixed: `.catch` on supabase builder → plain await.)

Deploy: GitHub Action ✓ (migration 0022). Frontend build green (88 modules).

2026-06-16 — M4 frontend (active combat UI, display-only)

Request Build the M4 frontend only (no backend changes): `ActiveCombatPanel`, `CombatEventLayer`, combat reports view, ~1–2s combat polling, `SendFleetPanel` allows `pirate_hunt`. Client display-only; `combat_events` cosmetic; `combat_ticks` truth/log; keep boundaries.

Work done (files)

- `src/features/combat/` — `combatTypes.ts`, `combatApi.ts` (read encounters/events/ ticks/reports + `request_retreat`), `useCombat.ts` (1.5s poll), `CombatEventLayer.tsx` (cosmetic missile/laser/explosion feed), `ActiveCombatPanel.tsx` (danger/waves/ survivors/pending rewards/Retreat + `combat_ticks` debug log), `CombatReportsView.tsx`.
- `SendFleetPanel.tsx` — dispatch to safe **and** `pirate_hunt` locations (danger label
`<font name="Sym"> </font>` combat warning).
- `FleetStatusPanel.tsx` — present hunt fleets show "in combat" (retreat via combat panel).
- `Dashboard.tsx` — renders `ActiveCombatPanel` per active encounter + `CombatReportsView`, using a separate faster `useCombat` poll. `index.css` — `bh-fade-in` for event feed.

Boundaries: client display-only; only action is `request_retreat`; no client math; events cosmetic, ticks read-only. No backend changes.

Verification: npm run build green (88 modules, no type errors); dev server HTTP 200 at http://localhost:5173/. Visual click-through handed to user.

2026-06-16 — M4 backend: server-authoritative pirate combat (verified)

Request Build M4 backend: active-feeling combat (2s server ticks), 20s retreat, single-resource metal rewards, 30-min forced auto-extract safety cap. Server owns all outcomes; client animates cosmetic events later. Strict boundaries; backend first.

Security finding (fixed in this milestone) Probed the live DB: M1–M3 internal SECURITY_DEFINER functions (e.g. base_reserve_units, fleet_set_present, process_fleet_movements) were **client-callable** — Postgres grants EXECUTE to PUBLIC by default and PostgREST exposes the whole public schema. That's an anti-cheat hole (client could mutate units/fleet state). Fixed in 0021_lock_function_execute: revoke execute on all public functions from public/anon/authenticated, alter default privileges to block future leaks, and grant execute only on the 6 client RPCs (get_world_map, bootstrap_me, send_fleet_to_location, request_leave_location, request_retreat, get_combat_reports). Verified denied post-deploy.

Work done (migrations 0012–0021)

- Base base_add_resources; Fleet fleet_combat_stats + fleet_apply_losses.
- Combat tables combat_encounters / combat_ticks (truth log) / combat_events (cosmetic stream); Reward reward_grants + idempotent reward_grant; Report combat_reports + report_create + get_combat_reports.
- combat_create_encounter, combat_set_retreating, process_combat_ticks() (2s, FOR UPDATE SKIP LOCKED, idempotent; one tick row + several event rows; wave scaling, power combat, losses, rewards, defeat/escaped/completed).
- Presence activity_start routes hunt_pirates→Combat; presence_request_leave combat-retreat branch. Player RPCs: allow hunt sends (+min_power), request_retreat.
- Config: combat_tick_seconds 12→2, retreat_delay 30→20, max_presence_seconds 1800, reward_metal_base 10. Cron process-combat-ticks every 2s.

Deploy: GitHub Action run 27623526054 ✓ — 0012–0021 applied (incl. 2s cron + lockdown).

Verification — verify:m4: 20/20 PASSED

- Lockdown: 4 internal fns denied to client.
- Success: dispatch hunt → arrival → encounter active → ticks/waves/events accrue (danger rising) → retreat → escaped → fleet returning + return movement → reward granted exactly once (315 metal in base). verify:m3 still 13/13 (lockdown safe).
- Defeat: 1 scout vs Pirate Den → wiped → defeat → fleet destroyed → defeat report → no return, no reward.

Next: M4 frontend (ActiveCombatPanel + cosmetic CombatEventLayer) — awaiting go.

2026-06-16 — ✓ M3 COMPLETE

Browser click-through passed; M3 accepted. Criteria met: units return correctly, fleets complete correctly, no duplicate fleets, no console errors, no backend errors. One UI wording bug found + fixed (arriving in arriving... → awaiting server confirmation... once the client clock hits zero, while the cron resolves; backend untouched).

M4 requirement captured (user): combat must feel MORE active than movement. Movement stays slow (cron ~30s OK). Combat needs **faster server combat steps** (tune game_config.combat_tick_seconds) and **client-side** combat_events **for missile/laser visuals** — cosmetic, driven by server-authoritative results, never client authority. Do NOT optimize movement's zero-countdown gap.

M4 not started — awaiting go-ahead.

2026-06-16 — M3 frontend (Command Center)

Request Build the M3 frontend to click the live loop: base → send fleet → countdown → present → leave → return → units restored. Keep modules separated; client only requests + renders; M2 map read-only.

Work done (files)

- src/game/movement/travelPreview.ts — client ETA PREVIEW math only (mirrors server formula; not authoritative).
- src/lib/catalog.ts — shared unit_types read.
- src/features/base/ — baseTypes.ts, baseApi.ts (ensureBase/fetch*), BasePanel.tsx (base + resources + units at base).
- src/features/fleets/ — fleetTypes.ts, fleetApi.ts (send/leave + reads), SendFleetPanel.tsx (pick safe location + quantities, preview ETA), FleetStatusPanel.tsx (status/dest/countdown + leave button).

- `src/features/dashboard/useGameState.ts` — single 3s poll loop; panels stay presentational. `Dashboard.tsx` composes the panels (Command Center).

Boundaries: base UI in features/base, fleet UI in features/fleets, preview-only math in game/movement; M2 map untouched/read-only; no client-side game authority (all mutations via RPCs); reusable for future combat/trading/captains.

Verification

- `npm run build green` (tsc + vite, 83 modules, no type errors).
- Dev server serving HTTP 200 at `http://localhost:5173/`.
- Backend loop already proven by `verify:m3` (13/13) — frontend calls the same RPCs.
- Visual/console click-through: handed to user (browser).

Bugs / fixes

- `_(none in build)_`

2026-06-16 — M3 backend built, deployed, and verified live

Request Build M3 (movement + presence spine, no combat), deploy via GitHub Action, verify the full backend loop. Keep systems separated; server authoritative.

Work done

- M3a migrations `0003-0005`: `game_config`, `unit_catalog`, `base_system` (bases/units/resources + `initialize_new_player` + `signup bootstrap` + `backfill`).
- M3b migrations `0006-0011`: `fleet_system`, `movement_system`, `presence_system`, `movement_processor`, `player_rpcs`, `cron_movement` (`pg_cron` 30s).
- Switched deploy to the free GitHub Action (3 secrets in GitHub UI). First run failed at Link project — invalid `SUPABASE_ACCESS_TOKEN` secret; after user re-added a valid `sbp_` token, re-run succeeded.
- Wrote `scripts/verify-m3.mjs` (throwaway-user integration test) + `verify:m3`.

Deploy result — GitHub Action run 27619768482: ✓ success

- Migrations `0003-0011` all applied to remote, incl. `0011` (`pg_cron` enabled, job `process-fleet-movements` scheduled every 30s, no permission error).

Verification — `verify:m3`: **13/13 PASSED** `bootstrap` → `base` → starting units(100/20/5)+resources → dispatch to "Safe Rally Point" → movement row (5.0s, dist 12.1) → units reserved 100→90 → processor resolves arrival → fleet present + presence active(none) → leave → return movement (`return_home`) → processor resolves → fleet completed → survivors merged 90→100.

Bugs / fixes

- Deploy 1 failed: bad `SUPABASE_ACCESS_TOKEN` secret (JWT could not be decoded) → user re-added valid token → re-run green.
- `verify:m3 v1`: Supabase rejected `.test` email domain + a Node/libuv exit crash (auth auto-refresh timer). Fixed: use `@example.com`, `autoRefreshToken:false`, clean exit via `process.exitCode`.
- Email confirmation was ON → signup rate-limited; user disabled "Confirm email".

Follow-ups

- A few throwaway `m3test.*@example.com` users exist in auth (each with a base); harmless, can prune later.
- M3 frontend (base view, send-fleet panel, fleet status) is next.

2026-06-16 — M2 verified live against real Supabase

Request Verify M2 against a real database before M3. Apply migrations (no manual SQL paste, no secrets in chat).

Setup

- Supabase project created (ref `d1kbwztrdvnnj1vaydut`, Free plan, Asia-Pacific).
- GitHub repo `gkwngns714-spec/byebaru` (private) created; full project pushed.
- User chose Supabase's **native GitHub integration** + connected the repo.

Work done

- `.env.local` written with Project URL + **publishable** key (`sb_publishable_...`); git-ignored. Frontend uses publishable key only (never secret/service_role).
- Secrets handled via local git-ignored `supabase/.secrets.env` (access token + db password), loaded into transient env vars, **never** printed or committed; file deleted immediately after `db push`.

- Applied migrations via `npx supabase link + npx supabase db push (20260616000001_init_profiles, 20260616000002_world_map)`.

Result — `npm run verify:m2: 11/11 PASSED`

- Data: 2 sectors / 2 zones / 5 locations; nested sectors→zones→locations; 3 pirate_hunt + 2 safe_zone.
- RLS read: anon can read sectors/zones/locations.
- RLS write-denial: insert blocked (42501 insufficient_privilege — SELECT-only grant), update/delete affect 0 rows.
- Frontend: dev server up at `http://localhost:5173/` for click-through.

Bugs / fixes

- Native GitHub integration did **not** auto-deploy on Free plan (first verify found no tables). Applied via CLI instead. Future migrations need a deploy decision (upgrade for native, or use the free GitHub Action with secrets in GitHub UI).
- The redundant custom Action `deploy-migrations.yml` fails on push (no secrets set); left in place pending the deploy-mechanism decision.

Follow-ups

- Rotate/revoke the temporary Supabase access token (it lived only in the deleted local file, but rotate as good hygiene).
- Decide future migration deploy mechanism before/at M3.

2026-06-16 — System boundaries approved; M2 (read-only world map)

Request Before coding, define strict system boundaries (sole writer per table, acyclic cross-system call graph via exposed functions only). Approve and persist, then build M2 as a **read-only** world map: `sectors/zones/locations` + seed + `get_world_map()` + map screen. No movement/fleets/presence/combat/rewards/resources.

Decisions made

- Approved all 5 beyond-spec ownership additions: **Fleet**, **Base**, **World State** systems as sole owners of their shared tables; `reward_grants` ledger as the only reward-application path; **Activity** table-less router. `_Why_` enforces every separation law with a single-writer-per-table rule and prevents hidden coupling.
- M2 scope locked to the 3 static Map tables only; `zone_state/location_state` deferred (they belong to World State, built later) so Map stays pure.

Work done

- Wrote `docs/SYSTEM_BOUNDARIES.md` (table→sole-writer matrix, per-system owns/exposes/forbidden, the 5 allowed call-edges, forbidden edges, invariant checklist).
- **M2 migration** `20260616000002_world_map.sql`: `sectors/zones/locations` (static, Map-owned) with CHECK constraints + FKs + `unique(sector,name)/(zone,name)`; public-read RLS, no write policies (no client writes); `get_world_map()` (nested jsonb, `stable`, granted to anon/authenticated); seed = 2 sectors / 2 zones / 5 locations (mix of `safe_zone` + `pirate_hunt`).
- **M2 frontend**: `features/map/` (`mapTypes.ts`, `mapApi.ts`, `MapPage.tsx`); `/map` route (auth-guarded); "Open galaxy map" link on Dashboard.
- Verified: `npm run build` green (tsc + vite, 75 modules). SQL not run locally (no psql/docker/supabase CLI on this machine) — reviewed by hand; first live run on migration apply.

Bugs / fixes

- `_ (none)_`

Follow-ups for user

- Apply migrations + set `.env.local`, then the map screen loads live data.
- M2 shows Map-owned fields only (name/type/danger/reward). Distance & travel-time need a base + movement formula → arrive in M3.

2026-06-16 — Foundation architecture & milestone plan (no code)

Request User supplied a detailed server-authoritative PvE design spec (map → location → movement → presence → activity → combat → retreat → return → report) and asked to **plan only, no code yet**, then persist the design as living docs.

Decisions made

- **Economy = combat-reward only (Option 1)**. Seed a starter base + starter units; resources come solely from pirate-combat rewards landing in `base_resources` at encounter end. `_Why_` the priority is proving the core world/loop foundation, not the

- economy. Adding production/buildings/training now would build too many systems at once and make bugs hard to isolate. Deferred: buildings, build queues, passive production, lazy resource accrual, unit training, research, trade/market, cargo.
- **Sequencing = movement+presence spine first (M3), combat second (M4).** _Why:_ spec keeps movement and combat as separate systems bridged by presence; proving the movement→presence→return spine on a harmless `safe_zone` first isolates any later combat bugs to the combat system (which the `combat_rounds` table is built to debug).
 - **Write architecture docs before any game code.** _Why:_ the spec is large and prescriptive; capturing it as `docs/ARCHITECTURE.md` makes it the source of truth so every milestone (and future session) follows the same modular, anti-cheat, server-authoritative rules instead of re-deriving them.

Gap resolutions agreed (added beyond original spec)

- `base_resources` table — rewards need somewhere to land (not an economy system).
- `initialize_new_player()` — seeds starter base + units + resources (no training in MVP).
- `game_config` table — tunable balance (`travel_scale`, `max_active_fleets`, `tick/retreat seconds`, reward multipliers, random variance) without code redeloys.

Work done

- Verified Supabase Cron supports sub-minute (seconds) schedules on Postgres 15.1.1.61+ → 30s movement / 10–15s combat / 60s location-state ticks are feasible.
- Wrote `docs/ARCHITECTURE.md` (core principle, world hierarchy, all systems, combat formulas, anti-cheat, RLS/RPC, state machines, constraints/locking/idempotency, cron timing, MVP table list, milestone roadmap M1–M6, deferred list).
- No game code or migrations written yet (next step: M2 world map, after review).

Bugs / fixes

- `_(none — planning only)_`

2026-06-16 — Rename to Byeharu

Request Change the game name to **Byeharu** (the initial scaffold used "Byeolharu"; user confirmed the shorter spelling).

Work done

- Renamed project folder `byeolharu` → `byecharu`.
- Updated `package.json` / `package-lock.json` name, `index.html` title, README, the migration comment, the Supabase client warning tag, and the AuthPage / Dashboard headings from "Byeolharu" to "Byecharu".
- Updated saved project memory.

Bugs / fixes

- `_(none)_`

2026-06-16 — Milestone 1: Scaffold + auth

Request Rebuild the PvE space-strategy game from scratch as a clean web-first project named **Byeolharu**. Stack: React + TypeScript + Vite, Tailwind, Zustand, Supabase (Postgres + Auth + RLS + RPC + pg_cron). Server-authoritative, modular systems, milestone-by-milestone. First milestone: scaffold + basic auth structure.

Work done

- Created Vite React+TS project at `C:\Users\디폴리스\byecharu`.
- Installed `zustand`, `@supabase/supabase-js`, `react-router-dom`, and `tailwindcss` + `@tailwindcss/vite` (Tailwind v4).
- Wired Tailwind via the Vite plugin (`vite.config.ts`) and `@import 'tailwindcss'` in `src/index.css`.
- Supabase client at `src/lib/supabase.ts`; env typing in `src/vite-env.d.ts`; `.env.example` with `VITE_SUPABASE_URL` / `VITE_SUPABASE_ANON_KEY`.
- Auth: Zustand store `src/store/authStore.ts` (session, signIn/signUp/signOut, `init()` listener); `src/features/auth/AuthPage.tsx` (login/signup); `src/app/RequireAuth.tsx` route guard; routing in `src/app/App.tsx`.
- Placeholder `src/features/dashboard/Dashboard.tsx`.
- DB: migration `supabase/migrations/20260616000001_init_profiles.sql` — `profiles` table, RLS (own-row read/update), auto-create-profile trigger on `auth.users` (SECURITY DEFINER).
- CI: `.github/workflows/deploy-migrations.yml` to supabase db push on push to main.
- Removed default Vite demo files (`App.tsx/App.css/sample assets`); updated `index.html` title and `.gitignore` for env files.

Bugs / fixes

- `_(none yet)_`

Open follow-ups

- User must create a Supabase project and fill `.env.local`.
 - For CI: add repo secrets `SUPABASE_ACCESS_TOKEN`, `SUPABASE_PROJECT_ID`, `SUPABASE_DB_PASSWORD`.
 - Run `npm run build` / typecheck once `.env.local` exists to confirm green.
-